Hibernate & JPA

A Complete Guide

From First Entity to Production Application — With Spring Boot 3, Hibernate 6, and Jakarta Persistence

2025 Edition • 30 Chapters

Table of Contents

Chapter 1 — Introduction
Chapter 2 — Development Environment
Chapter 3 — First JPA App
Chapter 4 — JPA Annotations
Chapter 5 — Primary Keys
Chapter 6 — CRUD Operations
Chapter 7 — Derived Queries
Chapter 8 — JPQL
Chapter 9 — Native SQL
Chapter 10 — Criteria API
Chapter 11 — Pagination and Sorting
Chapter 12 — One-to-One Relationships
Chapter 13 — One-to-Many Relationships
Chapter 14 — Many-to-Many Relationships
Chapter 15 — Embeddable Types
Chapter 16 — Inheritance Mapping
Chapter 17 — Entity Lifecycle
Chapter 18 — Fetching Strategies
Chapter 19 — Cascade Operations
Chapter 20 — Transaction Management
Chapter 21 — Caching
Chapter 22 — Bean Validation
Chapter 23 — DTOs and Projections
Chapter 24 — Repository and Service Layer
Chapter 25 — Error Handling
Chapter 26 — Flyway Migrations
Chapter 27 — Performance Tuning

Chapter 28 — REST API
Chapter 29 — Capstone Project
Chapter 30 — Glossary

Chapter 1: Introduction to ORM, JPA, and Hibernate

Learning Goals

By the end of this chapter, you will be able to:

- Explain what Object-Relational Mapping (ORM) is and why it exists
- Understand the pain points of working with raw JDBC
- Describe what JPA is and why it is a specification, not a library
- Explain what Hibernate is and its role as a JPA provider
- Distinguish between JPA, Hibernate, and Spring Data JPA
- Know when to use an ORM and when raw SQL is more appropriate
- Understand the layered architecture of a modern Java persistence stack

The Problem: Java Objects and Relational Databases Speak Different Languages

Every application needs to store data. When you build a Java application, you work with objects. A `Customer` has a name, an email, and a list of orders. An `Order` has a date, a total, and a list of items. These objects live in memory, they reference each other, they have behavior, and they form a rich graph of interconnected data.

But when it is time to save that data permanently, most applications use a relational database. Databases do not understand Java objects. They understand tables, rows, columns, foreign keys, and SQL. A `Customer` in the database is a row in the `customers` table. An `Order` is a row in the `orders` table. The relationship between them is a foreign key — a number in one table that references a number in another.

This fundamental mismatch is called the object-relational impedance mismatch. It is the gap between how you think about data in Java (objects, inheritance, collections, references) and how databases think about data (tables, rows, joins, foreign keys).

Let us look at a concrete example to understand this gap.

The Object World

In Java, a customer and their orders look like this:

```java
public class Customer {
    private Long id;
    private String name;
    private String email;
    private List<Order> orders; // Direct reference to related objects
}

public class Order {
    private Long id;
    private LocalDate orderDate;
    private BigDecimal total;
    private Customer customer; // Direct reference back
    private List<OrderItem> items;
}
```

You navigate from a customer to their orders by simply calling `customer.getOrders()`. You do not need to know anything about tables or SQL. The relationships are natural Java references.

The Relational World

In a database, the same data looks like this:

```sql
CREATE TABLE customers (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE
);

CREATE TABLE orders (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    order_date DATE NOT NULL,
    total DECIMAL(10, 2),
    customer_id BIGINT,
    FOREIGN KEY (customer_id) REFERENCES customers(id)
);
```

To get a customer's orders, you write a SQL JOIN query. To navigate from an order back to its customer, you look up the `customer_id` foreign key. There are no object references — only numbers linking rows together.

The Five Core Mismatches

The impedance mismatch shows up in several specific ways:

```
Mismatch              Java World                 Database World
-----------------------------------------------------------------------
Identity              equals() and hashCode()    Primary key (id column)
Inheritance           extends, implements        No direct equivalent
Associations          Object references          Foreign keys and join tables
Data types            LocalDate, BigDecimal,     DATE, DECIMAL, VARCHAR,
                      enums, custom types        INT
Granularity           Fine-grained objects       Flat table rows
                      (Address as a class)       (address columns in a row)
```

Identity: In Java, two objects can be equal (`equals()`) but be different instances. In a database, a row is uniquely identified by its primary key. Deciding when two objects represent the same database row is surprisingly tricky.

Inheritance: Java supports class inheritance. A `SavingsAccount` can extend `BankAccount`. Databases have no concept of inheritance. You need strategies to map a class hierarchy into tables.

Associations: In Java, a `Customer` has a `List<Order>`. In a database, the `orders` table has a `customer_id` column. The relationship is inverted — the child table holds the reference, not the parent.

Data types: Java has rich types like `LocalDate`, `BigDecimal`, enums, and custom classes. Databases have their own type system (VARCHAR, DECIMAL, TIMESTAMP). Converting between them is not always straightforward.

Granularity: In Java, you might model an `Address` as a separate class with street, city, state, and zip code. In a database, those might just be four columns in the `customers` table. The object model is more fine-grained than the table structure.

The Old Way: Raw JDBC

Before ORM frameworks existed, Java developers used JDBC (Java Database Connectivity) directly to talk to databases. JDBC is a low-level API included in the Java standard library. It

gives you complete control over SQL and database interaction, but that control comes at a significant cost in boilerplate code, error-prone manual mapping, and maintenance burden.

Let us see what a simple CRUD operation looks like with raw JDBC.

Saving a Customer with JDBC

```java
public class CustomerDao {

    private DataSource dataSource;

    public void save(Customer customer) {
        String sql = "INSERT INTO customers (name, email) VALUES (?, ?)";

        Connection connection = null;
        PreparedStatement statement = null;
        ResultSet generatedKeys = null;

        try {
            connection = dataSource.getConnection();
            statement = connection.prepareStatement(sql,
                Statement.RETURN_GENERATED_KEYS);

            statement.setString(1, customer.getName());
            statement.setString(2, customer.getEmail());
            statement.executeUpdate();

            // Get the auto-generated ID
            generatedKeys = statement.getGeneratedKeys();
            if (generatedKeys.next()) {
                customer.setId(generatedKeys.getLong(1));
            }

        } catch (SQLException e) {
            throw new RuntimeException("Failed to save customer", e);
        } finally {
            // Must close everything in reverse order
            try {
                if (generatedKeys != null) generatedKeys.close();
            } catch (SQLException e) { /* log and ignore */ }
            try {
                if (statement != null) statement.close();
            } catch (SQLException e) { /* log and ignore */ }
            try {
                if (connection != null) connection.close();
            } catch (SQLException e) { /* log and ignore */ }
        }
    }
}
```

That is over 30 lines of code to save a single object. And this is a simple example — no relationships, no transactions, no error recovery. Now imagine doing this for every entity in

your application, for every CRUD operation, for every query.

Finding a Customer by ID with JDBC

```java
public Customer findById(Long id) {
    String sql = "SELECT id, name, email FROM customers WHERE id = ?";

    Connection connection = null;
    PreparedStatement statement = null;
    ResultSet resultSet = null;

    try {
        connection = dataSource.getConnection();
        statement = connection.prepareStatement(sql);
        statement.setLong(1, id);
        resultSet = statement.executeQuery();

        if (resultSet.next()) {
            Customer customer = new Customer();
            customer.setId(resultSet.getLong("id"));
            customer.setName(resultSet.getString("name"));
            customer.setEmail(resultSet.getString("email"));
            return customer;
        }

        return null;

    } catch (SQLException e) {
        throw new RuntimeException("Failed to find customer", e);
    } finally {
        try { if (resultSet != null) resultSet.close(); } catch (SQLException e) {}
        try { if (statement != null) statement.close(); } catch (SQLException e) {}
        try { if (connection != null) connection.close(); } catch (SQLException e) {}
    }
}
```

The Pain Points of JDBC

After working with JDBC for even a small project, the problems become obvious:

1. Massive boilerplate: Every operation requires getting a connection, creating a statement, setting parameters, executing, reading results, and closing resources. The actual business logic is buried under infrastructure code.

2. Manual mapping: You must manually convert between `ResultSet` columns and Java object fields. If you add a column to the table, you must update every query and every mapping by hand.

3. Error-prone resource management: Forgetting to close a connection, statement, or result set causes resource leaks. The `finally` blocks are ugly and easy to get wrong.

4. SQL strings everywhere: SQL is written as strings inside Java code. There is no compile-time checking. A typo in a column name is only caught at runtime.

5. No relationship handling: If a `Customer` has orders, you must write separate queries to load them, manually set the references, and decide when to load related data.

6. No caching: Every query hits the database. If you call `findById(1)` twice in the same request, it runs two SQL queries for the same data.

7. Database portability: Your SQL might use features specific to one database (MySQL AUTO_INCREMENT vs PostgreSQL SERIAL). Switching databases means rewriting queries.

These are not minor inconveniences. In a real application with dozens of entities, hundreds of queries, and complex relationships, JDBC becomes a maintenance nightmare.

What Is ORM?

Object-Relational Mapping (ORM) is a programming technique that automatically converts data between a relational database and an object-oriented programming language. An ORM framework handles the translation for you, so you work with Java objects and the framework takes care of the SQL.

Think of an ORM as a translator. You speak Java, the database speaks SQL, and the ORM translates between them. You say "save this Customer object" and the ORM generates the appropriate `INSERT INTO customers...` SQL statement, executes it, and sets the generated ID back on your object.

What an ORM Does for You

```
Without ORM (JDBC):                  With ORM (Hibernate):
-----------------------------------  -----------------------------------
Write SQL strings by hand            SQL generated automatically
Map ResultSet to objects manually    Objects populated automatically
Manage connections and resources     Managed by the framework
Handle relationships manually       Relationships mapped with annotations
No caching                          Built-in caching
Database-specific SQL               Database-independent queries
30+ lines per operation             1-2 lines per operation
```

The Same Save Operation with an ORM

Remember the 30+ line JDBC `save` method? With an ORM, it becomes:

```java
entityManager.persist(customer);
```

One line. The ORM knows the `Customer` class maps to the `customers` table (because of annotations you defined once), it generates the INSERT SQL, executes it, retrieves the generated ID, and sets it on the object.

The Same Find Operation with an ORM

```java
Customer customer = entityManager.find(Customer.class, 1L);
```

One line. The ORM generates `SELECT id, name, email FROM customers WHERE id = 1`, executes it, creates a `Customer` object, populates its fields, and returns it.

How ORM Works Under the Hood

An ORM framework operates on a simple principle: you define the mapping between your Java classes and database tables once, and the framework uses that mapping to generate SQL and convert data automatically.

```
Your Code                    ORM Framework                   Database
-----------                  -------------                   --------
@Entity                      Reads annotations
Customer customer =          Generates SQL:
  new Customer("Alice");     INSERT INTO customers
entityManager.persist(       (name) VALUES ('Alice')     --> Executes SQL
  customer);                 Sets generated ID back      <-- Returns ID
                             customer.setId(1L)
```

The mapping is defined with annotations directly on your Java classes:

```java
@Entity                              // This class maps to a table
@Table(name = "customers")           // The table name is "customers"
public class Customer {

    @Id                              // This field is the primary key
    @GeneratedValue(strategy =       // The database generates IDs
        GenerationType.IDENTITY)
```

```
    private Long id;

    @Column(name = "name",           // Maps to the "name" column
            nullable = false,
            length = 100)
    private String name;

    @Column(unique = true)           // Maps to the "email" column (unique)
    private String email;
}
```

You define this mapping once. From then on, the ORM knows how to translate between the `Customer` class and the `customers` table. You never write SQL for basic operations again.

What Is JPA?

JPA (Jakarta Persistence API) is the official Java specification for object-relational mapping. It is not a library or a framework — it is a set of interfaces, annotations, and rules that define how ORM should work in Java.

Think of JPA as a contract. It says: "If you are an ORM framework, here is how your API should look. Here are the annotations developers should use. Here is how EntityManager should behave. Here is what `@Entity` means." But JPA itself does not provide the actual implementation. It is the blueprint, not the building.

JPA Is a Specification

This is a crucial distinction that trips up many beginners:

```
JPA (Specification)                 vs.            Hibernate (Implementation)
----------------------------------                 ----------------------------------
Defines the rules                                  Follows the rules
Provides interfaces                                Provides actual classes
jakarta.persistence.EntityManager                  org.hibernate.Session
(interface)                                        (implements EntityManager)
Cannot run on its own                              Can actually run and do work
Lives in the jakarta.persistence                   Lives in org.hibernate package
package
```

The JPA specification defines:

- Annotations: `@Entity`, `@Table`, `@Id`, `@Column`, `@OneToMany`, `@ManyToOne`, and dozens more. These are in the `jakarta.persistence` package.

- EntityManager interface: The core API for persisting, finding, updating, and deleting entities.
- JPQL: Java Persistence Query Language — a SQL-like language that queries entities (not tables).
- Criteria API: A programmatic, type-safe way to build queries.
- Transaction rules: How transactions should work with the persistence layer.
- Caching rules: How first-level and second-level caches should behave.
- Lifecycle events: `@PrePersist`, `@PostPersist`, `@PreUpdate`, and similar callbacks.

Why Use a Specification?

You might wonder: why not just use Hibernate directly? Why have a specification at all?

The answer is portability and standardization.

Because JPA is a standard, your code is not locked into any single ORM provider. If you write your code using JPA annotations (`jakarta.persistence.*`), you can theoretically switch from Hibernate to EclipseLink, OpenJPA, or any other JPA provider without changing your application code.

In practice, most projects use Hibernate and may occasionally use Hibernate-specific features. But coding to the JPA standard is still valuable because:

1. Your skills are transferable — JPA knowledge applies regardless of the provider
2. Standard annotations are well-documented — The JPA specification is clear and consistent
3. Spring Data JPA works with any provider — Spring does not care which JPA provider you use
4. Job interviews test JPA knowledge — Interviewers ask about JPA, not just Hibernate

A Brief History

JPA has evolved through several major versions:

```
Version    Year    Key Changes
-------------------------------------------------------------
JPA 1.0    2006    First release. Basic ORM annotations,
                   EntityManager, JPQL.
JPA 2.0    2009    Criteria API, @ElementCollection,
                   @MapKeyColumn, cache API, Bean Validation.
JPA 2.1    2013    Stored procedures, @Converter,
                   @NamedEntityGraph, CDI integration.
JPA 2.2    2017    Java 8 date/time support, Stream results,
                   @Repeatable annotations.
```

```
JPA 3.0    2020    Namespace change: javax.persistence
                   became jakarta.persistence (Jakarta EE).
JPA 3.1    2022    UUID generation, numeric functions,
                   JPQL enhancements, @Extract.
JPA 3.2    2024    (Ongoing) Additional query enhancements.
```

The most significant change was JPA 3.0, which moved from the `javax.persistence` package to `jakarta.persistence`. This happened because the Java EE project was transferred from Oracle to the Eclipse Foundation and renamed to Jakarta EE. If you see older tutorials or code using `javax.persistence`, they are using JPA 2.x or earlier. All modern code (including everything in this book) uses `jakarta.persistence`.

What Is Hibernate?

Hibernate is the most popular JPA provider (implementation). It is an open-source ORM framework that implements the full JPA specification and adds many powerful features of its own.

When you use JPA annotations like `@Entity`, `@Id`, and `@Table`, and call methods like `entityManager.persist()`, it is Hibernate that does the actual work behind the scenes. Hibernate translates your method calls into SQL, manages the database connection, handles caching, and converts results back into Java objects.

Hibernate's Role in the Stack

```
+-------------------------------------------+
|           Your Application Code           |
|  (Entities, Services, Repositories)       |
+-------------------------------------------+
                    |
                    v
+-------------------------------------------+
|         Spring Data JPA (Optional)        |
|  (JpaRepository, derived queries,         |
|   pagination, specifications)             |
+-------------------------------------------+
                    |
                    v
+-------------------------------------------+
|              JPA Specification            |
|  (jakarta.persistence annotations,        |
|   EntityManager interface, JPQL)          |
```

```
+-------------------------------------------+
                    |
                    v
+-------------------------------------------+
|          Hibernate (JPA Provider)         |
|  (Actual ORM implementation, SQL          |
|   generation, caching, session mgmt)      |
+-------------------------------------------+
                    |
                    v
+-------------------------------------------+
|              JDBC Driver                  |
|  (H2, PostgreSQL, MySQL driver)           |
+-------------------------------------------+
                    |
                    v
+-------------------------------------------+
|              Database                     |
|  (H2, PostgreSQL, MySQL, Oracle)          |
+-------------------------------------------+
```

Hibernate-Specific Features

While Hibernate implements everything in JPA, it also provides features that go beyond the specification:

Additional annotations:

- `@NaturalId` — Define a business key (like email or ISBN) for natural lookups
- `@Formula` — Derive a field value from a SQL expression without a real column
- `@DynamicUpdate` — Generate UPDATE statements that only include changed columns
- `@DynamicInsert` — Generate INSERT statements that skip null columns
- `@BatchSize` — Control batch fetching for collections
- `@Where` — Add a SQL WHERE clause to entity or collection loading (useful for soft deletes)
- `@Filter` — Define parameterized filters that can be enabled/disabled at runtime

Additional features:

- `StatelessSession` — A lightweight session for bulk operations without caching overhead
- Multi-tenancy support — Handle multiple tenants in a single database
- Hibernate Envers — Automatic entity auditing and versioning
- Hibernate Search — Full-text search integration with Lucene/Elasticsearch
- Bytecode enhancement — Instrument entity classes for lazy attribute loading

Throughout this book, we will clearly mark which features are JPA standard (portable across providers) and which are Hibernate-specific (only available with Hibernate). Most of the code you write will use standard JPA annotations.

A Brief History of Hibernate

```
Version      Year    Key Details
-------------------------------------------------------------
Hibernate 1  2002    First release by Gavin King. Predates JPA.
Hibernate 2  2003    Matured mapping and query capabilities.
Hibernate 3  2005    First to implement JPA 1.0.
Hibernate 4  2011    Multi-tenancy, service registry.
Hibernate 5  2015    JPA 2.1, Java 8 support, new bootstrap API.
             2017    JPA 2.2 support added.
Hibernate 6  2022    JPA 3.0/3.1, jakarta.persistence namespace,
                     rewritten query parser (SQM), improved
                     type system, better performance.
```

Hibernate actually predates JPA — it was so successful that many of its concepts became part of the JPA specification. Gavin King, who created Hibernate, was a key contributor to the JPA specification.

JPA vs Hibernate vs Spring Data JPA

This is one of the most common sources of confusion for beginners. Let us clear it up with a direct comparison.

JPA — The Contract

JPA defines the standard annotations and interfaces. It tells you what to write, but it cannot run anything on its own.

```java
// These imports are JPA (the standard)
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.EntityManager;
```

Hibernate — The Engine

Hibernate is the actual ORM engine that implements JPA. It does the real work: generating SQL, managing connections, caching entities.

```java
// These imports are Hibernate-specific
import org.hibernate.Session;
import org.hibernate.annotations.NaturalId;
import org.hibernate.annotations.BatchSize;
```

Spring Data JPA — The Convenience Layer

Spring Data JPA is a Spring module that sits on top of JPA (and therefore on top of Hibernate). It eliminates boilerplate by providing:

- Automatic repository implementations: Define an interface, and Spring creates the implementation for you.
- Derived query methods: Name your method `findByEmail`, and Spring generates the JPQL query.
- Built-in pagination and sorting: Out of the box with `Pageable` and `Sort`.
- Auditing support: Automatic `@CreatedDate`, `@LastModifiedDate`.

```java
// This import is Spring Data JPA
import org.springframework.data.jpa.repository.JpaRepository;

// You write an interface. Spring implements it automatically.
public interface CustomerRepository extends JpaRepository<Customer, Long> {
    List<Customer> findByNameContaining(String name);
    Optional<Customer> findByEmail(String email);
}
```

With Spring Data JPA, you do not even call `entityManager.persist()` directly. You call `customerRepository.save(customer)`, and Spring Data JPA calls the EntityManager for you.

The Complete Comparison

```
Feature                JPA             Hibernate          Spring Data JPA
-------------------------------------------------------------------------
What is it?            Specification   Implementation     Convenience layer
Can run alone?         No              Yes                No (needs JPA
                                                          provider)
Package                jakarta.        org.hibernate      org.springframework
                       persistence                        .data.jpa
Defines annotations?   Yes             Yes (extra ones)   Yes (extra ones)
Generates SQL?         No (defines     Yes                No (delegates to
                       rules only)                        JPA provider)
```

```
Provides caching?      Defines API      Implements it      No (uses JPA's)
Required?              Yes (standard)   Yes (or another    No (optional
                                        provider)          convenience)
```

An Analogy

Think of it like building a house:

- JPA is the building code — it defines the rules for how a house should be built (safety standards, structural requirements, electrical codes). But you cannot live in a building code.
- Hibernate is the construction company — it actually builds the house according to the building code. It follows the rules but also adds extra features (fancy lighting, smart home systems) that go beyond the basic code.
- Spring Data JPA is the property management company — it handles the day-to-day tasks (finding tenants, collecting rent, scheduling repairs) so you do not have to manage every detail yourself. It works with any construction company that follows the building code.

When to Use ORM vs Raw SQL

ORM is not always the right choice. Understanding when it helps and when it gets in the way is an important skill.

Use ORM (JPA/Hibernate) When

1. Your application is entity-centric: You are building a typical business application where you create, read, update, and delete objects. Examples: e-commerce, CRM, content management, user management.

2. You want rapid development: ORM dramatically reduces the amount of code you write. For standard CRUD operations, you write almost no SQL.

3. You need relationship management: Your entities have complex relationships (one-to-many, many-to-many) and you want the framework to handle loading and saving related objects.

4. You value database portability: You need to support multiple databases or might switch databases in the future.

5. You need caching: Hibernate's first-level and second-level caches can significantly improve performance for read-heavy applications.

6. You want auditing and lifecycle hooks: ORM frameworks provide automatic timestamps, audit logging, and lifecycle callbacks.

Use Raw SQL (JDBC or a Lightweight Library) When

1. You are doing complex analytics: Queries with many joins, subqueries, window functions, CTEs, or database-specific features are often easier and more performant in raw SQL.

2. You need maximum performance: For bulk operations (inserting millions of rows) or high-throughput systems, the overhead of ORM can be significant.

3. You are working with legacy databases: If the database schema cannot be changed and does not map cleanly to objects, ORM can fight you more than help you.

4. You need database-specific features: Stored procedures, materialized views, full-text search (without Hibernate Search), or database-specific syntax are easier with raw SQL.

5. Your application is simple: If you have a few tables and simple queries, the overhead of setting up ORM might not be worth it.

The Pragmatic Approach

In practice, most projects use ORM for the majority of operations and drop down to native SQL for specific cases. JPA and Hibernate support this approach. You can use JPQL for most queries, and when you need a complex database-specific query, you write native SQL with `@Query(nativeQuery = true)`. You do not have to choose one or the other exclusively.

The Technology Stack for This Book

Throughout this book, we will use the following technology stack:

```
Technology          Version          Purpose
------------------------------------------------------------
Java                17+              Programming language
Spring Boot         3.x              Application framework
Spring Data JPA     3.x              Repository abstraction
Hibernate           6.x              JPA provider (ORM engine)
JPA                 3.1              Persistence specification
                                     (jakarta.persistence)
```

```
H2 Database         2.x             In-memory database for
                                    learning and examples
Maven               3.x             Build and dependency
                                    management
Hibernate Validator 8.x             Bean validation (optional)
Flyway              9.x             Database migrations
                                    (later chapters)
```

Why This Stack?

Spring Boot auto-configures Hibernate, the data source, and transaction management. You can focus on learning JPA and Hibernate concepts without wrestling with XML configuration files or manual setup.

H2 Database runs entirely in memory. You do not need to install a database server. Every time you start your application, the database is fresh. This makes it perfect for learning and experimentation. In later chapters, when we discuss production patterns, we will note where you would use PostgreSQL or MySQL instead.

Maven is the most widely used build tool in the Java ecosystem. The `pom.xml` dependency declarations are straightforward, and most tutorials and documentation use Maven.

How This Book Is Structured

This book is divided into six parts that take you from complete beginner to building production-ready applications:

Part I: Foundations (Chapters 1-6) — Set up your environment, create your first entity, understand annotations, and perform basic CRUD operations. By the end of Part I, you can build a simple application that creates, reads, updates, and deletes data.

Part II: Querying Data (Chapters 7-11) — Master every way to query data: derived methods, JPQL, native SQL, Criteria API, and pagination. By the end of Part II, you can retrieve data in any shape you need.

Part III: Relationships and Mapping (Chapters 12-16) — Map real-world relationships between entities: one-to-one, one-to-many, many-to-many, embedded types, and inheritance. By the end of Part III, you can model complex domain models.

Part IV: Deep Dive (Chapters 17-22) — Understand how Hibernate works internally: entity lifecycle, fetching strategies, transactions, caching, and validation. By the end of Part IV, you

understand why things work the way they do.

Part V: Production Patterns (Chapters 23-27) — Learn professional patterns: DTOs, service layers, error handling, database migrations, and performance tuning. By the end of Part V, you write code that is production-quality.

Part VI: Capstone (Chapters 28-30) — Build a complete REST API and a full Library Management System project that applies everything you have learned.

Each chapter builds on the previous one. The concepts are introduced in a logical order, and later chapters reference earlier ones. You will get the most out of this book by reading the chapters in sequence.

Common Mistakes

1. Confusing JPA with Hibernate: JPA is a specification (a set of rules). Hibernate is an implementation (the actual library). You use JPA annotations in your code, but Hibernate is what runs at runtime. Many beginners say "I'm using JPA" when they mean "I'm using Hibernate through JPA annotations."
2. Thinking ORM eliminates the need to understand SQL: ORM generates SQL for you, but you still need to understand SQL to write efficient JPQL queries, debug performance issues, and use native queries when needed. ORM is a tool to reduce boilerplate, not a replacement for database knowledge.
3. Choosing ORM when it is not appropriate: For reporting-heavy applications, bulk data processing, or systems with very complex queries, raw SQL or a lightweight query builder (like jOOQ) might be a better choice. Do not force every project into an ORM.
4. Using Hibernate-specific features without realizing it: When you import from `org.hibernate.*`, you are using Hibernate-specific code that will not work with other JPA providers. This is fine if you know you are doing it, but be aware of the trade-off.
5. Ignoring the javax to jakarta migration: If you are reading older tutorials or using older libraries, they may use `javax.persistence.*` imports. Modern projects (Spring Boot 3.x, Hibernate 6.x) use `jakarta.persistence.*`. Mixing them causes compilation errors.

Best Practices

1. Code to the JPA specification first: Use `jakarta.persistence.*` annotations for your entities. Only reach for Hibernate-specific annotations (`org.hibernate.annotations.*`) when JPA does not provide what you need.
2. Learn SQL alongside ORM: Understanding what SQL Hibernate generates helps you write better JPQL, avoid N+1 problems, and debug performance issues. Always enable SQL logging during development.
3. Start simple: Begin with basic CRUD operations and simple queries. Add complexity (relationships, caching, optimization) only when you need it and understand it.
4. Use Spring Data JPA for productivity: In a Spring Boot application, there is no reason not to use Spring Data JPA. The repository interfaces eliminate enormous amounts of boilerplate with zero performance cost.
5. Keep entities clean: An entity class should represent a database table mapping. Do not put business logic, validation rules (beyond simple annotations), or presentation logic in entity classes.

Summary

In this chapter, you learned the foundational concepts that underpin everything else in this book:

- Object-relational impedance mismatch is the gap between Java's object model and the relational database model. It manifests in differences in identity, inheritance, associations, data types, and granularity.
- JDBC is Java's low-level database API. It works but requires enormous amounts of boilerplate code for connections, statements, result set mapping, and resource cleanup.
- ORM (Object-Relational Mapping) is a technique that automatically translates between objects and database tables. It reduces boilerplate, handles relationships, provides caching, and generates SQL.
- JPA (Jakarta Persistence API) is the Java standard specification for ORM. It defines the annotations (`@Entity`, `@Id`, `@Column`), interfaces (`EntityManager`), and query language (JPQL) — but it cannot run on its own.
- Hibernate is the most popular JPA implementation. It does the actual work of generating SQL, managing connections, caching data, and converting between objects and rows. It also provides features beyond the JPA standard.
- Spring Data JPA is a convenience layer on top of JPA that eliminates boilerplate by auto-implementing repository interfaces.

- ORM is not always the answer. For complex analytics, bulk operations, or database-specific features, raw SQL might be more appropriate. Most real projects use a combination of both.

Interview Questions

Q1: What is the object-relational impedance mismatch?

It is the fundamental mismatch between how Java represents data (objects, inheritance, references, collections) and how relational databases represent data (tables, rows, foreign keys, joins). ORM frameworks exist to bridge this gap.

Q2: What is the difference between JPA and Hibernate?

JPA is a specification — it defines the standard annotations, interfaces, and rules for ORM in Java. Hibernate is an implementation — it is the actual library that implements the JPA specification. You cannot use JPA without a provider like Hibernate, EclipseLink, or OpenJPA.

Q3: What are the main advantages of using an ORM over raw JDBC?

Reduced boilerplate code, automatic SQL generation, built-in caching, relationship management, database portability, entity lifecycle management, and type-safe queries through the Criteria API.

Q4: When would you choose raw SQL over an ORM?

For complex analytical queries, bulk data operations, database-specific features (stored procedures, materialized views), maximum performance requirements, or when working with legacy schemas that do not map cleanly to objects.

Q5: What changed from `javax.persistence` to `jakarta.persistence`?

When Java EE was transferred from Oracle to the Eclipse Foundation, it was renamed Jakarta EE. The package namespace changed from `javax.persistence` to `jakarta.persistence`. This happened in JPA 3.0. The API is functionally the same, but the package names are different. Spring Boot 3.x and Hibernate 6.x use the `jakarta` namespace.

Q6: What is the role of Spring Data JPA?

Spring Data JPA is a convenience layer on top of JPA. It auto-implements repository interfaces so you do not need to write boilerplate code for common operations. It provides derived query methods, pagination, sorting, and auditing support. It works with any JPA provider, not just Hibernate.

Q7: Can you use Hibernate without JPA?

Yes. Hibernate has its own native API (the `Session` interface) that predates JPA. However, modern best practice is to use JPA annotations and the `EntityManager` interface, which Hibernate implements. This keeps your code portable and follows the industry standard.

Q8: Name three Hibernate-specific features that are not part of the JPA specification.

`@NaturalId` for natural key lookups, `@Formula` for computed columns using SQL expressions, and `@BatchSize` for controlling batch fetching of collections. Other examples include `@DynamicUpdate`, `@Where`, `@Filter`, and `StatelessSession`.

Practice Exercises

Exercise 1: JDBC Pain Write a JDBC method that finds all customers whose name contains a given string, ordered by name. Notice how much boilerplate is required. Keep this code — you will compare it to the JPA version in Chapter 3.

Exercise 2: Mapping Analysis Consider a simple blog application with `Author`, `Post`, and `Comment` entities. For each pair, identify:

- What type of relationship exists (one-to-one, one-to-many, many-to-many)
- Which side holds the foreign key in the database
- What the impedance mismatch looks like (Java references vs foreign keys)

Exercise 3: JPA vs Hibernate Create a two-column table listing 10 annotations. For each, indicate whether it comes from the JPA specification (`jakarta.persistence.*`) or is Hibernate-specific (`org.hibernate.annotations.*`). Examples: `@Entity`, `@NaturalId`, `@Table`, `@Formula`, `@Column`, `@BatchSize`, `@Id`, `@DynamicUpdate`, `@GeneratedValue`, `@Where`.

Exercise 4: Stack Diagram Draw the technology stack for a Spring Boot application that uses Hibernate. Include: your application code, Spring Data JPA, JPA, Hibernate, JDBC driver, and the database. Label each layer with what it does and give one example of a class or annotation from that layer.

Exercise 5: When to Use ORM For each scenario below, decide whether ORM (Hibernate) or raw SQL would be the better choice, and explain why:

1. A CRUD application for managing employee records
2. A data warehouse reporting system with complex joins and window functions
3. A microservice that manages user profiles with addresses and roles
4. A batch job that inserts 10 million rows from a CSV file

5. An application that needs to support both PostgreSQL and MySQL

What Is Next?

In the next chapter, we will set up your development environment. You will install the JDK, configure Maven, create a Spring Boot project using Spring Initializr, and understand the project structure and key configuration files. By the end of Chapter 2, you will have a running Spring Boot application with Hibernate and H2 ready for the hands-on examples that follow.

Chapter 2: Setting Up Your Development Environment

Learning Goals

By the end of this chapter, you will be able to:

- Install and verify the Java Development Kit (JDK 17 or later)
- Understand what Maven is and how it manages dependencies
- Create a new Spring Boot 3.x project using Spring Initializr
- Understand the project structure of a Spring Boot JPA application
- Configure application.properties for Hibernate and H2
- Access the H2 database console to inspect your data
- Run your Spring Boot application for the first time

Prerequisites

Before we begin, let us make sure you have the foundational tools in place. Here is what you need and what this chapter will help you set up:

```
+-------------------------------------------------------------+
|                   Development Environment                   |
|                                                             |
|  +-----------------+  +-----------------+  +-------------+  |
|  |    JDK 17+      |  |   Maven 3.9+    |  |    IDE      |  |
|  |  (Java compiler |  |  (Build tool,   |  | (IntelliJ or |  |
|  |   and runtime)  |  |   dependency    |  |  VS Code)   |  |
|  |                 |  |   manager)      |  |             |  |
|  +-----------------+  +-----------------+  +-------------+  |
```

```
|                                                                  |
|  +------------------+  +------------------+                     |
|  | Spring Initializr|  |   H2 Console     |                     |
|  | (Project creator)|  | (DB browser)     |                     |
|  +------------------+  +------------------+                     |
+------------------------------------------------------------------+
```

You do not need to install a database server. H2 runs entirely inside your application.

Installing the JDK

The Java Development Kit (JDK) includes everything you need to compile and run Java applications. We need JDK 17 or later because Spring Boot 3.x requires it.

What Version to Install

```
JDK Version    Spring Boot 3.x    Recommended?
-------------------------------------------------------
JDK 11         Not supported       No
JDK 17         Supported (LTS)     Yes (minimum)
JDK 21         Supported (LTS)     Yes (latest LTS)
JDK 22+        Supported           Optional
```

LTS stands for Long-Term Support. JDK 17 and 21 are LTS releases, meaning they receive security updates for years. Either one is a great choice. This book uses JDK 17 syntax, which is compatible with all later versions.

Installing on macOS

The easiest way is using Homebrew:

```bash
# Install Homebrew if you do not have it
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/inst...

# Install JDK 17
brew install openjdk@17

# Add to PATH (add this line to your ~/.zshrc or ~/.bash_profile)
export PATH="/opt/homebrew/opt/openjdk@17/bin:$PATH"
```

```
# Verify installation
java -version
```

You should see output similar to:

```
openjdk version "17.0.10" 2024-01-16
OpenJDK Runtime Environment (build 17.0.10+7)
OpenJDK 64-Bit Server VM (build 17.0.10+7, mixed mode)
```

Installing on Windows

1. Download the JDK installer from https://adoptium.net (Eclipse Temurin)
2. Run the installer and check "Set JAVA_HOME variable"
3. Open a new Command Prompt and verify:

```bash
java -version
javac -version
```

Installing on Linux

```bash
# Ubuntu/Debian
sudo apt update
sudo apt install openjdk-17-jdk

# Fedora/RHEL
sudo dnf install java-17-openjdk-devel

# Verify
java -version
```

Verifying Your Installation

Run both commands to ensure the JDK (not just JRE) is installed:

```bash
# Check the runtime
java -version

# Check the compiler (this confirms JDK, not just JRE)
javac -version
```

If `java` works but `javac` does not, you installed a JRE (Java Runtime Environment) instead of a JDK. Go back and install the full JDK.

Understanding Maven

Maven is a build tool and dependency manager for Java projects. It handles two critical tasks:

1. Dependency management: You declare which libraries your project needs (like Hibernate, Spring Boot, H2), and Maven downloads them automatically.
2. Build lifecycle: Maven compiles your code, runs tests, and packages your application into a runnable JAR file.

How Maven Works

```
+-------------------------------------------------------------------+
|                        Your Project                               |
|                                                                   |
|  pom.xml                                                          |
|  (declares dependencies)                                          |
|                                                                   |
|  <dependency>                     Maven reads pom.xml             |
|    spring-boot-starter-data-jpa  and downloads JARs               |
|  </dependency>            -----> from Maven Central               |
|  <dependency>                     Repository (the internet)       |
|    h2database                                                     |
|  </dependency>                                                    |
|                                                                   |
|  +-------------------------------------------------------------+  |
|  |  ~/.m2/repository/ (local cache)                            |  |
|  |                                                             |  |
|  |  spring-boot-starter-data-jpa-3.2.x.jar                    |  |
|  |  hibernate-core-6.4.x.jar                                  |  |
|  |  h2-2.x.jar                                                |  |
|  |  jakarta.persistence-api-3.1.x.jar                         |  |
|  |  ... (all transitive dependencies)                          |  |
|  +-------------------------------------------------------------+  |
+-------------------------------------------------------------------+
```

When you add `spring-boot-starter-data-jpa` as a dependency, Maven does not just download that one JAR. It also downloads all of its transitive dependencies — the libraries that library depends on. This single starter pulls in:

```
spring-boot-starter-data-jpa
  |
  +-- spring-data-jpa          (Spring Data JPA)
  |     +-- spring-data-commons
  |
  +-- hibernate-core            (Hibernate 6.x)
  |     +-- jakarta.persistence-api  (JPA 3.1)
  |     +-- jboss-logging
  |     +-- byte-buddy          (bytecode generation)
  |
  +-- spring-boot-starter-jdbc  (JDBC support)
  |     +-- HikariCP            (connection pool)
  |     +-- spring-jdbc
  |
  +-- spring-boot-starter-aop   (AOP for transactions)
        +-- aspectjweaver
```

You declare one dependency, and Maven brings in over 20 JARs. This is why dependency management is essential — doing this by hand would be tedious and error-prone.

Installing Maven

Maven comes bundled with most IDEs (IntelliJ, Eclipse) and is included in the Maven Wrapper that Spring Initializr generates. You usually do not need to install Maven separately. But if you want it available on the command line:

macOS:

```bash
brew install maven
mvn -version
```

Windows: Download from https://maven.apache.org/download.cgi, unzip, and add the `bin` directory to your PATH.

Linux:

```bash
sudo apt install maven    # Ubuntu/Debian
sudo dnf install maven    # Fedora/RHEL
mvn -version
```

The Maven Wrapper

Every Spring Boot project generated by Spring Initializr includes a Maven Wrapper (`mvnw` on macOS/Linux, `mvnw.cmd` on Windows). This is a script that downloads and uses the correct Maven version for your project, without requiring a global Maven installation.

```bash
# Instead of:
mvn spring-boot:run

# You can use:
./mvnw spring-boot:run     # macOS/Linux
mvnw.cmd spring-boot:run   # Windows
```

Throughout this book, we will use `mvn` in examples, but you can always substitute `./mvnw` if you prefer the wrapper.

Choosing an IDE

You can use any Java IDE or text editor, but two options stand out for Spring Boot development:

IntelliJ IDEA (Recommended)

IntelliJ IDEA is the most popular IDE for Java and Spring Boot development. The Community Edition (free) supports Java and Maven. The Ultimate Edition (paid, free for students) adds Spring Boot support, database tools, and JPA entity designers.

```
IntelliJ IDEA Features for Hibernate/JPA:
+-----------------------------------------+
| Community (Free)    | Ultimate (Paid)     |
|---------------------|---------------------|
| Java editing        | Everything in CE    |
| Maven support       | Spring Boot support |
| Code completion     | JPA entity diagram  |
| Debugging           | Database tools      |
| Git integration     | JPQL completion     |
| Refactoring         | Hibernate console   |
+-----------------------------------------+
```

Download from: https://www.jetbrains.com/idea/download/

Visual Studio Code

VS Code is lightweight and free. With the right extensions, it becomes a capable Java IDE:

Required extensions:

- Extension Pack for Java (by Microsoft) — Java language support

- Spring Boot Extension Pack (by VMware) — Spring Boot support
- Maven for Java (by Microsoft) — Maven project support

Other Options

Eclipse (with Spring Tools Suite), NetBeans, or even a plain text editor with command-line Maven all work. The code in this book is IDE-independent.

Creating Your Project with Spring Initializr

Spring Initializr is a web tool that generates a ready-to-run Spring Boot project with the dependencies you choose. It is the standard way to start a new Spring Boot project.

Step-by-Step Setup

1. Open your browser and go to https://start.spring.io
2. Fill in the project settings:

```
Setting              Value                      Why
-----------------------------------------------------------------
Project              Maven                      Build tool
Language             Java                       Programming language
Spring Boot          3.3.x (latest stable)      Framework version
Group                com.example                Your organization
Artifact             hibernate-demo             Project name
Name                 hibernate-demo             Application name
Description          Learning Hibernate & JPA   Your description
Package name         com.example.hibernatedemo  Base package
Packaging            Jar                        Executable JAR
Java                 17                         JDK version
```

3. Add the following dependencies (click "Add Dependencies"):

```
+-----------------------------------------------------------+
|  Dependencies to Add                                      |
|                                                           |
|  [x] Spring Data JPA                                      |
|      Provides: JpaRepository, EntityManager,              |
|      auto-configuration for Hibernate                     |
|                                                           |
|  [x] H2 Database                                          |
|      Provides: In-memory database, no installation        |
|      needed, web console for browsing data                |
```

```
|                                                            |
|  [x] Spring Web                                            |
|      Provides: REST controllers, embedded Tomcat           |
|      (needed for H2 console and later chapters)            |
|                                                            |
|  [x] Spring Boot DevTools (optional)                       |
|      Provides: Automatic restart on code changes,          |
|      LiveReload, relaxed development settings              |
+------------------------------------------------------------+
```

4. Click "Generate" to download the project as a ZIP file.
5. Unzip the file and open the project in your IDE.

What Spring Initializr Creates

```
hibernate-demo/
|
+-- pom.xml                          <-- Maven configuration
+-- mvnw                             <-- Maven Wrapper (Mac/Linux)
+-- mvnw.cmd                         <-- Maven Wrapper (Windows)
+-- .mvn/                            <-- Maven Wrapper config
|
+-- src/
|   +-- main/
|   |   +-- java/
|   |   |   +-- com/example/hibernatedemo/
|   |   |       +-- HibernateDemoApplication.java   <-- Main class
|   |   |
|   |   +-- resources/
|   |       +-- application.properties              <-- Configuration
|   |       +-- static/                              <-- Static files
|   |       +-- templates/                           <-- Template files
|   |
|   +-- test/
|       +-- java/
|           +-- com/example/hibernatedemo/
|               +-- HibernateDemoApplicationTests.java  <-- Test class
|
+-- .gitignore
```

Let us examine the key files.

Understanding the Project Structure

The pom.xml File

The `pom.xml` is the heart of your Maven project. It declares your project metadata and dependencies. Here is what Spring Initializr generates (simplified):

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <!-- Spring Boot parent: provides dependency versions -->
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.0</version>
    </parent>

    <!-- Your project coordinates -->
    <groupId>com.example</groupId>
    <artifactId>hibernate-demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>hibernate-demo</name>
    <description>Learning Hibernate and JPA</description>

    <!-- Java version -->
    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <!-- Spring Data JPA (includes Hibernate 6.x) -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <!-- H2 In-Memory Database -->
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>

        <!-- Spring Web (for REST and H2 console) -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <!-- DevTools (optional, for hot reload) -->
```

```
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>

        <!-- Test dependencies -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

Key observations:

- No version numbers on dependencies: The Spring Boot parent manages all version numbers. When you declare `spring-boot-starter-data-jpa`, the parent decides which version of Hibernate, Spring Data, HikariCP, and Jakarta Persistence to use. This prevents version conflicts.
- `scope: runtime` for H2 means the H2 JAR is available when the app runs but not at compile time (you never import H2 classes directly in your code).
- `spring-boot-starter-data-jpa` is the single dependency that brings in JPA, Hibernate, Spring Data JPA, HikariCP (connection pool), and everything else you need.

How Dependencies Flow

```
What you declare          What you actually get
in pom.xml                (transitive dependencies)
--------------------      -----------------------------------

spring-boot-              +-- spring-data-jpa 3.3.x
starter-data-jpa     -->  +-- hibernate-core 6.5.x
                          +-- jakarta.persistence-api 3.1.x
                          +-- HikariCP 5.x
                          +-- spring-jdbc 6.1.x
                          +-- spring-tx 6.1.x (transactions)
```

```
                           +-- spring-orm 6.1.x
                           +-- spring-aop 6.1.x

h2                    -->  H2 Database Engine 2.x

spring-boot-               +-- spring-webmvc 6.1.x
starter-web           -->  +-- embedded Tomcat 10.x
                           +-- jackson (JSON processing)
```

The Main Application Class

```java
package com.example.hibernatedemo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class HibernateDemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(HibernateDemoApplication.class, args);
    }
}
```

This tiny class does a lot thanks to `@SpringBootApplication`, which combines three annotations:

```
@SpringBootApplication
  |
  +-- @SpringBootConfiguration   (This is a configuration class)
  |
  +-- @EnableAutoConfiguration   (Auto-configure based on dependencies)
  |     |
  |     +-- Detects spring-data-jpa on classpath
  |     |   --> Configures EntityManagerFactory, TransactionManager
  |     |
  |     +-- Detects h2 on classpath
  |     |   --> Configures DataSource with H2
  |     |
  |     +-- Detects spring-web on classpath
  |         --> Starts embedded Tomcat
  |
  +-- @ComponentScan             (Scan this package and sub-packages
                                  for @Component, @Service,
                                  @Repository, @Controller)
```

When you run this class, Spring Boot:

1. Starts an embedded Tomcat server
2. Creates an H2 in-memory database
3. Creates a Hibernate `SessionFactory` (which implements JPA's `EntityManagerFactory`)
4. Sets up transaction management
5. Scans for `@Entity` classes and creates the corresponding tables

All of this happens automatically because of the dependencies in your `pom.xml`.

Configuring application.properties

The `src/main/resources/application.properties` file controls how Spring Boot and Hibernate behave. The generated file is empty — Spring Boot uses sensible defaults. But for learning, we want to customize several settings.

Replace the empty file with:

```properties
# ===== Application Name =====
spring.application.name=hibernate-demo

# ===== H2 Database Configuration =====
# Use a named in-memory database (accessible from H2 console)
spring.datasource.url=jdbc:h2:mem:learndb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# ===== H2 Console (Web UI to browse your database) =====
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# ===== JPA / Hibernate Configuration =====
# Auto-create tables from @Entity classes (good for development)
spring.jpa.hibernate.ddl-auto=create-drop

# Show SQL statements in the console (essential for learning)
spring.jpa.show-sql=true

# Format the SQL for readability
spring.jpa.properties.hibernate.format_sql=true

# Show parameter values bound to SQL statements
logging.level.org.hibernate.orm.jdbc.bind=TRACE

# Hibernate dialect (optional - Hibernate auto-detects H2)
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
```

Let us understand each setting:

Database URL

```properties
spring.datasource.url=jdbc:h2:mem:learndb
```

This URL tells H2 to create an in-memory database named `learndb`. The format is:

```
jdbc:h2:mem:learndb
|    |  |   |
|    |  |   +-- Database name
|    |  +------ "mem" means in-memory (data lost on restart)
|    +--------- H2 database
+-------------- JDBC protocol
```

Other H2 URL formats you might see:

```
jdbc:h2:mem:testdb           In-memory, named "testdb"
jdbc:h2:file:./data/mydb     File-based, persists to disk
jdbc:h2:tcp://host/db        Server mode, remote access
```

DDL Auto Strategy

```properties
spring.jpa.hibernate.ddl-auto=create-drop
```

This controls what Hibernate does with the database schema on startup:

```
Strategy        On Startup                On Shutdown      Use Case
-------------------------------------------------------------------------------
none            Nothing                   Nothing          Production
validate        Validate schema matches   Nothing          Production
                entities (error if not)
update          Add new tables/columns,   Nothing          Careful dev
                never drops anything
create          Drop and recreate all     Nothing          Development
                tables
create-drop     Drop and recreate all     Drop all         Testing and
                tables                    tables           learning
```

For learning, `create-drop` is perfect. Every time you start the app, you get a fresh database with the correct schema. In production, you would use `validate` (and manage schema changes with Flyway, covered in Chapter 26).

SQL Logging

```properties
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
logging.level.org.hibernate.orm.jdbc.bind=TRACE
```

These settings make Hibernate print every SQL statement it generates. This is one of the most important learning tools. When you call `repository.save(customer)`, you will see exactly what SQL Hibernate runs:

```sql
Hibernate:
    insert
    into
        customers
        (email, name, id)
    values
        (?, ?, default)

-- With bind parameter logging:
binding parameter (1:VARCHAR) <- [alice@example.com]
binding parameter (2:VARCHAR) <- [Alice Johnson]
```

Always enable SQL logging during development. Understanding what SQL Hibernate generates is essential for writing efficient code and debugging performance issues.

The H2 Console

One of H2's best features is its built-in web console. It lets you browse tables, run SQL queries, and inspect your data — all in your browser.

Accessing the Console

1. Start your application (we will do this shortly)
2. Open your browser to: `http://localhost:8080/h2-console`
3. Enter the connection settings:

```
+-----------------------------------------+
|         H2 Console Login                |
|                                         |
|  Driver Class: org.h2.Driver            |
```

```
|  JDBC URL:     jdbc:h2:mem:learndb        |
|  User Name:    sa                         |
|  Password:     (leave empty)              |
|                                           |
|  [Connect]                                |
+-------------------------------------------+
```

Important: The JDBC URL must match exactly what is in your `application.properties`. If you used `jdbc:h2:mem:learndb` in properties, use the same URL in the console.

What You Can Do in the Console

```
+-------------------------------------------------------------------+
|  H2 Console                                                       |
|                                                                   |
|  Left panel:           |  Right panel:                            |
|  +-----------------+   |  +-----------------------------------+   |
|  | Tables:         |   |  | SQL Editor:                       |   |
|  |   CUSTOMERS     |   |  |                                   |   |
|  |   ORDERS        |   |  | SELECT * FROM customers;          |   |
|  |   ORDER_ITEMS   |   |  |                                   |   |
|  +-----------------+   |  | [Run]                             |   |
|                        |  |                                   |   |
|  Click a table to      |  | Results:                          |   |
|  see its columns       |  | | ID | NAME  | EMAIL            | |   |
|  and auto-generate     |  | | 1  | Alice | alice@example.com | |   |
|  a SELECT query        |  | | 2  | Bob   | bob@example.com   | |   |
|                        |  +-----------------------------------+   |
+-------------------------------------------------------------------+
```

The H2 console is invaluable for verifying that your entities created the right tables, your data was saved correctly, and your relationships are set up properly.

Running Your Application

From the Command Line

Navigate to your project directory and run:

```bash
# Using Maven
mvn spring-boot:run
```

```
# Or using the Maven Wrapper
./mvnw spring-boot:run
```

From Your IDE

IntelliJ IDEA: Open `HibernateDemoApplication.java` and click the green play button next to the `main` method.

VS Code: Open `HibernateDemoApplication.java` and click "Run" above the `main` method (or press F5).

What You Should See

When the application starts successfully, you will see output like this in the console:

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.3.0)

Starting HibernateDemoApplication using Java 17.0.10
HikariPool-1 - Starting...
HikariPool-1 - Start completed.
H2 console available at '/h2-console'. Database available at 'jdbc:h2:mem:learndb'
Hibernate: create table ...
Tomcat started on port 8080
Started HibernateDemoApplication in 2.3 seconds
```

The key lines to look for:

```
Startup Flow:
+-------------------------------------------------------------------+
|  1. Spring Boot banner appears                                     |
|  2. HikariCP connection pool starts                                |
|  3. H2 console enabled at /h2-console                              |
|  4. Hibernate creates tables (you will see CREATE TABLE statements)  |
|  5. Tomcat starts on port 8080                                     |
|  6. "Started HibernateDemoApplication" -- success!                  |
+-------------------------------------------------------------------+
```

If you see errors, check the troubleshooting section at the end of this chapter.

Verifying Everything Works

1. Open `http://localhost:8080/h2-console` in your browser
2. Connect using the settings shown earlier
3. You should see an empty database (no tables yet — we will create entities in Chapter 3)

Stopping the Application

Press `Ctrl+C` in the terminal, or click the stop button in your IDE.

Project Structure Deep Dive

As your project grows, you will organize code into packages. Here is the standard package structure for a Spring Boot JPA application:

```
com.example.hibernatedemo/
|
+-- HibernateDemoApplication.java     Main class (already exists)
|
+-- entity/                           JPA entity classes
|   +-- Customer.java                 @Entity classes go here
|   +-- Order.java
|   +-- Product.java
|
+-- repository/                       Data access layer
|   +-- CustomerRepository.java       JpaRepository interfaces
|   +-- OrderRepository.java
|
+-- service/                          Business logic layer
|   +-- CustomerService.java          @Service classes
|   +-- OrderService.java
|
+-- controller/                       REST API layer
|   +-- CustomerController.java       @RestController classes
|   +-- OrderController.java
|
+-- dto/                              Data Transfer Objects
|   +-- CustomerRequest.java          Request/Response DTOs
|   +-- CustomerResponse.java
|
+-- config/                           Configuration classes
|   +-- AppConfig.java                @Configuration classes
|
+-- exception/                        Custom exceptions
    +-- ResourceNotFoundException.java
```

How the Layers Connect

```
+-------------------------------------------------------------------+
|                                                                   |
|   HTTP Request                                                    |
|       |                                                           |
|       v                                                           |
|   +------------------+                                            |
|   |   Controller     |  Receives HTTP requests, validates input,  |
|   |   (@RestController)  returns HTTP responses                   |
|   +------------------+                                            |
|           |                                                       |
|           v                                                       |
|   +------------------+                                            |
|   |   Service        |  Business logic, transaction boundaries,   |
|   |   (@Service)     |  orchestrates repository calls             |
|   +------------------+                                            |
|           |                                                       |
|           v                                                       |
|   +------------------+                                            |
|   |   Repository     |  Data access, JPA queries,                 |
|   |   (@Repository / |  Spring Data JPA auto-implementation       |
|   |    JpaRepository)|                                            |
|   +------------------+                                            |
|           |                                                       |
|           v                                                       |
|   +------------------+                                            |
|   |   Entity         |  JPA entities, database table mapping      |
|   |   (@Entity)      |                                            |
|   +------------------+                                            |
|           |                                                       |
|           v                                                       |
|   +------------------+                                            |
|   |   Hibernate/JPA  |  ORM engine, SQL generation, caching       |
|   +------------------+                                            |
|           |                                                       |
|           v                                                       |
|   +------------------+                                            |
|   |   H2 Database    |  In-memory database                        |
|   +------------------+                                            |
|                                                                   |
+-------------------------------------------------------------------+
```

Each layer has a specific responsibility:

- Controller: Handles HTTP. Never contains database logic.
- Service: Contains business rules. Manages transactions.
- Repository: Talks to the database. No business logic.
- Entity: Maps Java objects to database tables. Pure data.

We will build this structure gradually over the next several chapters.

Understanding Auto-Configuration

One thing that surprises beginners about Spring Boot is how much happens automatically. Let us trace what Spring Boot does when it sees `spring-boot-starter-data-jpa` and `h2` in your dependencies:

```
Spring Boot Auto-Configuration Flow:
+-------------------------------------------------------------------+
|                                                                   |
|  1. Scans classpath                                               |
|     - Finds hibernate-core.jar --> "JPA provider detected"        |
|     - Finds h2.jar --> "H2 database detected"                     |
|     - Finds spring-data-jpa.jar --> "Spring Data JPA detected"    |
|                                                                   |
|  2. Creates DataSource                                            |
|     - Reads spring.datasource.* properties                       |
|     - Creates HikariDataSource (connection pool)                  |
|     - Connects to jdbc:h2:mem:learndb                             |
|                                                                   |
|  3. Creates EntityManagerFactory                                  |
|     - Scans for @Entity classes                                   |
|     - Builds Hibernate SessionFactory                             |
|     - Applies ddl-auto strategy (create-drop)                     |
|     - Creates/validates database tables                           |
|                                                                   |
|  4. Creates TransactionManager                                    |
|     - JpaTransactionManager                                       |
|     - Makes @Transactional work                                   |
|                                                                   |
|  5. Creates Repository Implementations                            |
|     - Finds interfaces extending JpaRepository                    |
|     - Generates implementing classes at runtime                   |
|     - Registers them as Spring beans                              |
|                                                                   |
+-------------------------------------------------------------------+
```

All of this happens in about 2 seconds when you start the application. Without Spring Boot, you would need to configure each of these components manually with XML or Java configuration classes. Spring Boot's auto-configuration is one of its biggest productivity advantages.

Application Properties Reference

Here is a complete reference of the most important properties for Hibernate and JPA in Spring Boot:

```properties
# ===== DataSource =====
spring.datasource.url=                    # JDBC URL
spring.datasource.username=               # Database username
spring.datasource.password=               # Database password
spring.datasource.driver-class-name=      # JDBC driver class

# ===== JPA / Hibernate =====
spring.jpa.hibernate.ddl-auto=            # none|validate|update|create|create-drop
spring.jpa.show-sql=                      # true|false (log SQL)
spring.jpa.database-platform=             # Hibernate dialect class
spring.jpa.open-in-view=                  # true|false (OSIV, default true)
spring.jpa.generate-ddl=                  # true|false

# ===== Hibernate-Specific =====
spring.jpa.properties.hibernate.format_sql=          # true|false
spring.jpa.properties.hibernate.use_sql_comments=    # true|false
spring.jpa.properties.hibernate.jdbc.batch_size=     # e.g., 25
spring.jpa.properties.hibernate.order_inserts=       # true|false
spring.jpa.properties.hibernate.order_updates=       # true|false
spring.jpa.properties.hibernate.generate_statistics= # true|false
spring.jpa.properties.hibernate.default_batch_fetch_size= # e.g., 16

# ===== H2 Console =====
spring.h2.console.enabled=                # true|false
spring.h2.console.path=                   # e.g., /h2-console

# ===== Connection Pool (HikariCP) =====
spring.datasource.hikari.maximum-pool-size=    # default 10
spring.datasource.hikari.minimum-idle=         # default same as max
spring.datasource.hikari.connection-timeout=   # default 30000 (30s)

# ===== Logging =====
logging.level.org.hibernate.SQL=               # DEBUG (show SQL)
logging.level.org.hibernate.orm.jdbc.bind=     # TRACE (show params)
logging.level.org.hibernate.stat=              # DEBUG (statistics)
```

You do not need to memorize these. Bookmark this page and refer back when you need to adjust a setting.

Troubleshooting Common Setup Issues

Problem: "Port 8080 already in use"

Another application is using port 8080. Either stop it or change the port:

```properties
server.port=8081
```

Problem: "Cannot determine embedded database driver class"

Your `pom.xml` is missing the H2 dependency, or it has the wrong scope:

```xml
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
```

Problem: "Failed to configure a DataSource"

Spring Boot cannot find a database. Ensure your `application.properties` has:

```properties
spring.datasource.url=jdbc:h2:mem:learndb
```

Problem: H2 Console shows "Database not found"

Make sure the JDBC URL in the H2 console login matches exactly what is in `application.properties`:

```
In application.properties:     jdbc:h2:mem:learndb
In H2 Console login:           jdbc:h2:mem:learndb   <-- Must match!
```

Problem: "java: error: release version 17 not supported"

Your IDE is using an older JDK. In IntelliJ:

1. Go to File > Project Structure > Project
2. Set Project SDK to JDK 17
3. Set Language Level to 17

In VS Code, set `java.configuration.runtimes` in settings.

Problem: Maven dependencies not downloading

```bash
# Force Maven to re-download dependencies
mvn dependency:purge-local-repository
mvn clean install
```

Common Mistakes

1. Not enabling SQL logging: Without `spring.jpa.show-sql=true`, you cannot see what Hibernate is doing. This is the number one debugging tool for JPA applications. Always enable it during development.
2. Using `ddl-auto=update` in production: The `update` strategy tries to modify your database schema automatically. It never drops columns or tables, which can leave orphaned structures. In production, use `validate` and manage schema changes with Flyway (Chapter 26).
3. Forgetting the Maven Wrapper: If you share your project with others who do not have Maven installed, they need the `mvnw`/`mvnw.cmd` files. Always commit these to version control.
4. Wrong H2 Console URL: The JDBC URL in the H2 console login page defaults to `jdbc:h2:~/test`. You must change it to match your `application.properties` (e.g., `jdbc:h2:mem:learndb`).
5. Mixing javax and jakarta imports: If you copy code from older tutorials, you might accidentally use `javax.persistence.*` imports. Spring Boot 3.x and Hibernate 6.x require `jakarta.persistence.*`. If your IDE suggests `javax`, you have an old dependency somewhere.

Best Practices

1. Use Spring Initializr for new projects: It generates a correctly configured project with compatible dependency versions. Do not try to set up a Spring Boot project from scratch.
2. Commit the Maven Wrapper: Include `mvnw`, `mvnw.cmd`, and the `.mvn/` directory in version control. This ensures everyone on your team uses the same Maven version.
3. Use profiles for different environments: Create `application-dev.properties` for development and `application-prod.properties` for production. Activate them with `spring.profiles.active=dev`.

4. Enable SQL logging during development, disable in production: Logging every SQL statement helps you learn and debug, but it creates unnecessary log noise in production.
5. Pin your Java version: Set `<java.version>17</java.version>` in `pom.xml` so the project compiles with the same Java version everywhere.

Summary

In this chapter, you set up a complete development environment for learning Hibernate and JPA:

- JDK 17+ provides the Java compiler and runtime needed for Spring Boot 3.x and Hibernate 6.x.
- Maven manages your dependencies and build process. The `pom.xml` file declares what your project needs, and Maven downloads everything automatically.
- Spring Initializr generates a ready-to-run Spring Boot project with the right dependencies (`spring-boot-starter-data-jpa`, `h2`, `spring-boot-starter-web`).
- application.properties controls database connection, Hibernate behavior (ddl-auto, SQL logging), and the H2 console.
- H2 Console provides a web-based database browser at `/h2-console` where you can inspect tables and run SQL queries.
- Spring Boot auto-configuration detects your dependencies and automatically creates the DataSource, EntityManagerFactory, TransactionManager, and repository implementations.
- The standard project structure organizes code into entity, repository, service, controller, and dto packages — each with a clear responsibility.

Interview Questions

Q1: What is the difference between `ddl-auto=update` and `ddl-auto=create-drop`?

`update` adds new tables and columns to match your entities but never drops existing structures. It is additive only. `create-drop` drops all tables on startup, recreates them from entity definitions, and drops them again on shutdown. `create-drop` is for testing; `update` is sometimes used in development but is risky for production.

Q2: Why does Spring Boot only need one dependency (`spring-boot-starter-data-jpa`) to set up Hibernate?

The starter is a meta-dependency that pulls in all required libraries through transitive dependencies: Hibernate Core, Jakarta Persistence API, Spring Data JPA, HikariCP (connection pool), Spring JDBC, and Spring ORM. Spring Boot's auto-configuration then wires everything together based on what it finds on the classpath.

Q3: What is HikariCP and why is it important?

HikariCP is a high-performance JDBC connection pool. Instead of creating a new database connection for every request (which is expensive), HikariCP maintains a pool of reusable connections. It is the default connection pool in Spring Boot and is widely regarded as the fastest available.

Q4: What happens when Spring Boot starts and finds `@Entity` classes with `ddl-auto=create-drop`?

Hibernate scans for all classes annotated with `@Entity`, generates `CREATE TABLE` SQL statements based on the field annotations (`@Column`, `@Id`, etc.), and executes them against the database. When the application shuts down, it generates and executes `DROP TABLE` statements for all entity tables.

Q5: How does Spring Boot know to use Hibernate as the JPA provider?

Spring Boot's auto-configuration detects `hibernate-core` on the classpath (pulled in by `spring-boot-starter-data-jpa`). It then creates a `LocalContainerEntityManagerFactoryBean` configured with Hibernate as the provider. If you added EclipseLink to the classpath instead, Spring Boot would detect and use that.

Q6: What is the purpose of `spring.jpa.open-in-view`?

Open-Session-In-View (OSIV) keeps the Hibernate session open for the entire HTTP request lifecycle, including the view rendering phase. This allows lazy-loaded associations to be fetched during view rendering without a `LazyInitializationException`. It defaults to `true` in Spring Boot but is considered an anti-pattern because it can lead to N+1 queries and should be set to `false` in most applications.

Practice Exercises

Exercise 1: Create Your Project Follow the steps in this chapter to create a Spring Boot project with Spring Initializr. Start the application and verify that:

- The Spring Boot banner appears in the console
- The H2 console is accessible at `http://localhost:8080/h2-console`
- You can connect to the database using the credentials from `application.properties`

Exercise 2: Experiment with Properties Try changing the following properties one at a time and observe the effect:

1. Change `spring.jpa.show-sql` to `false` — what disappears from the console?
2. Change `spring.h2.console.path` to `/db` — where is the console now?
3. Change `server.port` to `9090` — what URL do you use?
4. Remove the `spring.datasource.url` line — what error do you get?

Exercise 3: Explore the Dependencies Run this Maven command to see all dependencies (including transitive ones):

```bash
mvn dependency:tree
```

Find and list: the version of Hibernate Core, the version of the Jakarta Persistence API, and the version of HikariCP. Notice how many JARs the single `spring-boot-starter-data-jpa` brings in.

Exercise 4: Profile Setup Create two property files:

- `application-dev.properties` with `spring.jpa.show-sql=true` and `spring.h2.console.enabled=true`
- `application-prod.properties` with `spring.jpa.show-sql=false` and `spring.h2.console.enabled=false`

Run the app with each profile (`-Dspring.profiles.active=dev` or `prod`) and observe the differences.

What Is Next?

In the next chapter, we will build your first JPA application from scratch. You will create an `@Entity` class, understand the JPA `EntityManager`, set up a `JpaRepository`, and perform your first database operations. You will see the SQL that Hibernate generates for each operation and verify the results in the H2 console. By the end of Chapter 3, you will have a working application that creates, reads, updates, and deletes data.

Chapter 3: Your First JPA Application

Learning Goals

By the end of this chapter, you will be able to:

- Create a JPA entity class with proper annotations
- Understand the role of EntityManager in JPA
- Create a Spring Data JPA repository interface
- Perform basic CRUD operations (create, read, update, delete)
- See and understand the SQL that Hibernate generates
- Use the H2 console to verify data in the database
- Load initial data using CommandLineRunner
- Understand the difference between EntityManager and JpaRepository approaches

Your First Entity: The Student Class

An entity is a Java class that maps to a database table. Each instance of the entity represents a row in that table. Each field in the class maps to a column.

Let us start with a simple example — a `Student` entity.

What We Want to Achieve

```
Java Object                          Database Table: students
+--------------------+               +------+----------+-------------------+------+
```

```
| Student            |                | id   | name     | email              | age  |
|--------------------|     Maps       |------|----------|--------------------|------|
| id: 1              |  =========>    | 1    | Alice    | alice@example.com  | 20   |
| name: "Alice"      |                | 2    | Bob      | bob@example.com    | 22   |
| email: "alice@..." |                | 3    | Charlie  | charlie@ex.com     | 21   |
| age: 20            |                +------+----------+--------------------+------+
+--------------------+
```

Creating the Entity Class

Create the file `src/main/java/com/example/hibernatedemo/entity/Student.java`:

```java
package com.example.hibernatedemo.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.persistence.Column;

@Entity                                     // JPA: Marks this class as a database entity
@Table(name = "students")                   // JPA: Maps to the "students" table
public class Student {

    @Id                                     // JPA: This field is the primary key
    @GeneratedValue(strategy =              // JPA: Database generates the ID
        GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false,               // JPA: Maps to "name" column, NOT NULL
            length = 100)
    private String name;

    @Column(nullable = false,               // JPA: Maps to "email" column, UNIQUE
            unique = true)
    private String email;

    private int age;                        // No @Column = uses defaults

    // Default constructor (REQUIRED by JPA)
    public Student() {
    }

    // Convenience constructor
    public Student(String name, String email, int age) {
        this.name = name;
        this.email = email;
        this.age = age;
    }
```

```
    // Getters and Setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    @Override
    public String toString() {
        return "Student{id=" + id + ", name='" + name + "', email='" + email + "', age=...
    }
}
```

Annotation Breakdown

Let us examine each annotation and what it does:

```
Annotation                    What It Does                     SQL Equivalent
--------------------------------------------------------------------------------
@Entity                       Marks class as a JPA entity      (tells Hibernate
                              (required for persistence)       to manage this class)

@Table(name = "students")     Maps class to table name         CREATE TABLE students
```

```
@Id                             Marks the primary key field      PRIMARY KEY

@GeneratedValue(strategy =      Auto-generate ID values          IDENTITY / AUTO_INCREMENT
  GenerationType.IDENTITY)

@Column(nullable = false,       Column constraints               name VARCHAR(100) NOT NULL
  length = 100)

@Column(unique = true)          Unique constraint                email VARCHAR(255) UNIQUE
```

What Hibernate Generates

When you start the application with `ddl-auto=create-drop`, Hibernate reads the annotations and generates this SQL:

```sql
Hibernate:
    create table students (
        age integer not null,
        id bigint generated by default as identity,
        email varchar(255) not null unique,
        name varchar(100) not null,
        primary key (id)
    )
```

Notice how the annotations translated directly into SQL constraints. This is the ORM at work — you described the table structure in Java, and Hibernate created the SQL.

The Three Requirements for a JPA Entity

Every JPA entity must satisfy three requirements:

```
+-----------------------------------------------------------------+
|                  JPA Entity Requirements                        |
|                                                                 |
|  1. @Entity annotation on the class                             |
|     (tells JPA this is a managed entity)                        |
|                                                                 |
|  2. @Id annotation on a field                                   |
|     (every entity must have a primary key)                      |
|                                                                 |
|  3. A public or protected no-argument constructor               |
|     (JPA creates instances using reflection)                    |
|                                                                 |
|  Note: The class must NOT be final, and                         |
|        the fields must NOT be final                              |
|        (Hibernate creates proxy subclasses)                     |
+-----------------------------------------------------------------+
```

If you forget any of these, you will get an error at startup. The most common mistake is forgetting the no-argument constructor (the default constructor disappears when you add any other constructor).

Understanding the EntityManager

Before we use Spring Data JPA's convenient `JpaRepository`, let us understand what happens underneath. The EntityManager is the core JPA interface for all database operations. Spring Data JPA uses it internally, so understanding it will help you debug issues and write custom queries later.

What the EntityManager Does

```
Your Code              EntityManager              Database
---------              -------------              --------

persist(student)  -->  Tracks the object     -->  INSERT INTO students ...
                       in the persistence
                       context (cache)

find(Student, 1)  -->  Checks cache first    -->  SELECT * FROM students
                       Returns from cache          WHERE id = 1
                       if found

merge(student)    -->  Copies state from     -->  UPDATE students SET ...
                       detached object to          WHERE id = 1
                       managed object

remove(student)   -->  Marks for deletion    -->  DELETE FROM students
                                                  WHERE id = 1
```

The Persistence Context

The EntityManager maintains a persistence context — a first-level cache that tracks all entity objects during a transaction. This is one of JPA's most important concepts:

```
+-------------------------------------------------------------------+
|                    Persistence Context                            |
|                    (First-Level Cache)                            |
|                                                                  |
```

```
|  Transaction begins                                                |
|  |                                                                 |
|  |  persist(student1)  --> student1 is now "managed"               |
|  |                         (tracked by EntityManager)              |
|  |                                                                 |
|  |  find(Student, 1)   --> returns student1 from cache             |
|  |                         (NO SQL executed!)                      |
|  |                                                                 |
|  |  student1.setName("New Name")                                   |
|  |                     --> EntityManager detects the change         |
|  |                         automatically ("dirty checking")        |
|  |                                                                 |
|  Transaction commits                                               |
|  |                                                                 |
|  |  EntityManager flushes all changes to the database:             |
|  |  --> INSERT INTO students ... (for persist)                     |
|  |  --> UPDATE students SET name = 'New Name' ... (for setName)    |
|  |                                                                 |
|  Persistence context is cleared                                    |
+-------------------------------------------------------------------+
```

Key behaviors to remember:

1. `persist()` does not immediately execute SQL. It adds the entity to the persistence context. SQL runs at flush time (usually when the transaction commits).
2. `find()` checks the cache first. If you call `find(Student.class, 1L)` twice in the same transaction, the second call returns the cached object without hitting the database.
3. Changes are detected automatically. If you modify a managed entity (`student.setName("...")`), Hibernate detects the change and generates an UPDATE — you do not need to call any save method.

Using EntityManager Directly

While Spring Data JPA is the recommended approach, understanding EntityManager helps you grasp what happens underneath:

```java
package com.example.hibernatedemo.repository;

import com.example.hibernatedemo.entity.Student;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;

@Repository
@Transactional
public class StudentEntityManagerRepository {
```

```
    @PersistenceContext                     // JPA: Injects the EntityManager
    private EntityManager entityManager;

    // CREATE
    public Student save(Student student) {
        entityManager.persist(student);      // Adds to persistence context
        return student;                      // ID is set after flush
    }

    // READ
    public Student findById(Long id) {
        return entityManager.find(Student.class, id);  // Returns null if not found
    }

    // UPDATE (managed entities are updated automatically)
    public Student update(Student student) {
        return entityManager.merge(student);  // Merges detached entity
    }

    // DELETE
    public void delete(Long id) {
        Student student = entityManager.find(Student.class, id);
        if (student != null) {
            entityManager.remove(student);    // Marks for deletion
        }
    }
}
```

This works, but it is a lot of boilerplate for basic CRUD. That is exactly what Spring Data JPA eliminates.

Creating a Spring Data JPA Repository

Spring Data JPA lets you define a repository as a simple interface. Spring creates the implementation automatically at runtime.

The Repository Interface

Create `src/main/java/com/example/hibernatedemo/repository/StudentRepository.java`:

```java
package com.example.hibernatedemo.repository;

import com.example.hibernatedemo.entity.Student;
```

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {

    // Spring Data JPA generates the implementation automatically!
    // You get all of these methods for free:
    //
    // save(Student)          - Create or update
    // findById(Long)         - Find by primary key
    // findAll()              - Find all students
    // deleteById(Long)       - Delete by primary key
    // count()                - Count all students
    // existsById(Long)       - Check if exists
    // ... and more
}
```

That is it. One interface, zero implementation code. Let us understand what is happening:

How Spring Data JPA Works

```
What You Write:
+-----------------------------------------+
|  public interface StudentRepository      |
|      extends JpaRepository<Student, Long> |
|  { }                                     |
+-----------------------------------------+
        |
        | Spring Boot scans for JpaRepository interfaces
        | at startup and generates implementations
        v
What Spring Creates (at runtime):
+-----------------------------------------+
|  public class StudentRepositoryImpl      |
|      implements StudentRepository {      |
|                                          |
|    @PersistenceContext                    |
|    EntityManager em;                     |
|                                          |
|    public Student save(Student s) {      |
|      if (s.getId() == null) {            |
|        em.persist(s);                    |
|      } else {                            |
|        em.merge(s);                      |
|      }                                   |
|      return s;                           |
|    }                                     |
|                                          |
```

```
|    public Optional<Student>               |
|        findById(Long id) {               |
|      Student s = em.find(                 |
|          Student.class, id);              |
|      return Optional.ofNullable(s);       |
|    }                                      |
|                                           |
|    // ... all other methods               |
|  }                                        |
+-------------------------------------------+
```

You never see or write this implementation. Spring generates it at startup using proxy patterns and the EntityManager.

The JpaRepository Hierarchy

`JpaRepository` extends several parent interfaces, each adding more methods:

```
Repository<T, ID>                    (marker interface, no methods)
    |
    v
CrudRepository<T, ID>               (basic CRUD: save, findById, delete, count)
    |
    v
ListCrudRepository<T, ID>           (same but returns List instead of Iterable)
    |
    v
PagingAndSortingRepository<T, ID>   (adds pagination and sorting)
    |
    v
ListPagingAndSortingRepository      (pagination returning List)
    |
    v
JpaRepository<T, ID>                (adds flush, saveAndFlush, batch operations)
```

By extending `JpaRepository<Student, Long>`, you get all methods from every interface in the chain. The two type parameters are:

- `Student` — the entity class
- `Long` — the type of the primary key (`@Id` field)

Performing CRUD Operations

Now let us put it all together. We will create a `CommandLineRunner` that demonstrates all four CRUD operations and shows the SQL Hibernate generates.

Loading Initial Data

Create `src/main/java/com/example/hibernatedemo/DataLoader.java`:

```java
package com.example.hibernatedemo;

import com.example.hibernatedemo.entity.Student;
import com.example.hibernatedemo.repository.StudentRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class DataLoader implements CommandLineRunner {

    private final StudentRepository studentRepository;

    public DataLoader(StudentRepository studentRepository) {
        this.studentRepository = studentRepository;
    }

    @Override
    public void run(String... args) {
        System.out.println("=== Starting Data Loader ===\n");

        // ---- CREATE ----
        System.out.println("--- CREATE: Saving students ---");

        Student alice = new Student("Alice Johnson", "alice@example.com", 20);
        Student bob = new Student("Bob Smith", "bob@example.com", 22);
        Student charlie = new Student("Charlie Brown", "charlie@example.com", 21);

        studentRepository.save(alice);
        studentRepository.save(bob);
        studentRepository.save(charlie);

        System.out.println("Saved: " + alice);   // ID is now set!
        System.out.println("Saved: " + bob);
        System.out.println("Saved: " + charlie);

        // ---- READ ----
        System.out.println("\n--- READ: Finding students ---");

        // Find by ID
        studentRepository.findById(1L).ifPresent(student ->
            System.out.println("Found by ID 1: " + student)
        );

        // Find all
```

```
        System.out.println("\nAll students:");
        studentRepository.findAll().forEach(student ->
            System.out.println("  " + student)
        );

        // Count
        long count = studentRepository.count();
        System.out.println("Total students: " + count);

        // ---- UPDATE ----
        System.out.println("\n--- UPDATE: Modifying a student ---");

        studentRepository.findById(1L).ifPresent(student -> {
            System.out.println("Before update: " + student);
            student.setAge(21);
            student.setEmail("alice.johnson@example.com");
            studentRepository.save(student);
            System.out.println("After update:  " + student);
        });

        // ---- DELETE ----
        System.out.println("\n--- DELETE: Removing a student ---");

        System.out.println("Exists before delete (ID 3): " +
            studentRepository.existsById(3L));
        studentRepository.deleteById(3L);
        System.out.println("Exists after delete (ID 3):  " +
            studentRepository.existsById(3L));

        // Final state
        System.out.println("\n--- Final state ---");
        studentRepository.findAll().forEach(student ->
            System.out.println("  " + student)
        );
        System.out.println("Total students: " + studentRepository.count());

        System.out.println("\n=== Data Loader Complete ===");
    }
}
```

What Is CommandLineRunner?

```
Spring Boot Startup Flow:
+-------------------------------------------------------------------+
|                                                                   |
|  1. Spring Boot starts                                            |
|  2. Auto-configuration runs (DataSource, EntityManager, etc.)     |
|  3. Hibernate creates tables (ddl-auto=create-drop)               |
|  4. Component scanning finds @Component classes                  |
|  5. CommandLineRunner beans execute (our DataLoader)               |
|  6. Application is ready                                          |
```

```
|                                                                      |
|  CommandLineRunner is perfect for:                                   |
|  - Loading seed data                                                 |
|  - Running one-time setup                                            |
|  - Demonstrating features (like in this book)                        |
+----------------------------------------------------------------------+
```

Running the Application

Start the application and watch the console output. You will see Hibernate's SQL interleaved with your print statements:

CREATE — What Hibernate does when you call `save()`:

```sql
Hibernate:
    insert
    into
        students
        (age, email, name, id)
    values
        (?, ?, ?, default)
```

The `?` marks are bind parameters. With our logging configuration, you also see:

```
binding parameter (1:INTEGER) <- [20]
binding parameter (2:VARCHAR) <- [alice@example.com]
binding parameter (3:VARCHAR) <- [Alice Johnson]
```

READ — What Hibernate does when you call `findById()`:

```sql
Hibernate:
    select
        s1_0.id,
        s1_0.age,
        s1_0.email,
        s1_0.name
    from
        students s1_0
    where
        s1_0.id=?
```

READ ALL — What Hibernate does when you call `findAll()`:

```sql
Hibernate:
    select
        s1_0.id,
        s1_0.age,
        s1_0.email,
        s1_0.name
    from
        students s1_0
```

UPDATE — What Hibernate does when you modify a managed entity and call `save()`:

```sql
Hibernate:
    update
        students
    set
        age=?,
        email=?,
        name=?
    where
        id=?
```

DELETE — What Hibernate does when you call `deleteById()`:

```sql
Hibernate:
    select
        s1_0.id,
        s1_0.age,
        s1_0.email,
        s1_0.name
    from
        students s1_0
    where
        s1_0.id=?

Hibernate:
    delete
    from
        students
    where
        id=?
```

Notice that `deleteById()` first loads the entity with a SELECT (Hibernate needs the managed entity to trigger lifecycle events), then issues the DELETE.

The Complete CRUD Flow

```
Operation         Java Code                    SQL Generated
------------------------------------------------------------------------
CREATE            repository.save(new)         INSERT INTO students ...
READ (one)        repository.findById(1L)      SELECT ... WHERE id = 1
READ (all)        repository.findAll()         SELECT ... FROM students
UPDATE            entity.setName("...");       UPDATE students SET ...
                  repository.save(entity)      WHERE id = ?
DELETE            repository.deleteById(1L)    SELECT ... WHERE id = 1
                                               DELETE ... WHERE id = 1
EXISTS            repository.existsById(1L)    SELECT count(*)
                                               FROM students WHERE id = 1
COUNT             repository.count()           SELECT count(*)
                                               FROM students
```

Verifying Data in the H2 Console

After the application starts and the DataLoader runs, let us inspect the data:

1. Open `http://localhost:8080/h2-console`
2. Connect with JDBC URL: `jdbc:h2:mem:learndb`, Username: `sa`, Password: (empty)
3. You should see the `STUDENTS` table in the left panel

Run these queries to verify:

```sql
-- See all students
SELECT * FROM students;

-- Check the table structure
SHOW COLUMNS FROM students;

-- Count students
SELECT COUNT(*) FROM students;
```

Expected Results After DataLoader

```
H2 Console Query Results:

SELECT * FROM students;

+----+-----+--------------------------+--------------+
| ID | AGE | EMAIL                    | NAME         |
+----+-----+--------------------------+--------------+
| 1  | 21  | alice.johnson@example.com| Alice Johnson|
```

```
| 2  | 22  | bob@example.com          | Bob Smith    |
+----+-----+--------------------------+--------------+

Note: Charlie (ID 3) was deleted by the DataLoader.
      Alice's age and email were updated.
```

Understanding save() in Detail

The `save()` method in Spring Data JPA does something clever — it decides whether to INSERT or UPDATE based on the entity's ID:

```
repository.save(entity)
    |
    +-- Is entity.getId() == null?
    |       |
    |       +-- YES: This is a NEW entity
    |       |       --> entityManager.persist(entity)
    |       |       --> SQL: INSERT INTO students ...
    |       |       --> Generated ID is set on the entity
    |       |
    |       +-- NO: This is an EXISTING entity
    |               --> entityManager.merge(entity)
    |               --> SQL: SELECT ... (load current state)
    |               --> SQL: UPDATE students SET ... WHERE id = ?
    |
    +-- Returns the saved/updated entity
```

This means:

```java
// CREATE: id is null, so save() does INSERT
Student student = new Student("Alice", "alice@example.com", 20);
studentRepository.save(student);  // INSERT
// student.getId() is now 1 (set by Hibernate)

// UPDATE: id is 1 (not null), so save() does UPDATE
student.setAge(21);
studentRepository.save(student);  // UPDATE
```

persist() vs merge()

This distinction is important for understanding JPA behavior:

```
+-------------------------------------------------------------------+
|  persist()                      |  merge()                          |
|  (for NEW entities)             |  (for EXISTING/DETACHED entities) |
|                                 |                                   |
|  - Entity must NOT have an ID   |  - Entity must have an ID         |
|  - Makes the entity "managed"   |  - Returns a NEW managed copy     |
|  - Throws exception if entity   |  - The original object remains    |
|    already has an ID (some       |    detached                       |
|    providers)                   |  - If entity does not exist in    |
|  - More efficient for inserts   |    DB, creates a new one          |
+-------------------------------------------------------------------+
```

Spring Data JPA's `save()` method handles this decision for you, which is why it is the preferred approach.

Adding Custom Query Methods

Your `StudentRepository` currently only has the methods inherited from `JpaRepository`. You can add custom finder methods by following Spring Data JPA's naming conventions:

```java
package com.example.hibernatedemo.repository;

import com.example.hibernatedemo.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {

    // Spring Data JPA generates the query from the method name!

    // Find by exact email
    Optional<Student> findByEmail(String email);

    // Find all students with a specific name
    List<Student> findByName(String name);

    // Find students older than a given age
    List<Student> findByAgeGreaterThan(int age);

    // Find students by name containing a string (LIKE query)
    List<Student> findByNameContaining(String keyword);
```

```
    // Check if a student with this email exists
    boolean existsByEmail(String email);

    // Count students with a specific age
    long countByAge(int age);

    // Delete all students with a given name
    void deleteByName(String name);
}
```

How Method Name Parsing Works

Spring Data JPA parses the method name and generates JPQL/SQL:

```
Method Name                          Generated SQL
----------------------------------------------------------------------
findByEmail(email)                   SELECT ... WHERE email = ?

findByName(name)                     SELECT ... WHERE name = ?

findByAgeGreaterThan(age)            SELECT ... WHERE age > ?

findByNameContaining(keyword)        SELECT ... WHERE name LIKE '%keyword%'

existsByEmail(email)                 SELECT count(*) > 0
                                     WHERE email = ?

countByAge(age)                      SELECT count(*)
                                     WHERE age = ?

deleteByName(name)                   DELETE FROM students
                                     WHERE name = ?
```

Method Name Keywords

```
Keyword                Example                          SQL Clause
----------------------------------------------------------------------
And                    findByNameAndAge                 WHERE name=? AND age=?
Or                     findByNameOrEmail                WHERE name=? OR email=?
Is, Equals             findByNameIs                     WHERE name=?
Between                findByAgeBetween                 WHERE age BETWEEN ? AND ?
LessThan               findByAgeLessThan                WHERE age < ?
GreaterThan            findByAgeGreaterThan             WHERE age > ?
Like                   findByNameLike                   WHERE name LIKE ?
Containing             findByNameContaining             WHERE name LIKE '%?%'
StartingWith           findByNameStartingWith           WHERE name LIKE '?%'
```

```
EndingWith              findByNameEndingWith             WHERE name LIKE '%?'
OrderBy                 findByAgeOrderByNameAsc          ... ORDER BY name ASC
Not                     findByNameNot                    WHERE name <> ?
In                      findByAgeIn(Collection)          WHERE age IN (?, ?, ?)
True/False              findByActiveTrue                 WHERE active = true
IsNull                  findByEmailIsNull                WHERE email IS NULL
IsNotNull               findByEmailIsNotNull             WHERE email IS NOT NULL
```

We will explore derived query methods in much greater depth in Chapter 7.

Testing Custom Methods

Update your `DataLoader` to test some custom queries:

```java
@Override
public void run(String... args) {
    // Save some students
    studentRepository.save(new Student("Alice Johnson", "alice@example.com", 20));
    studentRepository.save(new Student("Bob Smith", "bob@example.com", 22));
    studentRepository.save(new Student("Charlie Brown", "charlie@example.com", 21));
    studentRepository.save(new Student("Alice Cooper", "cooper@example.com", 25));

    // Custom query: find by email
    System.out.println("\n--- Find by email ---");
    studentRepository.findByEmail("bob@example.com")
        .ifPresent(s -> System.out.println("Found: " + s));

    // Custom query: find by name containing
    System.out.println("\n--- Name containing 'Alice' ---");
    studentRepository.findByNameContaining("Alice")
        .forEach(s -> System.out.println("  " + s));

    // Custom query: age greater than 21
    System.out.println("\n--- Age greater than 21 ---");
    studentRepository.findByAgeGreaterThan(21)
        .forEach(s -> System.out.println("  " + s));

    // Custom query: exists by email
    System.out.println("\n--- Exists by email ---");
    System.out.println("alice@example.com exists: " +
        studentRepository.existsByEmail("alice@example.com"));
    System.out.println("nobody@example.com exists: " +
        studentRepository.existsByEmail("nobody@example.com"));

    // Custom query: count by age
    System.out.println("\n--- Count by age ---");
    System.out.println("Students aged 20: " + studentRepository.countByAge(20));
}
```

EntityManager vs JpaRepository: When to Use Which

You now know two ways to interact with the database. Here is when to use each:

```
+-----------------------------------------------------------------------+
|  Approach              | Use When                                      |
|------------------------|-----------------------------------------------|
|  JpaRepository         | - Standard CRUD operations                    |
|  (Spring Data JPA)     | - Derived query methods (findByName, etc.)    |
|                        | - Pagination and sorting                      |
|                        | - 95% of your data access needs               |
|                        |                                               |
|  EntityManager         | - Complex custom queries                      |
|  (JPA core)            | - Batch operations                            |
|                        | - Direct control over persistence context      |
|                        | - Stored procedure calls                      |
|                        | - When you need flush/clear/detach            |
+-----------------------------------------------------------------------+
```

In this book, we will primarily use `JpaRepository` because it handles the vast majority of use cases with minimal code. We will bring in `EntityManager` when we need fine-grained control (Chapter 17: Entity Lifecycle and Persistence Context).

A Complete Working Example

Let us put together a clean, complete example that you can run and experiment with.

Project Structure

```
src/main/java/com/example/hibernatedemo/
|
+-- HibernateDemoApplication.java       (main class)
+-- entity/
|   +-- Student.java                    (JPA entity)
+-- repository/
|   +-- StudentRepository.java          (Spring Data JPA repository)
+-- DataLoader.java                     (seed data and demos)
```

The Entity (Student.java)

```java
package com.example.hibernatedemo.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    private int age;

    public Student() {}

    public Student(String name, String email, int age) {
        this.name = name;
        this.email = email;
        this.age = age;
    }

    // Getters, setters, toString...

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }

    @Override
    public String toString() {
        return "Student{id=" + id + ", name='" + name +
               "', email='" + email + "', age=" + age + "}";
    }
}
```

The Repository (StudentRepository.java)

```java
package com.example.hibernatedemo.repository;

import com.example.hibernatedemo.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {

    Optional<Student> findByEmail(String email);

    List<Student> findByNameContaining(String keyword);

    List<Student> findByAgeGreaterThan(int age);

    List<Student> findByAgeBetween(int min, int max);

    boolean existsByEmail(String email);

    long countByAge(int age);
}
```

The Data Loader (DataLoader.java)

```java
package com.example.hibernatedemo;

import com.example.hibernatedemo.entity.Student;
import com.example.hibernatedemo.repository.StudentRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class DataLoader implements CommandLineRunner {

    private final StudentRepository repository;

    public DataLoader(StudentRepository repository) {
        this.repository = repository;
    }

    @Override
    public void run(String... args) {

        // === CREATE ===
        System.out.println("\n========== CREATE ==========");
        Student s1 = repository.save(new Student("Alice Johnson", "alice@university.com...
        Student s2 = repository.save(new Student("Bob Smith", "bob@university.com", 22));
```

```
        Student s3 = repository.save(new Student("Charlie Brown", "charlie@university.c...
        Student s4 = repository.save(new Student("Diana Prince", "diana@university.com"...
        Student s5 = repository.save(new Student("Eve Williams", "eve@university.com", ...
        System.out.println("Created 5 students. IDs: " +
            s1.getId() + ", " + s2.getId() + ", " + s3.getId() + ", " +
            s4.getId() + ", " + s5.getId());

        // === READ ===
        System.out.println("\n========== READ ==========");

        System.out.println("\nAll students:");
        repository.findAll().forEach(s -> System.out.println("  " + s));

        System.out.println("\nFind by ID 2:");
        repository.findById(2L).ifPresentOrElse(
            s -> System.out.println("  Found: " + s),
            () -> System.out.println("  Not found")
        );

        System.out.println("\nFind by email 'charlie@university.com':");
        repository.findByEmail("charlie@university.com")
            .ifPresent(s -> System.out.println("  Found: " + s));

        System.out.println("\nStudents with 'a' in name:");
        repository.findByNameContaining("a")
            .forEach(s -> System.out.println("  " + s));

        System.out.println("\nStudents older than 21:");
        repository.findByAgeGreaterThan(21)
            .forEach(s -> System.out.println("  " + s));

        System.out.println("\nStudents aged 20-21:");
        repository.findByAgeBetween(20, 21)
            .forEach(s -> System.out.println("  " + s));

        System.out.println("\nTotal count: " + repository.count());
        System.out.println("Count aged 20: " + repository.countByAge(20));
        System.out.println("Email exists (alice@university.com): " +
            repository.existsByEmail("alice@university.com"));

        // === UPDATE ===
        System.out.println("\n========== UPDATE ==========");

        repository.findById(1L).ifPresent(student -> {
            System.out.println("Before: " + student);
            student.setAge(21);
            student.setEmail("alice.j@university.com");
            repository.save(student);
            System.out.println("After:  " + student);
        });

        // === DELETE ===
        System.out.println("\n========== DELETE ==========");

        System.out.println("Before delete - count: " + repository.count());
```

```
        repository.deleteById(5L);
        System.out.println("After delete  - count: " + repository.count());

        // === FINAL STATE ===
        System.out.println("\n========== FINAL STATE ==========");
        repository.findAll().forEach(s -> System.out.println("  " + s));
    }
}
```

Expected Console Output

```
========== CREATE ==========
Hibernate: insert into students (age,email,name,id) values (?,?,?,default)
Hibernate: insert into students (age,email,name,id) values (?,?,?,default)
Hibernate: insert into students (age,email,name,id) values (?,?,?,default)
Hibernate: insert into students (age,email,name,id) values (?,?,?,default)
Hibernate: insert into students (age,email,name,id) values (?,?,?,default)
Created 5 students. IDs: 1, 2, 3, 4, 5

========== READ ==========

All students:
Hibernate: select s1_0.id,s1_0.age,s1_0.email,s1_0.name from students s1_0
  Student{id=1, name='Alice Johnson', email='alice@university.com', age=20}
  Student{id=2, name='Bob Smith', email='bob@university.com', age=22}
  Student{id=3, name='Charlie Brown', email='charlie@university.com', age=21}
  Student{id=4, name='Diana Prince', email='diana@university.com', age=23}
  Student{id=5, name='Eve Williams', email='eve@university.com', age=20}

Find by ID 2:
Hibernate: select ... from students s1_0 where s1_0.id=?
  Found: Student{id=2, name='Bob Smith', email='bob@university.com', age=22}

Students older than 21:
Hibernate: select ... from students s1_0 where s1_0.age>?
  Student{id=2, name='Bob Smith', email='bob@university.com', age=22}
  Student{id=4, name='Diana Prince', email='diana@university.com', age=23}

========== UPDATE ==========
Hibernate: select ... where s1_0.id=?
Before: Student{id=1, name='Alice Johnson', email='alice@university.com', age=20}
Hibernate: update students set age=?,email=?,name=? where id=?
After:  Student{id=1, name='Alice Johnson', email='alice.j@university.com', age=21}

========== DELETE ==========
Before delete - count: 5
Hibernate: select ... where s1_0.id=?
Hibernate: delete from students where id=?
After delete  - count: 4

========== FINAL STATE ==========
```

```
  Student{id=1, name='Alice Johnson', email='alice.j@university.com', age=21}
  Student{id=2, name='Bob Smith', email='bob@university.com', age=22}
  Student{id=3, name='Charlie Brown', email='charlie@university.com', age=21}
  Student{id=4, name='Diana Prince', email='diana@university.com', age=23}
```

What Happens Behind the Scenes: The Full Picture

Let us trace a single `save()` call from your code all the way to the database:

```
Your Code                                  What Happens
----------                                 -------------------------

repository.save(                           1. Spring Data JPA receives the call
  new Student("Alice",                     2. Checks if entity is new (id == null)
    "alice@uni.com", 20)                      --> Yes, it is new
);                                         3. Calls entityManager.persist(student)
                                           4. Hibernate adds student to the
                                              persistence context (first-level cache)
                                           5. Hibernate assigns a generated ID
                                              (depends on strategy)
                                           6. At flush time (transaction commit):
                                              a. Generates SQL:
                                                 INSERT INTO students
                                                 (age, email, name)
                                                 VALUES (20, 'alice@uni.com', 'Alice')
                                              b. Sends SQL to JDBC driver
                                              c. JDBC driver sends to H2 database
                                              d. H2 executes the INSERT
                                              e. H2 returns the generated ID
                                              f. Hibernate sets id on the entity
                                           7. Returns the entity with ID set
```

```
Full Stack Trace for repository.save():

+-----------------------------------------------------+
| StudentRepository.save(student)                     |  Spring Data JPA
+-----------------------------------------------------+
            |
            v
+-----------------------------------------------------+
| SimpleJpaRepository.save(student)                   |  Spring Data JPA
|   if (isNew(student))                               |  (auto-generated impl)
|     entityManager.persist(student)                  |
|   else                                              |
|     entityManager.merge(student)                    |
+-----------------------------------------------------+
```

```
            |
            v
+---------------------------------------------------+
| EntityManager.persist(student)                    |  JPA API
+---------------------------------------------------+
            |
            v
+---------------------------------------------------+
| SessionImpl.persist(student)                      |  Hibernate
|   - Add to persistence context                    |  (JPA provider)
|   - Schedule INSERT action                        |
|   - Generate ID (if IDENTITY strategy)            |
+---------------------------------------------------+
            |
            v (at flush/commit time)
+---------------------------------------------------+
| ActionQueue.executeInserts()                      |  Hibernate
|   - Generate SQL from entity metadata             |  (internal)
|   - Bind parameters                               |
+---------------------------------------------------+
            |
            v
+---------------------------------------------------+
| PreparedStatement.executeUpdate()                 |  JDBC
|   INSERT INTO students (age,email,name)           |
|   VALUES (20, 'alice@uni.com', 'Alice')           |
+---------------------------------------------------+
            |
            v
+---------------------------------------------------+
| H2 Database Engine                                |  Database
|   - Validates constraints (NOT NULL, UNIQUE)      |
|   - Inserts row                                   |
|   - Returns generated ID                          |
+---------------------------------------------------+
```

Understanding this full stack helps you debug issues. If you get a `ConstraintViolationException`, it comes from the database level. If you get a `TransientPropertyValueException`, it comes from Hibernate's validation. If you get a `DataIntegrityViolationException`, it is Spring wrapping a database error.

Common Mistakes

1. Forgetting the no-argument constructor: JPA creates entity instances using reflection, which requires a no-arg constructor. If you define any constructor, Java no longer provides the default one, so you must add it explicitly:

```java
// WRONG: Only has the parameterized constructor
@Entity
public class Student {
    public Student(String name, String email) { ... }
    // Crash! JPA cannot create instances
}

// CORRECT: Also has a no-arg constructor
@Entity
public class Student {
    public Student() {}  // Required by JPA
    public Student(String name, String email) { ... }
}
```

2. Making the entity class or fields `final`: Hibernate creates proxy subclasses for lazy loading. If the class is `final`, it cannot be subclassed:

```java
// WRONG
@Entity
public final class Student { ... }

// WRONG
@Entity
public class Student {
    private final String name;  // Cannot be set by reflection
}
```

3. Not using `Optional` with `findById()`: `findById()` returns `Optional<Student>`, not `Student`. Always handle the case where the entity might not exist:

```java
// WRONG: Can throw NoSuchElementException
Student student = repository.findById(999L).get();

// CORRECT: Handle the missing case
repository.findById(999L).ifPresentOrElse(
    student -> System.out.println("Found: " + student),
    () -> System.out.println("Student not found")
);

// ALSO CORRECT: With orElseThrow
Student student = repository.findById(999L)
    .orElseThrow(() -> new RuntimeException("Student not found"));
```

4. Expecting save() to always do INSERT: `save()` does INSERT when the ID is null and UPDATE (via merge) when the ID is not null. If you create a `new Student()` and manually set its ID, `save()` will try to UPDATE, not INSERT.
5. Not looking at the generated SQL: Always have `spring.jpa.show-sql=true` during development. If a simple operation is generating 10 SQL queries, something is wrong (probably an N+1 problem, covered in Chapter 18).

Best Practices

1. Always use `JpaRepository` as your starting point: It provides everything you need for basic operations. Only drop down to `EntityManager` when you need fine-grained control.
2. Use Optional returns for single-entity queries: `findById()` returns `Optional<Student>`, which forces callers to handle the null case. Define your custom finder methods to return `Optional` for single results:

```java
Optional<Student> findByEmail(String email);  // Good
Student findByEmail(String email);            // Returns null if not found (risky)
```

3. Use constructor injection for repositories: Always inject repositories through the constructor, not field injection:

```java
// GOOD: Constructor injection
private final StudentRepository repository;
public DataLoader(StudentRepository repository) {
    this.repository = repository;
}

// AVOID: Field injection
@Autowired
private StudentRepository repository;
```

4. Keep entities simple: Entity classes should only contain field mappings, getters, setters, and `toString()`. Business logic belongs in service classes.
5. Name your tables explicitly: Use `@Table(name = "students")` rather than relying on the default naming strategy. This makes your code self-documenting and avoids surprises when entity names change.

Summary

In this chapter, you built your first complete JPA application:

- Entity classes use `@Entity`, `@Id`, `@GeneratedValue`, `@Column`, and `@Table` annotations to map Java objects to database tables. These are all JPA standard annotations from `jakarta.persistence`.

- EntityManager is the core JPA interface. It provides `persist()`, `find()`, `merge()`, and `remove()` for direct database operations. Understanding it helps you debug JPA issues.
- The persistence context is a first-level cache managed by EntityManager. It tracks entity state changes and flushes them to the database at transaction commit time.
- JpaRepository is a Spring Data JPA interface that auto-implements all basic CRUD operations. You define an interface; Spring generates the implementation at startup.
- Derived query methods let you define custom queries by naming your method according to conventions (e.g., `findByEmail`, `findByAgeGreaterThan`). Spring parses the method name and generates the query.
- save() is smart — it calls `persist()` for new entities (null ID) and `merge()` for existing entities (non-null ID).
- SQL logging (`spring.jpa.show-sql=true`) shows you exactly what Hibernate does for each operation. Always enable it during development.

Interview Questions

Q1: What are the minimum requirements for a JPA entity class?

A JPA entity must have: (1) the `@Entity` annotation on the class, (2) an `@Id` annotation on a field to define the primary key, and (3) a public or protected no-argument constructor. Additionally, the class and its persistent fields must not be `final`.

Q2: What is the difference between `persist()` and `merge()` in JPA?

`persist()` is for new entities that are not yet in the database. It makes the entity managed and schedules an INSERT. `merge()` is for detached entities (previously managed but no longer tracked). It copies the state of the detached entity onto a new managed instance and schedules an UPDATE. `persist()` modifies the original object; `merge()` returns a new managed copy.

Q3: How does Spring Data JPA's `save()` method decide whether to INSERT or UPDATE?

It checks if the entity is "new" by examining the `@Id` field. If the ID is `null` (or zero for primitive types), the entity is considered new, and `persist()` is called (INSERT). If the ID has a value, `merge()` is called (UPDATE). This behavior can be customized by implementing the `Persistable` interface.

Q4: What is the persistence context in JPA?

The persistence context is a first-level cache managed by the EntityManager within a transaction. It tracks all managed entity instances. If you load the same entity twice in a

transaction, the second call returns the cached instance. It also performs "dirty checking" — detecting changes to managed entities and automatically generating UPDATE statements at flush time.

Q5: How do derived query methods work in Spring Data JPA?

Spring Data JPA parses the method name to build a query. For example, `findByNameContaining(String keyword)` is parsed as: `find` (query type) + `By` (predicate start) + `Name` (field name) + `Containing` (LIKE with wildcards). It generates: `SELECT ... FROM students WHERE name LIKE '%keyword%'`. The method must follow specific naming conventions documented by Spring Data.

Q6: Why does `deleteById()` execute a SELECT before the DELETE?

JPA requires an entity to be in the managed state before it can be removed. `deleteById()` first loads the entity using `findById()` (SELECT), makes it managed, then calls `remove()` (DELETE). This ensures lifecycle callbacks (`@PreRemove`, `@PostRemove`) are triggered. If you want to delete without loading the entity first, you can use a custom `@Query` with a DELETE statement.

Q7: What is the difference between `JpaRepository` and `CrudRepository`?

`CrudRepository` provides basic CRUD operations: `save`, `findById`, `findAll`, `delete`, `count`, `existsById`. `JpaRepository` extends `CrudRepository` and adds JPA-specific methods: `flush()`, `saveAndFlush()`, `saveAllAndFlush()`, `deleteInBatch()`, `deleteAllInBatch()`, `getById()`. It also extends `PagingAndSortingRepository` for pagination. For most applications, `JpaRepository` is the right choice.

Practice Exercises

Exercise 1: Create a Course Entity Create a `Course` entity with fields: `id` (Long, auto-generated), `title` (String, not null, max 200 chars), `code` (String, not null, unique, max 10 chars), `credits` (int), and `department` (String). Create a `CourseRepository` and a `DataLoader` that saves five courses and demonstrates all CRUD operations.

Exercise 2: Experiment with Derived Queries Add these methods to your `CourseRepository` and test them:

- Find courses by department
- Find courses with credits greater than a given value
- Find courses whose title contains a keyword

- Count courses by department
- Check if a course code exists

Exercise 3: Compare EntityManager and JpaRepository Write the same four operations (create, read, update, delete) using both `EntityManager` directly and `JpaRepository`. Compare the amount of code needed for each approach.

Exercise 4: Observe SQL Generation With `show-sql=true`, perform the following operations and record the SQL generated:

1. Save a new student
2. Call `findById()` twice for the same ID within the same method
3. Find a student, change their name, and call `save()`
4. Call `deleteById()` for an ID that does not exist

Analyze: How many SQL statements were generated? Were any unnecessary?

Exercise 5: Constraint Violations Try to save two students with the same email (which has a unique constraint). Observe the exception that is thrown. Then try to save a student with a null name (which has a not-null constraint). What exceptions do you get? How would you handle these gracefully?

What Is Next?

In the next chapter, we will take a deep dive into JPA annotations. You will learn every annotation you need for mapping Java classes to database tables: `@Column` options (length, precision, scale, insertable, updatable), `@Enumerated` for enum mapping, `@Temporal` for date types, `@Lob` for large objects, `@Transient` for non-persisted fields, naming strategies, and Hibernate-specific annotations like `@NaturalId`, `@Formula`, and `@DynamicUpdate`. By the end of Chapter 4, you will be able to map any Java class to any table structure.

Chapter 4: Understanding JPA Annotations in Depth

Learning Goals

By the end of this chapter, you will be able to:

- Use @Column to control column names, length, precision, nullability, and more
- Map Java enums to database columns with @Enumerated
- Handle date and time fields with Java 8+ types
- Store large text and binary data with @Lob
- Exclude fields from persistence with @Transient
- Use @Basic to control fetch behavior on individual fields
- Understand implicit vs physical naming strategies
- Use Hibernate-specific annotations: @NaturalId, @Formula, @DynamicUpdate, @DynamicInsert
- Know which annotations are JPA standard and which are Hibernate-specific

The Annotation Map

Before we dive into individual annotations, here is a visual map showing how JPA annotations map a Java class to a database table:

```
Java Class                              Database Table
+-------------------------------+       +-----------------------------------+
|  @Entity                      |       |                                   |
|  @Table(name = "products")    | ====> |  TABLE: products                  |
|                               |       |                                   |
```

```
|  @Id                           |       |  id BIGINT PRIMARY KEY           |
|  @GeneratedValue(IDENTITY)     |       |    GENERATED BY DEFAULT          |
|  private Long id;              |       |    AS IDENTITY                   |
|                                |       |                                  |
|  @Column(name = "product_name"|       |  product_name VARCHAR(200)       |
|    nullable = false,           |       |    NOT NULL                      |
|    length = 200)               |       |                                  |
|  private String name;          |       |                                  |
|                                |       |                                  |
|  @Column(precision = 10,       |       |  price DECIMAL(10,2)             |
|    scale = 2)                  |       |                                  |
|  private BigDecimal price;     |       |                                  |
|                                |       |                                  |
|  @Enumerated(STRING)           |       |  status VARCHAR(255)             |
|  private Status status;        |       |    (stores "ACTIVE", "DRAFT")    |
|                                |       |                                  |
|  @Lob                          |       |  description CLOB                |
|  private String description;   |       |    (large text)                  |
|                                |       |                                  |
|  @Transient                    |       |  (not in the table)              |
|  private double discount;      |       |                                  |
|                                |       |                                  |
|  private LocalDateTime         |       |  created_at TIMESTAMP            |
|    createdAt;                  |       |                                  |
+--------------------------------+       +----------------------------------+
```

@Entity and @Table

You have already seen these in Chapter 3. Let us understand their full options.

@Entity

```java
@Entity                           // Uses class name as entity name
@Entity(name = "ProductEntity")  // Custom entity name (used in JPQL)
```

The `name` attribute of `@Entity` sets the entity name used in JPQL queries, not the table name:

```java
@Entity(name = "ProductEntity")
public class Product { ... }

// In JPQL, you use the entity name:
// SELECT p FROM ProductEntity p WHERE p.price > 100
```

Most of the time, you do not need to set the entity name. The default (class name) is fine.

JPA Standard: `@Entity` is defined in `jakarta.persistence`.

@Table

```java
@Table(
    name = "products",                // Table name
    schema = "inventory",             // Database schema (optional)
    catalog = "mydb",                 // Database catalog (optional)
    uniqueConstraints = {             // Table-level unique constraints
        @UniqueConstraint(
            name = "uk_product_code",
            columnNames = {"code"}
        ),
        @UniqueConstraint(
            name = "uk_name_category",
            columnNames = {"name", "category"}  // Composite unique
        )
    },
    indexes = {                       // Table indexes
        @Index(
            name = "idx_product_name",
            columnList = "name"
        ),
        @Index(
            name = "idx_category_price",
            columnList = "category, price DESC"
        )
    }
)
```

When to use @Table: Always use it to explicitly set the table name. Without it, Hibernate uses a naming strategy to derive the table name from the class name, which can produce unexpected results (e.g., `OrderItem` might become `order_item` or `ORDERITEM` depending on the naming strategy).

JPA Standard: `@Table` is defined in `jakarta.persistence`.

@Column — Controlling Column Mapping

`@Column` is the most frequently used annotation after `@Entity` and `@Id`. It controls how a Java field maps to a database column.

All @Column Attributes

```java
@Column(
    name = "product_name",    // Column name in the database
    nullable = false,         // NOT NULL constraint (default: true)
    unique = false,           // UNIQUE constraint (default: false)
    length = 200,             // VARCHAR length (default: 255)
    precision = 10,           // Total digits for DECIMAL (default: 0)
    scale = 2,                // Decimal places for DECIMAL (default: 0)
    insertable = true,        // Include in INSERT statements (default: true)
    updatable = true,         // Include in UPDATE statements (default: true)
    columnDefinition = "",    // Raw SQL column definition (overrides everything)
    table = ""                // Secondary table name (advanced)
)
```

Practical Examples

String fields:

```java
// Short string with constraints
@Column(name = "username", nullable = false, unique = true, length = 50)
private String username;
// SQL: username VARCHAR(50) NOT NULL UNIQUE

// Default mapping (no @Column needed)
private String description;
// SQL: description VARCHAR(255)

// Long string (but not @Lob)
@Column(length = 2000)
private String bio;
// SQL: bio VARCHAR(2000)
```

Numeric fields:

```java
// Money/decimal fields — always specify precision and scale
@Column(precision = 10, scale = 2)
private BigDecimal price;
// SQL: price DECIMAL(10,2)
// Stores values like 99999999.99

@Column(precision = 5, scale = 4)
private BigDecimal taxRate;
// SQL: tax_rate DECIMAL(5,4)
// Stores values like 0.0825

// Integer fields usually need no @Column
private int quantity;
// SQL: quantity INTEGER NOT NULL
// (primitives are NOT NULL by default)
```

Read-only fields:

```java
// A field set only during INSERT, never updated
@Column(updatable = false)
private LocalDateTime createdAt;
// SQL generates: INSERT includes createdAt, UPDATE excludes it

// A field that comes from a database trigger or default
@Column(insertable = false, updatable = false)
private LocalDateTime lastModified;
// Never included in INSERT or UPDATE — managed by the database
```

Raw column definition:

```java
// Override everything with raw SQL
@Column(columnDefinition = "TEXT DEFAULT 'No description'")
private String notes;
// SQL: notes TEXT DEFAULT 'No description'

// Database-specific type
@Column(columnDefinition = "JSONB")
private String metadata;
// SQL: metadata JSONB (PostgreSQL specific)
```

When @Column Is Optional

You do not need `@Column` if you are happy with the defaults:

```
Field Type           Default Column Type    Default Nullable    Default Length
-------------------------------------------------------------------------------
String               VARCHAR(255)           true                255
int (primitive)      INTEGER                false (NOT NULL)    N/A
Integer (wrapper)    INTEGER                true                N/A
long (primitive)     BIGINT                 false (NOT NULL)    N/A
Long (wrapper)       BIGINT                 true                N/A
boolean              BOOLEAN                false               N/A
Boolean              BOOLEAN                true                N/A
double               DOUBLE                 false               N/A
BigDecimal           DECIMAL(38,2)*         true                N/A
LocalDate            DATE                   true                N/A
LocalDateTime        TIMESTAMP              true                N/A
```

*BigDecimal default precision varies by database dialect.

Key rule: Primitive types (`int`, `long`, `boolean`, `double`) are always NOT NULL. Wrapper types (`Integer`, `Long`, `Boolean`, `Double`) are nullable by default. Use wrapper types when null is a valid value.

```java
// primitive: NOT NULL in database (use when null is never valid)
private int age;

// wrapper: nullable in database (use when null is meaningful)
private Integer age;  // null could mean "age not provided"
```

JPA Standard: `@Column` is defined in `jakarta.persistence`.

@Enumerated — Mapping Java Enums

Java enums are a natural way to represent fixed sets of values. JPA provides `@Enumerated` to map them to database columns.

Two Strategies: ORDINAL vs STRING

```java
public enum Status {
    DRAFT,      // ordinal 0
    ACTIVE,     // ordinal 1
    ARCHIVED    // ordinal 2
}
```

```java
// ORDINAL (default) — stores the enum's position number
@Enumerated(EnumType.ORDINAL)
private Status status;
// Database value: 0, 1, or 2
// SQL: status INTEGER

// STRING — stores the enum's name as text
@Enumerated(EnumType.STRING)
private Status status;
// Database value: "DRAFT", "ACTIVE", or "ARCHIVED"
// SQL: status VARCHAR(255)
```

Why You Should Always Use STRING

```
+-----------------------------------------------------------------+
|   ORDINAL                          STRING                       |
|   (AVOID)                          (RECOMMENDED)                |
|                                                                 |
```

```
|   Stores: 0, 1, 2               Stores: "DRAFT", "ACTIVE",    |
|                                         "ARCHIVED"            |
|                                                               |
|   Problem: If you add a new     No problem: Adding new values |
|   enum value in the middle:     anywhere is safe:             |
|                                                               |
|   enum Status {                 enum Status {                 |
|     DRAFT,     // 0             DRAFT,                        |
|     REVIEW,    // 1 (NEW!)      REVIEW,    (NEW!)             |
|     ACTIVE,    // 2 (was 1!)    ACTIVE,                       |
|     ARCHIVED   // 3 (was 2!)    ARCHIVED                      |
|   }                             }                             |
|                                                               |
|   All existing "1" values now   "ACTIVE" still means ACTIVE   |
|   mean REVIEW instead of        regardless of position.       |
|   ACTIVE! DATA CORRUPTION!                                    |
+---------------------------------------------------------------+
```

Always use `EnumType.STRING`. It is slightly less storage-efficient (strings vs integers), but it is safe against reordering and self-documenting when you query the database directly.

Complete Enum Example

```java
public enum Priority {
    LOW, MEDIUM, HIGH, CRITICAL
}

public enum Category {
    ELECTRONICS, CLOTHING, BOOKS, FOOD, HOME
}

@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Category category;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 10)
    private Priority priority;
```

```
    // constructors, getters, setters...
}
```

Generated SQL:

```sql
CREATE TABLE products (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY,
    name VARCHAR(255) NOT NULL,
    category VARCHAR(20) NOT NULL,
    priority VARCHAR(10) NOT NULL,
    PRIMARY KEY (id)
);

-- Sample data:
INSERT INTO products (name, category, priority)
VALUES ('Laptop', 'ELECTRONICS', 'HIGH');
```

JPA Standard: `@Enumerated` is defined in `jakarta.persistence`.

Date and Time Mapping

Java 8+ Date/Time Types (Recommended)

With JPA 2.2+ and Hibernate 6, Java 8 date/time types are supported directly — no extra annotations needed:

```java
@Entity
public class Event {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;

    // DATE only (no time) — e.g., 2024-03-15
    private LocalDate eventDate;
    // SQL: event_date DATE

    // TIME only (no date) — e.g., 14:30:00
    private LocalTime startTime;
    // SQL: start_time TIME
```

```
    // DATE + TIME (no timezone) — e.g., 2024-03-15T14:30:00
    private LocalDateTime createdAt;
    // SQL: created_at TIMESTAMP

    // DATE + TIME + TIMEZONE OFFSET — e.g., 2024-03-15T14:30:00+05:30
    private OffsetDateTime publishedAt;
    // SQL: published_at TIMESTAMP WITH TIME ZONE

    // INSTANT — a point on the timeline (UTC)
    private Instant lastModified;
    // SQL: last_modified TIMESTAMP WITH TIME ZONE
}
```

Java Type to SQL Type Mapping

```
Java Type                  SQL Type                     Example Value
-----------------------------------------------------------------------
LocalDate                  DATE                         2024-03-15
LocalTime                  TIME                         14:30:00
LocalDateTime              TIMESTAMP                    2024-03-15 14:30:00
OffsetDateTime             TIMESTAMP WITH TIME ZONE     2024-03-15 14:30:00+05:30
Instant                    TIMESTAMP WITH TIME ZONE     2024-03-15 09:00:00Z
ZonedDateTime              TIMESTAMP WITH TIME ZONE     2024-03-15 14:30:00+05:30
Year                       INTEGER                      2024
YearMonth                  Not directly supported        (use converter)
```

Legacy Date Types and @Temporal

If you work with older code, you might encounter `java.util.Date` or `java.util.Calendar`. These require `@Temporal`:

```java
// LEGACY — avoid in new code
@Temporal(TemporalType.DATE)        // Store date only
private java.util.Date birthDate;

@Temporal(TemporalType.TIME)        // Store time only
private java.util.Date startTime;

@Temporal(TemporalType.TIMESTAMP)   // Store date + time
private java.util.Date createdAt;

// MODERN — prefer this in new code
private LocalDate birthDate;         // No @Temporal needed
private LocalTime startTime;         // No @Temporal needed
private LocalDateTime createdAt;     // No @Temporal needed
```

Best practice: Always use `java.time.*` types (LocalDate, LocalDateTime, Instant). They are clearer, immutable, and do not require `@Temporal`.

JPA Standard: `@Temporal` is defined in `jakarta.persistence`. Java 8+ date/time support is part of JPA 2.2+.

@Lob — Large Objects

`@Lob` marks a field as a Large Object, which maps to database types designed for large content.

```java
@Entity
public class Article {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;

    // Large text content
    @Lob
    private String content;
    // SQL: content CLOB (Character Large Object)
    // Can store megabytes of text

    // Binary data (images, files)
    @Lob
    private byte[] thumbnail;
    // SQL: thumbnail BLOB (Binary Large Object)
    // Can store binary data
}
```

CLOB vs BLOB

```
+-----------------------------------------------------------------+
|  @Lob on String          |  @Lob on byte[]                     |
|  (CLOB)                  |  (BLOB)                             |
|                          |                                      |
|  Character Large Object  |  Binary Large Object                 |
|  Stores: text content    |  Stores: images, PDFs, files        |
|  Size: up to 4 GB        |  Size: up to 4 GB                   |
|  SQL: CLOB / TEXT         |  SQL: BLOB / BYTEA                  |
|                          |                                      |
```

```
|  Use for:                |  Use for:                          |
|  - Article bodies        |  - Profile pictures                |
|  - HTML content          |  - File attachments                |
|  - JSON/XML documents    |  - Thumbnails                      |
+-------------------------------------------------------------+
```

Caution: Loading LOB data is expensive. If your entity has a large LOB field that you do not always need, consider lazy loading (with `@Basic(fetch = FetchType.LAZY)`) or moving the LOB to a separate entity.

JPA Standard: `@Lob` is defined in `jakarta.persistence`.

@Transient — Excluding Fields

`@Transient` tells JPA to ignore a field — it will not be mapped to any database column.

```java
@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Column(precision = 10, scale = 2)
    private BigDecimal price;

    @Column(precision = 5, scale = 4)
    private BigDecimal taxRate;

    // NOT stored in database — calculated at runtime
    @Transient
    private BigDecimal priceWithTax;

    // NOT stored in database — used only in application logic
    @Transient
    private boolean selected;

    // Calculate the transient field
    public BigDecimal getPriceWithTax() {
        if (price != null && taxRate != null) {
            return price.multiply(BigDecimal.ONE.add(taxRate));
        }
        return price;
```

```
    }
}
```

Three Ways to Exclude Fields

```
Method                    Annotation Required?    Works With
----------------------------------------------------------------
@Transient                Yes                     JPA (standard)
transient keyword         No                      Java serialization
                                                  AND JPA
static fields             No                      Never persisted
```

```java
@Transient                            // JPA ignores this field
private double discount;

private transient double discount;   // Both Java serialization AND JPA
                                     // ignore this field

private static int instanceCount;    // Static fields are never persisted
```

Best practice: Use the `@Transient` annotation (not the `transient` keyword) because it is explicit and only affects JPA, not Java serialization.

JPA Standard: `@Transient` is defined in `jakarta.persistence`.

@Basic — Fetch Type and Optionality

`@Basic` controls two things: whether a field is loaded lazily and whether it is optional (nullable).

```java
// Default behavior (you rarely need to write this)
@Basic(fetch = FetchType.EAGER, optional = true)
private String name;

// Lazy loading for large fields
@Basic(fetch = FetchType.LAZY)
@Lob
private String content;
// Content is NOT loaded when the entity is fetched.
// It is loaded only when getContent() is called.
```

```
// Field is required (NOT NULL)
@Basic(optional = false)
private String email;
// Hibernate validates non-null before sending to database
```

Lazy Loading Individual Fields

```
Default behavior (EAGER):
+-----------------------------------------------------------------+
| SELECT id, name, email, content FROM articles WHERE id = 1   |
|                                                                 |
| All columns loaded in one query, even if you only need       |
| the name. Large LOB columns waste bandwidth.                  |
+-----------------------------------------------------------------+

With @Basic(fetch = LAZY) on content:
+-----------------------------------------------------------------+
| Query 1: SELECT id, name, email FROM articles WHERE id = 1   |
|          (content NOT loaded)                                  |
|                                                                 |
| Query 2 (only if getContent() is called):                    |
|          SELECT content FROM articles WHERE id = 1            |
+-----------------------------------------------------------------+
```

Important: Lazy loading of basic fields requires Hibernate bytecode enhancement, which is not enabled by default in Spring Boot. Without it, `@Basic(fetch = LAZY)` is silently ignored and the field is loaded eagerly. To enable it, add to `pom.xml`:

```xml
<plugin>
    <groupId>org.hibernate.orm.tooling</groupId>
    <artifactId>hibernate-enhance-maven-plugin</artifactId>
    <version>${hibernate.version}</version>
    <executions>
        <execution>
            <configuration>
                <enableLazyInitialization>true</enableLazyInitialization>
            </configuration>
            <goals>
                <goal>enhance</goal>
            </goals>
        </execution>
    </executions>
</plugin>
```

For most applications, a simpler approach is to move large fields to a separate entity with a one-to-one relationship (covered in Chapter 12).

JPA Standard: `@Basic` is defined in `jakarta.persistence`.

Naming Strategies

When you do not explicitly specify a column or table name, Hibernate uses a naming strategy to derive the name from the Java field or class name.

How Names Are Derived

```
Java Name                  Implicit Strategy          Physical Strategy
                           (logical name)             (database name)
-----------------------------------------------------------------------
Class: OrderItem           OrderItem                  order_item
Field: firstName           firstName                  first_name
Field: createdAt           createdAt                  created_at
Field: isActive            isActive                   is_active
Class: HTMLParser          HTMLParser                  htmlparser
```

Spring Boot uses these default strategies:

```
+-----------------------------------------------------------------+
|  Naming Strategy Flow                                            |
|                                                                  |
|  Java name: "orderDate"                                          |
|       |                                                          |
|       v                                                          |
|  Implicit Strategy (SpringImplicitNamingStrategy):               |
|       Derives logical name from Java name                        |
|       "orderDate" --> "orderDate"                                |
|       |                                                          |
|       v                                                          |
|  Physical Strategy (CamelCaseToUnderscoresNamingStrategy):       |
|       Converts logical name to database name                     |
|       "orderDate" --> "order_date"                                |
|       |                                                          |
|       v                                                          |
|  Database column: order_date                                     |
+-----------------------------------------------------------------+
```

Overriding with @Column and @Table

When you specify a name explicitly, the naming strategy is bypassed:

```java
@Column(name = "order_date")     // Exact name — no naming strategy applied
private LocalDate orderDate;

@Table(name = "order_items")     // Exact name — no naming strategy applied
public class OrderItem { ... }
```

Changing the Naming Strategy

You can change the naming strategy in `application.properties`:

```properties
# Use exact Java names (no underscore conversion)
spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalN...

# With this strategy:
# orderDate --> orderDate (not order_date)
# OrderItem --> OrderItem (not order_item)
```

Best practice: Stick with the Spring Boot default (camelCase to snake_case) and explicitly name columns with `@Column(name = "...")` when the default does not match your database conventions.

Hibernate-Specific Annotations

The following annotations are not part of the JPA specification. They are provided by Hibernate and will not work with other JPA providers (EclipseLink, OpenJPA). Use them when JPA does not offer what you need, but be aware of the trade-off.

@NaturalId — Business Keys

A natural ID (or business key) is a unique identifier that has real-world meaning, as opposed to the surrogate primary key (the auto-generated `id`).

```java
import org.hibernate.annotations.NaturalId;

@Entity
@Table(name = "books")
public class Book {

    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;             // Surrogate key (database-generated)

    @NaturalId                   // Hibernate: Business key
    @Column(nullable = false, unique = true, length = 13)
    private String isbn;         // Natural key (real-world identifier)

    @Column(nullable = false)
    private String title;

    // ...
}
```

```
+-----------------------------------------------------------------+
|  Surrogate Key (id)           vs.     Natural Key (isbn)        |
|                                                                 |
|  - Auto-generated by DB               - Has real-world meaning |
|  - No business meaning                - ISBN, email, SSN, etc. |
|  - Always unique                      - Always unique          |
|  - Used for JOINs and FKs             - Used for lookups       |
|  - Never changes                      - Rarely changes         |
+-----------------------------------------------------------------+
```

With `@NaturalId`, Hibernate provides optimized lookup methods:

```java
// Standard JPA way (by surrogate key)
Book book = entityManager.find(Book.class, 1L);

// Hibernate way (by natural key) — uses Session, not EntityManager
Session session = entityManager.unwrap(Session.class);
Book book = session.byNaturalId(Book.class)
    .using("isbn", "978-0-13-468599-1")
    .load();
```

Hibernate also caches natural ID lookups in the first-level cache, so repeated lookups by the same natural ID do not hit the database.

Hibernate-specific: `@NaturalId` is from `org.hibernate.annotations`.

@Formula — Computed Columns

`@Formula` lets you define a field whose value is computed by a SQL expression rather than stored in a column:

```java
import org.hibernate.annotations.Formula;

@Entity
@Table(name = "order_items")
public class OrderItem {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private int quantity;

    @Column(precision = 10, scale = 2)
    private BigDecimal unitPrice;

    // Computed field - NOT a real column
    @Formula("quantity * unit_price")
    private BigDecimal totalPrice;

    // Another example: computed from a subquery
    @Formula("(SELECT COUNT(*) FROM reviews r WHERE r.product_id = id)")
    private int reviewCount;
}
```

```
Database Table: order_items
+----+----------+------------+
| id | quantity | unit_price |
|----|----------|------------|
| 1  | 3        | 29.99      |
| 2  | 1        | 149.99     |
+----+----------+------------+
Note: total_price is NOT a column - it does not exist in the table.

When Hibernate loads an OrderItem, it generates:
SELECT id, quantity, unit_price,
       quantity * unit_price AS formula_0   <-- @Formula
FROM order_items WHERE id = 1

Result: totalPrice = 89.97 (3 * 29.99)
```

Key points about @Formula:

- The expression uses SQL column names (not Java field names)
- The formula is read-only — you cannot set or update it
- The formula is evaluated by the database, so it supports SQL functions
- The value is calculated every time the entity is loaded (no caching)

Hibernate-specific: `@Formula` is from `org.hibernate.annotations`.

@DynamicUpdate — Efficient Updates

By default, Hibernate generates UPDATE statements that set all columns, even those that have not changed:

```sql
-- Default behavior: UPDATE ALL columns
UPDATE products
SET name = 'Widget', price = 29.99, description = 'A widget',
    category = 'TOOLS', status = 'ACTIVE', created_at = '2024-01-01'
WHERE id = 1
-- Even if only the price changed!
```

`@DynamicUpdate` tells Hibernate to generate UPDATE statements that only include changed columns:

```java
import org.hibernate.annotations.DynamicUpdate;

@Entity
@DynamicUpdate          // Only update changed columns
@Table(name = "products")
public class Product {
    // ...
}
```

```sql
-- With @DynamicUpdate: Only changed columns
UPDATE products
SET price = 29.99
WHERE id = 1
-- Only the price column!
```

When to Use @DynamicUpdate

```
+-----------------------------------------------------------------+
|  Use @DynamicUpdate when:          | Skip when:                  |
|                                    |                             |
|  - Table has many columns          | - Table has few columns     |
|    (15+) and updates are           |   (updating all is fine)    |
|    typically partial               |                             |
|  - Table has indexed columns       | - Every update changes      |
|    that should not be              |   most columns              |
|    unnecessarily touched           |                             |
|  - You need to avoid trigger       | - Simplicity is more        |
|    firing on unchanged columns     |   important than the minor  |
|                                    |   performance gain          |
+-----------------------------------------------------------------+
```

Hibernate-specific: `@DynamicUpdate` is from `org.hibernate.annotations`.

@DynamicInsert — Efficient Inserts

Similar to `@DynamicUpdate`, `@DynamicInsert` generates INSERT statements that skip null fields (letting the database use column defaults):

```java
import org.hibernate.annotations.DynamicInsert;

@Entity
@DynamicInsert
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    // Database has: status VARCHAR(20) DEFAULT 'DRAFT'
    @Column(length = 20)
    private String status;

    // ...
}
```

```sql
-- Without @DynamicInsert (default)
INSERT INTO products (name, status) VALUES ('Widget', NULL)
-- status becomes NULL, ignoring the database DEFAULT

-- With @DynamicInsert
INSERT INTO products (name) VALUES ('Widget')
-- status is omitted, so the database applies DEFAULT 'DRAFT'
```

Hibernate-specific: `@DynamicInsert` is from `org.hibernate.annotations`.

Complete Entity Example

Let us put everything together in a realistic entity:

```java
package com.example.hibernatedemo.entity;

import jakarta.persistence.*;
import org.hibernate.annotations.DynamicUpdate;
import org.hibernate.annotations.Formula;
import org.hibernate.annotations.NaturalId;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.LocalDateTime;

@Entity
@Table(
    name = "products",
    uniqueConstraints = {
        @UniqueConstraint(name = "uk_product_sku", columnNames = "sku")
    },
    indexes = {
        @Index(name = "idx_product_category", columnList = "category"),
        @Index(name = "idx_product_name", columnList = "name")
    }
)
@DynamicUpdate                                          // Hibernate: efficient updates
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NaturalId                                          // Hibernate: business key
    @Column(nullable = false, unique = true, length = 20)
    private String sku;                                 // e.g., "PROD-001"

    @Column(nullable = false, length = 200)
    private String name;

    @Lob                                                // JPA: large text
    private String description;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price;

    @Column(precision = 10, scale = 2)
    private BigDecimal costPrice;

    @Formula("price - cost_price")                      // Hibernate: computed field
    private BigDecimal profit;

    @Enumerated(EnumType.STRING)                        // JPA: enum as string
    @Column(nullable = false, length = 20)
    private Category category;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 15)
```

```
    private Status status;

    @Column(nullable = false)
    private int stockQuantity;

    @Column(nullable = false)
    private int reorderLevel;

    @Formula("CASE WHEN stock_quantity <= reorder_level " +  // Hibernate: computed
             "THEN true ELSE false END")
    private boolean needsReorder;

    private LocalDate launchDate;                       // JPA 2.2+: no @Temporal needed

    @Column(updatable = false)                          // Set once, never updated
    private LocalDateTime createdAt;

    private LocalDateTime updatedAt;

    @Transient                                          // JPA: not persisted
    private boolean featured;

    // Enums
    public enum Category {
        ELECTRONICS, CLOTHING, BOOKS, FOOD, HOME, SPORTS
    }

    public enum Status {
        DRAFT, ACTIVE, DISCONTINUED, OUT_OF_STOCK
    }

    // Default constructor (required by JPA)
    public Product() {
    }

    // Convenience constructor
    public Product(String sku, String name, BigDecimal price,
                   Category category) {
        this.sku = sku;
        this.name = name;
        this.price = price;
        this.category = category;
        this.status = Status.DRAFT;
        this.stockQuantity = 0;
        this.reorderLevel = 10;
        this.createdAt = LocalDateTime.now();
        this.updatedAt = LocalDateTime.now();
    }

    // Lifecycle callback — sets timestamps automatically
    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
        this.updatedAt = LocalDateTime.now();
    }
```

```
    @PreUpdate
    protected void onUpdate() {
        this.updatedAt = LocalDateTime.now();
    }

    // Getters and setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getSku() { return sku; }
    public void setSku(String sku) { this.sku = sku; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
    public BigDecimal getPrice() { return price; }
    public void setPrice(BigDecimal price) { this.price = price; }
    public BigDecimal getCostPrice() { return costPrice; }
    public void setCostPrice(BigDecimal costPrice) { this.costPrice = costPrice; }
    public BigDecimal getProfit() { return profit; }
    public Category getCategory() { return category; }
    public void setCategory(Category category) { this.category = category; }
    public Status getStatus() { return status; }
    public void setStatus(Status status) { this.status = status; }
    public int getStockQuantity() { return stockQuantity; }
    public void setStockQuantity(int stockQuantity) { this.stockQuantity = stockQuantit...
    public int getReorderLevel() { return reorderLevel; }
    public void setReorderLevel(int reorderLevel) { this.reorderLevel = reorderLevel; }
    public boolean isNeedsReorder() { return needsReorder; }
    public LocalDate getLaunchDate() { return launchDate; }
    public void setLaunchDate(LocalDate launchDate) { this.launchDate = launchDate; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public LocalDateTime getUpdatedAt() { return updatedAt; }
    public boolean isFeatured() { return featured; }
    public void setFeatured(boolean featured) { this.featured = featured; }

    @Override
    public String toString() {
        return "Product{sku='" + sku + "', name='" + name +
               "', price=" + price + ", category=" + category +
               ", status=" + status + ", stock=" + stockQuantity + "}";
    }
}
```

Generated SQL for This Entity

```sql
CREATE TABLE products (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY,
    sku VARCHAR(20) NOT NULL,
    name VARCHAR(200) NOT NULL,
```

```
    description CLOB,
    price DECIMAL(10,2) NOT NULL,
    cost_price DECIMAL(10,2),
    category VARCHAR(20) NOT NULL,
    status VARCHAR(15) NOT NULL,
    stock_quantity INTEGER NOT NULL,
    reorder_level INTEGER NOT NULL,
    launch_date DATE,
    created_at TIMESTAMP,
    updated_at TIMESTAMP,
    PRIMARY KEY (id),
    CONSTRAINT uk_product_sku UNIQUE (sku)
);

CREATE INDEX idx_product_category ON products (category);
CREATE INDEX idx_product_name ON products (name);
```

Notice:

- `profit` and `needsReorder` are NOT columns — they are `@Formula` computed fields
- `featured` is NOT a column — it is `@Transient`
- `description` is `CLOB` because of `@Lob`
- `category` and `status` are `VARCHAR` because of `@Enumerated(STRING)`
- `created_at` and `updated_at` are `TIMESTAMP` (no `@Temporal` needed for `LocalDateTime`)

Annotation Source Map

```
Annotation           Package                       Standard
-------------------------------------------------------------------
@Entity              jakarta.persistence           JPA
@Table               jakarta.persistence           JPA
@Id                  jakarta.persistence           JPA
@GeneratedValue      jakarta.persistence           JPA
@Column              jakarta.persistence           JPA
@Lob                 jakarta.persistence           JPA
@Enumerated          jakarta.persistence           JPA
@Transient           jakarta.persistence           JPA
@Basic               jakarta.persistence           JPA
@PrePersist          jakarta.persistence           JPA
@PreUpdate           jakarta.persistence           JPA
@Index               jakarta.persistence           JPA
@UniqueConstraint    jakarta.persistence           JPA
@NaturalId           org.hibernate.annotations     Hibernate-only
@Formula             org.hibernate.annotations     Hibernate-only
@DynamicUpdate       org.hibernate.annotations     Hibernate-only
@DynamicInsert       org.hibernate.annotations     Hibernate-only
```

Lifecycle Callbacks

JPA defines callback annotations that let you hook into entity lifecycle events:

```java
@Entity
public class AuditableEntity {

    @PrePersist     // Called before INSERT
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
    }

    @PostPersist    // Called after INSERT
    protected void afterCreate() {
        System.out.println("Entity created with ID: " + id);
    }

    @PreUpdate      // Called before UPDATE
    protected void onUpdate() {
        this.updatedAt = LocalDateTime.now();
    }

    @PostUpdate     // Called after UPDATE
    protected void afterUpdate() {
        System.out.println("Entity updated: " + id);
    }

    @PreRemove      // Called before DELETE
    protected void onDelete() {
        System.out.println("About to delete: " + id);
    }

    @PostRemove     // Called after DELETE
    protected void afterDelete() {
        System.out.println("Deleted: " + id);
    }

    @PostLoad       // Called after entity is loaded from DB
    protected void onLoad() {
        // Initialize transient fields
        this.displayName = firstName + " " + lastName;
    }
}
```

Lifecycle Callback Flow

```
+-----------------------------------------------------------------+
|  Operation: repository.save(new Entity())                       |
|                                                                 |
```

```
|  1. @PrePersist fires (set createdAt, validate, etc.)           |
|  2. INSERT INTO ... (SQL executes)                              |
|  3. @PostPersist fires (log creation, send notification, etc.)  |
+-----------------------------------------------------------------+

+-----------------------------------------------------------------+
|  Operation: entity.setName("new"); repository.save(entity)      |
|                                                                 |
|  1. @PreUpdate fires (set updatedAt, validate, etc.)            |
|  2. UPDATE ... SET ... (SQL executes)                           |
|  3. @PostUpdate fires (log update, send notification, etc.)     |
+-----------------------------------------------------------------+

+-----------------------------------------------------------------+
|  Operation: repository.deleteById(1L)                           |
|                                                                 |
|  1. SELECT ... (load the entity)                                |
|  2. @PostLoad fires (entity is loaded)                          |
|  3. @PreRemove fires (cleanup, validation, etc.)                |
|  4. DELETE FROM ... (SQL executes)                              |
|  5. @PostRemove fires (log deletion, etc.)                      |
+-----------------------------------------------------------------+
```

Common use case: Automatically setting `createdAt` and `updatedAt` timestamps without requiring the caller to remember:

```java
@PrePersist
protected void onCreate() {
    this.createdAt = LocalDateTime.now();
    this.updatedAt = LocalDateTime.now();
}

@PreUpdate
protected void onUpdate() {
    this.updatedAt = LocalDateTime.now();
}
```

We will explore a more sophisticated approach using Spring Data JPA's `@CreatedDate` and `@LastModifiedDate` in Chapter 21 (Auditing).

JPA Standard: All lifecycle callbacks (`@PrePersist`, `@PostPersist`, `@PreUpdate`, `@PostUpdate`, `@PreRemove`, `@PostRemove`, `@PostLoad`) are defined in `jakarta.persistence`.

Common Mistakes

1. Using `EnumType.ORDINAL` for enums: Adding or reordering enum constants silently corrupts existing data. Always use `EnumType.STRING`.
2. Using `java.util.Date` instead of `java.time.*`: In new code, always use `LocalDate`, `LocalDateTime`, or `Instant`. They are immutable, type-safe, and do not require `@Temporal`.
3. Forgetting that `@Formula` uses SQL column names: The formula expression runs in SQL, so you must use database column names (e.g., `unit_price`), not Java field names (e.g., `unitPrice`).
4. Expecting `@Lob` fields to lazy-load automatically: Without Hibernate bytecode enhancement, `@Lob` fields are loaded eagerly. This can cause performance issues when loading entities with large text/binary fields.
5. Not specifying `length` for String columns: Without `@Column(length = ...)`, all Strings default to VARCHAR(255). For names, use 100. For emails, use 150. For descriptions, use 2000 or `@Lob`. Being intentional about sizes catches bugs early.
6. Using primitive types when null is valid: `private int age` is NOT NULL in the database. If age can be unknown/optional, use `private Integer age` instead.
7. Confusing `@Entity(name)` with `@Table(name)`: `@Entity(name)` sets the entity name for JPQL queries. `@Table(name)` sets the database table name. They serve different purposes.

Best Practices

1. Always use `@Enumerated(EnumType.STRING)`: The minor storage cost is worth the safety against enum reordering.
2. Be explicit about column constraints: Even though JPA infers some things, always specify `nullable`, `length`, and `unique` when they matter. It makes your code self-documenting and catches mistakes at DDL time rather than runtime.
3. Use `@Column(updatable = false)` for creation timestamps: This prevents accidental modification of fields that should be set once.
4. Prefer JPA standard annotations: Use `jakarta.persistence.*` annotations whenever possible. Only use `org.hibernate.annotations.*` when JPA does not provide what you need.
5. Use lifecycle callbacks for automatic timestamps: `@PrePersist` and `@PreUpdate` ensure timestamps are always set, even if the calling code forgets.
6. Document Hibernate-specific annotations: When you use `@NaturalId`, `@Formula`, or `@DynamicUpdate`, add a comment noting it is Hibernate-specific. This helps if you ever need to switch providers.

Summary

In this chapter, you learned every major annotation for mapping Java classes to database tables:

- @Column controls column name, length, precision, scale, nullability, uniqueness, and whether the field is included in INSERT/UPDATE statements.
- @Enumerated(EnumType.STRING) maps Java enums as readable strings. Always use STRING, never ORDINAL.
- Java 8+ date/time types (LocalDate, LocalDateTime, Instant) map directly to SQL DATE/TIMESTAMP without needing `@Temporal`.
- @Lob maps large text (CLOB) and binary (BLOB) data. Consider lazy loading for large fields.
- @Transient excludes a field from database persistence.
- @Basic controls fetch type (EAGER/LAZY) and optionality for individual fields.
- Naming strategies determine how Java names translate to database names. Spring Boot defaults to camelCase-to-snake_case.
- @NaturalId (Hibernate) marks a business key for optimized lookups.
- @Formula (Hibernate) computes a field value from a SQL expression.
- @DynamicUpdate (Hibernate) generates UPDATE statements with only changed columns.
- Lifecycle callbacks (@PrePersist, @PreUpdate, etc.) let you hook into entity operations for automatic timestamps and validation.

Interview Questions

Q1: What is the difference between `@Column(unique = true)` and `@Table(uniqueConstraints = ...)`?

`@Column(unique = true)` creates a unique constraint on a single column. `@Table(uniqueConstraints = @UniqueConstraint(columnNames = {"col1", "col2"}))` can create composite unique constraints spanning multiple columns. Use `@Column` for single-column uniqueness and `@Table` for multi-column uniqueness or when you need to name the constraint.

Q2: Why should you always use `EnumType.STRING` instead of `EnumType.ORDINAL`?

`ORDINAL` stores the enum's position (0, 1, 2). If you add a new enum value or reorder them, existing data becomes corrupted because the numbers now map to different values. `STRING` stores the name ("ACTIVE", "DRAFT"), which is independent of position and remains correct even after reordering.

Q3: What is the difference between `@Transient` and `@Formula`?

`@Transient` marks a field that is completely ignored by JPA — it has no column and no SQL. `@Formula` marks a field that has no column but IS populated by a SQL expression when the entity is loaded. `@Formula` fields are read-only and computed by the database.

Q4: What is `@NaturalId` and when would you use it?

`@NaturalId` is a Hibernate-specific annotation that marks a field as a business key (like ISBN, email, or SKU). Unlike the primary key (`@Id`), a natural ID has real-world meaning. Hibernate provides optimized lookup methods for natural IDs and caches them in the first-level cache. Use it when you frequently look up entities by a business identifier.

Q5: How do primitive vs wrapper types affect nullability in JPA?

Primitive types (`int`, `long`, `boolean`) cannot be null in Java, so JPA maps them to NOT NULL columns. Wrapper types (`Integer`, `Long`, `Boolean`) can be null, so JPA maps them to nullable columns by default. Use primitives when null is never a valid value, and wrappers when null represents "unknown" or "not set."

Q6: What is the purpose of `@Column(insertable = false, updatable = false)`?

A field with both `insertable = false` and `updatable = false` is never included in INSERT or UPDATE SQL statements. The value comes from the database itself — perhaps a default value, a trigger, or a computed column. JPA reads the value but never writes it.

Q7: What does `@DynamicUpdate` do and when should you use it?

By default, Hibernate generates UPDATE statements that set all columns, even unchanged ones. `@DynamicUpdate` tells Hibernate to generate UPDATE statements that only include columns whose values have actually changed. Use it on entities with many columns (15+) where updates typically change only a few fields. This can improve performance and avoid unnecessary trigger firing on unchanged columns.

Q8: What are JPA lifecycle callbacks? Name four of them.

Lifecycle callbacks are methods annotated to run at specific points in an entity's lifecycle. `@PrePersist` runs before INSERT, `@PostPersist` runs after INSERT, `@PreUpdate` runs before UPDATE, `@PostUpdate` runs after UPDATE, `@PreRemove` runs before DELETE, `@PostRemove` runs after DELETE, and `@PostLoad` runs after an entity is loaded from the database. They are commonly used for setting timestamps and logging.

Practice Exercises

Exercise 1: Employee Entity Create an `Employee` entity with: `id` (auto-generated), `employeeCode` (natural ID, unique, 10 chars), `firstName` and `lastName` (not null, 50 chars each), `email` (unique), `salary` (BigDecimal with precision 10, scale 2), `department` (enum: HR, ENGINEERING, SALES, MARKETING), `hireDate` (LocalDate), `isActive` (boolean), `bio` (Lob), `createdAt` and `updatedAt` (auto-set via lifecycle callbacks). Write the entity class with all appropriate annotations.

Exercise 2: Annotation Identification For each annotation below, state whether it is JPA standard or Hibernate-specific: `@Entity`, `@NaturalId`, `@Column`, `@Formula`, `@Lob`, `@DynamicUpdate`, `@Transient`, `@Enumerated`, `@PrePersist`, `@DynamicInsert`, `@Basic`, `@Table`.

Exercise 3: SQL Prediction Given an entity with `@DynamicUpdate` and fields: `name`, `email`, `age`, `status`, predict the SQL generated when:

1. You change only `name` and call `save()`
2. You change `name`, `email`, and `age` and call `save()`
3. You change nothing and call `save()`

Exercise 4: Enum Disaster Create a `TaskStatus` enum with: TODO, IN_PROGRESS, DONE. Create a Task entity using `@Enumerated(ORDINAL)`. Save some tasks. Then add a new status REVIEW between IN_PROGRESS and DONE. Query the data. What happened to the DONE tasks?

Exercise 5: Formula Practice Create an `Invoice` entity with fields: `subtotal`, `taxRate`, `discountPercent`. Add `@Formula` fields for: `taxAmount` (subtotal taxRate), `discountAmount` (subtotal discountPercent / 100), and `total` (subtotal + taxAmount - discountAmount). Verify the computed values in the H2 console.

What Is Next?

In the next chapter, we will explore Primary Keys and Generation Strategies. You will learn every strategy for generating primary keys (IDENTITY, SEQUENCE, TABLE, UUID), when to use each one, how to work with composite keys using `@EmbeddedId` and `@IdClass`, and the difference between natural and surrogate keys. Choosing the right key strategy affects performance, portability, and how you design your entity relationships.

Chapter 5: Primary Keys and Generation Strategies

Learning Goals

By the end of this chapter, you will be able to:

- Explain why every entity needs a primary key
- Use all four JPA generation strategies: IDENTITY, SEQUENCE, TABLE, and AUTO
- Configure @SequenceGenerator and @TableGenerator
- Generate UUID primary keys
- Create composite primary keys with @EmbeddedId and @IdClass
- Understand the difference between natural and surrogate keys
- Choose the right generation strategy for your application
- Understand how each strategy affects batch insert performance

Why Primary Keys Matter

Every JPA entity must have a primary key. The primary key serves two critical purposes:

```
+-----------------------------------------------------------------+
|                   Primary Key Purposes                          |
|                                                                 |
|  1. DATABASE LEVEL                                              |
|     - Uniquely identifies every row in the table                |
|     - Enables efficient lookups (clustered index)               |
|     - Referenced by foreign keys in related tables              |
|                                                                 |
|  2. JPA LEVEL                                                   |
```

```
|     - Identifies entities in the persistence context (cache)   |
|     - Used by EntityManager.find(Class, id) for lookups        |
|     - Determines equality in the persistence context           |
|     - Used for merge() to decide insert vs update              |
+----------------------------------------------------------------+
```

Without a primary key, JPA cannot manage the entity. You will get an error at startup if any `@Entity` class is missing the `@Id` annotation.

The @Id and @GeneratedValue Annotations

The simplest primary key configuration requires two annotations:

```java
@Id                                          // This field is the primary key
@GeneratedValue(strategy = GenerationType.IDENTITY)  // Auto-generate values
private Long id;
```

`@Id` marks the field as the primary key. `@GeneratedValue` tells JPA to automatically generate values instead of requiring you to set them manually.

Without @GeneratedValue

If you omit `@GeneratedValue`, you must set the ID yourself before saving:

```java
@Entity
public class Country {
    @Id
    private String code;    // No @GeneratedValue — must be set manually

    private String name;
}

// You must set the ID:
Country usa = new Country();
usa.setCode("US");          // Required!
usa.setName("United States");
repository.save(usa);

// Without setting the code, you get an error
```

This is appropriate for natural keys (like country codes, currency codes, or ISBNs) where the value comes from the business domain, not from the database.

Generation Strategies

JPA defines four strategies in the `GenerationType` enum:

```
+-----------------------------------------------------------------+
|  Strategy        How It Works             Best For               |
|-----------------------------------------------------------------|
|  IDENTITY        Database auto-increment  Simple apps, H2, MySQL  |
|  SEQUENCE        Database sequence object  PostgreSQL, Oracle      |
|  TABLE           Simulated sequence table  Portability (rare)     |
|  AUTO            Provider chooses          Quick prototypes       |
+-----------------------------------------------------------------+
```

Let us explore each one in depth.

GenerationType.IDENTITY

The most straightforward strategy. The database generates the ID using its auto-increment feature.

```java
@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    // ...
}
```

How IDENTITY Works

```
Your Code                      Database
---------                      --------

Student s = new Student();
s.setName("Alice");
                               Table: students
repository.save(s);            id (AUTO_INCREMENT) | name
    |                          --------------------|-------
    v                          1                   | Alice
INSERT INTO students
  (name)                       The database assigns
  VALUES ('Alice')             id = 1 automatically
    |
    v
s.getId() == 1                 Hibernate reads the
                               generated ID back
```

SQL Generated

```sql
-- Table creation
CREATE TABLE students (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY,   -- H2
    -- id BIGINT AUTO_INCREMENT,                   -- MySQL
    -- id BIGSERIAL,                               -- PostgreSQL
    name VARCHAR(255) NOT NULL,
    PRIMARY KEY (id)
);

-- Insert (no id in the INSERT)
INSERT INTO students (name) VALUES ('Alice');
```

The Batch Insert Problem

IDENTITY has one significant limitation: it prevents JDBC batch inserts.

```
Why IDENTITY Breaks Batching:
+-------------------------------------------------------------------+
|                                                                   |
|  With IDENTITY, Hibernate MUST execute each INSERT immediately    |
|  to get the generated ID back from the database.                  |
|                                                                   |
|  Normal flow (what we want):                                      |
|  persist(s1) --> queue INSERT                                     |
|  persist(s2) --> queue INSERT                                     |
|  persist(s3) --> queue INSERT                                     |
|  flush()     --> execute all 3 INSERTs in one batch               |
|                                                                   |
```

```
|  IDENTITY flow (what actually happens):                          |
|  persist(s1) --> execute INSERT immediately --> get id=1          |
|  persist(s2) --> execute INSERT immediately --> get id=2          |
|  persist(s3) --> execute INSERT immediately --> get id=3          |
|                                                                  |
|  Each INSERT is a separate round-trip to the database!           |
|  With 1000 entities, that is 1000 round-trips instead of 1.      |
+------------------------------------------------------------------+
```

For most applications this is fine. But if you need to insert thousands of rows at once, consider SEQUENCE instead.

When to Use IDENTITY

- Simple applications where batch performance is not critical
- MySQL databases (which support AUTO_INCREMENT natively)
- H2 databases (which support both IDENTITY and SEQUENCE)
- When you want the simplest possible configuration

GenerationType.SEQUENCE

The SEQUENCE strategy uses a database sequence object to generate IDs. A sequence is a database feature that produces a series of unique numbers.

```java
@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE,
                    generator = "order_seq")
    @SequenceGenerator(
        name = "order_seq",              // Generator name (matches generator above)
        sequenceName = "orders_id_seq",  // Actual sequence name in database
        initialValue = 1,               // Starting value
        allocationSize = 50             // How many IDs to pre-allocate
    )
    private Long id;

    @Column(nullable = false)
    private String description;

    // ...
}
```

How SEQUENCE Works

```
Database Sequence: orders_id_seq
+-------+
| 1     | --> current value
+-------+

Your Code                       Database
---------                       --------

// First persist call:
repository.save(order1);
    |
    v
SELECT NEXT VALUE FOR            Sequence returns 1
  orders_id_seq                  (with allocationSize=50,
    |                            Hibernate pre-fetches 1-50)
    v
order1.id = 1                   Hibernate assigns from
                                its cached range

// Next 49 persist calls:
repository.save(order2);        order2.id = 2 (from cache!)
repository.save(order3);        order3.id = 3 (from cache!)
...                             ...
repository.save(order50);       order50.id = 50 (from cache!)

// 51st persist call:
repository.save(order51);
    |
    v
SELECT NEXT VALUE FOR            Sequence returns 51
  orders_id_seq                  (Hibernate pre-fetches 51-100)
    |
    v
order51.id = 51
```

The allocationSize Optimization

The `allocationSize` parameter is key to understanding SEQUENCE performance:

```
allocationSize = 1 (inefficient):
+-------------------------------------------------------------------+
|  persist(e1) --> SELECT NEXT VALUE FOR seq  --> id = 1            |
|  persist(e2) --> SELECT NEXT VALUE FOR seq  --> id = 2            |
|  persist(e3) --> SELECT NEXT VALUE FOR seq  --> id = 3            |
|  ...                                                              |
|  Each persist = 1 database call for the ID                        |
```

```
+-------------------------------------------------------------------+

allocationSize = 50 (default, efficient):
+-------------------------------------------------------------------+
|  persist(e1) --> SELECT NEXT VALUE FOR seq  --> ids 1-50 cached   |
|  persist(e2) --> (from cache)               --> id = 2            |
|  persist(e3) --> (from cache)               --> id = 3            |
|  ...                                                              |
|  persist(e50) --> (from cache)              --> id = 50           |
|  persist(e51) --> SELECT NEXT VALUE FOR seq  --> ids 51-100 cached|
|  ...                                                              |
|  50 persists = 1 database call for IDs!                           |
+-------------------------------------------------------------------+
```

With `allocationSize = 50`, Hibernate calls the sequence once and reserves 50 IDs. The next 49 `persist()` calls use cached IDs without touching the database. This is much more efficient than IDENTITY for batch operations.

Important: The sequence's INCREMENT BY in the database must match `allocationSize`. If they mismatch, IDs will have gaps or duplicates:

```sql
-- Database sequence must match allocationSize
CREATE SEQUENCE orders_id_seq
    START WITH 1
    INCREMENT BY 50;    -- Must match allocationSize = 50
```

SEQUENCE Enables Batch Inserts

Unlike IDENTITY, SEQUENCE allows Hibernate to batch INSERT statements:

```
SEQUENCE with batching enabled:
+-------------------------------------------------------------------+
|  persist(e1) --> assigns id=1 (from cache)                        |
|  persist(e2) --> assigns id=2 (from cache)                        |
|  persist(e3) --> assigns id=3 (from cache)                        |
|  flush()     --> sends ONE batch INSERT for all 3 entities:       |
|                                                                   |
|  INSERT INTO orders (id, description)                             |
|  VALUES (1, 'Order A'), (2, 'Order B'), (3, 'Order C')           |
|                                                                   |
|  One round-trip instead of three!                                 |
+-------------------------------------------------------------------+
```

To enable batch inserts in application.properties:

```properties
spring.jpa.properties.hibernate.jdbc.batch_size=25
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
```

When to Use SEQUENCE

- PostgreSQL and Oracle databases (native sequence support)
- When you need batch insert performance
- When you want to pre-allocate IDs (reduces database round-trips)
- Production applications with high-volume inserts

GenerationType.TABLE

The TABLE strategy uses a special database table to simulate sequences. It works on any database but has the worst performance.

```java
@Entity
@Table(name = "invoices")
public class Invoice {

    @Id
    @GeneratedValue(strategy = GenerationType.TABLE,
                    generator = "invoice_gen")
    @TableGenerator(
        name = "invoice_gen",
        table = "id_generator",          // Name of the generator table
        pkColumnName = "gen_name",       // Column that holds the sequence name
        valueColumnName = "gen_value",   // Column that holds the current value
        pkColumnValue = "invoice_id",    // The sequence name for this entity
        allocationSize = 50
    )
    private Long id;

    // ...
}
```

How TABLE Strategy Works

```
Generator Table: id_generator
+---------------+-----------+
```

```
| gen_name      | gen_value |
|---------------|-----------|
| invoice_id    | 50        |
| payment_id    | 100       |
| customer_id   | 25        |
+---------------+-----------+

When Hibernate needs a new batch of IDs:

1. SELECT gen_value FROM id_generator WHERE gen_name = 'invoice_id'
   (returns 50)

2. UPDATE id_generator SET gen_value = 100
   WHERE gen_name = 'invoice_id' AND gen_value = 50
   (increment by allocationSize)

3. Use IDs 51-100 from cache

This requires a SELECT + UPDATE + transaction lock
for every batch of IDs. Slow!
```

When to Use TABLE

Almost never. TABLE strategy exists for databases that do not support sequences (old MySQL versions), but modern databases all support either IDENTITY or SEQUENCE. TABLE strategy:

- Requires row-level locking on the generator table
- Creates a performance bottleneck under high concurrency
- Is more complex than IDENTITY or SEQUENCE

Only use TABLE if: You absolutely must support a database that has neither auto-increment nor sequences. This is extremely rare in modern applications.

GenerationType.AUTO

AUTO lets the JPA provider choose the strategy based on the database dialect:

```java
@Id
@GeneratedValue(strategy = GenerationType.AUTO)
private Long id;
```

```
// Or simply (AUTO is the default):
@Id
@GeneratedValue
private Long id;
```

What AUTO Chooses

```
Database          Hibernate 6 Chooses
----------------------------------------
H2                SEQUENCE
PostgreSQL        SEQUENCE
Oracle            SEQUENCE
MySQL 5.x         TABLE (not ideal!)
MySQL 8.x         SEQUENCE (if supported)
SQL Server        SEQUENCE
```

Warning: In Hibernate 6, AUTO defaults to SEQUENCE for most databases and falls back to TABLE for databases without sequence support. This can surprise you — if you expect IDENTITY behavior (simple auto-increment), explicitly use `GenerationType.IDENTITY`.

When to Use AUTO

- Quick prototypes where you do not care about the strategy
- When you might switch databases and want the provider to decide

Best practice: In production code, explicitly choose IDENTITY or SEQUENCE. Do not rely on AUTO — different Hibernate versions may choose differently.

UUID Primary Keys

UUIDs (Universally Unique Identifiers) are 128-bit identifiers that are globally unique without requiring a central authority. They look like: `550e8400-e29b-41d4-a716-446655440000`.

JPA 3.1+ UUID Generation

JPA 3.1 added built-in UUID support:

```java
@Entity
@Table(name = "documents")
public class Document {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;      // java.util.UUID

    @Column(nullable = false)
    private String title;

    // ...
}
```

```sql
-- Generated SQL (H2)
CREATE TABLE documents (
    id UUID NOT NULL,
    title VARCHAR(255) NOT NULL,
    PRIMARY KEY (id)
);

INSERT INTO documents (id, title) VALUES ('550e8400-e29b-...', 'My Document');
```

UUID as String

If your database does not have a native UUID type, you can store it as a string:

```java
@Entity
public class ApiToken {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    @Column(length = 36)
    private String id;    // Stored as VARCHAR(36)

    // ...
}
```

When to Use UUID

```
+-----------------------------------------------------------------+
|  UUID Advantages                | UUID Disadvantages              |
|---------------------------------|---------------------------------|
|  Globally unique across         | Larger than Long (16 bytes      |
|  systems and databases          | vs 8 bytes)                     |
|                                 |                                 |
```

```
|  Can be generated client-side  | Slower index lookups           |
|  (no DB round-trip)            | (random values fragment        |
|                                | B-tree indexes)                |
|  No sequence bottleneck        |                                |
|  under high concurrency        | Less human-readable            |
|                                | (hard to type in URLs)         |
|  Safe for distributed          |                                |
|  systems and microservices     | Larger foreign keys            |
|                                | in related tables              |
|  Hard to guess (security)      |                                |
+----------------------------------------------------------------+
```

Use UUID when:

- Building distributed systems or microservices that independently generate IDs
- IDs appear in URLs and should not be guessable (security)
- You merge data from multiple databases
- You need to generate IDs before saving to the database

Use Long when:

- Single database application
- Performance matters (Long indexes are faster)
- IDs are internal and not exposed to users
- Human readability is important (IDs like 1, 2, 3 are easier to work with)

Composite Primary Keys

Sometimes a table's primary key consists of multiple columns. For example, an enrollment might be uniquely identified by both `studentId` and `courseId`. JPA supports this with two approaches: `@EmbeddedId` and `@IdClass`.

Approach 1: @EmbeddedId (Recommended)

With `@EmbeddedId`, you create a separate class for the composite key:

```java
// Step 1: Create the key class
import jakarta.persistence.Embeddable;
import java.io.Serializable;
import java.util.Objects;
```

```
@Embeddable
public class EnrollmentId implements Serializable {

    @Column(name = "student_id")
    private Long studentId;

    @Column(name = "course_id")
    private Long courseId;

    // Default constructor (required)
    public EnrollmentId() {}

    public EnrollmentId(Long studentId, Long courseId) {
        this.studentId = studentId;
        this.courseId = courseId;
    }

    // Getters and setters
    public Long getStudentId() { return studentId; }
    public void setStudentId(Long studentId) { this.studentId = studentId; }
    public Long getCourseId() { return courseId; }
    public void setCourseId(Long courseId) { this.courseId = courseId; }

    // MUST override equals() and hashCode()
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        EnrollmentId that = (EnrollmentId) o;
        return Objects.equals(studentId, that.studentId) &&
               Objects.equals(courseId, that.courseId);
    }

    @Override
    public int hashCode() {
        return Objects.hash(studentId, courseId);
    }
}
```

```java
// Step 2: Use it in the entity
@Entity
@Table(name = "enrollments")
public class Enrollment {

    @EmbeddedId
    private EnrollmentId id;

    @Column(nullable = false)
    private LocalDate enrollmentDate;

    private String grade;
```

```
    // Constructors
    public Enrollment() {}

    public Enrollment(Long studentId, Long courseId) {
        this.id = new EnrollmentId(studentId, courseId);
        this.enrollmentDate = LocalDate.now();
    }

    // Getters, setters...
    public EnrollmentId getId() { return id; }
    public void setId(EnrollmentId id) { this.id = id; }
    public LocalDate getEnrollmentDate() { return enrollmentDate; }
    public void setEnrollmentDate(LocalDate enrollmentDate) {
        this.enrollmentDate = enrollmentDate;
    }
    public String getGrade() { return grade; }
    public void setGrade(String grade) { this.grade = grade; }
}
```

```java
// Step 3: Create the repository
public interface EnrollmentRepository
        extends JpaRepository<Enrollment, EnrollmentId> {
    //                                     ^^^^^^^^^^^^^
    //                           The ID type is the composite key class
}
```

```java
// Usage
EnrollmentId key = new EnrollmentId(1L, 101L);
Enrollment enrollment = new Enrollment(1L, 101L);
enrollment.setGrade("A");
repository.save(enrollment);

// Find by composite key
repository.findById(new EnrollmentId(1L, 101L));
```

How @EmbeddedId Maps to the Database

```
Java                                    Database
----                                    --------

Enrollment entity                       Table: enrollments
+---------------------------+           +------------+-----------+--------+-------+
| @EmbeddedId               |           | student_id | course_id | enroll | grade |
| EnrollmentId id:          |   ===>    |    (PK)    |   (PK)    | _date  |       |
|   studentId: 1            |           |------------|-----------|--------|-------|
|   courseId: 101            |           | 1          | 101       | 2024.. | A     |
| enrollmentDate: 2024-03   |           | 1          | 102       | 2024.. | B+    |
| grade: "A"                |           | 2          | 101       | 2024.. | null  |
```

```
+---------------------------+           +------------+-----------+--------+-------+

SQL:
CREATE TABLE enrollments (
    student_id BIGINT NOT NULL,
    course_id BIGINT NOT NULL,
    enrollment_date DATE NOT NULL,
    grade VARCHAR(255),
    PRIMARY KEY (student_id, course_id)
);
```

Approach 2: @IdClass

With `@IdClass`, the key fields are defined directly in the entity, and a separate class mirrors them:

```java
// Step 1: Create the ID class (mirrors the @Id fields)
public class EnrollmentId implements Serializable {

    private Long studentId;
    private Long courseId;

    public EnrollmentId() {}

    public EnrollmentId(Long studentId, Long courseId) {
        this.studentId = studentId;
        this.courseId = courseId;
    }

    // MUST have equals() and hashCode()
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        EnrollmentId that = (EnrollmentId) o;
        return Objects.equals(studentId, that.studentId) &&
               Objects.equals(courseId, that.courseId);
    }

    @Override
    public int hashCode() {
        return Objects.hash(studentId, courseId);
    }

    // Getters and setters...
}
```

```java
// Step 2: Use @IdClass on the entity
@Entity
@Table(name = "enrollments")
@IdClass(EnrollmentId.class)
public class Enrollment {

    @Id                                     // Part of composite key
    @Column(name = "student_id")
    private Long studentId;

    @Id                                     // Part of composite key
    @Column(name = "course_id")
    private Long courseId;

    @Column(nullable = false)
    private LocalDate enrollmentDate;

    private String grade;

    // ...
}
```

@EmbeddedId vs @IdClass

```
+-----------------------------------------------------------------+
|  @EmbeddedId                    | @IdClass                      |
|  (Recommended)                  |                               |
|---------------------------------|-------------------------------|
|  Key fields in separate class   | Key fields in entity AND      |
|  Entity has ONE @EmbeddedId     | separate class (duplicated)   |
|  field                          |                               |
|                                 |                               |
|  Access: entity.getId()         | Access: entity.getStudentId() |
|          .getStudentId()        |         entity.getCourseId()  |
|                                 |                               |
|  JPQL: e.id.studentId           | JPQL: e.studentId             |
|                                 |                               |
|  Cleaner: key is a proper       | Noisier: key fields are       |
|  value object                   | scattered in the entity       |
+-----------------------------------------------------------------+
```

Recommendation: Use `@EmbeddedId`. It groups key fields into a clean value object, avoids field duplication, and makes the composite key reusable.

Requirements for Composite Key Classes

Both `@EmbeddedId` and `@IdClass` key classes must:

```
+-----------------------------------------------------------------+
|  Composite Key Class Requirements                               |
|                                                                 |
|  1. Implement Serializable                                      |
|  2. Have a public no-argument constructor                       |
|  3. Override equals() and hashCode()                            |
|  4. Fields must match the key columns (for @IdClass)            |
|  5. Be public (or package-private)                              |
+-----------------------------------------------------------------+
```

The `equals()` and `hashCode()` requirement is critical. JPA uses these methods to identify entities in the persistence context. If they are wrong, you will get duplicate entities, cache misses, or wrong results.

Natural Keys vs Surrogate Keys

Surrogate Keys

A surrogate key is an artificial identifier with no business meaning. It exists solely to uniquely identify a row:

```java
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;    // 1, 2, 3, 4, 5 ... meaningless outside the database
```

Natural Keys

A natural key has real-world business meaning:

```java
@Id
private String isbn;           // "978-0-13-468599-1" — identifies a book
// or
@Id
private String currencyCode;   // "USD", "EUR" — identifies a currency
// or
@Id
private String email;          // "alice@example.com" — identifies a user
```

Comparison

```
+-----------------------------------------------------------------+
|  Surrogate Key                  | Natural Key                    |
|  (Recommended for most cases)   |                                |
|---------------------------------|--------------------------------|
|  Auto-generated (1, 2, 3...)    | Business-meaningful            |
|  Never changes                  | Might change (email, name)     |
|  Small and fast (Long = 8B)     | Can be large (VARCHAR)         |
|  Simple foreign keys            | Verbose foreign keys           |
|  Requires DB lookup for         | Can query by business value    |
|  business queries               | directly                       |
|                                 |                                |
|  Example:                       | Example:                       |
|  user.id = 42                   | user.email = "alice@ex.com"    |
|  order.user_id = 42             | order.user_email = "alice@..." |
+-----------------------------------------------------------------+
```

Best Practice: Surrogate Key + Natural Key Constraint

The recommended approach is to use a surrogate primary key for the `@Id` and add a unique constraint on the natural key:

```java
@Entity
@Table(name = "books")
public class Book {

    @Id                                                 // Surrogate key
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NaturalId                                          // Natural key (Hibernate)
    @Column(nullable = false, unique = true, length = 13)
    private String isbn;                                // Business identifier

    @Column(nullable = false)
    private String title;
}
```

This gives you the best of both worlds:

- Surrogate key for efficient joins, foreign keys, and internal operations
- Natural key for business lookups and external identification

---

Choosing the Right Strategy

Here is a decision guide:

```
                    Start Here
                        |
                        v
              Need distributed IDs?
              (microservices, multi-DB)
                /              \
              YES               NO
               |                 |
               v                 v
         Use UUID          Need batch inserts?
                           (1000+ rows at once)
                           /              \
                         YES               NO
                          |                 |
                          v                 v
                     Use SEQUENCE       Use IDENTITY
                     (allocationSize    (simplest setup)
                      = 50+)
```

Strategy Comparison Table

```
Feature            IDENTITY    SEQUENCE    TABLE      UUID
-------------------------------------------------------------
Simplicity         Excellent   Good        Poor       Good
Batch insert       No          Yes         Yes        Yes
  performance
Pre-allocation     No          Yes         Yes        N/A (client)
Database support   All         Most        All        All
ID readability     1, 2, 3     1, 2, 3     1, 2, 3    550e8400-...
Index performance  Excellent   Excellent   Excellent  Good*
Distributed safe   No          No          No         Yes
ID before save     No          Possible    Possible   Yes
Size (bytes)       8           8           8          16

* UUID index performance can be improved with ordered/time-based UUIDs
```

Recommendations by Database

```
Database           Recommended Strategy
-----------------------------------------------
H2 (learning)      IDENTITY (simple) or
                    SEQUENCE (for batch practice)
PostgreSQL          SEQUENCE (native support)
MySQL 8+           IDENTITY or SEQUENCE
Oracle              SEQUENCE (native support)
```

```
SQL Server          SEQUENCE
Multi-database      SEQUENCE (most portable)
Microservices       UUID
```

Practical Examples

Example 1: Simple Entity with IDENTITY

```java
@Entity
@Table(name = "categories")
public class Category {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String name;

    @Column(length = 200)
    private String description;

    public Category() {}

    public Category(String name) {
        this.name = name;
    }

    // getters, setters...
}
```

Example 2: Entity with SEQUENCE

```java
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE,
                    generator = "product_seq")
    @SequenceGenerator(
        name = "product_seq",
```

```
        sequenceName = "product_id_seq",
        allocationSize = 25
    )
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(precision = 10, scale = 2)
    private BigDecimal price;

    public Product() {}

    public Product(String name, BigDecimal price) {
        this.name = name;
        this.price = price;
    }

    // getters, setters...
}
```

Example 3: Entity with UUID

```java
@Entity
@Table(name = "api_keys")
public class ApiKey {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false)
    private String owner;

    private boolean active;

    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
        this.active = true;
    }

    public ApiKey() {}

    public ApiKey(String name, String owner) {
```

```
        this.name = name;
        this.owner = owner;
    }

    // getters, setters...
}
```

Example 4: Entity with Composite Key

```java
@Embeddable
public class CourseRatingId implements Serializable {

    @Column(name = "student_id")
    private Long studentId;

    @Column(name = "course_id")
    private Long courseId;

    public CourseRatingId() {}
    public CourseRatingId(Long studentId, Long courseId) {
        this.studentId = studentId;
        this.courseId = courseId;
    }

    // getters, setters, equals, hashCode...

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof CourseRatingId)) return false;
        CourseRatingId that = (CourseRatingId) o;
        return Objects.equals(studentId, that.studentId) &&
               Objects.equals(courseId, that.courseId);
    }

    @Override
    public int hashCode() {
        return Objects.hash(studentId, courseId);
    }
}

@Entity
@Table(name = "course_ratings")
public class CourseRating {

    @EmbeddedId
    private CourseRatingId id;

    @Column(nullable = false)
    private int rating;     // 1-5
```

```
    private String review;

    private LocalDateTime ratedAt;

    public CourseRating() {}

    public CourseRating(Long studentId, Long courseId, int rating) {
        this.id = new CourseRatingId(studentId, courseId);
        this.rating = rating;
        this.ratedAt = LocalDateTime.now();
    }

    // getters, setters...
}
```

Example 5: Natural Key (No @GeneratedValue)

```java
@Entity
@Table(name = "currencies")
public class Currency {

    @Id
    @Column(length = 3)
    private String code;        // "USD", "EUR", "GBP" — set manually

    @Column(nullable = false, length = 50)
    private String name;        // "US Dollar", "Euro", "British Pound"

    @Column(nullable = false, length = 5)
    private String symbol;      // "$", "EUR", "GBP"

    public Currency() {}

    public Currency(String code, String name, String symbol) {
        this.code = code;
        this.name = name;
        this.symbol = symbol;
    }

    // getters, setters...
}

// Usage:
Currency usd = new Currency("USD", "US Dollar", "$");
repository.save(usd);    // No ID generation — "USD" is the key

Currency found = repository.findById("USD").orElseThrow();
```

Common Mistakes

1. Using `GenerationType.AUTO` and being surprised: AUTO might choose TABLE strategy (which is slow) or SEQUENCE (which creates a `hibernate_sequences` table). Always be explicit about your strategy.
2. Mismatching `allocationSize` and database INCREMENT BY: If `@SequenceGenerator(allocationSize = 50)` but the database sequence has `INCREMENT BY 1`, IDs will have gaps of 50. They must match.
3. Forgetting `equals()` and `hashCode()` on composite key classes: Without these, JPA cannot correctly identify entities in the persistence context. Use `Objects.equals()` and `Objects.hash()` on all key fields.
4. Using a mutable field as a natural primary key: If you use `email` as `@Id` and the user changes their email, you cannot simply update it — the primary key changed, which breaks foreign key references. Use surrogate keys for `@Id` and mark the natural key with `@NaturalId` + unique constraint.
5. Not considering batch performance: If your application does bulk inserts, IDENTITY prevents batching. Switch to SEQUENCE with an appropriate `allocationSize`.
6. Using `int` instead of `Long` for auto-generated IDs: `int` limits you to about 2.1 billion IDs. While unlikely to overflow, `Long` (9.2 quintillion) is standard practice and costs nothing extra.

Best Practices

1. Use `Long` for surrogate primary keys: It is the convention in the JPA ecosystem. Use `Long` (wrapper, nullable) rather than `long` (primitive) so JPA can detect new entities by `id == null`.
2. Use IDENTITY for simple applications, SEQUENCE for performance-critical ones: IDENTITY is simpler to configure. SEQUENCE enables batch inserts and is more portable.
3. Always set `allocationSize` explicitly for SEQUENCE: The default is 50, which is usually fine. But make sure your database sequence matches.
4. Use `@EmbeddedId` over `@IdClass` for composite keys: It is cleaner, groups key fields logically, and avoids field duplication.
5. Never expose database IDs in public APIs without consideration: If your API returns `id: 1, 2, 3`, users can guess other IDs. For public-facing APIs, consider using UUIDs or a separate public identifier.

6. Do not use business fields as `@Id`: Use surrogate keys for `@Id` and enforce business uniqueness with `@Column(unique = true)` or `@NaturalId`.

Summary

In this chapter, you learned every strategy for generating primary keys in JPA:

- IDENTITY uses database auto-increment. It is the simplest strategy but prevents JDBC batch inserts because Hibernate must execute each INSERT immediately to get the generated ID.
- SEQUENCE uses database sequence objects. It is the most efficient strategy because `allocationSize` pre-allocates IDs in batches, reducing database round-trips and enabling batch inserts.
- TABLE simulates sequences with a database table. It works everywhere but has poor performance due to row-level locking. Avoid in modern applications.
- AUTO lets Hibernate choose the strategy based on the database dialect. Avoid in production — be explicit.
- UUID generates globally unique 128-bit identifiers. Use for distributed systems and microservices where IDs must be generated independently.
- Composite keys combine multiple columns into a primary key using `@EmbeddedId` (recommended) or `@IdClass`. The key class must implement `Serializable` and override `equals()` and `hashCode()`.
- Surrogate keys (auto-generated, meaningless IDs) are recommended for `@Id`. Natural keys (business-meaningful values) should use `@NaturalId` or unique constraints.

Interview Questions

Q1: What are the four JPA generation strategies?

IDENTITY (database auto-increment), SEQUENCE (database sequence object), TABLE (simulated sequence using a table), and AUTO (provider chooses). JPA 3.1 also added UUID as a strategy.

Q2: Why does IDENTITY strategy prevent batch inserts?

With IDENTITY, the database generates the ID during the INSERT. Hibernate must execute each INSERT immediately and read back the generated ID to set it on the entity and maintain

the persistence context. This prevents grouping multiple INSERTs into a single batch operation.

Q3: What is `allocationSize` in @SequenceGenerator?

`allocationSize` tells Hibernate how many IDs to pre-allocate from the sequence in one database call. With `allocationSize = 50`, Hibernate calls `SELECT NEXT VALUE` once and caches 50 IDs. The next 49 `persist()` calls use cached IDs without a database round-trip. The database sequence's INCREMENT BY must match this value.

Q4: What is the difference between @EmbeddedId and @IdClass?

Both define composite primary keys. `@EmbeddedId` uses an `@Embeddable` class as a single field in the entity (cleaner, key is a value object). `@IdClass` requires the key fields to appear in both the entity and the ID class (more verbose, duplicated fields). `@EmbeddedId` is generally preferred.

Q5: What requirements must a composite key class satisfy?

It must implement `Serializable`, have a public no-argument constructor, and override `equals()` and `hashCode()`. For `@EmbeddedId`, it must also be annotated with `@Embeddable`. For `@IdClass`, its field names and types must match the `@Id` fields in the entity.

Q6: When would you use UUID instead of Long for primary keys?

In distributed systems or microservices where multiple services independently generate IDs without coordination. Also when IDs are exposed in public URLs and should not be guessable, when merging data from multiple databases, or when you need to generate IDs client-side before persisting.

Q7: What is the difference between a natural key and a surrogate key?

A surrogate key is an auto-generated value with no business meaning (e.g., `id = 42`). A natural key has real-world significance (e.g., `isbn = "978-0-13-468599-1"`). Best practice is to use surrogate keys for `@Id` (stable, efficient, simple foreign keys) and natural keys as unique business identifiers with `@NaturalId` or `@Column(unique = true)`.

Q8: Which generation strategy would you recommend for a PostgreSQL production application and why?

SEQUENCE. PostgreSQL has excellent native sequence support, and SEQUENCE with an appropriate `allocationSize` (25-50) enables batch inserts, pre-allocates IDs to reduce database round-trips, and avoids the bottleneck of TABLE strategy. It is the most performant and portable strategy for databases that support sequences.

Practice Exercises

Exercise 1: Strategy Comparison Create three entities (`IdentityEntity`, `SequenceEntity`, `UuidEntity`) using different generation strategies. Save 10 instances of each and compare: the SQL generated, the ID values assigned, and the number of database calls made (visible in SQL logs).

Exercise 2: Sequence Configuration Create a `Product` entity with SEQUENCE strategy, `allocationSize = 5`. Save 12 products. How many times does Hibernate call `SELECT NEXT VALUE`? What happens if you restart the application and save more products — do IDs continue sequentially or have gaps?

Exercise 3: Composite Key Model a library loan system with a composite key. Create a `Loan` entity with a composite key of `memberId` and `bookId`, plus fields for `loanDate` and `returnDate`. Use `@EmbeddedId`. Create a repository and demonstrate saving, finding by composite key, and checking existence.

Exercise 4: Natural Key Create a `Country` entity with the ISO country code (e.g., "US", "GB", "JP") as the natural primary key (no `@GeneratedValue`). Create a DataLoader that saves 10 countries. What happens if you try to save a duplicate code?

Exercise 5: Batch Performance Test Create two entities: one with IDENTITY strategy and one with SEQUENCE strategy (allocationSize = 50). Enable batch inserts in `application.properties`. Save 1000 instances of each and compare the SQL logs. With SEQUENCE, how many `SELECT NEXT VALUE` calls do you see? With IDENTITY, is batching used?

What Is Next?

In the next chapter, we will explore Basic CRUD Operations with JPA in much greater depth. You will learn the full entity lifecycle (New, Managed, Detached, Removed), understand every method in `CrudRepository` and `JpaRepository`, master `save` vs `saveAndFlush` vs `saveAll`, and learn how flush and clear operations work. By the end of Chapter 6, you will have complete control over creating, reading, updating, and deleting entities.

Chapter 6: Basic CRUD Operations with JPA

Learning Goals

By the end of this chapter, you will be able to:

- Understand the four entity lifecycle states: New, Managed, Detached, Removed
- Use every method in CrudRepository and JpaRepository
- Distinguish between save, saveAndFlush, and saveAll
- Understand when and why Hibernate flushes changes to the database
- Use flush and clear operations for fine-grained control
- Handle Optional returns safely
- Perform batch save and delete operations
- Know how dirty checking eliminates the need for explicit updates

Entity Lifecycle States

Every JPA entity exists in one of four states. Understanding these states is fundamental to understanding how JPA manages your data.

```
+-------------------------------------------------------------------+
|                    Entity Lifecycle States                         |
|                                                                   |
|  +----------+     persist()      +-----------+                   |
|  |          | -----------------> |           |                   |
|  |   NEW    |                    |  MANAGED  | <---+             |
|  | (Transient)                   |           |     |             |
|  +----------+                    +-----------+     |             |
```

```
|       ^                          |    |    |      |              |
|       |                  remove()|    |    |merge()|             |
|       |                          |    |    |      |              |
|       |                          v    |    |      |              |
|       |                   +-----------+|    |  +-----------+      |
|       |                   |           ||    |  |           |      |
|       |                   |  REMOVED  ||    +->| DETACHED  |      |
|       |                   |           ||      |           |      |
|       |                   +-----------+|      +-----------+      |
|       |                                |         ^               |
|       |                                |         |               |
|       |                                +---------+               |
|       |                             detach() / clear()           |
|       |                             / transaction ends           |
|       |                                                          |
|       +--- garbage collected (no reference)                      |
+------------------------------------------------------------------+
```

State Descriptions

```
State        In Persistence    In Database    Tracked by       Example
             Context?                         EntityManager?
---------------------------------------------------------------------------
NEW          No                No             No               new Student()
(Transient)

MANAGED      Yes               Yes (or will   Yes              After persist()
                               be at flush)                    or find()

DETACHED     No                Yes            No               After detach(),
                                                               clear(), or
                                                               transaction end

REMOVED      Yes (marked       Yes (will be   Yes              After remove()
             for deletion)     deleted at                      (before flush)
                               flush)
```

Lifecycle in Action

```java
// 1. NEW — just created, not tracked by JPA
Student student = new Student("Alice", "alice@example.com", 20);
// student is NEW (transient)
// Not in persistence context, not in database

// 2. MANAGED — tracked by JPA, changes are detected
studentRepository.save(student);
```

```
// student is now MANAGED
// In persistence context, will be INSERTed at flush

// 3. MANAGED — changes are auto-detected (dirty checking)
student.setAge(21);
// NO save() call needed!
// Hibernate detects the change and will UPDATE at flush

// 4. DETACHED — no longer tracked
// (happens automatically at end of @Transactional method)
// Changes to detached entities are NOT tracked

// 5. Re-attach a DETACHED entity
student.setEmail("alice.new@example.com");
studentRepository.save(student);
// save() calls merge() for existing entities
// Creates a new MANAGED copy with the updated state

// 6. REMOVED — marked for deletion
studentRepository.delete(student);
// student is REMOVED
// Will be DELETEd at flush
```

Visualizing the Flow

```
Time -->

new Student("Alice")        repository.save(s)         s.setAge(21)
     |                           |                          |
     v                           v                          v
  +-----+                   +--------+                  +--------+
  | NEW |  -- persist() --> |MANAGED | -- auto-detect ->|MANAGED |
  +-----+                   +--------+    (dirty check) +--------+
                                 |                          |
                                 |                          |
                            (transaction                (transaction
                             commits)                   commits)
                                 |                          |
                                 v                          v
                            INSERT INTO               UPDATE students
                            students ...              SET age = 21 ...
```

CrudRepository Methods

`CrudRepository` is the base interface that provides fundamental CRUD operations. Let us examine every method.

Setup: The Employee Entity

For this chapter's examples, we will use an `Employee` entity:

```java
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Department department;

    @Column(precision = 10, scale = 2)
    private BigDecimal salary;

    private boolean active;

    public enum Department {
        ENGINEERING, MARKETING, SALES, HR, FINANCE
    }

    public Employee() {}

    public Employee(String name, String email, Department department,
                    BigDecimal salary) {
        this.name = name;
        this.email = email;
        this.department = department;
        this.salary = salary;
        this.active = true;
    }

    // getters, setters, toString...
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
```

```
    public void setEmail(String email) { this.email = email; }
    public Department getDepartment() { return department; }
    public void setDepartment(Department department) { this.department = department; }
    public BigDecimal getSalary() { return salary; }
    public void setSalary(BigDecimal salary) { this.salary = salary; }
    public boolean isActive() { return active; }
    public void setActive(boolean active) { this.active = active; }

    @Override
    public String toString() {
        return "Employee{id=" + id + ", name='" + name + "', department=" +
               department + ", salary=" + salary + "}";
    }
}
```

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
}
```

CREATE Operations

save(S entity)

Saves a single entity. If the entity is new (ID is null), it calls `persist()` (INSERT). If the entity already exists (ID is not null), it calls `merge()` (UPDATE).

```java
// CREATE - new entity (id is null)
Employee emp = new Employee("Alice", "alice@company.com",
    Employee.Department.ENGINEERING, new BigDecimal("95000"));
Employee saved = employeeRepository.save(emp);
System.out.println("Saved with ID: " + saved.getId());  // ID is now set
```

```sql
-- SQL generated:
INSERT INTO employees (active, department, email, name, salary, id)
VALUES (true, 'ENGINEERING', 'alice@company.com', 'Alice', 95000.00, default)
```

saveAll(Iterable<S> entities)

Saves multiple entities in one call:

```java
List<Employee> employees = List.of(
    new Employee("Bob", "bob@company.com",
        Employee.Department.MARKETING, new BigDecimal("85000")),
    new Employee("Charlie", "charlie@company.com",
        Employee.Department.SALES, new BigDecimal("78000")),
    new Employee("Diana", "diana@company.com",
        Employee.Department.HR, new BigDecimal("82000"))
);

List<Employee> saved = employeeRepository.saveAll(employees);
System.out.println("Saved " + saved.size() + " employees");
```

```
saveAll() internals:
+-------------------------------------------------------------------+
|  for each entity in the list:                                     |
|    if entity.getId() == null:                                     |
|      entityManager.persist(entity)   --> schedules INSERT         |
|    else:                                                          |
|      entityManager.merge(entity)     --> schedules UPDATE         |
|                                                                   |
|  All INSERTs/UPDATEs execute at flush time                        |
|                                                                   |
|  Note: With IDENTITY strategy, each INSERT executes immediately.  |
|  With SEQUENCE strategy and batch_size configured, INSERTs can    |
|  be batched for better performance.                               |
+-------------------------------------------------------------------+
```

READ Operations

findById(ID id)

Returns an `Optional<T>` containing the entity, or empty if not found:

```java
// Safe way - with Optional
Optional<Employee> optional = employeeRepository.findById(1L);

// Pattern 1: ifPresent
optional.ifPresent(emp ->
    System.out.println("Found: " + emp)
);

// Pattern 2: ifPresentOrElse
optional.ifPresentOrElse(
    emp -> System.out.println("Found: " + emp),
    ()  -> System.out.println("Employee not found")
);
```

```
// Pattern 3: orElseThrow (for service methods)
Employee emp = employeeRepository.findById(1L)
    .orElseThrow(() -> new RuntimeException("Employee not found with ID: 1"));

// Pattern 4: orElse (with default value)
Employee emp = employeeRepository.findById(1L)
    .orElse(new Employee("Unknown", "unknown@company.com",
        Employee.Department.HR, BigDecimal.ZERO));
```

```sql
-- SQL generated:
SELECT e.id, e.active, e.department, e.email, e.name, e.salary
FROM employees e
WHERE e.id = ?
```

findAll()

Returns all entities in the table:

```java
List<Employee> all = employeeRepository.findAll();
System.out.println("Total employees: " + all.size());
all.forEach(System.out::println);
```

```sql
-- SQL generated:
SELECT e.id, e.active, e.department, e.email, e.name, e.salary
FROM employees e
```

Warning: `findAll()` loads every row into memory. For tables with thousands of rows, use pagination instead (Chapter 11).

findAllById(Iterable<ID> ids)

Returns entities matching a list of IDs:

```java
List<Employee> selected = employeeRepository.findAllById(List.of(1L, 3L, 5L));
System.out.println("Found " + selected.size() + " employees");
```

```sql
-- SQL generated:
SELECT e.id, e.active, e.department, e.email, e.name, e.salary
FROM employees e
WHERE e.id IN (?, ?, ?)
```

Note: If some IDs do not exist, the method returns only the entities that were found — no exception is thrown. The returned list might be smaller than the input list.

existsById(ID id)

Checks if an entity exists without loading it:

```java
boolean exists = employeeRepository.existsById(1L);
System.out.println("Employee 1 exists: " + exists);
```

```sql
-- SQL generated (efficient — does not load the entity):
SELECT COUNT(*) FROM employees WHERE id = ?
-- Returns true if count > 0
```

count()

Returns the total number of entities:

```java
long total = employeeRepository.count();
System.out.println("Total employees: " + total);
```

```sql
-- SQL generated:
SELECT COUNT(*) FROM employees
```

UPDATE Operations

JPA handles updates differently than you might expect. You do not call an explicit "update" method. Instead, you modify a managed entity and JPA detects the changes automatically.

Dirty Checking — Automatic Updates

```java
@Service
@Transactional    // Keeps entities managed for the duration of the method
public class EmployeeService {

    private final EmployeeRepository repository;

    public EmployeeService(EmployeeRepository repository) {
        this.repository = repository;
```

```
    }

    public Employee updateSalary(Long id, BigDecimal newSalary) {
        Employee emp = repository.findById(id)
            .orElseThrow(() -> new RuntimeException("Not found"));

        emp.setSalary(newSalary);   // Modify the managed entity

        // NO save() call needed!
        // Hibernate detects the change (dirty checking)
        // and generates an UPDATE at transaction commit

        return emp;
    }
}
```

```
Dirty Checking Flow:
+-------------------------------------------------------------------+
|  @Transactional method starts                                      |
|    |                                                               |
|    v                                                               |
|  findById(1L)                                                      |
|    --> SELECT ... FROM employees WHERE id = 1                      |
|    --> Employee loaded, state snapshot saved:                       |
|        {name:"Alice", salary:95000, department:ENGINEERING}        |
|    |                                                               |
|    v                                                               |
|  emp.setSalary(105000)                                             |
|    --> Java object changed in memory                               |
|    --> No SQL yet!                                                 |
|    |                                                               |
|    v                                                               |
|  @Transactional method ends (transaction commits)                  |
|    --> Hibernate compares current state vs snapshot:                |
|        Current:  {name:"Alice", salary:105000, department:ENG}     |
|        Snapshot: {name:"Alice", salary:95000,  department:ENG}     |
|        DIFF:     salary changed!                                   |
|    --> Generates and executes:                                      |
|        UPDATE employees SET salary = 105000 WHERE id = 1           |
|    --> COMMIT                                                      |
+-------------------------------------------------------------------+
```

This is one of JPA's most powerful features. You work with plain Java objects, and the framework handles persistence automatically.

Explicit save() for Updates

You can also use `save()` to update entities. This is necessary when:

- The entity is detached (not in a `@Transactional` context)
- You want to be explicit about the save operation

```java
// Load the entity (becomes detached after the method returns)
Employee emp = employeeRepository.findById(1L).orElseThrow();

// Modify the detached entity
emp.setDepartment(Employee.Department.FINANCE);
emp.setSalary(new BigDecimal("110000"));

// Must call save() to merge the detached entity
employeeRepository.save(emp);
```

```
save() for detached entities:
+-------------------------------------------------------------------+
|  save(detachedEmployee)                                            |
|    |                                                               |
|    v                                                               |
|  Is id null?                                                       |
|    NO --> entityManager.merge(detachedEmployee)                    |
|           |                                                        |
|           v                                                        |
|           SELECT ... WHERE id = ?   (load current state)           |
|           Copy detached state onto managed entity                  |
|           UPDATE ... SET ... WHERE id = ?   (at flush)             |
+-------------------------------------------------------------------+
```

When to Use save() vs Dirty Checking

```
+-------------------------------------------------------------------+
|  Approach              When to Use                                 |
|-------------------------------------------------------------------|
|  Dirty checking        Inside @Transactional methods               |
|  (no save() needed)    Entity is managed (loaded in this           |
|                        transaction)                                |
|                        Cleaner code, fewer SQL calls               |
|                                                                    |
|  Explicit save()       Entity is detached                          |
|                        In controllers or non-transactional         |
|                        code                                        |
|                        When you want to be explicit                |
+-------------------------------------------------------------------+
```

DELETE Operations

deleteById(ID id)

Deletes an entity by its primary key:

```java
employeeRepository.deleteById(3L);
```

```sql
-- SQL generated (two queries):
-- 1. Load the entity first
SELECT e.id, e.active, e.department, e.email, e.name, e.salary
FROM employees e WHERE e.id = ?

-- 2. Delete it
DELETE FROM employees WHERE id = ?
```

Why two queries? JPA requires the entity to be in the managed state before deletion. This allows lifecycle callbacks (`@PreRemove`, `@PostRemove`) to fire. If you want a single-query delete, use a custom `@Query` (Chapter 8).

delete(S entity)

Deletes a specific entity instance:

```java
Employee emp = employeeRepository.findById(1L).orElseThrow();
employeeRepository.delete(emp);
```

```sql
-- If entity is already managed, only the DELETE:
DELETE FROM employees WHERE id = ?
```

deleteAll()

Deletes all entities one by one (slow for large tables):

```java
employeeRepository.deleteAll();
```

```sql
-- Loads ALL entities first, then deletes one by one:
SELECT e.id, e.active, ... FROM employees e
DELETE FROM employees WHERE id = 1
DELETE FROM employees WHERE id = 2
DELETE FROM employees WHERE id = 3
-- ... for each entity!
```

Warning: `deleteAll()` loads every entity into memory and deletes them individually. For large tables, use `deleteAllInBatch()` instead (JpaRepository method).

deleteAllById(Iterable<ID> ids)

Deletes entities matching a list of IDs:

```java
employeeRepository.deleteAllById(List.of(2L, 4L, 6L));
```

JpaRepository — Additional Methods

`JpaRepository` extends `CrudRepository` with several useful methods:

Flush Operations

flush()

Forces Hibernate to send all pending SQL statements to the database immediately:

```java
Employee emp = new Employee("Eve", "eve@company.com",
    Employee.Department.ENGINEERING, new BigDecimal("92000"));
employeeRepository.save(emp);

// SQL is NOT yet sent to the database (it is queued)

employeeRepository.flush();

// NOW the INSERT has been sent to the database
// (but the transaction is not yet committed)
```

```
Normal flow (without flush):
+-------------------------------------------------------------------+
|  save(emp)       --> entity queued in persistence context          |
|  save(emp2)      --> entity queued in persistence context          |
|  save(emp3)      --> entity queued in persistence context          |
|  (end of @Transactional)                                          |
|       --> flush: INSERT emp, INSERT emp2, INSERT emp3              |
|       --> commit transaction                                      |
+-------------------------------------------------------------------+

With explicit flush:
+-------------------------------------------------------------------+
|  save(emp)       --> entity queued in persistence context          |
|  flush()         --> INSERT emp sent to database NOW               |
|  save(emp2)      --> entity queued in persistence context          |
|  flush()         --> INSERT emp2 sent to database NOW              |
|  (end of @Transactional)                                          |
|       --> nothing to flush (already done)                         |
```

```
|       --> commit transaction                                        |
+-------------------------------------------------------------------+
```

When to flush:

- When you need the database to validate constraints immediately
- When you need to call a stored procedure after the INSERT
- When you need the generated ID before the transaction commits
- When doing batch processing and need to manage memory

saveAndFlush(S entity)

Saves and immediately flushes to the database:

```java
// Equivalent to save() + flush()
Employee emp = employeeRepository.saveAndFlush(
    new Employee("Frank", "frank@company.com",
        Employee.Department.SALES, new BigDecimal("75000"))
);
// INSERT has been sent to the database immediately
// emp.getId() is guaranteed to be set
```

saveAllAndFlush(Iterable<S> entities)

Saves multiple entities and flushes:

```java
List<Employee> batch = List.of(emp1, emp2, emp3);
List<Employee> saved = employeeRepository.saveAllAndFlush(batch);
// All INSERTs sent to database immediately
```

Batch Delete Operations

deleteInBatch(Iterable<T> entities)

Deletes entities in a single SQL DELETE statement (no loading required):

```java
List<Employee> toDelete = employeeRepository.findAllById(List.of(1L, 2L, 3L));
employeeRepository.deleteInBatch(toDelete);
```

```sql
-- SINGLE query (much faster than deleteAll):
DELETE FROM employees WHERE id IN (?, ?, ?)
```

deleteAllInBatch()

Deletes all rows with a single SQL statement:

```java
employeeRepository.deleteAllInBatch();
```

```sql
-- SINGLE query:
DELETE FROM employees
```

Comparison: delete vs deleteInBatch

```
+-------------------------------------------------------------------+
|  Method                  SQL Queries          Lifecycle Callbacks  |
|-------------------------------------------------------------------|
|  delete(entity)          SELECT + DELETE      Yes (@PreRemove,     |
|                          per entity           @PostRemove fire)    |
|                                                                    |
|  deleteById(id)          SELECT + DELETE      Yes                  |
|                          per entity                                |
|                                                                    |
|  deleteAll()             SELECT all +         Yes                  |
|                          DELETE per entity                         |
|                                                                    |
|  deleteInBatch(list)     Single DELETE        No! (entities not    |
|                          WHERE id IN (...)    loaded, callbacks    |
|                                               do not fire)         |
|                                                                    |
|  deleteAllInBatch()      Single DELETE        No!                  |
|                          FROM table                                |
+-------------------------------------------------------------------+
```

Key trade-off: `deleteInBatch` and `deleteAllInBatch` are much faster but skip JPA lifecycle callbacks. Use them when performance matters and you do not have `@PreRemove`/`@PostRemove` callbacks.

getReferenceById(ID id)

Returns a lazy proxy — no database query until you access a field:

```java
Employee ref = employeeRepository.getReferenceById(1L);
// NO SQL executed yet! ref is a Hibernate proxy

System.out.println(ref.getName());
```

```
// NOW the SQL executes:
// SELECT ... FROM employees WHERE id = ?
```

```
getReferenceById vs findById:
+-------------------------------------------------------------------+
|                                                                   |
|  findById(1L):                                                    |
|    --> Immediately: SELECT ... WHERE id = 1                       |
|    --> Returns the actual entity (or empty Optional)              |
|    --> Use when you need the entity data NOW                      |
|                                                                   |
|  getReferenceById(1L):                                            |
|    --> No query! Returns a proxy object                           |
|    --> Query fires when you access any field                      |
|    --> Throws EntityNotFoundException if not found (on access)    |
|    --> Use when you only need a reference (for setting FKs)       |
|                                                                   |
+-------------------------------------------------------------------+
```

`getReferenceById()` is useful when you need an entity reference for setting a relationship, without actually loading the entity data:

```java
// Setting a foreign key without loading the parent entity
Order order = new Order();
order.setCustomer(customerRepository.getReferenceById(42L));
// No SELECT on customers table! Just sets customer_id = 42
orderRepository.save(order);
```

Complete CRUD Service Example

Here is a complete service class demonstrating all CRUD patterns:

```java
package com.example.hibernatedemo.service;

import com.example.hibernatedemo.entity.Employee;
import com.example.hibernatedemo.repository.EmployeeRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.util.List;
import java.util.Optional;
```

```
@Service
@Transactional
public class EmployeeService {

    private final EmployeeRepository repository;

    public EmployeeService(EmployeeRepository repository) {
        this.repository = repository;
    }

    // --- CREATE ---

    public Employee createEmployee(String name, String email,
                                   Employee.Department dept, BigDecimal salary) {
        Employee emp = new Employee(name, email, dept, salary);
        return repository.save(emp);
    }

    public List<Employee> createEmployees(List<Employee> employees) {
        return repository.saveAll(employees);
    }

    // --- READ ---

    @Transactional(readOnly = true)    // Optimization: no dirty checking
    public Employee getEmployee(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new RuntimeException(
                "Employee not found with id: " + id));
    }

    @Transactional(readOnly = true)
    public List<Employee> getAllEmployees() {
        return repository.findAll();
    }

    @Transactional(readOnly = true)
    public boolean employeeExists(Long id) {
        return repository.existsById(id);
    }

    @Transactional(readOnly = true)
    public long countEmployees() {
        return repository.count();
    }

    // --- UPDATE ---

    public Employee updateEmployee(Long id, String name, String email) {
        Employee emp = repository.findById(id)
            .orElseThrow(() -> new RuntimeException("Not found: " + id));

        emp.setName(name);
        emp.setEmail(email);
```

```
        // No save() needed! Dirty checking handles the UPDATE
        return emp;
    }

    public Employee updateSalary(Long id, BigDecimal newSalary) {
        Employee emp = repository.findById(id)
            .orElseThrow(() -> new RuntimeException("Not found: " + id));

        emp.setSalary(newSalary);
        // Dirty checking will generate: UPDATE employees SET salary = ? WHERE id = ?

        return emp;
    }

    public Employee deactivateEmployee(Long id) {
        Employee emp = repository.findById(id)
            .orElseThrow(() -> new RuntimeException("Not found: " + id));

        emp.setActive(false);
        return emp;
    }

    // --- DELETE ---

    public void deleteEmployee(Long id) {
        if (!repository.existsById(id)) {
            throw new RuntimeException("Employee not found with id: " + id);
        }
        repository.deleteById(id);
    }

    public void deleteEmployees(List<Long> ids) {
        List<Employee> employees = repository.findAllById(ids);
        repository.deleteInBatch(employees);  // Efficient batch delete
    }

    public void deleteAllEmployees() {
        repository.deleteAllInBatch();  // Single DELETE statement
    }
}
```

@Transactional(readOnly = true)

Notice the `@Transactional(readOnly = true)` on read methods. This is a performance optimization:

```
+-------------------------------------------------------------------+
|  @Transactional(readOnly = true)                                  |
|                                                                   |
|  What it does:                                                    |
```

```
|  1. Tells Hibernate to skip dirty checking for all entities       |
|     loaded in this transaction (no snapshot comparison)            |
|  2. Tells the JDBC driver this is a read-only transaction         |
|     (driver may optimize accordingly)                              |
|  3. Prevents accidental writes (some databases throw errors       |
|     on writes in read-only transactions)                           |
|                                                                    |
|  Performance benefit:                                              |
|  - No snapshot stored for dirty checking                           |
|  - Less memory used                                                |
|  - Faster transaction commit (nothing to flush)                    |
|                                                                    |
|  Use for: findById, findAll, count, exists, any read-only method  |
+-------------------------------------------------------------------+
```

Understanding Flush Modes

Hibernate does not execute SQL immediately when you call `save()` or modify an entity. Instead, it queues changes and sends them to the database during a flush. Understanding when flushes happen is important for debugging.

When Does Hibernate Flush?

```
+-------------------------------------------------------------------+
|  Automatic Flush Triggers                                          |
|                                                                    |
|  1. Transaction commit                                             |
|     @Transactional method returns --> flush + commit               |
|                                                                    |
|  2. Before a JPQL/SQL query                                        |
|     If you persist an entity and then query for it,                |
|     Hibernate flushes first to ensure the query sees               |
|     the new data                                                   |
|                                                                    |
|  3. Explicit flush() call                                          |
|     repository.flush() or entityManager.flush()                    |
|                                                                    |
|  4. saveAndFlush() call                                            |
|     Combines save + immediate flush                                |
+-------------------------------------------------------------------+
```

Flush Before Query Example

```java
@Transactional
public void demonstrateFlushBeforeQuery() {
    // Save a new employee (queued, not yet in DB)
    Employee emp = new Employee("Grace", "grace@company.com",
        Employee.Department.ENGINEERING, new BigDecimal("100000"));
    employeeRepository.save(emp);

    // Query all employees - Hibernate flushes FIRST
    // so the query includes the new employee
    long count = employeeRepository.count();
    // Flush happens here automatically (before the COUNT query)
    // Then: SELECT COUNT(*) FROM employees

    System.out.println("Count: " + count);  // Includes Grace
}
```

```
Timeline:
+-------------------------------------------------------------------+
|  save(grace)     --> queued in persistence context (no SQL)        |
|                                                                    |
|  count()         --> Hibernate detects a pending query             |
|                  --> AUTO flush: INSERT INTO employees ... (grace) |
|                  --> Then: SELECT COUNT(*) FROM employees          |
|                  --> Returns count including grace                 |
|                                                                    |
|  method returns  --> transaction commits                           |
+-------------------------------------------------------------------+
```

Working with Optional

Spring Data JPA methods that return a single entity use `Optional<T>`. Here are the key patterns:

Safe Patterns

```java
// Pattern 1: orElseThrow - use in service methods
public Employee getEmployee(Long id) {
    return repository.findById(id)
        .orElseThrow(() -> new EmployeeNotFoundException(id));
}

// Pattern 2: ifPresent - use when action is optional
public void logIfExists(Long id) {
```

```
    repository.findById(id)
        .ifPresent(emp -> log.info("Found employee: {}", emp.getName()));
}

// Pattern 3: map — transform the value
public String getEmployeeName(Long id) {
    return repository.findById(id)
        .map(Employee::getName)
        .orElse("Unknown");
}

// Pattern 4: filter — add conditions
public Optional<Employee> findActiveEmployee(Long id) {
    return repository.findById(id)
        .filter(Employee::isActive);
}

// Pattern 5: or — fallback to another lookup
public Employee findByIdOrEmail(Long id, String email) {
    return repository.findById(id)
        .or(() -> repository.findByEmail(email))
        .orElseThrow(() -> new RuntimeException("Not found"));
}
```

Anti-Patterns to Avoid

```java
// BAD: Calling .get() without checking
Employee emp = repository.findById(id).get();
// Throws NoSuchElementException if not found!

// BAD: Checking isPresent() then get()
Optional<Employee> opt = repository.findById(id);
if (opt.isPresent()) {
    Employee emp = opt.get();
    // ... use emp
}
// BETTER: Use ifPresent, map, or orElseThrow

// BAD: Using Optional as a field type
public class MyService {
    private Optional<Employee> lastEmployee;  // Don't do this!
}
// Optional is for return values, not fields
```

Batch Operations and Performance

Saving in Batches

For large numbers of entities, saving one at a time is inefficient:

```java
// SLOW: 1000 individual save calls
for (int i = 0; i < 1000; i++) {
    repository.save(new Employee("Emp-" + i, "emp" + i + "@company.com",
        Employee.Department.ENGINEERING, new BigDecimal("50000")));
}

// BETTER: saveAll with a list
List<Employee> batch = new ArrayList<>();
for (int i = 0; i < 1000; i++) {
    batch.add(new Employee("Emp-" + i, "emp" + i + "@company.com",
        Employee.Department.ENGINEERING, new BigDecimal("50000")));
}
repository.saveAll(batch);  // Single method call
```

Clearing the Persistence Context

When saving very large numbers of entities, the persistence context grows and consumes memory. Periodically flush and clear:

```java
@Transactional
public void importEmployees(List<Employee> employees) {
    int batchSize = 50;

    for (int i = 0; i < employees.size(); i++) {
        entityManager.persist(employees.get(i));

        if (i > 0 && i % batchSize == 0) {
            entityManager.flush();    // Send SQL to database
            entityManager.clear();    // Free memory (detach all entities)
            System.out.println("Flushed batch at index " + i);
        }
    }
    // Final flush happens at transaction commit
}
```

```
Memory usage during batch processing:
+-------------------------------------------------------------------+
|                                                                   |
|  Without flush/clear:                                             |
|  Memory: [##########] [####################] [################...]|
|  Entities in context keep growing --> OutOfMemoryError            |
|                                                                   |
|  With flush/clear every 50:                                       |
|  Memory: [####] flush/clear [####] flush/clear [####] ...         |
```

```
|  Memory stays bounded                                                |
+----------------------------------------------------------------------+
```

Common Mistakes

1. Calling save() on already-managed entities inside @Transactional: Dirty checking handles updates automatically. Calling `save()` on a managed entity is redundant (though not harmful — it just triggers an unnecessary merge).

```java
@Transactional
public void updateName(Long id, String name) {
    Employee emp = repository.findById(id).orElseThrow();
    emp.setName(name);
    // repository.save(emp);  // UNNECESSARY! Dirty checking handles it
}
```

2. Using deleteAll() on large tables: `deleteAll()` loads every entity then deletes them one by one. Use `deleteAllInBatch()` for a single DELETE statement.
3. Calling .get() on Optional without checking: Always use `orElseThrow()`, `orElse()`, `ifPresent()`, or `map()`. Never call `.get()` directly.
4. Forgetting @Transactional: Without `@Transactional`, each repository call runs in its own transaction. This means entities become detached between calls, dirty checking does not work, and multiple database operations are not atomic.
5. Not using readOnly = true for read operations: Read-only transactions skip dirty checking and snapshot storage, improving performance for query-heavy methods.
6. Expecting deleteById() to throw an exception for non-existent IDs: In Spring Data JPA 3.x, `deleteById()` for a non-existent ID does not throw — it simply does nothing (after running a SELECT). Check with `existsById()` first if you need to verify existence.

Best Practices

1. Use @Transactional at the service layer: Put `@Transactional` on service classes or methods, not on repositories or controllers. The service layer defines the business transaction boundary.

2. Use @Transactional(readOnly = true) for read operations: This is a free performance optimization. Mark all read-only service methods with it.
3. Let dirty checking handle updates: Inside `@Transactional` methods, modify managed entities directly. Do not call `save()` unless the entity is detached.
4. Use deleteInBatch for bulk deletes: When deleting multiple entities and you do not need lifecycle callbacks, `deleteInBatch()` is significantly faster than `deleteAll()`.
5. Use orElseThrow() for required lookups: When an entity must exist, use `orElseThrow()` with a descriptive exception. This makes the error message clear and the code intention obvious.
6. Use saveAll() instead of save() in a loop: `saveAll()` is more efficient and expresses the batch intent clearly.

Summary

In this chapter, you mastered CRUD operations in JPA:

- Entity lifecycle states (New, Managed, Detached, Removed) determine how JPA tracks and persists changes. Managed entities are tracked; detached entities are not.
- Dirty checking automatically detects changes to managed entities and generates UPDATE statements at flush time. Inside `@Transactional` methods, you do not need to call `save()` after modifying an entity.
- CrudRepository provides `save`, `findById`, `findAll`, `delete`, `count`, and `existsById`. These cover most basic data access needs.
- JpaRepository adds `flush()`, `saveAndFlush()`, `deleteInBatch()`, `deleteAllInBatch()`, and `getReferenceById()` for finer control over persistence and performance.
- Flush sends queued SQL to the database. It happens automatically at transaction commit, before queries, and when you call `flush()` explicitly.
- Optional must be handled safely using `orElseThrow()`, `ifPresent()`, `map()`, or `orElse()`. Never call `.get()` without checking.
- @Transactional(readOnly = true) optimizes read operations by skipping dirty checking.
- Batch operations (`saveAll`, `deleteInBatch`, `deleteAllInBatch`) are significantly more efficient than individual operations for large datasets.

Interview Questions

Q1: What are the four entity lifecycle states in JPA?

New (transient — not tracked, not in DB), Managed (tracked by EntityManager, in persistence context), Detached (was managed but no longer tracked — after transaction ends or clear()), and Removed (marked for deletion, will be deleted at flush). Only managed entities are tracked for dirty checking.

Q2: What is dirty checking and how does it work?

Dirty checking is JPA's mechanism for automatically detecting changes to managed entities. When an entity is loaded, Hibernate takes a snapshot of its state. At flush time (usually transaction commit), Hibernate compares the current state against the snapshot. If any field has changed, it generates and executes an UPDATE statement. This means you do not need to call save() explicitly for updates to managed entities.

Q3: What is the difference between save() and saveAndFlush()?

`save()` adds the entity to the persistence context and schedules an INSERT or UPDATE, but the SQL is not immediately sent to the database. `saveAndFlush()` does the same but also immediately flushes the persistence context, sending the SQL to the database right away. Use `saveAndFlush()` when you need the database to validate constraints immediately or when you need the generated ID before the transaction commits.

Q4: Why does deleteById() execute a SELECT before the DELETE?

JPA requires entities to be in the managed state before removal. This ensures lifecycle callbacks (`@PreRemove`, `@PostRemove`) fire correctly. `deleteById()` first loads the entity with `findById()`, making it managed, then calls `remove()` on it. If you do not need lifecycle callbacks, use `deleteInBatch()` or a custom `@Query` with a DELETE statement for better performance.

Q5: What is the difference between deleteAll() and deleteAllInBatch()?

`deleteAll()` loads every entity into memory, then deletes them one by one with individual DELETE statements. It fires lifecycle callbacks but is very slow for large tables. `deleteAllInBatch()` executes a single `DELETE FROM table` SQL statement — no entity loading, no lifecycle callbacks, but much faster.

Q6: What does @Transactional(readOnly = true) do?

It marks the transaction as read-only, which provides two optimizations: (1) Hibernate skips dirty checking — no snapshots are taken and no comparison is done at flush time, saving memory and CPU. (2) The JDBC driver is hinted that this is read-only, allowing potential database-level optimizations. Use it for all methods that only read data.

Q7: When should you call flush() explicitly?

Explicit flush is needed when: (1) you need the database to validate constraints before the transaction commits, (2) you need to interleave SQL execution with other logic, (3) you are doing batch processing and need to clear the persistence context periodically to manage memory (flush then clear), (4) you need the generated ID from an INSERT before the transaction ends.

Q8: What is getReferenceById() and how does it differ from findById()?

`getReferenceById()` returns a lazy proxy without executing a database query. The actual SQL fires only when you access a non-ID field. It is useful for setting foreign key references without loading the entity. `findById()` immediately executes a SELECT and returns the full entity (or empty Optional). If the entity does not exist, `getReferenceById()` throws `EntityNotFoundException` on field access, while `findById()` returns an empty Optional.

Practice Exercises

Exercise 1: CRUD Service Create a `BookService` with methods: `createBook`, `getBook`, `getAllBooks`, `updateTitle`, `deleteBook`. Use proper `@Transactional` annotations (readOnly for reads). Demonstrate dirty checking by updating a book without calling `save()`.

Exercise 2: Batch Performance Save 500 employees using three approaches: (a) individual `save()` calls in a loop, (b) `saveAll()` with a list, (c) `saveAll()` with flush/clear every 50. Enable SQL logging and count the INSERT statements for each approach. Which is fastest?

Exercise 3: Delete Comparison Create 100 employees. Then delete them using: (a) `deleteAll()`, (b) `deleteAllInBatch()`, (c) `deleteAllById()` with all IDs. Count the SQL statements for each approach and measure which is fastest.

Exercise 4: Lifecycle States Write a `CommandLineRunner` that demonstrates all four entity states with print statements and SQL logging:

1. Create a `new Employee()` and print its ID (should be null — NEW state)
2. Call `save()` and print its ID (should be set — MANAGED state)
3. Modify the entity and observe the UPDATE at flush (MANAGED + dirty checking)
4. Call `delete()` and observe the DELETE (REMOVED state)

Exercise 5: Optional Handling Create a service that finds an employee by ID. Write five different implementations using: `orElseThrow()`, `ifPresent()`, `map()`, `orElse()`, and `filter()`. Write a test or runner that calls each one with both existing and non-existing IDs.

What Is Next?

In the next chapter, we will explore Spring Data Derived Query Methods in depth. You will learn the complete naming convention for query methods, all available keywords (Between, Like, In, OrderBy, and many more), how to use different return types (List, Optional, Stream, Page), and how to write complex finders without any SQL or JPQL. By the end of Chapter 7, you will be able to express most queries simply by naming a method.

Chapter 7: Spring Data Derived Query Methods

Learning Goals

By the end of this chapter, you will be able to:

- Write query methods using Spring Data JPA naming conventions
- Use all major query keywords (Between, Like, In, OrderBy, and more)
- Return different result types: List, Optional, Stream, and primitives
- Limit query results with First and Top
- Combine multiple conditions with And and Or
- Write delete and count query methods
- Use @Modifying for update and delete operations
- Debug derived query methods when they fail to parse

How Derived Queries Work

Spring Data JPA reads your method name, parses it into parts, and generates JPQL (and ultimately SQL) automatically. You write zero query code — the method name IS the query.

```
Your Method Name                      What Spring Data Does
--------------------                  ----------------------------------------

findByEmail(String email)             1. Parses method name:
                                         find = query type
                                         By = predicate start
                                         Email = field name
                                      2. Generates JPQL:
```

```
                                          SELECT e FROM Employee e
                                          WHERE e.email = ?1
                                       3. Hibernate generates SQL:
                                          SELECT ... FROM employees
                                          WHERE email = ?
```

The Parsing Formula

Every derived query method follows this structure:

```
+-------------------------------------------------------------------+
|                                                                   |
|  [action] + By + [conditions] + [ordering]                        |
|                                                                   |
|  action:     find, read, get, query, search, stream,              |
|              count, exists, delete                                |
|                                                                   |
|  By:         separates the action from the conditions             |
|                                                                   |
|  conditions: FieldName + [Keyword] + And/Or + FieldName + ...     |
|              e.g., NameContaining, AgeGreaterThan,                |
|              EmailAndDepartment                                   |
|                                                                   |
|  ordering:   OrderByFieldAsc, OrderByFieldDesc                    |
|              (optional, at the end)                                |
|                                                                   |
+-------------------------------------------------------------------+

Examples:
  findByName                    --> WHERE name = ?
  findByNameAndAge              --> WHERE name = ? AND age = ?
  findByAgeGreaterThanOrderByNameAsc
                                --> WHERE age > ? ORDER BY name ASC
  countByDepartment             --> SELECT COUNT(*) WHERE department = ?
  existsByEmail                 --> SELECT COUNT(*) > 0 WHERE email = ?
  deleteByActive                --> DELETE WHERE active = ?
```

Setup: The Product Entity

For this chapter, we will use a `Product` entity with diverse field types:

```java
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200)
    private String name;

    @Column(nullable = false, unique = true, length = 20)
    private String sku;

    @Column(precision = 10, scale = 2)
    private BigDecimal price;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private Category category;

    private int stockQuantity;

    private boolean active;

    private LocalDate launchDate;

    @Column(length = 100)
    private String brand;

    public enum Category {
        ELECTRONICS, CLOTHING, BOOKS, FOOD, HOME, SPORTS
    }

    // constructors, getters, setters, toString...
    public Product() {}

    public Product(String name, String sku, BigDecimal price,
                   Category category, int stock, String brand) {
        this.name = name;
        this.sku = sku;
        this.price = price;
        this.category = category;
        this.stockQuantity = stock;
        this.active = true;
        this.launchDate = LocalDate.now();
        this.brand = brand;
    }
}
```

Equality and Simple Matching

Exact Match

```java
public interface ProductRepository extends JpaRepository<Product, Long> {

    // Find by exact field value
    List<Product> findByCategory(Product.Category category);
    // SQL: SELECT ... FROM products WHERE category = ?

    Optional<Product> findBySku(String sku);
    // SQL: SELECT ... FROM products WHERE sku = ?

    List<Product> findByActive(boolean active);
    // SQL: SELECT ... FROM products WHERE active = ?

    List<Product> findByBrand(String brand);
    // SQL: SELECT ... FROM products WHERE brand = ?
}
```

Note: `findBy` and `findAllBy` are equivalent. Both return a collection. For single results, use `Optional<T>` as the return type.

Is, Equals (Synonyms for Exact Match)

```java
// These three are identical:
List<Product> findByCategory(Product.Category category);
List<Product> findByCategoryIs(Product.Category category);
List<Product> findByCategoryEquals(Product.Category category);
// All generate: WHERE category = ?
```

Not, IsNot

```java
List<Product> findByCategoryNot(Product.Category category);
// SQL: WHERE category <> ?

List<Product> findByActiveIsNot(boolean active);
// SQL: WHERE active <> ?
```

Comparison Keywords

Numeric Comparisons

```java
// Greater than
List<Product> findByPriceGreaterThan(BigDecimal price);
// SQL: WHERE price > ?

// Greater than or equal
List<Product> findByPriceGreaterThanEqual(BigDecimal price);
// SQL: WHERE price >= ?

// Less than
List<Product> findByPriceLessThan(BigDecimal price);
// SQL: WHERE price < ?

// Less than or equal
List<Product> findByPriceLessThanEqual(BigDecimal price);
// SQL: WHERE price <= ?

// Between (inclusive on both ends)
List<Product> findByPriceBetween(BigDecimal min, BigDecimal max);
// SQL: WHERE price BETWEEN ? AND ?

// Stock quantity examples
List<Product> findByStockQuantityGreaterThan(int quantity);
// SQL: WHERE stock_quantity > ?

List<Product> findByStockQuantityLessThanEqual(int quantity);
// SQL: WHERE stock_quantity <= ?
```

Date Comparisons

```java
// Products launched after a date
List<Product> findByLaunchDateAfter(LocalDate date);
// SQL: WHERE launch_date > ?

// Products launched before a date
List<Product> findByLaunchDateBefore(LocalDate date);
// SQL: WHERE launch_date < ?

// Products launched between two dates
List<Product> findByLaunchDateBetween(LocalDate start, LocalDate end);
// SQL: WHERE launch_date BETWEEN ? AND ?
```

Keyword Reference: Comparisons

```
Keyword              SQL Equivalent        Example Method
------------------------------------------------------------------------
GreaterThan          >                     findByPriceGreaterThan
GreaterThanEqual     >=                    findByPriceGreaterThanEqual
LessThan             <                     findByPriceLessThan
LessThanEqual        <=                    findByPriceLessThanEqual
Between              BETWEEN x AND y       findByPriceBetween
After                >                     findByLaunchDateAfter
Before               <                     findByLaunchDateBefore
```

Note: `After` and `Before` are equivalent to `GreaterThan` and `LessThan` but read better with date fields.

String Matching Keywords

Like, NotLike

```java
// Requires caller to provide wildcards
List<Product> findByNameLike(String pattern);
// Usage: findByNameLike("%Widget%")
// SQL: WHERE name LIKE ?

List<Product> findByNameNotLike(String pattern);
// SQL: WHERE name NOT LIKE ?
```

Containing, StartingWith, EndingWith

These methods add the `%` wildcards automatically:

```java
// Contains substring (adds % on both sides)
List<Product> findByNameContaining(String keyword);
// Usage: findByNameContaining("Widget")
// SQL: WHERE name LIKE '%Widget%'

// Starts with prefix (adds % on the right)
List<Product> findByNameStartingWith(String prefix);
// Usage: findByNameStartingWith("Super")
// SQL: WHERE name LIKE 'Super%'

// Ends with suffix (adds % on the left)
```

```
List<Product> findByNameEndingWith(String suffix);
// Usage: findByNameEndingWith("Pro")
// SQL: WHERE name LIKE '%Pro'
```

```
Like vs Containing vs StartingWith vs EndingWith:
+-------------------------------------------------------------------+
|  Method                          Pattern Sent to SQL              |
|-------------------------------------------------------------------|
|  findByNameLike("%Widget%")      WHERE name LIKE '%Widget%'       |
|  findByNameContaining("Widget")  WHERE name LIKE '%Widget%'       |
|  findByNameStartingWith("Wi")    WHERE name LIKE 'Wi%'            |
|  findByNameEndingWith("get")     WHERE name LIKE '%get'           |
|                                                                   |
|  Like:          You provide the % wildcards                       |
|  Containing:    % added on both sides automatically               |
|  StartingWith:  % added on right side automatically               |
|  EndingWith:    % added on left side automatically                |
+-------------------------------------------------------------------+
```

Case-Insensitive Matching

```java
// Ignore case for string comparisons
List<Product> findByNameIgnoreCase(String name);
// SQL: WHERE UPPER(name) = UPPER(?)

List<Product> findByNameContainingIgnoreCase(String keyword);
// SQL: WHERE UPPER(name) LIKE UPPER('%keyword%')

List<Product> findByBrandIgnoreCase(String brand);
// SQL: WHERE UPPER(brand) = UPPER(?)
```

Keyword Reference: Strings

```
Keyword              SQL Equivalent        Wildcard Handling
-----------------------------------------------------------------------
Like                 LIKE                  Manual (you add %)
NotLike              NOT LIKE              Manual
Containing           LIKE '%x%'            Automatic (both sides)
StartingWith         LIKE 'x%'             Automatic (right side)
EndingWith           LIKE '%x'             Automatic (left side)
IgnoreCase           UPPER(col)=UPPER(?)   Case-insensitive
```

Null Checking

```java
// Field is null
List<Product> findByBrandIsNull();
// SQL: WHERE brand IS NULL

// Field is not null
List<Product> findByBrandIsNotNull();
// SQL: WHERE brand IS NOT NULL

// Also valid syntax:
List<Product> findByBrandNull();        // Same as IsNull
List<Product> findByBrandNotNull();     // Same as IsNotNull
```

Boolean Keywords

```java
// True
List<Product> findByActiveTrue();
// SQL: WHERE active = true

// False
List<Product> findByActiveFalse();
// SQL: WHERE active = false
```

These are cleaner than `findByActive(true)` when you always query for a specific boolean value.

Collection Keywords: In and NotIn

```java
// Find products in a set of categories
List<Product> findByCategoryIn(Collection<Product.Category> categories);
// Usage: findByCategoryIn(List.of(Category.ELECTRONICS, Category.BOOKS))
// SQL: WHERE category IN (?, ?)

// Find products NOT in a set of categories
List<Product> findByCategoryNotIn(Collection<Product.Category> categories);
```

```
// SQL: WHERE category NOT IN (?, ?)

// Works with any field type
List<Product> findByBrandIn(Collection<String> brands);
// Usage: findByBrandIn(List.of("Apple", "Samsung", "Sony"))
// SQL: WHERE brand IN (?, ?, ?)
```

Combining Conditions: And, Or

And (Both conditions must match)

```java
// Two conditions
List<Product> findByCategoryAndActive(Product.Category category, boolean active);
// SQL: WHERE category = ? AND active = ?

// Three conditions
List<Product> findByCategoryAndActiveAndPriceGreaterThan(
    Product.Category category, boolean active, BigDecimal price);
// SQL: WHERE category = ? AND active = ? AND price > ?

// String + numeric
List<Product> findByBrandAndPriceLessThan(String brand, BigDecimal price);
// SQL: WHERE brand = ? AND price < ?
```

Or (Either condition must match)

```java
List<Product> findByCategoryOrBrand(Product.Category category, String brand);
// SQL: WHERE category = ? OR brand = ?

List<Product> findByPriceLessThanOrStockQuantityGreaterThan(
    BigDecimal price, int stock);
// SQL: WHERE price < ? OR stock_quantity > ?
```

Combining And with Or

For complex conditions mixing And and Or, the method name becomes unwieldy. Use `@Query` instead (Chapter 8):

```java
// This works but is hard to read:
List<Product> findByCategoryAndPriceGreaterThanOrBrandAndActiveTrue(
    Product.Category cat, BigDecimal price, String brand);
// SQL: WHERE (category = ? AND price > ?) OR (brand = ? AND active = true)

// Better: Use @Query for complex logic (Chapter 8)
```

Ordering Results

OrderBy in Method Name

```java
// Ascending (default)
List<Product> findByCategoryOrderByNameAsc(Product.Category category);
// SQL: WHERE category = ? ORDER BY name ASC

// Descending
List<Product> findByCategoryOrderByPriceDesc(Product.Category category);
// SQL: WHERE category = ? ORDER BY price DESC

// Multiple sort fields
List<Product> findByCategoryOrderByPriceDescNameAsc(Product.Category category);
// SQL: WHERE category = ? ORDER BY price DESC, name ASC

// Order all results (no filter)
List<Product> findAllByOrderByNameAsc();
// SQL: SELECT ... FROM products ORDER BY name ASC
// Note: "AllBy" + "OrderBy" — the By after All is required
```

Sort Parameter (More Flexible)

Instead of baking the sort order into the method name, pass a `Sort` parameter:

```java
// Method definition — accepts Sort parameter
List<Product> findByCategory(Product.Category category, Sort sort);

// Usage — sort is decided at call time
List<Product> cheapest = repository.findByCategory(
    Category.ELECTRONICS,
    Sort.by("price").ascending()
);

List<Product> newest = repository.findByCategory(
```

```
    Category.ELECTRONICS,
    Sort.by("launchDate").descending()
);

// Multiple sort fields
List<Product> sorted = repository.findByCategory(
    Category.ELECTRONICS,
    Sort.by(Sort.Order.desc("price"), Sort.Order.asc("name"))
);
```

```
OrderBy in method name vs Sort parameter:
+-------------------------------------------------------------------+
|  OrderBy in Name                  Sort Parameter                  |
|  findByCategoryOrderByPriceDesc   findByCategory(cat, Sort)       |
|                                                                   |
|  Fixed sort order                 Flexible sort order             |
|  Sort is part of the method       Sort is decided by caller       |
|  Simpler for one sort order       Better when sort varies         |
|  New method for each sort         One method handles all sorts    |
+-------------------------------------------------------------------+
```

Limiting Results: First, Top

First and Top

```java
// Get the first result
Optional<Product> findFirstByCategoryOrderByPriceAsc(Product.Category category);
// SQL: WHERE category = ? ORDER BY price ASC LIMIT 1
// Returns the cheapest product in a category

// Get the top result (same as First)
Optional<Product> findTopByCategoryOrderByPriceDesc(Product.Category category);
// SQL: WHERE category = ? ORDER BY price DESC LIMIT 1
// Returns the most expensive product in a category

// Get top N results
List<Product> findTop3ByCategoryOrderByPriceDesc(Product.Category category);
// SQL: WHERE category = ? ORDER BY price DESC LIMIT 3
// Returns the 3 most expensive products

List<Product> findFirst5ByOrderByLaunchDateDesc();
// SQL: SELECT ... FROM products ORDER BY launch_date DESC LIMIT 5
// Returns the 5 most recently launched products
```

```
// Count variant
List<Product> findTop10ByActiveTrueOrderByStockQuantityAsc();
// SQL: WHERE active = true ORDER BY stock_quantity ASC LIMIT 10
// Returns 10 products with lowest stock
```

```
First vs Top:
+-------------------------------------------------------------------+
|  "First" and "Top" are synonyms — they behave identically         |
|                                                                   |
|  findFirst...     = findTop...                                    |
|  findFirst3...    = findTop3...                                   |
|  findFirst10...   = findTop10...                                  |
|                                                                   |
|  Without a number: returns 1 result (Optional<T> or T)            |
|  With a number N:  returns up to N results (List<T>)              |
+-------------------------------------------------------------------+
```

Return Types

Derived query methods support various return types:

```
Return Type           When to Use                     Example
-----------------------------------------------------------------------
List<T>               Multiple results expected       findByCategory(cat)
Optional<T>           Zero or one result              findBySku(sku)
T                     Exactly one result (risky)      findBySku(sku)
                      Throws if not found!
Stream<T>             Process results lazily           streamByCategory(cat)
long                  Count queries                   countByCategory(cat)
boolean               Existence checks                existsBySku(sku)
void                  Delete methods                  deleteByCategory(cat)
Page<T>               Paginated results               findByCategory(cat, pageable)
Slice<T>              Paginated without total count   findByCategory(cat, pageable)
```

Stream Return Type

```java
// Returns a Java Stream for lazy processing
@QueryHints(@QueryHint(name = HINT_FETCH_SIZE, value = "25"))
Stream<Product> streamByCategory(Product.Category category);
```

```java
// Usage - MUST be in @Transactional and close the stream
@Transactional(readOnly = true)
public void processExpensiveProducts() {
    try (Stream<Product> stream = repository.streamByCategory(Category.ELECTRONICS)) {
        stream
            .filter(p -> p.getPrice().compareTo(new BigDecimal("500")) > 0)
            .forEach(p -> System.out.println("Expensive: " + p.getName()));
    }
}
```

Important: Streams from repositories must be used within a `@Transactional` context and should be closed (use try-with-resources).

Count and Exists Methods

Count

```java
// Count by field value
long countByCategory(Product.Category category);
// SQL: SELECT COUNT(*) FROM products WHERE category = ?

long countByActiveTrue();
// SQL: SELECT COUNT(*) FROM products WHERE active = true

long countByPriceGreaterThan(BigDecimal price);
// SQL: SELECT COUNT(*) FROM products WHERE price > ?

long countByBrandAndCategory(String brand, Product.Category category);
// SQL: SELECT COUNT(*) FROM products WHERE brand = ? AND category = ?
```

Exists

```java
// Check existence
boolean existsBySku(String sku);
// SQL: SELECT COUNT(*) FROM products WHERE sku = ?
// Returns true if count > 0

boolean existsByNameContaining(String keyword);
// SQL: SELECT COUNT(*) FROM products WHERE name LIKE '%keyword%'

boolean existsByCategoryAndActiveTrue(Product.Category category);
// SQL: SELECT COUNT(*) FROM products WHERE category = ? AND active = true
```

Delete Methods

Derived Delete Methods

```java
// Delete by field value
void deleteByCategory(Product.Category category);
// SQL: SELECT ... WHERE category = ?   (loads entities)
//      DELETE FROM products WHERE id = ?   (per entity)

long deleteByActiveFalse();
// Returns the count of deleted entities
// SQL: Same as above — loads then deletes individually

void deleteByBrand(String brand);
// SQL: loads by brand, deletes each individually

// With @Transactional (required for delete methods)
@Transactional
public void removeInactiveProducts() {
    long count = productRepository.deleteByActiveFalse();
    System.out.println("Deleted " + count + " inactive products");
}
```

How Derived Deletes Work

```
Derived delete flow:
+-------------------------------------------------------------------+
|  deleteByCategory(ELECTRONICS)                                     |
|    |                                                               |
|    v                                                               |
|  1. SELECT ... FROM products WHERE category = 'ELECTRONICS'        |
|     (loads ALL matching entities into persistence context)         |
|    |                                                               |
|    v                                                               |
|  2. For EACH entity:                                               |
|     - @PreRemove callback fires                                    |
|     - entityManager.remove(entity)                                 |
|     - DELETE FROM products WHERE id = ?                            |
|     - @PostRemove callback fires                                   |
|    |                                                               |
|    v                                                               |
|  Result: N SELECT + N DELETE statements                            |
|  (not efficient for bulk deletes!)                                 |
+-------------------------------------------------------------------+
```

Important: Derived delete methods load entities first and delete them one by one. For bulk deletes, use `@Query` with `@Modifying` (covered below) or `deleteInBatch()`.

@Modifying for Bulk Updates and Deletes

When you need to update or delete many rows efficiently (without loading entities), use `@Query` with `@Modifying`:

```java
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    // Bulk UPDATE - single SQL statement, no entity loading
    @Modifying
    @Query("UPDATE Product p SET p.active = false WHERE p.category = :category")
    int deactivateByCategory(@Param("category") Product.Category category);
    // Returns the number of rows updated

    // Bulk DELETE - single SQL statement, no entity loading
    @Modifying
    @Query("DELETE FROM Product p WHERE p.active = false")
    int deleteInactiveProducts();
    // Returns the number of rows deleted

    // Bulk UPDATE with multiple fields
    @Modifying
    @Query("UPDATE Product p SET p.price = p.price * :multiplier " +
           "WHERE p.category = :category")
    int adjustPriceByCategory(@Param("multiplier") BigDecimal multiplier,
                              @Param("category") Product.Category category);
}
```

```
Derived delete vs @Modifying delete:
+-----------------------------------------------------------------+
|  Derived: deleteByActiveFalse()                                  |
|    --> SELECT all inactive products (loads into memory)           |
|    --> DELETE each one individually                               |
|    --> N+1 SQL statements                                        |
|    --> Lifecycle callbacks fire                                   |
|                                                                  |
|  @Modifying: @Query("DELETE FROM Product WHERE active = false")   |
|    --> Single DELETE statement                                   |
|    --> No entities loaded into memory                             |
|    --> 1 SQL statement                                           |
|    --> Lifecycle callbacks do NOT fire                            |
+-----------------------------------------------------------------+
```

@Modifying with clearAutomatically

After a `@Modifying` query, the persistence context might contain stale data (entities that were updated in the database but not in memory). Use `clearAutomatically = true` to clear the cache:

```java
@Modifying(clearAutomatically = true)
@Query("UPDATE Product p SET p.price = p.price * 1.1 WHERE p.category = :cat")
int applyPriceIncrease(@Param("cat") Product.Category category);
// After this runs, all managed entities are cleared from the cache
// Next findById will load fresh data from the database
```

```
Without clearAutomatically:
+-------------------------------------------------------------------+
|  Product p = findById(1L);      // price = 100.00 (cached)        |
|  applyPriceIncrease(ELECTRONICS); // DB: price = 110.00           |
|  Product p2 = findById(1L);    // price = 100.00 (STALE CACHE!)  |
+-------------------------------------------------------------------+

With clearAutomatically = true:
+-------------------------------------------------------------------+
|  Product p = findById(1L);      // price = 100.00 (cached)        |
|  applyPriceIncrease(ELECTRONICS); // DB: price = 110.00           |
|  // Persistence context cleared automatically                     |
|  Product p2 = findById(1L);    // price = 110.00 (fresh from DB) |
+-------------------------------------------------------------------+
```

Complete Repository Example

Here is a comprehensive repository showcasing all the query method patterns:

```java
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    // === Exact Match ===
    Optional<Product> findBySku(String sku);
    List<Product> findByCategory(Product.Category category);
    List<Product> findByBrand(String brand);
    List<Product> findByActiveTrue();
    List<Product> findByActiveFalse();

    // === Comparisons ===
    List<Product> findByPriceGreaterThan(BigDecimal price);
```

```
    List<Product> findByPriceLessThanEqual(BigDecimal price);
    List<Product> findByPriceBetween(BigDecimal min, BigDecimal max);
    List<Product> findByStockQuantityLessThan(int quantity);
    List<Product> findByLaunchDateAfter(LocalDate date);
    List<Product> findByLaunchDateBetween(LocalDate start, LocalDate end);

    // === String Matching ===
    List<Product> findByNameContaining(String keyword);
    List<Product> findByNameContainingIgnoreCase(String keyword);
    List<Product> findByNameStartingWith(String prefix);
    List<Product> findByBrandIgnoreCase(String brand);

    // === Null Checks ===
    List<Product> findByBrandIsNull();
    List<Product> findByBrandIsNotNull();

    // === Collections ===
    List<Product> findByCategoryIn(Collection<Product.Category> categories);
    List<Product> findByBrandIn(Collection<String> brands);

    // === Combined Conditions ===
    List<Product> findByCategoryAndActiveTrue(Product.Category category);
    List<Product> findByBrandAndPriceLessThan(String brand, BigDecimal price);
    List<Product> findByCategoryAndPriceBetween(
        Product.Category category, BigDecimal min, BigDecimal max);

    // === Ordering ===
    List<Product> findByCategoryOrderByPriceAsc(Product.Category category);
    List<Product> findByActiveTrueOrderByLaunchDateDesc();
    List<Product> findByCategory(Product.Category category, Sort sort);

    // === Limiting ===
    Optional<Product> findFirstByCategoryOrderByPriceAsc(Product.Category cat);
    List<Product> findTop5ByCategoryOrderByPriceDesc(Product.Category cat);
    List<Product> findFirst10ByOrderByLaunchDateDesc();

    // === Count and Exists ===
    long countByCategory(Product.Category category);
    long countByActiveTrue();
    boolean existsBySku(String sku);
    boolean existsByCategoryAndPriceGreaterThan(
        Product.Category category, BigDecimal price);

    // === Derived Deletes ===
    @Transactional
    long deleteByActiveFalse();

    @Transactional
    void deleteByCategory(Product.Category category);

    // === Bulk Operations with @Modifying ===
    @Modifying(clearAutomatically = true)
    @Query("UPDATE Product p SET p.active = false WHERE p.category = :category")
    int deactivateByCategory(@Param("category") Product.Category category);
```

```
    @Modifying(clearAutomatically = true)
    @Query("DELETE FROM Product p WHERE p.active = false AND p.stockQuantity = 0")
    int removeDeadStock();
}
```

Testing Derived Queries

Here is a DataLoader that tests various query methods:

```java
@Component
public class QueryDemoRunner implements CommandLineRunner {

    private final ProductRepository repository;

    public QueryDemoRunner(ProductRepository repository) {
        this.repository = repository;
    }

    @Override
    public void run(String... args) {
        seedData();
        runQueries();
    }

    private void seedData() {
        repository.saveAll(List.of(
            new Product("Laptop Pro 15", "LAP-001", new BigDecimal("1299.99"),
                Product.Category.ELECTRONICS, 50, "Apple"),
            new Product("Wireless Mouse", "MOU-001", new BigDecimal("29.99"),
                Product.Category.ELECTRONICS, 200, "Logitech"),
            new Product("Java Programming", "BOK-001", new BigDecimal("49.99"),
                Product.Category.BOOKS, 100, null),
            new Product("Running Shoes", "SHO-001", new BigDecimal("89.99"),
                Product.Category.SPORTS, 75, "Nike"),
            new Product("Cotton T-Shirt", "TSH-001", new BigDecimal("19.99"),
                Product.Category.CLOTHING, 300, "Uniqlo"),
            new Product("Bluetooth Speaker", "SPK-001", new BigDecimal("79.99"),
                Product.Category.ELECTRONICS, 0, "Sony"),
            new Product("Cookbook Basics", "BOK-002", new BigDecimal("34.99"),
                Product.Category.BOOKS, 60, null),
            new Product("Laptop Stand", "STD-001", new BigDecimal("45.99"),
                Product.Category.ELECTRONICS, 150, "Amazon")
        ));
    }

    private void runQueries() {
```

```
        System.out.println("\n=== Derived Query Demonstrations ===\n");

        // Exact match
        System.out.println("--- Electronics ---");
        repository.findByCategory(Product.Category.ELECTRONICS)
            .forEach(p -> System.out.println("  " + p.getName() + " - $" + p.getPrice()));

        // Price range
        System.out.println("\n--- Price $30-$100 ---");
        repository.findByPriceBetween(new BigDecimal("30"), new BigDecimal("100"))
            .forEach(p -> System.out.println("  " + p.getName() + " - $" + p.getPrice()));

        // String search
        System.out.println("\n--- Name containing 'Laptop' ---");
        repository.findByNameContainingIgnoreCase("laptop")
            .forEach(p -> System.out.println("  " + p.getName()));

        // Null check
        System.out.println("\n--- No brand ---");
        repository.findByBrandIsNull()
            .forEach(p -> System.out.println("  " + p.getName()));

        // Combined conditions
        System.out.println("\n--- Electronics under $100 ---");
        repository.findByBrandAndPriceLessThan("Apple", new BigDecimal("2000"))
            .forEach(p -> System.out.println("  " + p.getName() + " - $" + p.getPrice()));

        // Top N
        System.out.println("\n--- Top 3 most expensive electronics ---");
        repository.findTop5ByCategoryOrderByPriceDesc(Product.Category.ELECTRONICS)
            .forEach(p -> System.out.println("  " + p.getName() + " - $" + p.getPrice()));

        // Cheapest in category
        System.out.println("\n--- Cheapest electronic ---");
        repository.findFirstByCategoryOrderByPriceAsc(Product.Category.ELECTRONICS)
            .ifPresent(p -> System.out.println("  " + p.getName() + " - $" + p.getPrice...

        // Count
        System.out.println("\n--- Counts ---");
        System.out.println("  Electronics: " + repository.countByCategory(Product.Categ...
        System.out.println("  Books: " + repository.countByCategory(Product.Category.BO...
        System.out.println("  Active: " + repository.countByActiveTrue());

        // Exists
        System.out.println("\n--- Exists ---");
        System.out.println("  SKU LAP-001: " + repository.existsBySku("LAP-001"));
        System.out.println("  SKU XXX-999: " + repository.existsBySku("XXX-999"));

        // Sort parameter
        System.out.println("\n--- Electronics sorted by price ascending ---");
        repository.findByCategory(Product.Category.ELECTRONICS,
                Sort.by("price").ascending())
            .forEach(p -> System.out.println("  " + p.getName() + " - $" + p.getPrice()));
    }
}
```

Complete Keyword Reference

```
Keyword              Example                          SQL Clause
----------------------------------------------------------------------
And                  findByNameAndPrice               WHERE x=? AND y=?
Or                   findByNameOrPrice                WHERE x=? OR y=?
Is, Equals           findByName, findByNameIs         WHERE x = ?
Not, IsNot           findByNameNot                    WHERE x <> ?
Between              findByPriceBetween               WHERE x BETWEEN ? AND ?
LessThan             findByPriceLessThan              WHERE x < ?
LessThanEqual        findByPriceLessThanEqual         WHERE x <= ?
GreaterThan          findByPriceGreaterThan           WHERE x > ?
GreaterThanEqual     findByPriceGreaterThanEqual      WHERE x >= ?
After                findByDateAfter                  WHERE x > ?
Before               findByDateBefore                 WHERE x < ?
IsNull, Null         findByBrandIsNull                WHERE x IS NULL
IsNotNull, NotNull   findByBrandIsNotNull             WHERE x IS NOT NULL
Like                 findByNameLike                   WHERE x LIKE ?
NotLike              findByNameNotLike                WHERE x NOT LIKE ?
Containing           findByNameContaining             WHERE x LIKE '%?%'
StartingWith         findByNameStartingWith           WHERE x LIKE '?%'
EndingWith           findByNameEndingWith             WHERE x LIKE '%?'
IgnoreCase           findByNameIgnoreCase             WHERE UPPER(x)=UPPER(?)
In                   findByCategoryIn                 WHERE x IN (?)
NotIn                findByCategoryNotIn              WHERE x NOT IN (?)
True                 findByActiveTrue                 WHERE x = true
False                findByActiveFalse                WHERE x = false
OrderBy              ...OrderByNameAsc                ORDER BY x ASC
Not                  findByNameNot                    WHERE x <> ?
First, Top           findFirst3By...                  LIMIT 3
```

Debugging Derived Queries

Common Error: Property Not Found

```
PropertyReferenceException:
No property 'emailAddress' found for type 'Product'
```

This means the field name in your method does not match the entity field name. Check for typos:

```java
// Entity has field: private String email;

// WRONG: emailAddress does not match
List<Product> findByEmailAddress(String email);

// CORRECT: matches the field name
List<Product> findByEmail(String email);
```

Common Error: Ambiguous Property Path

If your entity has nested objects (covered in later chapters), Spring Data might misparse the method name:

```java
// Entity has: private String orderDate;
// AND also:   private Order order; (with field date inside)

// Spring might parse "OrderDate" as "order.date" instead of "orderDate"
List<Product> findByOrderDate(LocalDate date);

// Fix: Use underscores to force correct parsing
List<Product> findByOrder_Date(LocalDate date);  // order.date
// vs
List<Product> findByOrderDate(LocalDate date);    // orderDate field
```

Verifying Generated SQL

Always check the SQL generated by your derived queries. Enable SQL logging:

```properties
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Then call the method and verify the SQL in the console matches your intent.

Common Mistakes

1. Method name does not match field name: `findByUserName` fails if the field is `username` (one word). The method name is parsed based on camelCase — each capital letter

starts a new word.

2. Using Like without wildcards: `findByNameLike("Widget")` generates `WHERE name LIKE 'Widget'` (exact match, not a pattern). Use `findByNameContaining("Widget")` instead, which adds `%` automatically.
3. Expecting derived deletes to be efficient: `deleteByCategory(BOOKS)` loads all matching entities then deletes them one by one. For bulk deletes, use `@Modifying` with `@Query`.
4. Missing @Transactional on delete methods: Derived delete and `@Modifying` methods require a transaction. Either annotate the calling service method with `@Transactional` or put `@Transactional` on the repository method.
5. Forgetting clearAutomatically on @Modifying: After a bulk UPDATE, cached entities in the persistence context are stale. Use `@Modifying(clearAutomatically = true)` to avoid reading outdated data.
6. Method names that are too long: `findByCategoryAndActiveTrueAndPriceGreaterThanAndBrandNotNullOrderByPriceAsc` is technically valid but unreadable. Use `@Query` for complex queries.

Best Practices

1. Use Containing instead of Like: `findByNameContaining("Widget")` is cleaner than `findByNameLike("%Widget%")` because wildcards are handled automatically.
2. Use Sort parameter for flexible ordering: Instead of creating multiple methods with different OrderBy clauses, create one method that accepts a `Sort` parameter.
3. Return Optional for single-result queries: When a query should return zero or one result, use `Optional<T>`. This forces callers to handle the empty case.
4. Use @Modifying for bulk operations: When updating or deleting many rows, `@Modifying` with `@Query` is much more efficient than derived methods.
5. Keep method names readable: If a method name exceeds ~60 characters, switch to `@Query` with JPQL (Chapter 8). Readability matters more than avoiding SQL.
6. Use IgnoreCase for user-facing searches: When users search by name or title, they do not expect case-sensitive matching. Add `IgnoreCase` to string search methods.

Summary

In this chapter, you learned how to write queries simply by naming methods:

- Derived query methods are parsed from method names into JPQL and SQL. Spring Data JPA supports dozens of keywords for building queries without writing any query language.
- Comparison keywords (GreaterThan, LessThan, Between, After, Before) handle numeric and date comparisons.
- String keywords (Containing, StartingWith, EndingWith, Like, IgnoreCase) handle text searching with automatic wildcard management.
- Logical operators (And, Or) combine multiple conditions. For complex logic, prefer `@Query`.
- Result limiting (First, Top, Top3, Top10) returns a subset of results, useful for "most recent" or "top N" queries.
- Return types include List, Optional, Stream, long (count), boolean (exists), and void (deletes).
- Sort parameter allows flexible ordering without creating multiple methods.
- @Modifying enables efficient bulk UPDATE and DELETE operations with a single SQL statement.

Interview Questions

Q1: How does Spring Data JPA generate a query from a method name?

Spring Data parses the method name into components: the action prefix (find, count, exists, delete), the By keyword, field names, condition keywords (GreaterThan, Containing, etc.), logical operators (And, Or), and ordering (OrderBy). It maps field names to entity properties and keywords to JPQL/SQL clauses, then generates the query automatically at application startup.

Q2: What is the difference between findByNameLike and findByNameContaining?

`findByNameLike` expects the caller to provide the wildcard characters (e.g., `"%Widget%"`). `findByNameContaining` automatically wraps the parameter with `%` on both sides. Similarly, `StartingWith` adds `%` on the right, and `EndingWith` adds `%` on the left.

Q3: How do you limit query results in derived methods?

Use `First` or `Top` keywords. `findFirstByOrderByPriceAsc()` returns the single cheapest product. `findTop5ByOrderByPriceDesc()` returns the five most expensive. Without a number, it returns one result; with a number N, it returns up to N results.

Q4: What is the difference between a derived delete method and a @Modifying delete?

A derived delete method (`deleteByCategory(cat)`) loads all matching entities into the persistence context, fires lifecycle callbacks (@PreRemove), and deletes them one by one with individual DELETE statements. A `@Modifying @Query("DELETE FROM Product p WHERE p.category = :cat")` executes a single DELETE SQL statement, does not load entities, and does not fire lifecycle callbacks. @Modifying is much faster for bulk operations.

Q5: What does @Modifying(clearAutomatically = true) do?

After executing a bulk UPDATE or DELETE query, the persistence context may contain stale entities that no longer match the database state. `clearAutomatically = true` clears the entire persistence context after the query executes, ensuring subsequent queries fetch fresh data from the database.

Q6: When should you use a Sort parameter instead of OrderBy in the method name?

Use a Sort parameter when the sort order needs to be flexible — when different callers might want different orderings. Use OrderBy in the method name when the sort order is fixed and always the same. Sort parameters avoid creating multiple methods with different OrderBy clauses.

Q7: Can you combine And and Or in derived query methods?

Yes, but the behavior can be confusing because And takes precedence over Or (just like in SQL). For complex conditions mixing And and Or, the method name becomes long and hard to read. It is better to use `@Query` with JPQL for complex logic.

Q8: How do you debug a derived query method that is not working correctly?

Enable SQL logging (`spring.jpa.show-sql=true`) and parameter logging (`logging.level.org.hibernate.orm.jdbc.bind=TRACE`). Call the method and compare the generated SQL with your expected query. Common issues include: property name mismatches (check field names), wrong keyword usage (e.g., Like without wildcards), and ambiguous property paths in entities with nested objects.

Practice Exercises

Exercise 1: Employee Queries Create an `EmployeeRepository` with derived methods for:

- Find by department
- Find by salary greater than a given amount
- Find by name containing a keyword (case-insensitive)
- Find by department and salary between two values

- Find the top 3 highest-paid employees
- Count employees by department
- Check if an email exists

Test each method with sample data and verify the generated SQL.

Exercise 2: Keyword Exploration For the Product entity, write derived query methods that use each of these keywords at least once: Between, Containing, IgnoreCase, In, IsNull, True, OrderBy, First, And, Or. Verify each generates correct SQL.

Exercise 3: Sort Flexibility Create a method `findByActiveTrue(Sort sort)`. Call it with five different Sort configurations: by name ascending, by price descending, by stock ascending, by launch date descending, and by category ascending then price descending. Verify each produces the correct ORDER BY clause.

Exercise 4: Bulk Operations Create @Modifying methods to: (a) deactivate all products with stock = 0, (b) increase prices by 10% for a given category, (c) delete all inactive products. Compare the SQL generated versus equivalent derived methods.

Exercise 5: Search Feature Build a simple product search feature using derived methods. Create methods for:

- Search by name keyword (case-insensitive)
- Filter by category
- Filter by price range
- Filter by brand (in a list of brands)
- Combine: search by name AND category AND price range

Write a service method that calls each one with test data.

What Is Next?

In the next chapter, we will explore JPQL — The JPA Query Language. While derived query methods are powerful, they have limits — you cannot express joins, aggregate functions, subqueries, or complex conditions through method names alone. JPQL gives you full query power while remaining database-independent. You will learn SELECT, WHERE, JOIN, GROUP BY, aggregate functions, named parameters, and how to use `@Query` for custom queries that go beyond what method names can express.

Chapter 8: JPQL — The JPA Query Language

Learning Goals

By the end of this chapter, you will be able to:

- Write JPQL queries using SELECT, FROM, WHERE, JOIN, GROUP BY, HAVING, and ORDER BY
- Understand how JPQL differs from SQL
- Use named parameters (:name) and positional parameters (?1)
- Write aggregate queries with COUNT, SUM, AVG, MIN, MAX
- Use the @Query annotation in Spring Data JPA repositories
- Create @NamedQuery for reusable entity-level queries
- Perform JPQL UPDATE and DELETE operations
- Use constructor expressions to return DTOs directly from queries

What Is JPQL?

JPQL (Java Persistence Query Language) is a query language defined by the JPA specification. It looks similar to SQL, but with one fundamental difference: JPQL queries entities and their fields, not tables and columns.

```
+-----------------------------------------------------------------+
|  SQL vs JPQL                                                    |
|                                                                 |
|  SQL:   SELECT * FROM employees WHERE department = 'ENGINEERING'  |
|         ^queries tables and columns                             |
```

```
|                                                                    |
|  JPQL:  SELECT e FROM Employee e WHERE e.department = 'ENGINEERING'|
|         ^queries entities and fields                               |
|                                                                    |
|  Key differences:                                                  |
|  - SQL uses table names          JPQL uses entity class names      |
|  - SQL uses column names         JPQL uses field names             |
|  - SQL is database-specific      JPQL is database-independent      |
|  - SQL returns rows              JPQL returns entity objects       |
+--------------------------------------------------------------------+
```

Why JPQL Instead of SQL?

```
JPQL Advantages:
+--------------------------------------------------------------------+
|  1. Database-independent                                           |
|     Same query works on H2, PostgreSQL, MySQL, Oracle              |
|     Hibernate translates JPQL to the correct SQL dialect           |
|                                                                    |
|  2. Object-oriented                                                |
|     Queries reference entity fields, not columns                   |
|     Navigate relationships with dot notation: e.department.name    |
|                                                                    |
|  3. Type-safe with compile-time checking                           |
|     Field names are validated at startup                           |
|     Typos are caught early, not at runtime                         |
|                                                                    |
|  4. Automatic mapping                                              |
|     Results are automatically mapped to entity objects              |
|     No manual ResultSet parsing                                    |
+--------------------------------------------------------------------+
```

JPQL Syntax Overview

The Basic Structure

```
SELECT [what to return]
FROM [EntityName] [alias]
WHERE [conditions]
GROUP BY [fields]
HAVING [group conditions]
ORDER BY [fields] [ASC|DESC]
```

JPQL vs SQL Side-by-Side

```
JPQL                                    SQL Generated by Hibernate
---------------------------------------  ---------------------------------------
SELECT e                                 SELECT e.id, e.name, e.email,
FROM Employee e                                 e.department, e.salary,
                                                e.active
                                         FROM employees e

SELECT e                                 SELECT e.id, e.name, e.email,
FROM Employee e                                 e.department, e.salary,
WHERE e.department = :dept                      e.active
                                         FROM employees e
                                         WHERE e.department = ?

SELECT e.name, e.salary                  SELECT e.name, e.salary
FROM Employee e                          FROM employees e
WHERE e.salary > :min                    WHERE e.salary > ?
ORDER BY e.salary DESC                   ORDER BY e.salary DESC

SELECT COUNT(e)                          SELECT COUNT(e.id)
FROM Employee e                          FROM employees e
WHERE e.active = true                    WHERE e.active = true
```

Notice:

- `Employee` (JPQL) maps to `employees` (SQL table)
- `e.department` (JPQL field) maps to `e.department` (SQL column, via naming strategy)
- `SELECT e` returns the full entity; Hibernate expands it to all columns
- `:dept` (named parameter) becomes `?` in the actual SQL

Setup: Entities for This Chapter

We will use two related entities to demonstrate joins:

```java
@Entity
@Table(name = "departments")
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    @Column(nullable = false, unique = true, length = 50)
    private String name;

    @Column(length = 200)
    private String location;

    @OneToMany(mappedBy = "department")
    private List<Employee> employees = new ArrayList<>();

    public Department() {}
    public Department(String name, String location) {
        this.name = name;
        this.location = location;
    }

    // getters, setters...
}
```

```java
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(precision = 10, scale = 2)
    private BigDecimal salary;

    private boolean active;

    private LocalDate hireDate;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;

    public Employee() {}
    public Employee(String name, String email, BigDecimal salary,
                    boolean active, LocalDate hireDate) {
        this.name = name;
        this.email = email;
        this.salary = salary;
        this.active = active;
        this.hireDate = hireDate;
    }
```

```
    // getters, setters...
}
```

Note: We will cover `@ManyToOne` and `@OneToMany` in detail in Chapter 13. For now, just understand that an Employee belongs to a Department, and a Department has many Employees.

Using @Query in Spring Data JPA

The `@Query` annotation lets you write JPQL (or native SQL) directly on repository methods:

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // JPQL query using @Query
    @Query("SELECT e FROM Employee e WHERE e.salary > :minSalary")
    List<Employee> findHighEarners(@Param("minSalary") BigDecimal minSalary);

    // Equivalent derived method (for comparison):
    // List<Employee> findBySalaryGreaterThan(BigDecimal salary);
}
```

Named Parameters vs Positional Parameters

```java
// Named parameters - recommended (clear and self-documenting)
@Query("SELECT e FROM Employee e WHERE e.name = :name AND e.active = :active")
List<Employee> findByNameAndStatus(@Param("name") String name,
                                   @Param("active") boolean active);

// Positional parameters - shorter but less readable
@Query("SELECT e FROM Employee e WHERE e.name = ?1 AND e.active = ?2")
List<Employee> findByNameAndStatus(String name, boolean active);
```

```
Named vs Positional Parameters:
+-------------------------------------------------------------------+
|  Named (:name)                    Positional (?1)                 |
|  @Param("name") String name       String name  (position 1)      |
|                                                                   |
|  More readable                    Shorter                         |
|  Order does not matter            Order must match ?1, ?2, ?3     |
```

```
|  Self-documenting                 Fragile if params reordered     |
|  Recommended                      Acceptable for simple queries   |
+-------------------------------------------------------------------+
```

Best practice: Always use named parameters. They make queries readable and safe against parameter reordering.

SELECT Queries

Selecting Full Entities

```java
// Select all employees
@Query("SELECT e FROM Employee e")
List<Employee> findAllEmployees();

// With a condition
@Query("SELECT e FROM Employee e WHERE e.active = true")
List<Employee> findActiveEmployees();

// The alias 'e' is required — JPQL needs it for field references
```

Selecting Specific Fields

```java
// Select only name — returns List<String>
@Query("SELECT e.name FROM Employee e WHERE e.active = true")
List<String> findActiveEmployeeNames();

// Select two fields — returns List<Object[]>
@Query("SELECT e.name, e.salary FROM Employee e ORDER BY e.salary DESC")
List<Object[]> findNamesAndSalaries();
```

When selecting multiple fields, the return type is `Object[]`. You access fields by index:

```java
List<Object[]> results = repository.findNamesAndSalaries();
for (Object[] row : results) {
    String name = (String) row[0];
    BigDecimal salary = (BigDecimal) row[1];
    System.out.println(name + ": $" + salary);
}
```

This is awkward. A better approach is constructor expressions (covered later in this chapter).

DISTINCT

```java
// Remove duplicates
@Query("SELECT DISTINCT e.department FROM Employee e")
List<Department> findDistinctDepartments();

@Query("SELECT DISTINCT e.name FROM Employee e WHERE e.active = true")
List<String> findDistinctActiveNames();
```

WHERE Clause

Comparison Operators

```java
// Equals
@Query("SELECT e FROM Employee e WHERE e.name = :name")
List<Employee> findByExactName(@Param("name") String name);

// Not equals
@Query("SELECT e FROM Employee e WHERE e.department <> :dept")
List<Employee> findNotInDepartment(@Param("dept") Department dept);

// Greater than, less than
@Query("SELECT e FROM Employee e WHERE e.salary > :min AND e.salary < :max")
List<Employee> findBySalaryRange(@Param("min") BigDecimal min,
                                 @Param("max") BigDecimal max);

// BETWEEN
@Query("SELECT e FROM Employee e WHERE e.salary BETWEEN :min AND :max")
List<Employee> findBySalaryBetween(@Param("min") BigDecimal min,
                                   @Param("max") BigDecimal max);
```

String Operations

```java
// LIKE with wildcards
@Query("SELECT e FROM Employee e WHERE e.name LIKE :pattern")
List<Employee> findByNamePattern(@Param("pattern") String pattern);
// Usage: findByNamePattern("%Alice%")
```

```
// LIKE with CONCAT (add wildcards in the query)
@Query("SELECT e FROM Employee e WHERE e.name LIKE CONCAT('%', :keyword, '%')")
List<Employee> searchByName(@Param("keyword") String keyword);
// Usage: searchByName("Alice")  — no wildcards needed from caller

// UPPER / LOWER for case-insensitive
@Query("SELECT e FROM Employee e WHERE LOWER(e.name) LIKE LOWER(CONCAT('%', :keyword, '...
List<Employee> searchByNameIgnoreCase(@Param("keyword") String keyword);
```

NULL Checks

```java
@Query("SELECT e FROM Employee e WHERE e.department IS NULL")
List<Employee> findUnassignedEmployees();

@Query("SELECT e FROM Employee e WHERE e.department IS NOT NULL")
List<Employee> findAssignedEmployees();
```

IN Clause

```java
@Query("SELECT e FROM Employee e WHERE e.department.name IN :deptNames")
List<Employee> findByDepartmentNames(@Param("deptNames") List<String> deptNames);
// Usage: findByDepartmentNames(List.of("Engineering", "Sales"))
// SQL: WHERE d.name IN (?, ?)
```

Logical Operators

```java
// AND
@Query("SELECT e FROM Employee e WHERE e.active = true AND e.salary > :min")
List<Employee> findActiveHighEarners(@Param("min") BigDecimal min);

// OR
@Query("SELECT e FROM Employee e WHERE e.department.name = :dept1 OR e.department.name ...
List<Employee> findInEitherDepartment(@Param("dept1") String dept1,
                                      @Param("dept2") String dept2);

// NOT
@Query("SELECT e FROM Employee e WHERE NOT (e.active = true AND e.salary < :max)")
List<Employee> findNotActiveLowEarners(@Param("max") BigDecimal max);

// Complex conditions with parentheses
@Query("SELECT e FROM Employee e WHERE " +
       "(e.department.name = :dept AND e.salary > :min) OR " +
```

```
        "(e.active = false)")
List<Employee> findComplexCondition(@Param("dept") String dept,
                                    @Param("min") BigDecimal min);
```

JOIN Queries

JPQL supports implicit joins (via dot navigation) and explicit joins.

Implicit Join (Dot Navigation)

```java
// Navigate from Employee to Department using dot notation
@Query("SELECT e FROM Employee e WHERE e.department.name = :deptName")
List<Employee> findByDepartmentName(@Param("deptName") String deptName);
// Hibernate generates: SELECT ... FROM employees e
//                      JOIN departments d ON e.department_id = d.id
//                      WHERE d.name = ?
```

Hibernate automatically generates the JOIN when you navigate a relationship with dot notation. This is convenient but gives you less control.

Explicit JOIN

```java
// Explicit inner join
@Query("SELECT e FROM Employee e JOIN e.department d WHERE d.name = :deptName")
List<Employee> findByDepartmentNameWithJoin(@Param("deptName") String deptName);

// Explicit inner join selecting from both entities
@Query("SELECT e.name, d.name FROM Employee e JOIN e.department d")
List<Object[]> findEmployeeAndDepartmentNames();
```

LEFT JOIN

```java
// Left join — includes employees without a department
@Query("SELECT e FROM Employee e LEFT JOIN e.department d")
List<Employee> findAllWithOptionalDepartment();

// Left join with condition
```

```
@Query("SELECT e FROM Employee e LEFT JOIN e.department d " +
       "WHERE d.name = :dept OR d IS NULL")
List<Employee> findByDeptOrUnassigned(@Param("dept") String dept);
```

JOIN FETCH (Performance Optimization)

`JOIN FETCH` loads the related entity in the same query, avoiding the N+1 problem:

```java
// Without JOIN FETCH — N+1 problem:
// Query 1: SELECT * FROM employees
// Query 2: SELECT * FROM departments WHERE id = 1  (for emp1)
// Query 3: SELECT * FROM departments WHERE id = 2  (for emp2)
// ... N additional queries for N employees!

// With JOIN FETCH — single query:
@Query("SELECT e FROM Employee e JOIN FETCH e.department")
List<Employee> findAllWithDepartment();
// Generates: SELECT e.*, d.* FROM employees e
//            JOIN departments d ON e.department_id = d.id
// ONE query loads everything!
```

```
N+1 Problem vs JOIN FETCH:
+-------------------------------------------------------------------+
|  Without JOIN FETCH:                                               |
|  Query 1: SELECT * FROM employees           (returns 100 emps)    |
|  Query 2: SELECT * FROM departments WHERE id=1                    |
|  Query 3: SELECT * FROM departments WHERE id=2                    |
|  ...                                                               |
|  Query 101: SELECT * FROM departments WHERE id=100                |
|  Total: 101 queries!                                               |
|                                                                    |
|  With JOIN FETCH:                                                  |
|  Query 1: SELECT e.*, d.* FROM employees e                        |
|           JOIN departments d ON e.department_id = d.id             |
|  Total: 1 query!                                                   |
+-------------------------------------------------------------------+
```

We will cover the N+1 problem in much greater depth in Chapter 18.

LEFT JOIN FETCH

```java
// Left join fetch — includes employees without departments
@Query("SELECT e FROM Employee e LEFT JOIN FETCH e.department")
List<Employee> findAllWithOptionalDepartment();
```

Aggregate Functions

JPQL supports the standard SQL aggregate functions:

```
Function     Purpose                Example
-------------------------------------------------------
COUNT        Count rows             COUNT(e)
SUM          Sum values             SUM(e.salary)
AVG          Average value          AVG(e.salary)
MIN          Minimum value          MIN(e.salary)
MAX          Maximum value          MAX(e.salary)
```

Basic Aggregations

```java
// Count all employees
@Query("SELECT COUNT(e) FROM Employee e")
long countAll();

// Count active employees
@Query("SELECT COUNT(e) FROM Employee e WHERE e.active = true")
long countActive();

// Sum of salaries
@Query("SELECT SUM(e.salary) FROM Employee e WHERE e.active = true")
BigDecimal totalActiveSalary();

// Average salary
@Query("SELECT AVG(e.salary) FROM Employee e")
Double averageSalary();

// Min and Max salary
@Query("SELECT MIN(e.salary) FROM Employee e")
BigDecimal minSalary();

@Query("SELECT MAX(e.salary) FROM Employee e")
BigDecimal maxSalary();

// Multiple aggregates in one query
@Query("SELECT COUNT(e), AVG(e.salary), MIN(e.salary), MAX(e.salary) " +
       "FROM Employee e WHERE e.active = true")
Object[] getSalaryStats();
// Access: Object[] stats = repository.getSalaryStats();
// long count = (long) stats[0];
// double avg = (double) stats[1];
// BigDecimal min = (BigDecimal) stats[2];
// BigDecimal max = (BigDecimal) stats[3];
```

GROUP BY and HAVING

GROUP BY

```java
// Count employees per department
@Query("SELECT d.name, COUNT(e) FROM Employee e " +
       "JOIN e.department d GROUP BY d.name")
List<Object[]> countByDepartment();

// Average salary per department
@Query("SELECT d.name, AVG(e.salary) FROM Employee e " +
       "JOIN e.department d GROUP BY d.name ORDER BY AVG(e.salary) DESC")
List<Object[]> avgSalaryByDepartment();

// Usage:
List<Object[]> results = repository.countByDepartment();
for (Object[] row : results) {
    String deptName = (String) row[0];
    Long count = (Long) row[1];
    System.out.println(deptName + ": " + count + " employees");
}
```

HAVING (Filter Groups)

```java
// Departments with more than 5 employees
@Query("SELECT d.name, COUNT(e) FROM Employee e " +
       "JOIN e.department d GROUP BY d.name HAVING COUNT(e) > :minCount")
List<Object[]> largeDepartments(@Param("minCount") long minCount);

// Departments with average salary above a threshold
@Query("SELECT d.name, AVG(e.salary) FROM Employee e " +
       "JOIN e.department d GROUP BY d.name HAVING AVG(e.salary) > :minAvg")
List<Object[]> highPayingDepartments(@Param("minAvg") double minAvg);
```

```
GROUP BY + HAVING Flow:
+-------------------------------------------------------------------+
|  Data:                                                            |
|  Engineering: Alice(95K), Bob(105K), Charlie(88K)  = 3 employees  |
|  Marketing:   Diana(78K), Eve(82K)                 = 2 employees  |
|  Sales:       Frank(72K)                           = 1 employee   |
|                                                                   |
|  Query: SELECT d.name, COUNT(e), AVG(e.salary)                    |
```

```
|         FROM Employee e JOIN e.department d                        |
|         GROUP BY d.name                                            |
|         HAVING COUNT(e) >= 2                                       |
|                                                                   |
|  Result:                                                          |
|  +---------------+-------+----------+                             |
|  | Department    | Count | Avg Sal  |                             |
|  |---------------|-------|----------|                             |
|  | Engineering   | 3     | 96000.00 |                             |
|  | Marketing     | 2     | 80000.00 |                             |
|  +---------------+-------+----------+                             |
|  (Sales excluded — only 1 employee, fails HAVING COUNT >= 2)     |
+-------------------------------------------------------------------+
```

Constructor Expressions — Returning DTOs

Instead of returning `Object[]` for multi-field queries, you can use constructor expressions to return DTOs directly:

Step 1: Create a DTO

```java
package com.example.hibernatedemo.dto;

public class EmployeeSummary {

    private String name;
    private String departmentName;
    private BigDecimal salary;

    // Constructor must match the JPQL SELECT fields exactly
    public EmployeeSummary(String name, String departmentName, BigDecimal salary) {
        this.name = name;
        this.departmentName = departmentName;
        this.salary = salary;
    }

    // Getters
    public String getName() { return name; }
    public String getDepartmentName() { return departmentName; }
    public BigDecimal getSalary() { return salary; }

    @Override
    public String toString() {
        return name + " (" + departmentName + ") — $" + salary;
    }
}
```

Step 2: Use Constructor Expression in JPQL

```java
@Query("SELECT new com.example.hibernatedemo.dto.EmployeeSummary(" +
       "e.name, d.name, e.salary) " +
       "FROM Employee e JOIN e.department d " +
       "WHERE e.active = true ORDER BY e.salary DESC")
List<EmployeeSummary> findActiveEmployeeSummaries();
```

The `new com.example...EmployeeSummary(e.name, d.name, e.salary)` tells JPQL to create a DTO instance for each row, passing the selected fields to the constructor.

```
Constructor Expression vs Object[]:
+-----------------------------------------------------------------+
|  Object[] (fragile):                                            |
|  Object[] row = results.get(0);                                 |
|  String name = (String) row[0];        // index-based, fragile    |
|  String dept = (String) row[1];        // wrong index = ClassCast |
|  BigDecimal sal = (BigDecimal) row[2]; // no compile-time safety  |
|                                                                 |
|  Constructor Expression (type-safe):                            |
|  EmployeeSummary summary = results.get(0);                      |
|  String name = summary.getName();      // type-safe             |
|  String dept = summary.getDepartmentName(); // self-documenting   |
|  BigDecimal sal = summary.getSalary(); // compile-time checked    |
+-----------------------------------------------------------------+
```

More DTO Examples

```java
// Department statistics DTO
public class DepartmentStats {
    private String departmentName;
    private long employeeCount;
    private double averageSalary;

    public DepartmentStats(String departmentName, long employeeCount,
                           double averageSalary) {
        this.departmentName = departmentName;
        this.employeeCount = employeeCount;
        this.averageSalary = averageSalary;
    }
    // getters...
}

// Query returning the DTO
```

```
@Query("SELECT new com.example.hibernatedemo.dto.DepartmentStats(" +
       "d.name, COUNT(e), AVG(e.salary)) " +
       "FROM Employee e JOIN e.department d " +
       "GROUP BY d.name ORDER BY COUNT(e) DESC")
List<DepartmentStats> getDepartmentStatistics();
```

@NamedQuery — Reusable Entity-Level Queries

`@NamedQuery` defines queries on the entity class itself. They are validated at application startup (typos are caught early):

```java
@Entity
@Table(name = "employees")
@NamedQuery(
    name = "Employee.findByDepartmentName",
    query = "SELECT e FROM Employee e JOIN e.department d WHERE d.name = :deptName"
)
@NamedQuery(
    name = "Employee.findActiveHighEarners",
    query = "SELECT e FROM Employee e WHERE e.active = true AND e.salary > :minSalary O...
)
@NamedQuery(
    name = "Employee.countByDepartment",
    query = "SELECT d.name, COUNT(e) FROM Employee e JOIN e.department d GROUP BY d.name"
)
public class Employee {
    // ... entity fields
}
```

Using @NamedQuery in Repositories

Spring Data JPA automatically resolves named queries if they follow the convention `EntityName.methodName`:

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Matches @NamedQuery(name = "Employee.findByDepartmentName")
    List<Employee> findByDepartmentName(@Param("deptName") String deptName);

    // Matches @NamedQuery(name = "Employee.findActiveHighEarners")
    List<Employee> findActiveHighEarners(@Param("minSalary") BigDecimal minSalary);
}
```

```
@NamedQuery vs @Query:
+-------------------------------------------------------------------+
|  @NamedQuery                        @Query                         |
|  (on Entity class)                  (on Repository method)         |
|                                                                    |
|  Validated at startup               Validated at startup            |
|  Defined on the entity              Defined on the repository      |
|  Reusable across repositories       Tied to one method             |
|  Query separate from method         Query next to method           |
|  Convention: Entity.methodName       Explicit annotation            |
|                                                                    |
|  Use when: query is reused          Use when: query is specific    |
|  across multiple repositories       to one repository method       |
+-------------------------------------------------------------------+
```

Best practice: Use `@Query` on repository methods for most cases — it keeps the query close to where it is used. Use `@NamedQuery` when the same query is needed in multiple places or you want centralized query management on the entity.

UPDATE and DELETE in JPQL

JPQL supports bulk UPDATE and DELETE operations that execute as single SQL statements:

Bulk UPDATE

```java
@Modifying
@Query("UPDATE Employee e SET e.salary = e.salary * :multiplier " +
       "WHERE e.department.name = :dept")
int adjustSalariesByDepartment(@Param("multiplier") BigDecimal multiplier,
                               @Param("dept") String dept);

@Modifying
@Query("UPDATE Employee e SET e.active = false WHERE e.hireDate < :cutoff")
int deactivateBefore(@Param("cutoff") LocalDate cutoff);

@Modifying(clearAutomatically = true)
@Query("UPDATE Employee e SET e.department = :newDept " +
       "WHERE e.department = :oldDept")
int transferDepartment(@Param("oldDept") Department oldDept,
                       @Param("newDept") Department newDept);
```

Bulk DELETE

```java
@Modifying
@Query("DELETE FROM Employee e WHERE e.active = false")
int deleteInactiveEmployees();

@Modifying
@Query("DELETE FROM Employee e WHERE e.hireDate < :cutoff AND e.active = false")
int purgeOldInactiveEmployees(@Param("cutoff") LocalDate cutoff);
```

Important Rules for @Modifying

```
+-----------------------------------------------------------------+
|  @Modifying Rules                                                |
|                                                                  |
|  1. Must return int or void                                      |
|     int = number of rows affected                                |
|                                                                  |
|  2. Must be in a @Transactional context                          |
|     Either on the repository method or the calling service       |
|                                                                  |
|  3. Does NOT trigger lifecycle callbacks                         |
|     @PreUpdate, @PreRemove do NOT fire                           |
|                                                                  |
|  4. Does NOT update the persistence context                      |
|     Use clearAutomatically = true to avoid stale data            |
|                                                                  |
|  5. Uses JPQL syntax (entity/field names, not table/column)      |
|     UPDATE Employee e SET e.salary = ... (not UPDATE employees)  |
+-----------------------------------------------------------------+
```

JPQL Functions

JPQL provides built-in functions for string manipulation, math, and dates:

String Functions

```java
// CONCAT
@Query("SELECT CONCAT(e.name, ' (', d.name, ')') FROM Employee e JOIN e.department d")
List<String> findFormattedNames();
```

```
// LENGTH
@Query("SELECT e FROM Employee e WHERE LENGTH(e.name) > :minLength")
List<Employee> findLongNames(@Param("minLength") int minLength);

// UPPER, LOWER
@Query("SELECT e FROM Employee e WHERE UPPER(e.name) = UPPER(:name)")
List<Employee> findByNameCaseInsensitive(@Param("name") String name);

// SUBSTRING
@Query("SELECT SUBSTRING(e.email, 1, LOCATE('@', e.email) - 1) FROM Employee e")
List<String> findEmailUsernames();

// TRIM
@Query("SELECT e FROM Employee e WHERE TRIM(e.name) = :name")
List<Employee> findByTrimmedName(@Param("name") String name);

// LOCATE (find position of substring)
@Query("SELECT e FROM Employee e WHERE LOCATE('admin', e.email) > 0")
List<Employee> findAdminEmails();
```

Math Functions

```java
// ABS
@Query("SELECT e FROM Employee e WHERE ABS(e.salary - :target) < :tolerance")
List<Employee> findBySalaryNear(@Param("target") BigDecimal target,
                                @Param("tolerance") BigDecimal tolerance);

// MOD
@Query("SELECT e FROM Employee e WHERE MOD(e.id, 2) = 0")
List<Employee> findWithEvenIds();

// SIZE (collection size)
@Query("SELECT d FROM Department d WHERE SIZE(d.employees) > :minSize")
List<Department> findLargeDepartments(@Param("minSize") int minSize);
```

Date Functions

```java
// CURRENT_DATE, CURRENT_TIME, CURRENT_TIMESTAMP
@Query("SELECT e FROM Employee e WHERE e.hireDate < CURRENT_DATE")
List<Employee> findHiredBeforeToday();

// YEAR, MONTH, DAY (JPA 3.1+)
@Query("SELECT e FROM Employee e WHERE YEAR(e.hireDate) = :year")
List<Employee> findHiredInYear(@Param("year") int year);
```

Function Reference

```
Category     Function                Purpose
-------------------------------------------------------------
String       CONCAT(s1, s2)          Concatenate strings
             LENGTH(s)               String length
             UPPER(s), LOWER(s)      Case conversion
             SUBSTRING(s, pos, len)  Extract substring
             TRIM(s)                 Remove whitespace
             LOCATE(sub, s)          Find substring position

Math         ABS(n)                  Absolute value
             MOD(n, d)               Modulo (remainder)
             SQRT(n)                 Square root
             SIZE(collection)        Collection size

Date         CURRENT_DATE            Today's date
             CURRENT_TIME            Current time
             CURRENT_TIMESTAMP       Current date+time
             YEAR(d), MONTH(d)       Extract date parts
             DAY(d)                  (JPA 3.1+)

Conditional  CASE WHEN ... THEN      Conditional expressions
             COALESCE(a, b)          First non-null value
             NULLIF(a, b)            NULL if equal
```

CASE Expressions

JPQL supports conditional expressions similar to SQL CASE:

```java
// Simple CASE
@Query("SELECT e.name, " +
       "CASE e.active WHEN true THEN 'Active' ELSE 'Inactive' END " +
       "FROM Employee e")
List<Object[]> findWithStatusLabel();

// Searched CASE (with conditions)
@Query("SELECT e.name, " +
       "CASE WHEN e.salary >= 100000 THEN 'Senior' " +
       "     WHEN e.salary >= 70000 THEN 'Mid' " +
       "     ELSE 'Junior' END " +
       "FROM Employee e ORDER BY e.salary DESC")
List<Object[]> findWithSalaryBand();

// CASE in WHERE clause
```

```
@Query("SELECT e FROM Employee e WHERE " +
       "CASE WHEN e.active = true THEN e.salary ELSE 0 END > :min")
List<Employee> findActiveAboveMinSalary(@Param("min") BigDecimal min);
```

COALESCE and NULLIF

```java
// COALESCE - returns first non-null value
@Query("SELECT e.name, COALESCE(d.name, 'Unassigned') " +
       "FROM Employee e LEFT JOIN e.department d")
List<Object[]> findWithDepartmentOrDefault();

// NULLIF - returns null if values are equal
@Query("SELECT e.name, NULLIF(e.salary, 0) FROM Employee e")
List<Object[]> findSalariesNullIfZero();
```

Subqueries

JPQL supports subqueries in WHERE and HAVING clauses:

```java
// Employees with salary above the average
@Query("SELECT e FROM Employee e WHERE e.salary > " +
       "(SELECT AVG(e2.salary) FROM Employee e2)")
List<Employee> findAboveAverageSalary();

// Employees in the largest department
@Query("SELECT e FROM Employee e WHERE e.department = " +
       "(SELECT e2.department FROM Employee e2 " +
       " GROUP BY e2.department ORDER BY COUNT(e2) DESC LIMIT 1)")
List<Employee> findInLargestDepartment();

// Departments that have at least one employee earning above a threshold
@Query("SELECT d FROM Department d WHERE d IN " +
       "(SELECT e.department FROM Employee e WHERE e.salary > :min)")
List<Department> findDepartmentsWithHighEarners(@Param("min") BigDecimal min);

// EXISTS subquery
@Query("SELECT d FROM Department d WHERE EXISTS " +
       "(SELECT e FROM Employee e WHERE e.department = d AND e.active = true)")
List<Department> findDepartmentsWithActiveEmployees();
```

Complete Repository Example

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // === Basic Queries ===
    @Query("SELECT e FROM Employee e WHERE e.active = true ORDER BY e.name")
    List<Employee> findAllActive();

    @Query("SELECT e FROM Employee e WHERE e.salary BETWEEN :min AND :max")
    List<Employee> findBySalaryRange(@Param("min") BigDecimal min,
                                     @Param("max") BigDecimal max);

    // === String Search ===
    @Query("SELECT e FROM Employee e " +
           "WHERE LOWER(e.name) LIKE LOWER(CONCAT('%', :keyword, '%'))")
    List<Employee> searchByName(@Param("keyword") String keyword);

    // === Join Queries ===
    @Query("SELECT e FROM Employee e JOIN FETCH e.department " +
           "WHERE e.active = true")
    List<Employee> findActiveWithDepartment();

    @Query("SELECT e FROM Employee e JOIN e.department d " +
           "WHERE d.name = :dept AND e.salary > :min")
    List<Employee> findByDepartmentAndMinSalary(@Param("dept") String dept,
                                                @Param("min") BigDecimal min);

    // === Aggregate Queries ===
    @Query("SELECT AVG(e.salary) FROM Employee e WHERE e.active = true")
    Double findAverageActiveSalary();

    @Query("SELECT d.name, COUNT(e), AVG(e.salary) " +
           "FROM Employee e JOIN e.department d " +
           "GROUP BY d.name ORDER BY COUNT(e) DESC")
    List<Object[]> getDepartmentStats();

    // === Constructor Expression (DTO) ===
    @Query("SELECT new com.example.hibernatedemo.dto.EmployeeSummary(" +
           "e.name, d.name, e.salary) " +
           "FROM Employee e JOIN e.department d " +
           "ORDER BY e.salary DESC")
    List<EmployeeSummary> findEmployeeSummaries();

    // === Subquery ===
    @Query("SELECT e FROM Employee e WHERE e.salary > " +
           "(SELECT AVG(e2.salary) FROM Employee e2)")
    List<Employee> findAboveAverageSalary();

    // === Bulk Operations ===
    @Modifying(clearAutomatically = true)
    @Query("UPDATE Employee e SET e.salary = e.salary * :factor " +
```

```
            "WHERE e.department.name = :dept")
    int adjustSalaries(@Param("factor") BigDecimal factor,
                       @Param("dept") String dept);

    @Modifying
    @Query("DELETE FROM Employee e WHERE e.active = false " +
           "AND e.hireDate < :cutoff")
    int purgeOldInactive(@Param("cutoff") LocalDate cutoff);
}
```

Common Mistakes

1. Using table/column names instead of entity/field names: JPQL uses Java entity names and field names, not database names. `SELECT * FROM employees` is SQL. `SELECT e FROM Employee e` is JPQL.
2. Forgetting the alias: `SELECT FROM Employee WHERE active = true` fails. JPQL requires an alias: `SELECT e FROM Employee e WHERE e.active = true`.
3. Using SELECT * in JPQL: JPQL does not support `SELECT *`. Use `SELECT e` to select the full entity, or `SELECT e.name, e.email` for specific fields.
4. Missing @Modifying on UPDATE/DELETE queries: Without `@Modifying`, Spring Data expects a SELECT query and fails with an error.
5. Forgetting @Param for named parameters: Each `:paramName` in the query must have a matching `@Param("paramName")` on the method parameter.
6. Using JOIN FETCH with pagination: `JOIN FETCH` with `Pageable` causes Hibernate to apply pagination in memory (not in SQL), which loads all data first. Use `@EntityGraph` instead for paginated fetch joins.
7. Not using constructor expressions for multi-field results: Returning `List<Object[]>` is fragile and error-prone. Use constructor expressions (`new DTO(...)`) for type-safe results.

Best Practices

1. Use named parameters (:name) over positional (?1): They are more readable and safe against parameter reordering.
2. Use constructor expressions for projections: When you need only specific fields, return a DTO with `SELECT new DTO(...)` instead of `Object[]`.

3. Use JOIN FETCH to prevent N+1: When loading entities with relationships that you will access, use `JOIN FETCH` to load everything in one query.
4. Use @Modifying for bulk operations: Bulk UPDATE and DELETE are much faster than loading and modifying entities one by one.
5. Keep queries in @Query annotations: Placing JPQL directly on repository methods keeps the query close to its usage. Use `@NamedQuery` only when queries need to be shared.
6. Test your queries: Always check the generated SQL (via logging) to ensure it matches your intent, especially for complex joins and subqueries.

Summary

In this chapter, you mastered JPQL — the standard query language for JPA:

- JPQL queries entities and fields, not tables and columns. It is database-independent and object-oriented.
- @Query on repository methods lets you write custom JPQL when derived methods are insufficient.
- Named parameters (`:name` with `@Param`) are preferred over positional parameters (`?1`) for readability and safety.
- JOIN and JOIN FETCH navigate entity relationships. JOIN FETCH loads related entities in a single query, preventing the N+1 problem.
- Aggregate functions (COUNT, SUM, AVG, MIN, MAX) with GROUP BY and HAVING enable statistical queries.
- Constructor expressions (`SELECT new DTO(...)`) return type-safe DTOs instead of fragile `Object[]` arrays.
- @NamedQuery defines reusable queries on the entity class, validated at startup.
- Bulk UPDATE and DELETE with `@Modifying` execute single SQL statements without loading entities.
- JPQL functions (CONCAT, UPPER, LOWER, CASE, COALESCE) handle string manipulation, math, dates, and conditional logic.

Interview Questions

Q1: What is the difference between JPQL and SQL?

JPQL queries entity classes and their fields using Java names (e.g., `SELECT e FROM Employee e`). SQL queries database tables and columns (e.g., `SELECT * FROM employees`). JPQL is database-independent — Hibernate translates it to the appropriate SQL dialect. JPQL does not support `SELECT *`, requires aliases, and returns entity objects rather than raw rows.

Q2: What is a constructor expression in JPQL and why use it?

A constructor expression uses `SELECT new com.example.DTO(field1, field2)` to create DTO instances directly from query results. It provides type-safe results with proper field names instead of fragile `Object[]` arrays with index-based access. The DTO must have a constructor matching the selected fields exactly.

Q3: What is the difference between JOIN and JOIN FETCH in JPQL?

`JOIN` creates a SQL join and can filter/select based on the joined entity, but does not eagerly load the relationship. `JOIN FETCH` creates a SQL join AND loads the related entity data into the persistence context, preventing lazy loading and the N+1 problem. Use `JOIN` when you only need to filter; use `JOIN FETCH` when you will access the related entity's data.

Q4: How do named parameters differ from positional parameters?

Named parameters use `:paramName` syntax and require `@Param("paramName")` on method parameters. Positional parameters use `?1`, `?2` and match by method parameter order. Named parameters are more readable, self-documenting, and safe against parameter reordering. They are the recommended approach.

Q5: What does @Modifying do and what are its limitations?

`@Modifying` marks a `@Query` method as an UPDATE or DELETE operation. It executes a single SQL statement and returns the number of affected rows. Limitations: it does not trigger JPA lifecycle callbacks (@PreUpdate, @PreRemove), it does not update the persistence context (use `clearAutomatically = true`), and it requires a `@Transactional` context.

Q6: Can JPQL query across multiple entity relationships?

Yes. JPQL supports navigating relationships with dot notation (e.g., `e.department.name`) and explicit joins (`JOIN e.department d`). You can chain multiple joins: `SELECT e FROM Employee e JOIN e.department d JOIN d.company c WHERE c.name = :company`. Hibernate generates the appropriate SQL JOINs.

Q7: What is the difference between @Query and @NamedQuery?

`@Query` is placed on a repository method and associates the JPQL with that specific method. `@NamedQuery` is placed on the entity class and defines a named query that can be referenced by convention (`EntityName.methodName`). Both are validated at startup. `@Query` is more common because it keeps the query close to its usage.

Q8: How do aggregate functions with GROUP BY work in JPQL?

Aggregate functions (COUNT, SUM, AVG, MIN, MAX) operate on groups defined by GROUP BY. `SELECT d.name, COUNT(e) FROM Employee e JOIN e.department d GROUP BY d.name` counts employees per department. HAVING filters groups after aggregation: `HAVING COUNT(e) > 5` keeps only departments with more than 5 employees. Results are typically returned as `Object[]` or DTOs via constructor expressions.

Practice Exercises

Exercise 1: Employee Search Write JPQL queries with @Query for: (a) find employees hired in a specific year, (b) find employees whose name starts with a letter, (c) find the top 5 highest-paid active employees, (d) find employees in departments located in a specific city.

Exercise 2: Aggregate Reports Write JPQL queries that return: (a) total salary cost per department, (b) the department with the highest average salary, (c) the number of employees hired each year, (d) departments where the average salary exceeds a given threshold.

Exercise 3: Constructor Expressions Create DTOs for: (a) `EmployeeCard` with name, email, and department name, (b) `SalaryReport` with department name, employee count, total salary, and average salary. Write JPQL queries using constructor expressions to populate them.

Exercise 4: JOIN FETCH Create a query that loads all employees with their departments in a single query. First, run it WITHOUT JOIN FETCH and count the SQL queries generated. Then add JOIN FETCH and verify only one query runs.

Exercise 5: Bulk Operations Write @Modifying queries to: (a) give a 10% raise to all employees in a department, (b) deactivate all employees with hire dates before 2020, (c) transfer all employees from one department to another. Test each and verify the number of rows affected.

What Is Next?

In the next chapter, we will explore Native SQL Queries and Result Mapping. While JPQL covers most use cases, sometimes you need database-specific features, complex subqueries, window functions, or raw SQL performance. You will learn when to use native SQL over JPQL, how to map native query results to entities and DTOs, and how to safely mix

native SQL with JPA.

Chapter 9: Native SQL Queries and Result Mapping

Learning Goals

By the end of this chapter, you will be able to:

- Know when to use native SQL instead of JPQL
- Write native SQL queries with @Query(nativeQuery = true)
- Use named and positional parameters in native queries
- Map native query results to entities automatically
- Map native query results to DTOs using interfaces and classes
- Use @SqlResultSetMapping and @ColumnResult for complex mappings
- Use @ConstructorResult to map results to DTO constructors
- Combine native SQL with Spring Data JPA pagination
- Understand the trade-offs of native SQL vs JPQL

When to Use Native SQL

JPQL handles the vast majority of queries. But there are times when you need to drop down to raw SQL:

```
+-----------------------------------------------------------------+
|  Use Native SQL When:                                           |
|                                                                 |
|  1. Database-specific features                                  |
|     - Window functions (ROW_NUMBER, RANK, LAG, LEAD)            |
|     - Common Table Expressions (WITH ... AS)                    |
```

```
|     - Database-specific functions (PostgreSQL jsonb, etc.)         |
|     - Full-text search (tsvector in PostgreSQL)                    |
|                                                                    |
|  2. Complex queries that JPQL cannot express                       |
|     - UNION / INTERSECT / EXCEPT                                   |
|     - Recursive queries                                            |
|     - Complex subqueries in FROM clause                            |
|                                                                    |
|  3. Performance-critical queries                                   |
|     - Hand-tuned SQL with optimizer hints                          |
|     - Queries where JPQL generates suboptimal SQL                  |
|                                                                    |
|  4. Legacy database integration                                    |
|     - Views that do not map to entities                            |
|     - Stored procedures and functions                              |
|     - Schemas you cannot modify                                    |
+--------------------------------------------------------------------+

+--------------------------------------------------------------------+
|  Stick with JPQL When:                                             |
|                                                                    |
|  - Standard CRUD and simple joins                                  |
|  - You want database portability                                   |
|  - Derived query methods or @Query with JPQL work fine             |
|  - You want compile-time entity/field name validation              |
+--------------------------------------------------------------------+
```

Basic Native Queries with @Query

Add `nativeQuery = true` to the `@Query` annotation to write raw SQL:

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // JPQL (for comparison):
    @Query("SELECT e FROM Employee e WHERE e.active = true")
    List<Employee> findActiveJpql();

    // Native SQL (same result):
    @Query(value = "SELECT * FROM employees WHERE active = true",
           nativeQuery = true)
    List<Employee> findActiveNative();
}
```

```
JPQL vs Native SQL in @Query:
+--------------------------------------------------------------------+
```

```
|  JPQL:                                                          |
|  @Query("SELECT e FROM Employee e WHERE e.active = true")        |
|  - Uses entity name (Employee) and field names (e.active)        |
|  - Database-independent                                          |
|  - Validated at startup                                          |
|                                                                  |
|  Native SQL:                                                     |
|  @Query(value = "SELECT * FROM employees WHERE active = true",   |
|         nativeQuery = true)                                      |
|  - Uses table name (employees) and column names (active)         |
|  - Database-specific                                             |
|  - NOT validated at startup (errors at runtime)                  |
+------------------------------------------------------------------+
```

Parameters in Native Queries

```java
// Named parameters (use :paramName)
@Query(value = "SELECT * FROM employees WHERE department_id = :deptId AND salary > :min...
       nativeQuery = true)
List<Employee> findByDeptAndSalary(@Param("deptId") Long deptId,
                                   @Param("minSalary") BigDecimal minSalary);

// Positional parameters (use ?1, ?2)
@Query(value = "SELECT * FROM employees WHERE department_id = ?1 AND salary > ?2",
       nativeQuery = true)
List<Employee> findByDeptAndSalary(Long deptId, BigDecimal minSalary);
```

Native Queries That Return Entities

When you use `SELECT *` or select all columns that match the entity's mapping, Spring Data JPA automatically maps the result to the entity class:

```java
// Returns List<Employee> — automatic mapping
@Query(value = "SELECT * FROM employees WHERE active = true ORDER BY name",
       nativeQuery = true)
List<Employee> findActiveOrderByName();

// Also works with specific columns matching entity fields
@Query(value = "SELECT id, name, email, salary, active, hire_date, department_id " +
               "FROM employees WHERE salary > :min",
       nativeQuery = true)
List<Employee> findHighEarners(@Param("min") BigDecimal min);
```

Important: The column names in the result set must match the entity's column mappings. If you use `@Column(name = "hire_date")`, the SQL must select `hire_date`, not `hireDate`.

Database-Specific Features

This is where native SQL really shines — using features that JPQL does not support.

Window Functions

```java
// Rank employees by salary within each department
@Query(value = """
    SELECT e.*,
           RANK() OVER (PARTITION BY e.department_id ORDER BY e.salary DESC) as salary_...
    FROM employees e
    WHERE e.active = true
    """,
       nativeQuery = true)
List<Object[]> findEmployeesWithSalaryRank();

// Row number for pagination alternative
@Query(value = """
    SELECT * FROM (
        SELECT e.*, ROW_NUMBER() OVER (ORDER BY e.salary DESC) as rn
        FROM employees e
        WHERE e.active = true
    ) ranked WHERE rn BETWEEN :start AND :end
    """,
       nativeQuery = true)
List<Employee> findByRankRange(@Param("start") int start, @Param("end") int end);
```

Common Table Expressions (CTEs)

```java
// Find departments where average salary exceeds the company average
@Query(value = """
    WITH company_avg AS (
        SELECT AVG(salary) as avg_salary FROM employees WHERE active = true
    ),
    dept_avg AS (
        SELECT department_id, AVG(salary) as avg_salary, COUNT(*) as emp_count
        FROM employees WHERE active = true
        GROUP BY department_id
    )
    SELECT d.id, d.name, da.avg_salary, da.emp_count
    FROM departments d
    JOIN dept_avg da ON d.id = da.department_id
    CROSS JOIN company_avg ca
```

```
    WHERE da.avg_salary > ca.avg_salary
    ORDER BY da.avg_salary DESC
    """,
       nativeQuery = true)
List<Object[]> findAboveAverageDepartments();
```

UNION Queries

```java
// Combine results from multiple queries
@Query(value = """
    SELECT name, email, 'ACTIVE' as status FROM employees WHERE active = true
    UNION ALL
    SELECT name, email, 'INACTIVE' as status FROM employees WHERE active = false
    ORDER BY status, name
    """,
       nativeQuery = true)
List<Object[]> findAllWithStatus();
```

H2-Specific Functions

```java
// H2 date functions
@Query(value = "SELECT * FROM employees WHERE YEAR(hire_date) = :year",
       nativeQuery = true)
List<Employee> findHiredInYear(@Param("year") int year);

// H2 string functions
@Query(value = "SELECT * FROM employees WHERE LOCATE(:keyword, LOWER(name)) > 0",
       nativeQuery = true)
List<Employee> searchName(@Param("keyword") String keyword);
```

Mapping Results to DTOs

Native queries often return data that does not map to an entity. You need to map results to custom objects.

Approach 1: Interface-Based Projections (Recommended)

The simplest and cleanest approach. Define an interface with getter methods matching the column names:

```java
// Step 1: Define a projection interface
public interface EmployeeSummaryProjection {
    String getName();
    String getEmail();
    BigDecimal getSalary();
    String getDepartmentName();  // Matches column alias "department_name"
}

// Step 2: Use it in the repository
@Query(value = """
    SELECT e.name, e.email, e.salary, d.name AS department_name
    FROM employees e
    JOIN departments d ON e.department_id = d.id
    WHERE e.active = true
    ORDER BY e.salary DESC
    """,
       nativeQuery = true)
List<EmployeeSummaryProjection> findActiveSummaries();
```

```java
// Usage - clean and type-safe
List<EmployeeSummaryProjection> summaries = repository.findActiveSummaries();
for (EmployeeSummaryProjection s : summaries) {
    System.out.println(s.getName() + " (" + s.getDepartmentName() +
                       ") - $" + s.getSalary());
}
```

```
How Interface Projections Work:
+-------------------------------------------------------------------+
|  SQL result row:                                                  |
|  | name    | email           | salary  | department_name |        |
|  | Alice   | alice@co.com    | 95000   | Engineering     |        |
|                                                                   |
|  Spring creates a proxy object implementing the interface:        |
|  proxy.getName()           --> "Alice"                            |
|  proxy.getEmail()          --> "alice@co.com"                     |
|  proxy.getSalary()         --> 95000                              |
|  proxy.getDepartmentName() --> "Engineering"                      |
|                                                                   |
|  Matching rules:                                                  |
|  - Method name getXxx maps to column "xxx" or "XXX"               |
|  - Use column aliases to match: d.name AS department_name         |
|  - Case-insensitive matching                                      |
+-------------------------------------------------------------------+
```

Approach 2: Class-Based Projections (DTO Classes)

Use a concrete class instead of an interface:

```java
// DTO class
public class DepartmentReport {
    private String departmentName;
    private long employeeCount;
    private double averageSalary;
    private BigDecimal totalSalary;

    // Constructor must match the query columns in order
    public DepartmentReport(String departmentName, long employeeCount,
                            double averageSalary, BigDecimal totalSalary) {
        this.departmentName = departmentName;
        this.employeeCount = employeeCount;
        this.averageSalary = averageSalary;
        this.totalSalary = totalSalary;
    }

    // Getters
    public String getDepartmentName() { return departmentName; }
    public long getEmployeeCount() { return employeeCount; }
    public double getAverageSalary() { return averageSalary; }
    public BigDecimal getTotalSalary() { return totalSalary; }
}
```

For native queries with class-based projections, you typically need `@SqlResultSetMapping` or return `Object[]` and map manually. Interface projections are simpler for native queries.

Approach 3: Manual Mapping from Object[]

For complex results, sometimes manual mapping is the most practical:

```java
@Query(value = """
    SELECT d.name as dept_name,
           COUNT(e.id) as emp_count,
           COALESCE(AVG(e.salary), 0) as avg_salary,
           COALESCE(SUM(e.salary), 0) as total_salary
    FROM departments d
    LEFT JOIN employees e ON d.id = e.department_id AND e.active = true
    GROUP BY d.name
    ORDER BY emp_count DESC
    """,
       nativeQuery = true)
List<Object[]> findDepartmentReportRaw();
```

```java
// Service method that maps the results
public List<DepartmentReport> getDepartmentReport() {
    return repository.findDepartmentReportRaw().stream()
        .map(row -> new DepartmentReport(
            (String) row[0],                                // dept_name
```

```
            ((Number) row[1]).longValue(),                  // emp_count
            ((Number) row[2]).doubleValue(),                // avg_salary
            row[3] != null ? new BigDecimal(row[3].toString()) : BigDecimal.ZERO
        ))
        .toList();
}
```

```
Projection Approaches Comparison:
+-------------------------------------------------------------------+
|  Approach                  Pros                 Cons              |
|-------------------------------------------------------------------|
|  Interface Projection      Clean, automatic     Cannot have       |
|  (Recommended)             Spring proxy magic   logic/methods     |
|                                                                   |
|  Class-Based DTO           Full control,        More setup for    |
|                            can add methods      native queries    |
|                                                                   |
|  Object[] + manual map     Works with any       Fragile, index-   |
|                            query shape          based, type-unsafe|
+-------------------------------------------------------------------+
```

@SqlResultSetMapping

For advanced result mapping, JPA provides `@SqlResultSetMapping` on the entity class. This is useful when you need to map native query results to entities or DTOs with precise control.

Mapping to an Entity

```java
@Entity
@Table(name = "employees")
@SqlResultSetMapping(
    name = "EmployeeMapping",
    entities = @EntityResult(
        entityClass = Employee.class,
        fields = {
            @FieldResult(name = "id", column = "emp_id"),
            @FieldResult(name = "name", column = "emp_name"),
            @FieldResult(name = "email", column = "emp_email"),
            @FieldResult(name = "salary", column = "emp_salary"),
            @FieldResult(name = "active", column = "emp_active")
        }
    )
)
```

```
public class Employee {
    // ...
}
```

This is used when the SQL column names do not match the entity field mappings (perhaps from a complex join or view).

@ConstructorResult — Mapping to DTOs

`@ConstructorResult` maps native query columns to a DTO constructor:

```java
@Entity
@Table(name = "employees")
@SqlResultSetMapping(
    name = "EmployeeSummaryMapping",
    classes = @ConstructorResult(
        targetClass = EmployeeSummary.class,
        columns = {
            @ColumnResult(name = "emp_name", type = String.class),
            @ColumnResult(name = "dept_name", type = String.class),
            @ColumnResult(name = "emp_salary", type = BigDecimal.class)
        }
    )
)
public class Employee {
    // ...
}
```

```java
// The DTO
public class EmployeeSummary {
    private String name;
    private String departmentName;
    private BigDecimal salary;

    public EmployeeSummary(String name, String departmentName, BigDecimal salary) {
        this.name = name;
        this.departmentName = departmentName;
        this.salary = salary;
    }
    // getters...
}
```

```java
// Using with EntityManager (not directly supported in @Query)
@Repository
public class CustomEmployeeRepository {
```

```
    @PersistenceContext
    private EntityManager em;

    public List<EmployeeSummary> findSummaries() {
        return em.createNativeQuery(
            "SELECT e.name AS emp_name, d.name AS dept_name, e.salary AS emp_salary " +
            "FROM employees e JOIN departments d ON e.department_id = d.id",
            "EmployeeSummaryMapping"
        ).getResultList();
    }
}
```

```
@SqlResultSetMapping usage:
+-----------------------------------------------------------------+
|  For simple native queries:                                      |
|    Use interface projections (easiest)                            |
|                                                                  |
|  For complex column name remapping:                              |
|    Use @SqlResultSetMapping with @EntityResult                   |
|                                                                  |
|  For mapping to DTOs from EntityManager:                         |
|    Use @SqlResultSetMapping with @ConstructorResult              |
|                                                                  |
|  For Spring Data @Query with native SQL:                         |
|    Interface projections work best                               |
|    @SqlResultSetMapping is used with EntityManager directly      |
+-----------------------------------------------------------------+
```

Native Queries with Pagination

Spring Data JPA supports pagination with native queries, but requires a separate count query:

```java
// Must provide countQuery for pagination
@Query(value = "SELECT * FROM employees WHERE active = true ORDER BY name",
       countQuery = "SELECT COUNT(*) FROM employees WHERE active = true",
       nativeQuery = true)
Page<Employee> findActivePagedNative(Pageable pageable);
```

```
Why countQuery is needed:
+-----------------------------------------------------------------+
|  For JPQL, Hibernate can derive the count query automatically:    |
|  @Query("SELECT e FROM Employee e WHERE e.active = true")         |
|  --> Hibernate derives: SELECT COUNT(e) FROM Employee e            |
|                         WHERE e.active = true                     |
```

```
|                                                                  |
|  For native SQL, Hibernate cannot parse the SQL to derive a      |
|  count query. You must provide it explicitly:                    |
|  @Query(value = "SELECT * FROM employees WHERE active = true",   |
|         countQuery = "SELECT COUNT(*) FROM employees ...",        |
|         nativeQuery = true)                                      |
+------------------------------------------------------------------+
```

Pagination with Projections

```java
// Interface projection with pagination
@Query(value = "SELECT e.name, e.email, e.salary FROM employees e WHERE e.active = true",
       countQuery = "SELECT COUNT(*) FROM employees WHERE active = true",
       nativeQuery = true)
Page<EmployeeBasicProjection> findActiveProjectedPaged(Pageable pageable);

public interface EmployeeBasicProjection {
    String getName();
    String getEmail();
    BigDecimal getSalary();
}
```

Usage

```java
// Page 0, 10 items per page, sorted by salary descending
Pageable pageable = PageRequest.of(0, 10, Sort.by("salary").descending());
Page<Employee> page = repository.findActivePagedNative(pageable);

System.out.println("Total elements: " + page.getTotalElements());
System.out.println("Total pages: " + page.getTotalPages());
System.out.println("Current page: " + page.getNumber());
page.getContent().forEach(e -> System.out.println("  " + e));
```

Native UPDATE and DELETE

```java
// Native UPDATE
@Modifying
@Query(value = "UPDATE employees SET salary = salary * :factor WHERE department_id = :d...
       nativeQuery = true)
```

```
int adjustSalariesNative(@Param("factor") BigDecimal factor,
                         @Param("deptId") Long deptId);

// Native DELETE
@Modifying
@Query(value = "DELETE FROM employees WHERE active = false AND hire_date < :cutoff",
       nativeQuery = true)
int purgeOldInactive(@Param("cutoff") LocalDate cutoff);

// Native INSERT (rare, but possible)
@Modifying
@Query(value = "INSERT INTO employees (name, email, salary, active, department_id) " +
               "VALUES (:name, :email, :salary, true, :deptId)",
       nativeQuery = true)
void insertEmployee(@Param("name") String name, @Param("email") String email,
                    @Param("salary") BigDecimal salary, @Param("deptId") Long deptId);
```

Note: Native `@Modifying` queries follow the same rules as JPQL ones — they require `@Transactional` and do not update the persistence context. Use `clearAutomatically = true` when needed.

Calling Database Functions and Stored Procedures

Calling Database Functions

```java
// H2 built-in functions
@Query(value = "SELECT * FROM employees WHERE DATEDIFF('YEAR', hire_date, CURRENT_DATE)...
       nativeQuery = true)
List<Employee> findLongTermEmployees(@Param("years") int years);

// Using CASE in native SQL
@Query(value = """
    SELECT e.name,
           CASE WHEN e.salary >= 100000 THEN 'Senior'
                WHEN e.salary >= 70000 THEN 'Mid-Level'
                ELSE 'Junior' END AS level
    FROM employees e
    WHERE e.active = true
    ORDER BY e.salary DESC
    """,
       nativeQuery = true)
List<Object[]> findEmployeeLevels();
```

Calling Stored Procedures (JPA Standard)

```java
// Define the stored procedure call on the entity
@Entity
@NamedStoredProcedureQuery(
    name = "Employee.raiseByDepartment",
    procedureName = "raise_by_department",
    parameters = {
        @StoredProcedureParameter(mode = ParameterMode.IN,
            name = "dept_id", type = Long.class),
        @StoredProcedureParameter(mode = ParameterMode.IN,
            name = "raise_pct", type = BigDecimal.class),
        @StoredProcedureParameter(mode = ParameterMode.OUT,
            name = "affected_count", type = Integer.class)
    }
)
public class Employee { ... }

// Call from repository
@Procedure(name = "Employee.raiseByDepartment")
int raiseByDepartment(@Param("dept_id") Long deptId,
                      @Param("raise_pct") BigDecimal raisePct);
```

Mixing Native SQL and JPQL

In practice, most repositories use a combination of derived methods, JPQL, and native SQL:

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // === Derived Methods (simplest queries) ===
    List<Employee> findByActiveTrue();
    Optional<Employee> findByEmail(String email);
    long countByDepartment_Name(String deptName);

    // === JPQL (standard queries with entity navigation) ===
    @Query("SELECT e FROM Employee e JOIN FETCH e.department WHERE e.active = true")
    List<Employee> findActiveWithDepartment();

    @Query("SELECT new com.example.dto.EmployeeSummary(e.name, d.name, e.salary) " +
           "FROM Employee e JOIN e.department d ORDER BY e.salary DESC")
    List<EmployeeSummary> findSummaries();

    // === Native SQL (database-specific features) ===
    @Query(value = """
        SELECT e.*, RANK() OVER (PARTITION BY e.department_id
                                 ORDER BY e.salary DESC) as rnk
        FROM employees e
```

```
        WHERE e.active = true
        """,
           nativeQuery = true)
    List<Object[]> findWithDepartmentRank();

    @Query(value = """
        WITH dept_stats AS (
            SELECT department_id, AVG(salary) as avg_sal
            FROM employees WHERE active = true
            GROUP BY department_id
        )
        SELECT e.* FROM employees e
        JOIN dept_stats ds ON e.department_id = ds.department_id
        WHERE e.salary > ds.avg_sal * 1.2
        """,
           nativeQuery = true)
    List<Employee> findTopPerformers();
}
```

```
Decision Guide: Which Query Type to Use?
+-----------------------------------------------------------------+
|  Start Here                                                     |
|      |                                                          |
|      v                                                          |
|  Can a derived method express it?                               |
|      |                                                          |
|   YES --> Use derived method (findByNameContaining, etc.)       |
|      |                                                          |
|    NO                                                           |
|      |                                                          |
|      v                                                          |
|  Can JPQL express it? (joins, aggregates, subqueries)           |
|      |                                                          |
|   YES --> Use @Query with JPQL                                  |
|      |                                                          |
|    NO                                                           |
|      |                                                          |
|      v                                                          |
|  Need DB-specific features?                                     |
|  (window functions, CTEs, UNION, hints)                         |
|      |                                                          |
|   YES --> Use @Query(nativeQuery = true) with native SQL        |
+-----------------------------------------------------------------+
```

Common Mistakes

1. Using entity/field names in native queries: Native SQL uses table and column names, not entity and field names. `SELECT * FROM Employee` fails; use `SELECT * FROM employees`.
2. Forgetting countQuery with pagination: `Page<T>` return type with native queries requires an explicit `countQuery`. Without it, you get an error.
3. Column name mismatches with interface projections: The getter method name (`getDepartmentName`) must match the column name or alias (`department_name`). Case-insensitive, but underscores matter.
4. Losing database portability: Native SQL ties your code to a specific database. If you switch from H2 to PostgreSQL, some native queries might need changes (different date functions, syntax variations).
5. Not using clearAutomatically on @Modifying native queries: Same as JPQL — bulk updates via native SQL do not update the persistence context.
6. Expecting native queries to be validated at startup: Unlike JPQL, native SQL is not parsed by Hibernate at startup. Syntax errors are only caught at runtime when the query executes.

Best Practices

1. Prefer JPQL over native SQL: Use native SQL only when JPQL cannot express what you need. JPQL is portable, validated at startup, and works with entity relationships naturally.
2. Use interface projections for native query results: They are the simplest and cleanest way to map non-entity results from native queries.
3. Always provide countQuery for paginated native queries: This is required and prevents runtime errors.
4. Use text blocks (triple quotes) for long queries: Java text blocks (`"""..."""`) make multi-line SQL queries much more readable than string concatenation.
5. Comment your native queries: Since native SQL is not self-documenting like JPQL (which uses entity names), add comments explaining why native SQL is needed and what database-specific features it uses.
6. Test native queries against your target database: If you develop with H2 but deploy to PostgreSQL, test native queries against PostgreSQL early. SQL dialects differ.

Summary

In this chapter, you learned when and how to use native SQL queries:

- Native SQL is needed for database-specific features (window functions, CTEs, UNION), complex queries that JPQL cannot express, and performance-critical hand-tuned SQL.
- @Query(nativeQuery = true) enables native SQL in Spring Data JPA repositories. It uses table/column names instead of entity/field names.
- Interface projections are the recommended way to map native query results to DTOs. Define an interface with getters matching column names/aliases.
- @SqlResultSetMapping with @ConstructorResult provides advanced result mapping for complex native queries used with EntityManager.
- Pagination with native queries requires an explicit `countQuery` parameter.
- Native @Modifying works the same as JPQL — use for bulk UPDATE/DELETE, requires @Transactional, does not update the persistence context.
- The decision hierarchy: Use derived methods first, then JPQL, then native SQL. Each step adds power but reduces portability.

Interview Questions

Q1: When should you use native SQL instead of JPQL?

Use native SQL for: database-specific features (window functions, CTEs, full-text search), UNION queries, recursive queries, complex subqueries in FROM clauses, optimizer hints, queries against views or legacy schemas, and when JPQL generates suboptimal SQL that needs hand-tuning.

Q2: How do you map native query results to DTOs in Spring Data JPA?

The simplest approach is interface-based projections: define an interface with getter methods matching column names or aliases, and use it as the return type. For more complex mappings, use `@SqlResultSetMapping` with `@ConstructorResult` on the entity class, or manually map `Object[]` results in a service method.

Q3: Why does a native query with Page return type require a countQuery?

For JPQL, Hibernate can automatically derive a count query by analyzing the query structure. For native SQL, Hibernate cannot parse the SQL to derive a count. The count query must be provided explicitly so Spring Data can determine the total number of elements for pagination metadata.

Q4: What is the key difference between native SQL and JPQL in terms of naming?

JPQL uses entity class names and Java field names (e.g., `SELECT e FROM Employee e WHERE e.firstName = :name`). Native SQL uses database table names and column names (e.g., `SELECT * FROM employees WHERE first_name = :name`). This is the most common source of errors when switching between the two.

Q5: Are native queries validated at application startup?

No. JPQL queries (both @Query and @NamedQuery) are parsed and validated at startup — a typo causes an immediate error. Native SQL is passed directly to the database at runtime. Syntax errors are only caught when the query is actually executed.

Q6: How do interface projections work with native queries?

Spring Data creates a dynamic proxy implementing the projection interface. When a native query returns results, Spring maps each column to the corresponding getter method by matching the getter name to the column name or alias (case-insensitive). `getDepartmentName()` maps to a column named `department_name` or `DEPARTMENT_NAME`.

Practice Exercises

Exercise 1: Window Functions Write a native query that ranks employees by salary within each department using `RANK() OVER (PARTITION BY ...)`. Return the results with employee name, department, salary, and rank. Use an interface projection.

Exercise 2: CTE Query Write a native query using a CTE that finds employees whose salary is more than 20% above their department's average. Test it with sample data.

Exercise 3: Projection Comparison Write the same query (employees with department names, salary above a threshold) three ways: (a) returning `Object[]`, (b) using an interface projection, (c) using JPQL with a constructor expression. Compare the code clarity.

Exercise 4: Paginated Native Query Write a native query that returns all active employees with pagination support. Include the countQuery. Test with `PageRequest.of(0, 5)` and verify the total elements, total pages, and content.

Exercise 5: Mixed Repository Create a repository for a `Product` entity that uses all three query types: 3 derived methods, 3 JPQL queries, and 3 native queries. Choose each type based on the complexity of the query. Explain your choices.

What Is Next?

In the next chapter, we will explore the Criteria API — Type-Safe Dynamic Queries. While JPQL and native SQL are string-based (and prone to typos), the Criteria API lets you build queries programmatically with compile-time type safety. You will learn how to use `CriteriaBuilder`, `CriteriaQuery`, and `Predicate` for dynamic query construction, and how Spring Data Specifications make the Criteria API practical for everyday use.

Chapter 10: Criteria API — Type-Safe Dynamic Queries

Learning Goals

By the end of this chapter, you will be able to:

- Understand why the Criteria API exists and when to use it
- Build queries programmatically using CriteriaBuilder and CriteriaQuery
- Use Root, Join, and Predicate to construct WHERE clauses
- Create dynamic queries with conditional predicates
- Use the JPA Metamodel for compile-time type safety
- Apply aggregate functions, grouping, and ordering with Criteria API
- Implement Spring Data Specifications for reusable query fragments
- Combine specifications with and/or logic for complex filtering

Why the Criteria API?

JPQL and native SQL are string-based. You write the query as a string, and errors are caught only at runtime (or startup for JPQL). The Criteria API solves this by letting you build queries with Java code:

```
String-Based Queries (JPQL / SQL):
+-----------------------------------------------------------------+
|  @Query("SELECT e FROM Employee e WHERE e.departmnt = :dept")     |
|                                              ^                    |
|                                              Typo!                |
|                                              Not caught until     |
```

```
|                                                  runtime / startup    |
+-----------------------------------------------------------------------+

Criteria API (Programmatic):
+-----------------------------------------------------------------------+
|  cb.equal(root.get("departmnt"), dept)                                |
|                        ^                                              |
|                        Still a string - caught at runtime             |
|                                                                       |
|  cb.equal(root.get(Employee_.department), dept)                       |
|                        ^                                              |
|                        Metamodel field - caught at COMPILE TIME       |
+-----------------------------------------------------------------------+
```

But the biggest advantage is dynamic queries. Consider a search form:

```
Search Employees Form:
+-----------------------------------------------------------------------+
|  Name:       [____________]  (optional)                               |
|  Department: [____________]  (optional)                               |
|  Min Salary: [____________]  (optional)                               |
|  Active:     [ ] Yes  [ ] No  [ ] Either  (optional)                 |
|                                                                       |
|  [Search]                                                             |
+-----------------------------------------------------------------------+
```

With JPQL, building this query dynamically is ugly:

```java
// JPQL approach: string concatenation nightmare
public List<Employee> search(String name, String dept,
                             BigDecimal minSalary, Boolean active) {
    StringBuilder jpql = new StringBuilder("SELECT e FROM Employee e WHERE 1=1");
    Map<String, Object> params = new HashMap<>();

    if (name != null) {
        jpql.append(" AND LOWER(e.name) LIKE :name");
        params.put("name", "%" + name.toLowerCase() + "%");
    }
    if (dept != null) {
        jpql.append(" AND e.department.name = :dept");
        params.put("dept", dept);
    }
    if (minSalary != null) {
        jpql.append(" AND e.salary >= :minSalary");
        params.put("minSalary", minSalary);
    }
    if (active != null) {
        jpql.append(" AND e.active = :active");
        params.put("active", active);
```

```
    }

    TypedQuery<Employee> query = em.createQuery(jpql.toString(), Employee.class);
    params.forEach(query::setParameter);
    return query.getResultList();
}
```

With the Criteria API, it is cleaner and type-safe:

```java
// Criteria API approach: programmatic and clean
public List<Employee> search(String name, String dept,
                             BigDecimal minSalary, Boolean active) {
    CriteriaBuilder cb = em.getCriteriaBuilder();
    CriteriaQuery<Employee> cq = cb.createQuery(Employee.class);
    Root<Employee> root = cq.from(Employee.class);

    List<Predicate> predicates = new ArrayList<>();

    if (name != null) {
        predicates.add(cb.like(cb.lower(root.get("name")),
                       "%" + name.toLowerCase() + "%"));
    }
    if (dept != null) {
        predicates.add(cb.equal(root.get("department").get("name"), dept));
    }
    if (minSalary != null) {
        predicates.add(cb.greaterThanOrEqualTo(root.get("salary"), minSalary));
    }
    if (active != null) {
        predicates.add(cb.equal(root.get("active"), active));
    }

    cq.select(root).where(predicates.toArray(new Predicate[0]));
    return em.createQuery(cq).getResultList();
}
```

Criteria API Building Blocks

The Criteria API consists of four main components:

```
Criteria API Architecture:
+-------------------------------------------------------------------+
|                                                                   |
|  CriteriaBuilder (cb)                                             |
|  +-------------------------+                                      |
```

```
|  | Factory for everything:  |                                    |
|  | - CriteriaQuery          |                                    |
|  | - Predicates (WHERE)     |                                    |
|  | - Expressions            |                                    |
|  | - Aggregates             |                                    |
|  | - Ordering               |                                    |
|  +--------------------------+                                    |
|           |                                                      |
|           v                                                      |
|  CriteriaQuery<T> (cq)                                          |
|  +--------------------------+                                    |
|  | Represents the query:    |                                    |
|  | - .select()              |                                    |
|  | - .where()               |                                    |
|  | - .orderBy()             |                                    |
|  | - .groupBy()             |                                    |
|  | - .having()              |                                    |
|  +--------------------------+                                    |
|           |                                                      |
|           v                                                      |
|  Root<T> (root)                                                  |
|  +--------------------------+                                    |
|  | Represents the FROM:     |                                    |
|  | - .get("fieldName")      |                                    |
|  | - .join("relationship")  |                                    |
|  | - .fetch("relationship") |                                    |
|  +--------------------------+                                    |
|           |                                                      |
|           v                                                      |
|  Predicate                                                       |
|  +--------------------------+                                    |
|  | Represents a condition:  |                                    |
|  | cb.equal(a, b)           |                                    |
|  | cb.like(a, pattern)      |                                    |
|  | cb.greaterThan(a, b)     |                                    |
|  | cb.and(p1, p2)           |                                    |
|  | cb.or(p1, p2)            |                                    |
|  +--------------------------+                                    |
|                                                                  |
+------------------------------------------------------------------+
```

Step-by-Step: Building a Criteria Query

```java
// Step 1: Get CriteriaBuilder from EntityManager
CriteriaBuilder cb = entityManager.getCriteriaBuilder();

// Step 2: Create a CriteriaQuery for the result type
CriteriaQuery<Employee> cq = cb.createQuery(Employee.class);

// Step 3: Define the FROM clause (the root entity)
```

```
Root<Employee> root = cq.from(Employee.class);

// Step 4: Build predicates (WHERE conditions)
Predicate isActive = cb.equal(root.get("active"), true);
Predicate highSalary = cb.greaterThan(root.get("salary"), new BigDecimal("80000"));

// Step 5: Compose the query
cq.select(root)
  .where(cb.and(isActive, highSalary))
  .orderBy(cb.desc(root.get("salary")));

// Step 6: Execute
List<Employee> results = entityManager.createQuery(cq).getResultList();
```

```
This generates SQL equivalent to:
+-----------------------------------------------------------------+
|  SELECT * FROM employees                                         |
|  WHERE active = true AND salary > 80000                          |
|  ORDER BY salary DESC                                            |
+-----------------------------------------------------------------+
```

CriteriaBuilder Methods Reference

Comparison Predicates

```java
CriteriaBuilder cb = em.getCriteriaBuilder();
Root<Employee> root = cq.from(Employee.class);

// Equality
cb.equal(root.get("name"), "Alice")           // name = 'Alice'
cb.notEqual(root.get("active"), false)        // active != false

// Comparison
cb.greaterThan(root.get("salary"), 50000)     // salary > 50000
cb.greaterThanOrEqualTo(root.get("salary"), 50000)  // salary >= 50000
cb.lessThan(root.get("salary"), 100000)       // salary < 100000
cb.lessThanOrEqualTo(root.get("salary"), 100000)    // salary <= 100000
cb.between(root.get("salary"), 50000, 100000) // salary BETWEEN 50000 AND 100000

// String matching
cb.like(root.get("name"), "%Alice%")          // name LIKE '%Alice%'
cb.like(root.get("name"), "A%")               // name LIKE 'A%'
cb.notLike(root.get("name"), "%test%")        // name NOT LIKE '%test%'
```

```
// Null checks
cb.isNull(root.get("email"))                  // email IS NULL
cb.isNotNull(root.get("email"))               // email IS NOT NULL

// Collection
root.get("name").in("Alice", "Bob", "Charlie") // name IN ('Alice', 'Bob', 'Charlie')
cb.not(root.get("name").in("Alice", "Bob"))    // name NOT IN ('Alice', 'Bob')
```

Logical Operators

```java
Predicate isActive = cb.equal(root.get("active"), true);
Predicate highSalary = cb.greaterThan(root.get("salary"), 80000);
Predicate inEngineering = cb.equal(root.get("department").get("name"), "Engineering");

// AND
cb.and(isActive, highSalary)                  // active = true AND salary > 80000
cb.and(isActive, highSalary, inEngineering)   // all three combined with AND

// OR
cb.or(highSalary, inEngineering)              // salary > 80000 OR dept = 'Engineering'

// NOT
cb.not(isActive)                              // NOT (active = true)

// Complex combinations
cb.and(
    isActive,
    cb.or(highSalary, inEngineering)
)
// active = true AND (salary > 80000 OR dept = 'Engineering')
```

```
Predicate Composition:
+-------------------------------------------------------------------+
|                                                                    |
|  Simple:     cb.equal(root.get("active"), true)                   |
|              --> WHERE active = true                               |
|                                                                    |
|  AND:        cb.and(p1, p2)                                       |
|              --> WHERE active = true AND salary > 80000            |
|                                                                    |
|  OR:         cb.or(p1, p2)                                        |
|              --> WHERE active = true OR salary > 80000             |
|                                                                    |
|  Nested:     cb.and(p1, cb.or(p2, p3))                            |
|              --> WHERE active = true                               |
|                  AND (salary > 80000 OR dept = 'Engineering')      |
|                                                                    |
|  Array:      cb.and(predicates.toArray(new Predicate[0]))         |
|              --> WHERE p1 AND p2 AND p3 AND ...                    |
```

```
|              (perfect for dynamic queries!)                        |
|                                                                   |
+-------------------------------------------------------------------+
```

Joins with Criteria API

Basic Join

```java
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Employee> cq = cb.createQuery(Employee.class);
Root<Employee> employee = cq.from(Employee.class);

// INNER JOIN to department
Join<Employee, Department> department = employee.join("department");

// WHERE clause using the joined entity
cq.select(employee)
  .where(cb.equal(department.get("name"), "Engineering"));

// Generated SQL: SELECT e.* FROM employees e
//                JOIN departments d ON e.department_id = d.id
//                WHERE d.name = 'Engineering'
```

Join Types

```java
// INNER JOIN (default)
Join<Employee, Department> dept = employee.join("department");
Join<Employee, Department> dept = employee.join("department", JoinType.INNER);

// LEFT JOIN
Join<Employee, Department> dept = employee.join("department", JoinType.LEFT);

// Multiple joins
Join<Employee, Department> dept = employee.join("department");
Join<Department, Company> company = dept.join("company");

cq.where(cb.equal(company.get("name"), "TechCorp"));
// SELECT e.* FROM employees e
// JOIN departments d ON e.department_id = d.id
// JOIN companies c ON d.company_id = c.id
// WHERE c.name = 'TechCorp'
```

```
Join Diagram:
+-------------------------------------------------------------------+
|                                                                   |
|  Root<Employee> employee                                          |
|       |                                                           |
|       |-- .join("department") --> Join<Employee, Department>       |
|       |       |                                                   |
|       |       |-- .join("company") --> Join<Department, Company>   |
|       |                                                           |
|       |-- .join("projects") --> Join<Employee, Project>            |
|       |                                                           |
|       |-- .get("name") --> Path<String>  (for WHERE/SELECT)       |
|       |-- .get("salary") --> Path<BigDecimal>                     |
|                                                                   |
|  Each join produces a new root you can navigate further           |
+-------------------------------------------------------------------+
```

Fetch Joins (Solving N+1)

```java
// Fetch join - loads the relationship eagerly in the same query
Root<Employee> employee = cq.from(Employee.class);
employee.fetch("department", JoinType.LEFT);

// This prevents the N+1 problem:
// Instead of 1 query for employees + N queries for departments,
// it loads everything in a single query
```

Aggregate Functions and Grouping

Aggregate Functions

```java
CriteriaBuilder cb = em.getCriteriaBuilder();

// Count
CriteriaQuery<Long> countQuery = cb.createQuery(Long.class);
Root<Employee> root = countQuery.from(Employee.class);
countQuery.select(cb.count(root));
countQuery.where(cb.equal(root.get("active"), true));
Long count = em.createQuery(countQuery).getSingleResult();

// Sum
CriteriaQuery<BigDecimal> sumQuery = cb.createQuery(BigDecimal.class);
```

```
Root<Employee> sumRoot = sumQuery.from(Employee.class);
sumQuery.select(cb.sum(sumRoot.get("salary")));
BigDecimal totalSalary = em.createQuery(sumQuery).getSingleResult();

// Average
CriteriaQuery<Double> avgQuery = cb.createQuery(Double.class);
Root<Employee> avgRoot = avgQuery.from(Employee.class);
avgQuery.select(cb.avg(avgRoot.get("salary")));
Double avgSalary = em.createQuery(avgQuery).getSingleResult();

// Min / Max
CriteriaQuery<BigDecimal> maxQuery = cb.createQuery(BigDecimal.class);
Root<Employee> maxRoot = maxQuery.from(Employee.class);
maxQuery.select(cb.max(maxRoot.get("salary")));
BigDecimal maxSalary = em.createQuery(maxQuery).getSingleResult();
```

GROUP BY and HAVING

```java
// Department salary report: average salary per department, only where avg > 70000
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Object[]> cq = cb.createQuery(Object[].class);
Root<Employee> employee = cq.from(Employee.class);
Join<Employee, Department> dept = employee.join("department");

cq.multiselect(
    dept.get("name"),                        // department name
    cb.count(employee),                      // employee count
    cb.avg(employee.get("salary")),          // average salary
    cb.sum(employee.get("salary"))           // total salary
);

cq.groupBy(dept.get("name"));
cq.having(cb.gt(cb.avg(employee.get("salary")), 70000));
cq.orderBy(cb.desc(cb.avg(employee.get("salary"))));

List<Object[]> results = em.createQuery(cq).getResultList();
for (Object[] row : results) {
    System.out.printf("Dept: %s, Count: %d, Avg: %.2f, Total: %s%n",
        row[0], row[1], row[2], row[3]);
}

// Generated SQL:
// SELECT d.name, COUNT(e.id), AVG(e.salary), SUM(e.salary)
// FROM employees e JOIN departments d ON e.department_id = d.id
// GROUP BY d.name
// HAVING AVG(e.salary) > 70000
// ORDER BY AVG(e.salary) DESC
```

JPA Metamodel — Compile-Time Type Safety

String-based field names (`root.get("salary")`) can have typos. The JPA Metamodel generates static classes that provide compile-time checking.

What the Metamodel Looks Like

For each entity class, a corresponding metamodel class is generated:

```
Entity Class:                          Generated Metamodel Class:
+-----------------------------+      +-----------------------------+
| @Entity                     |      | @StaticMetamodel(           |
| public class Employee {     |      |   Employee.class)           |
|   @Id                       |      | public abstract class       |
|   private Long id;          | ---> |   Employee_ {               |
|   private String name;      |      |   static volatile           |
|   private BigDecimal salary;|      |     SingularAttribute       |
|   private boolean active;   |      |       <Employee, Long> id;  |
|   private Department dept;  |      |   static volatile           |
| }                           |      |     SingularAttribute       |
+-----------------------------+      |       <Employee, String>    |
                                     |       name;                 |
                                     |   static volatile           |
                                     |     SingularAttribute       |
                                     |       <Employee, BigDecimal>|
                                     |       salary;               |
                                     |   // ... more fields        |
                                     | }                           |
                                     +-----------------------------+
```

Setting Up Metamodel Generation

Add the Hibernate JPA Metamodel Generator to your `pom.xml`:

```xml
<dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-jpamodelgen</artifactId>
    <scope>provided</scope>
</dependency>
```

Spring Boot's dependency management handles the version. The `provided` scope means it is only needed during compilation, not at runtime.

After adding this dependency and building (`mvn compile`), you will find generated classes like `Employee_` in your `target/generated-sources` directory.

Using the Metamodel

```java
// Without metamodel (string-based, error-prone):
root.get("salary")            // typo in "salary" only caught at runtime
root.get("department")        // type unknown to compiler

// With metamodel (type-safe, compile-time checked):
root.get(Employee_.salary)    // compiler verifies "salary" exists and is BigDecimal
root.get(Employee_.department) // compiler knows this is a Department
```

Full example:

```java
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Employee> cq = cb.createQuery(Employee.class);
Root<Employee> root = cq.from(Employee.class);

// All field references are compile-time checked
cq.select(root)
  .where(cb.and(
      cb.equal(root.get(Employee_.active), true),
      cb.greaterThan(root.get(Employee_.salary), new BigDecimal("80000")),
      cb.like(root.get(Employee_.name), "%Alice%")
  ))
  .orderBy(cb.desc(root.get(Employee_.salary)));

List<Employee> results = em.createQuery(cq).getResultList();
```

If you rename `salary` to `compensation` in the `Employee` entity, `Employee_.salary` immediately causes a compilation error, telling you exactly where to update.

Spring Data Specifications

The Criteria API is powerful but verbose. Spring Data Specifications provide a cleaner interface by wrapping Criteria API predicates into reusable, composable objects.

The Specification Interface

```java
// JPA Standard (what Spring wraps)
public interface Specification<T> extends Serializable {
    Predicate toPredicate(Root<T> root,
```

```
                            CriteriaQuery<?> query,
                            CriteriaBuilder criteriaBuilder);
}
```

```
How Specifications Work:
+-------------------------------------------------------------------+
|                                                                   |
|  Specification<Employee>                                          |
|  +----------------------------+                                   |
|  | toPredicate(root, cq, cb)  |                                   |
|  | {                          |                                   |
|  |   return cb.equal(         |----> Returns ONE Predicate        |
|  |     root.get("active"),    |                                   |
|  |     true);                 |                                   |
|  | }                          |                                   |
|  +----------------------------+                                   |
|                                                                   |
|  Multiple Specifications can be combined:                         |
|  spec1.and(spec2).and(spec3)  ----> AND chain of predicates       |
|  spec1.or(spec2)              ----> OR combination                |
|                                                                   |
|  Repository uses them:                                            |
|  repository.findAll(spec)     ----> Executes with combined WHERE  |
|  repository.findAll(spec, pageable) ----> With pagination         |
+-------------------------------------------------------------------+
```

Setting Up: Extend JpaSpecificationExecutor

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long>,
                                            JpaSpecificationExecutor<Employee> {
    // JpaSpecificationExecutor adds:
    // List<T> findAll(Specification<T> spec)
    // Page<T> findAll(Specification<T> spec, Pageable pageable)
    // long count(Specification<T> spec)
    // boolean exists(Specification<T> spec)
    // Optional<T> findOne(Specification<T> spec)
}
```

Creating Specifications

```java
public class EmployeeSpecifications {

    public static Specification<Employee> isActive() {
```

```
        return (root, query, cb) -> cb.equal(root.get("active"), true);
    }

    public static Specification<Employee> hasName(String name) {
        return (root, query, cb) ->
            cb.like(cb.lower(root.get("name")),
                    "%" + name.toLowerCase() + "%");
    }

    public static Specification<Employee> inDepartment(String deptName) {
        return (root, query, cb) ->
            cb.equal(root.get("department").get("name"), deptName);
    }

    public static Specification<Employee> salaryGreaterThan(BigDecimal min) {
        return (root, query, cb) ->
            cb.greaterThan(root.get("salary"), min);
    }

    public static Specification<Employee> salaryBetween(BigDecimal min,
                                                        BigDecimal max) {
        return (root, query, cb) -> cb.between(root.get("salary"), min, max);
    }

    public static Specification<Employee> hiredAfter(LocalDate date) {
        return (root, query, cb) ->
            cb.greaterThan(root.get("hireDate"), date);
    }

    public static Specification<Employee> hiredInYear(int year) {
        return (root, query, cb) -> {
            LocalDate start = LocalDate.of(year, 1, 1);
            LocalDate end = LocalDate.of(year, 12, 31);
            return cb.between(root.get("hireDate"), start, end);
        };
    }
}
```

Combining Specifications

```java
// Import static for cleaner code
import static com.example.spec.EmployeeSpecifications.*;

// Simple usage
List<Employee> activeEmployees = repository.findAll(isActive());

// Combine with AND
List<Employee> activeEngineers = repository.findAll(
    isActive().and(inDepartment("Engineering"))
);
```

```
// Combine with AND and OR
Specification<Employee> spec = isActive()
    .and(salaryGreaterThan(new BigDecimal("70000")))
    .and(inDepartment("Engineering").or(inDepartment("Product")));

List<Employee> results = repository.findAll(spec);

// With pagination
Page<Employee> page = repository.findAll(spec, PageRequest.of(0, 20));
```

```
Specification Composition:
+-------------------------------------------------------------------+
|                                                                   |
|  isActive()                                                       |
|      .and(salaryGreaterThan(70000))                               |
|      .and(inDepartment("Engineering").or(inDepartment("Product")))|
|                                                                   |
|  Generates:                                                       |
|  WHERE active = true                                              |
|    AND salary > 70000                                             |
|    AND (department.name = 'Engineering'                           |
|         OR department.name = 'Product')                           |
+-------------------------------------------------------------------+
```

Dynamic Search with Specifications

This is where Specifications truly shine — building queries from user input:

```java
@Service
public class EmployeeSearchService {

    private final EmployeeRepository repository;

    public EmployeeSearchService(EmployeeRepository repository) {
        this.repository = repository;
    }

    public Page<Employee> search(EmployeeSearchCriteria criteria, Pageable pageable) {

        Specification<Employee> spec = Specification.where(null); // start with no filter

        if (criteria.getName() != null && !criteria.getName().isBlank()) {
            spec = spec.and(EmployeeSpecifications.hasName(criteria.getName()));
        }
```

```
        if (criteria.getDepartment() != null) {
            spec = spec.and(EmployeeSpecifications.inDepartment(criteria.getDepartment(...
        }

        if (criteria.getMinSalary() != null) {
            spec = spec.and(EmployeeSpecifications.salaryGreaterThan(criteria.getMinSal...
        }

        if (criteria.getMaxSalary() != null) {
            spec = spec.and(EmployeeSpecifications.salaryLessThan(criteria.getMaxSalary...
        }

        if (criteria.getActiveOnly() != null && criteria.getActiveOnly()) {
            spec = spec.and(EmployeeSpecifications.isActive());
        }

        if (criteria.getHiredAfter() != null) {
            spec = spec.and(EmployeeSpecifications.hiredAfter(criteria.getHiredAfter()));
        }

        return repository.findAll(spec, pageable);
    }
}
```

```java
// The search criteria DTO
public class EmployeeSearchCriteria {
    private String name;
    private String department;
    private BigDecimal minSalary;
    private BigDecimal maxSalary;
    private Boolean activeOnly;
    private LocalDate hiredAfter;

    // getters and setters
}
```

```java
// Controller usage
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    private final EmployeeSearchService searchService;

    @GetMapping("/search")
    public Page<Employee> search(
            @RequestParam(required = false) String name,
            @RequestParam(required = false) String department,
            @RequestParam(required = false) BigDecimal minSalary,
            @RequestParam(required = false) BigDecimal maxSalary,
```

```
            @RequestParam(required = false) Boolean activeOnly,
            @RequestParam(required = false) LocalDate hiredAfter,
            Pageable pageable) {

        EmployeeSearchCriteria criteria = new EmployeeSearchCriteria();
        criteria.setName(name);
        criteria.setDepartment(department);
        criteria.setMinSalary(minSalary);
        criteria.setMaxSalary(maxSalary);
        criteria.setActiveOnly(activeOnly);
        criteria.setHiredAfter(hiredAfter);

        return searchService.search(criteria, pageable);
    }
}
```

```
Dynamic Query Building Flow:
+-------------------------------------------------------------------+
|                                                                    |
|  User request: /api/employees/search?name=Ali&minSalary=60000    |
|                                                                    |
|  Step 1: Start with empty spec (WHERE true)                        |
|                                                                    |
|  Step 2: name="Ali" is present                                    |
|          --> spec AND hasName("Ali")                               |
|          WHERE LOWER(name) LIKE '%ali%'                            |
|                                                                    |
|  Step 3: department=null, skip                                     |
|                                                                    |
|  Step 4: minSalary=60000 is present                               |
|          --> spec AND salaryGreaterThan(60000)                     |
|          WHERE LOWER(name) LIKE '%ali%' AND salary > 60000         |
|                                                                    |
|  Step 5: Execute with pagination                                   |
|          repository.findAll(spec, pageable)                        |
|                                                                    |
+-------------------------------------------------------------------+
```

Specifications with Joins

When you need to filter by related entities:

```java
public class EmployeeSpecifications {

    // Join to department and filter by department name
```

```
    public static Specification<Employee> inDepartment(String deptName) {
        return (root, query, cb) -> {
            Join<Employee, Department> dept = root.join("department");
            return cb.equal(dept.get("name"), deptName);
        };
    }

    // Join to projects (many-to-many) and filter
    public static Specification<Employee> onProject(String projectName) {
        return (root, query, cb) -> {
            Join<Employee, Project> projects = root.join("projects");
            return cb.equal(projects.get("name"), projectName);
        };
    }

    // Avoid duplicate results with distinct
    public static Specification<Employee> onProjectDistinct(String projectName) {
        return (root, query, cb) -> {
            query.distinct(true);  // Prevent duplicates from joins
            Join<Employee, Project> projects = root.join("projects");
            return cb.equal(projects.get("name"), projectName);
        };
    }
}
```

Criteria API vs JPQL vs Native SQL

```
+-------------------------------------------------------------------+
| Feature          | Derived    | JPQL      | Criteria  | Native   |
|                  | Methods    |           | API/Spec  | SQL      |
|-------------------------------------------------------------------|
| Syntax           | Method     | String    | Java API  | String   |
|                  | names      |           |           |          |
| Type safety      | High       | Low       | High      | None     |
|                  |            |           | (w/meta)  |          |
| Dynamic queries  | No         | Ugly      | Elegant   | Ugly     |
| Reusable pieces  | No         | No        | Yes       | No       |
|                  |            |           | (specs)   |          |
| Readability      | Excellent  | Good      | Verbose   | Good     |
|                  | (simple)   |           |           |          |
| DB portability   | Yes        | Yes       | Yes       | No       |
| Complex queries  | Very       | Good      | Good      | Full     |
|                  | limited    |           |           | power    |
| Learning curve   | Easy       | Medium    | Steep     | Medium   |
|                  |            |           |           |          |
| Best for         | Simple     | Fixed     | Dynamic   | DB-      |
|                  | lookups    | queries   | filters   | specific |
|                  |            |           | & search  | features |
```

```
+-----------------------------------------------------------------+
```

Common Mistakes

1. Using string field names instead of metamodel: `root.get("salary")` works but is error-prone. Use `root.get(Employee_.salary)` with the JPA Metamodel for compile-time safety.
2. Forgetting distinct with joins: Joining a collection (`@OneToMany`, `@ManyToMany`) can produce duplicate root entities. Add `query.distinct(true)` in your specification.
3. Creating new joins in each specification: If two specifications both join the same relationship, you get two joins in the SQL. Extract the join into a shared specification or check if the join already exists.
4. Not using Specification.where(null) for empty start: When building dynamic specs, start with `Specification.where(null)` so `.and()` chains work even when all filters are null.
5. Overusing Criteria API for simple queries: For straightforward queries, derived methods or JPQL are simpler and more readable. Reserve Criteria API for dynamic filtering scenarios.
6. Forgetting to extend JpaSpecificationExecutor: Specifications only work if your repository extends `JpaSpecificationExecutor<T>` in addition to `JpaRepository<T, ID>`.

Best Practices

1. Use Specifications for search/filter endpoints: Any endpoint where users can optionally filter by multiple fields is a perfect use case for Specifications.
2. Group specifications in a dedicated class: Create `EmployeeSpecifications`, `OrderSpecifications`, etc. This keeps them organized and reusable.
3. Enable the JPA Metamodel: The `hibernate-jpamodelgen` dependency is lightweight and catches field name errors at compile time instead of runtime.
4. Combine with pagination: Specifications work seamlessly with `Pageable`. Always return `Page<T>` for search endpoints.
5. Keep specifications focused: Each specification should represent one filter condition. Complex logic comes from combining simple specifications, not from writing one giant specification.

6. Use the right tool for the job: Derived methods for simple lookups, JPQL for fixed queries with joins, Specifications for dynamic search, native SQL for database-specific features.

Summary

In this chapter, you learned how to build type-safe, dynamic queries:

- Criteria API lets you build queries programmatically using `CriteriaBuilder`, `CriteriaQuery`, `Root`, and `Predicate`. It eliminates string-based query errors.
- The main components: `CriteriaBuilder` creates predicates and expressions, `CriteriaQuery` represents the query structure, `Root` defines the FROM entity, and `Predicate` defines WHERE conditions.
- JPA Metamodel (`Employee_`) provides compile-time type safety by generating static field references from your entity classes.
- Spring Data Specifications wrap the Criteria API in a clean, composable interface. Each specification is a single predicate that can be combined with `.and()` and `.or()`.
- Dynamic search is the primary use case — building queries from optional user-provided filter criteria without ugly string concatenation.
- Joins in Criteria API use `root.join("relationship")` and `root.fetch("relationship")` for eager loading.

Interview Questions

Q1: What is the main advantage of the Criteria API over JPQL?

The Criteria API excels at dynamic query construction. When a query's conditions depend on user input (optional filters in a search form), the Criteria API builds the query programmatically without string concatenation. Combined with the JPA Metamodel, it also provides compile-time type safety.

Q2: What is a Spring Data Specification?

A Specification is an implementation of the `Specification<T>` interface that encapsulates a single query predicate. It takes a `Root`, `CriteriaQuery`, and `CriteriaBuilder` and returns a `Predicate`. Specifications can be combined using `.and()` and `.or()` to create complex queries from simple, reusable pieces.

Q3: What is the JPA Metamodel and why should you use it?

The JPA Metamodel generates static classes (like `Employee_`) that contain typed references to entity fields. Instead of `root.get("salary")` (where a typo only fails at runtime), you use `root.get(Employee_.salary)` which fails at compile time if the field does not exist or the type is wrong.

Q4: How do you handle optional search criteria with Specifications?

Start with `Specification.where(null)` as a base. For each non-null criterion, chain with `.and(someSpec)`. Null specifications are ignored, so only provided criteria are added to the query. This avoids `if-else` chains building JPQL strings.

Q5: What is JpaSpecificationExecutor?

It is a Spring Data interface that your repository must extend to use Specifications. It adds methods like `findAll(Specification<T>)`, `findAll(Specification<T>, Pageable)`, `count(Specification<T>)`, and `exists(Specification<T>)`.

Q6: When would you use the Criteria API vs JPQL?

Use Criteria API (via Specifications) for dynamic queries with optional filters, search endpoints, and when type safety is important. Use JPQL for fixed queries with known structure, especially when readability matters more than dynamism.

Practice Exercises

Exercise 1: Basic Criteria Query Write a Criteria API query that finds all employees with salary between two values, ordered by salary descending. Execute it using EntityManager.

Exercise 2: Specification Factory Create a `ProductSpecifications` class for a `Product` entity with fields (name, category, price, inStock). Write specifications for: `hasCategory`, `priceBetween`, `isInStock`, `nameContains`. Test combining them.

Exercise 3: Dynamic Product Search Build a `ProductSearchService` that accepts an optional name, category, min price, max price, and inStock flag. Use Specifications to build the query dynamically. Return paginated results.

Exercise 4: Metamodel Setup Add the `hibernate-jpamodelgen` dependency to a project and refactor string-based Criteria queries to use the generated metamodel classes. Verify that renaming a field causes a compile error.

Exercise 5: Complex Specification Write a specification that finds employees who are either (a) in the "Engineering" department with salary above 90,000 or (b) in the "Sales" department with salary above 70,000, and who were hired in the last 2 years. Combine at least 3 specifications.

What Is Next?

In the next chapter, we will explore Pagination, Sorting, and Query Optimization. You will learn how to efficiently return subsets of data with `Pageable`, `Page`, and `Slice`, sort by multiple fields, write custom count queries, and use `@EntityGraph` to control fetching at the query level — essential skills for building performant APIs.

Chapter 11: Pagination, Sorting, and Query Optimization

Learning Goals

By the end of this chapter, you will be able to:

- Understand why pagination is essential for performance and user experience
- Use Pageable, PageRequest, and Sort to paginate and sort query results
- Distinguish between Page, Slice, and List return types
- Implement pagination with derived methods, JPQL, native SQL, and Specifications
- Sort by multiple fields and handle custom sort mappings
- Write custom count queries for complex paginated queries
- Use @EntityGraph to optimize fetching at the query level
- Solve the N+1 problem with fetch joins and entity graphs
- Apply @QueryHints for query plan caching

Why Pagination Matters

Without pagination, a query returns every matching row. This works when you have 50 employees but fails catastrophically with 500,000.

```
Without Pagination:
+-------------------------------------------------------------------+
|                                                                   |
|  Client                        Server                  Database    |
|    |                             |                        |        |
|    |  GET /api/employees         |                        |        |
```

```
|    |--------------------------->|                        |        |
|    |                            |  SELECT * FROM         |        |
|    |                            |  employees             |        |
|    |                            |------------------------>|        |
|    |                            |                        |        |
|    |                            |  500,000 rows          |        |
|    |                            |<------------------------|        |
|    |                            |                        |        |
|    |  500,000 JSON objects      |                        |        |
|    |  (50 MB response!)         |                        |        |
|    |<---------------------------|                        |        |
|    |                            |                        |        |
|  Problems:                                                         |
|  - Database loads ALL rows into memory                             |
|  - Server converts ALL rows to JSON                                |
|  - Network transfers 50 MB                                         |
|  - Client tries to render 500,000 items                            |
|  - Everyone is unhappy                                             |
+--------------------------------------------------------------------+

With Pagination:
+--------------------------------------------------------------------+
|                                                                    |
|  Client                       Server                  Database     |
|    |                            |                        |        |
|    |  GET /api/employees        |                        |        |
|    |    ?page=0&size=20         |                        |        |
|    |--------------------------->|                        |        |
|    |                            |  SELECT * FROM         |        |
|    |                            |  employees             |        |
|    |                            |  LIMIT 20 OFFSET 0     |        |
|    |                            |------------------------>|        |
|    |                            |                        |        |
|    |                            |  20 rows               |        |
|    |                            |<------------------------|        |
|    |                            |                        |        |
|    |  20 JSON objects           |                        |        |
|    |  + metadata (total: 500K)  |                        |        |
|    |<---------------------------|                        |        |
|    |                            |                        |        |
|  Result:                                                           |
|  - Database reads only 20 rows                                     |
|  - Fast response, small payload                                    |
|  - Client renders one page at a time                               |
+--------------------------------------------------------------------+
```

Pageable, PageRequest, and Sort

Spring Data JPA provides three core types for pagination:

```
Pagination Types:
+-------------------------------------------------------------------+
|                                                                   |
|  Pageable (interface)                                             |
|  +----------------------------+                                   |
|  | Defines WHAT to retrieve:  |                                   |
|  | - Page number (0-based)    |                                   |
|  | - Page size                |                                   |
|  | - Sort order               |                                   |
|  +----------------------------+                                   |
|           |                                                       |
|  PageRequest (implementation)                                     |
|  +----------------------------+                                   |
|  | Creates a Pageable:        |                                   |
|  | PageRequest.of(0, 20)      |                                   |
|  | PageRequest.of(0, 20,      |                                   |
|  |   Sort.by("name"))         |                                   |
|  +----------------------------+                                   |
|           |                                                       |
|  Sort                                                             |
|  +----------------------------+                                   |
|  | Defines ordering:          |                                   |
|  | Sort.by("name")            |                                   |
|  | Sort.by("name").ascending()|                                   |
|  | Sort.by("salary").desc()   |                                   |
|  | Sort.by("dept", "name")    |                                   |
|  +----------------------------+                                   |
|                                                                   |
+-------------------------------------------------------------------+
```

Creating PageRequest

```java
// Page 0 (first page), 20 items per page, no sorting
Pageable pageable = PageRequest.of(0, 20);

// Page 2 (third page), 10 items per page
Pageable pageable = PageRequest.of(2, 10);

// With sorting
Pageable pageable = PageRequest.of(0, 20, Sort.by("name"));

// With descending sort
Pageable pageable = PageRequest.of(0, 20, Sort.by("salary").descending());

// With direction parameter
Pageable pageable = PageRequest.of(0, 20, Sort.Direction.DESC, "salary");

// With multiple sort fields
Pageable pageable = PageRequest.of(0, 20,
    Sort.by("department.name").ascending()
        .and(Sort.by("salary").descending()));
```

```
Page Numbering (0-based):
+-----------------------------------------------------------------+
|                                                                  |
|  Data: [A, B, C, D, E, F, G, H, I, J, K, L, M, N, O]           |
|  Page size: 5                                                    |
|                                                                  |
|  Page 0: [A, B, C, D, E]    (offset 0)                          |
|  Page 1: [F, G, H, I, J]    (offset 5)                          |
|  Page 2: [K, L, M, N, O]    (offset 10)                         |
|                                                                  |
|  Total elements: 15                                              |
|  Total pages: 3                                                  |
|  Page 0 is the first page (not page 1!)                          |
+-----------------------------------------------------------------+
```

Sort API

Spring Data's `Sort` class provides a fluent API for defining sort orders:

```java
// Single field, ascending (default)
Sort sort = Sort.by("name");

// Single field, explicit direction
Sort sort = Sort.by(Sort.Direction.ASC, "name");
Sort sort = Sort.by(Sort.Direction.DESC, "salary");

// Fluent ascending / descending
Sort sort = Sort.by("salary").descending();
Sort sort = Sort.by("name").ascending();

// Multiple fields
Sort sort = Sort.by("department").ascending()
               .and(Sort.by("salary").descending());

// Shorthand for multiple fields, same direction
Sort sort = Sort.by(Sort.Direction.ASC, "lastName", "firstName");

// Unsorted
Sort sort = Sort.unsorted();

// Null handling (JPA standard)
Sort sort = Sort.by("email")
    .ascending()
    .nullsLast();  // or .nullsFirst()
```

Sort with Repository Methods

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Sort parameter on derived methods
    List<Employee> findByActive(boolean active, Sort sort);

    // Sort parameter with JPQL
    @Query("SELECT e FROM Employee e WHERE e.department.name = :dept")
    List<Employee> findByDepartment(@Param("dept") String dept, Sort sort);
}

// Usage
List<Employee> sorted = repository.findByActive(true,
    Sort.by("salary").descending());
```

Page vs Slice vs List

Spring Data JPA offers three return types for paginated queries, each with different performance characteristics:

```
Return Type Comparison:
+-------------------------------------------------------------------+
|                                                                   |
|  Page<T>                                                          |
|  +--------------------------------+                               |
|  | Content: List<T>               |  Executes TWO queries:       |
|  | Page number: 0                 |  1. SELECT ... LIMIT/OFFSET  |
|  | Page size: 20                  |  2. SELECT COUNT(*)          |
|  | Total elements: 1,542          |                              |
|  | Total pages: 78                |  Use when: UI needs total    |
|  | Has next: true                 |  page count (page 1 of 78)  |
|  | Has previous: false            |                              |
|  +--------------------------------+                               |
|                                                                   |
|  Slice<T>                                                         |
|  +--------------------------------+                               |
|  | Content: List<T>               |  Executes ONE query:         |
|  | Page number: 0                 |  SELECT ... LIMIT (size+1)   |
|  | Page size: 20                  |  Fetches 21 rows, returns 20 |
|  | Has next: true                 |  Checks if 21st exists       |
|  | Has previous: false            |                              |
|  | (NO total count!)              |  Use when: infinite scroll   |
|  +--------------------------------+  or "Load More" button       |
|                                                                   |
```

```
|  List<T>                                                          |
|  +--------------------------------+                               |
|  | Just the data, no metadata      |  Executes ONE query:         |
|  | [item1, item2, ..., item20]     |  SELECT ... LIMIT/OFFSET     |
|  |                                 |                              |
|  | (NO count, NO page info)        |  Use when: you only need     |
|  +--------------------------------+  the data subset             |
|                                                                   |
+-------------------------------------------------------------------+
```

Using Page

```java
// Repository
Page<Employee> findByActive(boolean active, Pageable pageable);

// Service
Pageable pageable = PageRequest.of(0, 20, Sort.by("name"));
Page<Employee> page = repository.findByActive(true, pageable);

// Page metadata
page.getContent();           // List<Employee> - the 20 employees
page.getNumber();            // 0 - current page number
page.getSize();              // 20 - page size
page.getTotalElements();     // 1542 - total matching rows
page.getTotalPages();        // 78 - total pages (1542 / 20, rounded up)
page.hasNext();              // true - there are more pages
page.hasPrevious();          // false - this is the first page
page.isFirst();              // true
page.isLast();               // false
page.getNumberOfElements();  // 20 - elements on THIS page
```

Using Slice

```java
// Repository
Slice<Employee> findByActive(boolean active, Pageable pageable);

// Service
Slice<Employee> slice = repository.findByActive(true, PageRequest.of(0, 20));

// Slice metadata (NO totalElements or totalPages)
slice.getContent();           // List<Employee>
slice.getNumber();            // 0
slice.getSize();              // 20
slice.hasNext();              // true (knows because it fetched 21 rows)
slice.hasPrevious();          // false
// slice.getTotalElements()   // DOES NOT EXIST on Slice!
// slice.getTotalPages()      // DOES NOT EXIST on Slice!
```

When to Use Which

```
Decision Guide:
+-------------------------------------------------------------------+
|                                                                    |
|  Need total count for "Page X of Y" display?                      |
|      |                                                             |
|   YES --> Use Page<T>                                              |
|      |    (extra COUNT query is acceptable)                        |
|      |                                                             |
|    NO                                                              |
|      |                                                             |
|      v                                                             |
|  Need "Load More" / "Next" button?                                |
|      |                                                             |
|   YES --> Use Slice<T>                                             |
|      |    (avoids COUNT query, knows if more data exists)          |
|      |                                                             |
|    NO                                                              |
|      |                                                             |
|      v                                                             |
|  Just need the data?                                               |
|      |                                                             |
|   YES --> Use List<T> with Pageable                                |
|           (simplest, no metadata overhead)                         |
+-------------------------------------------------------------------+
|                                                                    |
|  Performance Impact:                                               |
|  Table with 1M rows, active = true matches 500K:                  |
|                                                                    |
|  Page<T>:  SELECT ... LIMIT 20    (~1ms)                           |
|          + SELECT COUNT(*) WHERE active = true  (~200ms)           |
|          = ~201ms total                                            |
|                                                                    |
|  Slice<T>: SELECT ... LIMIT 21    (~1ms)                           |
|          = ~1ms total                                              |
|                                                                    |
|  The COUNT query can be expensive on large tables!                 |
+-------------------------------------------------------------------+
```

Pagination with Different Query Types

Derived Methods

```java
// Page return
Page<Employee> findByActive(boolean active, Pageable pageable);

// Slice return
Slice<Employee> findByDepartmentName(String deptName, Pageable pageable);

// List return (still paginated — uses LIMIT/OFFSET — but no metadata)
List<Employee> findBySalaryGreaterThan(BigDecimal salary, Pageable pageable);

// With Sort only (no pagination)
List<Employee> findByActive(boolean active, Sort sort);
```

JPQL with Pagination

```java
// JPQL with Page return — count query derived automatically
@Query("SELECT e FROM Employee e WHERE e.active = true")
Page<Employee> findActiveEmployees(Pageable pageable);

// JPQL with explicit count query (for complex queries)
@Query(value = "SELECT e FROM Employee e JOIN e.department d WHERE d.name = :dept",
       countQuery = "SELECT COUNT(e) FROM Employee e JOIN e.department d WHERE d.name =...
Page<Employee> findByDept(@Param("dept") String dept, Pageable pageable);

// Slice (no count needed)
@Query("SELECT e FROM Employee e WHERE e.salary > :min")
Slice<Employee> findHighEarners(@Param("min") BigDecimal min, Pageable pageable);
```

Native SQL with Pagination

```java
// Native SQL — MUST provide countQuery for Page return
@Query(value = "SELECT * FROM employees WHERE active = true",
       countQuery = "SELECT COUNT(*) FROM employees WHERE active = true",
       nativeQuery = true)
Page<Employee> findActiveNative(Pageable pageable);

// Slice return — no countQuery needed
@Query(value = "SELECT * FROM employees WHERE salary > :min",
       nativeQuery = true)
Slice<Employee> findHighEarnersNative(@Param("min") BigDecimal min, Pageable pageable);
```

Specifications with Pagination

```java
// Already built in to JpaSpecificationExecutor
Page<Employee> findAll(Specification<Employee> spec, Pageable pageable);

// Usage
Specification<Employee> spec = EmployeeSpecifications.isActive()
    .and(EmployeeSpecifications.salaryGreaterThan(new BigDecimal("50000")));

Page<Employee> page = repository.findAll(spec,
    PageRequest.of(0, 20, Sort.by("salary").descending()));
```

Custom Count Queries

For complex queries with joins or subqueries, the auto-derived count query might be incorrect or slow. You can provide your own:

```java
// Problem: Auto-derived count with FETCH JOIN fails
@Query("SELECT e FROM Employee e JOIN FETCH e.department WHERE e.active = true")
Page<Employee> findActiveWithDept(Pageable pageable);
// ERROR! Hibernate cannot derive a COUNT query from a FETCH JOIN

// Solution: Provide explicit countQuery
@Query(value = "SELECT e FROM Employee e JOIN FETCH e.department WHERE e.active = true",
       countQuery = "SELECT COUNT(e) FROM Employee e WHERE e.active = true")
Page<Employee> findActiveWithDept(Pageable pageable);
```

```
Why Custom Count Queries Are Needed:
+-----------------------------------------------------------------+
|                                                                  |
|  Auto-derived count:                                             |
|  Query:  SELECT e FROM Employee e                                |
|          JOIN FETCH e.department                                  |
|          WHERE e.active = true                                   |
|                                                                  |
|  Hibernate tries: SELECT COUNT(e) FROM Employee e                |
|                   JOIN FETCH e.department    <-- ERROR!          |
|                   WHERE e.active = true                          |
|                                                                  |
|  FETCH JOIN does not make sense in a COUNT query.                |
|  Hibernate cannot strip it automatically for native queries.     |
|                                                                  |
|  Custom count:                                                   |
|  countQuery = "SELECT COUNT(e) FROM Employee e                   |
|                WHERE e.active = true"                            |
|  - Simple, fast, no join needed for counting                     |
+-----------------------------------------------------------------+
```

@EntityGraph — Query-Level Fetch Control

`@EntityGraph` (JPA 2.1 standard) lets you control which relationships are eagerly loaded per query, without changing the entity's default fetch type.

```
Fetching Without @EntityGraph (N+1 Problem):
+-------------------------------------------------------------------+
|                                                                    |
|  Query: findAll()                                                  |
|                                                                    |
|  SQL 1:  SELECT * FROM employees                  (1 query)        |
|  Result: 100 employees                                             |
|                                                                    |
|  For EACH employee, accessing employee.getDepartment():            |
|  SQL 2:  SELECT * FROM departments WHERE id = 1   (query 2)       |
|  SQL 3:  SELECT * FROM departments WHERE id = 2   (query 3)       |
|  SQL 4:  SELECT * FROM departments WHERE id = 1   (query 4)       |
|  ...                                                               |
|  SQL 101: SELECT * FROM departments WHERE id = 5  (query 101)     |
|                                                                    |
|  Total: 1 + 100 = 101 queries!  (the N+1 problem)                 |
+-------------------------------------------------------------------+

Fetching With @EntityGraph:
+-------------------------------------------------------------------+
|                                                                    |
|  Query: findAll() with @EntityGraph({"department"})                |
|                                                                    |
|  SQL 1:  SELECT e.*, d.*                                           |
|          FROM employees e                                          |
|          LEFT JOIN departments d ON e.department_id = d.id         |
|                                                        (1 query)   |
|  Result: 100 employees with departments already loaded             |
|                                                                    |
|  Total: 1 query! Everything loaded in a single JOIN.               |
+-------------------------------------------------------------------+
```

Using @EntityGraph on Repository Methods

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Load department eagerly for this specific query
```

```
    @EntityGraph(attributePaths = {"department"})
    List<Employee> findByActive(boolean active);

    // Load multiple relationships
    @EntityGraph(attributePaths = {"department", "projects"})
    List<Employee> findAll();

    // Combine with JPQL
    @EntityGraph(attributePaths = {"department"})
    @Query("SELECT e FROM Employee e WHERE e.salary > :min")
    List<Employee> findHighEarnersWithDept(@Param("min") BigDecimal min);

    // Combine with pagination
    @EntityGraph(attributePaths = {"department"})
    Page<Employee> findByActive(boolean active, Pageable pageable);

    // Nested paths (department's company)
    @EntityGraph(attributePaths = {"department", "department.company"})
    List<Employee> findAll(Sort sort);
}
```

@NamedEntityGraph (on the Entity)

For reusable fetch plans, define the graph on the entity class:

```java
@Entity
@Table(name = "employees")
@NamedEntityGraph(
    name = "Employee.withDepartment",
    attributeNodes = @NamedAttributeNode("department")
)
@NamedEntityGraph(
    name = "Employee.withDepartmentAndProjects",
    attributeNodes = {
        @NamedAttributeNode("department"),
        @NamedAttributeNode("projects")
    }
)
@NamedEntityGraph(
    name = "Employee.full",
    attributeNodes = {
        @NamedAttributeNode(value = "department",
            subgraph = "department-with-company"),
        @NamedAttributeNode("projects")
    },
    subgraphs = @NamedSubgraph(
        name = "department-with-company",
        attributeNodes = @NamedAttributeNode("company")
    )
)
```

```
public class Employee {
    // ...
}
```

```java
// Reference by name in repository
@EntityGraph("Employee.withDepartment")
List<Employee> findByActive(boolean active);

@EntityGraph("Employee.full")
Optional<Employee> findById(Long id);
```

```
@EntityGraph Types:
+-------------------------------------------------------------------+
|                                                                    |
|  FETCH graph (default):                                            |
|  - Attributes in the graph: loaded EAGERLY                        |
|  - Attributes NOT in the graph: loaded LAZILY                     |
|  - Overrides the entity's default fetch type                      |
|                                                                    |
|  LOAD graph:                                                       |
|  - Attributes in the graph: loaded EAGERLY                        |
|  - Attributes NOT in the graph: use entity's DEFAULT fetch type   |
|                                                                    |
|  @EntityGraph(                                                     |
|    attributePaths = {"department"},                                |
|    type = EntityGraph.EntityGraphType.FETCH  // or LOAD            |
|  )                                                                 |
|                                                                    |
|  Most of the time, FETCH (default) is what you want.              |
+-------------------------------------------------------------------+
```

@QueryHints for Optimization

JPA `@QueryHints` let you pass hints to the persistence provider for optimization:

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Read-only hint - Hibernate skips dirty checking for these entities
    @QueryHints(@QueryHint(name = "org.hibernate.readOnly", value = "true"))
    @Query("SELECT e FROM Employee e WHERE e.active = true")
    List<Employee> findActiveReadOnly();
```

```
    // Comment hint — adds a SQL comment for debugging in query logs
    @QueryHints(@QueryHint(name = "org.hibernate.comment",
                           value = "Employee active list"))
    List<Employee> findByActive(boolean active);

    // Fetch size hint — controls JDBC fetch size for large result sets
    @QueryHints(@QueryHint(name = "org.hibernate.fetchSize", value = "50"))
    @Query("SELECT e FROM Employee e")
    List<Employee> findAllWithFetchSize();

    // Timeout hint — query timeout in seconds
    @QueryHints(@QueryHint(name = "jakarta.persistence.query.timeout",
                           value = "5000"))
    @Query("SELECT e FROM Employee e WHERE e.active = true")
    List<Employee> findActiveWithTimeout();
}
```

```
Common @QueryHints:
+-------------------------------------------------------------------+
|  Hint                                      Effect                 |
|-------------------------------------------------------------------|
|  org.hibernate.readOnly = "true"           Skip dirty checking    |
|                                            (faster for read-only  |
|                                            data, saves memory)    |
|                                                                   |
|  org.hibernate.fetchSize = "50"            JDBC rows fetched per  |
|                                            database round-trip    |
|                                                                   |
|  org.hibernate.comment = "..."             SQL comment for logging |
|                                                                   |
|  org.hibernate.cacheable = "true"          Use query cache        |
|                                                                   |
|  jakarta.persistence.query.timeout         Query timeout (ms)     |
|  = "5000"                                                         |
+-------------------------------------------------------------------+
```

Pagination in REST Controllers

A complete example of exposing paginated data through a REST API:

```java
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    private final EmployeeRepository repository;
```

```
    public EmployeeController(EmployeeRepository repository) {
        this.repository = repository;
    }

    // Spring Boot auto-resolves Pageable from query parameters:
    // GET /api/employees?page=0&size=20&sort=name,asc&sort=salary,desc
    @GetMapping
    public Page<Employee> getAll(Pageable pageable) {
        return repository.findAll(pageable);
    }

    // With default values
    @GetMapping("/active")
    public Page<Employee> getActive(
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "20") int size,
            @RequestParam(defaultValue = "name") String sortBy,
            @RequestParam(defaultValue = "asc") String direction) {

        Sort sort = direction.equalsIgnoreCase("desc")
            ? Sort.by(sortBy).descending()
            : Sort.by(sortBy).ascending();

        return repository.findByActive(true,
            PageRequest.of(page, size, sort));
    }
}
```

```
REST Pagination Request/Response:
+-------------------------------------------------------------------+
|                                                                    |
|  Request:                                                          |
|  GET /api/employees?page=0&size=3&sort=salary,desc               |
|                                                                    |
|  Response:                                                         |
|  {                                                                 |
|    "content": [                                                    |
|      {"id": 5, "name": "Alice", "salary": 120000},               |
|      {"id": 12, "name": "Bob", "salary": 110000},                |
|      {"id": 3, "name": "Charlie", "salary": 95000}               |
|    ],                                                              |
|    "pageable": {                                                   |
|      "pageNumber": 0,                                              |
|      "pageSize": 3,                                                |
|      "sort": { "sorted": true, "orders": [...] }                 |
|    },                                                              |
|    "totalElements": 150,                                           |
|    "totalPages": 50,                                               |
|    "first": true,                                                  |
|    "last": false,                                                  |
|    "numberOfElements": 3                                           |
|  }                                                                 |
```

```
|                                                                    |
+--------------------------------------------------------------------+
```

Limiting Page Size

Always limit the maximum page size to prevent abuse:

```java
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addArgumentResolvers(List<HandlerMethodArgumentResolver> resolvers) {
        PageableHandlerMethodArgumentResolver resolver =
            new PageableHandlerMethodArgumentResolver();
        resolver.setMaxPageSize(100);         // Max 100 items per page
        resolver.setFallbackPageable(
            PageRequest.of(0, 20));           // Default: page 0, size 20
        resolvers.add(resolver);
    }
}
```

Transforming Page Content

Often you need to convert entity pages to DTO pages:

```java
// Entity to DTO conversion using Page.map()
public Page<EmployeeDto> getEmployees(Pageable pageable) {
    Page<Employee> entityPage = repository.findByActive(true, pageable);

    // map() preserves all pagination metadata
    return entityPage.map(employee -> new EmployeeDto(
        employee.getId(),
        employee.getName(),
        employee.getEmail(),
        employee.getDepartment().getName()
    ));
}
```

```
Page.map() Transformation:
+--------------------------------------------------------------------+
|                                                                    |
|  Page<Employee>                      Page<EmployeeDto>             |
```

```
|  +-------------------+              +--------------------+        |
|  | Employee(id=1,    |    map()     | EmployeeDto(id=1,  |        |
|  |   name="Alice",   | ----------> |   name="Alice",    |        |
|  |   email="a@co",   |             |   email="a@co",    |        |
|  |   dept=Dept(...))  |             |   deptName="Eng")  |        |
|  +-------------------+              +--------------------+        |
|  | totalElements: 150|    same      | totalElements: 150 |        |
|  | totalPages: 8     | ----------> | totalPages: 8      |        |
|  | number: 0         |  metadata   | number: 0          |        |
|  +-------------------+  preserved  +--------------------+        |
+-----------------------------------------------------------------+
```

Keyset Pagination (Scroll API)

For very large datasets, offset-based pagination becomes slow because the database must skip over all previous rows. Keyset pagination (also called cursor-based pagination) is more efficient:

```
Offset vs Keyset Pagination Performance:
+-----------------------------------------------------------------+
|                                                                 |
|  Offset pagination (page 1000, size 20):                        |
|  SELECT * FROM employees ORDER BY id LIMIT 20 OFFSET 20000      |
|  --> Database reads 20,020 rows, discards 20,000, returns 20    |
|  --> Gets SLOWER as page number increases                       |
|                                                                 |
|  Keyset pagination (after id = 20000):                          |
|  SELECT * FROM employees WHERE id > 20000 ORDER BY id LIMIT 20  |
|  --> Database uses index to jump directly, reads only 20 rows   |
|  --> SAME speed regardless of position                          |
|                                                                 |
|  Performance:                                                   |
|  Page 1:    Offset ~1ms     Keyset ~1ms      (same)             |
|  Page 100:  Offset ~10ms    Keyset ~1ms                         |
|  Page 1000: Offset ~100ms   Keyset ~1ms      (big difference!)  |
|  Page 10K:  Offset ~1000ms  Keyset ~1ms      (10x faster)       |
+-----------------------------------------------------------------+
```

Spring Data JPA 3.1+ supports keyset pagination through the `ScrollPosition` and `Window` API:

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
```

```
    Window<Employee> findByActiveTrue(OffsetScrollPosition position, Limit limit, Sort ...
}

// Usage
Window<Employee> firstPage = repository.findByActiveTrue(
    ScrollPosition.offset(),        // start from beginning
    Limit.of(20),                   // 20 items
    Sort.by("salary").descending()  // sort order
);

// Get next page using the last item's position
if (firstPage.hasNext()) {
    Window<Employee> secondPage = repository.findByActiveTrue(
        firstPage.positionAt(firstPage.size() - 1),
        Limit.of(20),
        Sort.by("salary").descending()
    );
}
```

Common Mistakes

1. Forgetting that pages are 0-based: Page 0 is the first page. Passing page=1 returns the second page of results. If your frontend uses 1-based pagination, subtract 1 before creating `PageRequest`.
2. Using Page when Slice is sufficient: If your UI shows "Load More" instead of "Page 1 of 50", use `Slice<T>`. The COUNT query for `Page<T>` can be expensive on large tables.
3. Not providing countQuery for complex JPQL or native queries: Queries with FETCH JOIN, UNION, or subqueries cannot have their count query auto-derived. Provide an explicit `countQuery`.
4. Sorting by non-indexed columns: Pagination with `ORDER BY` on a column without a database index forces a full table sort. Add indexes on frequently sorted columns.
5. Using offset pagination on huge datasets: Offset-based pagination (LIMIT/OFFSET) gets slower as the offset grows because the database must scan and discard earlier rows. For datasets with millions of rows, consider keyset pagination.
6. Not limiting max page size: Without a max page size limit, a client can request `?size=1000000` and pull your entire database into memory. Always configure a maximum.
7. Ignoring @EntityGraph with pagination: Paginating a query that triggers N+1 lazy loading means each page of 20 items generates 21 queries (1 + 20). Add `@EntityGraph` to eagerly load the needed relationships in one query.

Best Practices

1. Always paginate list endpoints: Never return unbounded lists from an API. Default to `PageRequest.of(0, 20)` and let clients override with query parameters.
2. Choose Page vs Slice based on UI needs: Use `Page<T>` for traditional pagination controls ("Page 3 of 47"). Use `Slice<T>` for infinite scroll or "Load More" patterns. Use `List<T>` when you do not need pagination metadata.
3. Add @EntityGraph to paginated queries: Prevent N+1 by eagerly loading relationships used in the response, especially when converting entities to DTOs.
4. Set a maximum page size: Configure `PageableHandlerMethodArgumentResolver` with `setMaxPageSize(100)` to prevent abuse.
5. Use Page.map() for DTO conversion: The `map()` method preserves all pagination metadata, making it perfect for entity-to-DTO conversion.
6. Index your sort columns: If users can sort by salary, department name, or hire date, ensure those columns have database indexes.
7. Use Pageable directly in controller parameters: Spring Boot auto-resolves `Pageable` from query parameters (`?page=0&size=20&sort=name,asc`), eliminating manual parsing.

Summary

In this chapter, you learned how to efficiently retrieve subsets of data:

- Pageable and PageRequest define what page to retrieve, how many items per page, and the sort order. Pages are 0-indexed.
- Page<T> returns data plus full pagination metadata (total count, total pages) but requires an extra COUNT query. Slice<T> skips the count and just knows if more data exists. List<T> returns only the data.
- Sort provides a fluent API for single-field, multi-field, and directional sorting with null handling.
- Pagination works with all query types: derived methods, JPQL, native SQL, and Specifications.
- @EntityGraph controls eager fetching per query, solving the N+1 problem without changing entity defaults. Use `attributePaths` for inline definition or `@NamedEntityGraph` for reusable fetch plans.
- @QueryHints pass optimization hints like read-only mode, fetch size, and query timeout.

- Page.map() transforms entity pages to DTO pages while preserving all pagination metadata.
- Keyset pagination outperforms offset pagination on large datasets by using indexed lookups instead of OFFSET.

Interview Questions

Q1: What is the difference between Page and Slice in Spring Data JPA?

`Page<T>` executes an additional COUNT query to know the total number of elements and total pages. It provides full navigation metadata. `Slice<T>` fetches one extra row (size+1) to determine if a next page exists, but does not count the total. Use Page for traditional "Page X of Y" UIs; use Slice for infinite scroll or "Load More" patterns where total count is unnecessary.

Q2: How does Spring Boot resolve Pageable from HTTP request parameters?

Spring Boot's `PageableHandlerMethodArgumentResolver` reads `page`, `size`, and `sort` query parameters and creates a `Pageable` instance. For example, `?page=0&size=20&sort=name,asc&sort=salary,desc` creates `PageRequest.of(0, 20, Sort.by("name").ascending().and(Sort.by("salary").descending()))`.

Q3: What is the N+1 problem and how does @EntityGraph solve it?

The N+1 problem occurs when loading N entities triggers N additional queries to load their lazy relationships (1 query for the list + N queries for each related entity). `@EntityGraph` specifies which relationships should be eagerly loaded in a single JOIN query, reducing N+1 queries to 1 query.

Q4: When would you provide a custom countQuery?

When the auto-derived count query fails or is inefficient. Common cases: queries with FETCH JOIN (count cannot include FETCH), native SQL queries (count cannot be auto-derived), complex queries with subqueries or UNIONs, and when the count can be simplified for performance.

Q5: What is keyset pagination and when should you use it?

Keyset pagination (cursor-based) uses the last seen value as a filter (`WHERE id > lastId`) instead of OFFSET. It maintains constant performance regardless of position because the database uses an index to jump directly to the right spot. Use it when paginating millions of rows where offset-based pagination becomes too slow at high page numbers.

Q6: How do you convert a Page of entities to a Page of DTOs?

Use `Page.map()`. This method transforms each element using a mapping function while preserving all pagination metadata (total elements, total pages, page number, sort). Example: `entityPage.map(e -> new EmployeeDto(e.getId(), e.getName()))`.

Practice Exercises

Exercise 1: Basic Pagination Create a `ProductRepository` with a `Page<Product> findByCategory(String category, Pageable pageable)` method. Write a test that loads page 0 with 5 items, then page 1, and verifies the content and metadata.

Exercise 2: Slice vs Page Implement the same query returning `Slice<Product>` and `Page<Product>`. Enable SQL logging and compare the number of queries executed. Verify that Slice does not have totalElements.

Exercise 3: Multi-Field Sorting Create an endpoint `GET /api/products` that supports sorting by `category` (ascending) and then `price` (descending). Test with sample data and verify the order.

Exercise 4: EntityGraph Create a `Product` entity with a lazy `@ManyToOne` relationship to `Category`. Write a query that returns 20 products with pagination. First observe the N+1 problem in SQL logs, then add `@EntityGraph(attributePaths = {"category"})` and verify it is reduced to 1 query.

Exercise 5: Paginated Search API Build a `GET /api/employees/search` endpoint that accepts optional filters (name, department, minSalary, maxSalary) and pagination parameters. Use Specifications from Chapter 10 combined with `Pageable`. Return `Page<EmployeeDto>` using `Page.map()`.

What Is Next?

In the next chapter, we begin Part III: Relationships and Mapping. We will start with One-to-One Relationships — how to map them with `@OneToOne`, choose between unidirectional and bidirectional, share primary keys with `@MapsId`, and handle lazy loading and cascading for one-to-one associations.

Chapter 12: One-to-One Relationships

Learning Goals

By the end of this chapter, you will be able to:

- Understand when to use one-to-one relationships in a domain model
- Map unidirectional one-to-one relationships with @OneToOne and @JoinColumn
- Map bidirectional one-to-one relationships with mappedBy
- Share primary keys between related entities using @MapsId
- Control lazy vs eager loading for one-to-one associations
- Apply cascade operations to propagate persistence actions
- Choose between foreign key, shared primary key, and join table strategies
- Avoid common pitfalls in one-to-one mappings

Understanding One-to-One Relationships

A one-to-one relationship means that one entity is associated with exactly one instance of another entity. Examples from the real world:

```
One-to-One Examples:
+-------------------------------------------------------------------+
|                                                                   |
|  User        <------>  UserProfile                                |
|  (login info)          (bio, avatar, preferences)                 |
|                                                                   |
|  Employee    <------>  ParkingSpot                                 |
|  (employee data)       (assigned parking location)                |
```

```
|                                                                    |
|  Order       <------>  ShippingDetails                             |
|  (order info)          (tracking number, carrier, address)         |
|                                                                    |
|  Country     <------>  Capital                                     |
|  (country info)        (capital city details)                      |
|                                                                    |
+--------------------------------------------------------------------+
```

When to Use One-to-One

One-to-one relationships are appropriate when:

1. Separation of concerns: You want to keep a large entity manageable by splitting it into two tables (User + UserProfile)
2. Optional data: The related data might not exist for every row (not every employee has a parking spot)
3. Different access patterns: Some data is accessed frequently, other data rarely (user login vs profile details)
4. Security isolation: Sensitive data (salary details, medical records) in a separate table with different access controls

```
Why Split into Two Tables?
+--------------------------------------------------------------------+
|                                                                     |
|  Option A: One Big Table                                            |
|  +-----------------------------------------------------------+    |
|  | users                                                     |    |
|  | id | name | email | password | bio | avatar | theme | ... |    |
|  +-----------------------------------------------------------+    |
|  Problems:                                                          |
|  - Wide rows (many columns) = slower queries                       |
|  - Loading login page loads ALL profile data too                    |
|  - NULL columns for users without profiles                          |
|                                                                     |
|  Option B: Two Tables with One-to-One                               |
|  +---------------------------+  +---------------------------+     |
|  | users                     |  | user_profiles             |     |
|  | id | name | email | pass  |  | id | user_id | bio | ... |     |
|  +---------------------------+  +---------------------------+     |
|  Benefits:                                                          |
|  - Narrow rows = faster queries                                    |
|  - Login page only loads users table                               |
|  - Profile data loaded on demand                                    |
|  - Clean separation of concerns                                    |
+--------------------------------------------------------------------+
```

Database Schema for One-to-One

There are three ways to implement one-to-one in a database:

```
Strategy 1: Foreign Key (Most Common)
+-------------------------------------------------------------------+
|                                                                   |
|  users                          user_profiles                     |
|  +--------+---------+          +--------+----------+---------+    |
|  | id(PK) | name    |          | id(PK) | user_id  | bio     |    |
|  |--------+---------|          |--------+----------+---------|    |
|  | 1      | Alice   |<---+    | 10     | 1 (FK,UQ)| Hello...|    |
|  | 2      | Bob     |<---|-+  | 11     | 2 (FK,UQ)| Hi...   |    |
|  | 3      | Charlie |    | |  +--------+----------+---------+    |
|  +--------+---------+    | |                                     |
|                           | +-- user_id is a FOREIGN KEY          |
|                           +---- with a UNIQUE constraint          |
|                                 (ensures one-to-one)              |
+-------------------------------------------------------------------+

Strategy 2: Shared Primary Key
+-------------------------------------------------------------------+
|                                                                   |
|  users                          user_profiles                     |
|  +--------+---------+          +--------+---------+               |
|  | id(PK) | name    |          | id(PK) | bio     |               |
|  |--------+---------|          |--------+---------|               |
|  | 1      | Alice   |<------  | 1(FK)  | Hello...|               |
|  | 2      | Bob     |<------  | 2(FK)  | Hi...   |               |
|  +--------+---------+          +--------+---------+               |
|                                                                   |
|  user_profiles.id IS BOTH the primary key AND foreign key         |
|  Same value in both tables (user 1 -> profile 1)                  |
|  Most efficient: no extra column needed                           |
+-------------------------------------------------------------------+

Strategy 3: Join Table (Rare for One-to-One)
+-------------------------------------------------------------------+
|                                                                   |
|  users             user_profile_mapping       user_profiles       |
|  +------+------+   +----------+------------+  +------+-------+   |
|  |id(PK)|name  |   |user_id   |profile_id  |  |id(PK)|bio    |   |
|  |------+------|   |----------+------------|  |------+-------|   |
|  | 1    |Alice |   | 1(FK,UQ) | 10(FK,UQ)  |  | 10   |Hello..|   |
|  | 2    |Bob   |   | 2(FK,UQ) | 11(FK,UQ)  |  | 11   |Hi..   |   |
|  +------+------+   +----------+------------+  +------+-------+   |
|                                                                   |
|  Extra join table. Rarely used for one-to-one.                    |
|  Useful when you cannot modify either table's schema.             |
+-------------------------------------------------------------------+
```

Unidirectional One-to-One (Foreign Key)

The simplest form: one entity knows about the other, but not vice versa.

```
Unidirectional: User --> UserProfile
+-------------------------------------------------------------------+
|                                                                   |
|  User                          UserProfile                        |
|  +----------------------+    +----------------------+            |
|  | id: Long             |    | id: Long             |            |
|  | name: String         |    | bio: String          |            |
|  | email: String        |    | avatarUrl: String    |            |
|  | profile: UserProfile -+--->| theme: String        |            |
|  +----------------------+    +----------------------+            |
|                                                                   |
|  User can access its profile: user.getProfile()                   |
|  UserProfile does NOT know about User                             |
+-------------------------------------------------------------------+
```

```java
@Entity
@Table(name = "user_profiles")
public class UserProfile {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(length = 500)
    private String bio;

    @Column(name = "avatar_url")
    private String avatarUrl;

    @Column(length = 20)
    private String theme;

    // Default constructor required by JPA
    protected UserProfile() {}

    public UserProfile(String bio, String avatarUrl, String theme) {
        this.bio = bio;
        this.avatarUrl = avatarUrl;
        this.theme = theme;
    }

    // Getters and setters
```

```
    public Long getId() { return id; }
    public String getBio() { return bio; }
    public void setBio(String bio) { this.bio = bio; }
    public String getAvatarUrl() { return avatarUrl; }
    public void setAvatarUrl(String avatarUrl) { this.avatarUrl = avatarUrl; }
    public String getTheme() { return theme; }
    public void setTheme(String theme) { this.theme = theme; }
}
```

```java
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @OneToOne(cascade = CascadeType.ALL, orphanRemoval = true)  // JPA standard
    @JoinColumn(name = "profile_id", unique = true)              // JPA standard
    private UserProfile profile;

    // Default constructor required by JPA
    protected User() {}

    public User(String name, String email) {
        this.name = name;
        this.email = email;
    }

    // Getters and setters
    public Long getId() { return id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public UserProfile getProfile() { return profile; }
    public void setProfile(UserProfile profile) { this.profile = profile; }
}
```

Understanding the Annotations

```
@OneToOne Breakdown:
+-------------------------------------------------------------------+
```

```
|                                                                    |
|  @OneToOne(                                                        |
|      cascade = CascadeType.ALL,   // When User is saved/deleted,   |
|                                   // also save/delete the Profile |
|      orphanRemoval = true         // If profile is set to null,    |
|                                   // delete the orphaned Profile   |
|  )                                                                 |
|                                                                    |
|  @JoinColumn(                                                      |
|      name = "profile_id",         // Column name in users table    |
|      unique = true                // UNIQUE constraint enforces    |
|                                   // one-to-one (not one-to-many)  |
|  )                                                                 |
|  private UserProfile profile;                                      |
|                                                                    |
|  Generated SQL:                                                    |
|  ALTER TABLE users ADD COLUMN profile_id BIGINT UNIQUE;            |
|  ALTER TABLE users ADD FOREIGN KEY (profile_id)                    |
|      REFERENCES user_profiles(id);                                 |
+--------------------------------------------------------------------+
```

Saving and Loading

```java
@Service
@Transactional
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    // Save user with profile - cascade saves both
    public User createUserWithProfile(String name, String email,
                                      String bio, String avatarUrl) {
        User user = new User(name, email);

        UserProfile profile = new UserProfile(bio, avatarUrl, "light");
        user.setProfile(profile);  // Set the relationship

        return userRepository.save(user);  // Saves BOTH user and profile
    }

    // Loading user automatically loads profile (EAGER by default)
    public User getUser(Long id) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("User not found"));

        // profile is already loaded (EAGER is default for @OneToOne)
```

```
        System.out.println("Profile: " + user.getProfile().getBio());

        return user;
    }

    // Replace profile - orphanRemoval deletes the old one
    public void updateProfile(Long userId, String newBio) {
        User user = userRepository.findById(userId).orElseThrow();

        if (user.getProfile() != null) {
            user.getProfile().setBio(newBio);
        } else {
            user.setProfile(new UserProfile(newBio, null, "light"));
        }
        // No explicit save needed - dirty checking handles it
    }

    // Remove profile - orphanRemoval deletes the orphaned profile
    public void removeProfile(Long userId) {
        User user = userRepository.findById(userId).orElseThrow();
        user.setProfile(null);  // Profile becomes orphan, gets deleted
    }
}
```

```
Cascade in Action:
+-------------------------------------------------------------------+
|                                                                   |
|  userRepository.save(user)                                        |
|       |                                                           |
|       v                                                           |
|  CascadeType.ALL means:                                          |
|  1. INSERT INTO user_profiles (bio, avatar_url, theme)            |
|     VALUES ('Hello', 'pic.jpg', 'light')  --> returns id = 10    |
|  2. INSERT INTO users (name, email, profile_id)                   |
|     VALUES ('Alice', 'alice@co.com', 10)                         |
|                                                                   |
|  userRepository.delete(user)                                      |
|       |                                                           |
|       v                                                           |
|  CascadeType.ALL means:                                          |
|  1. DELETE FROM users WHERE id = 1                                |
|  2. DELETE FROM user_profiles WHERE id = 10                       |
|                                                                   |
|  user.setProfile(null); // orphanRemoval                          |
|       |                                                           |
|       v                                                           |
|  1. UPDATE users SET profile_id = NULL WHERE id = 1               |
|  2. DELETE FROM user_profiles WHERE id = 10  (orphan removed)     |
+-------------------------------------------------------------------+
```

Bidirectional One-to-One

Both entities know about each other. One side is the owning side (has the foreign key), the other is the inverse side (uses `mappedBy`).

```
Bidirectional: User <--> UserProfile
+-------------------------------------------------------------------+
|                                                                   |
|  User (Owning Side)               UserProfile (Inverse Side)     |
|  +-----------------------+        +-----------------------+       |
|  | id: Long              |        | id: Long              |       |
|  | name: String          |        | bio: String           |       |
|  | email: String         |        | avatarUrl: String     |       |
|  | profile: UserProfile -+------->| theme: String         |       |
|  +-----------------------+   <----+-user: User             |       |
|                                   +-----------------------+       |
|                                                                   |
|  Owning side: has @JoinColumn (foreign key in its table)          |
|  Inverse side: has mappedBy = "profile" (no FK, just a reference) |
|                                                                   |
|  user.getProfile()  --> works                                     |
|  profile.getUser()  --> also works                                |
+-------------------------------------------------------------------+
```

```java
// Owning side - unchanged from unidirectional
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @OneToOne(cascade = CascadeType.ALL, orphanRemoval = true)
    @JoinColumn(name = "profile_id", unique = true)
    private UserProfile profile;

    // Helper method to maintain both sides
    public void setProfile(UserProfile profile) {
        // Remove old profile's back-reference
        if (this.profile != null) {
            this.profile.setUser(null);
        }
        this.profile = profile;
```

```
        // Set new profile's back-reference
        if (profile != null) {
            profile.setUser(this);
        }
    }

    // ... constructors, other getters
}
```

```java
// Inverse side - add the back-reference
@Entity
@Table(name = "user_profiles")
public class UserProfile {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(length = 500)
    private String bio;

    @Column(name = "avatar_url")
    private String avatarUrl;

    @Column(length = 20)
    private String theme;

    @OneToOne(mappedBy = "profile")  // "profile" is the field name in User
    private User user;

    // ... constructors, getters, setters
    public User getUser() { return user; }
    public void setUser(User user) { this.user = user; }
}
```

```
Owning Side vs Inverse Side:
+-------------------------------------------------------------------+
|                                                                    |
|  Owning Side (User)            Inverse Side (UserProfile)          |
|  ---------------------------   ---------------------------         |
|  Has @JoinColumn               Has mappedBy                       |
|  Controls the foreign key      Read-only mirror                   |
|  Database writes go here       No database column                 |
|  Setting this side updates DB  Setting only this side does NOT     |
|                                update the database!               |
|                                                                    |
|  CRITICAL:                                                         |
|  user.setProfile(profile)  --> UPDATES the database                |
|  profile.setUser(user)     --> does NOT update database alone!     |
|                                                                    |
|  Always set the OWNING side. The inverse side is for navigation    |
```

```
|  convenience only.                                                  |
+---------------------------------------------------------------------+
```

Importance of Helper Methods

Always use helper methods that synchronize both sides:

```java
// WRONG: Setting only the inverse side
UserProfile profile = new UserProfile("Hello", null, "light");
profile.setUser(user);  // This alone does NOT set the FK!
// Database: users.profile_id is still NULL

// CORRECT: Setting the owning side (with helper)
user.setProfile(profile);  // Helper sets BOTH sides
// Database: users.profile_id = profile.id
```

Shared Primary Key with @MapsId

The most efficient one-to-one strategy: the child table uses the parent's primary key as its own primary key. No extra foreign key column is needed.

```
Shared Primary Key:
+---------------------------------------------------------------------+
|                                                                     |
|  users                           user_profiles                      |
|  +--------+---------+          +--------+---------+                |
|  | id(PK) | name    |          | id(PK) | bio     |                |
|  |--------+---------|          | (FK)   |         |                |
|  | 1      | Alice   |<---------| 1      | Hello...|                |
|  | 2      | Bob     |<---------| 2      | Hi...   |                |
|  | 3      | Charlie |          +--------+---------+                |
|  +--------+---------+                                               |
|                                                                     |
|  user_profiles.id = users.id (same value)                           |
|  No separate profile_id column needed                               |
|  One less column, one less index = more efficient                   |
+---------------------------------------------------------------------+
```

```java
@Entity
@Table(name = "users")
```

```
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @OneToOne(mappedBy = "user", cascade = CascadeType.ALL,
              orphanRemoval = true)
    private UserProfile profile;

    // Helper
    public void setProfile(UserProfile profile) {
        if (this.profile != null) {
            this.profile.setUser(null);
        }
        this.profile = profile;
        if (profile != null) {
            profile.setUser(this);
        }
    }

    // ... constructors, getters
}
```

```java
@Entity
@Table(name = "user_profiles")
public class UserProfile {

    @Id  // This ID will be copied from User's ID
    private Long id;

    @Column(length = 500)
    private String bio;

    @Column(name = "avatar_url")
    private String avatarUrl;

    @OneToOne
    @MapsId  // The magic: profile.id = user.id
    @JoinColumn(name = "id")  // The FK column IS the PK column
    private User user;

    protected UserProfile() {}

    public UserProfile(String bio, String avatarUrl) {
        this.bio = bio;
```

```
        this.avatarUrl = avatarUrl;
    }

    // Getters and setters
    public Long getId() { return id; }
    public String getBio() { return bio; }
    public void setBio(String bio) { this.bio = bio; }
    public String getAvatarUrl() { return avatarUrl; }
    public void setAvatarUrl(String avatarUrl) { this.avatarUrl = avatarUrl; }
    public User getUser() { return user; }
    public void setUser(User user) { this.user = user; }
}
```

```
How @MapsId Works:
+-------------------------------------------------------------------+
|                                                                   |
|  1. User is saved:                                                |
|     INSERT INTO users (name, email) VALUES ('Alice', 'a@co.com')  |
|     --> Generated id = 1                                          |
|                                                                   |
|  2. UserProfile is saved (via cascade):                           |
|     @MapsId copies user.id into profile.id automatically          |
|     INSERT INTO user_profiles (id, bio, avatar_url)               |
|     VALUES (1, 'Hello', 'pic.jpg')                                |
|            ^                                                      |
|            |-- Same value as user.id!                             |
|                                                                   |
|  3. Loading profile by user ID is instant:                        |
|     SELECT * FROM user_profiles WHERE id = 1                      |
|     (PK lookup - the fastest possible query)                      |
|                                                                   |
|  No @GeneratedValue on UserProfile.id!                            |
|  The ID comes from the User, not from the database.               |
+-------------------------------------------------------------------+
```

When to Use @MapsId

```
Strategy Comparison:
+-------------------------------------------------------------------+
|                                                                   |
|  Strategy          Extra Column?  Lookup Speed   Recommended?     |
|  ----------------------------------------------------------------|
|  Foreign Key       Yes (FK col)   FK index       Good default     |
|  (@JoinColumn)                    lookup                          |
|                                                                   |
|  Shared PK         No             PK lookup      Best for true    |
|  (@MapsId)                        (fastest)      1:1 always       |
|                                                                   |
|  Join Table        Yes (whole     Two joins      Only for legacy  |
```

```
|  (@JoinTable)      extra table)   (slowest)      schemas          |
|                                                                    |
|  Use @MapsId when:                                                 |
|  - The child entity ALWAYS exists with its parent                  |
|  - You want maximum performance (PK lookups)                       |
|  - You do not need independent IDs                                 |
|                                                                    |
|  Use Foreign Key when:                                             |
|  - The relationship is optional (child may not exist)              |
|  - The child might be shared or reassigned                         |
|  - You need independent auto-generated IDs                         |
+--------------------------------------------------------------------+
```

Lazy Loading with One-to-One

By default, `@OneToOne` uses `FetchType.EAGER` — the related entity is loaded immediately with the parent. This can be a performance concern.

```
Default @OneToOne Fetching:
+--------------------------------------------------------------------+
|                                                                    |
|  userRepository.findById(1L);                                      |
|                                                                    |
|  SQL Generated (EAGER, default):                                   |
|  SELECT u.*, p.*                                                   |
|  FROM users u                                                      |
|  LEFT JOIN user_profiles p ON u.profile_id = p.id                  |
|  WHERE u.id = 1                                                    |
|                                                                    |
|  Always loads profile even if you only need user.name!             |
+--------------------------------------------------------------------+
```

Setting Lazy Loading

```java
@OneToOne(fetch = FetchType.LAZY, cascade = CascadeType.ALL)
@JoinColumn(name = "profile_id", unique = true)
private UserProfile profile;
```

The Lazy Loading Caveat for @OneToOne

Important: Lazy loading on the inverse side (mappedBy) of a `@OneToOne` does not always work as expected with standard JPA:

```
Lazy Loading Caveat:
+-------------------------------------------------------------------+
|                                                                   |
|  Owning Side (has @JoinColumn):                                   |
|  @OneToOne(fetch = FetchType.LAZY)                                |
|  @JoinColumn(name = "profile_id")                                 |
|  private UserProfile profile;                                     |
|  --> LAZY works correctly                                         |
|  --> Hibernate creates a proxy (loads on first access)            |
|                                                                   |
|  Inverse Side (has mappedBy):                                     |
|  @OneToOne(mappedBy = "user", fetch = FetchType.LAZY)             |
|  private User user;                                               |
|  --> LAZY may NOT work!                                           |
|  --> Hibernate cannot create a proxy because it does not know     |
|      if the relationship is null or not without querying           |
|  --> It must query to check, which defeats the purpose of LAZY    |
|                                                                   |
|  Solutions:                                                       |
|  1. Use @MapsId (shared PK) — eliminates the problem              |
|  2. Use optional = false if always present                        |
|  3. Accept eager loading on the inverse side                      |
|  4. Use Hibernate bytecode enhancement (advanced)                 |
+-------------------------------------------------------------------+
```

```java
// Making lazy work on inverse side with optional = false
@OneToOne(mappedBy = "user", fetch = FetchType.LAZY, optional = false)
private UserProfile profile;
// Tells Hibernate: profile is NEVER null, so it can safely create a proxy
```

Optional vs Required Relationships

```java
// Optional (default) — profile may be null
@OneToOne(cascade = CascadeType.ALL, optional = true)  // default is true
@JoinColumn(name = "profile_id", unique = true)
private UserProfile profile;

// Required — profile must always exist
@OneToOne(cascade = CascadeType.ALL, optional = false)
@JoinColumn(name = "profile_id", nullable = false, unique = true)
private UserProfile profile;
```

```
Optional vs Required:
+-------------------------------------------------------------------+
|                                                                   |
|  optional = true (default):                                       |
|  - LEFT JOIN used when loading                                    |
|  - profile_id column can be NULL                                  |
|  - user.getProfile() may return null                              |
|  - Use when: not all users have profiles                          |
|                                                                   |
|  optional = false:                                                |
|  - INNER JOIN used when loading (more efficient)                  |
|  - profile_id column is NOT NULL                                  |
|  - user.getProfile() never returns null                           |
|  - Use when: every user MUST have a profile                       |
|  - Also helps lazy loading work on the inverse side               |
+-------------------------------------------------------------------+
```

Complete Example: Employee and ParkingSpot

A practical bidirectional one-to-one with @MapsId:

```java
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @OneToOne(mappedBy = "employee", cascade = CascadeType.ALL,
              orphanRemoval = true, fetch = FetchType.LAZY)
    private ParkingSpot parkingSpot;

    protected Employee() {}

    public Employee(String name, String email) {
        this.name = name;
        this.email = email;
    }
```

```
    // Convenience method - keeps both sides in sync
    public void assignParkingSpot(String spotNumber, String level) {
        ParkingSpot spot = new ParkingSpot(spotNumber, level);
        spot.setEmployee(this);
        this.parkingSpot = spot;
    }

    public void removeParkingSpot() {
        if (this.parkingSpot != null) {
            this.parkingSpot.setEmployee(null);
            this.parkingSpot = null;  // orphanRemoval deletes it
        }
    }

    // Getters
    public Long getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public ParkingSpot getParkingSpot() { return parkingSpot; }
}
```

```java
@Entity
@Table(name = "parking_spots")
public class ParkingSpot {

    @Id
    private Long id;

    @Column(name = "spot_number", nullable = false)
    private String spotNumber;

    @Column(nullable = false)
    private String level;

    @OneToOne
    @MapsId
    @JoinColumn(name = "id")
    private Employee employee;

    protected ParkingSpot() {}

    public ParkingSpot(String spotNumber, String level) {
        this.spotNumber = spotNumber;
        this.level = level;
    }

    // Getters and setters
    public Long getId() { return id; }
    public String getSpotNumber() { return spotNumber; }
    public void setSpotNumber(String spotNumber) { this.spotNumber = spotNumber; }
    public String getLevel() { return level; }
    public void setLevel(String level) { this.level = level; }
```

```
    public Employee getEmployee() { return employee; }
    public void setEmployee(Employee employee) { this.employee = employee; }
}
```

Repository and Service

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    @EntityGraph(attributePaths = {"parkingSpot"})
    Optional<Employee> findByEmail(String email);

    @Query("SELECT e FROM Employee e LEFT JOIN FETCH e.parkingSpot WHERE e.parkingSpot ...
    List<Employee> findAllWithParkingSpots();
}
```

```java
@Service
@Transactional
public class ParkingService {

    private final EmployeeRepository employeeRepository;

    public ParkingService(EmployeeRepository employeeRepository) {
        this.employeeRepository = employeeRepository;
    }

    public void assignSpot(Long employeeId, String spotNumber, String level) {
        Employee employee = employeeRepository.findById(employeeId)
            .orElseThrow(() -> new RuntimeException("Employee not found"));

        employee.assignParkingSpot(spotNumber, level);
        // Cascade saves the ParkingSpot automatically
    }

    public void removeSpot(Long employeeId) {
        Employee employee = employeeRepository.findById(employeeId)
            .orElseThrow(() -> new RuntimeException("Employee not found"));

        employee.removeParkingSpot();
        // orphanRemoval deletes the ParkingSpot automatically
    }

    public String getSpotInfo(Long employeeId) {
        Employee employee = employeeRepository.findById(employeeId)
            .orElseThrow(() -> new RuntimeException("Employee not found"));

        ParkingSpot spot = employee.getParkingSpot();
        if (spot == null) {
```

```
            return employee.getName() + " has no assigned parking spot";
        }
        return employee.getName() + " - Spot " + spot.getSpotNumber() +
               " (Level " + spot.getLevel() + ")";
    }
}
```

Testing

```java
@SpringBootTest
@Transactional
class ParkingServiceTest {

    @Autowired
    private ParkingService parkingService;

    @Autowired
    private EmployeeRepository employeeRepository;

    @Test
    void shouldAssignAndRemoveParkingSpot() {
        // Create employee
        Employee emp = new Employee("Alice", "alice@company.com");
        employeeRepository.save(emp);

        // Assign parking spot
        parkingService.assignSpot(emp.getId(), "A-15", "Ground");

        // Verify
        Employee loaded = employeeRepository.findById(emp.getId()).orElseThrow();
        assertNotNull(loaded.getParkingSpot());
        assertEquals("A-15", loaded.getParkingSpot().getSpotNumber());
        assertEquals("Ground", loaded.getParkingSpot().getLevel());
        assertEquals(loaded.getId(), loaded.getParkingSpot().getId()); // Same ID!

        // Remove parking spot
        parkingService.removeSpot(emp.getId());

        // Verify removal
        Employee reloaded = employeeRepository.findById(emp.getId()).orElseThrow();
        assertNull(reloaded.getParkingSpot());
    }
}
```

Common Mistakes

1. Only setting the inverse side (mappedBy) and expecting the FK to update:

```java
// WRONG
profile.setUser(user);  // inverse side only — FK not updated!
// CORRECT
user.setProfile(profile);  // owning side — FK IS updated
```

2. Forgetting `unique = true` on @JoinColumn: Without the unique constraint, the database allows multiple rows referencing the same parent — making it a one-to-many relationship at the database level.
3. Using @GeneratedValue with @MapsId: When using `@MapsId`, the ID is copied from the parent entity. Adding `@GeneratedValue` causes a conflict. Remove it from the child entity.
4. Not using helper methods for bidirectional relationships: Failing to synchronize both sides leads to inconsistent in-memory state, even if the database is correct after flush.
5. Assuming LAZY works on the inverse side: The `mappedBy` side of `@OneToOne` often loads eagerly regardless of `FetchType.LAZY` setting. Use `@MapsId` or `optional = false` to mitigate.
6. Cascading in both directions: Having `cascade = CascadeType.ALL` on both sides of a bidirectional relationship can cause issues. Cascade from the parent (owning side) only.

Best Practices

1. Prefer @MapsId for true one-to-one: When the child entity always exists with its parent and has no independent identity, shared primary keys are the most efficient mapping.
2. Keep cascade on the parent side only: The parent entity should cascade operations to the child. The inverse side (mappedBy) should not cascade back.
3. Use bidirectional helper methods: Write methods like `setProfile()` that synchronize both sides of the relationship to prevent inconsistencies.
4. Set fetch = FetchType.LAZY on the owning side: Override the default EAGER loading when you do not always need the related entity. Use `@EntityGraph` to eagerly load it when needed.
5. Use orphanRemoval for owned children: When the child cannot exist without the parent (like a UserProfile without a User), enable `orphanRemoval = true`.
6. Mark required relationships with optional = false: This generates NOT NULL constraints, uses INNER JOIN (faster), and helps lazy loading work on inverse sides.

Summary

In this chapter, you learned how to map one-to-one relationships:

- @OneToOne marks a single-valued association. By default it is EAGER fetched. Use `@JoinColumn` on the owning side to define the foreign key column.
- Unidirectional means only one entity references the other. Bidirectional means both entities reference each other, with `mappedBy` on the inverse side.
- @MapsId creates a shared primary key — the child's ID equals the parent's ID. This is the most efficient strategy for true one-to-one relationships.
- The owning side (with `@JoinColumn`) controls the foreign key. Always set the owning side. The inverse side (`mappedBy`) is for navigation convenience only.
- Cascade propagates operations from parent to child. orphanRemoval deletes children that are no longer referenced.
- Lazy loading works reliably on the owning side but may not work on the inverse side (mappedBy) of `@OneToOne` without `@MapsId` or `optional = false`.
- Helper methods that synchronize both sides of a bidirectional relationship are essential for consistent in-memory state.

Interview Questions

Q1: What is the difference between unidirectional and bidirectional one-to-one?

Unidirectional: only one entity has a reference to the other (e.g., User has UserProfile, but UserProfile does not know about User). Bidirectional: both entities reference each other. The owning side has `@JoinColumn`, the inverse side has `@OneToOne(mappedBy = "fieldName")`.

Q2: What does @MapsId do and when would you use it?

`@MapsId` makes the child entity use the parent entity's primary key as its own primary key. The child's ID equals the parent's ID. Use it for true one-to-one relationships where the child always exists with the parent and does not need an independent identity. It eliminates an extra column and provides the fastest lookup (PK-based).

Q3: Why is it important to set the owning side in a bidirectional relationship?

The owning side (the entity with `@JoinColumn`) controls the foreign key in the database. Only changes made to the owning side are persisted. If you only set the inverse side (`mappedBy`), the database foreign key is not updated, even though the in-memory state appears correct.

Q4: Does lazy loading work on both sides of a @OneToOne bidirectional relationship?

Lazy loading works reliably on the owning side (where `@JoinColumn` is). On the inverse side (`mappedBy`), lazy loading may not work because Hibernate cannot determine if the relationship is null without querying the database. Solutions include using `@MapsId`, setting `optional = false`, or using Hibernate bytecode enhancement.

Q5: What is the difference between orphanRemoval and CascadeType.REMOVE?

`CascadeType.REMOVE` deletes the child when the parent is deleted. `orphanRemoval = true` also deletes the child when the relationship is broken (e.g., `user.setProfile(null)`). orphanRemoval is a superset of CascadeType.REMOVE — it covers both parent deletion and relationship removal.

Q6: What happens if you forget `unique = true` on @JoinColumn for a one-to-one?

Without the unique constraint, the database allows multiple entities to reference the same target, effectively making it a one-to-many relationship at the database level. JPA will still treat it as one-to-one in your Java code, but the database constraint is not enforced, which could lead to data integrity issues.

Practice Exercises

Exercise 1: Unidirectional One-to-One Create `Order` and `ShippingDetails` entities with a unidirectional `@OneToOne` from Order to ShippingDetails. Use `CascadeType.ALL`. Write a service that creates an order with shipping details and test it.

Exercise 2: Bidirectional with @MapsId Create `Student` and `StudentCard` entities with a bidirectional one-to-one using `@MapsId`. The StudentCard should share the Student's primary key. Write helper methods and test that both sides stay synchronized.

Exercise 3: Lazy Loading Verification Create a one-to-one relationship with `FetchType.LAZY`. Enable SQL logging and verify: (a) That accessing the parent does NOT load the child immediately (b) That accessing the child triggers a second query (c) That using `@EntityGraph` loads both in one query

Exercise 4: orphanRemoval Create entities where the child uses `orphanRemoval = true`. Test three scenarios: (a) Saving parent with child (cascade) (b) Replacing the child with a new one (old child should be deleted) (c) Setting the child to null (old child should be deleted)

Exercise 5: Strategy Comparison Implement the same one-to-one relationship (Country and Capital) three ways: foreign key, shared primary key (@MapsId), and join table. Enable SQL

logging and compare the generated schema and queries for each strategy.

What Is Next?

In the next chapter, we will explore One-to-Many and Many-to-One Relationships — the most common relationship type in database modeling. You will learn how to map parent-child relationships, understand the owning side concept in a many-to-one context, write bidirectional helper methods, and avoid the common pitfalls of collection-valued associations.

Chapter 13: One-to-Many and Many-to-One Relationships

Learning Goals

By the end of this chapter, you will be able to:

- Understand one-to-many and many-to-one relationships and how they relate
- Map @ManyToOne (the owning side) with @JoinColumn
- Map @OneToMany (the inverse side) with mappedBy
- Write bidirectional helper methods to keep both sides synchronized
- Choose between unidirectional and bidirectional mappings
- Use orphanRemoval to auto-delete detached children
- Avoid the unidirectional @OneToMany without @JoinColumn anti-pattern
- Handle cascading correctly in parent-child relationships
- Query across one-to-many relationships with JPQL and derived methods

Understanding One-to-Many Relationships

One-to-many is the most common relationship type in database modeling. One parent entity owns a collection of child entities.

```
One-to-Many Examples:
+-------------------------------------------------------------------+
|                                                                   |
|  Department  ----< Employee      (One dept has many employees)    |
|  Author      ----< Book          (One author writes many books)   |
|  Order       ----< OrderItem     (One order has many line items)  |
```

```
|  Post        ----< Comment       (One post has many comments)     |
|  Customer    ----< Invoice       (One customer has many invoices) |
|  Category    ----< Product       (One category has many products) |
|                                                                   |
|  Legend:  ----<  means "has many"                                 |
+-------------------------------------------------------------------+
```

Two Sides of the Same Coin

One-to-many and many-to-one are the same relationship seen from different sides:

```
Same Relationship, Different Perspectives:
+-------------------------------------------------------------------+
|                                                                   |
|  From Department's perspective:                                   |
|  "A Department has MANY Employees"    --> @OneToMany              |
|                                                                   |
|  From Employee's perspective:                                     |
|  "An Employee belongs to ONE Department"  --> @ManyToOne          |
|                                                                   |
|  Department (1) -------- (*) Employee                             |
|       |                        |                                  |
|       |  @OneToMany             |  @ManyToOne                     |
|       |  List<Employee>         |  Department department           |
|       |  (collection side)      |  (single-value side)            |
|       |  (inverse side)         |  (owning side)                  |
|                                                                   |
|  In the DATABASE, the FK is always on the "many" side:            |
|  employees table has department_id (FK)                           |
|  departments table does NOT have employee columns                 |
+-------------------------------------------------------------------+
```

Database Schema

```
Database Tables:
+-------------------------------------------------------------------+
|                                                                   |
|  departments                      employees                       |
|  +--------+-----------+          +--------+--------+----------+   |
|  | id(PK) | name      |          | id(PK) | name   | dept_id  |   |
|  |--------+-----------|          |--------+--------+----------|   |
|  | 1      | Engineering|<---+    | 10     | Alice  | 1 (FK)   |   |
|  | 2      | Marketing |<---|-+  | 11     | Bob    | 1 (FK)   |   |
|  | 3      | Sales     |    | |  | 12     | Carol  | 2 (FK)   |   |
|  +--------+-----------+    | |  | 13     | Dave   | 2 (FK)   |   |
|                             | |  | 14     | Eve    | 1 (FK)   |   |
|                             | |  +--------+--------+----------+   |
```

```
|                              | |                                  |
|  The FK is on the "many" side (employees table)                  |
|  NOT on the "one" side (departments table)                       |
|  This is always the case for one-to-many / many-to-one           |
+------------------------------------------------------------------+
```

@ManyToOne — The Owning Side

Start with `@ManyToOne` because it is simpler and more important. The "many" side always owns the relationship because that is where the foreign key lives.

```java
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @ManyToOne(fetch = FetchType.LAZY)        // JPA standard
    @JoinColumn(name = "department_id")        // JPA standard — FK column
    private Department department;

    protected Employee() {}

    public Employee(String name, String email) {
        this.name = name;
        this.email = email;
    }

    // Getters and setters
    public Long getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public Department getDepartment() { return department; }
    public void setDepartment(Department department) {
        this.department = department;
    }
}
```

```
@ManyToOne Annotation Breakdown:
+-----------------------------------------------------------------+
|                                                                 |
|  @ManyToOne(                                                    |
|      fetch = FetchType.LAZY  // Override default EAGER           |
|  )                           // Loads department only when       |
|                              // employee.getDepartment() is called |
|                                                                 |
|  @JoinColumn(                                                   |
|      name = "department_id"  // Column name in employees table   |
|  )                           // FK referencing departments.id    |
|                                                                 |
|  Default fetch: EAGER (loads department with every employee!)   |
|  Best practice: Always set FetchType.LAZY on @ManyToOne         |
|                                                                 |
|  Generated DDL:                                                 |
|  ALTER TABLE employees ADD COLUMN department_id BIGINT;         |
|  ALTER TABLE employees ADD FOREIGN KEY (department_id)          |
|      REFERENCES departments(id);                                |
+-----------------------------------------------------------------+
```

Using @ManyToOne Alone (Unidirectional)

You can use `@ManyToOne` without a corresponding `@OneToMany`. This is called unidirectional many-to-one:

```java
// Employee knows its department
Employee employee = new Employee("Alice", "alice@co.com");
employee.setDepartment(engineeringDept);
employeeRepository.save(employee);

// Query employees by department
List<Employee> engineers = employeeRepository.findByDepartmentName("Engineering");

// Navigate from employee to department
String deptName = employee.getDepartment().getName();
```

This is perfectly valid and often sufficient. You only need `@OneToMany` on the Department side if you want to navigate from a department to its employees using `department.getEmployees()`.

@OneToMany with mappedBy — The Inverse Side

Add `@OneToMany` to the parent entity when you need collection navigation:

```java
@Entity
@Table(name = "departments")
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    @OneToMany(mappedBy = "department",       // Field name in Employee
               cascade = CascadeType.ALL,     // Cascade operations to children
               orphanRemoval = true)          // Delete children removed from list
    private List<Employee> employees = new ArrayList<>();

    protected Department() {}

    public Department(String name) {
        this.name = name;
    }

    // ===== Bidirectional Helper Methods =====

    public void addEmployee(Employee employee) {
        employees.add(employee);
        employee.setDepartment(this);   // Set the owning side!
    }

    public void removeEmployee(Employee employee) {
        employees.remove(employee);
        employee.setDepartment(null);   // Clear the owning side!
    }

    // Getters
    public Long getId() { return id; }
    public String getName() { return name; }
    public List<Employee> getEmployees() {
        return Collections.unmodifiableList(employees);
    }
}
```

```
Bidirectional Mapping:
+-------------------------------------------------------------------+
|                                                                   |
|  Department                        Employee                       |
|  +-------------------------+     +-------------------------+      |
|  | @OneToMany(             |     | @ManyToOne(             |      |
|  |   mappedBy="department",|     |   fetch=LAZY)           |      |
|  |   cascade=ALL,          |     | @JoinColumn(            |      |
|  |   orphanRemoval=true)   |     |   name="department_id") |      |
|  | List<Employee> employees|---->| Department department   |      |
|  +-------------------------+     +-------------------------+      |
```

```
|  INVERSE side                    OWNING side                      |
|  (no FK column)                  (has FK column)                  |
|  (mappedBy points to the         (JoinColumn defines              |
|   field on the owning side)       the FK column name)             |
|                                                                   |
|  RULE: mappedBy = "department"                                    |
|  means: "look at the 'department' field in Employee               |
|          to find the FK mapping"                                  |
+-------------------------------------------------------------------+
```

Why Helper Methods Are Critical

Without helper methods, it is easy to create inconsistent state:

```
Without Helper Methods — What Goes Wrong:
+-------------------------------------------------------------------+
|                                                                   |
|  // WRONG: Only setting the collection side                       |
|  department.getEmployees().add(employee);                         |
|  // In-memory: department knows about employee                    |
|  // But employee.department is still null!                        |
|  // Database: department_id is NOT set (owning side not updated)  |
|                                                                   |
|  // WRONG: Only setting the @ManyToOne side                       |
|  employee.setDepartment(department);                              |
|  // In-memory: employee knows its department                      |
|  // But department.employees does not contain this employee!      |
|  // Database: FK IS set (owning side was updated)                 |
|  // But the in-memory collection is out of sync                   |
|                                                                   |
|  // CORRECT: Helper method sets BOTH sides                        |
|  department.addEmployee(employee);                                |
|  // In-memory: both sides are synchronized                        |
|  // Database: FK is correctly set                                 |
+-------------------------------------------------------------------+
```

```java
// The helper methods in Department:

public void addEmployee(Employee employee) {
    employees.add(employee);          // Update the collection
    employee.setDepartment(this);     // Update the owning side (sets FK)
}

public void removeEmployee(Employee employee) {
    employees.remove(employee);       // Remove from collection
```

```
    employee.setDepartment(null);     // Clear the owning side (clears FK)
}
```

Saving and Loading Parent-Child Data

Creating a Department with Employees

```java
@Service
@Transactional
public class DepartmentService {

    private final DepartmentRepository departmentRepository;

    public DepartmentService(DepartmentRepository departmentRepository) {
        this.departmentRepository = departmentRepository;
    }

    public Department createDepartmentWithEmployees() {
        Department dept = new Department("Engineering");

        // Use helper methods - cascade persists the employees
        dept.addEmployee(new Employee("Alice", "alice@co.com"));
        dept.addEmployee(new Employee("Bob", "bob@co.com"));
        dept.addEmployee(new Employee("Carol", "carol@co.com"));

        return departmentRepository.save(dept);
        // ONE save call persists department AND all 3 employees
    }
}
```

```
Cascade Persist Flow:
+-------------------------------------------------------------------+
|                                                                   |
|  departmentRepository.save(dept)                                  |
|       |                                                           |
|       v                                                           |
|  1. INSERT INTO departments (name) VALUES ('Engineering')         |
|     --> Generated id = 1                                          |
|                                                                   |
|  2. CascadeType.ALL propagates PERSIST to children:               |
|     INSERT INTO employees (name, email, department_id)            |
|     VALUES ('Alice', 'alice@co.com', 1)                           |
|                                                                   |
|     INSERT INTO employees (name, email, department_id)            |
|     VALUES ('Bob', 'bob@co.com', 1)                               |
```

```
|                                                                  |
|     INSERT INTO employees (name, email, department_id)           |
|     VALUES ('Carol', 'carol@co.com', 1)                          |
|                                                                  |
|  Result: 4 INSERT statements from 1 save() call                 |
+------------------------------------------------------------------+
```

Loading and Navigating

```java
// Load department - employees are LAZY by default
Department dept = departmentRepository.findById(1L).orElseThrow();
// SQL: SELECT * FROM departments WHERE id = 1

// Access employees - triggers lazy loading
List<Employee> employees = dept.getEmployees();
// SQL: SELECT * FROM employees WHERE department_id = 1

// Load with employees eagerly (avoid N+1)
@Query("SELECT d FROM Department d LEFT JOIN FETCH d.employees WHERE d.id = :id")
Optional<Department> findByIdWithEmployees(@Param("id") Long id);
// SQL: SELECT d.*, e.* FROM departments d
//      LEFT JOIN employees e ON d.id = e.department_id WHERE d.id = 1
```

Adding and Removing Children

```java
@Transactional
public void addEmployeeToDepartment(Long deptId, String name, String email) {
    Department dept = departmentRepository.findById(deptId).orElseThrow();
    dept.addEmployee(new Employee(name, email));
    // Cascade persists the new employee. No explicit save needed.
}

@Transactional
public void removeEmployeeFromDepartment(Long deptId, Long employeeId) {
    Department dept = departmentRepository.findByIdWithEmployees(deptId)
        .orElseThrow();

    dept.getEmployees().stream()
        .filter(e -> e.getId().equals(employeeId))
        .findFirst()
        .ifPresent(dept::removeEmployee);
    // orphanRemoval deletes the employee from the database
}

@Transactional
public void transferEmployee(Long fromDeptId, Long toDeptId, Long employeeId) {
```

```
    Department fromDept = departmentRepository.findByIdWithEmployees(fromDeptId)
        .orElseThrow();
    Department toDept = departmentRepository.findById(toDeptId).orElseThrow();

    Employee employee = fromDept.getEmployees().stream()
        .filter(e -> e.getId().equals(employeeId))
        .findFirst()
        .orElseThrow();

    fromDept.removeEmployee(employee);  // Removes from old department
    toDept.addEmployee(employee);       // Adds to new department
    // FK updated automatically (owning side changed)
}
```

```
Transfer Employee Flow:
+-------------------------------------------------------------------+
|                                                                   |
|  Before:                                                          |
|  Engineering [Alice, Bob, Carol]    Marketing [Dave]              |
|                                                                   |
|  transferEmployee(engId, mktId, bobId)                            |
|                                                                   |
|  1. fromDept.removeEmployee(bob)                                  |
|     - Engineering.employees: [Alice, Carol]                       |
|     - bob.department = null                                       |
|                                                                   |
|  2. toDept.addEmployee(bob)                                       |
|     - Marketing.employees: [Dave, Bob]                            |
|     - bob.department = Marketing                                  |
|                                                                   |
|  3. Hibernate flushes:                                            |
|     UPDATE employees SET department_id = 2 WHERE id = 11          |
|     (Bob's FK changes from Engineering to Marketing)              |
|                                                                   |
|  After:                                                           |
|  Engineering [Alice, Carol]    Marketing [Dave, Bob]              |
+-------------------------------------------------------------------+
```

The Unidirectional @OneToMany Anti-Pattern

A unidirectional `@OneToMany` (without `mappedBy`, without a corresponding `@ManyToOne`) should be avoided. It causes Hibernate to create an extra join table:

```
Anti-Pattern: Unidirectional @OneToMany
+-------------------------------------------------------------------+
```

```
|                                                                    |
|  @Entity                                                           |
|  public class Department {                                         |
|      @OneToMany        // NO mappedBy, NO @JoinColumn              |
|      private List<Employee> employees;                             |
|  }                                                                 |
|                                                                    |
|  @Entity                                                           |
|  public class Employee {                                           |
|      // NO @ManyToOne field!                                       |
|  }                                                                 |
|                                                                    |
|  Hibernate creates THREE tables:                                   |
|  +-------------+    +---------------------+    +-------------+    |
|  | departments |    | departments_employees|    | employees   |    |
|  | id | name   |    | dept_id | emp_id    |    | id | name   |    |
|  +-------------+    +---------------------+    +-------------+    |
|                                                                    |
|  Problems:                                                         |
|  - Extra join table (unnecessary for one-to-many)                  |
|  - Extra JOINs in every query                                      |
|  - INSERT and DELETE operations are more complex                   |
|  - Hibernate deletes ALL rows in join table then re-inserts        |
|    when the collection changes (terrible performance!)             |
+--------------------------------------------------------------------+
```

The Fix

If you truly need a unidirectional `@OneToMany` (the child does not have a `@ManyToOne` reference), use `@JoinColumn` to specify the FK column directly:

```java
// Acceptable: Unidirectional @OneToMany WITH @JoinColumn
@Entity
public class Department {

    @OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
    @JoinColumn(name = "department_id")  // FK column in employees table
    private List<Employee> employees = new ArrayList<>();
}
```

But the best practice is still to use a bidirectional mapping:

```
Recommended Approaches (best to worst):
+--------------------------------------------------------------------+
|                                                                    |
|  1. Bidirectional @ManyToOne + @OneToMany(mappedBy)   BEST         |
|     - Most efficient SQL                                           |
|     - Navigate both directions                                     |
```

```
|     - FK managed by @ManyToOne side                                |
|                                                                    |
|  2. @ManyToOne only (no @OneToMany)                  GOOD          |
|     - Simplest mapping                                             |
|     - Use JPQL joins when you need the collection                  |
|     - Avoids collection management complexity                      |
|                                                                    |
|  3. Unidirectional @OneToMany WITH @JoinColumn        OK           |
|     - Works but generates extra UPDATE statements                  |
|     - INSERT employee, then UPDATE employee set FK                 |
|                                                                    |
|  4. Unidirectional @OneToMany without @JoinColumn     AVOID!       |
|     - Creates unnecessary join table                               |
|     - Terrible performance on collection changes                   |
+--------------------------------------------------------------------+
```

Cascade and orphanRemoval in Depth

CascadeType Options

```java
@OneToMany(mappedBy = "department",
           cascade = CascadeType.ALL,     // Shortcut for all types below
           orphanRemoval = true)
private List<Employee> employees;
```

```
Cascade Types Explained:
+--------------------------------------------------------------------+
|                                                                    |
|  CascadeType      When It Fires         What It Does               |
|  ------------------------------------------------------------------|
|  PERSIST           save(parent)          Also save new children   |
|                                                                    |
|  MERGE             merge(parent)         Also merge children      |
|                    (reattach detached)                             |
|                                                                    |
|  REMOVE            delete(parent)        Also delete children     |
|                                                                    |
|  REFRESH           refresh(parent)       Also refresh children    |
|                    (reload from DB)                                |
|                                                                    |
|  DETACH            detach(parent)        Also detach children     |
|                    (remove from context)                           |
|                                                                    |
|  ALL               All of the above      Everything cascades      |
|                                                                    |
```

```
+-----------------------------------------------------------------+
|                                                                 |
|  orphanRemoval = true:                                          |
|  When a child is REMOVED from the collection (but the parent    |
|  is NOT deleted), delete the orphaned child.                    |
|                                                                 |
|  Example:                                                       |
|  dept.removeEmployee(bob);                                      |
|  // orphanRemoval: DELETE FROM employees WHERE id = bob.id      |
|  // without orphanRemoval: bob just has department_id = null    |
+-----------------------------------------------------------------+
```

When to Use Each Cascade

```
Cascade Decision Guide:
+-----------------------------------------------------------------+
|                                                                 |
|  Parent owns children completely?                               |
|  (Children cannot exist without parent)                         |
|  Example: Order --> OrderItems                                  |
|      |                                                          |
|   YES --> cascade = CascadeType.ALL, orphanRemoval = true       |
|                                                                 |
|  Parent creates children but they are shared?                   |
|  (Children may belong to other parents too)                     |
|  Example: Department --> Employees                              |
|      |                                                          |
|   MAYBE --> cascade = {PERSIST, MERGE}                          |
|             (do NOT cascade REMOVE - would delete shared data)  |
|             (do NOT use orphanRemoval - employee moves depts)   |
|                                                                 |
|  Parent just references existing children?                      |
|  (Children are managed independently)                           |
|  Example: Tag --> Products (many-to-many via join)              |
|      |                                                          |
|   NO --> No cascade at all                                      |
+-----------------------------------------------------------------+
```

Collection Types: List vs Set

JPA supports several collection types for `@OneToMany`:

```java
// List — ordered, allows duplicates
@OneToMany(mappedBy = "department")
private List<Employee> employees = new ArrayList<>();

// Set — unordered, no duplicates
@OneToMany(mappedBy = "department")
private Set<Employee> employees = new HashSet<>();
```

```
List vs Set for @OneToMany:
+-------------------------------------------------------------------+
|                                                                   |
|  List<Employee>                    Set<Employee>                  |
|  ---------------------------       ---------------------------    |
|  Preserves insertion order         No guaranteed order            |
|  Allows duplicates (rare in DB)    No duplicates                  |
|  Uses ArrayList internally         Uses HashSet internally        |
|  Remove by index or value          Remove by value only           |
|                                                                   |
|  Hibernate behavior:               Hibernate behavior:            |
|  - remove(obj) works fine          - Requires equals/hashCode!    |
|  - No extra requirements           - Without proper equals,       |
|                                      contains() and remove()      |
|                                      will not work correctly      |
|                                                                   |
|  Recommendation:                                                  |
|  Use List for simplicity unless you need Set semantics.           |
|  If using Set, ALWAYS implement equals() and hashCode()           |
|  based on a business key (NOT the auto-generated id).             |
+-------------------------------------------------------------------+
```

Implementing equals and hashCode for Entities

When using `Set<Employee>`, you need proper `equals()` and `hashCode()`:

```java
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String email;

    // ... other fields

    @Override
```

```
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Employee employee = (Employee) o;
        // Use business key (email), NOT id
        return email != null && email.equals(employee.email);
    }

    @Override
    public int hashCode() {
        // Use business key (email), NOT id
        return email != null ? email.hashCode() : 0;
    }
}
```

```
Why NOT Use id for equals/hashCode:
+-------------------------------------------------------------------+
|                                                                   |
|  Problem with id-based equals:                                    |
|                                                                   |
|  Employee e1 = new Employee("Alice", "alice@co.com");             |
|  // e1.id is null (not yet saved)                                 |
|                                                                   |
|  Set<Employee> set = new HashSet<>();                             |
|  set.add(e1);                                                     |
|  // hashCode based on id = null --> hashCode = 0                  |
|                                                                   |
|  employeeRepository.save(e1);                                     |
|  // e1.id is now 1 --> hashCode changes!                          |
|                                                                   |
|  set.contains(e1); // FALSE! hashCode changed after insertion     |
|  // The Set cannot find the element because the hash bucket       |
|  // changed. The element is "lost" in the Set.                    |
|                                                                   |
|  Solution: Use a stable business key (email, ISBN, SSN)           |
|  that does not change between new and persisted states.           |
+-------------------------------------------------------------------+
```

Querying One-to-Many Relationships

Derived Query Methods

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
```

```
    // Find employees by department name
    List<Employee> findByDepartmentName(String deptName);

    // Count employees in a department
    long countByDepartmentId(Long deptId);

    // Find by department and salary
    List<Employee> findByDepartmentNameAndSalaryGreaterThan(
        String deptName, BigDecimal minSalary);

    // Check if department has any employees
    boolean existsByDepartmentId(Long deptId);
}

@Repository
public interface DepartmentRepository extends JpaRepository<Department, Long> {

    // Find departments that have employees with a certain name
    List<Department> findByEmployeesName(String employeeName);

    // Find departments with employees earning above threshold
    @Query("SELECT DISTINCT d FROM Department d JOIN d.employees e WHERE e.salary > :min")
    List<Department> findDepartmentsWithHighEarners(@Param("min") BigDecimal min);
}
```

JPQL Queries

```java
// JOIN FETCH to avoid N+1
@Query("SELECT d FROM Department d LEFT JOIN FETCH d.employees WHERE d.name = :name")
Optional<Department> findByNameWithEmployees(@Param("name") String name);

// Aggregate query
@Query("SELECT d.name, COUNT(e), AVG(e.salary) " +
       "FROM Department d LEFT JOIN d.employees e " +
       "GROUP BY d.name ORDER BY COUNT(e) DESC")
List<Object[]> getDepartmentStatistics();

// Find departments with employee count
@Query("SELECT new com.example.dto.DeptSummary(d.name, SIZE(d.employees)) " +
       "FROM Department d")
List<DeptSummary> getDepartmentSummaries();
```

@OrderBy — Ordering Collection Elements

You can specify a default ordering for collection elements:

```java
@Entity
public class Department {

    @OneToMany(mappedBy = "department")
    @OrderBy("name ASC")  // JPA standard — order by employee name
    private List<Employee> employees = new ArrayList<>();
}

// Multiple sort fields
@OneToMany(mappedBy = "post")
@OrderBy("createdAt DESC, author ASC")  // Newest first, then by author
private List<Comment> comments = new ArrayList<>();
```

```
@OrderBy Behavior:
+-------------------------------------------------------------------+
|                                                                   |
|  When Hibernate loads the collection, it adds ORDER BY to SQL:    |
|                                                                   |
|  Without @OrderBy:                                                |
|  SELECT * FROM employees WHERE department_id = 1                  |
|  (no guaranteed order)                                            |
|                                                                   |
|  With @OrderBy("name ASC"):                                       |
|  SELECT * FROM employees WHERE department_id = 1 ORDER BY name    |
|  (always alphabetical)                                            |
|                                                                   |
|  Note: @OrderBy uses JPQL field names, not column names.          |
|  Use "name" not "employee_name".                                  |
+-------------------------------------------------------------------+
```

Complete Example: Post and Comments

A practical blog example with comments as owned children:

```java
@Entity
@Table(name = "posts")
public class Post {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;
```

```
    @Column(columnDefinition = "TEXT")
    private String content;

    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @OneToMany(mappedBy = "post", cascade = CascadeType.ALL,
               orphanRemoval = true)
    @OrderBy("createdAt ASC")
    private List<Comment> comments = new ArrayList<>();

    protected Post() {}

    public Post(String title, String content) {
        this.title = title;
        this.content = content;
        this.createdAt = LocalDateTime.now();
    }

    // Helper methods
    public Comment addComment(String author, String text) {
        Comment comment = new Comment(author, text);
        comments.add(comment);
        comment.setPost(this);
        return comment;
    }

    public void removeComment(Comment comment) {
        comments.remove(comment);
        comment.setPost(null);
    }

    public int getCommentCount() {
        return comments.size();
    }

    // Getters
    public Long getId() { return id; }
    public String getTitle() { return title; }
    public String getContent() { return content; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public List<Comment> getComments() {
        return Collections.unmodifiableList(comments);
    }
}
```

```java
@Entity
@Table(name = "comments")
public class Comment {

    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String author;

    @Column(nullable = false, columnDefinition = "TEXT")
    private String text;

    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "post_id", nullable = false)
    private Post post;

    protected Comment() {}

    public Comment(String author, String text) {
        this.author = author;
        this.text = text;
        this.createdAt = LocalDateTime.now();
    }

    // Getters and setters
    public Long getId() { return id; }
    public String getAuthor() { return author; }
    public String getText() { return text; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public Post getPost() { return post; }
    public void setPost(Post post) { this.post = post; }
}
```

```java
@Repository
public interface PostRepository extends JpaRepository<Post, Long> {

    @EntityGraph(attributePaths = {"comments"})
    Optional<Post> findWithCommentsById(Long id);

    @Query("SELECT p FROM Post p LEFT JOIN FETCH p.comments WHERE p.title LIKE %:keywor...
    List<Post> searchWithComments(@Param("keyword") String keyword);

    @Query("SELECT p FROM Post p WHERE SIZE(p.comments) >= :minComments")
    List<Post> findPopularPosts(@Param("minComments") int minComments);
}
```

```java
@Service
@Transactional
public class BlogService {
```

```
    private final PostRepository postRepository;

    public BlogService(PostRepository postRepository) {
        this.postRepository = postRepository;
    }

    public Post createPostWithComments() {
        Post post = new Post("My First Post", "This is the content...");
        post.addComment("Alice", "Great post!");
        post.addComment("Bob", "Thanks for sharing.");
        return postRepository.save(post);
    }

    public void addComment(Long postId, String author, String text) {
        Post post = postRepository.findById(postId).orElseThrow();
        post.addComment(author, text);
        // Cascade saves the new comment
    }

    public void deleteComment(Long postId, Long commentId) {
        Post post = postRepository.findWithCommentsById(postId).orElseThrow();
        post.getComments().stream()
            .filter(c -> c.getId().equals(commentId))
            .findFirst()
            .ifPresent(post::removeComment);
        // orphanRemoval deletes the comment
    }

    @Transactional(readOnly = true)
    public Post getPostWithComments(Long postId) {
        return postRepository.findWithCommentsById(postId)
            .orElseThrow(() -> new RuntimeException("Post not found"));
    }
}
```

Avoiding the N+1 Problem

The N+1 problem is especially common with `@OneToMany`:

```
The N+1 Problem:
+-------------------------------------------------------------------+
|                                                                   |
|  // Load all departments                                          |
|  List<Department> departments = departmentRepository.findAll();   |
|  // SQL: SELECT * FROM departments   --> returns 10 departments   |
|                                                                   |
|  // Access each department's employees                            |
|  for (Department dept : departments) {                            |
```

```
|      System.out.println(dept.getName() + ": " +                    |
|          dept.getEmployees().size());  // Lazy load triggered!     |
|  }                                                                 |
|                                                                    |
|  SQL queries generated:                                            |
|  1. SELECT * FROM departments                        (1 query)     |
|  2. SELECT * FROM employees WHERE department_id = 1  (query 2)     |
|  3. SELECT * FROM employees WHERE department_id = 2  (query 3)     |
|  ...                                                               |
|  11. SELECT * FROM employees WHERE department_id = 10 (query 11)   |
|                                                                    |
|  Total: 1 + 10 = 11 queries (N+1 where N=10)                      |
+--------------------------------------------------------------------+

Solutions:
+--------------------------------------------------------------------+
|                                                                    |
|  1. JOIN FETCH in JPQL:                                            |
|     @Query("SELECT d FROM Department d LEFT JOIN FETCH             |
|             d.employees")                                          |
|     List<Department> findAllWithEmployees();                       |
|     --> 1 query with LEFT JOIN                                     |
|                                                                    |
|  2. @EntityGraph:                                                  |
|     @EntityGraph(attributePaths = {"employees"})                   |
|     List<Department> findAll();                                    |
|     --> 1 query with LEFT JOIN                                     |
|                                                                    |
|  3. @BatchSize (Hibernate-specific):                               |
|     @OneToMany(mappedBy = "department")                            |
|     @BatchSize(size = 10)                                          |
|     private List<Employee> employees;                              |
|     --> Loads employees for up to 10 departments in 1 query        |
|     --> Reduces N+1 to 1 + ceil(N/10)                              |
+--------------------------------------------------------------------+
```

Common Mistakes

1. Modifying only the @OneToMany collection without setting @ManyToOne: The collection side (inverse) does not control the FK. You must set the `@ManyToOne` side for the database to be updated.
2. Using unidirectional @OneToMany without @JoinColumn: This creates an unnecessary join table with terrible performance. Always add `@JoinColumn` or use a bidirectional mapping.
3. Cascading REMOVE on shared children: If an employee can transfer between departments, `cascade = CascadeType.REMOVE` would delete the employee when the

department is deleted. Use `cascade = {PERSIST, MERGE}` instead.

4. Forgetting to initialize the collection: `private List<Employee> employees;` (null) will throw `NullPointerException` when you call `addEmployee()`. Always initialize: `= new ArrayList<>()`.
5. Loading large collections unnecessarily: If a department has 10,000 employees, `dept.getEmployees()` loads all of them. Use JPQL queries with pagination instead of navigating through the collection.
6. Not setting fetch = FetchType.LAZY on @ManyToOne: The default for `@ManyToOne` is EAGER. Loading 1,000 employees loads 1,000 departments eagerly. Always set `FetchType.LAZY`.

Best Practices

1. Always set `fetch = FetchType.LAZY` on @ManyToOne: The default EAGER loading is almost never what you want. Load the parent entity on demand with `@EntityGraph` or `JOIN FETCH`.
2. Use bidirectional mappings with helper methods: The combination of `@ManyToOne` + `@OneToMany(mappedBy)` with `addChild()`/`removeChild()` helpers is the cleanest and most efficient approach.
3. Use `cascade = CascadeType.ALL` and `orphanRemoval` only for truly owned children: Order items belong to an order (cascade ALL). Employees do not belong to a department in the same way (they can transfer).
4. Return unmodifiable collections from getters: `return Collections.unmodifiableList(employees)` forces callers to use helper methods, maintaining consistency.
5. Prefer JPQL queries over collection navigation for large sets: Instead of `dept.getEmployees().stream().filter(...)`, use `employeeRepository.findByDepartmentIdAndSalaryGreaterThan(deptId, min)`.
6. Initialize collections to avoid NullPointerException: Always write `= new ArrayList<>()` or `= new HashSet<>()` in the field declaration.

Summary

In this chapter, you learned how to map the most common relationship type:

- @ManyToOne is the owning side and should always have `fetch = FetchType.LAZY`. The foreign key column lives in the child table.
- @OneToMany(mappedBy) is the inverse side. It provides collection navigation but does not control the foreign key. The `mappedBy` value is the field name on the child entity.
- Helper methods (`addChild()`, `removeChild()`) are essential for keeping both sides of a bidirectional relationship synchronized.
- Cascade propagates persistence operations from parent to child. Use `CascadeType.ALL` for truly owned children; use `{PERSIST, MERGE}` for shared children.
- orphanRemoval deletes children that are removed from the collection, not just when the parent is deleted.
- Avoid unidirectional @OneToMany without @JoinColumn — it creates a join table with poor performance.
- The N+1 problem occurs when lazy-loading collections in a loop. Solve it with `JOIN FETCH`, `@EntityGraph`, or `@BatchSize`.
- @OrderBy defines a default ordering for collection elements.

Interview Questions

Q1: What is the difference between @ManyToOne and @OneToMany?

They are two sides of the same relationship. `@ManyToOne` is on the child entity (many employees belong to one department) and is the owning side that controls the foreign key. `@OneToMany` is on the parent entity (one department has many employees) and is the inverse side that provides collection navigation via `mappedBy`.

Q2: Why is it important to use helper methods in bidirectional relationships?

Helper methods synchronize both sides of the relationship in memory. Without them, you might only set the collection side (inverse), which does not update the foreign key. Or you might only set the `@ManyToOne` side, which updates the FK but leaves the parent's collection inconsistent. Helper methods ensure both sides are always in sync.

Q3: What happens if you use @OneToMany without mappedBy or @JoinColumn?

Hibernate creates an unnecessary join table (e.g., `departments_employees`) to manage the relationship. This adds an extra table, requires extra JOINs, and causes poor performance — especially for collection modifications where Hibernate deletes all join table rows and re-inserts them.

Q4: When should you use orphanRemoval = true?

When the child entity cannot meaningfully exist without its parent. Examples: OrderItem without an Order, Comment without a Post. When you remove the child from the parent's collection, `orphanRemoval` automatically deletes it from the database. Do not use it when children can be transferred to another parent (e.g., Employee transferring departments).

Q5: What is the N+1 problem and how do you solve it with @OneToMany?

Loading N parent entities and then accessing their lazy collections triggers N additional queries (1 for parents + N for each collection). Solutions: (1) `JOIN FETCH` in JPQL loads everything in one query, (2) `@EntityGraph` specifies eager loading per query, (3) `@BatchSize` (Hibernate) batches multiple collection loads into fewer queries.

Q6: Why should @ManyToOne default fetch type be changed to LAZY?

The JPA default for `@ManyToOne` is `FetchType.EAGER`, which loads the parent entity immediately with every child query. If you load 1,000 employees, it joins the departments table 1,000 times even if you do not need department data. Setting `FetchType.LAZY` defers loading until you actually call `employee.getDepartment()`.

Practice Exercises

Exercise 1: Basic Bidirectional Mapping Create `Category` and `Product` entities with a bidirectional one-to-many relationship. Write helper methods, a service to add/remove products from categories, and test all operations.

Exercise 2: Cascade Behavior Create `Order` and `OrderItem` entities where items are fully owned by the order. Test: (a) creating an order with 3 items in one save, (b) removing an item (orphanRemoval), (c) deleting the order (cascade removes all items).

Exercise 3: N+1 Detection and Fix Load all departments and print each department's employee count. Enable SQL logging. Count the queries (observe N+1). Fix it with (a) JOIN FETCH and (b) @EntityGraph. Verify the query count drops to 1.

Exercise 4: Transfer Between Parents Implement an employee transfer feature. Write a service method that moves an employee from one department to another. Verify the FK is updated and both collections are correct. Handle edge cases (employee not found, same department).

Exercise 5: Blog Application Build a blog with `Author`, `Post`, and `Comment` entities. Author has many Posts (bidirectional). Post has many Comments (bidirectional, owned). Write a service with: create post, add comment, delete comment, get post with comments, get author with all posts and comments (solve N+1).

What Is Next?

In the next chapter, we will explore Many-to-Many Relationships — the most complex relationship type. You will learn how to use `@ManyToMany` with `@JoinTable`, add extra columns to join tables by breaking them into two `@OneToMany` relationships, handle the equals/hashCode challenge with Sets, and choose the right collection type for many-to-many associations.

Chapter 14: Many-to-Many Relationships

Learning Goals

By the end of this chapter, you will be able to:

- Understand many-to-many relationships and how they are stored in databases
- Map @ManyToMany with @JoinTable in JPA
- Configure join table column names with @JoinColumn and @InverseJoinColumn
- Implement bidirectional many-to-many with helper methods
- Add extra columns to a join table by converting to two @OneToMany relationships
- Choose between Set and List for many-to-many collections
- Understand the equals/hashCode contract for entities in Sets
- Query many-to-many relationships with JPQL and derived methods
- Avoid common performance pitfalls with many-to-many

Understanding Many-to-Many Relationships

A many-to-many relationship exists when entities on both sides can be associated with multiple entities on the other side.

```
Many-to-Many Examples:
+-------------------------------------------------------------------+
|                                                                   |
|  Student >----< Course     (A student takes many courses,         |
|                             a course has many students)           |
|                                                                   |
```

```
|  Book >----< Author        (A book can have many authors,         |
|                             an author can write many books)       |
|                                                                   |
|  User >----< Role          (A user can have many roles,           |
|                             a role can be assigned to many users) |
|                                                                   |
|  Product >----< Tag        (A product can have many tags,         |
|                             a tag applies to many products)       |
|                                                                   |
|  Employee >----< Project   (An employee works on many projects,   |
|                             a project has many employees)         |
|                                                                   |
|  Legend:  >----<  means "many-to-many"                            |
+-------------------------------------------------------------------+
```

How Databases Store Many-to-Many

Relational databases cannot represent many-to-many directly. They use a join table (also called an association table, junction table, or link table):

```
Database Schema for Student-Course:
+-------------------------------------------------------------------+
|                                                                   |
|  students              student_courses           courses          |
|  +------+--------+    +------------+-----------+ +------+-------+ |
|  |id(PK)| name   |    |student_id  | course_id | |id(PK)| title | |
|  |------+--------|    |  (FK)      |   (FK)    | |------+-------| |
|  | 1    | Alice  |    | 1          | 101       | | 101  | Math  | |
|  | 2    | Bob    |    | 1          | 102       | | 102  | Sci   | |
|  | 3    | Carol  |    | 2          | 101       | | 103  | Eng   | |
|  +------+--------+    | 2          | 103       | +------+-------+ |
|                        | 3          | 102       |                 |
|                        +------------+-----------+                 |
|                                                                    |
|  Reading the join table:                                           |
|  Alice (1) takes Math (101) and Sci (102)                         |
|  Bob (2) takes Math (101) and Eng (103)                           |
|  Carol (3) takes Sci (102)                                        |
|                                                                    |
|  The join table has:                                               |
|  - NO primary key of its own (composite PK of both FKs)          |
|  - ONLY foreign keys (no extra data)                               |
|  - Two FK columns referencing each side                            |
+-------------------------------------------------------------------+
```

Basic @ManyToMany Mapping

The Owning Side

One side must be the owning side (defines the `@JoinTable`). By convention, choose whichever side makes more sense as the "primary" entity:

```java
@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @ManyToMany                                         // JPA standard
    @JoinTable(
        name = "student_courses",                       // Join table name
        joinColumns = @JoinColumn(name = "student_id"),       // FK to this entity
        inverseJoinColumns = @JoinColumn(name = "course_id")  // FK to the other entity
    )
    private Set<Course> courses = new HashSet<>();

    protected Student() {}

    public Student(String name, String email) {
        this.name = name;
        this.email = email;
    }

    // ===== Helper Methods =====
    public void enrollIn(Course course) {
        courses.add(course);
        course.getStudentsInternal().add(this);
    }

    public void dropCourse(Course course) {
        courses.remove(course);
        course.getStudentsInternal().remove(this);
    }

    // Getters
    public Long getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public Set<Course> getCourses() {
        return Collections.unmodifiableSet(courses);
    }
```

```
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Student student = (Student) o;
        return email != null && email.equals(student.email);
    }

    @Override
    public int hashCode() {
        return email != null ? email.hashCode() : 0;
    }
}
```

```
@JoinTable Annotation Breakdown:
+-------------------------------------------------------------------+
|                                                                   |
|  @JoinTable(                                                      |
|      name = "student_courses",    // Name of the join table       |
|                                                                   |
|      joinColumns = @JoinColumn(   // FK pointing to THIS entity   |
|          name = "student_id"      // (Student.id)                 |
|      ),                                                           |
|                                                                   |
|      inverseJoinColumns =         // FK pointing to the OTHER     |
|          @JoinColumn(             // entity (Course.id)           |
|              name = "course_id"                                   |
|          )                                                        |
|  )                                                                |
|                                                                   |
|  Generated DDL:                                                   |
|  CREATE TABLE student_courses (                                   |
|      student_id BIGINT NOT NULL,                                  |
|      course_id BIGINT NOT NULL,                                   |
|      PRIMARY KEY (student_id, course_id),                         |
|      FOREIGN KEY (student_id) REFERENCES students(id),            |
|      FOREIGN KEY (course_id) REFERENCES courses(id)               |
|  );                                                               |
+-------------------------------------------------------------------+
```

The Inverse Side

```java
@Entity
@Table(name = "courses")
public class Course {

    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    @Column(nullable = false)
    private int credits;

    @ManyToMany(mappedBy = "courses")   // Inverse side - points to owning field
    private Set<Student> students = new HashSet<>();

    protected Course() {}

    public Course(String title, int credits) {
        this.title = title;
        this.credits = credits;
    }

    // Internal access for helper methods (package-private)
    Set<Student> getStudentsInternal() {
        return students;
    }

    // Getters
    public Long getId() { return id; }
    public String getTitle() { return title; }
    public int getCredits() { return credits; }
    public Set<Student> getStudents() {
        return Collections.unmodifiableSet(students);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Course course = (Course) o;
        return title != null && title.equals(course.title);
    }

    @Override
    public int hashCode() {
        return title != null ? title.hashCode() : 0;
    }
}
```

```
Bidirectional @ManyToMany:
+-------------------------------------------------------------------+
|                                                                   |
|  Student (Owning Side)              Course (Inverse Side)         |
|  +---------------------------+     +---------------------------+  |
|  | @ManyToMany               |     | @ManyToMany(              |  |
|  | @JoinTable(               |     |   mappedBy = "courses")   |  |
```

```
|  |   name="student_courses", |     | Set<Student> students     |  |
|  |   joinColumns=...,        |     +---------------------------+  |
|  |   inverseJoinColumns=...) |                                    |
|  | Set<Course> courses       |                                    |
|  +---------------------------+                                    |
|                                                                   |
|  The owning side defines the @JoinTable.                          |
|  The inverse side uses mappedBy.                                  |
|  Helper methods on the owning side update BOTH sides.             |
+-------------------------------------------------------------------+
```

Working with Many-to-Many

Enrolling Students in Courses

```java
@Service
@Transactional
public class EnrollmentService {

    private final StudentRepository studentRepository;
    private final CourseRepository courseRepository;

    public EnrollmentService(StudentRepository studentRepository,
                             CourseRepository courseRepository) {
        this.studentRepository = studentRepository;
        this.courseRepository = courseRepository;
    }

    public void enrollStudentInCourse(Long studentId, Long courseId) {
        Student student = studentRepository.findById(studentId).orElseThrow();
        Course course = courseRepository.findById(courseId).orElseThrow();

        student.enrollIn(course);
        // Helper method updates both sides
        // Hibernate inserts into student_courses join table
    }

    public void dropStudentFromCourse(Long studentId, Long courseId) {
        Student student = studentRepository.findById(studentId).orElseThrow();
        Course course = courseRepository.findById(courseId).orElseThrow();

        student.dropCourse(course);
        // Helper method removes from both sides
        // Hibernate deletes from student_courses join table
    }
}
```

```
Enroll Flow (SQL Generated):
+-------------------------------------------------------------------+
|                                                                   |
|  student.enrollIn(course);                                        |
|                                                                   |
|  1. Hibernate detects change in courses Set (dirty checking)      |
|  2. On flush:                                                     |
|     INSERT INTO student_courses (student_id, course_id)           |
|     VALUES (1, 101)                                               |
|                                                                   |
|  student.dropCourse(course);                                      |
|                                                                   |
|  1. Hibernate detects removal from courses Set                    |
|  2. On flush:                                                     |
|     DELETE FROM student_courses                                   |
|     WHERE student_id = 1 AND course_id = 101                     |
|                                                                   |
|  Note: Only the join table is affected.                           |
|  The students and courses tables are NOT modified.                |
+-------------------------------------------------------------------+
```

Loading Data

```java
// Load student with courses eagerly
@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {

    @EntityGraph(attributePaths = {"courses"})
    Optional<Student> findWithCoursesById(Long id);

    @Query("SELECT DISTINCT s FROM Student s LEFT JOIN FETCH s.courses " +
           "WHERE s.name LIKE %:name%")
    List<Student> searchWithCourses(@Param("name") String name);
}

@Repository
public interface CourseRepository extends JpaRepository<Course, Long> {

    @EntityGraph(attributePaths = {"students"})
    Optional<Course> findWithStudentsById(Long id);

    @Query("SELECT c FROM Course c WHERE SIZE(c.students) >= :min")
    List<Course> findPopularCourses(@Param("min") int min);
}
```

Why Set Is Preferred Over List for @ManyToMany

```
List vs Set for @ManyToMany:
+-------------------------------------------------------------------+
|                                                                   |
|  With List<Course>:                                               |
|  student.dropCourse(course);                                      |
|                                                                   |
|  Hibernate generates:                                             |
|  1. DELETE FROM student_courses WHERE student_id = 1  (ALL rows!) |
|  2. INSERT INTO student_courses (student_id, course_id)           |
|     VALUES (1, 102)                                (re-insert!)   |
|  3. INSERT INTO student_courses (student_id, course_id)           |
|     VALUES (1, 103)                                (re-insert!)   |
|                                                                   |
|  Removes ALL entries, then re-inserts the remaining ones!         |
|  If the student has 50 courses and drops 1: 1 DELETE + 49 INSERTs |
|                                                                   |
|  With Set<Course>:                                                |
|  student.dropCourse(course);                                      |
|                                                                   |
|  Hibernate generates:                                             |
|  1. DELETE FROM student_courses                                   |
|     WHERE student_id = 1 AND course_id = 101       (just 1 row!) |
|                                                                   |
|  Removes ONLY the specific entry! Much more efficient.            |
|                                                                   |
|  RULE: Always use Set<T> for @ManyToMany collections.             |
|  List<T> causes terrible performance on modifications.            |
+-------------------------------------------------------------------+
```

This is one of the most important performance rules in Hibernate. Using `List` for `@ManyToMany` causes Hibernate to delete and recreate all join table entries whenever the collection changes. Using `Set` allows surgical inserts and deletes.

Adding Extra Columns to the Join Table

The basic `@ManyToMany` only supports a join table with two foreign key columns. But what if you need extra columns?

```
Problem: Need Extra Data in the Relationship:
+-------------------------------------------------------------------+
|                                                                   |
|  Student enrolls in Course:                                       |
|  - enrollment_date: When did they enroll?                         |
|  - grade: What grade did they receive?                            |
|  - semester: Which semester?                                      |
```

```
|                                                                    |
|  These cannot be stored with @ManyToMany!                          |
|  @ManyToMany only creates a join table with two FK columns.        |
|                                                                    |
|  Solution: Create an explicit join ENTITY                          |
|  and use two @ManyToOne / @OneToMany relationships instead.        |
|                                                                    |
|  Before:                                                           |
|  Student >----< Course                                             |
|  (simple @ManyToMany, no extra columns)                            |
|                                                                    |
|  After:                                                            |
|  Student ----< Enrollment >---- Course                             |
|  (two @OneToMany through an Enrollment entity)                     |
+--------------------------------------------------------------------+
```

The Enrollment Entity (Join Entity)

```java
@Entity
@Table(name = "enrollments")
public class Enrollment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "student_id", nullable = false)
    private Student student;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "course_id", nullable = false)
    private Course course;

    // ===== Extra columns! =====
    @Column(name = "enrolled_at", nullable = false)
    private LocalDate enrolledAt;

    @Column(length = 2)
    private String grade;    // "A", "B+", "C", etc.

    @Column(nullable = false, length = 20)
    private String semester;  // "Fall 2025", "Spring 2026"

    protected Enrollment() {}

    public Enrollment(Student student, Course course, String semester) {
        this.student = student;
        this.course = course;
        this.semester = semester;
```

```
        this.enrolledAt = LocalDate.now();
    }

    // Getters and setters
    public Long getId() { return id; }
    public Student getStudent() { return student; }
    public Course getCourse() { return course; }
    public LocalDate getEnrolledAt() { return enrolledAt; }
    public String getGrade() { return grade; }
    public void setGrade(String grade) { this.grade = grade; }
    public String getSemester() { return semester; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Enrollment that = (Enrollment) o;
        return student != null && course != null &&
               student.equals(that.student) && course.equals(that.course) &&
               semester != null && semester.equals(that.semester);
    }

    @Override
    public int hashCode() {
        return Objects.hash(
            student != null ? student.getEmail() : null,
            course != null ? course.getTitle() : null,
            semester
        );
    }
}
```

Updated Student and Course Entities

```java
@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @OneToMany(mappedBy = "student", cascade = CascadeType.ALL,
               orphanRemoval = true)
```

```
    private Set<Enrollment> enrollments = new HashSet<>();

    protected Student() {}

    public Student(String name, String email) {
        this.name = name;
        this.email = email;
    }

    // Helper methods
    public Enrollment enrollIn(Course course, String semester) {
        Enrollment enrollment = new Enrollment(this, course, semester);
        enrollments.add(enrollment);
        course.getEnrollmentsInternal().add(enrollment);
        return enrollment;
    }

    public void drop(Course course, String semester) {
        enrollments.removeIf(e ->
            e.getCourse().equals(course) && e.getSemester().equals(semester));
    }

    public Long getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public Set<Enrollment> getEnrollments() {
        return Collections.unmodifiableSet(enrollments);
    }

    // equals/hashCode based on email (same as before)
}
```

```java
@Entity
@Table(name = "courses")
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    private int credits;

    @OneToMany(mappedBy = "course", cascade = CascadeType.ALL,
               orphanRemoval = true)
    private Set<Enrollment> enrollments = new HashSet<>();

    protected Course() {}

    public Course(String title, int credits) {
```

```
        this.title = title;
        this.credits = credits;
    }

    Set<Enrollment> getEnrollmentsInternal() {
        return enrollments;
    }

    public Long getId() { return id; }
    public String getTitle() { return title; }
    public int getCredits() { return credits; }
    public Set<Enrollment> getEnrollments() {
        return Collections.unmodifiableSet(enrollments);
    }

    // equals/hashCode based on title (same as before)
}
```

```
Database Schema with Join Entity:
+-------------------------------------------------------------------+
|                                                                   |
|  students             enrollments              courses            |
|  +------+--------+   +------+------+------+   +------+-------+   |
|  |id(PK)| name   |   |id(PK)|stu_id|crs_id|  |id(PK)| title |   |
|  |------+--------|   |      |(FK)  |(FK)  |   |------+-------|   |
|  | 1    | Alice  |   | 1    | 1    | 101  |   | 101  | Math  |   |
|  | 2    | Bob    |   | 2    | 1    | 102  |   | 102  | Sci   |   |
|  +------+--------+   | 3    | 2    | 101  |   | 103  | Eng   |   |
|                       +------+------+------+   +------+-------+   |
|                       |enrolled_at  |grade |                      |
|                       |-------------|------|                      |
|                       |2025-09-01   | A    |                      |
|                       |2025-09-01   | B+   |                      |
|                       |2025-09-05   | null |                      |
|                       +-------------+------+                      |
|                                                                   |
|  Now we can store enrollment_date, grade, semester, etc.          |
|  Each enrollment is its own entity with its own id.               |
+-------------------------------------------------------------------+
```

Using the Join Entity

```java
@Service
@Transactional
public class EnrollmentService {

    private final StudentRepository studentRepository;
    private final CourseRepository courseRepository;
```

```
    private final EnrollmentRepository enrollmentRepository;

    // Constructor...

    public void enroll(Long studentId, Long courseId, String semester) {
        Student student = studentRepository.findById(studentId).orElseThrow();
        Course course = courseRepository.findById(courseId).orElseThrow();
        student.enrollIn(course, semester);
    }

    public void assignGrade(Long enrollmentId, String grade) {
        Enrollment enrollment = enrollmentRepository.findById(enrollmentId)
            .orElseThrow();
        enrollment.setGrade(grade);
    }

    @Transactional(readOnly = true)
    public List<Enrollment> getStudentTranscript(Long studentId) {
        return enrollmentRepository.findByStudentIdOrderBySemesterDesc(studentId);
    }

    @Transactional(readOnly = true)
    public double getStudentGPA(Long studentId) {
        return enrollmentRepository.findByStudentId(studentId).stream()
            .filter(e -> e.getGrade() != null)
            .mapToDouble(this::gradeToPoints)
            .average()
            .orElse(0.0);
    }

    private double gradeToPoints(Enrollment e) {
        return switch (e.getGrade()) {
            case "A" -> 4.0;
            case "A-" -> 3.7;
            case "B+" -> 3.3;
            case "B" -> 3.0;
            case "B-" -> 2.7;
            case "C+" -> 2.3;
            case "C" -> 2.0;
            default -> 0.0;
        };
    }
}
```

```java
@Repository
public interface EnrollmentRepository extends JpaRepository<Enrollment, Long> {

    List<Enrollment> findByStudentId(Long studentId);
    List<Enrollment> findByStudentIdOrderBySemesterDesc(Long studentId);
    List<Enrollment> findByCourseId(Long courseId);
    List<Enrollment> findBySemester(String semester);
```

```
    @Query("SELECT e FROM Enrollment e JOIN FETCH e.course " +
           "WHERE e.student.id = :studentId AND e.grade IS NOT NULL")
    List<Enrollment> findGradedByStudent(@Param("studentId") Long studentId);

    @Query("SELECT e.course.title, AVG(CASE " +
           "WHEN e.grade = 'A' THEN 4.0 WHEN e.grade = 'B' THEN 3.0 " +
           "WHEN e.grade = 'C' THEN 2.0 ELSE 0 END) " +
           "FROM Enrollment e WHERE e.grade IS NOT NULL " +
           "GROUP BY e.course.title ORDER BY 2 DESC")
    List<Object[]> getCourseAverageGrades();

    boolean existsByStudentIdAndCourseIdAndSemester(
        Long studentId, Long courseId, String semester);
}
```

@ManyToMany vs Join Entity: Decision Guide

```
When to Use Which:
+-------------------------------------------------------------------+
|                                                                   |
|  Use @ManyToMany when:                                            |
|  +-------------------------------+                                |
|  | - Join table has ONLY two FKs |                                |
|  | - No extra data on the link   |                                |
|  | - Simple tag/label/role       |                                |
|  |   associations                |                                |
|  | - You do not need to query    |                                |
|  |   the relationship itself     |                                |
|  +-------------------------------+                                |
|                                                                   |
|  Examples:                                                        |
|  - Product <-> Tag                                                |
|  - User <-> Role                                                  |
|  - Article <-> Category                                           |
|                                                                   |
|  Use Join Entity when:                                            |
|  +-------------------------------+                                |
|  | - Join table needs extra      |                                |
|  |   columns (date, status,     |                               |
|  |   quantity, grade, etc.)      |                                |
|  | - You need to query the       |                                |
|  |   relationship itself         |                                |
|  | - The link has its own        |                                |
|  |   lifecycle                   |                                |
|  +-------------------------------+                                |
|                                                                   |
|  Examples:                                                        |
|  - Student <-> Course (with grade, semester)                      |
```

```
|  - Order <-> Product (with quantity, price)                        |
|  - Employee <-> Project (with role, start/end date)                |
|  - User <-> Group (with joined_at, is_admin)                       |
+--------------------------------------------------------------------+
|                                                                    |
|  Pro tip: When in doubt, use a join entity.                        |
|  Requirements often grow, and adding columns to @ManyToMany        |
|  later requires a significant refactor. A join entity is           |
|  more work upfront but more flexible long-term.                    |
+--------------------------------------------------------------------+
```

Practical Example: User and Role

A simple `@ManyToMany` example — no extra columns needed:

```java
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
        name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id")
    )
    private Set<Role> roles = new HashSet<>();

    protected User() {}

    public User(String username, String password) {
        this.username = username;
        this.password = password;
    }

    public void addRole(Role role) {
        roles.add(role);
```

```
        role.getUsersInternal().add(this);
    }

    public void removeRole(Role role) {
        roles.remove(role);
        role.getUsersInternal().remove(this);
    }

    public boolean hasRole(String roleName) {
        return roles.stream().anyMatch(r -> r.getName().equals(roleName));
    }

    // Getters
    public Long getId() { return id; }
    public String getUsername() { return username; }
    public Set<Role> getRoles() { return Collections.unmodifiableSet(roles); }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        User user = (User) o;
        return username != null && username.equals(user.username);
    }

    @Override
    public int hashCode() {
        return username != null ? username.hashCode() : 0;
    }
}
```

```java
@Entity
@Table(name = "roles")
public class Role {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    @ManyToMany(mappedBy = "roles")
    private Set<User> users = new HashSet<>();

    protected Role() {}

    public Role(String name) {
        this.name = name;
    }

    Set<User> getUsersInternal() {
```

```
        return users;
    }

    public Long getId() { return id; }
    public String getName() { return name; }
    public Set<User> getUsers() { return Collections.unmodifiableSet(users); }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Role role = (Role) o;
        return name != null && name.equals(role.name);
    }

    @Override
    public int hashCode() {
        return name != null ? name.hashCode() : 0;
    }
}
```

```java
@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByUsername(String username);

    @EntityGraph(attributePaths = {"roles"})
    Optional<User> findWithRolesByUsername(String username);

    // Find users who have a specific role
    @Query("SELECT u FROM User u JOIN u.roles r WHERE r.name = :roleName")
    List<User> findByRoleName(@Param("roleName") String roleName);

    // Find users with multiple roles
    @Query("SELECT u FROM User u WHERE SIZE(u.roles) >= :count")
    List<User> findUsersWithMultipleRoles(@Param("count") int count);
}
```

Cascade Rules for Many-to-Many

```
Cascade Rules for @ManyToMany:
+-----------------------------------------------------------------+
|                                                                  |
|  CRITICAL: Do NOT use CascadeType.REMOVE or CascadeType.ALL      |
|  on @ManyToMany relationships!                                   |
|                                                                  |
```

```
|  Why?                                                              |
|  - If Student has cascade REMOVE on courses:                       |
|    Deleting a student would DELETE the shared courses!             |
|    Other students would lose their courses.                        |
|                                                                    |
|  - If User has cascade REMOVE on roles:                            |
|    Deleting a user would DELETE the shared roles!                  |
|    Other users would lose their role assignments.                  |
|                                                                    |
|  Safe cascade options for @ManyToMany:                             |
|  +----------------------------+                                    |
|  | cascade = {CascadeType.PERSIST, CascadeType.MERGE}             |
|  | or no cascade at all       |                                    |
|  +----------------------------+                                    |
|                                                                    |
|  For join entities (@OneToMany on both sides):                     |
|  cascade = CascadeType.ALL with orphanRemoval = true is OK         |
|  because the enrollment/link is owned by the parent.               |
+--------------------------------------------------------------------+
```

Querying Many-to-Many with JPQL

```java
// Find all students enrolled in a specific course
@Query("SELECT s FROM Student s JOIN s.courses c WHERE c.title = :title")
List<Student> findStudentsByCourse(@Param("title") String courseTitle);

// Find students who take both Math and Science
@Query("SELECT s FROM Student s JOIN s.courses c1 JOIN s.courses c2 " +
       "WHERE c1.title = 'Math' AND c2.title = 'Science'")
List<Student> findStudentsTakingMathAndScience();

// Count enrollments per course
@Query("SELECT c.title, SIZE(c.students) FROM Course c ORDER BY SIZE(c.students) DESC")
List<Object[]> getCoursPopularity();

// Find courses with no students
@Query("SELECT c FROM Course c WHERE c.students IS EMPTY")
List<Course> findEmptyCourses();

// Find students not enrolled in any course
@Query("SELECT s FROM Student s WHERE s.courses IS EMPTY")
List<Student> findUnenrolledStudents();
```

Removing All Associations Before Deleting

When deleting an entity in a many-to-many relationship, you must remove the associations first:

```java
@Service
@Transactional
public class StudentService {

    private final StudentRepository studentRepository;

    // Delete a student - must clear associations first
    public void deleteStudent(Long studentId) {
        Student student = studentRepository.findWithCoursesById(studentId)
            .orElseThrow();

        // Remove student from all courses
        for (Course course : new HashSet<>(student.getCourses())) {
            student.dropCourse(course);
        }

        // Now safe to delete - join table entries already removed
        studentRepository.delete(student);
    }
}
```

```
Deletion Flow:
+-------------------------------------------------------------------+
|                                                                   |
|  WITHOUT clearing associations:                                   |
|  studentRepository.delete(student);                               |
|  --> ERROR! FK constraint violation                               |
|  student_courses still has rows referencing this student           |
|                                                                   |
|  WITH clearing associations:                                      |
|  1. student.dropCourse(math);                                     |
|     DELETE FROM student_courses WHERE student_id=1 AND course_id=101 |
|  2. student.dropCourse(science);                                  |
|     DELETE FROM student_courses WHERE student_id=1 AND course_id=102 |
|  3. studentRepository.delete(student);                            |
|     DELETE FROM students WHERE id = 1     (now safe!)             |
+-------------------------------------------------------------------+
```

Common Mistakes

1. Using List instead of Set for @ManyToMany: List causes Hibernate to delete ALL join table entries and re-insert the remaining ones on every modification. Set allows targeted insert/delete. Always use `Set<T>`.
2. Using CascadeType.REMOVE or ALL on @ManyToMany: This deletes the shared entity when one side is deleted. A student deletion would delete shared courses. Use `{PERSIST, MERGE}` or no cascade.
3. Forgetting helper methods: Setting only one side of the relationship causes inconsistency. Always synchronize both sides with `addX()` and `removeX()` helper methods.
4. Not implementing equals/hashCode when using Set: Without proper `equals()` and `hashCode()`, `Set.contains()`, `Set.remove()`, and `Set.add()` do not work correctly. Use a business key, not the auto-generated `id`.
5. Using @ManyToMany when extra columns are needed: If you need dates, quantities, statuses, or any extra data on the association, you cannot use `@ManyToMany`. Convert to a join entity with two `@ManyToOne` relationships.
6. Not loading associations before deletion: Deleting an entity with active many-to-many associations causes FK constraint violations. Clear all associations first, then delete.

Best Practices

1. Always use Set for @ManyToMany collections: This is a hard rule for performance. `Set` gives O(1) insert/remove on the join table. `List` gives O(n) re-insert.
2. Implement equals/hashCode using business keys: Use a unique, immutable field like `email`, `username`, or `title` — never the auto-generated `id` which is null before persistence.
3. Start with a join entity if you might need extra columns: Requirements evolve. Converting `@ManyToMany` to a join entity later is a significant refactor. When in doubt, use a join entity upfront.
4. Do not cascade REMOVE on @ManyToMany: Cascade only `PERSIST` and `MERGE`, or use no cascade and manage entities independently.
5. Use @EntityGraph or JOIN FETCH to load associations: Avoid N+1 when accessing the many-to-many collections by fetching eagerly at the query level.
6. Consider whether you really need the inverse side: A unidirectional `@ManyToMany` (only on the owning side) is simpler. Add the inverse side only if you need to navigate from both directions.

Summary

In this chapter, you learned how to map the most complex relationship type:

- @ManyToMany creates a join table with two foreign key columns. One side is the owning side (defines `@JoinTable`), the other uses `mappedBy`.
- Always use Set, not List, for `@ManyToMany` collections. List causes full delete-and-reinsert of the join table on every change.
- Helper methods synchronize both sides of the bidirectional relationship, just as with `@OneToMany`.
- Never cascade REMOVE on `@ManyToMany`. It would delete shared entities. Use `{PERSIST, MERGE}` or no cascade.
- Join entities replace `@ManyToMany` when you need extra columns (date, grade, quantity) on the association. This converts the relationship into two `@ManyToOne` / `@OneToMany` pairs.
- equals/hashCode must use business keys (not auto-generated IDs) when entities are stored in Sets.
- Clear associations before deleting entities in many-to-many relationships to avoid FK constraint violations.

Interview Questions

Q1: How does a relational database represent a many-to-many relationship?

With a join table (also called association table or junction table). The join table has two foreign key columns, each referencing one side of the relationship. The composite of both foreign keys forms the primary key. For example, `student_courses` has `student_id` and `course_id`, both as foreign keys.

Q2: Why should you use Set instead of List for @ManyToMany collections?

When using `List`, Hibernate handles collection changes by deleting ALL rows from the join table and re-inserting the remaining ones. With `Set`, Hibernate can insert or delete individual rows. For a student with 50 courses who drops 1, `List` generates 1 DELETE + 49 INSERTs, while `Set` generates just 1 DELETE.

Q3: When should you convert @ManyToMany to a join entity?

When the association needs extra columns beyond the two foreign keys. Examples: enrollment date, grade, quantity, role, status, or any metadata about the relationship itself. Also when you need to query the relationship directly (e.g., "find all enrollments from this

semester").

Q4: Why should you not use CascadeType.REMOVE on @ManyToMany?

Because both sides of a many-to-many share the referenced entities. If a student has `cascade = REMOVE` on courses, deleting the student would also delete the courses — which are shared with other students. This would cascade into data loss for unrelated entities.

Q5: How do you implement equals and hashCode for entities used in Sets?

Use a natural business key (like `email`, `username`, or `ISBN`) that is unique, immutable, and available before the entity is persisted. Never use the auto-generated `id` because it is `null` for new entities, which breaks the `hashCode` contract when the entity is added to a Set before being saved.

Q6: What is the difference between the owning side and inverse side in @ManyToMany?

The owning side defines the `@JoinTable` and controls the join table's SQL (inserts and deletes). The inverse side uses `mappedBy` and is read-only — changes to the inverse collection are not persisted. Only changes to the owning side's collection trigger database updates.

Practice Exercises

Exercise 1: Simple Many-to-Many Create `Article` and `Tag` entities with a basic `@ManyToMany` using `Set`. Write helper methods, a service to add/remove tags, and verify the join table entries.

Exercise 2: Join Entity Create `Employee`, `Project`, and `Assignment` (join entity) where an assignment has `role` (String) and `startDate`/`endDate` (LocalDate). Write a service with: assign employee to project, end assignment, find all active assignments for a project.

Exercise 3: Set vs List Performance Map the same many-to-many relationship twice — once with `Set` and once with `List`. Enable SQL logging. Add 5 associations, then remove 1. Count and compare the SQL statements generated.

Exercise 4: User-Role Authorization Build a complete User-Role system with: create user, create role, assign role to user, remove role, check `hasRole()`, find all users with a specific role, find users with multiple roles. Write repository queries using JPQL.

Exercise 5: Safe Deletion Create entities with many-to-many relationships. Write a test that: (a) tries to delete an entity without clearing associations (expect error), (b) clears associations first then deletes (expect success). Verify the join table is cleaned up properly.

What Is Next?

In the next chapter, we will explore Embeddable Types and Element Collections. You will learn how to use `@Embeddable` and `@Embedded` to model value objects (like Address) that are stored in the parent table, and `@ElementCollection` for collections of simple values or embeddables that are stored in a separate table without being full entities.

Chapter 15: Embeddable Types and Element Collections

Learning Goals

By the end of this chapter, you will be able to:

- Understand what embeddable types are and when to use them
- Create value objects with @Embeddable and @Embedded
- Override column names with @AttributeOverride
- Use @ElementCollection to store collections of value types
- Configure @CollectionTable for element collections
- Distinguish between entities and value objects
- Nest embeddable types within other embeddables
- Use embeddable types as map keys
- Query embeddable fields with JPQL and derived methods

Entities vs Value Objects

Before diving into embeddable types, you need to understand a fundamental distinction:

```
Entities vs Value Objects:
+-------------------------------------------------------------------+
|                                                                   |
|  Entity                            Value Object                   |
|  +--------------------------+     +--------------------------+    |
|  | Has its own identity     |     | No identity of its own   |    |
|  | (id field / primary key) |     | (no id / no PK)          |    |
|  |                          |     |                          |    |
```

```
|  | Has its own lifecycle    |     | Belongs to an entity     |    |
|  | (exists independently)  |     | (cannot exist alone)     |    |
|  |                          |     |                          |    |
|  | Tracked by Hibernate    |     | Embedded in the owner's  |    |
|  | (managed entity)        |     | table (not a separate    |    |
|  |                          |     | table by default)        |    |
|  |                          |     |                          |    |
|  | Example: Customer,       |     | Example: Address, Money, |    |
|  | Order, Product           |     | PhoneNumber, DateRange   |    |
|  +--------------------------+     +--------------------------+    |
|                                                                    |
|  Question: Does this "thing" have meaning on its own?              |
|                                                                    |
|  "123 Main St, Springfield"  --> NO, it is part of a Customer    |
|  "Customer #42, Alice"       --> YES, it exists independently    |
|                                                                    |
|  If NO --> Value Object --> @Embeddable                            |
|  If YES --> Entity --> @Entity                                     |
+-------------------------------------------------------------------+
```

Real-World Analogies

Think about a house address. An address does not exist independently — it describes where a person lives. You would not have an "addresses" table with its own ID that customers reference via foreign key. Instead, the address fields (street, city, state, zip) are part of the customer record. That is a value object.

Now think about a product in an order. A product exists independently — it has its own ID, its own lifecycle, and multiple orders can reference it. That is an entity.

@Embeddable and @Embedded

Creating an Embeddable Class

```java
@Embeddable                          // JPA standard
public class Address {

    @Column(nullable = false)
    private String street;

    @Column(nullable = false, length = 100)
    private String city;
```

```
    @Column(nullable = false, length = 2)
    private String state;

    @Column(name = "zip_code", nullable = false, length = 10)
    private String zipCode;

    protected Address() {}           // Required by JPA

    public Address(String street, String city, String state, String zipCode) {
        this.street = street;
        this.city = city;
        this.state = state;
        this.zipCode = zipCode;
    }

    // Getters
    public String getStreet() { return street; }
    public String getCity() { return city; }
    public String getState() { return state; }
    public String getZipCode() { return zipCode; }

    // Value objects should implement equals and hashCode
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Address address = (Address) o;
        return Objects.equals(street, address.street) &&
               Objects.equals(city, address.city) &&
               Objects.equals(state, address.state) &&
               Objects.equals(zipCode, address.zipCode);
    }

    @Override
    public int hashCode() {
        return Objects.hash(street, city, state, zipCode);
    }

    @Override
    public String toString() {
        return street + ", " + city + ", " + state + " " + zipCode;
    }
}
```

Using @Embedded in an Entity

```java
@Entity
@Table(name = "customers")
public class Customer {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Embedded                               // JPA standard
    private Address address;

    protected Customer() {}

    public Customer(String name, Address address) {
        this.name = name;
        this.address = address;
    }

    // Getters and setters
    public Long getId() { return id; }
    public String getName() { return name; }
    public Address getAddress() { return address; }
    public void setAddress(Address address) { this.address = address; }
}
```

```
How @Embedded Maps to the Database:
+-------------------------------------------------------------------+
|                                                                   |
|  Java:                           Database (customers table):      |
|  +------------------+          +------+------+------+------+--+  |
|  | Customer         |          | id   | name |street| city |..|  |
|  | - id: Long       |          |------+------+------+------+--|  |
|  | - name: String   |          | 1    | Alice|123   | NYC  |..|  |
|  | - address: Address|          | 2    | Bob  |456   | LA   |..|  |
|  |   - street       |--------->| (all address columns are     |  |
|  |   - city         |          |  embedded in customers table)|  |
|  |   - state        |          +------+------+------+------+--+  |
|  |   - zipCode      |                                            |
|  +------------------+          No separate "addresses" table!     |
|                                                                   |
|  The embeddable fields become columns in the OWNER's table.       |
|  There is NO join, NO foreign key, NO separate table.             |
+-------------------------------------------------------------------+
```

Usage

```java
// Creating
Address address = new Address("123 Main St", "Springfield", "IL", "62701");
```

```
Customer customer = new Customer("Alice Johnson", address);
customerRepository.save(customer);

// Reading
Customer found = customerRepository.findById(1L).orElseThrow();
System.out.println(found.getAddress().getCity());  // "Springfield"

// Updating (replace the entire value object)
found.setAddress(new Address("456 Oak Ave", "Chicago", "IL", "60601"));
// Hibernate: UPDATE customers SET street='456 Oak Ave', city='Chicago', ...
```

@AttributeOverride — Renaming Embedded Columns

When you embed the same type twice, you get column name conflicts. `@AttributeOverride` resolves this:

```java
@Entity
@Table(name = "companies")
public class Company {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Embedded
    @AttributeOverrides({
        @AttributeOverride(name = "street",  column = @Column(name = "billing_street")),
        @AttributeOverride(name = "city",    column = @Column(name = "billing_city")),
        @AttributeOverride(name = "state",   column = @Column(name = "billing_state")),
        @AttributeOverride(name = "zipCode", column = @Column(name = "billing_zip"))
    })
    private Address billingAddress;

    @Embedded
    @AttributeOverrides({
        @AttributeOverride(name = "street",  column = @Column(name = "shipping_street")),
        @AttributeOverride(name = "city",    column = @Column(name = "shipping_city")),
        @AttributeOverride(name = "state",   column = @Column(name = "shipping_state")),
        @AttributeOverride(name = "zipCode", column = @Column(name = "shipping_zip"))
    })
    private Address shippingAddress;
```

```
    // constructor, getters, setters...
}
```

```
@AttributeOverride in the Database:
+-------------------------------------------------------------------+
|                                                                   |
|  companies table:                                                 |
|  +----+------+----------------+----------------+                  |
|  | id | name | billing_street | shipping_street|                  |
|  |    |      | billing_city   | shipping_city  |                  |
|  |    |      | billing_state  | shipping_state |                  |
|  |    |      | billing_zip    | shipping_zip   |                  |
|  +----+------+----------------+----------------+                  |
|                                                                   |
|  Without @AttributeOverride, both Address fields would try        |
|  to create "street", "city", "state", "zip_code" columns,        |
|  causing a conflict. @AttributeOverride gives each a unique       |
|  column name prefix.                                              |
+-------------------------------------------------------------------+
```

@Embeddable for Money, DateRange, and Other Value Types

Embeddable types are perfect for domain concepts that have multiple fields but no identity:

Money

```java
@Embeddable
public class Money {

    @Column(nullable = false, precision = 19, scale = 2)
    private BigDecimal amount;

    @Column(nullable = false, length = 3)
    @Enumerated(EnumType.STRING)
    private Currency currency;

    public enum Currency { USD, EUR, GBP, JPY }

    protected Money() {}
```

```
    public Money(BigDecimal amount, Currency currency) {
        this.amount = amount;
        this.currency = currency;
    }

    public Money(double amount, Currency currency) {
        this(BigDecimal.valueOf(amount), currency);
    }

    // Getters
    public BigDecimal getAmount() { return amount; }
    public Currency getCurrency() { return currency; }

    // Business methods
    public Money add(Money other) {
        if (this.currency != other.currency) {
            throw new IllegalArgumentException("Cannot add different currencies");
        }
        return new Money(this.amount.add(other.amount), this.currency);
    }

    @Override
    public boolean equals(Object o) { /* compare amount and currency */ }

    @Override
    public int hashCode() { return Objects.hash(amount, currency); }

    @Override
    public String toString() { return amount + " " + currency; }
}
```

```java
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Embedded
    @AttributeOverrides({
        @AttributeOverride(name = "amount", column = @Column(name = "price_amount")),
        @AttributeOverride(name = "currency", column = @Column(name = "price_currency"))
    })
    private Money price;

    // ...
}
```

DateRange

```java
@Embeddable
public class DateRange {

    @Column(name = "start_date")
    private LocalDate startDate;

    @Column(name = "end_date")
    private LocalDate endDate;

    protected DateRange() {}

    public DateRange(LocalDate startDate, LocalDate endDate) {
        if (endDate != null && startDate != null && endDate.isBefore(startDate)) {
            throw new IllegalArgumentException("End date cannot be before start date");
        }
        this.startDate = startDate;
        this.endDate = endDate;
    }

    public boolean isActive() {
        LocalDate today = LocalDate.now();
        return (startDate == null || !today.isBefore(startDate)) &&
               (endDate == null || !today.isAfter(endDate));
    }

    public long getDurationDays() {
        if (startDate == null || endDate == null) return 0;
        return ChronoUnit.DAYS.between(startDate, endDate);
    }

    // Getters, equals, hashCode, toString...
}
```

```java
@Entity
public class Campaign {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Embedded
    private DateRange activePeriod;

    // ...
}
```

Nesting Embeddable Types

Embeddable classes can contain other embeddable classes:

```java
@Embeddable
public class ContactInfo {

    @Column(nullable = false)
    private String email;

    @Column(name = "phone_number")
    private String phoneNumber;

    @Embedded
    private Address address;

    protected ContactInfo() {}

    public ContactInfo(String email, String phoneNumber, Address address) {
        this.email = email;
        this.phoneNumber = phoneNumber;
        this.address = address;
    }

    // Getters...
}
```

```java
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Embedded
    private ContactInfo contactInfo;

    // ...
}
```

```
Nested Embeddable Mapping:
+-------------------------------------------------------------------+
|                                                                   |
|  Java:                                                            |
|  Employee                                                         |
|    |-- id                                                         |
```

```
|    |-- name                                                    |
|    +-- contactInfo (ContactInfo)                               |
|          |-- email                                             |
|          |-- phoneNumber                                       |
|          +-- address (Address)                                 |
|                |-- street                                      |
|                |-- city                                        |
|                |-- state                                       |
|                +-- zipCode                                     |
|                                                                |
|  Database (employees table):                                   |
|  +----+------+-------+--------+--------+------+-------+-------+  |
|  | id | name | email | phone_ | street | city | state | zip_  |  |
|  |    |      |       | number |        |      |       | code  |  |
|  +----+------+-------+--------+--------+------+-------+-------+  |
|                                                                 |
|  All fields flattened into one table. No joins needed.          |
+-----------------------------------------------------------------+
```

@ElementCollection — Collections of Value Types

What if an entity has a collection of value objects? You cannot use `@OneToMany` because value objects are not entities. Use `@ElementCollection` instead:

```java
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ElementCollection                      // JPA standard
    @CollectionTable(                       // JPA standard
        name = "employee_phone_numbers",    // Separate table name
        joinColumns = @JoinColumn(name = "employee_id")  // FK column
    )
    @Column(name = "phone_number")          // Column for the value
    private Set<String> phoneNumbers = new HashSet<>();

    // ...
}
```

```
@ElementCollection of Strings:
+-------------------------------------------------------------------+
|                                                                   |
|  Java:                                                            |
|  Employee                                                         |
|    |-- id: 1                                                      |
|    |-- name: "Alice"                                              |
|    +-- phoneNumbers: ["555-0101", "555-0102", "555-0103"]         |
|                                                                   |
|  Database:                                                        |
|                                                                   |
|  employees                  employee_phone_numbers                |
|  +----+-------+            +-------------+--------------+         |
|  | id | name  |            | employee_id | phone_number |         |
|  |----+-------|            |-------------+--------------|         |
|  | 1  | Alice |<-----------| 1           | 555-0101     |         |
|  | 2  | Bob   |     +------| 1           | 555-0102     |         |
|  +----+-------+     |      | 1           | 555-0103     |         |
|                      +------| 2           | 555-0201     |         |
|                             +-------------+--------------+         |
|                                                                   |
|  A SEPARATE TABLE is created for the collection.                  |
|  This is different from @Embedded (same table).                   |
|  Each element is a row. The FK links back to the owner entity.    |
+-------------------------------------------------------------------+
```

@ElementCollection of Embeddable Types

```java
@Embeddable
public class Skill {

    @Column(nullable = false)
    private String name;

    @Enumerated(EnumType.STRING)
    private SkillLevel level;

    public enum SkillLevel { BEGINNER, INTERMEDIATE, ADVANCED, EXPERT }

    protected Skill() {}

    public Skill(String name, SkillLevel level) {
        this.name = name;
        this.level = level;
    }

    // Getters, equals, hashCode...
}
```

```java
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ElementCollection(fetch = FetchType.LAZY)
    @CollectionTable(
        name = "employee_skills",
        joinColumns = @JoinColumn(name = "employee_id")
    )
    private List<Skill> skills = new ArrayList<>();

    public void addSkill(String name, Skill.SkillLevel level) {
        skills.add(new Skill(name, level));
    }

    public void removeSkill(String name) {
        skills.removeIf(s -> s.getName().equals(name));
    }

    // ...
}
```

```
@ElementCollection of Embeddable:
+-------------------------------------------------------------------+
|                                                                   |
|  employees                   employee_skills                      |
|  +----+-------+             +-------------+--------+-----------+  |
|  | id | name  |             | employee_id | name   | level     |  |
|  |----+-------|             |-------------+--------+-----------|  |
|  | 1  | Alice |<------------| 1           | Java   | EXPERT    |  |
|  +----+-------+      +-----| 1           | SQL    | ADVANCED  |  |
|                       |     | 1           | React  | BEGINNER  |  |
|                       |     +-------------+--------+-----------+  |
|                       |                                           |
|  Each Skill embeddable becomes a row in employee_skills.          |
|  The Skill fields (name, level) become columns.                   |
|  No separate "skills" entity — these are value objects.           |
+-------------------------------------------------------------------+
```

@ElementCollection of Enums

```java
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String username;

    @ElementCollection(fetch = FetchType.EAGER)  // Roles usually loaded eagerly
    @CollectionTable(name = "user_roles",
                     joinColumns = @JoinColumn(name = "user_id"))
    @Column(name = "role")
    @Enumerated(EnumType.STRING)
    private Set<Role> roles = new HashSet<>();

    public enum Role { USER, EDITOR, ADMIN, SUPER_ADMIN }

    public void addRole(Role role) { roles.add(role); }
    public void removeRole(Role role) { roles.remove(role); }
    public boolean hasRole(Role role) { return roles.contains(role); }

    // ...
}
```

@ElementCollection with Map

You can also use `@ElementCollection` with a `Map`:

```java
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ElementCollection
    @CollectionTable(name = "product_attributes",
                     joinColumns = @JoinColumn(name = "product_id"))
    @MapKeyColumn(name = "attribute_name")       // Column for the map key
    @Column(name = "attribute_value")             // Column for the map value
    private Map<String, String> attributes = new HashMap<>();
```

```
    public void setAttribute(String key, String value) {
        attributes.put(key, value);
    }

    public String getAttribute(String key) {
        return attributes.get(key);
    }

    // ...
}
```

```
@ElementCollection with Map:
+-------------------------------------------------------------------+
|                                                                   |
|  Java:                                                            |
|  Product                                                          |
|    |-- id: 1                                                      |
|    |-- name: "Laptop"                                             |
|    +-- attributes:                                                |
|          "color" -> "silver"                                      |
|          "weight" -> "1.4kg"                                      |
|          "screen" -> "14 inch"                                    |
|                                                                   |
|  Database:                                                        |
|                                                                   |
|  products                   product_attributes                    |
|  +----+--------+           +------------+----------------+-----+  |
|  | id | name   |           | product_id | attribute_name |value|  |
|  |----+--------|           |------------+----------------+-----|  |
|  | 1  | Laptop |<----------| 1          | color          |silv |  |
|  +----+--------+           | 1          | weight         |1.4kg|  |
|                             | 1          | screen         |14in |  |
|                             +------------+----------------+-----+  |
+-------------------------------------------------------------------+
```

Querying Embeddable Fields

Derived Query Methods

```java
@Repository
public interface CustomerRepository extends JpaRepository<Customer, Long> {

    // Query by embedded field - use dot notation
```

```
    List<Customer> findByAddressCity(String city);
    List<Customer> findByAddressState(String state);
    List<Customer> findByAddressCityAndAddressState(String city, String state);
    List<Customer> findByAddressZipCode(String zipCode);
}
```

JPQL with Embeddable Fields

```java
// Navigate embedded fields with dot notation
@Query("SELECT c FROM Customer c WHERE c.address.city = :city")
List<Customer> findByCity(@Param("city") String city);

// Use embedded fields in SELECT
@Query("SELECT c.name, c.address.city, c.address.state FROM Customer c")
List<Object[]> findNamesAndLocations();

// Use embedded fields in ORDER BY
@Query("SELECT c FROM Customer c ORDER BY c.address.state, c.address.city")
List<Customer> findAllOrderedByLocation();
```

Querying @ElementCollection

```java
// Find employees who have a specific skill
@Query("SELECT e FROM Employee e JOIN e.skills s WHERE s.name = :skillName")
List<Employee> findBySkill(@Param("skillName") String skillName);

// Find employees with a specific skill at a specific level
@Query("SELECT e FROM Employee e JOIN e.skills s " +
       "WHERE s.name = :skill AND s.level = :level")
List<Employee> findBySkillAndLevel(@Param("skill") String skill,
                                   @Param("level") Skill.SkillLevel level);

// Count skills per employee
@Query("SELECT e.name, SIZE(e.skills) FROM Employee e GROUP BY e.name")
List<Object[]> countSkillsPerEmployee();

// Find users with a specific role
@Query("SELECT u FROM User u JOIN u.roles r WHERE r = :role")
List<User> findByRole(@Param("role") User.Role role);
```

@ElementCollection vs @OneToMany — When to Use Which

```
Decision Guide: @ElementCollection vs @OneToMany
+-------------------------------------------------------------------+
|                                                                   |
|  Does the "child" data have its own identity?                     |
|  (Does it have a meaningful ID? Is it referenced elsewhere?)      |
|      |                                                            |
|   YES --> Use @OneToMany with a proper @Entity                    |
|      |    Example: Order --> OrderItem (OrderItem has its own ID,  |
|      |    quantity, price, product reference)                     |
|      |                                                            |
|    NO --> Does it have multiple fields?                            |
|           |                                                       |
|        YES --> @ElementCollection of @Embeddable                  |
|           |    Example: Employee --> Skill (name + level)          |
|           |                                                       |
|        NO  --> @ElementCollection of simple types                 |
|                Example: Employee --> phoneNumbers (Set<String>)    |
|                         User --> roles (Set<Role>)                |
|                                                                   |
+-------------------------------------------------------------------+
|                                                                   |
|  Key Differences:                                                 |
|                                                                   |
|  @ElementCollection              @OneToMany                      |
|  -------------------------       --------------------------       |
|  No ID on child                  Child has its own @Id            |
|  Child is a value object         Child is a managed entity        |
|  Cannot be shared                Can be shared (FK references)    |
|  Always owned by parent          Can exist independently          |
|  Deleted when parent deleted     Lifecycle can be independent     |
|  No lazy/eager per item          Each item independently loadable |
|  No cascading needed             Requires cascade configuration   |
|  Simpler but less flexible       More complex but more powerful   |
|                                                                   |
|  IMPORTANT: @ElementCollection replaces the ENTIRE collection     |
|  on update (DELETE all, then INSERT all). For large or            |
|  frequently modified collections, use @OneToMany instead.         |
+-------------------------------------------------------------------+
```

The Hidden Performance Trap

```
@ElementCollection Update Behavior:
+-------------------------------------------------------------------+
|                                                                   |
|  Adding ONE phone number to an employee with 5 numbers:           |
```

```
|                                                                  |
|  employee.getPhoneNumbers().add("555-0106");                     |
|                                                                  |
|  Hibernate executes:                                             |
|  1. DELETE FROM employee_phone_numbers WHERE employee_id = 1     |
|  2. INSERT INTO employee_phone_numbers (employee_id, phone_number)|
|     VALUES (1, '555-0101')                                       |
|  3. INSERT ... VALUES (1, '555-0102')                            |
|  4. INSERT ... VALUES (1, '555-0103')                            |
|  5. INSERT ... VALUES (1, '555-0104')                            |
|  6. INSERT ... VALUES (1, '555-0105')                            |
|  7. INSERT ... VALUES (1, '555-0106')   <-- the new one          |
|                                                                  |
|  7 SQL statements to add 1 phone number!                         |
|  Hibernate DELETES all rows then RE-INSERTS everything.          |
|                                                                  |
|  For small, rarely-modified collections: fine.                   |
|  For large or frequently-modified collections: use @OneToMany.   |
+------------------------------------------------------------------+
```

Null Embeddable Behavior

When all fields of an embedded object are null, Hibernate treats the entire embeddable as null:

```java
Customer customer = new Customer("Alice", null);  // No address
customerRepository.save(customer);

Customer found = customerRepository.findById(1L).orElseThrow();
found.getAddress();  // Returns null (not an Address with null fields)
```

```
Null Embeddable Behavior:
+------------------------------------------------------------------+
|                                                                  |
|  Database row:                                                   |
|  | id | name  | street | city | state | zip_code |               |
|  | 1  | Alice | NULL   | NULL | NULL  | NULL     |               |
|                                                                  |
|  When ALL embedded columns are NULL:                             |
|  customer.getAddress() returns NULL (not new Address(null...))   |
|                                                                  |
|  When ANY embedded column is NOT NULL:                           |
|  | id | name  | street | city   | state | zip_code |             |
|  | 2  | Bob   | NULL   | Chicago| NULL  | NULL     |             |
|  customer.getAddress() returns Address(null, "Chicago", null, null)|
```

```
|                                                                  |
|  To avoid NullPointerException, check for null:                  |
|  if (customer.getAddress() != null) {                            |
|      String city = customer.getAddress().getCity();               |
|  }                                                               |
+------------------------------------------------------------------+
```

Common Mistakes

1. Giving an @Embeddable class an @Id field: Embeddable types are value objects — they do not have their own identity. If you need an ID, it should be an `@Entity`, not an `@Embeddable`.
2. Using @ElementCollection for large collections: Hibernate deletes and re-inserts all rows on every modification. For collections with more than a few dozen items, or collections modified frequently, use `@OneToMany` with a proper entity instead.
3. Forgetting @AttributeOverride when embedding the same type twice: Two `@Embedded Address` fields in the same entity will create duplicate column names. Use `@AttributeOverride` to give each set of columns unique names.
4. Not implementing equals and hashCode on @Embeddable classes: Value objects should be compared by their field values, not by identity. Without proper `equals()`, operations like `Set.contains()` and `List.remove()` will not work correctly.
5. Using @ElementCollection with FetchType.EAGER unnecessarily: The default is LAZY, which is correct for most cases. Only use EAGER for very small collections that are always needed (like user roles).
6. Not checking for null on optional embedded fields: When all columns of an embedded field are null, Hibernate returns `null` for the entire embeddable, which can cause `NullPointerException`.

Best Practices

1. Use @Embeddable for domain value objects: Address, Money, DateRange, PhoneNumber — any concept that has multiple fields but no independent identity should be an embeddable.
2. Make embeddable classes immutable when possible: Value objects should ideally not change after creation. Provide a constructor, getters, but no setters. To "update" a value,

replace the entire object.

3. Always implement equals, hashCode, and toString: Value objects are defined by their content, not their identity. These methods should compare all fields.
4. Keep @ElementCollection small: Use it for 0-20 items that rarely change (tags, roles, phone numbers). For larger or frequently modified collections, use `@OneToMany`.
5. Use @Embedded to reduce entity bloat: Instead of 10 address-related fields cluttering your entity, group them into an `Address` embeddable. This makes the entity cleaner and the address concept reusable.
6. Prefer @Embeddable over creating a separate entity table: If the data has no independent lifecycle and is always accessed with its owner, embedding is simpler and more performant (no joins needed).

Summary

In this chapter, you learned how to model value objects in JPA:

- @Embeddable marks a class as a value object (no identity, no table, no primary key). Its fields become columns in the owning entity's table.
- @Embedded places an embeddable object inside an entity. The embeddable's fields are flattened into the entity's table.
- @AttributeOverride renames embedded columns to avoid conflicts when the same embeddable type is used multiple times.
- @ElementCollection stores a collection of value objects (or simple types) in a separate table linked by a foreign key. Unlike `@OneToMany`, the elements have no identity.
- @CollectionTable configures the table name and join column for element collections.
- Value objects vs entities: Value objects have no identity, belong to an entity, and cannot exist alone. Entities have their own identity and lifecycle.
- Performance warning: `@ElementCollection` replaces all rows on modification (DELETE + INSERT all). Use `@OneToMany` for large or frequently modified collections.

Interview Questions

Q1: What is the difference between @Embeddable and @Entity?

An `@Entity` has its own identity (`@Id`), its own database table, and its own lifecycle — it can exist independently. An `@Embeddable` is a value object with no identity, no table, and no independent lifecycle. Its fields are embedded as columns in the owning entity's table. Use `@Entity` for things like Customer, Order, Product. Use `@Embeddable` for things like Address, Money, DateRange.

Q2: When would you use @ElementCollection instead of @OneToMany?

Use `@ElementCollection` when the collection items are value objects (no identity, cannot exist independently, always belong to one owner). Examples: phone numbers, tags, roles, skills. Use `@OneToMany` when the items are entities with their own identity that may be shared or have independent lifecycles. Also prefer `@OneToMany` for large collections, since `@ElementCollection` deletes and re-inserts all items on every modification.

Q3: What happens when you embed the same @Embeddable type twice without @AttributeOverride?

It causes a column name conflict. Both embedded fields try to create columns with the same names (e.g., two "street" columns), resulting in a mapping error at startup. Use `@AttributeOverride` to give each set of columns unique names.

Q4: How does Hibernate handle null values for @Embedded fields?

When ALL columns of an embedded field are null in the database, Hibernate returns `null` for the entire embeddable object (not an embeddable instance with null fields). If ANY column has a non-null value, Hibernate creates an embeddable instance.

Q5: What is the performance concern with @ElementCollection?

When modifying an element collection, Hibernate typically deletes ALL existing rows for that owner and re-inserts them all. Adding one element to a collection of 100 items generates 1 DELETE + 101 INSERT statements. This makes `@ElementCollection` unsuitable for large or frequently modified collections.

Q6: Can embeddable types be nested?

Yes. An `@Embeddable` class can contain another `@Embedded` field that references another `@Embeddable` class. For example, a `ContactInfo` embeddable can contain an `Address` embeddable. All fields from both levels are flattened into the owning entity's table columns.

Practice Exercises

Exercise 1: Address Embeddable Create an `Address` embeddable and use it in a `Customer` entity. Save, load, and update addresses. Query customers by city and state using derived methods.

Exercise 2: Money Value Object Create a `Money` embeddable with amount and currency. Use it in a `Product` entity. Add an `@AttributeOverride` to have both `originalPrice` and `salePrice` Money fields.

Exercise 3: Element Collection of Strings Add a `Set<String>` of tags to a `Product` entity using `@ElementCollection`. Write methods to add/remove tags. Query products by tag using JPQL.

Exercise 4: Element Collection of Embeddables Create an `Education` embeddable (institution, degree, year) and add `@ElementCollection` of `Education` to an `Employee` entity. Save, query, and verify.

Exercise 5: Comparison Test Model phone numbers both ways: (a) `@ElementCollection` of strings, and (b) `@OneToMany` with a `PhoneNumber` entity. Add 10 numbers, then add 1 more. Enable SQL logging and compare the number of SQL statements. Observe the DELETE + re-INSERT behavior of `@ElementCollection`.

What Is Next?

In the next chapter, we will explore Inheritance Mapping Strategies — how to map Java class hierarchies (abstract classes, subclasses) to database tables. You will learn the three JPA strategies (SINGLE_TABLE, JOINED, TABLE_PER_CLASS), when to use each, and the simpler `@MappedSuperclass` alternative for sharing common fields.

Chapter 16: Inheritance Mapping Strategies

Learning Goals

By the end of this chapter, you will be able to:

- Understand the challenge of mapping Java inheritance to relational databases
- Use @MappedSuperclass for shared fields without inheritance mapping
- Implement SINGLE_TABLE inheritance strategy with discriminator columns
- Implement JOINED inheritance strategy with separate tables
- Implement TABLE_PER_CLASS inheritance strategy with union queries
- Use @DiscriminatorColumn and @DiscriminatorValue to control type identification
- Write polymorphic queries that return mixed entity types
- Choose the right inheritance strategy for your use case
- Understand the performance trade-offs of each strategy

The Challenge: Inheritance in a Relational World

Java supports class inheritance. Databases do not. When you have a class hierarchy, you must decide how to represent it in tables.

```
Java Inheritance:
+-------------------------------------------------------------------+
|                                                                   |
|              Payment (abstract)                                   |
|              +-------------------+                                |
|              | id: Long          |                                |
```

```
|              | amount: BigDecimal|                                |
|              | createdAt: Date   |                                |
|              +-------------------+                                |
|                  /          \                                     |
|                 /            \                                    |
|  CreditCardPayment          BankTransferPayment                  |
|  +-----------------+        +----------------------+             |
|  | cardNumber      |        | bankName             |             |
|  | expiryMonth     |        | accountNumber        |             |
|  | expiryYear      |        | routingNumber        |             |
|  +-----------------+        +----------------------+             |
|                                                                  |
|  Question: How do we store this in tables?                       |
|  - One table for all types?                                      |
|  - Separate tables joined by FK?                                 |
|  - Completely separate tables?                                   |
|  - Or just share fields without real inheritance?                |
+------------------------------------------------------------------+
```

JPA provides four approaches:

```
Inheritance Mapping Options:
+------------------------------------------------------------------+
|                                                                  |
|  1. @MappedSuperclass                                            |
|     Not true inheritance mapping. Shares fields only.            |
|     No polymorphic queries. Each subclass has its own table.     |
|                                                                  |
|  2. SINGLE_TABLE (default JPA inheritance strategy)              |
|     ONE table for all types. Discriminator column identifies type.|
|     Fastest queries. Nullable columns for subclass-specific data. |
|                                                                  |
|  3. JOINED                                                       |
|     Separate table per class. Joined by primary key.             |
|     Normalized. Slower queries (JOINs needed).                   |
|                                                                  |
|  4. TABLE_PER_CLASS                                              |
|     Completely separate tables. No shared table.                 |
|     UNION queries for polymorphism. Rarely used.                 |
+------------------------------------------------------------------+
```

@MappedSuperclass — Shared Fields Without Inheritance

`@MappedSuperclass` is not a true inheritance strategy. It simply lets you define common fields in a base class that are inherited by child entities. Each child gets its own independent table with

the base class fields included.

```java
@MappedSuperclass   // NOT @Entity — this is not a table
public abstract class BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }

    // Getters
    public Long getId() { return id; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public LocalDateTime getUpdatedAt() { return updatedAt; }
}
```

```java
@Entity
@Table(name = "products")
public class Product extends BaseEntity {
    @Column(nullable = false)
    private String name;
    private BigDecimal price;
    // ...
}

@Entity
@Table(name = "customers")
public class Customer extends BaseEntity {
    @Column(nullable = false)
    private String fullName;
    private String email;
    // ...
}
```

```
@MappedSuperclass Tables:
+-------------------------------------------------------------------+
|                                                                   |
|  products                           customers                     |
|  +------+----------+----------+   +------+----------+----------+ |
|  |id(PK)| name     | price    |   |id(PK)| full_name| email    | |
|  |      | created_at           |   |      | created_at           | |
|  |      | updated_at           |   |      | updated_at           | |
|  +------+----------+----------+   +------+----------+----------+ |
|                                                                   |
|  Each entity has its OWN table with base fields copied in.        |
|  No relationship between the tables.                              |
|  You CANNOT write: "SELECT b FROM BaseEntity b"                   |
|  (BaseEntity is not an entity — no polymorphic queries)           |
+-------------------------------------------------------------------+
```

When to Use @MappedSuperclass

```
@MappedSuperclass Use Cases:
+-------------------------------------------------------------------+
|                                                                   |
|  USE @MappedSuperclass when:                                      |
|  - You want shared audit fields (createdAt, updatedAt, createdBy) |
|  - You want a common id strategy across all entities              |
|  - The subclasses are UNRELATED (Product and Customer)            |
|  - You do NOT need polymorphic queries                            |
|                                                                   |
|  DO NOT use @MappedSuperclass when:                               |
|  - The classes are truly related (CreditCard and BankTransfer     |
|    are both Payments)                                             |
|  - You need to query "all payments" across types                  |
|  - You need a FK to the base type (order.payment_id)              |
+-------------------------------------------------------------------+
```

SINGLE_TABLE — One Table for All Types

The `SINGLE_TABLE` strategy stores the entire class hierarchy in a single database table. A discriminator column identifies which subclass each row represents.

```java
@Entity
@Table(name = "payments")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)  // JPA standard
```

```
@DiscriminatorColumn(
    name = "payment_type",                              // JPA standard
    discriminatorType = DiscriminatorType.STRING
)
public abstract class Payment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private BigDecimal amount;

    @Column(name = "created_at", nullable = false)
    private LocalDateTime createdAt = LocalDateTime.now();

    protected Payment() {}

    public Payment(BigDecimal amount) {
        this.amount = amount;
    }

    public Long getId() { return id; }
    public BigDecimal getAmount() { return amount; }
    public LocalDateTime getCreatedAt() { return createdAt; }
}
```

```java
@Entity
@DiscriminatorValue("CREDIT_CARD")     // JPA standard
public class CreditCardPayment extends Payment {

    @Column(name = "card_number")
    private String cardNumber;

    @Column(name = "expiry_month")
    private Integer expiryMonth;

    @Column(name = "expiry_year")
    private Integer expiryYear;

    protected CreditCardPayment() {}

    public CreditCardPayment(BigDecimal amount, String cardNumber,
                             int expiryMonth, int expiryYear) {
        super(amount);
        this.cardNumber = cardNumber;
        this.expiryMonth = expiryMonth;
        this.expiryYear = expiryYear;
    }

    // Getters
    public String getCardNumber() { return cardNumber; }
```

```
    public Integer getExpiryMonth() { return expiryMonth; }
    public Integer getExpiryYear() { return expiryYear; }
}
```

```java
@Entity
@DiscriminatorValue("BANK_TRANSFER")
public class BankTransferPayment extends Payment {

    @Column(name = "bank_name")
    private String bankName;

    @Column(name = "account_number")
    private String accountNumber;

    @Column(name = "routing_number")
    private String routingNumber;

    protected BankTransferPayment() {}

    public BankTransferPayment(BigDecimal amount, String bankName,
                               String accountNumber, String routingNumber) {
        super(amount);
        this.bankName = bankName;
        this.accountNumber = accountNumber;
        this.routingNumber = routingNumber;
    }

    // Getters
    public String getBankName() { return bankName; }
    public String getAccountNumber() { return accountNumber; }
    public String getRoutingNumber() { return routingNumber; }
}
```

```
SINGLE_TABLE Database Layout:
+-------------------------------------------------------------------+
|                                                                   |
|  payments (ONE table for all payment types)                       |
|  +----+--------+------+----------+-------+--------+-----+------+ |
|  | id | type   |amount| created  | card  | expiry | bank| acct | |
|  |    |        |      |          | _num  |_mo/_yr | name| _num | |
|  |----|--------|------|----------|-------|--------|-----|------| |
|  | 1  | CREDIT | 99.99| 2025-... | 4111  | 12/26  | null| null | |
|  |    | _CARD  |      |          |       |        |     |      | |
|  | 2  | BANK_  |250.00| 2025-... | null  | null   | Chase| 1234| |
|  |    |TRANSFER|      |          |       |        |     |      | |
|  | 3  | CREDIT | 49.99| 2025-... | 5500  | 03/27  | null| null | |
|  |    | _CARD  |      |          |       |        |     |      | |
|  +----+--------+------+----------+-------+--------+-----+------+ |
|                                                                   |
|  Discriminator column: payment_type                               |
|  - "CREDIT_CARD" rows have card columns, null bank columns        |
```

```
|  - "BANK_TRANSFER" rows have bank columns, null card columns      |
|  - Subclass columns MUST be nullable (different types use them)   |
+-------------------------------------------------------------------+
```

Querying SINGLE_TABLE

```java
@Repository
public interface PaymentRepository extends JpaRepository<Payment, Long> {

    // Polymorphic - returns ALL payment types
    List<Payment> findAll();
    // SQL: SELECT * FROM payments

    // Type-specific - returns only credit cards
    @Query("SELECT p FROM CreditCardPayment p WHERE p.amount > :min")
    List<CreditCardPayment> findLargeCreditCardPayments(@Param("min") BigDecimal min);
    // SQL: SELECT * FROM payments WHERE payment_type = 'CREDIT_CARD' AND amount > ?

    // Find by discriminator value
    @Query("SELECT p FROM Payment p WHERE TYPE(p) = CreditCardPayment")
    List<Payment> findCreditCardPayments();

    // Find all payments above an amount (polymorphic)
    List<Payment> findByAmountGreaterThan(BigDecimal amount);
    // SQL: SELECT * FROM payments WHERE amount > ?
    // Returns mix of CreditCardPayment and BankTransferPayment objects
}
```

```
SINGLE_TABLE Query Performance:
+-------------------------------------------------------------------+
|                                                                   |
|  Polymorphic query: "Find all payments over $100"                 |
|  SELECT * FROM payments WHERE amount > 100                        |
|  --> SINGLE table scan, NO joins                                  |
|  --> FASTEST possible                                             |
|                                                                   |
|  Type-specific query: "Find all credit card payments"             |
|  SELECT * FROM payments WHERE payment_type = 'CREDIT_CARD'        |
|  --> Still SINGLE table, just adds WHERE clause                   |
|  --> Still very fast                                              |
|                                                                   |
|  INSERT: Single INSERT into one table                             |
|  --> Fastest possible                                             |
+-------------------------------------------------------------------+
```

JOINED — One Table Per Class, Linked by PK

The `JOINED` strategy creates a separate table for the base class and each subclass. They are linked by a shared primary key.

```java
@Entity
@Table(name = "vehicles")
@Inheritance(strategy = InheritanceType.JOINED)        // JPA standard
@DiscriminatorColumn(name = "vehicle_type")            // Optional for JOINED
public abstract class Vehicle {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String make;

    @Column(nullable = false)
    private String model;

    @Column(nullable = false)
    private int year;

    @Column(nullable = false)
    private BigDecimal price;

    protected Vehicle() {}

    public Vehicle(String make, String model, int year, BigDecimal price) {
        this.make = make;
        this.model = model;
        this.year = year;
        this.price = price;
    }

    // Getters
    public Long getId() { return id; }
    public String getMake() { return make; }
    public String getModel() { return model; }
    public int getYear() { return year; }
    public BigDecimal getPrice() { return price; }
}
```

```java
@Entity
@Table(name = "cars")
@DiscriminatorValue("CAR")
public class Car extends Vehicle {

    @Column(name = "num_doors")
```

```
    private int numDoors;

    @Column(name = "trunk_size")
    private int trunkSize;   // in liters

    protected Car() {}

    public Car(String make, String model, int year, BigDecimal price,
               int numDoors, int trunkSize) {
        super(make, model, year, price);
        this.numDoors = numDoors;
        this.trunkSize = trunkSize;
    }

    public int getNumDoors() { return numDoors; }
    public int getTrunkSize() { return trunkSize; }
}
```

```java
@Entity
@Table(name = "trucks")
@DiscriminatorValue("TRUCK")
public class Truck extends Vehicle {

    @Column(name = "payload_capacity")
    private double payloadCapacity;   // in tons

    @Column(name = "num_axles")
    private int numAxles;

    protected Truck() {}

    public Truck(String make, String model, int year, BigDecimal price,
                 double payloadCapacity, int numAxles) {
        super(make, model, year, price);
        this.payloadCapacity = payloadCapacity;
        this.numAxles = numAxles;
    }

    public double getPayloadCapacity() { return payloadCapacity; }
    public int getNumAxles() { return numAxles; }
}
```

```
JOINED Database Layout:
+-------------------------------------------------------------------+
|                                                                    |
|  vehicles (base table)                                             |
|  +------+------+--------+------+----------+                      |
|  |id(PK)| make | model  | year | price    |                      |
|  |------+------+--------+------+----------|                      |
|  | 1    | Toyota| Camry | 2024 | 35000    |                      |
|  | 2    | Ford | F-150  | 2024 | 55000    |                      |
```

```
|  | 3    | Honda| Civic  | 2025 | 28000    |                      |
|  +------+------+--------+------+----------+                      |
|                                                                     |
|  cars (subclass table)       trucks (subclass table)               |
|  +------+-------+-------+   +------+---------+-------+            |
|  |id(PK)| doors | trunk |   |id(PK)| payload | axles |            |
|  | (FK) |       |       |   | (FK) |         |       |            |
|  |------+-------+-------|   |------+---------+-------|            |
|  | 1    | 4     | 500   |   | 2    | 2.5     | 2     |            |
|  | 3    | 4     | 400   |   +------+---------+-------+            |
|  +------+-------+-------+                                           |
|                                                                     |
|  cars.id and trucks.id are BOTH primary keys AND foreign keys      |
|  referencing vehicles.id. Same value, shared primary key.          |
|                                                                     |
|  No null columns! Each table only has relevant fields.             |
+--------------------------------------------------------------------+
```

Querying JOINED Strategy

```
JOINED Query Performance:
+--------------------------------------------------------------------+
|                                                                     |
|  Polymorphic query: "Find all vehicles over $30K"                  |
|  SELECT v.*, c.*, t.*                                              |
|  FROM vehicles v                                                   |
|  LEFT JOIN cars c ON v.id = c.id                                   |
|  LEFT JOIN trucks t ON v.id = t.id                                 |
|  WHERE v.price > 30000                                             |
|  --> Requires JOIN across ALL subclass tables                      |
|  --> Slower than SINGLE_TABLE                                      |
|  --> More JOINs with more subclasses                               |
|                                                                     |
|  Type-specific query: "Find all cars"                              |
|  SELECT v.*, c.*                                                   |
|  FROM vehicles v                                                   |
|  INNER JOIN cars c ON v.id = c.id                                  |
|  --> Only ONE join (the specific subclass)                         |
|  --> Reasonably fast                                               |
|                                                                     |
|  INSERT: TWO inserts (base + subclass table)                       |
|  --> Slightly slower than SINGLE_TABLE                             |
+--------------------------------------------------------------------+
```

TABLE_PER_CLASS — Completely Separate Tables

Each concrete class gets its own table with ALL fields (inherited + own). No shared table exists.

```java
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)   // JPA standard
public abstract class Notification {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)  // IDENTITY won't work here
    private Long id;

    @Column(nullable = false)
    private String recipient;

    @Column(nullable = false, columnDefinition = "TEXT")
    private String message;

    @Column(name = "sent_at")
    private LocalDateTime sentAt;

    protected Notification() {}

    public Notification(String recipient, String message) {
        this.recipient = recipient;
        this.message = message;
        this.sentAt = LocalDateTime.now();
    }

    // Getters
    public Long getId() { return id; }
    public String getRecipient() { return recipient; }
    public String getMessage() { return message; }
    public LocalDateTime getSentAt() { return sentAt; }
}

@Entity
@Table(name = "email_notifications")
public class EmailNotification extends Notification {
    @Column(name = "subject_line")
    private String subjectLine;
    // constructor, getters...
}

@Entity
@Table(name = "sms_notifications")
public class SmsNotification extends Notification {
    @Column(name = "phone_number")
```

```
    private String phoneNumber;
    // constructor, getters...
}
```

```
TABLE_PER_CLASS Database Layout:
+-------------------------------------------------------------------+
|                                                                   |
|  email_notifications (ALL fields duplicated)                      |
|  +------+-----------+---------+----------+----------+             |
|  |id(PK)| recipient | message | sent_at  | subject  |             |
|  |------+-----------+---------+----------+----------|             |
|  | 1    | alice@co  | Hello.. | 2025-... | Welcome  |             |
|  | 3    | bob@co    | Update..| 2025-... | News     |             |
|  +------+-----------+---------+----------+----------+             |
|                                                                   |
|  sms_notifications (ALL fields duplicated)                        |
|  +------+-----------+---------+----------+----------+             |
|  |id(PK)| recipient | message | sent_at  | phone    |             |
|  |------+-----------+---------+----------+----------|             |
|  | 2    | charlie   | Code... | 2025-... | 555-0123 |             |
|  +------+-----------+---------+----------+----------+             |
|                                                                   |
|  No base table exists. Each table has ALL fields.                 |
|  IDs must be globally unique (cannot use IDENTITY).               |
|  recipient, message, sent_at are DUPLICATED in each table.        |
+-------------------------------------------------------------------+
```

```
TABLE_PER_CLASS Query Performance:
+-------------------------------------------------------------------+
|                                                                   |
|  Polymorphic query: "Find all notifications"                      |
|  SELECT * FROM email_notifications                                |
|  UNION ALL                                                        |
|  SELECT * FROM sms_notifications                                  |
|  --> Requires UNION across ALL subclass tables                    |
|  --> SLOWEST for polymorphic queries                              |
|  --> More tables = more UNIONs                                    |
|                                                                   |
|  Type-specific query: "Find all email notifications"              |
|  SELECT * FROM email_notifications                                |
|  --> SINGLE table, NO joins, NO unions                            |
|  --> FASTEST for type-specific queries                            |
|                                                                   |
|  INSERT: Single INSERT into one table                             |
|  --> Fast (like SINGLE_TABLE)                                     |
+-------------------------------------------------------------------+
```

Strategy Comparison

```
Complete Comparison:
+-------------------------------------------------------------------+
|                                                                   |
|  Criteria          SINGLE_TABLE  JOINED    TABLE_PER_CLASS        |
|  ----------------------------------------------------------------|
|  Polymorphic       Fastest       Moderate  Slowest (UNION)        |
|  query speed       (1 table)     (JOINs)   (UNION ALL)           |
|                                                                   |
|  Type-specific     Fast (WHERE   Moderate  Fastest                |
|  query speed       on discrim)   (1 JOIN)  (1 table, no join)    |
|                                                                   |
|  INSERT speed      Fastest       Moderate  Fast                   |
|                    (1 INSERT)    (2 INSERTs)(1 INSERT)            |
|                                                                   |
|  Data integrity    Weak          Strong    Moderate               |
|                    (nullable     (NOT NULL (NOT NULL OK)          |
|                    subclass cols) OK)                              |
|                                                                   |
|  Normalization     Poor          Good      Poor                   |
|                    (1 wide table)(separate (fields duplicated     |
|                    with nulls)   tables)    in each table)        |
|                                                                   |
|  Schema changes    Easy          Moderate  Hard (must alter       |
|                    (1 table)     (base or   ALL tables for        |
|                                  sub table) base field changes)   |
|                                                                   |
|  FK references     Easy          Easy      Difficult              |
|  from other        (FK to 1      (FK to    (FK to which table?)  |
|  entities          table)        base)                             |
|                                                                   |
|  @GeneratedValue   All types     All types AUTO or SEQUENCE       |
|  strategies        work          work      (IDENTITY fails)       |
+-------------------------------------------------------------------+
```

Choosing a Strategy

```
Decision Guide:
+-------------------------------------------------------------------+
|                                                                   |
|  Do you need polymorphic queries?                                 |
|  ("Find all payments", "Find all vehicles")                       |
|      |                                                            |
|    NO --> Use @MappedSuperclass (simplest, no inheritance)        |
|      |                                                            |
|   YES                                                             |
|      |                                                            |
|      v                                                            |
```

```
|  How many subclass-specific fields?                                |
|      |                                                             |
|   FEW (< 5 per subclass)                                          |
|      |--> SINGLE_TABLE (fast, simple, some null columns OK)        |
|      |                                                             |
|   MANY (subclasses are very different)                             |
|      |--> JOINED (normalized, no wasted space, but slower)         |
|      |                                                             |
|  Are polymorphic queries rare?                                     |
|  Mostly query one type at a time?                                  |
|      |                                                             |
|   YES --> TABLE_PER_CLASS (fastest per-type, slowest polymorphic)  |
|                                                                    |
|  Default recommendation: SINGLE_TABLE                              |
|  It is the simplest, fastest for most queries, and the             |
|  JPA default for a reason. Use JOINED only when the subclass       |
|  differences are large and data integrity matters.                 |
+--------------------------------------------------------------------+
```

Referencing Inherited Entities from Other Entities

A key advantage of SINGLE_TABLE and JOINED is that other entities can have a foreign key to the base class:

```java
@Entity
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "payment_id")
    private Payment payment;    // Can be CreditCard or BankTransfer

    // ... other fields
}
```

```
FK to Inherited Entity:
+--------------------------------------------------------------------+
|                                                                    |
|  SINGLE_TABLE and JOINED:                                          |
|  orders.payment_id --> payments.id                                 |
|  Works perfectly. FK points to ONE table.                          |
```

```
|                                                                  |
|  TABLE_PER_CLASS:                                                |
|  orders.payment_id --> ??? (credit_cards.id or bank_transfers.id?)|
|  PROBLEMATIC! FK must reference a specific table,                |
|  but there is no shared base table.                              |
|  Most databases cannot enforce this FK constraint.               |
+------------------------------------------------------------------+
```

---

instanceof and Polymorphic Operations

When you load entities polymorphically, you can use `instanceof` to handle type-specific logic:

```java
@Service
@Transactional(readOnly = true)
public class PaymentReportService {

    private final PaymentRepository paymentRepository;

    public void printPaymentReport() {
        List<Payment> payments = paymentRepository.findAll();

        for (Payment payment : payments) {
            System.out.printf("$%.2f on %s", payment.getAmount(),
                payment.getCreatedAt());

            if (payment instanceof CreditCardPayment cc) {
                System.out.printf(" via Credit Card ending %s",
                    cc.getCardNumber().substring(cc.getCardNumber().length() - 4));
            } else if (payment instanceof BankTransferPayment bt) {
                System.out.printf(" via Bank Transfer from %s", bt.getBankName());
            }

            System.out.println();
        }
    }
}
```

TYPE() in JPQL

```java
// Find payments of a specific type
@Query("SELECT p FROM Payment p WHERE TYPE(p) = CreditCardPayment")
List<Payment> findCreditCardPayments();
```

```
@Query("SELECT p FROM Payment p WHERE TYPE(p) IN (CreditCardPayment, BankTransferPaymen...
List<Payment> findElectronicPayments();

// Count by type
@Query("SELECT TYPE(p), COUNT(p) FROM Payment p GROUP BY TYPE(p)")
List<Object[]> countByType();
```

Common Mistakes

1. Using TABLE_PER_CLASS with IDENTITY generation: `GenerationType.IDENTITY` relies on database auto-increment, but IDs must be globally unique across tables. Use `AUTO`, `SEQUENCE`, or `TABLE` instead.
2. Not making subclass columns nullable with SINGLE_TABLE: In SINGLE_TABLE, a credit card row has null bank columns and vice versa. Subclass-specific columns MUST be nullable. Adding `@Column(nullable = false)` on subclass fields causes insert failures.
3. Adding too many subclasses to SINGLE_TABLE: If you have 10 subclasses each with 5 unique columns, the table has 50+ columns where most are null. Consider JOINED for better data integrity.
4. Using TABLE_PER_CLASS with FK relationships: Other entities cannot have proper FK constraints to TABLE_PER_CLASS entities because there is no shared table to reference.
5. Choosing JOINED for small hierarchies: If subclasses differ by only 2-3 fields, JOINED adds unnecessary JOINs. SINGLE_TABLE is simpler and faster.
6. Forgetting @DiscriminatorValue: Without an explicit discriminator value, JPA uses the class name as the default. This is brittle — renaming the class changes stored data.

Best Practices

1. Default to SINGLE_TABLE: It is the fastest for queries, simplest to implement, and handles most use cases. Accept the nullable columns trade-off.
2. Use @MappedSuperclass for audit fields: Common fields like `id`, `createdAt`, `updatedAt` across unrelated entities belong in a `@MappedSuperclass`, not an inheritance hierarchy.
3. Always set @DiscriminatorValue explicitly: Do not rely on the default class name. Explicit values are stable across refactoring and renaming.

4. Avoid deep inheritance hierarchies: One level of subclassing is clean. Two or more levels become difficult to manage and query. Prefer composition over deep inheritance.
5. Use JOINED only when subclasses are significantly different: If subclasses have many unique columns and you need NOT NULL constraints, JOINED is appropriate. Otherwise, SINGLE_TABLE is simpler.
6. Avoid TABLE_PER_CLASS in most cases: It has limited support for FK relationships, requires UNION for polymorphic queries, and does not support IDENTITY generation. Use it only when you almost never query polymorphically.

Summary

In this chapter, you learned how to map Java inheritance hierarchies to database tables:

- @MappedSuperclass shares fields across unrelated entities without creating inheritance in the database. No polymorphic queries, each subclass has its own table.
- SINGLE_TABLE stores the entire hierarchy in one table with a discriminator column. Fastest for queries but subclass columns must be nullable.
- JOINED creates a separate table per class linked by primary key. Normalized and supports NOT NULL on subclass columns, but requires JOINs for queries.
- TABLE_PER_CLASS puts everything in separate tables with duplicated base fields. Fastest for type-specific queries but requires UNION for polymorphic queries.
- Default to SINGLE_TABLE for most use cases. Use JOINED when subclasses have many unique fields and data integrity matters. Use @MappedSuperclass for shared audit/timestamp fields. Avoid TABLE_PER_CLASS unless polymorphic queries are rare.

Interview Questions

Q1: What are the JPA inheritance strategies?

JPA provides three true inheritance strategies: `SINGLE_TABLE` (one table with discriminator), `JOINED` (separate tables linked by PK), and `TABLE_PER_CLASS` (independent tables with duplicated fields). Additionally, `@MappedSuperclass` provides field sharing without true inheritance mapping.

Q2: What is a discriminator column?

A discriminator column is a special column in the database table that identifies which subclass a particular row represents. For example, a `payment_type` column with values `CREDIT_CARD` or `BANK_TRANSFER`. It is defined with `@DiscriminatorColumn` on the base class and `@DiscriminatorValue` on each subclass.

Q3: Why must subclass-specific columns be nullable in SINGLE_TABLE?

Because all types share one table. A credit card row has values for `card_number` and `expiry_month` but null for `bank_name` and `account_number`. A bank transfer row is the opposite. Each row only uses its own subclass columns — the rest must accept null.

Q4: When would you choose JOINED over SINGLE_TABLE?

When subclasses have many unique columns (e.g., 15+ each), when you need NOT NULL constraints on subclass columns, or when the table would become excessively wide with SINGLE_TABLE. JOINED provides better normalization at the cost of JOIN queries.

Q5: What is the limitation of TABLE_PER_CLASS with GenerationType.IDENTITY?

`IDENTITY` uses database auto-increment, which generates IDs per table. Two tables could generate the same ID (both have row with id=1). Since polymorphic queries use UNION across tables, IDs must be globally unique. Use `SEQUENCE` or `TABLE` generation instead.

Q6: How do you perform a type-specific query in JPQL?

Use the `TYPE()` function: `SELECT p FROM Payment p WHERE TYPE(p) = CreditCardPayment`. You can also use `IN`: `WHERE TYPE(p) IN (CreditCardPayment, BankTransferPayment)`. Alternatively, query the subclass directly: `SELECT c FROM CreditCardPayment c`.

Practice Exercises

Exercise 1: @MappedSuperclass Create a `BaseEntity` with `id`, `createdAt`, `updatedAt` and `@PrePersist`/`@PreUpdate` callbacks. Create `Product` and `Category` entities extending it. Verify that both entities have audit timestamps set automatically.

Exercise 2: SINGLE_TABLE Payment System Implement the Payment hierarchy (CreditCardPayment, BankTransferPayment) with SINGLE_TABLE strategy. Create an Order entity with a `@ManyToOne` to Payment. Test polymorphic queries, type-specific queries, and the `TYPE()` function.

Exercise 3: JOINED Vehicle System Implement the Vehicle hierarchy (Car, Truck, Motorcycle) with JOINED strategy. Add 5 vehicles of mixed types. Enable SQL logging and compare the queries generated for: (a) findAll(), (b) finding only cars, (c) inserting a new truck.

Exercise 4: Strategy Comparison Map the same 3-class hierarchy with all three strategies (in separate packages). Insert 100 records of mixed types. Benchmark: (a) polymorphic findAll(), (b) type-specific find, (c) insert speed. Compare query counts and shapes.

Exercise 5: Shape Hierarchy Design a `Shape` hierarchy with `Circle` (radius), `Rectangle` (width, height), and `Triangle` (base, height, sideA, sideB). Add an `area()` method to each. Choose the most appropriate inheritance strategy and justify your choice. Implement and test with polymorphic queries.

What Is Next?

In the next chapter, we enter Part IV: Deep Dive and start with Entity Lifecycle and Persistence Context. You will learn how Hibernate manages entities internally — the four lifecycle states (New, Managed, Detached, Removed), how the first-level cache works, how dirty checking detects changes automatically, and when and how Hibernate flushes changes to the database.

Chapter 17: Entity Lifecycle and Persistence Context

Learning Goals

By the end of this chapter, you will be able to:

- Understand the four entity lifecycle states: New, Managed, Detached, Removed
- Explain what the persistence context is and how it works as a first-level cache
- Describe how dirty checking detects changes automatically
- Know when and how Hibernate flushes changes to the database
- Use flush modes to control when SQL is executed
- Work with detached entities: merge, reattach, and handle common issues
- Understand the open-session-in-view pattern and why it is controversial
- Use EntityManager methods (persist, merge, find, remove, detach, clear, flush)
- Avoid common pitfalls with entity state transitions

The Four Entity States

Every JPA entity exists in one of four states relative to the persistence context (the first-level cache managed by Hibernate):

```
Entity Lifecycle States:
+-------------------------------------------------------------------+
|                                                                   |
|                      +----------+                                 |
|         new          |          |       entityManager.persist()     |
|      keyword         |   NEW    |----------------------------+     |
```

```
|    (constructor)    | (Transient)                          |     |
|                     +----------+                           |     |
|                                                            v     |
|                                                     +---------+ |
|  entityManager  +----------+                        |         | |
|  .find()        |          |   entityManager.remove()  | MANAGED | |
|  ------------->| MANAGED  |------------------------->|         | |
|  @Query         |          |                        +---------+ |
|  repository     +----------+                          |     ^   |
|  .findById()         |  ^                              |     |   |
|                      |  |   entityManager.merge()      |     |   |
|  entityManager       |  +-------------------------------+     |   |
|  .detach()           |                                        |   |
|  .clear()            v                                        |   |
|  transaction    +----------+                                  |   |
|  ends           |          |   entityManager.merge()          |   |
|                 | DETACHED |-----------------------------------+   |
|                 |          |                                       |
|                 +----------+                                      |
|                                                                   |
|                 +----------+                                      |
|                 |          |   flush() / commit()                  |
|                 | REMOVED  |-------> DELETE SQL executed           |
|                 |          |         entity gone from DB           |
|                 +----------+                                      |
|                                                                   |
+-------------------------------------------------------------------+
```

State Definitions

```
State        In Persistence    Has DB     Tracked by      Changes
             Context?          Row?       Hibernate?      Auto-saved?
-----------------------------------------------------------------------
NEW          No                No         No              No
(Transient)  Just created      Not yet    Not watching    Must call
             with new          in any     this object     persist()
                               table

MANAGED      Yes               Yes (or    Yes             Yes!
                               will be    Dirty checking  Changes saved
                               on flush)  is active       on flush

DETACHED     No                Yes        No              No
             Was managed,      Row        Not watching    Must call
             now disconnected  still      anymore         merge()
                               exists

REMOVED      Yes               Yes (until Yes             N/A
             Scheduled for     DELETE     Scheduled for   Will be
             deletion          runs)      removal         deleted
```

The Persistence Context

The persistence context is the central concept in JPA. Think of it as a workspace where Hibernate tracks entities and their changes.

```
Persistence Context (First-Level Cache):
+-------------------------------------------------------------------+
|                                                                    |
|  EntityManager (Spring @Transactional scope)                       |
|  +-----------------------------------------------------------------+|
|  |  Persistence Context                                            ||
|  |  +-------------------------------------------------------------+||
|  |  |  Identity Map (first-level cache)                           |||
|  |  |                                                             |||
|  |  |  Key: (Entity Type + ID)     Value: Entity Instance         |||
|  |  |  --------------------------  -------------------------     |||
|  |  |  (Employee, 1)               Employee{id=1, name="Alice"} |||
|  |  |  (Employee, 2)               Employee{id=2, name="Bob"}   |||
|  |  |  (Department, 10)            Department{id=10, name="Eng"}|||
|  |  |                                                             |||
|  |  |  Guarantees:                                                |||
|  |  |  - Same ID always returns SAME Java object (identity)       |||
|  |  |  - Loaded entities are cached (no duplicate queries)        |||
|  |  |  - Changes are tracked (dirty checking)                     |||
|  |  +-------------------------------------------------------------+||
|  |                                                                 ||
|  |  Action Queue (pending SQL)                                     ||
|  |  +-------------------------------------------------------------+||
|  |  |  INSERT Employee(id=3, name="Carol")                        |||
|  |  |  UPDATE Employee SET name="Alicia" WHERE id=1               |||
|  |  |  DELETE FROM Employee WHERE id=2                            |||
|  |  +-------------------------------------------------------------+||
|  +-----------------------------------------------------------------+|
|                                                                    |
|  On flush/commit: Action Queue SQL is sent to the database         |
+-------------------------------------------------------------------+
```

Identity Guarantee

The persistence context ensures that for any given entity type and ID, you always get the same Java object instance:

```java
@Transactional
public void demonstrateIdentity() {
```

```
    Employee e1 = entityManager.find(Employee.class, 1L);
    // SQL: SELECT * FROM employees WHERE id = 1

    Employee e2 = entityManager.find(Employee.class, 1L);
    // NO SQL! Returns cached instance from persistence context

    System.out.println(e1 == e2);  // true - exact same object!

    // Even queries return the same instance
    Employee e3 = employeeRepository.findByEmail("alice@co.com");
    // SQL runs, but if it returns id=1, Hibernate returns
    // the SAME object already in the persistence context

    System.out.println(e1 == e3);  // true - same object again!
}
```

```
Identity Guarantee Flow:
+-------------------------------------------------------------------+
|                                                                   |
|  find(Employee.class, 1L)                                         |
|       |                                                           |
|       v                                                           |
|  Is (Employee, 1) in the persistence context?                     |
|       |                                                           |
|    YES --> Return the cached instance (no SQL)                    |
|       |                                                           |
|    NO --> Execute SELECT, create Employee object,                 |
|           store in persistence context, return it                 |
|                                                                   |
|  Result: Within a transaction, the SAME id always returns         |
|  the SAME Java object. This is called "repeatable read"           |
|  at the application level.                                        |
+-------------------------------------------------------------------+
```

Dirty Checking

Hibernate automatically detects changes to managed entities. You do not need to call `save()` or `update()` — just modify the object and Hibernate generates the appropriate UPDATE SQL on flush.

```java
@Transactional
public void updateEmployeeName(Long id, String newName) {
    Employee employee = employeeRepository.findById(id).orElseThrow();
    // employee is now MANAGED
```

```
    employee.setName(newName);
    // That's it! No save() call needed.
    // Hibernate detects the change and generates UPDATE on flush.
}
// When @Transactional ends, Hibernate flushes:
// UPDATE employees SET name = 'newName' WHERE id = ?
```

```
How Dirty Checking Works:
+-------------------------------------------------------------------+
|                                                                   |
|  1. Entity loaded from DB:                                        |
|     Employee{id=1, name="Alice", salary=80000}                    |
|     Hibernate takes a SNAPSHOT of the original values             |
|     Snapshot: {name="Alice", salary=80000}                        |
|                                                                   |
|  2. Your code modifies the entity:                                |
|     employee.setName("Alicia");                                   |
|     employee.setSalary(85000);                                    |
|     Current: {name="Alicia", salary=85000}                        |
|                                                                   |
|  3. On flush, Hibernate compares current vs snapshot:             |
|     name:   "Alicia" != "Alice"    --> CHANGED                   |
|     salary: 85000 != 80000         --> CHANGED                   |
|                                                                   |
|  4. Hibernate generates UPDATE for changed fields:                |
|     UPDATE employees                                              |
|     SET name = 'Alicia', salary = 85000                           |
|     WHERE id = 1                                                  |
|                                                                   |
|  No explicit save() needed! Managed entities are auto-tracked.   |
+-------------------------------------------------------------------+
```

When save() IS Needed

```
When to Call save() / persist():
+-------------------------------------------------------------------+
|                                                                   |
|  NEED save():                                                     |
|  - Creating a NEW entity (not yet in DB)                          |
|    Employee e = new Employee("Alice", "alice@co.com");            |
|    employeeRepository.save(e);  // persist new entity             |
|                                                                   |
|  - Merging a DETACHED entity (was managed, now disconnected)      |
|    employee = employeeRepository.save(detachedEmployee);          |
|    // merge back into persistence context                         |
|                                                                   |
|  DO NOT NEED save():                                              |
|  - Modifying a MANAGED entity (loaded in current transaction)     |
```

```
|    Employee e = employeeRepository.findById(1L).orElseThrow();    |
|    e.setName("Alicia");                                           |
|    // Auto-saved on flush. Calling save() here works but          |
|    // is unnecessary and misleading.                              |
+-------------------------------------------------------------------+
```

Flushing

Flushing is the process of synchronizing the persistence context with the database — sending all pending SQL statements.

```
Flush Triggers:
+-------------------------------------------------------------------+
|                                                                   |
|  Hibernate flushes automatically when:                            |
|                                                                   |
|  1. Transaction commits (@Transactional method ends)              |
|     --> All pending changes flushed then committed                |
|                                                                   |
|  2. Before a JPQL/HQL query executes                              |
|     --> Ensures query sees the latest data                        |
|     employee.setName("Alicia");                                   |
|     List<Employee> result = em.createQuery(                       |
|         "SELECT e FROM Employee e WHERE e.name = 'Alicia'")       |
|         .getResultList();                                         |
|     --> Hibernate flushes the UPDATE before running SELECT        |
|     --> Query correctly finds the employee                        |
|                                                                   |
|  3. When you call entityManager.flush() explicitly                |
|     --> Forces immediate flush                                    |
|                                                                   |
|  Flushing does NOT commit the transaction!                        |
|  The SQL is sent to the DB but can still be rolled back.          |
+-------------------------------------------------------------------+
```

Flush Ordering

Hibernate flushes SQL in a specific order to satisfy foreign key constraints:

```
Flush SQL Order:
+-------------------------------------------------------------------+
|                                                                   |
|  1. INSERT statements (new entities)                              |
```

```
|     - Parent entities first (departments before employees)        |
|                                                                   |
|  2. UPDATE statements (modified entities)                         |
|     - In order of modification                                    |
|                                                                   |
|  3. DELETE collection elements                                    |
|     - Remove from join tables first                               |
|                                                                   |
|  4. DELETE statements (removed entities)                          |
|     - Child entities first (employees before departments)         |
|     - Reverse order of INSERT                                     |
|                                                                   |
|  This ordering ensures FK constraints are never violated.         |
|  Hibernate handles this automatically.                            |
+-------------------------------------------------------------------+
```

Flush Modes

```java
// JPA standard flush modes
entityManager.setFlushMode(FlushModeType.AUTO);    // Default - flush before queries
entityManager.setFlushMode(FlushModeType.COMMIT);  // Only flush on commit
```

```
Flush Modes:
+-------------------------------------------------------------------+
|                                                                   |
|  FlushModeType.AUTO (default):                                    |
|  - Flushes before every JPQL/HQL query                            |
|  - Flushes on commit                                              |
|  - Safest: queries always see latest changes                      |
|  - May cause extra flushes (performance overhead)                 |
|                                                                   |
|  FlushModeType.COMMIT:                                            |
|  - Flushes ONLY on commit                                         |
|  - Queries might NOT see uncommitted changes                      |
|  - Fewer flushes (better performance for batch operations)        |
|  - Use when you know queries don't need pending changes           |
|                                                                   |
|  Recommendation: Keep AUTO for correctness.                       |
|  Switch to COMMIT only for performance-critical batch operations. |
+-------------------------------------------------------------------+
```

EntityManager Methods and State Transitions

persist() — New to Managed

```java
@Transactional
public void persistExample() {
    Employee employee = new Employee("Alice", "alice@co.com");
    // State: NEW (transient) — not tracked, no DB row

    entityManager.persist(employee);
    // State: MANAGED — tracked by persistence context
    // No SQL yet! INSERT is queued for flush.
    // employee.getId() may be set immediately (IDENTITY strategy)
    // or later (SEQUENCE strategy)

    employee.setName("Alicia");
    // Still MANAGED — this change will be included in the INSERT
    // or generate an UPDATE after the INSERT
}
// Transaction commits: INSERT INTO employees (name, email) VALUES ('Alicia', 'alice@co...
```

```
persist() Flow:
+-------------------------------------------------------------------+
|                                                                   |
|  Employee e = new Employee(...)     --> State: NEW                |
|       |                                                           |
|       v                                                           |
|  entityManager.persist(e)           --> State: MANAGED            |
|       |                                - Added to persistence ctx |
|       |                                - INSERT queued            |
|       |                                - ID may be assigned       |
|       v                                                           |
|  e.setName("Alicia")               --> Still MANAGED              |
|       |                                - Change detected (dirty)  |
|       v                                                           |
|  Transaction commit                 --> FLUSH                     |
|       |                                - INSERT executed          |
|       v                                - COMMIT                   |
|  After commit                       --> State: DETACHED           |
|                                        - No longer tracked        |
+-------------------------------------------------------------------+
```

find() — Database to Managed

```java
@Transactional
public void findExample() {
    Employee employee = entityManager.find(Employee.class, 1L);
    // SQL: SELECT * FROM employees WHERE id = 1
    // State: MANAGED — loaded into persistence context
```

```
    // Second find - no SQL! Returned from cache.
    Employee same = entityManager.find(Employee.class, 1L);
    assert employee == same;  // Same Java object instance

    // find returns null if not found (does NOT throw)
    Employee notFound = entityManager.find(Employee.class, 999L);
    assert notFound == null;
}
```

merge() — Detached to Managed

```java
@Transactional
public Employee mergeExample(Employee detachedEmployee) {
    // detachedEmployee was loaded in a previous transaction
    // State: DETACHED - has an ID but not tracked

    detachedEmployee.setName("Updated Name");
    // This change is NOT tracked! Entity is detached.

    Employee managedEmployee = entityManager.merge(detachedEmployee);
    // State of managedEmployee: MANAGED (new copy in persistence context)
    // State of detachedEmployee: STILL DETACHED (not the same object!)

    assert detachedEmployee != managedEmployee;  // Different objects!
    // Always use the RETURNED entity, not the original.

    return managedEmployee;
}
```

```
merge() - Critical Detail:
+-----------------------------------------------------------------+
|                                                                 |
|  Employee detached = loadedInPreviousTransaction();             |
|  detached.setName("New Name");                                  |
|                                                                 |
|  Employee managed = entityManager.merge(detached);              |
|                                                                 |
|  What merge() does:                                             |
|  1. Finds entity with same ID in persistence context            |
|     - If found: copies state from detached to managed instance  |
|     - If not found: loads from DB, then copies state            |
|  2. Returns the MANAGED copy (not the original!)                |
|                                                                 |
|  CRITICAL: detached and managed are DIFFERENT objects!          |
|  detached is still detached. managed is the one being tracked.  |
|                                                                 |
|  Common mistake:                                                |
|  entityManager.merge(detached);                                 |
```

```
|  detached.setSalary(100000);  // BUG! Changes to detached         |
|                               // are NOT tracked!                 |
|                                                                   |
|  Correct:                                                         |
|  Employee managed = entityManager.merge(detached);                |
|  managed.setSalary(100000);   // Correct — changes to managed     |
|                               // ARE tracked                      |
+-------------------------------------------------------------------+
```

remove() — Managed to Removed

```java
@Transactional
public void removeExample(Long id) {
    Employee employee = entityManager.find(Employee.class, id);
    // State: MANAGED

    entityManager.remove(employee);
    // State: REMOVED — scheduled for deletion
    // DELETE is queued, not yet executed

    // You can still read the entity's data
    System.out.println(employee.getName());  // Works

    // But it will be deleted on flush/commit
}
// Transaction commits: DELETE FROM employees WHERE id = ?
```

detach() and clear()

```java
@Transactional
public void detachExample() {
    Employee employee = entityManager.find(Employee.class, 1L);
    // State: MANAGED

    entityManager.detach(employee);
    // State: DETACHED — removed from persistence context
    // Future changes are NOT tracked

    employee.setName("New Name");
    // This change is LOST — entity is detached, no dirty checking

    // clear() detaches ALL entities
    Employee e1 = entityManager.find(Employee.class, 1L);
    Employee e2 = entityManager.find(Employee.class, 2L);

    entityManager.clear();
```

```
    // Both e1 and e2 are now DETACHED
    // Persistence context is empty
    // Pending changes are DISCARDED (not flushed!)
}
```

```
detach() vs clear():
+-------------------------------------------------------------------+
|                                                                   |
|  detach(entity):                                                  |
|  - Removes ONE entity from persistence context                    |
|  - Other entities remain managed                                  |
|  - Pending changes for THIS entity are discarded                  |
|                                                                   |
|  clear():                                                         |
|  - Removes ALL entities from persistence context                  |
|  - Persistence context is completely emptied                      |
|  - ALL pending changes are discarded                              |
|  - Typically preceded by flush() to save pending changes:         |
|    entityManager.flush();  // save everything to DB               |
|    entityManager.clear();  // free memory                         |
|                                                                   |
|  Use case for flush() + clear():                                  |
|  Batch processing — periodically flush and clear to avoid         |
|  memory buildup when processing thousands of entities.            |
+-------------------------------------------------------------------+
```

Spring Data JPA and the Persistence Context

When you use Spring Data JPA repositories, the persistence context is managed automatically through `@Transactional`:

```
Spring @Transactional and Persistence Context:
+-------------------------------------------------------------------+
|                                                                   |
|  @Service                                                         |
|  public class EmployeeService {                                   |
|                                                                   |
|      @Transactional                                               |
|      public void updateEmployee(Long id, String name) {           |
|          |                                                        |
|          |  Transaction begins                                    |
|          |  EntityManager created                                  |
|          |  Persistence context opened                             |
|          v                                                        |
|          Employee e = repository.findById(id).orElseThrow();      |
```

```
|          |  SQL: SELECT ... (entity loaded, state: MANAGED)       |
|          v                                                        |
|          e.setName(name);                                         |
|          |  No SQL yet — change tracked by dirty checking         |
|          v                                                        |
|          // Method ends, @Transactional triggers:                 |
|          // 1. Flush (dirty checking generates UPDATE SQL)        |
|          // 2. Commit (transaction committed)                     |
|          // 3. Persistence context closed                          |
|          // 4. All entities become DETACHED                       |
|      }                                                            |
|  }                                                                |
+-------------------------------------------------------------------+
```

save() in Spring Data JPA

Spring Data's `save()` method internally calls either `persist()` or `merge()` depending on the entity state:

```java
// SimpleJpaRepository.save() — what Spring does internally:
@Transactional
public <S extends T> S save(S entity) {
    if (entityInformation.isNew(entity)) {
        entityManager.persist(entity);    // NEW entity --> persist
        return entity;
    } else {
        return entityManager.merge(entity);  // Existing entity --> merge
    }
}
```

```
How Spring Data save() Decides:
+-------------------------------------------------------------------+
|                                                                   |
|  repository.save(entity)                                          |
|       |                                                           |
|       v                                                           |
|  Is the entity new?                                               |
|  (id == null, or @Version is null/0)                              |
|       |                                                           |
|    YES --> entityManager.persist(entity)                          |
|       |   Returns the SAME object (now managed)                   |
|       |                                                           |
|    NO  --> entityManager.merge(entity)                            |
|            Returns a NEW managed copy                             |
|            (original may be different object!)                    |
|                                                                   |
|  This is why save() returns the entity — always use the           |
|  returned value, not the original:                                |
|  Employee saved = repository.save(employee);  // Use 'saved'     |
```

```
+-----------------------------------------------------------------+
```

Open Session in View (OSIV)

Spring Boot enables Open Session in View by default. This keeps the persistence context open for the entire HTTP request, even after the service method's `@Transactional` completes.

```
With OSIV (spring.jpa.open-in-view=true, the default):
+-----------------------------------------------------------------+
|                                                                  |
|  HTTP Request Lifecycle:                                         |
|  +-------------------------------------------------------------+  |
|  |  Filter opens persistence context                          |  |
|  |       |                                                    |  |
|  |  Controller                                                |  |
|  |       |                                                    |  |
|  |  @Transactional Service method                             |  |
|  |  +-----------------------------------------------+          |  |
|  |  | Transaction + persistence context active     |          |  |
|  |  | Entities are MANAGED                         |          |  |
|  |  | Changes are auto-flushed on commit           |          |  |
|  |  +-----------------------------------------------+          |  |
|  |  Transaction committed, but persistence context STAYS OPEN  |  |
|  |       |                                                    |  |
|  |  Controller continues — entities still MANAGED (read-only)  |  |
|  |  Lazy loading WORKS here (extra queries triggered)         |  |
|  |       |                                                    |  |
|  |  JSON serialization — lazy collections loaded on demand    |  |
|  |       |                                                    |  |
|  |  Filter closes persistence context                         |  |
|  +-------------------------------------------------------------+  |
+-----------------------------------------------------------------+

Without OSIV (spring.jpa.open-in-view=false):
+-----------------------------------------------------------------+
|                                                                  |
|  HTTP Request Lifecycle:                                         |
|  +-------------------------------------------------------------+  |
|  |  Controller                                                |  |
|  |       |                                                    |  |
|  |  @Transactional Service method                             |  |
|  |  +-----------------------------------------------+          |  |
|  |  | Transaction + persistence context active     |          |  |
|  |  | Entities are MANAGED                         |          |  |
|  |  +-----------------------------------------------+          |  |
|  |  Transaction committed, persistence context CLOSED          |  |
|  |  Entities become DETACHED                                  |  |
```

```
|  |       |                                                        |  |
|  |  Controller — entities are DETACHED                           |  |
|  |  Lazy loading FAILS: LazyInitializationException!             |  |
|  |       |                                                        |  |
|  |  Must use DTOs or fetch everything in the service             |  |
|  +-------------------------------------------------------------+  |
+-----------------------------------------------------------------+
```

OSIV: The Debate

```
OSIV Pros and Cons:
+-----------------------------------------------------------------+
|                                                                  |
|  PROS (why Spring enables it by default):                        |
|  - Convenient: lazy loading works everywhere                     |
|  - No LazyInitializationException surprises                      |
|  - Less upfront thinking about what to fetch                     |
|                                                                  |
|  CONS (why many teams disable it):                               |
|  - Hidden N+1 queries: lazy loads in the view layer              |
|    generate unexpected SQL, hard to find and optimize            |
|  - Database connection held for entire request                   |
|    (including JSON serialization and network I/O)                |
|  - Violates separation of concerns: data access in               |
|    the presentation layer                                        |
|  - Can cause accidental writes if entity is modified             |
|    outside the service transaction                               |
|                                                                  |
|  Recommendation for production:                                  |
|  spring.jpa.open-in-view=false                                   |
|  - Forces you to fetch everything needed in the service layer    |
|  - Use DTOs or @EntityGraph to load data explicitly              |
|  - Better performance, clearer architecture                      |
+-----------------------------------------------------------------+
```

To disable OSIV:

```properties
# application.properties
spring.jpa.open-in-view=false
```

Batch Processing with flush() and clear()

When processing many entities, the persistence context grows and slows down. Use `flush()` + `clear()` periodically:

```java
@Service
public class BatchService {

    @PersistenceContext
    private EntityManager entityManager;

    @Transactional
    public void importEmployees(List<EmployeeData> dataList) {
        int batchSize = 50;

        for (int i = 0; i < dataList.size(); i++) {
            EmployeeData data = dataList.get(i);
            Employee employee = new Employee(data.getName(), data.getEmail());
            entityManager.persist(employee);

            // Every 50 entities: flush to DB and clear the cache
            if ((i + 1) % batchSize == 0) {
                entityManager.flush();  // Send INSERTs to DB
                entityManager.clear();  // Free memory
            }
        }
        // Final flush for remaining entities
        entityManager.flush();
        entityManager.clear();
    }
}
```

```
Batch Processing Memory:
+-------------------------------------------------------------------+
|                                                                   |
|  Without flush + clear:                                           |
|                                                                   |
|  Persistence context grows with each persist:                     |
|  [1] [1,2] [1,2,3] ... [1,2,3,...,10000]                         |
|  Memory: 10,000 entities + 10,000 snapshots                       |
|  Dirty checking: compares ALL 10,000 on every flush               |
|  --> OutOfMemoryError with large datasets                         |
|                                                                   |
|  With flush + clear (batch size 50):                              |
|                                                                   |
|  [1..50] --> flush + clear --> [51..100] --> flush + clear -->    |
|  [101..150] --> ...                                               |
|  Memory: max 50 entities at any time                              |
|  Dirty checking: compares only 50 entities per flush              |
|  --> Constant memory usage, scales to millions                    |
+-------------------------------------------------------------------+
```

Common Mistakes

1. Calling save() on managed entities unnecessarily: When an entity is loaded within `@Transactional`, changes are auto-detected. Calling `repository.save(entity)` after `entity.setName()` is redundant and confusing.
2. Using the original entity after merge(): `merge()` returns a NEW managed copy. The original remains detached. Always use the returned entity: `Employee managed = em.merge(detached);`.
3. Calling clear() without flush(): `clear()` discards ALL pending changes. If you clear without flushing first, unsaved modifications are silently lost.
4. Expecting lazy loading after transaction ends: Without OSIV, accessing a lazy collection after the `@Transactional` method returns throws `LazyInitializationException`. Load all needed data within the transaction.
5. Modifying detached entities and expecting auto-save: Detached entities are not tracked. Changes are ignored unless you call `merge()` or `save()` within a new transaction.
6. Not understanding that flush is not commit: `flush()` sends SQL to the database but does NOT commit. The transaction can still be rolled back. Only `commit` (at the end of `@Transactional`) makes changes permanent.

Best Practices

1. Disable OSIV in production: Set `spring.jpa.open-in-view=false`. Fetch all required data in the service layer using `@EntityGraph`, `JOIN FETCH`, or DTOs.
2. Do not call save() on managed entities: If you loaded an entity within `@Transactional`, just modify it. Dirty checking handles the rest. Only call `save()` for new entities or when merging detached ones.
3. Always use the return value of save() and merge(): The returned entity is the managed instance. The passed-in entity might be a different (detached) object.
4. Use flush() + clear() for batch operations: When processing hundreds or thousands of entities, periodically flush and clear to prevent memory buildup and slow dirty checking.
5. Keep transactions short: Long transactions hold database connections and locks. Do your processing, save the results, and end the transaction. Avoid calling external APIs or doing I/O within a transaction.
6. Use @Transactional(readOnly = true) for read-only operations: This tells Hibernate to skip dirty checking, saving CPU and memory. It also hints to the database to use a read-only transaction.

Summary

In this chapter, you learned how Hibernate manages entities internally:

- Four lifecycle states: NEW (transient, no DB row), MANAGED (tracked, auto-saved), DETACHED (disconnected, not tracked), REMOVED (scheduled for deletion).
- Persistence context is a first-level cache that guarantees identity (same ID = same object), caches loaded entities, and tracks changes via dirty checking.
- Dirty checking automatically detects changes to managed entities by comparing current values against a snapshot taken at load time. No explicit save needed.
- Flushing sends pending SQL to the database. It happens automatically before queries (AUTO mode) and on transaction commit. It is NOT a commit.
- persist() makes a new entity managed. merge() copies a detached entity into the persistence context (returns a new managed copy). remove() schedules deletion. detach() and clear() remove entities from tracking.
- OSIV keeps the persistence context open for the entire HTTP request. Convenient but causes hidden N+1 queries and holds DB connections too long. Disable in production.
- Batch processing requires periodic `flush()` + `clear()` to prevent memory exhaustion.

Interview Questions

Q1: What are the four JPA entity lifecycle states?

NEW (transient): created with `new`, no persistence context or DB row. MANAGED: tracked by the persistence context, changes auto-detected. DETACHED: was managed but disconnected (transaction ended, or explicitly detached). REMOVED: scheduled for deletion, will be deleted on flush.

Q2: What is the persistence context and what guarantees does it provide?

The persistence context is Hibernate's first-level cache. It guarantees: (1) identity — the same entity ID always returns the same Java object within a transaction, (2) caching — repeated finds for the same ID do not hit the database, (3) dirty checking — changes to managed entities are automatically detected and flushed as SQL.

Q3: Do you need to call save() after modifying a managed entity?

No. Managed entities are tracked by dirty checking. When you modify a field, Hibernate detects the change and generates an UPDATE on flush/commit. Calling `save()` is only needed for new entities (persist) or detached entities (merge).

Q4: What is the difference between flush() and commit()?

`flush()` sends pending SQL statements to the database but does not end the transaction — changes can still be rolled back. `commit()` finalizes the transaction, making all changes permanent. Hibernate flushes automatically before commit.

Q5: Why does merge() return a different object?

`merge()` copies the state of a detached entity into a managed instance (either an existing managed entity with the same ID, or a newly loaded one). The returned managed copy is tracked by the persistence context. The original detached entity remains detached and untracked.

Q6: What is Open Session in View and should you use it?

OSIV keeps the persistence context open for the entire HTTP request, allowing lazy loading in controllers and view rendering. Spring Boot enables it by default. It should be disabled in production (`spring.jpa.open-in-view=false`) because it causes hidden N+1 queries, holds database connections too long, and violates the separation between service and presentation layers.

Q7: How do you handle batch processing to avoid OutOfMemoryError?

Periodically call `entityManager.flush()` followed by `entityManager.clear()` (e.g., every 50 entities). `flush()` sends pending SQL to the database, and `clear()` removes all entities from the persistence context, freeing memory. This keeps memory usage constant regardless of the total number of entities.

Practice Exercises

Exercise 1: State Transitions Write a test method that creates an entity, persists it, modifies it, detaches it, merges it, and removes it. After each step, verify the entity's state by checking if the persistence context contains it (`entityManager.contains(entity)`).

Exercise 2: Identity Guarantee Load the same entity by ID twice and by a query. Verify that all three references point to the exact same Java object (==). Then clear the persistence context and load again — verify you get a different object instance.

Exercise 3: Dirty Checking Load an entity, modify three fields, and let the transaction commit. Enable SQL logging and verify that Hibernate generates a single UPDATE with all three changes — without any explicit save() call.

Exercise 4: merge() Behavior Create a method that returns a detached entity (load in one transaction, return after transaction ends). In a new transaction, modify the detached entity and merge it. Verify: (a) the returned entity is different from the original, (b) only the returned entity is managed, (c) the database is updated.

Exercise 5: Batch Import Write a batch import method that inserts 1,000 entities. First implement without flush/clear and observe memory growth. Then add flush/clear every 50 entities. Enable Hibernate statistics to compare the two approaches.

What Is Next?

In the next chapter, we will tackle Fetching Strategies: Solving the N+1 Problem. You will learn the difference between lazy and eager fetching, understand why the N+1 problem is the most common Hibernate performance issue, and master the tools to solve it: JOIN FETCH, @EntityGraph, @BatchSize, and subselect fetching.

Chapter 18: Fetching Strategies — Solving the N+1 Problem

Learning Goals

By the end of this chapter, you will be able to:

- Understand the difference between lazy and eager fetching
- Explain why the N+1 problem is the most common Hibernate performance issue
- Detect the N+1 problem using SQL logging
- Solve N+1 with JOIN FETCH in JPQL queries
- Use @EntityGraph for query-level fetch control
- Apply @BatchSize (Hibernate) to batch lazy loads
- Use @Fetch(FetchMode.SUBSELECT) for collection fetching
- Understand and fix LazyInitializationException
- Choose the right fetching strategy for different use cases
- Profile and measure fetching performance

Lazy vs Eager Fetching

Every JPA relationship has a fetch type that controls when related data is loaded from the database.

```
Fetch Types:
+-------------------------------------------------------------------+
|                                                                   |
|  FetchType.EAGER:                                                 |
|  Load the related entity IMMEDIATELY with the parent.             |
```

```
|  One query loads everything via JOIN.                              |
|                                                                    |
|  Employee e = em.find(Employee.class, 1L);                         |
|  SQL: SELECT e.*, d.*                                              |
|       FROM employees e                                             |
|       LEFT JOIN departments d ON e.department_id = d.id            |
|       WHERE e.id = 1                                               |
|  --> Department loaded immediately, even if never accessed         |
|                                                                    |
|  FetchType.LAZY:                                                   |
|  Load the related entity ONLY when it is accessed.                 |
|  Parent loaded first, related data loaded on demand.               |
|                                                                    |
|  Employee e = em.find(Employee.class, 1L);                         |
|  SQL: SELECT e.* FROM employees e WHERE e.id = 1                   |
|  --> Department NOT loaded yet                                     |
|                                                                    |
|  e.getDepartment().getName();  // NOW it loads                     |
|  SQL: SELECT d.* FROM departments d WHERE d.id = 5                |
|  --> Department loaded on first access                             |
+--------------------------------------------------------------------+
```

JPA Default Fetch Types

```
Default Fetch Types:
+--------------------------------------------------------------------+
|                                                                    |
|  Annotation         Default FetchType   Recommendation             |
|  ------------------------------------------------------------------|
|  @ManyToOne          EAGER              Change to LAZY!            |
|  @OneToOne           EAGER              Change to LAZY!            |
|  @OneToMany          LAZY               Keep LAZY (good default)   |
|  @ManyToMany         LAZY               Keep LAZY (good default)   |
|                                                                    |
|  Single-value associations (@ManyToOne, @OneToOne) default to      |
|  EAGER because loading one extra row seems cheap.                  |
|  BUT this compounds: 1000 employees = 1000 department loads!      |
|                                                                    |
|  RULE: Make EVERYTHING lazy, then eagerly fetch what you           |
|  need per query using JOIN FETCH or @EntityGraph.                  |
+--------------------------------------------------------------------+
```

Always override the defaults:

```java
// BAD — uses default EAGER (loads department with every query)
@ManyToOne
private Department department;
```

```
// GOOD — lazy by default, load explicitly when needed
@ManyToOne(fetch = FetchType.LAZY)
private Department department;
```

The N+1 Problem Explained

The N+1 problem is the single most common cause of poor Hibernate performance. It happens when loading N entities triggers N additional queries for their relationships.

```
The N+1 Problem — Step by Step:
+-------------------------------------------------------------------+
|                                                                   |
|  Goal: Display 10 departments with their employee counts         |
|                                                                   |
|  Code:                                                            |
|  List<Department> depts = departmentRepository.findAll();         |
|  for (Department d : depts) {                                     |
|      System.out.println(d.getName() + ": " +                      |
|          d.getEmployees().size());  // Triggers lazy load!        |
|  }                                                                |
|                                                                   |
|  SQL queries generated:                                           |
|  ----------------------------------------------------------------|
|  Query 1:  SELECT * FROM departments                (1 query)    |
|            Returns 10 departments                                 |
|                                                                   |
|  Query 2:  SELECT * FROM employees                               |
|            WHERE department_id = 1                  (dept #1)     |
|  Query 3:  SELECT * FROM employees                               |
|            WHERE department_id = 2                  (dept #2)     |
|  Query 4:  SELECT * FROM employees                               |
|            WHERE department_id = 3                  (dept #3)     |
|  ...                                                              |
|  Query 11: SELECT * FROM employees                               |
|            WHERE department_id = 10                 (dept #10)    |
|                                                                   |
|  Total: 1 + 10 = 11 queries   (N+1 where N=10)                   |
|                                                                   |
|  With 100 departments:  101 queries                               |
|  With 1000 departments: 1001 queries!                             |
+-------------------------------------------------------------------+
```

N+1 with @ManyToOne (Eager Default)

The N+1 problem also happens with `@ManyToOne` when the default EAGER fetch type is not overridden:

```
@ManyToOne N+1:
+-------------------------------------------------------------------+
|                                                                   |
|  @ManyToOne  // Default: EAGER!                                   |
|  private Department department;                                   |
|                                                                   |
|  List<Employee> employees = employeeRepository.findAll();         |
|                                                                   |
|  SQL queries:                                                     |
|  Query 1:  SELECT * FROM employees               (loads 100 emps)|
|                                                                   |
|  Hibernate sees EAGER @ManyToOne, loads each department:          |
|  Query 2:  SELECT * FROM departments WHERE id = 1                 |
|  Query 3:  SELECT * FROM departments WHERE id = 2                 |
|  Query 4:  SELECT * FROM departments WHERE id = 1  (duplicate!)   |
|  Query 5:  SELECT * FROM departments WHERE id = 3                 |
|  ...                                                              |
|                                                                   |
|  Even with persistence context caching (same dept not queried     |
|  twice), you still get 1 + number_of_unique_departments queries!  |
|                                                                   |
|  FIX: @ManyToOne(fetch = FetchType.LAZY)                          |
|  Then use JOIN FETCH when you actually need the department.       |
+-------------------------------------------------------------------+
```

Solution 1: JOIN FETCH in JPQL

`JOIN FETCH` tells Hibernate to load the relationship in the same query using a SQL JOIN:

```java
@Repository
public interface DepartmentRepository extends JpaRepository<Department, Long> {

    // WITHOUT JOIN FETCH — causes N+1
    @Query("SELECT d FROM Department d")
    List<Department> findAllBasic();
    // 1 query + N lazy loads when accessing employees

    // WITH JOIN FETCH — single query
    @Query("SELECT d FROM Department d LEFT JOIN FETCH d.employees")
    List<Department> findAllWithEmployees();
    // 1 query! All departments AND their employees loaded together
}
```

```
JOIN FETCH Generated SQL:
+-------------------------------------------------------------------+
|                                                                   |
|  JPQL:  SELECT d FROM Department d LEFT JOIN FETCH d.employees    |
|                                                                   |
|  SQL:   SELECT d.*, e.*                                           |
|         FROM departments d                                        |
|         LEFT JOIN employees e ON d.id = e.department_id           |
|                                                                   |
|  Result set (denormalized):                                       |
|  | d.id | d.name      | e.id | e.name   | e.department_id |       |
|  |------|-------------|------|---------|-----------------|       |
|  | 1    | Engineering | 10   | Alice   | 1               |       |
|  | 1    | Engineering | 11   | Bob     | 1               |       |
|  | 1    | Engineering | 12   | Carol   | 1               |       |
|  | 2    | Marketing   | 13   | Dave    | 2               |       |
|  | 2    | Marketing   | 14   | Eve     | 2               |       |
|  | 3    | Sales       | NULL | NULL    | NULL            |       |
|  (LEFT JOIN includes Sales even though it has no employees)       |
|                                                                   |
|  Hibernate de-duplicates the result into:                         |
|  Department(1, "Engineering") --> [Alice, Bob, Carol]             |
|  Department(2, "Marketing")   --> [Dave, Eve]                     |
|  Department(3, "Sales")       --> []                              |
|                                                                   |
|  Total: 1 query instead of N+1!                                   |
+-------------------------------------------------------------------+
```

Multiple JOIN FETCH

```java
// Fetch department AND projects for employees
@Query("SELECT e FROM Employee e " +
       "LEFT JOIN FETCH e.department " +
       "LEFT JOIN FETCH e.projects " +
       "WHERE e.active = true")
List<Employee> findActiveWithDeptAndProjects();
```

JOIN FETCH Limitations

```
JOIN FETCH Limitations:
+-------------------------------------------------------------------+
|                                                                   |
|  1. Cannot JOIN FETCH multiple collections (bags):                |
|     SELECT d FROM Department d                                    |
|       LEFT JOIN FETCH d.employees                                 |
|       LEFT JOIN FETCH d.projects    <-- MultipleBagFetchException!|
|                                                                   |
```

```
|     Reason: Fetching two Lists creates a Cartesian product        |
|     with ambiguous results.                                       |
|                                                                   |
|     Fixes:                                                        |
|     a. Use Set instead of List for collections                    |
|     b. Fetch in separate queries (two queries, each with          |
|        one JOIN FETCH)                                            |
|     c. Use @BatchSize on one collection                           |
|                                                                   |
|  2. Cannot use JOIN FETCH with COUNT for pagination:              |
|     Page<Department> findAll(Pageable p)                          |
|     with JOIN FETCH fails for count query derivation.             |
|     --> Provide explicit countQuery.                              |
|                                                                   |
|  3. LEFT JOIN FETCH vs JOIN FETCH:                                |
|     LEFT JOIN FETCH: includes parents with no children            |
|     JOIN FETCH: excludes parents with no children                 |
|     Usually you want LEFT JOIN FETCH.                             |
+-------------------------------------------------------------------+
```

DISTINCT with JOIN FETCH

JOIN FETCH with `@OneToMany` can produce duplicate parent entities:

```java
// WITHOUT DISTINCT - departments may appear multiple times!
@Query("SELECT d FROM Department d LEFT JOIN FETCH d.employees")
List<Department> findAllWithEmployees();
// Engineering appears 3 times (once for each employee)

// WITH DISTINCT - each department appears once
@Query("SELECT DISTINCT d FROM Department d LEFT JOIN FETCH d.employees")
List<Department> findAllWithEmployeesDistinct();
// Engineering appears once with 3 employees
```

```
DISTINCT Behavior:
+-------------------------------------------------------------------+
|                                                                   |
|  Without DISTINCT:                                                |
|  findAllWithEmployees() returns:                                  |
|  [Engineering, Engineering, Engineering, Marketing, Marketing]    |
|  (3 copies of Engineering, 2 copies of Marketing)                |
|                                                                   |
|  With DISTINCT:                                                   |
|  findAllWithEmployeesDistinct() returns:                          |
|  [Engineering, Marketing, Sales]                                  |
|  (one of each, with employees fully loaded)                       |
|                                                                   |
|  Hibernate 6 applies DISTINCT at the entity level,                |
|  not the SQL level (avoids adding DISTINCT to SQL which           |
```

```
|  would slow down the database query).                              |
+-------------------------------------------------------------------+
```

Solution 2: @EntityGraph

`@EntityGraph` (JPA 2.1) specifies fetch plans at the query level without writing JPQL:

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Override findAll() with eager department loading
    @Override
    @EntityGraph(attributePaths = {"department"})
    List<Employee> findAll();

    // Fetch department for specific queries
    @EntityGraph(attributePaths = {"department"})
    List<Employee> findByActive(boolean active);

    // Fetch multiple relationships
    @EntityGraph(attributePaths = {"department", "projects"})
    Optional<Employee> findById(Long id);

    // Nested paths
    @EntityGraph(attributePaths = {"department", "department.company"})
    List<Employee> findBySalaryGreaterThan(BigDecimal min);

    // Combine with JPQL
    @EntityGraph(attributePaths = {"department"})
    @Query("SELECT e FROM Employee e WHERE e.salary > :min")
    List<Employee> findHighEarnersWithDept(@Param("min") BigDecimal min);
}
```

```
@EntityGraph vs JOIN FETCH:
+-------------------------------------------------------------------+
|                                                                   |
|  @EntityGraph:                                                    |
|  + Works on derived query methods (findByX)                       |
|  + Clean — no JPQL needed                                         |
|  + Easy to add/remove relationships                               |
|  - Cannot control JOIN type (always LEFT JOIN)                    |
|  - Cannot filter the fetched collection                           |
|                                                                   |
|  JOIN FETCH:                                                      |
|  + Full control over JOIN type (LEFT, INNER)                      |
|  + Can add WHERE conditions on the joined entity                  |
```

```
|  + More explicit about what happens                                |
|  - Requires JPQL query                                             |
|  - More verbose                                                    |
|                                                                    |
|  When to use which:                                                |
|  - Simple eager loading on derived methods: @EntityGraph           |
|  - Complex queries with conditions on joined data: JOIN FETCH      |
|  - Both solve the N+1 problem equally well                         |
+--------------------------------------------------------------------+
```

Solution 3: @BatchSize (Hibernate-Specific)

`@BatchSize` tells Hibernate to batch multiple lazy loads into a single query using `IN (...)`:

```java
@Entity
@Table(name = "departments")
public class Department {

    @OneToMany(mappedBy = "department")
    @BatchSize(size = 10)    // Hibernate-specific
    private List<Employee> employees = new ArrayList<>();
}
```

```
@BatchSize Behavior:
+--------------------------------------------------------------------+
|                                                                    |
|  Without @BatchSize — standard lazy loading (N+1):                 |
|  Query 1: SELECT * FROM departments  (10 departments)              |
|  Query 2: SELECT * FROM employees WHERE department_id = 1          |
|  Query 3: SELECT * FROM employees WHERE department_id = 2          |
|  ...                                                               |
|  Query 11: SELECT * FROM employees WHERE department_id = 10        |
|  Total: 11 queries                                                 |
|                                                                    |
|  With @BatchSize(size = 10):                                       |
|  Query 1: SELECT * FROM departments  (10 departments)              |
|  Query 2: SELECT * FROM employees                                  |
|           WHERE department_id IN (1, 2, 3, 4, 5, 6, 7, 8, 9, 10) |
|  Total: 2 queries!                                                 |
|                                                                    |
|  With 25 departments and @BatchSize(size = 10):                    |
|  Query 1: SELECT * FROM departments  (25 departments)              |
|  Query 2: SELECT ... WHERE department_id IN (1,2,...,10)           |
|  Query 3: SELECT ... WHERE department_id IN (11,12,...,20)         |
|  Query 4: SELECT ... WHERE department_id IN (21,22,...,25)         |
|  Total: 4 queries (1 + ceil(25/10) = 4)                            |
```

```
|                                                                    |
|  N+1 reduced to 1 + ceil(N / batchSize)                            |
+--------------------------------------------------------------------+
```

Global @BatchSize

Set a default batch size for all lazy collections:

```properties
# application.properties
spring.jpa.properties.hibernate.default_batch_fetch_size=16
```

This applies `@BatchSize(size = 16)` to all lazy collections and `@ManyToOne` relationships without needing to annotate each one.

```
When to Use @BatchSize:
+--------------------------------------------------------------------+
|                                                                    |
|  @BatchSize is great when:                                         |
|  - You cannot use JOIN FETCH (multiple collections)                |
|  - You want a "good enough" global optimization                    |
|  - You do not want to write custom JPQL for every query            |
|  - Some queries need the collection, others do not                 |
|                                                                    |
|  @BatchSize is NOT ideal when:                                     |
|  - You always need the collection (JOIN FETCH is faster: 1 query)  |
|  - You need precise control over the SQL                           |
+--------------------------------------------------------------------+
```

Solution 4: @Fetch(FetchMode.SUBSELECT) (Hibernate-Specific)

`SUBSELECT` fetching loads ALL collections in a single query using a subquery:

```java
@Entity
public class Department {

    @OneToMany(mappedBy = "department")
    @Fetch(FetchMode.SUBSELECT)    // Hibernate-specific
    private List<Employee> employees = new ArrayList<>();
}
```

```
SUBSELECT Behavior:
+-----------------------------------------------------------------+
|                                                                 |
|  Query 1: SELECT * FROM departments WHERE active = true         |
|           (returns 50 departments)                              |
|                                                                 |
|  When any department's employees are accessed:                  |
|  Query 2: SELECT * FROM employees WHERE department_id IN        |
|           (SELECT id FROM departments WHERE active = true)      |
|           (loads ALL employees for ALL 50 departments at once)  |
|                                                                 |
|  Total: 2 queries regardless of number of departments!          |
|                                                                 |
|  Comparison:                                                    |
|  N+1 (no optimization):     1 + 50 = 51 queries                |
|  @BatchSize(size = 10):     1 + 5  = 6 queries                 |
|  @Fetch(SUBSELECT):         1 + 1  = 2 queries (always)        |
|  JOIN FETCH:                1 query (but data is denormalized)  |
+-----------------------------------------------------------------+
```

Solving MultipleBagFetchException

When you try to JOIN FETCH two `List<>` collections, Hibernate throws `MultipleBagFetchException`:

```java
// FAILS! Cannot fetch two Lists at once.
@Query("SELECT d FROM Department d " +
       "LEFT JOIN FETCH d.employees " +
       "LEFT JOIN FETCH d.projects")
List<Department> findWithAll();
// --> MultipleBagFetchException
```

```
Solutions for MultipleBagFetchException:
+-----------------------------------------------------------------+
|                                                                 |
|  Solution 1: Use Set instead of List                            |
|  @OneToMany(mappedBy = "department")                            |
|  private Set<Employee> employees = new HashSet<>();             |
|  --> Sets do not cause Cartesian product ambiguity              |
|  --> JOIN FETCH of multiple Sets works                          |
|                                                                 |
|  Solution 2: Fetch in separate queries                          |
|  @EntityGraph(attributePaths = {"employees"})                   |
|  List<Department> findAllWithEmployees();                       |
```

```
|                                                                    |
|  @EntityGraph(attributePaths = {"projects"})                       |
|  List<Department> findAllWithProjects();                           |
|  --> Call both methods; persistence context merges results         |
|                                                                    |
|  Solution 3: Use @BatchSize on one collection                      |
|  JOIN FETCH the primary collection                                 |
|  @BatchSize on the secondary collection                            |
|  --> 1 JOIN FETCH + batched lazy loading                           |
|                                                                    |
|  Solution 4: Fetch one, use SUBSELECT for the other                |
|  JOIN FETCH employees                                               |
|  @Fetch(FetchMode.SUBSELECT) on projects                           |
+--------------------------------------------------------------------+
```

Solution 2 in Practice: Two-Query Approach

```java
@Service
@Transactional(readOnly = true)
public class DepartmentService {

    private final DepartmentRepository repository;

    public List<Department> findAllWithEverything() {
        // Query 1: Load departments + employees
        List<Department> departments = repository.findAllWithEmployees();

        // Query 2: Load same departments + projects
        // Persistence context already has these departments,
        // so this just adds the projects to the same instances
        repository.findAllWithProjects();

        // Both collections are now loaded on the same objects
        return departments;
    }
}
```

LazyInitializationException

This exception occurs when you access a lazy relationship outside the persistence context (after the transaction has ended):

```
LazyInitializationException:
+-------------------------------------------------------------------+
|                                                                   |
|  @Service                                                         |
|  public class EmployeeService {                                   |
|                                                                   |
|      @Transactional                                               |
|      public Employee getEmployee(Long id) {                       |
|          return employeeRepository.findById(id).orElseThrow();    |
|      }                                                            |
|  }                                                                |
|                                                                   |
|  @RestController                                                  |
|  public class EmployeeController {                                |
|                                                                   |
|      public String getDeptName(@PathVariable Long id) {           |
|          Employee e = service.getEmployee(id);                    |
|          // Transaction already ended!                            |
|          // Persistence context is closed!                        |
|          return e.getDepartment().getName();                      |
|          //      ^^^^^^^^^^^^^^^^                                 |
|          // LazyInitializationException!                          |
|          // Cannot initialize proxy — no Session                  |
|      }                                                            |
|  }                                                                |
|                                                                   |
+-------------------------------------------------------------------+
```

Fixes for LazyInitializationException

```
How to Fix LazyInitializationException:
+-------------------------------------------------------------------+
|                                                                   |
|  Fix 1: Load the data within the transaction (BEST)               |
|  @Transactional(readOnly = true)                                  |
|  public EmployeeDto getEmployee(Long id) {                        |
|      Employee e = repository.findById(id).orElseThrow();          |
|      return new EmployeeDto(                                      |
|          e.getName(),                                             |
|          e.getDepartment().getName()  // Loaded inside @Tx        |
|      );                                                           |
|  }                                                                |
|                                                                   |
|  Fix 2: Use JOIN FETCH or @EntityGraph                            |
|  @EntityGraph(attributePaths = {"department"})                    |
|  Optional<Employee> findWithDeptById(Long id);                    |
|  --> Department loaded eagerly with the employee                  |
|                                                                   |
|  Fix 3: Use a DTO projection (no entity, no lazy issue)           |
|  @Query("SELECT new com.example.EmployeeDto(e.name, d.name) " +  |
|         "FROM Employee e JOIN e.department d WHERE e.id = :id")   |
```

```
|  Optional<EmployeeDto> findDtoById(@Param("id") Long id);        |
|                                                                   |
|  DO NOT FIX WITH:                                                 |
|  - Making everything EAGER (causes N+1 everywhere)               |
|  - Enabling OSIV (masks the problem, causes hidden N+1)          |
|  - Hibernate.initialize() outside transaction (too late)          |
+-------------------------------------------------------------------+
```

Detecting N+1 Problems

Enable SQL Logging

```properties
# application.properties
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Better: use logging with parameter values
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.orm.jdbc.bind=TRACE
```

Hibernate Statistics

```properties
# Enable statistics
spring.jpa.properties.hibernate.generate_statistics=true
```

```java
// Programmatic check
@Autowired
private EntityManagerFactory emf;

public void checkQueryCount() {
    Statistics stats = emf.unwrap(SessionFactory.class).getStatistics();
    stats.clear();

    // Execute your code here
    departmentService.getAllWithEmployees();

    long queryCount = stats.getQueryExecutionCount();
    System.out.println("Queries executed: " + queryCount);
    // If this shows 11 for 10 departments, you have N+1!
}
```

Assert Query Count in Tests

```java
// Using datasource-proxy or a custom counter
@Test
void shouldLoadDepartmentsInOneQuery() {
    // Enable query counter
    long before = getQueryCount();

    List<Department> depts = repository.findAllWithEmployees();
    depts.forEach(d -> d.getEmployees().size()); // Force access

    long queries = getQueryCount() - before;
    assertThat(queries).isEqualTo(1); // Exactly 1 query, no N+1
}
```

Fetching Strategy Decision Guide

```
Which Fetching Strategy to Use:
+-----------------------------------------------------------------+
|                                                                 |
|  Always need the related data?                                  |
|      |                                                          |
|   YES                                                           |
|      |                                                          |
|      v                                                          |
|  Single collection or @ManyToOne?                               |
|      |                                                          |
|   YES --> JOIN FETCH or @EntityGraph                            |
|      |    (1 query, most efficient)                             |
|      |                                                          |
|   Multiple collections?                                         |
|      |                                                          |
|      +--> Can use Set? --> JOIN FETCH both                      |
|      |                                                          |
|      +--> Must use List? --> Two queries or                     |
|           JOIN FETCH one + @BatchSize on the other              |
|                                                                 |
|  Sometimes need the related data?                               |
|      |                                                          |
|      +--> @BatchSize (global default)                           |
|      |    Lazy by default, batch-loads when accessed            |
|      |    Best "general purpose" optimization                   |
|      |                                                          |
|      +--> @Fetch(SUBSELECT) (always loads ALL in 1 extra query)  |
|                                                                 |
|  Never need the related data?                                   |
```

```
|      |                                                                |
|      +--> FetchType.LAZY, no optimization needed                     |
|           (never accessed = never loaded = zero overhead)            |
+----------------------------------------------------------------------+
```

```
Performance Comparison (10 departments, each with employees):
+----------------------------------------------------------------------+
|                                                                      |
|  Strategy                        Queries   Notes                     |
|  ---------------------------------------------------------------------|
|  Default LAZY (N+1)              11        1 + 10 lazy loads         |
|  @BatchSize(size = 10)           2         1 + 1 batch IN(...)       |
|  @BatchSize(size = 5)            3         1 + 2 batches             |
|  @Fetch(SUBSELECT)               2         1 + 1 subquery            |
|  JOIN FETCH                      1         Everything in 1 query     |
|  @EntityGraph                    1         Same as JOIN FETCH        |
|  EAGER (default on @ManyToOne)   11        Same as N+1!              |
|                                                                      |
|  Note: JOIN FETCH returns more data (denormalized result set)        |
|  which uses more memory and network bandwidth.                       |
|  @BatchSize/SUBSELECT return normalized data in fewer queries.       |
|  For large result sets, 2 queries may be faster than 1 large one.    |
+----------------------------------------------------------------------+
```

Real-World Example: Employee Dashboard

```java
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;
    private BigDecimal salary;
    private boolean active;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(name = "employee_projects",
```

```
        joinColumns = @JoinColumn(name = "employee_id"),
        inverseJoinColumns = @JoinColumn(name = "project_id"))
    private Set<Project> projects = new HashSet<>();
}
```

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // For list view (just need department name)
    @EntityGraph(attributePaths = {"department"})
    List<Employee> findByActive(boolean active);

    // For detail view (need everything)
    @Query("SELECT DISTINCT e FROM Employee e " +
           "LEFT JOIN FETCH e.department " +
           "LEFT JOIN FETCH e.projects " +
           "WHERE e.id = :id")
    Optional<Employee> findDetailById(@Param("id") Long id);

    // For reporting (no relationships needed - use DTO)
    @Query("SELECT new com.example.dto.EmployeeReport(" +
           "e.name, e.salary, d.name) " +
           "FROM Employee e JOIN e.department d " +
           "WHERE e.active = true")
    List<EmployeeReport> findActiveReport();
}
```

```java
@Service
@Transactional(readOnly = true)
public class EmployeeService {

    private final EmployeeRepository repository;

    // List endpoint - loads department for display
    public List<EmployeeListDto> getActiveEmployees() {
        return repository.findByActive(true).stream()
            .map(e -> new EmployeeListDto(
                e.getId(),
                e.getName(),
                e.getDepartment().getName() // Already loaded via @EntityGraph
            ))
            .toList();
    }

    // Detail endpoint - loads everything
    public EmployeeDetailDto getEmployeeDetail(Long id) {
        Employee e = repository.findDetailById(id).orElseThrow();
        return new EmployeeDetailDto(
            e.getId(),
            e.getName(),
```

```
            e.getDepartment().getName(),
            e.getProjects().stream().map(Project::getName).toList()
        );
    }

    // Report endpoint - no entities at all
    public List<EmployeeReport> getReport() {
        return repository.findActiveReport();
    }
}
```

```
Different Fetch Plans for Different Use Cases:
+-------------------------------------------------------------------+
|                                                                   |
|  Use Case           Relationships Needed    Approach              |
|  -----------------------------------------------------------------|
|  Employee list      department name         @EntityGraph({dept})  |
|  Employee detail    dept + projects         JOIN FETCH both       |
|  Salary report      dept name only          DTO projection        |
|  Employee search    none (just employee)    No fetch optimization |
|  Bulk update        none                    JPQL UPDATE           |
|                                                                   |
|  Key insight: There is no single "right" fetch plan.              |
|  Different screens/APIs need different data, so use               |
|  different queries with different fetch strategies.               |
+-------------------------------------------------------------------+
```

Common Mistakes

1. Leaving @ManyToOne as default EAGER: This is the #1 cause of N+1 in real applications. Always set `fetch = FetchType.LAZY` on `@ManyToOne` and `@OneToOne`.
2. Adding EAGER to fix LazyInitializationException: This "fixes" one place but adds unnecessary joins everywhere else. Use `@EntityGraph` or `JOIN FETCH` on the specific queries that need the data.
3. Using JOIN FETCH without DISTINCT: `@OneToMany` JOIN FETCH produces duplicate parent entities. Always add `DISTINCT` in the JPQL.
4. Fetching two Lists with JOIN FETCH: Two `List<>` collections cause `MultipleBagFetchException`. Convert to `Set<>` or use separate queries.
5. Not detecting N+1 during development: Enable SQL logging and Hibernate statistics during development. Check query counts for list operations. N+1 is invisible without logging.

6. Applying the same fetch strategy everywhere: Different queries need different fetch plans. A list endpoint needs fewer relationships than a detail endpoint.

Best Practices

1. Make all relationships LAZY: Set `FetchType.LAZY` on every `@ManyToOne`, `@OneToOne`, `@OneToMany`, and `@ManyToMany`. Then selectively fetch what you need.
2. Set a global default_batch_fetch_size: Add `hibernate.default_batch_fetch_size=16` as a baseline optimization. This reduces N+1 to N/16+1 without any code changes.
3. Use @EntityGraph for simple eager loading: When you need department with employees, `@EntityGraph(attributePaths = {"department"})` is cleaner than writing JPQL.
4. Use JOIN FETCH for complex queries: When you need control over JOIN type, filtering, or multiple conditions on the joined entity.
5. Use DTO projections when you do not need entities: A `SELECT new EmployeeDto(e.name, d.name)` query avoids all lazy loading issues because there are no entities to lazy-load.
6. Always enable SQL logging during development: `logging.level.org.hibernate.SQL=DEBUG` makes N+1 immediately visible. Fix it before it reaches production.
7. Match fetch plans to use cases: Create different repository methods for different screens: `findListView()`, `findDetailView()`, `findForReport()`.

Summary

In this chapter, you learned how to tame Hibernate's fetching behavior:

- Lazy vs Eager: Lazy loads on access, Eager loads immediately. Default to LAZY for everything.
- N+1 problem: Loading N entities triggers N additional queries for their relationships. The most common Hibernate performance issue.
- JOIN FETCH: Loads relationships in one query via SQL JOIN. Most efficient but requires JPQL and careful handling of DISTINCT and multiple collections.
- @EntityGraph: Declarative fetch plans on repository methods. Cleaner than JPQL for simple cases. Same performance as JOIN FETCH.

- @BatchSize: Batches lazy loads into IN(...) queries. Good global default (`hibernate.default_batch_fetch_size`). Reduces N+1 to ceil(N/batchSize)+1.
- @Fetch(SUBSELECT): Loads all collections in one subquery. Always 2 queries total regardless of N.
- LazyInitializationException: Occurs when accessing lazy data outside the persistence context. Fix by loading data within `@Transactional`, using `@EntityGraph`/JOIN FETCH, or using DTO projections.
- Different use cases need different fetch plans. There is no universal "right" fetching strategy.

Interview Questions

Q1: What is the N+1 problem in Hibernate?

The N+1 problem occurs when loading a list of N entities triggers N additional queries to load their lazy relationships. For example, loading 100 departments and accessing each department's employees generates 1 query for departments + 100 queries for employees = 101 total. It is the most common cause of poor Hibernate performance.

Q2: What are the JPA default fetch types and why should you change them?

`@ManyToOne` and `@OneToOne` default to `EAGER`. `@OneToMany` and `@ManyToMany` default to `LAZY`. The EAGER defaults on single-value associations cause N+1 problems when loading collections of entities. Best practice: set everything to `LAZY` and use `JOIN FETCH` or `@EntityGraph` for specific queries that need related data.

Q3: What is the difference between JOIN FETCH and @EntityGraph?

Both solve N+1 by loading relationships eagerly in a single query. `JOIN FETCH` is used in JPQL queries and gives full control over JOIN type and filtering. `@EntityGraph` is declarative (annotation-based), works with derived query methods, and is cleaner for simple cases. They produce similar SQL.

Q4: How does @BatchSize reduce the N+1 problem?

Instead of loading each lazy collection one at a time (`N` individual queries), `@BatchSize(size=10)` groups lazy loads into batches of 10 using `WHERE id IN (1,2,...,10)`. This reduces N+1 queries to `1 + ceil(N/batchSize)` queries. It is a good default optimization that works without changing queries.

Q5: What causes MultipleBagFetchException and how do you fix it?

It occurs when JOIN FETCHing two `List<>` collections simultaneously, which creates an ambiguous Cartesian product. Fixes: (1) change `List` to `Set` for the collections, (2) fetch collections in separate queries, (3) JOIN FETCH one and use `@BatchSize` on the other.

Q6: What is LazyInitializationException and what is the correct fix?

It occurs when accessing a lazy relationship after the persistence context is closed (transaction ended). The correct fix is to load all needed data within the `@Transactional` service method using `JOIN FETCH`, `@EntityGraph`, or DTO projections. Do NOT fix by making relationships EAGER or enabling OSIV.

Practice Exercises

Exercise 1: N+1 Detection Create `Department` and `Employee` entities with a lazy `@OneToMany`. Load all departments and access each department's employees. Enable SQL logging and count the queries. Verify it is N+1.

Exercise 2: JOIN FETCH Fix Fix the N+1 from Exercise 1 using JOIN FETCH. Verify the query count drops to 1. Add DISTINCT and verify no duplicate departments.

Exercise 3: @BatchSize Comparison Apply `@BatchSize(size = 5)` to the employees collection. With 20 departments, verify the query count is 1 + 4 = 5 (instead of 21). Then try the global `default_batch_fetch_size` property.

Exercise 4: Multiple Collections Add a `@ManyToMany projects` collection to Department. Try to JOIN FETCH both `employees` and `projects`. Observe `MultipleBagFetchException`. Fix it using: (a) Set instead of List, (b) two separate queries.

Exercise 5: Real-World Fetch Plans Create repository methods for three use cases: (a) list view with department name only, (b) detail view with department + projects, (c) report with just names and salaries (DTO). Verify each generates optimal SQL.

What Is Next?

In the next chapter, we will explore Cascade Operations and Lifecycle Propagation. You will learn how cascade types control the propagation of persist, merge, remove, refresh, and detach operations through entity relationships, and how lifecycle callbacks (`@PrePersist`, `@PostUpdate`, etc.) let you hook into entity state changes.

Chapter 19: Cascade Operations and Lifecycle Propagation

Learning Goals

By the end of this chapter, you will be able to:

- Understand what cascading means in JPA relationships
- Use each CascadeType (PERSIST, MERGE, REMOVE, REFRESH, DETACH, ALL)
- Explain the difference between CascadeType.REMOVE and orphanRemoval
- Apply lifecycle callbacks (@PrePersist, @PostPersist, @PreUpdate, @PostUpdate, @PreRemove, @PostRemove)
- Create reusable @EntityListeners for cross-cutting concerns
- Know when to cascade and when not to cascade
- Avoid dangerous cascade configurations that cause data loss
- Implement audit logging with lifecycle callbacks

What Is Cascading?

Cascading means propagating a persistence operation from a parent entity to its related child entities. When you save, delete, or refresh a parent, the same operation can automatically apply to its children.

```
Cascading Concept:
+-----------------------------------------------------------------+
|                                                                 |
|  Without Cascade:                                               |
|  Order order = new Order();                                     |
```

```
|  OrderItem item1 = new OrderItem("Widget", 2);                  |
|  OrderItem item2 = new OrderItem("Gadget", 1);                  |
|  order.addItem(item1);                                           |
|  order.addItem(item2);                                           |
|                                                                  |
|  orderRepository.save(order);   // Saves order only              |
|  itemRepository.save(item1);    // Must save each item manually!  |
|  itemRepository.save(item2);    // Tedious and error-prone       |
|                                                                  |
|  With Cascade:                                                   |
|  @OneToMany(cascade = CascadeType.ALL)                           |
|  private List<OrderItem> items;                                  |
|                                                                  |
|  orderRepository.save(order);   // Saves order AND all items!    |
|  // One call does everything.                                    |
+------------------------------------------------------------------+
```

```
Cascade Flow:
+------------------------------------------------------------------+
|                                                                  |
|  Operation on Parent      CascadeType      Effect on Children    |
|  ----------------------------------------------------------------|
|  persist(order)           PERSIST           persist(item1)       |
|                                             persist(item2)       |
|                                                                  |
|  merge(order)             MERGE             merge(item1)         |
|                                             merge(item2)         |
|                                                                  |
|  remove(order)            REMOVE            remove(item1)        |
|                                             remove(item2)        |
|                                                                  |
|  refresh(order)           REFRESH           refresh(item1)       |
|                                             refresh(item2)       |
|                                                                  |
|  detach(order)            DETACH            detach(item1)        |
|                                             detach(item2)        |
|                                                                  |
|  ALL = all of the above                                          |
+------------------------------------------------------------------+
```

CascadeType.PERSIST

Propagates the `persist` (save) operation from parent to children:

```java
@Entity
public class Order {

    @OneToMany(mappedBy = "order", cascade = CascadeType.PERSIST)
    private List<OrderItem> items = new ArrayList<>();

    public void addItem(OrderItem item) {
        items.add(item);
        item.setOrder(this);
    }
}
```

```java
// One save() persists the order AND all items
Order order = new Order("ORD-001");
order.addItem(new OrderItem("Widget", 2, new BigDecimal("9.99")));
order.addItem(new OrderItem("Gadget", 1, new BigDecimal("24.99")));
orderRepository.save(order);

// SQL generated:
// INSERT INTO orders (order_number) VALUES ('ORD-001')
// INSERT INTO order_items (name, quantity, price, order_id) VALUES ('Widget', 2, 9.99, 1)
// INSERT INTO order_items (name, quantity, price, order_id) VALUES ('Gadget', 1, 24.99...
```

```
CascadeType.PERSIST Use Cases:
+-------------------------------------------------------------------+
|                                                                   |
|  USE when: Parent creates children as part of its lifecycle       |
|  - Order creates OrderItems                                       |
|  - Post creates Comments                                          |
|  - Invoice creates LineItems                                      |
|                                                                   |
|  DO NOT USE when: Children exist independently                    |
|  - Department does not create Employees                           |
|    (employees are hired separately)                               |
|  - Student does not create Courses                                |
|    (courses exist before enrollment)                              |
+-------------------------------------------------------------------+
```

CascadeType.MERGE

Propagates the `merge` operation — reattaches detached parent and children:

```java
@Entity
public class Order {
    @OneToMany(mappedBy = "order", cascade = {CascadeType.PERSIST, CascadeType.MERGE})
    private List<OrderItem> items = new ArrayList<>();
}
```

```java
// Load order in one transaction
Order detachedOrder = orderService.getOrder(1L);
// Transaction ends, order is DETACHED

// Modify in a different context (e.g., REST API receives modified order)
detachedOrder.getItems().get(0).setQuantity(5);
detachedOrder.addItem(new OrderItem("New Item", 1, new BigDecimal("14.99")));

// Merge back in a new transaction
orderService.updateOrder(detachedOrder);

@Transactional
public Order updateOrder(Order detachedOrder) {
    return orderRepository.save(detachedOrder);  // Calls merge internally
    // Cascade MERGE: merges order AND all items
    // Modified items get UPDATE, new items get INSERT
}
```

CascadeType.REMOVE

Propagates the `remove` (delete) operation — deleting the parent deletes all children:

```java
@Entity
public class Order {
    @OneToMany(mappedBy = "order",
               cascade = {CascadeType.PERSIST, CascadeType.MERGE, CascadeType.REMOVE})
    private List<OrderItem> items = new ArrayList<>();
}
```

```java
// Deleting the order also deletes all items
orderRepository.deleteById(1L);

// SQL generated:
// DELETE FROM order_items WHERE order_id = 1   (children first)
// DELETE FROM orders WHERE id = 1              (then parent)
```

```
CascadeType.REMOVE — Danger Zone:
+-----------------------------------------------------------------+
|                                                                  |
|  SAFE to cascade REMOVE:                                         |
|  - Parent OWNS children completely                               |
|  - Children cannot exist without parent                          |
|  - Deleting parent means children are meaningless                |
|  Examples: Order->Items, Post->Comments, Invoice->Lines          |
|                                                                  |
|  DANGEROUS to cascade REMOVE:                                    |
|  - Children are SHARED between parents                           |
|  - Children have independent lifecycles                          |
|  - Deleting parent should NOT delete children                    |
|  Examples:                                                       |
|  - Department->Employees (employees should transfer, not die)    |
|  - Student->Courses (courses serve other students)               |
|  - User->Roles (roles are shared, must not be deleted)           |
|                                                                  |
|  NEVER use CascadeType.REMOVE on @ManyToMany!                   |
|  It would delete the shared entity, not just the link.           |
+-----------------------------------------------------------------+
```

CascadeType.REFRESH and CascadeType.DETACH

Less commonly used but useful in specific scenarios:

```java
// REFRESH: Reloads entity state from database, discarding in-memory changes
@OneToMany(cascade = CascadeType.REFRESH)
// When you call entityManager.refresh(order), items are also refreshed

// DETACH: Removes entity from persistence context
@OneToMany(cascade = CascadeType.DETACH)
// When you call entityManager.detach(order), items are also detached
```

CascadeType.ALL

Shorthand for all cascade types combined:

```java
@OneToMany(mappedBy = "order", cascade = CascadeType.ALL)
private List<OrderItem> items;

// Equivalent to:
@OneToMany(mappedBy = "order",
    cascade = {CascadeType.PERSIST, CascadeType.MERGE,
               CascadeType.REMOVE, CascadeType.REFRESH,
               CascadeType.DETACH})
private List<OrderItem> items;
```

```
When to Use CascadeType.ALL:
+-------------------------------------------------------------------+
|                                                                   |
|  Use ALL when children are FULLY OWNED by parent:                 |
|  - Children created with parent                                   |
|  - Children updated with parent                                   |
|  - Children deleted with parent                                   |
|  - Children never shared or moved to another parent               |
|                                                                   |
|  Common pattern:                                                  |
|  @OneToMany(mappedBy = "parent",                                  |
|             cascade = CascadeType.ALL,                             |
|             orphanRemoval = true)                                  |
|  --> Complete lifecycle ownership                                  |
|                                                                   |
|  Use {PERSIST, MERGE} when children are NOT fully owned:          |
|  - Children may be moved between parents                          |
|  - Deleting parent should NOT delete children                     |
|  - Children have independent lifecycles                           |
+-------------------------------------------------------------------+
```

---

orphanRemoval vs CascadeType.REMOVE

These are often confused. They handle different scenarios:

```
orphanRemoval vs CascadeType.REMOVE:
+-------------------------------------------------------------------+
|                                                                   |
|  CascadeType.REMOVE:                                              |
|  Triggers when: PARENT is deleted                                 |
|  Effect: Children are deleted along with parent                   |
|                                                                   |
|  orderRepository.delete(order);                                   |
|  --> Deletes order AND all items                                  |
|                                                                   |
|  orphanRemoval = true:                                            |
```

```
|  Triggers when: CHILD is removed from parent's collection         |
|  Effect: Removed child is deleted from database                   |
|                                                                   |
|  order.removeItem(item);                                          |
|  --> item is deleted from database (it is an "orphan")            |
|  --> Order itself is NOT deleted                                  |
|                                                                   |
+-------------------------------------------------------------------+
|                                                                   |
|  Example showing the difference:                                  |
|                                                                   |
|  Without orphanRemoval:                                           |
|  order.getItems().remove(item);                                   |
|  --> item.order_id set to NULL (item still exists in DB)          |
|  --> Orphaned row with no parent!                                 |
|                                                                   |
|  With orphanRemoval = true:                                       |
|  order.getItems().remove(item);                                   |
|  --> DELETE FROM order_items WHERE id = ?                         |
|  --> Item completely removed from database                        |
|                                                                   |
|  orphanRemoval INCLUDES CascadeType.REMOVE behavior.             |
|  If parent is deleted, children are also deleted.                 |
+-------------------------------------------------------------------+
```

```java
@Entity
public class Order {

    @OneToMany(mappedBy = "order",
               cascade = CascadeType.ALL,
               orphanRemoval = true)        // Delete items removed from list
    private List<OrderItem> items = new ArrayList<>();

    public void removeItem(OrderItem item) {
        items.remove(item);         // Triggers orphanRemoval
        item.setOrder(null);        // Clear the relationship
    }

    public void clearItems() {
        items.clear();              // All items become orphans -> all deleted
    }

    public void replaceItems(List<OrderItem> newItems) {
        items.clear();                          // Delete old items (orphans)
        newItems.forEach(this::addItem);        // Add new items (cascade persist)
    }
}
```

JPA Lifecycle Callbacks

JPA provides annotations that let you hook into entity state changes. These methods are called automatically by Hibernate at specific points in the entity lifecycle.

```
Lifecycle Callback Timing:
+-------------------------------------------------------------------+
|                                                                   |
|  persist(entity)                                                  |
|       |                                                           |
|       v                                                           |
|  @PrePersist    --> Before INSERT SQL is generated                |
|       |             Set defaults, validate, generate timestamps   |
|       v                                                           |
|  INSERT SQL executed                                              |
|       |                                                           |
|       v                                                           |
|  @PostPersist   --> After INSERT completes                        |
|                     ID is available, log creation                 |
|                                                                   |
|  modify managed entity                                            |
|       |                                                           |
|       v                                                           |
|  @PreUpdate     --> Before UPDATE SQL is generated                |
|       |             Update timestamps, validate changes           |
|       v                                                           |
|  UPDATE SQL executed                                              |
|       |                                                           |
|       v                                                           |
|  @PostUpdate    --> After UPDATE completes                        |
|                     Log changes, trigger notifications            |
|                                                                   |
|  remove(entity)                                                   |
|       |                                                           |
|       v                                                           |
|  @PreRemove     --> Before DELETE SQL is generated                |
|       |             Cleanup, validate deletion is allowed         |
|       v                                                           |
|  DELETE SQL executed                                              |
|       |                                                           |
|       v                                                           |
|  @PostRemove    --> After DELETE completes                        |
|                     Log deletion, cleanup external resources      |
|                                                                   |
|  @PostLoad      --> After entity is loaded from database          |
|                     Initialize transient fields, compute values   |
+-------------------------------------------------------------------+
```

Using Lifecycle Callbacks on the Entity

```java
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private BigDecimal price;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @Transient  // Not persisted - computed after loading
    private String displayPrice;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }

    @PostLoad
    protected void onLoad() {
        displayPrice = "$" + price.setScale(2).toPlainString();
    }

    @PreRemove
    protected void onRemove() {
        System.out.println("About to delete product: " + name);
    }
}
```

Rules for Lifecycle Callback Methods

```
Lifecycle Callback Rules:
+-------------------------------------------------------------------+
|                                                                   |
|  1. Must be void return type                                      |
|  2. Must take no arguments (when on the entity itself)            |
|  3. Can be any access modifier (public, protected, private)       |
|  4. Should NOT call EntityManager methods                         |
```

```
|     (no persist, merge, remove, or queries inside callbacks)      |
|  5. Should NOT modify other entities                              |
|  6. Should be fast (called synchronously, blocks the operation)   |
|  7. Exceptions thrown will roll back the transaction               |
|                                                                   |
|  Valid uses:                                                       |
|  - Set timestamps                                                  |
|  - Validate data                                                   |
|  - Initialize computed fields                                      |
|  - Log events                                                      |
|                                                                   |
|  Invalid uses (will cause issues):                                |
|  - Load or save other entities                                    |
|  - Call repository methods                                        |
|  - Start new transactions                                         |
+-------------------------------------------------------------------+
```

@EntityListeners — Reusable Lifecycle Logic

Instead of putting lifecycle callbacks on every entity, extract them into a separate listener class:

```java
// Reusable audit listener
public class AuditListener {

    @PrePersist
    public void setCreatedAt(Object entity) {
        if (entity instanceof Auditable auditable) {
            auditable.setCreatedAt(LocalDateTime.now());
            auditable.setUpdatedAt(LocalDateTime.now());
        }
    }

    @PreUpdate
    public void setUpdatedAt(Object entity) {
        if (entity instanceof Auditable auditable) {
            auditable.setUpdatedAt(LocalDateTime.now());
        }
    }
}
```

```java
// Interface for auditable entities
public interface Auditable {
    void setCreatedAt(LocalDateTime createdAt);
```

```
    void setUpdatedAt(LocalDateTime updatedAt);
}
```

```java
// Apply the listener to any entity
@Entity
@Table(name = "products")
@EntityListeners(AuditListener.class)    // JPA standard
public class Product implements Auditable {

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @Override
    public void setCreatedAt(LocalDateTime createdAt) {
        this.createdAt = createdAt;
    }

    @Override
    public void setUpdatedAt(LocalDateTime updatedAt) {
        this.updatedAt = updatedAt;
    }
}

@Entity
@Table(name = "orders")
@EntityListeners(AuditListener.class)    // Same listener, different entity
public class Order implements Auditable {
    // Same audit fields...
}
```

Spring Data JPA Auditing (Easier Approach)

Spring Data JPA provides built-in auditing support that is simpler than manual `@EntityListeners`:

```java
// 1. Enable auditing
@Configuration
@EnableJpaAuditing
public class JpaConfig {
}

// 2. Use Spring's audit annotations
@Entity
@EntityListeners(AuditingEntityListener.class)  // Spring's listener
public class Product {
```

```
    @CreatedDate                        // Spring Data annotation
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate                   // Spring Data annotation
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @CreatedBy                          // Spring Data annotation
    @Column(name = "created_by", updatable = false)
    private String createdBy;

    @LastModifiedBy                     // Spring Data annotation
    @Column(name = "modified_by")
    private String modifiedBy;
}

// 3. Provide the current user (for @CreatedBy / @LastModifiedBy)
@Component
public class AuditorAwareImpl implements AuditorAware<String> {

    @Override
    public Optional<String> getCurrentAuditor() {
        // In a real app, get from SecurityContext:
        // return Optional.of(SecurityContextHolder.getContext()
        //     .getAuthentication().getName());
        return Optional.of("system");
    }
}
```

```
Spring Data Auditing vs Manual Callbacks:
+-------------------------------------------------------------------+
|                                                                   |
|  Manual @PrePersist / @PreUpdate:                                 |
|  - More control                                                   |
|  - Works without Spring                                           |
|  - Must implement in each entity or via listener                  |
|                                                                   |
|  Spring Data @CreatedDate / @LastModifiedDate:                    |
|  - Less code                                                      |
|  - Built-in @CreatedBy / @LastModifiedBy support                  |
|  - Just add annotations + enable auditing                         |
|  - Recommended for Spring Boot applications                      |
+-------------------------------------------------------------------+
```

Practical Example: Audit Trail with Callbacks

```java
// Audit log entity
@Entity
@Table(name = "audit_logs")
public class AuditLog {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "entity_type", nullable = false)
    private String entityType;

    @Column(name = "entity_id", nullable = false)
    private Long entityId;

    @Column(nullable = false)
    private String action;  // CREATED, UPDATED, DELETED

    @Column(name = "performed_at", nullable = false)
    private LocalDateTime performedAt;

    @Column(name = "performed_by")
    private String performedBy;

    public AuditLog() {}

    public AuditLog(String entityType, Long entityId, String action) {
        this.entityType = entityType;
        this.entityId = entityId;
        this.action = action;
        this.performedAt = LocalDateTime.now();
        this.performedBy = "system"; // Replace with actual user
    }

    // Getters...
}
```

```java
// Service that manually creates audit logs
// (Cannot use EntityManager in lifecycle callbacks,
//  so use application events or service-level logging)
@Service
public class ProductService {

    private final ProductRepository productRepository;
    private final AuditLogRepository auditLogRepository;

    @Transactional
    public Product createProduct(String name, BigDecimal price) {
        Product product = productRepository.save(new Product(name, price));

        auditLogRepository.save(
            new AuditLog("Product", product.getId(), "CREATED"));
```

```
        return product;
    }

    @Transactional
    public Product updateProduct(Long id, String name, BigDecimal price) {
        Product product = productRepository.findById(id).orElseThrow();
        product.setName(name);
        product.setPrice(price);

        auditLogRepository.save(
            new AuditLog("Product", product.getId(), "UPDATED"));

        return product;
    }

    @Transactional
    public void deleteProduct(Long id) {
        Product product = productRepository.findById(id).orElseThrow();

        auditLogRepository.save(
            new AuditLog("Product", id, "DELETED"));

        productRepository.delete(product);
    }
}
```

Cascade Decision Framework

```
Complete Cascade Decision Guide:
+-------------------------------------------------------------------+
|                                                                   |
|  Question 1: Does the parent CREATE the children?                 |
|       |                                                           |
|    YES --> Include CascadeType.PERSIST                            |
|    NO  --> Do not include PERSIST                                 |
|            (save children independently)                          |
|                                                                   |
|  Question 2: Are children modified through the parent?            |
|       |                                                           |
|    YES --> Include CascadeType.MERGE                              |
|    NO  --> Do not include MERGE                                   |
|                                                                   |
|  Question 3: Should deleting the parent delete children?          |
|       |                                                           |
|    YES --> Is the child SHARED with other parents?                |
|       |       |                                                   |
|       |    YES --> DO NOT cascade REMOVE! (would delete shared)   |
```

```
|       |    NO  --> Include CascadeType.REMOVE                    |
|       |                                                          |
|    NO  --> Do not include REMOVE                                 |
|                                                                  |
|  Question 4: Should removing a child from the collection         |
|              delete it from the database?                         |
|       |                                                          |
|    YES --> Add orphanRemoval = true                              |
|    NO  --> Leave orphanRemoval = false (default)                 |
|                                                                  |
+------------------------------------------------------------------+
|                                                                  |
|  Common Patterns:                                                |
|                                                                  |
|  Fully owned children (Order->Items):                            |
|    cascade = ALL, orphanRemoval = true                           |
|                                                                  |
|  Parent creates but does not delete children:                    |
|    cascade = {PERSIST, MERGE}                                    |
|                                                                  |
|  Independent entities with reference only:                       |
|    No cascade at all                                             |
|                                                                  |
|  @ManyToMany (shared entities):                                  |
|    cascade = {PERSIST, MERGE} at most                            |
|    NEVER cascade REMOVE                                          |
+------------------------------------------------------------------+
```

Common Mistakes

1. Using CascadeType.ALL on @ManyToMany: This includes REMOVE, which deletes shared entities. A student deletion would delete courses that other students are enrolled in. Use `{PERSIST, MERGE}` or no cascade.
2. Using CascadeType.REMOVE on non-owned children: If employees can transfer departments, `cascade = REMOVE` on `Department.employees` would delete transferred employees when the old department is deleted.
3. Calling EntityManager methods inside lifecycle callbacks: `@PrePersist`, `@PreUpdate`, etc. should not call `entityManager.persist()`, `merge()`, or run queries. This causes unpredictable behavior and potential infinite loops.
4. Confusing orphanRemoval with CascadeType.REMOVE: REMOVE triggers on parent deletion. orphanRemoval triggers on child removal from the collection. They handle different scenarios but orphanRemoval includes REMOVE behavior.
5. Using orphanRemoval with @ManyToMany: Removing a student from a course's student list should not delete the student. orphanRemoval is for owned children only.

6. Forgetting that cascade operates on the relationship, not the entity: Cascade is defined on the `@OneToMany`, `@ManyToOne`, etc. annotation. It affects how operations propagate through that specific relationship.

Best Practices

1. Default cascade for owned children: Use `cascade = CascadeType.ALL, orphanRemoval = true` for children that cannot exist without their parent (OrderItem, Comment, LineItem).
2. Minimal cascade for shared references: Use `cascade = {PERSIST, MERGE}` or no cascade for entities that have independent lifecycles (Employee, Course, Role).
3. Never cascade REMOVE on @ManyToMany: The shared entity would be deleted, affecting all other relationships.
4. Use Spring Data JPA auditing for timestamps: `@CreatedDate` and `@LastModifiedDate` with `@EnableJpaAuditing` is cleaner than manual `@PrePersist`/`@PreUpdate`.
5. Keep lifecycle callbacks simple and fast: Set timestamps, validate data, initialize fields. Do not perform database operations or call external services.
6. Use @MappedSuperclass for common audit fields: Put `createdAt`, `updatedAt`, `createdBy`, `modifiedBy` in a base class so all entities inherit them.

Summary

In this chapter, you learned how to manage operation propagation and entity lifecycle events:

- CascadeType.PERSIST: Saves children when parent is saved. Use for parent-creates-children patterns.
- CascadeType.MERGE: Reattaches children when parent is merged. Use with PERSIST for full write propagation.
- CascadeType.REMOVE: Deletes children when parent is deleted. Only for fully owned children.
- CascadeType.ALL: All cascade types combined. Use with orphanRemoval for complete ownership.
- orphanRemoval: Deletes children removed from the collection (not just when parent is deleted). Includes REMOVE behavior.
- Lifecycle callbacks (`@PrePersist`, `@PostLoad`, etc.) hook into entity state changes for timestamps, validation, and computed fields.

- @EntityListeners extract lifecycle logic into reusable classes.
- Spring Data JPA auditing (`@CreatedDate`, `@LastModifiedDate`, `@EnableJpaAuditing`) provides the simplest approach for audit timestamps and user tracking.

Interview Questions

Q1: What is the difference between CascadeType.REMOVE and orphanRemoval?

`CascadeType.REMOVE` deletes children when the PARENT is deleted. `orphanRemoval` deletes children when they are removed from the PARENT'S COLLECTION (the parent still exists). orphanRemoval also includes REMOVE behavior. Example: `order.removeItem(item)` triggers orphanRemoval (deletes item); `orderRepository.delete(order)` triggers both.

Q2: When should you NOT use CascadeType.ALL?

When children are shared between parents or have independent lifecycles. Examples: Employees (can transfer departments), Courses (shared by students), Roles (shared by users). Using ALL would cascade REMOVE, deleting shared entities when one parent is deleted.

Q3: What are JPA lifecycle callbacks and name three of them?

Lifecycle callbacks are methods annotated with JPA annotations that are called automatically at specific points in an entity's lifecycle. Examples: `@PrePersist` (before INSERT), `@PreUpdate` (before UPDATE), `@PostLoad` (after SELECT), `@PreRemove` (before DELETE), `@PostPersist` (after INSERT).

Q4: What restrictions apply to lifecycle callback methods?

They must return void, take no arguments (when on the entity), should not call EntityManager methods (persist, merge, remove, queries), should not modify other entities, and should be fast. Exceptions thrown inside callbacks will roll back the transaction.

Q5: How does Spring Data JPA auditing work?

Enable with `@EnableJpaAuditing`, add `@EntityListeners(AuditingEntityListener.class)` to entities, and annotate fields with `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, `@LastModifiedBy`. For user tracking, implement `AuditorAware<String>` as a Spring bean to provide the current user.

Q6: What cascade types are safe for @ManyToMany?

Only `PERSIST` and `MERGE` are safe. Never use `REMOVE`, `ALL`, or `orphanRemoval` on `@ManyToMany` because they would delete shared entities, not just the association.

Practice Exercises

Exercise 1: Cascade PERSIST Create `Invoice` and `LineItem` entities. Configure cascade PERSIST. Write a service that creates an invoice with 3 line items in a single save. Verify all 4 rows are created.

Exercise 2: orphanRemoval Add `orphanRemoval = true` to the Invoice-LineItem relationship. Write a test that: (a) creates an invoice with 3 items, (b) removes 1 item from the list, (c) verifies only 2 items remain in the database.

Exercise 3: Dangerous Cascade Create `Department` and `Employee` with `cascade = ALL`. Delete a department with employees. Observe that employees are deleted. Change to `cascade = {PERSIST, MERGE}`. Try to delete the department again (expect FK constraint error). Discuss which behavior is correct.

Exercise 4: Audit with Spring Data Enable Spring Data JPA auditing. Create a `BaseEntity` with `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, `@LastModifiedBy`. Implement `AuditorAware`. Create a Product entity extending BaseEntity. Verify timestamps and user are set automatically.

Exercise 5: Lifecycle Callbacks Create a `Document` entity with `@PrePersist` (generate a UUID slug), `@PreUpdate` (increment a version counter), and `@PostLoad` (compute a word count from content). Write tests verifying each callback fires at the correct time.

What Is Next?

In the next chapter, we will explore Transaction Management — one of the most important topics for production applications. You will learn about ACID properties, Spring's `@Transactional` annotation, propagation levels, isolation levels, read-only optimization, rollback rules, and common transaction pitfalls like self-invocation and checked exceptions.

Chapter 20: Transaction Management

Learning Goals

By the end of this chapter, you will be able to:

- Understand ACID properties and why transactions matter
- Use Spring's @Transactional annotation effectively
- Explain and apply propagation levels (REQUIRED, REQUIRES_NEW, etc.)
- Understand isolation levels and choose the right one
- Use readOnly transactions for performance optimization
- Configure rollback rules for checked and unchecked exceptions
- Avoid common pitfalls: self-invocation, checked exception swallowing
- Use programmatic transactions with TransactionTemplate
- Debug transaction issues in Spring Boot applications

Why Transactions Matter

A transaction groups multiple database operations into a single unit of work — either ALL succeed or ALL fail. Without transactions, a failure midway through an operation leaves your data in an inconsistent state.

```
Without Transactions — Data Corruption:
+-------------------------------------------------------------------+
|                                                                   |
|  Transfer $500 from Account A to Account B:                       |
|                                                                   |
```

```
|  Step 1: UPDATE accounts SET balance = balance - 500              |
|          WHERE id = 'A'                        --> SUCCESS       |
|          Account A: $1000 --> $500                               |
|                                                                  |
|  Step 2: UPDATE accounts SET balance = balance + 500              |
|          WHERE id = 'B'                        --> CRASH!        |
|          Server crashes, database restarts                       |
|                                                                  |
|  Result: $500 disappeared! A lost money, B never received it.    |
|                                                                  |
+------------------------------------------------------------------+

With Transactions - Atomicity:
+------------------------------------------------------------------+
|                                                                  |
|  BEGIN TRANSACTION                                               |
|                                                                  |
|  Step 1: UPDATE accounts SET balance = balance - 500              |
|          WHERE id = 'A'                        --> SUCCESS       |
|                                                                  |
|  Step 2: UPDATE accounts SET balance = balance + 500              |
|          WHERE id = 'B'                        --> CRASH!        |
|                                                                  |
|  ROLLBACK (automatic on crash)                                   |
|  Account A: Still $1000 (change was rolled back)                 |
|  Account B: Still $500 (never changed)                           |
|                                                                  |
|  Result: Data is consistent. No money lost.                      |
+------------------------------------------------------------------+
```

ACID Properties

```
ACID Properties:
+------------------------------------------------------------------+
|                                                                  |
|  A - Atomicity                                                   |
|  "All or nothing"                                                |
|  Either all operations in the transaction succeed,               |
|  or none of them do. Partial changes are rolled back.            |
|                                                                  |
|  C - Consistency                                                 |
|  "Rules are always followed"                                     |
|  The database moves from one valid state to another.             |
|  Constraints (FK, unique, check) are never violated.             |
|                                                                  |
|  I - Isolation                                                   |
|  "Transactions don't interfere"                                  |
|  Concurrent transactions see a consistent view of data.          |
```

```
|  One transaction's uncommitted changes are invisible to others.   |
|                                                                   |
|  D - Durability                                                   |
|  "Committed changes survive crashes"                               |
|  Once a transaction is committed, the data is permanently saved.  |
|  Even if the server crashes immediately after commit.             |
+-------------------------------------------------------------------+
```

Spring's @Transactional

In Spring Boot, you manage transactions declaratively with `@Transactional`:

```java
@Service
public class TransferService {

    private final AccountRepository accountRepository;

    @Transactional    // Everything in this method runs in ONE transaction
    public void transfer(Long fromId, Long toId, BigDecimal amount) {
        Account from = accountRepository.findById(fromId).orElseThrow();
        Account to = accountRepository.findById(toId).orElseThrow();

        if (from.getBalance().compareTo(amount) < 0) {
            throw new InsufficientFundsException("Not enough balance");
        }

        from.setBalance(from.getBalance().subtract(amount));
        to.setBalance(to.getBalance().add(amount));

        // No explicit save needed - dirty checking handles it
        // If any exception is thrown, everything rolls back
    }
}
```

```
How @Transactional Works:
+-------------------------------------------------------------------+
|                                                                   |
|  Spring creates a PROXY around your service class:                |
|                                                                   |
|  Caller --> [Proxy] --> [Your Service Method]                     |
|                |                                                  |
|                |  1. Begin transaction                            |
|                |  2. Open EntityManager / persistence context     |
|                |  3. Call your method                              |
|                |  4a. Method returns normally:                     |
|                |      --> Flush (dirty checking, SQL sent)        |
```

```
|                |      --> Commit transaction                      |
|                |  4b. Method throws RuntimeException:             |
|                |      --> Rollback transaction                    |
|                |  5. Close EntityManager                          |
|                                                                   |
|  The proxy intercepts the method call and wraps it in a          |
|  transaction. Your code does not manage transactions explicitly.  |
+-------------------------------------------------------------------+
```

@Transactional Attributes

```java
@Transactional(
    readOnly = false,                    // Default: false
    propagation = Propagation.REQUIRED,  // Default: REQUIRED
    isolation = Isolation.DEFAULT,       // Default: database default
    timeout = -1,                        // Default: no timeout (-1)
    rollbackFor = {},                    // Additional exception types to rollback
    noRollbackFor = {}                   // Exception types to NOT rollback
)
```

Propagation Levels

Propagation defines what happens when a transactional method calls another transactional method.

```
Propagation Levels:
+-------------------------------------------------------------------+
|                                                                   |
|  REQUIRED (default):                                              |
|  "Join existing or create new"                                    |
|  If a transaction exists, join it. If not, create a new one.     |
|                                                                   |
|  Caller                    Service A              Service B       |
|    |                          |                      |            |
|    |  call A.methodA()        |                      |            |
|    |------------------------->|                      |            |
|    |  [Tx1 starts]            |                      |            |
|    |                          |  call B.methodB()    |            |
|    |                          |--------------------->|            |
|    |                          |  [Joins Tx1]         |            |
|    |                          |  (same transaction!) |            |
|    |                          |<---------------------|            |
|    |  [Tx1 commits]           |                      |            |
```

```
|    |<-------------------------|                     |            |
|                                                                    |
+--------------------------------------------------------------------+

+--------------------------------------------------------------------+
|                                                                    |
|  REQUIRES_NEW:                                                     |
|  "Always create a new transaction"                                 |
|  Suspends the current transaction and starts a new one.            |
|  The new transaction commits/rollbacks independently.              |
|                                                                    |
|  Caller                    Service A              Service B        |
|    |                          |                      |            |
|    |  call A.methodA()        |                      |            |
|    |-------------------------->|                      |            |
|    |  [Tx1 starts]            |                      |            |
|    |                          |  call B.methodB()    |            |
|    |                          |--------------------->|            |
|    |                          |  [Tx1 suspended]     |            |
|    |                          |  [Tx2 starts]        |            |
|    |                          |  [Tx2 commits]       |            |
|    |                          |<---------------------|            |
|    |                          |  [Tx1 resumes]       |            |
|    |  [Tx1 commits]           |                      |            |
|    |<-------------------------|                      |            |
|                                                                    |
|  Use case: Audit logging that must persist even if the             |
|  main transaction rolls back.                                      |
+--------------------------------------------------------------------+
```

```
All Propagation Levels:
+--------------------------------------------------------------------+
|                                                                    |
|  Level            Existing Tx?  No Existing Tx?  Use When          |
|  ------------------------------------------------------------------|
|  REQUIRED         Join it       Create new       Default, most     |
|  (default)                                       cases             |
|                                                                    |
|  REQUIRES_NEW     Suspend,      Create new       Independent ops   |
|                   create new                     (audit logs)      |
|                                                                    |
|  SUPPORTS         Join it       Run without      Read-only         |
|                                 transaction      queries           |
|                                                                    |
|  NOT_SUPPORTED    Suspend it    Run without      External API      |
|                                 transaction      calls             |
|                                                                    |
|  MANDATORY        Join it       THROW            Must be called    |
|                                 exception!       within Tx         |
|                                                                    |
|  NEVER            THROW         Run without      Must NOT be       |
|                   exception!    transaction      called in Tx      |
|                                                                    |
```

```
|  NESTED           Create        Create new       Savepoints       |
|                   savepoint                      (partial rollback)|
+-------------------------------------------------------------------+
```

REQUIRES_NEW Example: Audit Logging

```java
@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final AuditService auditService;

    @Transactional
    public void cancelOrder(Long orderId) {
        Order order = orderRepository.findById(orderId).orElseThrow();
        order.setStatus("CANCELLED");

        // This runs in a SEPARATE transaction
        // Even if cancelOrder() rolls back, the audit log persists
        auditService.log("ORDER_CANCELLED", orderId);

        // If this throws, the order cancellation rolls back
        // but the audit log is already committed
        processRefund(order);
    }
}

@Service
public class AuditService {

    private final AuditLogRepository auditLogRepository;

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void log(String action, Long entityId) {
        auditLogRepository.save(new AuditLog(action, entityId));
        // Commits independently of the caller's transaction
    }
}
```

Isolation Levels

Isolation controls how concurrent transactions interact and what data they can see.

```
Concurrency Problems:
+-------------------------------------------------------------------+
|                                                                   |
|  Dirty Read:                                                      |
|  Tx1 writes data, Tx2 reads it BEFORE Tx1 commits.              |
|  If Tx1 rolls back, Tx2 has read data that never existed.        |
|                                                                   |
|  Tx1: UPDATE salary = 100000 ... (not committed yet)             |
|  Tx2: SELECT salary --> reads 100000 (dirty!)                    |
|  Tx1: ROLLBACK                                                    |
|  Tx2 now has stale data that was never actually committed.       |
|                                                                   |
|  Non-Repeatable Read:                                             |
|  Tx2 reads data twice and gets different results because          |
|  Tx1 committed a change between the two reads.                   |
|                                                                   |
|  Tx2: SELECT salary --> 80000                                     |
|  Tx1: UPDATE salary = 100000 + COMMIT                             |
|  Tx2: SELECT salary --> 100000 (different from first read!)      |
|                                                                   |
|  Phantom Read:                                                    |
|  Tx2 runs a range query twice and gets different rows because    |
|  Tx1 inserted or deleted rows between the two queries.           |
|                                                                   |
|  Tx2: SELECT * WHERE dept='Eng' --> 10 rows                      |
|  Tx1: INSERT new employee in Eng + COMMIT                         |
|  Tx2: SELECT * WHERE dept='Eng' --> 11 rows (phantom row!)      |
+-------------------------------------------------------------------+
```

```
Isolation Levels:
+-------------------------------------------------------------------+
|                                                                   |
|  Level               Dirty    Non-Repeatable  Phantom             |
|                      Read     Read            Read                |
|  ----------------------------------------------------------------|
|  READ_UNCOMMITTED    Possible Possible        Possible            |
|                      (worst isolation, best performance)          |
|                                                                   |
|  READ_COMMITTED      Prevented Possible       Possible            |
|                      (most common default)                        |
|                                                                   |
|  REPEATABLE_READ     Prevented Prevented      Possible            |
|                      (good isolation)                             |
|                                                                   |
|  SERIALIZABLE        Prevented Prevented      Prevented           |
|                      (best isolation, worst performance)          |
|                                                                   |
|  DEFAULT             Uses the database's default level            |
|                      (usually READ_COMMITTED)                     |
+-------------------------------------------------------------------+
```

```java
// Set isolation on a specific method
@Transactional(isolation = Isolation.REPEATABLE_READ)
public BigDecimal calculateBalance(Long accountId) {
    // Multiple reads guaranteed to see the same data
    Account account = accountRepository.findById(accountId).orElseThrow();
    List<Transaction> txns = transactionRepository.findByAccountId(accountId);
    return computeBalance(account, txns);
}
```

```
Choosing an Isolation Level:
+-------------------------------------------------------------------+
|                                                                   |
|  DEFAULT (READ_COMMITTED) — use for most operations               |
|  Prevents dirty reads. Good enough for most CRUD.                 |
|                                                                   |
|  REPEATABLE_READ — use for financial calculations                 |
|  When you read data multiple times and need consistency.          |
|  Example: Balance calculation, report generation.                 |
|                                                                   |
|  SERIALIZABLE — use rarely, for critical operations               |
|  Maximum safety but worst performance (transactions queue up).    |
|  Example: Seat reservation, inventory deduction.                  |
|                                                                   |
|  READ_UNCOMMITTED — almost never use                              |
|  Only for approximate analytics where dirty reads are acceptable. |
+-------------------------------------------------------------------+
```

Read-Only Transactions

```java
@Transactional(readOnly = true)
public List<Employee> getAllEmployees() {
    return employeeRepository.findAll();
}
```

```
readOnly = true Optimizations:
+-------------------------------------------------------------------+
|                                                                   |
|  What readOnly = true does:                                       |
|                                                                   |
|  1. Hibernate skips dirty checking                                |
|     - No snapshot taken on entity load                            |
|     - No field-by-field comparison on flush                       |
|     - Saves CPU and memory                                        |
|                                                                   |
```

```
|  2. Hibernate sets flush mode to MANUAL                          |
|     - No automatic flushing before queries                       |
|     - No UPDATE/INSERT/DELETE SQL generated                      |
|                                                                  |
|  3. Database hint (some drivers)                                 |
|     - Some JDBC drivers optimize for read-only connections       |
|     - May route to read replicas in master-slave setups          |
|                                                                  |
|  Use readOnly = true for:                                        |
|  - All GET/read endpoints                                        |
|  - Report generation                                             |
|  - Search and listing operations                                 |
|  - Any method that does not modify data                          |
|                                                                  |
|  WARNING: If you modify an entity in a readOnly transaction,     |
|  the change is SILENTLY IGNORED (not an error, just not saved).  |
+------------------------------------------------------------------+
```

Rollback Rules

By default, Spring rolls back on unchecked exceptions (RuntimeException and its subclasses) but commits on checked exceptions:

```
Default Rollback Behavior:
+------------------------------------------------------------------+
|                                                                  |
|  Exception Type              Default Behavior                    |
|  ---------------------------------------------------------------|
|  RuntimeException             ROLLBACK                           |
|  (NullPointerException,       (transaction is aborted)           |
|   IllegalArgumentException,                                      |
|   DataIntegrityViolation...)                                     |
|                                                                  |
|  Error                        ROLLBACK                           |
|  (OutOfMemoryError, etc.)                                        |
|                                                                  |
|  Checked Exception            COMMIT!                            |
|  (IOException, SQLException,  (transaction is committed!)        |
|   custom checked exceptions)  This surprises many developers.    |
+------------------------------------------------------------------+
```

Customizing Rollback

```java
// Rollback on a specific checked exception
@Transactional(rollbackFor = BusinessException.class)
public void processOrder(Order order) throws BusinessException {
    // If BusinessException is thrown, transaction rolls back
    // (normally checked exceptions would commit)
}

// Rollback on all exceptions
@Transactional(rollbackFor = Exception.class)
public void criticalOperation() throws Exception {
    // Any exception causes rollback
}

// Do NOT rollback on a specific runtime exception
@Transactional(noRollbackFor = ItemNotFoundException.class)
public void processItems(List<Long> ids) {
    for (Long id : ids) {
        try {
            processItem(id);
        } catch (ItemNotFoundException e) {
            // Log and continue - transaction should NOT rollback
            log.warn("Item {} not found, skipping", id);
        }
    }
}
```

Common Pitfall: Self-Invocation

The most common `@Transactional` bug — calling a transactional method from within the same class bypasses the proxy:

```
Self-Invocation Problem:
+-------------------------------------------------------------------+
|                                                                   |
|  @Service                                                         |
|  public class OrderService {                                      |
|                                                                   |
|      public void processOrders(List<Long> ids) {  // No @Tx      |
|          for (Long id : ids) {                                    |
|              processOrder(id);  // Calls method below             |
|          }                                                        |
|      }                                                            |
|                                                                   |
|      @Transactional                                               |
|      public void processOrder(Long id) {  // Has @Tx             |
|          // But @Transactional is IGNORED!                        |
|          // Because this call bypasses the proxy.                 |
```

```
|      }                                                              |
|  }                                                                  |
|                                                                     |
|  Why? Spring's proxy intercepts calls FROM OUTSIDE the class.    |
|  Internal calls (this.processOrder) go directly to the method,   |
|  skipping the proxy that manages the transaction.                  |
|                                                                     |
|  External call:  Caller --> [Proxy] --> processOrder  (Tx works)  |
|  Self-invocation: processOrders --> processOrder     (Tx skipped!)|
|                                                                     |
+---------------------------------------------------------------------+
```

Fixes for Self-Invocation

```
Fix 1: Move the method to a separate service (BEST):
+---------------------------------------------------------------------+
|  @Service                                                           |
|  public class OrderBatchService {                                   |
|      private final OrderService orderService;  // Inject            |
|                                                                     |
|      public void processOrders(List<Long> ids) {                    |
|          for (Long id : ids) {                                      |
|              orderService.processOrder(id);  // External call!      |
|          }                      // Goes through proxy -- Tx works   |
|      }                                                              |
|  }                                                                  |
|                                                                     |
|  @Service                                                           |
|  public class OrderService {                                        |
|      @Transactional                                                 |
|      public void processOrder(Long id) { ... }                     |
|  }                                                                  |
+---------------------------------------------------------------------+

Fix 2: Put @Transactional on the calling method:
+---------------------------------------------------------------------+
|  @Transactional  // Wrap the entire loop in one transaction         |
|  public void processOrders(List<Long> ids) {                        |
|      for (Long id : ids) {                                          |
|          processOrder(id);  // Internal call, but we're already     |
|      }                      // inside a transaction                 |
|  }                                                                  |
+---------------------------------------------------------------------+
```

Programmatic Transactions

For cases where `@Transactional` is not flexible enough, use `TransactionTemplate`:

```java
@Service
public class BatchService {

    private final TransactionTemplate transactionTemplate;
    private final EmployeeRepository employeeRepository;

    public BatchService(PlatformTransactionManager txManager,
                        EmployeeRepository employeeRepository) {
        this.transactionTemplate = new TransactionTemplate(txManager);
        this.employeeRepository = employeeRepository;
    }

    public void processBatch(List<EmployeeData> dataList) {
        for (int i = 0; i < dataList.size(); i += 50) {
            int end = Math.min(i + 50, dataList.size());
            List<EmployeeData> batch = dataList.subList(i, end);

            // Each batch in its own transaction
            transactionTemplate.execute(status -> {
                for (EmployeeData data : batch) {
                    employeeRepository.save(new Employee(data.getName()));
                }
                return null;
            });
            // Transaction committed. If next batch fails, previous batches are safe.
        }
    }

    public Employee findOrCreate(String email) {
        return transactionTemplate.execute(status -> {
            return employeeRepository.findByEmail(email)
                .orElseGet(() -> employeeRepository.save(new Employee(email)));
        });
    }
}
```

```
@Transactional vs TransactionTemplate:
+-------------------------------------------------------------------+
|                                                                   |
|  @Transactional (declarative):                                    |
|  + Clean, simple, less code                                       |
|  + Covers 95% of use cases                                       |
|  - Entire method is one transaction                               |
|  - Self-invocation issues                                         |
|                                                                   |
|  TransactionTemplate (programmatic):                              |
|  + Fine-grained control                                           |
|  + Multiple transactions in one method                            |
|  + No self-invocation issues                                      |
|  - More verbose code                                              |
|  - Must manage template manually                                  |
```

```
|                                                                  |
|  Use @Transactional for most cases.                              |
|  Use TransactionTemplate for batch processing where you need     |
|  multiple smaller transactions within one method.                |
+------------------------------------------------------------------+
```

Transaction Best Practices Summary

```
Transaction Best Practices:
+------------------------------------------------------------------+
|                                                                  |
|  1. Put @Transactional on service methods, not repositories      |
|     Repositories already run in a transaction per method.        |
|     Services define the business transaction boundary.           |
|                                                                  |
|  2. Use readOnly = true for all read operations                  |
|     Saves CPU (no dirty checking) and memory (no snapshots).     |
|                                                                  |
|  3. Keep transactions short                                      |
|     Do not call external APIs, send emails, or do file I/O       |
|     inside a transaction. These hold DB connections and locks.   |
|                                                                  |
|  4. Do not swallow exceptions silently                           |
|     Catching an exception inside @Transactional prevents         |
|     rollback. Rethrow or use noRollbackFor explicitly.           |
|                                                                  |
|  5. Beware of self-invocation                                    |
|     Calling @Transactional methods from the same class           |
|     bypasses the proxy. Split into separate services.            |
|                                                                  |
|  6. Use rollbackFor for checked exceptions                       |
|     @Transactional(rollbackFor = Exception.class) if your        |
|     methods throw checked exceptions that should rollback.       |
|                                                                  |
|  7. Default isolation is usually fine                            |
|     READ_COMMITTED covers most cases. Use REPEATABLE_READ        |
|     only for financial calculations or multi-read consistency.   |
+------------------------------------------------------------------+
```

Common Mistakes

1. Self-invocation bypasses @Transactional: Calling a `@Transactional` method from within the same class goes directly to the method, not through the proxy. The transaction annotation is ignored.
2. Checked exceptions do not trigger rollback: By default, only `RuntimeException` and `Error` cause rollback. A method throwing `IOException` will COMMIT the transaction. Use `rollbackFor` to change this.
3. Long transactions holding connections: Calling external APIs, sending emails, or performing file I/O inside a transaction holds the database connection for the entire duration. Move non-DB work outside the transaction.
4. Catching exceptions and preventing rollback: Wrapping code in try-catch inside `@Transactional` prevents the exception from reaching the proxy, so the transaction commits with partial changes.
5. Using @Transactional on private methods: Spring proxies cannot intercept private methods. `@Transactional` on a private method is silently ignored.
6. Modifying data in readOnly transactions: Changes to entities in a `readOnly = true` transaction are silently discarded. No error is thrown, but the data is not saved.

Summary

- Transactions ensure atomicity — all operations succeed or all fail. ACID properties (Atomicity, Consistency, Isolation, Durability) guarantee data integrity.
- @Transactional is Spring's declarative transaction management. It creates a proxy that begins a transaction before the method and commits/rollbacks after.
- Propagation controls behavior when transactional methods call each other. `REQUIRED` (default) joins existing transactions. `REQUIRES_NEW` creates independent transactions.
- Isolation levels control concurrent transaction visibility. `READ_COMMITTED` (default) prevents dirty reads. Higher levels provide more safety at the cost of performance.
- readOnly = true skips dirty checking and snapshots, improving performance for read operations.
- Rollback rules: Unchecked exceptions rollback, checked exceptions commit by default. Use `rollbackFor` to customize.
- Self-invocation bypasses the proxy. Split transactional methods into separate services.
- TransactionTemplate provides programmatic control for batch processing with multiple transactions.

Interview Questions

Q1: What are the ACID properties?

Atomicity (all or nothing), Consistency (valid state to valid state), Isolation (concurrent transactions don't interfere), Durability (committed data survives crashes).

Q2: What is the default propagation level and what does it do?

`REQUIRED` (default). If a transaction already exists, the method joins it. If no transaction exists, a new one is created. This means nested service calls share the same transaction.

Q3: When would you use REQUIRES_NEW propagation?

When an operation must commit independently of the calling transaction. Common use case: audit logging that must persist even if the main business operation rolls back.

Q4: Why do checked exceptions not trigger rollback by default?

Spring follows the EJB convention where checked exceptions indicate recoverable business conditions (order not found, insufficient funds) that the caller can handle. Unchecked exceptions indicate programming errors that should abort the transaction. Use `rollbackFor` to change this behavior.

Q5: What is the self-invocation problem with @Transactional?

When a method within a class calls another `@Transactional` method in the same class, the call goes directly to the method (bypassing the Spring proxy). The `@Transactional` annotation is ignored because there is no proxy to intercept the call. Fix by moving the transactional method to a separate service bean.

Q6: What does readOnly = true do?

It tells Hibernate to skip dirty checking (no snapshot comparison) and set flush mode to MANUAL (no SQL updates generated). This saves CPU and memory. Some JDBC drivers may also route to read replicas. Changes made to entities are silently discarded.

Practice Exercises

Exercise 1: Basic Transaction Create a transfer service that moves money between two accounts. Test: (a) successful transfer, (b) rollback on insufficient funds (throw RuntimeException), (c) verify both accounts unchanged after rollback.

Exercise 2: Propagation Create `OrderService.placeOrder()` (REQUIRED) that calls `AuditService.logAction()` (REQUIRES_NEW). Throw an exception in `placeOrder` after calling `logAction`. Verify the order is rolled back but the audit log is committed.

Exercise 3: Self-Invocation Create a service with `processAll()` calling `@Transactional processOne()`. Observe that `processOne`'s transaction is ignored. Fix by extracting to a separate service. Verify with SQL logging.

Exercise 4: Rollback Rules Create a method that throws a custom checked exception. Observe it commits. Add `rollbackFor`. Verify it now rolls back.

Exercise 5: Read-Only Optimization Enable Hibernate statistics. Run a query with and without `readOnly = true`. Compare the dirty checking overhead in statistics output.

What Is Next?

In the next chapter, we will explore Second-Level Cache and Query Cache — how to cache entity data across transactions for improved performance. You will learn the difference between the first-level cache (persistence context) and the second-level cache, how to configure cache providers, and when caching helps vs when it hurts.

Chapter 21: Second-Level Cache and Query Cache

Learning Goals

By the end of this chapter, you will be able to:

- Understand the difference between first-level and second-level cache
- Know when second-level caching improves performance
- Configure a second-level cache provider (Caffeine/Ehcache) with Spring Boot
- Mark entities as cacheable with @Cacheable and @Cache
- Choose the right cache concurrency strategy (READ_ONLY, READ_WRITE, etc.)
- Enable and use the query cache for repeated queries
- Monitor cache hit rates and performance
- Know when caching hurts more than it helps
- Invalidate and manage cache entries

First-Level Cache Recap

The first-level cache is the persistence context you learned about in Chapter 17. It exists within a single transaction.

```
First-Level Cache (Persistence Context):
+-------------------------------------------------------------------+
|                                                                   |
|  Scope: One transaction (one @Transactional method)               |
|                                                                   |
|  Transaction A:                    Transaction B:                  |
```

```
|  +----------------------+         +----------------------+        |
|  | find(Employee, 1)    |         | find(Employee, 1)    |        |
|  | --> SQL SELECT       |         | --> SQL SELECT       |        |
|  | find(Employee, 1)    |         | (separate query!)    |        |
|  | --> FROM CACHE       |         |                      |        |
|  | (no SQL, same Tx)    |         |                      |        |
|  +----------------------+         +----------------------+        |
|                                                                   |
|  Limitation: Cache is NOT shared between transactions.            |
|  Every new transaction starts with an empty cache.                |
|  find(Employee, 1) in Tx B hits the DB again.                    |
+-------------------------------------------------------------------+
```

Second-Level Cache (L2 Cache)

The second-level cache is a shared cache that lives across transactions. When an entity is loaded, it is stored in the L2 cache. Future transactions can retrieve it without hitting the database.

```
L2 Cache Architecture:
+-------------------------------------------------------------------+
|                                                                   |
|  Transaction A           Transaction B           Transaction C    |
|  +-----------+           +-----------+           +-----------+    |
|  | L1 Cache  |           | L1 Cache  |           | L1 Cache  |    |
|  |(per-Tx)   |           |(per-Tx)   |           |(per-Tx)   |    |
|  +-----------+           +-----------+           +-----------+    |
|       |                       |                       |           |
|       v                       v                       v           |
|  +-----------------------------------------------------------+   |
|  |              Second-Level Cache (L2)                       |   |
|  |              Shared across all transactions                |   |
|  |                                                            |   |
|  |  (Employee, 1) -> {name="Alice", salary=80000}             |   |
|  |  (Employee, 2) -> {name="Bob", salary=75000}               |   |
|  |  (Department, 10) -> {name="Engineering"}                  |   |
|  +-----------------------------------------------------------+   |
|       |                                                           |
|       v  (only on cache MISS)                                     |
|  +-----------------------------------------------------------+   |
|  |                     Database                               |   |
|  +-----------------------------------------------------------+   |
|                                                                   |
|  Flow for find(Employee, 1):                                      |
|  1. Check L1 (persistence context) --> miss                      |
|  2. Check L2 (shared cache) --> HIT! Return cached data          |
|  3. If L2 miss --> query database, store in L1 AND L2            |
```

```
+-----------------------------------------------------------------+
```

What Gets Cached

```
L2 Cache Stores DEHYDRATED Data:
+-----------------------------------------------------------------+
|                                                                  |
|  The L2 cache does NOT store Java objects.                       |
|  It stores disassembled (dehydrated) state:                     |
|                                                                  |
|  Cache Key: (com.example.Employee, 1)                            |
|  Cache Value: {                                                  |
|    "name" -> "Alice",                                            |
|    "email" -> "alice@co.com",                                    |
|    "salary" -> 80000,                                            |
|    "department_id" -> 10    // FK value, not the Department object |
|  }                                                               |
|                                                                  |
|  When retrieved from L2:                                         |
|  1. Hibernate reads the dehydrated data                          |
|  2. Creates a new Employee object                                |
|  3. Populates the fields                                         |
|  4. Puts it in the L1 cache (persistence context)               |
|  5. Returns the entity                                           |
|                                                                  |
|  Note: The department (FK) is not automatically loaded.          |
|  It may trigger another cache lookup or DB query.                |
+-----------------------------------------------------------------+
```

Setting Up L2 Cache with Spring Boot

Step 1: Add Dependencies

```xml
<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-cache</artifactId>
</dependency>

<!-- Caffeine (recommended for single-server apps) -->
<dependency>
    <groupId>com.github.ben-manes.caffeine</groupId>
```

```
    <artifactId>caffeine</artifactId>
</dependency>

<!-- Hibernate JCache integration -->
<dependency>
    <groupId>org.hibernate.orm</groupId>
    <artifactId>hibernate-jcache</artifactId>
</dependency>
```

Step 2: Configure Hibernate Properties

```properties
# application.properties

# Enable L2 cache
spring.jpa.properties.hibernate.cache.use_second_level_cache=true

# Enable query cache (optional)
spring.jpa.properties.hibernate.cache.use_query_cache=true

# Cache provider
spring.jpa.properties.hibernate.cache.region.factory_class=jcache

# Cache statistics (for monitoring)
spring.jpa.properties.hibernate.generate_statistics=true

# JCache provider
spring.cache.jcache.config=classpath:ehcache.xml
```

Step 3: Mark Entities as Cacheable

```java
@Entity
@Table(name = "departments")
@Cacheable                                          // JPA standard
@org.hibernate.annotations.Cache(
    usage = CacheConcurrencyStrategy.READ_WRITE     // Hibernate-specific
)
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    @OneToMany(mappedBy = "department")
```

```
    @org.hibernate.annotations.Cache(
        usage = CacheConcurrencyStrategy.READ_WRITE  // Cache the collection too
    )
    private List<Employee> employees = new ArrayList<>();

    // ...
}
```

```
Caching Annotations:
+-------------------------------------------------------------------+
|                                                                   |
|  @Cacheable (JPA standard, jakarta.persistence)                   |
|  - Marks entity as eligible for L2 caching                        |
|  - Simple on/off flag                                             |
|  - No cache strategy control                                      |
|                                                                   |
|  @Cache (Hibernate-specific, org.hibernate.annotations)           |
|  - Defines the caching STRATEGY                                   |
|  - Controls concurrency behavior                                  |
|  - Can be applied to entities AND collections                     |
|                                                                   |
|  You typically use BOTH:                                          |
|  @Cacheable           --> "This entity can be cached"             |
|  @Cache(usage = ...)  --> "Cache it with this strategy"           |
+-------------------------------------------------------------------+
```

Cache Concurrency Strategies

The strategy controls how the cache handles concurrent reads and writes:

```
Cache Concurrency Strategies:
+-------------------------------------------------------------------+
|                                                                   |
|  READ_ONLY:                                                       |
|  - Entity is never modified after creation                        |
|  - Fastest (no locking, no invalidation)                          |
|  - Throws exception if you try to update a cached entity          |
|  - Use for: lookup tables, enums, static reference data           |
|  Example: Country, Currency, Status codes                         |
|                                                                   |
|  NONSTRICT_READ_WRITE:                                            |
|  - Entity is rarely modified                                      |
|  - No locking — brief window of stale data after update           |
|  - Good performance, slightly inconsistent during writes          |
|  - Use for: data that changes rarely, staleness is acceptable     |
|  Example: Product catalog, user profiles                          |
```

```
|                                                                    |
|  READ_WRITE:                                                       |
|  - Entity is read and written frequently                           |
|  - Uses soft locks to prevent stale reads during updates           |
|  - Good balance of consistency and performance                     |
|  - Use for: most business entities                                 |
|  Example: Orders, accounts, inventory                              |
|                                                                    |
|  TRANSACTIONAL:                                                    |
|  - Full transaction isolation in the cache                         |
|  - Requires a JTA transaction manager                              |
|  - Highest consistency, lowest performance                         |
|  - Rarely needed in Spring Boot applications                      |
+--------------------------------------------------------------------+
```

```
Choosing a Strategy:
+--------------------------------------------------------------------+
|                                                                    |
|  Is the data immutable (never updated)?                            |
|      |                                                             |
|   YES --> READ_ONLY (fastest, safest)                              |
|      |                                                             |
|    NO                                                              |
|      |                                                             |
|  Is brief staleness acceptable?                                    |
|  (user sees old data for a few ms during updates)                  |
|      |                                                             |
|   YES --> NONSTRICT_READ_WRITE (fast, simple)                      |
|      |                                                             |
|    NO --> READ_WRITE (consistent, slightly slower)                 |
+--------------------------------------------------------------------+
```

Practical Examples

Read-Only Cache: Country Lookup

```java
@Entity
@Table(name = "countries")
@Cacheable
@org.hibernate.annotations.Cache(usage = CacheConcurrencyStrategy.READ_ONLY)
public class Country {

    @Id
    private String code;   // "US", "GB", "FR"
```

```
    @Column(nullable = false)
    private String name;   // "United States", "United Kingdom"

    // Constructor, getters (no setters — immutable)
}
```

```java
// First call: database query, stored in L2 cache
Country us = countryRepository.findById("US").orElseThrow();
// SQL: SELECT * FROM countries WHERE code = 'US'

// Second call (same or different transaction): L2 cache hit!
Country usAgain = countryRepository.findById("US").orElseThrow();
// No SQL! Served from L2 cache.

// 1000th call: still from cache, zero DB queries
```

Read-Write Cache: Product

```java
@Entity
@Table(name = "products")
@Cacheable
@org.hibernate.annotations.Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false)
    private BigDecimal price;

    private boolean active;

    // Getters, setters...
}
```

```
Read-Write Cache Flow:
+-------------------------------------------------------------------+
|                                                                    |
|  Read:                                                             |
|  find(Product, 1) --> L1 miss --> L2 HIT --> return from cache   |
|                       (no database query)                          |
|                                                                    |
|  Write:                                                            |
```

```
|  product.setPrice(newPrice);                                        |
|  On flush/commit:                                                   |
|  1. UPDATE products SET price = ? WHERE id = 1  (DB updated)      |
|  2. L2 cache entry for (Product, 1) is invalidated                |
|  3. Next read will query DB and repopulate cache                    |
|                                                                     |
|  Soft Lock during update:                                           |
|  1. Before UPDATE: L2 entry is "soft-locked"                      |
|  2. Other transactions reading (Product, 1) see the lock          |
|     and go directly to the database (avoid stale read)            |
|  3. After COMMIT: lock released, cache updated                      |
+---------------------------------------------------------------------+
```

Caching Collections

```java
@Entity
@Cacheable
@org.hibernate.annotations.Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Department {

    @OneToMany(mappedBy = "department")
    @org.hibernate.annotations.Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
    private List<Employee> employees = new ArrayList<>();
}
```

```
Collection Cache:
+---------------------------------------------------------------------+
|                                                                     |
|  The collection cache stores the LIST OF IDs, not entity data:    |
|                                                                     |
|  Cache Region: "com.example.Department.employees"                   |
|  Cache Key: Department id = 10                                      |
|  Cache Value: [1, 2, 3, 5, 8]  (employee IDs only)               |
|                                                                     |
|  When loading department.getEmployees():                            |
|  1. Look up collection cache for Department 10 --> [1,2,3,5,8]    |
|  2. For each ID, look up entity cache for Employee                  |
|     Employee 1 --> L2 HIT (cached entity data)                    |
|     Employee 2 --> L2 HIT                                         |
|     Employee 3 --> L2 MISS --> DB query, then cache it              |
|     ...                                                             |
|                                                                     |
|  Collection cache + entity cache work TOGETHER.                     |
|  If employees are also @Cacheable, most lookups are cache hits.   |
+---------------------------------------------------------------------+
```

Query Cache

The query cache stores query results (lists of entity IDs) keyed by the query string and parameters:

```properties
# Enable query cache
spring.jpa.properties.hibernate.cache.use_query_cache=true
```

```java
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    // Enable query cache for this specific query
    @QueryHints(@QueryHint(name = "org.hibernate.cacheable", value = "true"))
    List<Product> findByActiveTrue();

    @QueryHints(@QueryHint(name = "org.hibernate.cacheable", value = "true"))
    @Query("SELECT p FROM Product p WHERE p.price < :max AND p.active = true")
    List<Product> findAffordableProducts(@Param("max") BigDecimal max);
}
```

```
Query Cache Behavior:
+-----------------------------------------------------------------+
|                                                                 |
|  First call: findByActiveTrue()                                 |
|  1. Query cache MISS (never executed before)                    |
|  2. Execute SQL: SELECT * FROM products WHERE active = true     |
|  3. Store in query cache:                                       |
|     Key: "findByActiveTrue" + params                            |
|     Value: [1, 3, 5, 7, 9]  (list of product IDs)              |
|  4. Store each product in entity L2 cache                       |
|                                                                 |
|  Second call: findByActiveTrue()                                |
|  1. Query cache HIT! Returns [1, 3, 5, 7, 9]                   |
|  2. Look up each ID in entity L2 cache                          |
|  3. No SQL at all!                                              |
|                                                                 |
|  INVALIDATION:                                                  |
|  Any INSERT, UPDATE, or DELETE on the products table            |
|  invalidates ALL query cache entries for products.              |
|  This is aggressive — even updating product 99 invalidates      |
|  the cache for findByActiveTrue.                                |
+-----------------------------------------------------------------+
```

When Query Cache Helps vs Hurts

```
Query Cache Decision:
+-------------------------------------------------------------------+
|                                                                   |
|  Query cache HELPS when:                                          |
|  - The SAME query is executed frequently                          |
|  - The underlying data RARELY changes                             |
|  - The result set is small                                        |
|  Examples:                                                        |
|  - findAllActiveCountries() (changes yearly)                      |
|  - findAllRoles() (changes rarely)                                |
|  - findProductsByCategory("popular") (changes daily)              |
|                                                                   |
|  Query cache HURTS when:                                          |
|  - The table is updated frequently                                |
|  - Queries have many different parameter combinations             |
|  - Result sets are large                                          |
|  Examples:                                                        |
|  - findByUserId(userId) (thousands of unique IDs)                |
|  - Real-time data (prices, stock levels, chat messages)           |
|  - Tables with frequent writes (audit logs, events)              |
|                                                                   |
|  Rule of thumb: Query cache is most useful for                    |
|  "read-mostly, shared-across-users" data.                        |
+-------------------------------------------------------------------+
```

Monitoring Cache Performance

Hibernate Statistics

```java
@RestController
@RequestMapping("/api/admin/cache")
public class CacheMonitorController {

    @Autowired
    private EntityManagerFactory emf;

    @GetMapping("/stats")
    public Map<String, Object> getCacheStats() {
        Statistics stats = emf.unwrap(SessionFactory.class).getStatistics();

        Map<String, Object> result = new LinkedHashMap<>();
        result.put("L2 cache hits", stats.getSecondLevelCacheHitCount());
        result.put("L2 cache misses", stats.getSecondLevelCacheMissCount());
        result.put("L2 cache puts", stats.getSecondLevelCachePutCount());
        result.put("Query cache hits", stats.getQueryCacheHitCount());
```

```
        result.put("Query cache misses", stats.getQueryCacheMissCount());

        long hits = stats.getSecondLevelCacheHitCount();
        long misses = stats.getSecondLevelCacheMissCount();
        double hitRate = (hits + misses) > 0
            ? (double) hits / (hits + misses) * 100 : 0;
        result.put("L2 hit rate", String.format("%.1f%%", hitRate));

        return result;
    }
}
```

```
Good Cache Hit Rates:
+-------------------------------------------------------------------+
|                                                                   |
|  Hit Rate          Assessment                                     |
|  ----------------------------------------------------------------|
|  > 90%             Excellent - cache is highly effective          |
|  70-90%            Good - working well for most reads             |
|  50-70%            Moderate - review if entities change too often |
|  < 50%             Poor - cache may not be worth the overhead     |
|                    Consider removing caching for this entity      |
|                                                                   |
|  If hit rate is low, possible causes:                             |
|  - Entity is updated too frequently (invalidates cache)           |
|  - Cache size too small (entries evicted prematurely)             |
|  - Entities loaded by different access patterns (IDs vs queries)  |
+-------------------------------------------------------------------+
```

When NOT to Cache

```
Do NOT Cache:
+-------------------------------------------------------------------+
|                                                                   |
|  1. Frequently updated entities                                   |
|     - Cache is invalidated on every update                        |
|     - High invalidation rate = low hit rate = wasted memory       |
|     - Example: real-time stock prices, live chat messages         |
|                                                                   |
|  2. Entities with large data                                      |
|     - LOBs, large text fields, binary data                        |
|     - Consumes too much cache memory                              |
|     - Better to query on demand                                   |
|                                                                   |
|  3. Security-sensitive data                                       |
|     - User sessions, tokens, permissions that change frequently   |
```

```
|     - Stale cache = security vulnerability                        |
|                                                                   |
|  4. Data that must always be fresh                                |
|     - Inventory counts (must be real-time for ordering)           |
|     - Account balances (stale balance = wrong transaction)        |
|                                                                   |
|  5. Entities only loaded once (unique access patterns)            |
|     - Each entity loaded by a different user, never reused        |
|     - Cache fills up with never-reaccessed entries                |
|                                                                   |
|  DO Cache:                                                        |
|  - Reference data (countries, currencies, categories)             |
|  - Configuration (settings, feature flags)                        |
|  - Slowly changing data (product catalog, user profiles)          |
|  - Frequently read, shared across users                           |
+-------------------------------------------------------------------+
```

Cache Configuration with Caffeine

```java
@Configuration
public class CacheConfig {

    @Bean
    public HibernatePropertiesCustomizer hibernateSecondLevelCacheCustomizer() {
        return properties -> {
            properties.put("hibernate.cache.use_second_level_cache", "true");
            properties.put("hibernate.cache.use_query_cache", "true");
            properties.put("hibernate.cache.region.factory_class",
                "org.hibernate.cache.jcache.internal.JCacheRegionFactory");
            properties.put("hibernate.javax.cache.provider",
                "com.github.benmanes.caffeine.jcache.spi.CaffeineCachingProvider");
        };
    }
}
```

Or use Ehcache with an XML configuration:

```xml
<!-- src/main/resources/ehcache.xml -->
<config xmlns='http://www.ehcache.org/v3'>

    <!-- Default cache template -->
    <cache-template name="default">
        <expiry>
            <ttl unit="minutes">60</ttl>
```

```
        </expiry>
        <heap unit="entries">1000</heap>
    </cache-template>

    <!-- Entity-specific cache -->
    <cache alias="com.example.entity.Product" uses-template="default">
        <heap unit="entries">5000</heap>
    </cache>

    <!-- Collection cache -->
    <cache alias="com.example.entity.Department.employees" uses-template="default">
        <heap unit="entries">500</heap>
    </cache>

    <!-- Query cache region -->
    <cache alias="default-query-results-region" uses-template="default">
        <expiry>
            <ttl unit="minutes">5</ttl>
        </expiry>
    </cache>

</config>
```

Common Mistakes

1. Caching entities that change frequently: If an entity is updated every few seconds, the cache is constantly invalidated. Hit rate drops below 50%, and you pay the overhead of caching without the benefit.
2. Enabling query cache without entity cache: The query cache stores entity IDs. If those entities are not in the L2 entity cache, each ID triggers a database lookup — potentially worse than no caching.
3. Caching without monitoring: Without statistics, you cannot tell if caching is helping. Enable `hibernate.generate_statistics=true` and monitor hit rates.
4. Using READ_ONLY for mutable entities: `READ_ONLY` throws an exception on update. If the entity might ever be modified, use `NONSTRICT_READ_WRITE` or `READ_WRITE`.
5. Expecting the L2 cache to work with JPQL queries: `find()` by ID uses L2 cache. JPQL queries go to the database by default. You need the query cache (with `@QueryHint`) for JPQL result caching.
6. Not configuring cache sizes: Without size limits, caches grow unbounded. Always set maximum entry counts and TTL (time-to-live) for eviction.

Best Practices

1. Start without caching, add when needed: Premature caching adds complexity. Profile first, identify hot spots, then cache strategically.
2. Cache reference data first: Countries, currencies, categories, roles — these change rarely and are read constantly. Best ROI for caching.
3. Always monitor cache statistics: Track hit rates, miss rates, and eviction counts. Remove caching for entities with low hit rates.
4. Set appropriate TTLs: Reference data can cache for hours. Business data should cache for minutes. No cache should live forever without validation.
5. Cache both entities AND collections: If you cache a Department, also cache its employees collection. Otherwise, accessing employees still hits the database.
6. Use the query cache sparingly: Only for frequently repeated queries on stable data. The aggressive invalidation (any table change clears all query caches for that table) makes it counterproductive for volatile data.

Summary

- First-level cache (persistence context) exists per transaction. Same entity ID returns the same object within a transaction.
- Second-level cache is shared across transactions. Stores dehydrated entity state. Reduces database queries for frequently read entities.
- Cache concurrency strategies: `READ_ONLY` for immutable data, `NONSTRICT_READ_WRITE` for rarely changed data, `READ_WRITE` for general use.
- Query cache stores query results (entity IDs) keyed by query + parameters. Invalidated on any table modification. Best for stable, frequently repeated queries.
- Collection cache stores the list of child IDs, not entity data. Works together with entity cache.
- Monitor cache statistics to ensure caching is actually helping. Low hit rates mean caching is adding overhead without benefit.
- Cache reference data (countries, roles, categories) for best ROI. Avoid caching frequently updated or security-sensitive data.

Interview Questions

Q1: What is the difference between L1 and L2 cache in Hibernate?

L1 (first-level) cache is the persistence context — scoped to a single transaction, always on, cannot be disabled. L2 (second-level) cache is shared across transactions, optional, and requires explicit configuration. L1 stores entity objects; L2 stores dehydrated data (field values, not objects).

Q2: What does the second-level cache store?

Dehydrated entity state: field values and foreign key IDs, not Java objects. When retrieved from L2, Hibernate creates a new entity object, populates fields from the cached data, and places it in the L1 cache.

Q3: When should you use READ_ONLY vs READ_WRITE cache strategy?

`READ_ONLY` for immutable reference data that is never updated (countries, currencies). It is fastest because no locking or invalidation is needed. `READ_WRITE` for mutable business data that is both read and written. It uses soft locks to prevent stale reads during updates.

Q4: How does the query cache work and when should you use it?

The query cache stores the list of entity IDs returned by a query, keyed by the query string and parameters. On cache hit, it returns the IDs and looks up each entity in the L2 entity cache. It is invalidated when any row in the queried table is modified. Use it only for frequently repeated queries on stable data.

Q5: What is a common mistake when setting up L2 caching?

Enabling the query cache without also caching the entities. The query cache stores IDs, not data. If the entity is not in the L2 cache, each ID triggers a database SELECT — potentially slower than running the query directly.

Practice Exercises

Exercise 1: L2 Cache Setup Add Caffeine or Ehcache L2 cache to your project. Mark the `Country` entity as `READ_ONLY` cacheable. Load the same country by ID in two separate transactions. Verify with SQL logging that the second load has no SQL query.

Exercise 2: Cache Statistics Enable Hibernate statistics. Load a cacheable entity 100 times across different service calls. Print the cache hit count, miss count, and hit rate. Verify the hit

rate is close to 99%.

Exercise 3: Query Cache Enable the query cache. Mark `findByActiveTrue()` as cacheable. Call it 5 times. Verify only the first call generates SQL. Then update one product and call again — verify the cache was invalidated (SQL is executed).

Exercise 4: Collection Cache Cache a `Department.employees` collection. Load a department and access its employees in two transactions. Verify the second access has no SQL for the collection or the employee entities.

Exercise 5: Cache Comparison Create two services: one with L2 caching enabled and one without. Load the same 100 products 10 times each. Use `System.nanoTime()` to measure the time difference. Document the performance improvement.

What Is Next?

In the next chapter, we will explore Bean Validation with Hibernate Validator — how to declaratively validate entity data using annotations like `@NotNull`, `@Size`, `@Email`, and `@Pattern`. You will learn to create custom validators, use validation groups, and integrate validation with Spring MVC for automatic request validation.

Chapter 22: Bean Validation with Hibernate Validator

Learning Goals

By the end of this chapter, you will be able to:

- Understand the Jakarta Bean Validation specification and its role
- Use built-in constraint annotations (@NotNull, @Size, @Email, @Min, @Max, etc.)
- Distinguish between @NotNull, @NotEmpty, and @NotBlank
- Apply validation to JPA entities for database-level enforcement
- Create custom constraint annotations with validators
- Use validation groups for context-dependent validation
- Integrate validation with Spring MVC (@Valid in controllers)
- Handle and format validation error responses
- Understand the relationship between Bean Validation and JPA constraints

What Is Bean Validation?

Jakarta Bean Validation (formerly javax.validation) is a Java specification for declarative data validation. You annotate fields with constraints, and the framework validates them automatically.

Hibernate Validator is the reference implementation — just as Hibernate ORM implements JPA, Hibernate Validator implements Bean Validation. Spring Boot includes it automatically with `spring-boot-starter-web` or `spring-boot-starter-validation`.

```
Bean Validation in the Stack:
+-------------------------------------------------------------------+
|                                                                   |
|  Your Entity / DTO                                                |
|  @NotBlank String name;                                           |
|  @Email String email;                                             |
|  @Min(0) BigDecimal price;                                        |
|       |                                                           |
|       v                                                           |
|  Jakarta Bean Validation API (Specification)                      |
|  jakarta.validation.constraints.*                                 |
|       |                                                           |
|       v                                                           |
|  Hibernate Validator (Implementation)                             |
|  Reads annotations, runs validation logic                         |
|       |                                                           |
|       v                                                           |
|  Validation Result                                                |
|  Set<ConstraintViolation> violations                              |
|  (empty = valid, non-empty = invalid with error messages)         |
+-------------------------------------------------------------------+
```

Built-In Constraint Annotations

Null and Blank Checks

```java
@Entity
@Table(name = "users")
public class User {

    @NotNull        // Field cannot be null (allows empty string "")
    private String username;

    @NotEmpty       // Field cannot be null AND cannot be empty ("")
    private String firstName;     // Rejects: null, ""
                                  // Accepts: " ", "Alice"

    @NotBlank       // Cannot be null, empty, or only whitespace
    private String lastName;      // Rejects: null, "", "   "
                                  // Accepts: "Smith", " Smith "
}
```

```
@NotNull vs @NotEmpty vs @NotBlank:
+-------------------------------------------------------------------+
|                                                                   |
```

```
|  Value          @NotNull     @NotEmpty    @NotBlank                |
|  ----------------------------------------------------------------|
|  null            FAIL         FAIL         FAIL                   |
|  ""              PASS         FAIL         FAIL                   |
|  "   "           PASS         PASS         FAIL                   |
|  "Alice"         PASS         PASS         PASS                   |
|                                                                    |
|  Use @NotNull for: non-String fields (Long, BigDecimal, etc.)    |
|  Use @NotBlank for: String fields where you need real content    |
|  Use @NotEmpty for: String or Collection fields (not just blank) |
+-------------------------------------------------------------------+
```

String Constraints

```java
@Size(min = 2, max = 100)               // String length between 2 and 100
private String name;

@Size(max = 500)                         // Max 500 characters
private String description;

@Email                                   // Must be a valid email format
private String email;

@Pattern(regexp = "^[A-Z]{2}-\\d{4}$")  // Must match regex pattern
private String productCode;              // e.g., "AB-1234"

@Pattern(regexp = "^\\+?[1-9]\\d{1,14}$", message = "Invalid phone number")
private String phone;
```

Number Constraints

```java
@Min(0)                                  // Must be >= 0
private BigDecimal price;

@Max(150)                                // Must be <= 150
private int age;

@Positive                                // Must be > 0
private BigDecimal salary;

@PositiveOrZero                          // Must be >= 0
private int quantity;

@Negative                                // Must be < 0
private BigDecimal discount;             // Stored as negative value
```

```
@DecimalMin(value = "0.01")              // Must be >= 0.01
private BigDecimal minimumOrderAmount;

@DecimalMax(value = "999999.99")         // Must be <= 999999.99
private BigDecimal maximumPrice;

@Digits(integer = 6, fraction = 2)      // Max 6 integer, 2 decimal digits
private BigDecimal amount;               // e.g., 123456.78
```

Date and Time Constraints

```java
@Past                                    // Must be in the past
private LocalDate birthDate;

@PastOrPresent                           // Must be in the past or today
private LocalDate hireDate;

@Future                                  // Must be in the future
private LocalDate expirationDate;

@FutureOrPresent                         // Must be today or later
private LocalDateTime appointmentTime;
```

Collection Constraints

```java
@NotEmpty                                // Collection must not be null or empty
@Size(min = 1, max = 10)                 // Must have 1-10 elements
private List<String> tags;
```

```
Complete Built-In Constraints Reference:
+-------------------------------------------------------------------+
|                                                                    |
|  Annotation        Applies To        Validates                    |
|  ----------------------------------------------------------------|
|  @NotNull           Any              Not null                     |
|  @NotEmpty          String/Coll      Not null, not empty          |
|  @NotBlank          String           Not null, not blank          |
|  @Null              Any              Must be null                 |
|  @Size              String/Coll      Length/size in range          |
|  @Min               Number           >= value                     |
|  @Max               Number           <= value                     |
|  @Positive          Number           > 0                          |
|  @PositiveOrZero    Number           >= 0                         |
|  @Negative          Number           < 0                          |
|  @NegativeOrZero    Number           <= 0                         |
|  @DecimalMin        Number           >= value (as string)         |
```

```
|  @DecimalMax        Number           <= value (as string)         |
|  @Digits            Number           Integer + fraction digits    |
|  @Email             String           Valid email format            |
|  @Pattern           String           Matches regex                |
|  @Past              Date/Time        In the past                  |
|  @PastOrPresent     Date/Time        Past or now                  |
|  @Future            Date/Time        In the future                |
|  @FutureOrPresent   Date/Time        Now or future                |
|  @AssertTrue        Boolean          Must be true                 |
|  @AssertFalse       Boolean          Must be false                |
+-------------------------------------------------------------------+
```

Validated Entity Example

```java
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Name is required")
    @Size(min = 2, max = 100, message = "Name must be between 2 and 100 characters")
    @Column(nullable = false, length = 100)
    private String name;

    @NotBlank(message = "Email is required")
    @Email(message = "Email must be a valid email address")
    @Column(nullable = false, unique = true)
    private String email;

    @NotNull(message = "Salary is required")
    @Positive(message = "Salary must be positive")
    @Digits(integer = 8, fraction = 2, message = "Salary must have at most 8 integer an...
    @Column(nullable = false)
    private BigDecimal salary;

    @PastOrPresent(message = "Hire date cannot be in the future")
    @Column(name = "hire_date")
    private LocalDate hireDate;

    @Size(max = 500, message = "Bio must be at most 500 characters")
    private String bio;

    protected Employee() {}
```

```
    public Employee(String name, String email, BigDecimal salary) {
        this.name = name;
        this.email = email;
        this.salary = salary;
        this.hireDate = LocalDate.now();
    }

    // Getters and setters...
}
```

```
Bean Validation vs JPA Constraints:
+-------------------------------------------------------------------+
|                                                                   |
|  Bean Validation (@NotBlank, @Size, @Email):                      |
|  - Validates in JAVA before SQL is executed                       |
|  - Runs when entity is persisted or controller receives input     |
|  - Provides user-friendly error messages                          |
|  - Can validate DTOs (not just entities)                          |
|                                                                   |
|  JPA Constraints (@Column(nullable, length, unique)):             |
|  - Creates DATABASE constraints (NOT NULL, VARCHAR(100), UNIQUE)  |
|  - Enforced by the database itself                                |
|  - Last line of defense — catches issues Bean Validation missed   |
|  - Error messages are database-level (not user-friendly)          |
|                                                                   |
|  Best practice: Use BOTH.                                         |
|  @NotBlank + @Column(nullable = false)                            |
|  @Size(max = 100) + @Column(length = 100)                        |
|  Bean Validation catches errors early with good messages.         |
|  JPA constraints provide database-level data integrity.           |
+-------------------------------------------------------------------+
```

Validation in Spring MVC Controllers

Validating Request Bodies

```java
// Request DTO (not an entity — separate class for API input)
public class CreateEmployeeRequest {

    @NotBlank(message = "Name is required")
    @Size(min = 2, max = 100)
    private String name;
```

```
    @NotBlank(message = "Email is required")
    @Email(message = "Must be a valid email")
    private String email;

    @NotNull(message = "Salary is required")
    @Positive(message = "Salary must be positive")
    private BigDecimal salary;

    // Getters and setters...
}
```

```java
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    private final EmployeeService employeeService;

    @PostMapping
    public ResponseEntity<Employee> createEmployee(
            @Valid @RequestBody CreateEmployeeRequest request) {
        //  ^^^^^^ @Valid triggers validation!
        // If validation fails, Spring throws MethodArgumentNotValidException
        // BEFORE your code runs.

        Employee employee = employeeService.create(
            request.getName(),
            request.getEmail(),
            request.getSalary()
        );
        return ResponseEntity.status(HttpStatus.CREATED).body(employee);
    }
}
```

```
@Valid Request Flow:
+-------------------------------------------------------------------+
|                                                                   |
|  POST /api/employees                                              |
|  { "name": "", "email": "not-an-email", "salary": -5 }           |
|       |                                                           |
|       v                                                           |
|  Spring deserializes JSON --> CreateEmployeeRequest               |
|       |                                                           |
|       v                                                           |
|  @Valid triggers Bean Validation                                  |
|       |                                                           |
|  Violations found:                                                |
|  - name: "Name is required" (blank)                               |
|  - email: "Must be a valid email"                                 |
|  - salary: "Salary must be positive" (-5)                        |
|       |                                                           |
|       v                                                           |
```

```
|  MethodArgumentNotValidException thrown                          |
|  Controller method is NEVER called                               |
|       |                                                          |
|       v                                                          |
|  @ControllerAdvice handles it --> 400 Bad Request with errors    |
+------------------------------------------------------------------+
```

Global Error Handler

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidationErrors(
            MethodArgumentNotValidException ex) {

        Map<String, String> fieldErrors = new LinkedHashMap<>();
        for (FieldError error : ex.getBindingResult().getFieldErrors()) {
            fieldErrors.put(error.getField(), error.getDefaultMessage());
        }

        Map<String, Object> response = new LinkedHashMap<>();
        response.put("status", 400);
        response.put("error", "Validation Failed");
        response.put("fieldErrors", fieldErrors);

        return ResponseEntity.badRequest().body(response);
    }
}
```

```
Error Response Example:
+------------------------------------------------------------------+
|                                                                  |
|  {                                                               |
|    "status": 400,                                                |
|    "error": "Validation Failed",                                 |
|    "fieldErrors": {                                              |
|      "name": "Name is required",                                 |
|      "email": "Must be a valid email",                           |
|      "salary": "Salary must be positive"                         |
|    }                                                             |
|  }                                                               |
+------------------------------------------------------------------+
```

Custom Constraint Annotations

When built-in constraints are not enough, create your own:

Example: @ValidPhoneNumber

```java
// Step 1: Define the annotation
@Documented
@Constraint(validatedBy = PhoneNumberValidator.class)
@Target({ElementType.FIELD, ElementType.PARAMETER})
@Retention(RetentionPolicy.RUNTIME)
public @interface ValidPhoneNumber {
    String message() default "Invalid phone number format";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```

```java
// Step 2: Implement the validator
public class PhoneNumberValidator
        implements ConstraintValidator<ValidPhoneNumber, String> {

    private static final Pattern PHONE_PATTERN =
        Pattern.compile("^\\+?[1-9]\\d{6,14}$");

    @Override
    public void initialize(ValidPhoneNumber annotation) {
        // Optional initialization
    }

    @Override
    public boolean isValid(String value, ConstraintValidatorContext context) {
        if (value == null) {
            return true;  // Use @NotNull separately for null check
        }
        return PHONE_PATTERN.matcher(value).matches();
    }
}
```

```java
// Step 3: Use it
public class ContactRequest {

    @NotBlank
    private String name;

    @ValidPhoneNumber    // Custom constraint!
    private String phone;
}
```

Example: @UniqueEmail (Database-Aware Validation)

```java
@Documented
@Constraint(validatedBy = UniqueEmailValidator.class)
@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
public @interface UniqueEmail {
    String message() default "Email is already registered";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```

```java
@Component  // Must be a Spring bean to inject repositories
public class UniqueEmailValidator
        implements ConstraintValidator<UniqueEmail, String> {

    private final UserRepository userRepository;

    public UniqueEmailValidator(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public boolean isValid(String email, ConstraintValidatorContext context) {
        if (email == null) return true;
        return !userRepository.existsByEmail(email);
    }
}
```

```java
public class RegisterRequest {
    @NotBlank
    @Email
    @UniqueEmail   // Checks database for duplicates!
    private String email;
}
```

Validation Groups

Groups let you apply different validation rules in different contexts:

```java
// Define group marker interfaces
public interface OnCreate {}
public interface OnUpdate {}
```

```java
public class ProductRequest {

    @Null(groups = OnCreate.class, message = "ID must not be provided for creation")
    @NotNull(groups = OnUpdate.class, message = "ID is required for update")
    private Long id;

    @NotBlank(groups = {OnCreate.class, OnUpdate.class})
    @Size(min = 2, max = 100)
    private String name;

    @NotNull(groups = OnCreate.class, message = "Price is required")
    @Positive
    private BigDecimal price;
}
```

```java
@RestController
@RequestMapping("/api/products")
public class ProductController {

    @PostMapping
    public ResponseEntity<Product> create(
            @Validated(OnCreate.class) @RequestBody ProductRequest request) {
        //  ^^^^^^^^^^^^^^^^^^^^^^^^^^ Use @Validated (not @Valid) for groups
        // Validates: id must be null, name required, price required
    }

    @PutMapping("/{id}")
    public ResponseEntity<Product> update(
            @PathVariable Long id,
            @Validated(OnUpdate.class) @RequestBody ProductRequest request) {
        // Validates: id must not be null, name required
        // Price NOT validated (not in OnUpdate group)
    }
}
```

```
Validation Groups:
+-------------------------------------------------------------------+
|                                                                   |
|  Without groups: Same rules for create and update                 |
|  Problem: Create requires price, update may not                   |
|           Create has no ID, update requires ID                    |
|                                                                   |
|  With groups:                                                     |
|  OnCreate: id=@Null, name=@NotBlank, price=@NotNull              |
|  OnUpdate: id=@NotNull, name=@NotBlank                           |
|                                                                   |
|  Note: Use @Validated (Spring) not @Valid (Jakarta)               |
|  @Valid does not support groups parameter.                        |
+-------------------------------------------------------------------+
```

Cascading Validation with @Valid

Validate nested objects by adding `@Valid` to the field:

```java
public class OrderRequest {

    @NotBlank
    private String customerName;

    @NotEmpty(message = "Order must have at least one item")
    @Valid    // Cascade validation to each OrderItemRequest
    private List<OrderItemRequest> items;
}

public class OrderItemRequest {

    @NotBlank(message = "Product name is required")
    private String productName;

    @NotNull
    @Positive(message = "Quantity must be positive")
    private Integer quantity;

    @NotNull
    @Positive(message = "Price must be positive")
    private BigDecimal price;
}
```

```
Cascading @Valid:
+-------------------------------------------------------------------+
|                                                                   |
|  Without @Valid on the list:                                      |
|  - OrderRequest is validated (customerName, items not empty)      |
|  - OrderItemRequest fields are NOT validated                      |
|  - Invalid items slip through!                                    |
|                                                                   |
|  With @Valid on the list:                                         |
|  - OrderRequest is validated                                      |
|  - EACH OrderItemRequest in the list is also validated            |
|  - If item[2].quantity is -1, validation fails with:              |
|    field: "items[2].quantity"                                     |
|    message: "Quantity must be positive"                            |
+-------------------------------------------------------------------+
```

Programmatic Validation

Validate entities manually in service methods:

```java
@Service
public class EmployeeService {

    private final Validator validator;   // jakarta.validation.Validator

    public EmployeeService(Validator validator) {
        this.validator = validator;
    }

    public void validateEmployee(Employee employee) {
        Set<ConstraintViolation<Employee>> violations = validator.validate(employee);

        if (!violations.isEmpty()) {
            StringBuilder sb = new StringBuilder("Validation failed:\n");
            for (ConstraintViolation<Employee> v : violations) {
                sb.append("  ")
                   .append(v.getPropertyPath())
                   .append(": ")
                   .append(v.getMessage())
                   .append("\n");
            }
            throw new IllegalArgumentException(sb.toString());
        }
    }

    // Validate with specific group
    public void validateForCreate(ProductRequest request) {
        Set<ConstraintViolation<ProductRequest>> violations =
            validator.validate(request, OnCreate.class);
        // Handle violations...
    }
}
```

Custom Error Messages

Inline Messages

```java
@NotBlank(message = "First name is required")
private String firstName;
```

```
@Size(min = 8, max = 64, message = "Password must be between {min} and {max} characters")
private String password;  // {min} and {max} are interpolated from the annotation

@Min(value = 18, message = "Must be at least {value} years old")
private int age;
```

Messages from Properties File

```java
@NotBlank(message = "{employee.name.required}")
private String name;

@Email(message = "{employee.email.invalid}")
private String email;
```

```properties
# src/main/resources/ValidationMessages.properties
employee.name.required=Employee name is required
employee.email.invalid=Please provide a valid email address
employee.salary.positive=Salary must be a positive amount

# With interpolation
password.size=Password must be between {min} and {max} characters
```

Common Mistakes

1. Using @NotNull on String fields instead of @NotBlank: `@NotNull` allows empty strings `""`. For strings where you need actual content, use `@NotBlank`.
2. Using @Valid instead of @Validated for groups: `@Valid` (Jakarta) does not support validation groups. Use `@Validated` (Spring) when you need group-specific validation.
3. Returning null from custom validators for null input: Custom validators should return `true` for null values and let `@NotNull` handle null checks separately. This follows the Jakarta Bean Validation convention.
4. Not validating nested objects: Adding `@NotEmpty` to a `List<OrderItem>` validates the list is not empty but does NOT validate each item's fields. Add `@Valid` to cascade validation.
5. Putting validation only on entities, not on DTOs: Validate request DTOs at the controller level. Entity-level validation is a safety net, not the primary validation point.
6. Not handling MethodArgumentNotValidException: Without a `@ControllerAdvice` handler, Spring returns a generic 400 error. Add a handler that returns structured field errors.

Best Practices

1. Validate at the boundary: Validate incoming data in controllers (DTOs with `@Valid`) and as a safety net on entities. Do not rely solely on database constraints.
2. Use @NotBlank for strings, @NotNull for everything else: This covers the most common validation needs correctly.
3. Keep validation messages user-friendly: Use `message` attributes or message properties files. "Name is required" is better than "must not be blank."
4. Use separate DTOs for create and update: With different validation groups or completely separate DTO classes instead of reusing the same class.
5. Return structured error responses: Include field names, messages, and HTTP 400 status. Frontend developers need to map errors to form fields.
6. Use @Valid on nested objects and collections: Always cascade validation into complex request structures.

Summary

- Jakarta Bean Validation is a specification for declarative data validation. Hibernate Validator is the implementation (included in Spring Boot).
- Built-in constraints: `@NotNull`, `@NotBlank`, `@Size`, `@Email`, `@Min`, `@Max`, `@Positive`, `@Past`, `@Future`, `@Pattern`, and more.
- @NotBlank is preferred for String fields (rejects null, empty, and whitespace). `@NotNull` is for non-String fields.
- @Valid in controllers triggers automatic validation before the method executes. Validation failures throw `MethodArgumentNotValidException`.
- Custom constraints are created with an annotation + a `ConstraintValidator` implementation. The validator can be a Spring bean for database-aware validation.
- Validation groups (`OnCreate`, `OnUpdate`) apply different rules in different contexts. Use `@Validated` (not `@Valid`) for group support.
- Cascading @Valid on nested objects and collections ensures deeply nested data is validated.
- Use both Bean Validation (Java-level, user-friendly messages) and JPA constraints (database-level, data integrity).

Interview Questions

Q1: What is the difference between @NotNull, @NotEmpty, and @NotBlank?

`@NotNull` only checks that the value is not null (allows empty string `""`). `@NotEmpty` checks not null AND not empty (rejects `null` and `""`but allows `" "`). `@NotBlank` checks not null, not empty, AND not whitespace-only (rejects `null`, `""`, and `" "`).

Q2: How does @Valid work in a Spring controller?

When `@Valid` is placed on a `@RequestBody` parameter, Spring deserializes the request body, then runs Bean Validation on the resulting object before the controller method executes. If validation fails, `MethodArgumentNotValidException` is thrown and the method is never called.

Q3: How do you create a custom validation annotation?

Create an annotation with `@Constraint(validatedBy = MyValidator.class)` and the required attributes (`message`, `groups`, `payload`). Then create a class implementing `ConstraintValidator<MyAnnotation, FieldType>` with an `isValid()` method. Return `true` for valid, `false` for invalid.

Q4: What are validation groups and when should you use them?

Groups are marker interfaces that let you apply different validation rules in different contexts. For example, `@Null(groups = OnCreate.class)` on an ID field ensures no ID is sent during creation, while `@NotNull(groups = OnUpdate.class)` ensures an ID is provided during updates. Use `@Validated(OnCreate.class)` in the controller.

Q5: Should you validate on entities, DTOs, or both?

Both. Validate on DTOs at the controller layer for user-friendly error messages and early rejection. Validate on entities as a safety net to ensure data integrity regardless of how the entity is created. The DTO validation is the primary line of defense; entity validation is the secondary.

Practice Exercises

Exercise 1: Validated Entity Create a `Product` entity with: `@NotBlank` name, `@Positive` price, `@Size(max=500)` description, `@PastOrPresent` createdAt. Write a test that persists invalid data and verifies `ConstraintViolationException` is thrown.

Exercise 2: Controller Validation Create a `POST /api/products` endpoint with a `CreateProductRequest` DTO. Add `@Valid`. Write a `@ControllerAdvice` that returns structured field errors. Test with invalid input and verify the error response format.

Exercise 3: Custom Constraint Create a `@StrongPassword` annotation that requires: at least 8 characters, one uppercase, one lowercase, one digit, one special character. Write the validator and test with various passwords.

Exercise 4: Validation Groups Create a `UserRequest` with `OnCreate` and `OnUpdate` groups. On create: password is required, email must be unique. On update: password is optional, ID is required. Implement both endpoints and test.

Exercise 5: Cascading Validation Create an `OrderRequest` with a `List<@Valid OrderItemRequest>`. Validate that: order has at least one item, each item has a positive quantity and price. Test with a request where item[2] has invalid data and verify the error path includes the index.

What Is Next?

In the next chapter, we enter Part V: Production Patterns and start with DTOs, Projections, and Data Transfer. You will learn why exposing JPA entities directly in APIs is dangerous, how to create proper DTOs, use MapStruct for automated mapping, and leverage JPA projections for efficient data retrieval.

Chapter 23: DTOs, Projections, and Data Transfer

Learning Goals

By the end of this chapter, you will be able to:

- Explain why exposing JPA entities directly in APIs is problematic
- Create DTO classes for API request and response data
- Map between entities and DTOs manually and with MapStruct
- Use JPA interface-based projections for efficient read queries
- Use class-based projections with constructor expressions in JPQL
- Apply dynamic projections for flexible query return types
- Choose between DTOs, projections, and entities for different use cases
- Structure DTOs for create, update, and response scenarios

Why Not Expose Entities Directly?

It is tempting to return JPA entities directly from controllers. It works, but causes serious problems in production:

```
Problems with Exposing Entities:
+-------------------------------------------------------------------+
|                                                                   |
|  1. SECURITY — Leaking sensitive data                             |
|     @Entity User has: password, salt, loginAttempts,              |
|     internalNotes, isAdmin                                        |
|     Returning User in a GET /api/users endpoint                   |
|     exposes ALL fields in the JSON response!                      |
```

```
|                                                                    |
|  2. COUPLING — API tied to database schema                         |
|     If you rename a column or add a field, the API changes.        |
|     Frontend breaks. Mobile app breaks. Partner APIs break.        |
|                                                                    |
|  3. PERFORMANCE — Over-fetching                                    |
|     A list endpoint returns ALL entity fields + triggers           |
|     lazy loading of relationships during JSON serialization.       |
|     You send 20 fields when the client needs 3.                  |
|                                                                    |
|  4. CIRCULAR REFERENCES — Infinite JSON                            |
|     Department has List<Employee>, Employee has Department.        |
|     Jackson serializes Department -> employees -> department ->    |
|     employees -> ... StackOverflowError!                           |
|                                                                    |
|  5. LAZY LOADING — LazyInitializationException                     |
|     Entity returned from @Transactional method is detached.        |
|     JSON serializer accesses lazy field -> Exception!              |
|     (Unless OSIV is on, which has its own problems)                |
+--------------------------------------------------------------------+
```

```
Entity vs DTO:
+--------------------------------------------------------------------+
|                                                                    |
|  Entity (internal):           DTO (external):                      |
|  @Entity                      public class UserResponse {         |
|  public class User {              Long id;                         |
|      Long id;                     String name;                     |
|      String name;                 String email;                    |
|      String email;                String role;                     |
|      String passwordHash;    }                                     |
|      String salt;                                                  |
|      int loginAttempts;       Only exposes what the                |
|      String internalNotes;    client needs to see.                 |
|      boolean isAdmin;         No sensitive data.                   |
|      Department department;   No circular references.              |
|      List<Role> roles;        No lazy loading issues.              |
|  }                                                                 |
|                                                                    |
+--------------------------------------------------------------------+
```

DTO Pattern

A Data Transfer Object (DTO) is a simple class that carries data between layers. It has no business logic, no persistence annotations, and no database awareness.

Response DTOs

```java
// What the API returns to clients
public class EmployeeResponse {
    private Long id;
    private String name;
    private String email;
    private BigDecimal salary;
    private String departmentName;  // Flattened from department.name
    private LocalDate hireDate;

    public EmployeeResponse() {}

    public EmployeeResponse(Long id, String name, String email,
                            BigDecimal salary, String departmentName,
                            LocalDate hireDate) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.salary = salary;
        this.departmentName = departmentName;
        this.hireDate = hireDate;
    }

    // Getters and setters...
}
```

Request DTOs

```java
// What the client sends for creating
public class CreateEmployeeRequest {

    @NotBlank(message = "Name is required")
    @Size(min = 2, max = 100)
    private String name;

    @NotBlank @Email
    private String email;

    @NotNull @Positive
    private BigDecimal salary;

    @NotNull
    private Long departmentId;   // Client sends ID, not the entity

    // Getters and setters...
}

// What the client sends for updating (some fields optional)
```

```
public class UpdateEmployeeRequest {

    @Size(min = 2, max = 100)
    private String name;          // Optional — only update if provided

    @Email
    private String email;

    @Positive
    private BigDecimal salary;

    // Getters and setters...
}
```

```
DTO Flow Through Layers:
+-------------------------------------------------------------------+
|                                                                   |
|  Client          Controller         Service          Repository   |
|    |                 |                  |                 |        |
|    | JSON request    |                  |                 |        |
|    |---------------->|                  |                 |        |
|    |                 | CreateEmployee   |                 |        |
|    |                 | Request (DTO)    |                 |        |
|    |                 |---------------->|                 |        |
|    |                 |                  | Convert DTO     |        |
|    |                 |                  | to Entity       |        |
|    |                 |                  |---------------->|        |
|    |                 |                  |    save(entity)  |        |
|    |                 |                  |<----------------|        |
|    |                 |                  | Convert Entity   |        |
|    |                 |  EmployeeResponse| to DTO          |        |
|    |                 |<----------------|                 |        |
|    | JSON response   |                  |                 |        |
|    |<----------------|                  |                 |        |
|                                                                   |
|  DTOs cross layer boundaries. Entities stay in the service layer. |
+-------------------------------------------------------------------+
```

Manual Mapping

The simplest approach — write mapping methods by hand:

```java
@Service
@Transactional
public class EmployeeService {
```

```
    private final EmployeeRepository employeeRepository;
    private final DepartmentRepository departmentRepository;

    // === Create ===
    public EmployeeResponse create(CreateEmployeeRequest request) {
        Department department = departmentRepository
            .findById(request.getDepartmentId())
            .orElseThrow(() -> new RuntimeException("Department not found"));

        Employee employee = new Employee();
        employee.setName(request.getName());
        employee.setEmail(request.getEmail());
        employee.setSalary(request.getSalary());
        employee.setDepartment(department);
        employee.setHireDate(LocalDate.now());

        Employee saved = employeeRepository.save(employee);
        return toResponse(saved);
    }

    // === Read ===
    @Transactional(readOnly = true)
    public EmployeeResponse getById(Long id) {
        Employee employee = employeeRepository.findWithDeptById(id)
            .orElseThrow(() -> new RuntimeException("Employee not found"));
        return toResponse(employee);
    }

    // === List ===
    @Transactional(readOnly = true)
    public Page<EmployeeResponse> getAll(Pageable pageable) {
        return employeeRepository.findAll(pageable)
            .map(this::toResponse);
    }

    // === Update ===
    public EmployeeResponse update(Long id, UpdateEmployeeRequest request) {
        Employee employee = employeeRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Employee not found"));

        if (request.getName() != null) employee.setName(request.getName());
        if (request.getEmail() != null) employee.setEmail(request.getEmail());
        if (request.getSalary() != null) employee.setSalary(request.getSalary());

        return toResponse(employee);
    }

    // === Mapping Method ===
    private EmployeeResponse toResponse(Employee e) {
        return new EmployeeResponse(
            e.getId(),
            e.getName(),
            e.getEmail(),
            e.getSalary(),
```

```
            e.getDepartment() != null ? e.getDepartment().getName() : null,
            e.getHireDate()
        );
    }
}
```

```
Manual Mapping Trade-offs:
+-------------------------------------------------------------------+
|                                                                    |
|  Pros:                                                             |
|  + No extra dependencies                                          |
|  + Full control over mapping logic                                 |
|  + Easy to debug (just step through the method)                    |
|  + Works with any project                                          |
|                                                                    |
|  Cons:                                                             |
|  - Tedious for entities with many fields                           |
|  - Easy to forget a field when entity changes                     |
|  - Repetitive code across services                                 |
|  - No compile-time detection of missing mappings                   |
+-------------------------------------------------------------------+
```

MapStruct — Automated Mapping

MapStruct generates mapping code at compile time. You define an interface, and MapStruct creates the implementation:

Setup

```xml
<!-- pom.xml -->
<dependency>
    <groupId>org.mapstruct</groupId>
    <artifactId>mapstruct</artifactId>
    <version>1.5.5.Final</version>
</dependency>

<!-- Annotation processor -->
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <configuration>
        <annotationProcessorPaths>
            <path>
```

```
                <groupId>org.mapstruct</groupId>
                <artifactId>mapstruct-processor</artifactId>
                <version>1.5.5.Final</version>
            </path>
        </annotationProcessorPaths>
    </configuration>
</plugin>
```

Mapper Interface

```java
@Mapper(componentModel = "spring")   // Generates a Spring @Component bean
public interface EmployeeMapper {

    // Auto-maps matching field names (id, name, email, salary, hireDate)
    @Mapping(source = "department.name", target = "departmentName")
    EmployeeResponse toResponse(Employee employee);

    // Map a list
    List<EmployeeResponse> toResponseList(List<Employee> employees);

    // Create entity from request (ignore id and department - set separately)
    @Mapping(target = "id", ignore = true)
    @Mapping(target = "department", ignore = true)
    @Mapping(target = "hireDate", ignore = true)
    Employee toEntity(CreateEmployeeRequest request);

    // Update existing entity from request (null values are skipped)
    @BeanMapping(nullValuePropertyMappingStrategy = NullValuePropertyMappingStrategy.IG...
    @Mapping(target = "id", ignore = true)
    @Mapping(target = "department", ignore = true)
    void updateEntity(UpdateEmployeeRequest request, @MappingTarget Employee employee);
}
```

```java
// Usage in service
@Service
public class EmployeeService {

    private final EmployeeRepository employeeRepository;
    private final DepartmentRepository departmentRepository;
    private final EmployeeMapper mapper;   // Inject the mapper

    public EmployeeResponse create(CreateEmployeeRequest request) {
        Employee employee = mapper.toEntity(request);
        employee.setHireDate(LocalDate.now());

        Department dept = departmentRepository.findById(request.getDepartmentId())
            .orElseThrow();
        employee.setDepartment(dept);
```

```
        return mapper.toResponse(employeeRepository.save(employee));
    }

    public EmployeeResponse update(Long id, UpdateEmployeeRequest request) {
        Employee employee = employeeRepository.findById(id).orElseThrow();
        mapper.updateEntity(request, employee);  // Updates non-null fields
        return mapper.toResponse(employee);
    }
}
```

```
MapStruct at Compile Time:
+-------------------------------------------------------------------+
|                                                                   |
|  You write:                                                       |
|  @Mapper interface EmployeeMapper {                               |
|      EmployeeResponse toResponse(Employee employee);              |
|  }                                                                |
|                                                                   |
|  MapStruct generates (at compile time):                           |
|  @Component                                                      |
|  public class EmployeeMapperImpl implements EmployeeMapper {      |
|      @Override                                                    |
|      public EmployeeResponse toResponse(Employee employee) {     |
|          if (employee == null) return null;                        |
|          EmployeeResponse dto = new EmployeeResponse();           |
|          dto.setId(employee.getId());                              |
|          dto.setName(employee.getName());                          |
|          dto.setEmail(employee.getEmail());                        |
|          dto.setSalary(employee.getSalary());                      |
|          dto.setHireDate(employee.getHireDate());                  |
|          if (employee.getDepartment() != null) {                  |
|              dto.setDepartmentName(                                |
|                  employee.getDepartment().getName());              |
|          }                                                        |
|          return dto;                                               |
|      }                                                            |
|  }                                                                |
|                                                                   |
|  No reflection. No runtime overhead. Just plain method calls.    |
+-------------------------------------------------------------------+
```

JPA Interface-Based Projections

Instead of loading a full entity and mapping to a DTO, you can ask JPA to load only the fields you need:

```java
// Projection interface — only the fields needed for the list view
public interface EmployeeListProjection {
    Long getId();
    String getName();
    String getEmail();

    @Value("#{target.department.name}")  // SpEL for nested fields
    String getDepartmentName();
}
```

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Returns projections — Hibernate loads ONLY the needed columns
    List<EmployeeListProjection> findByActive(boolean active);

    // Projections with pagination
    Page<EmployeeListProjection> findByDepartmentName(String dept, Pageable pageable);

    // Projections with JPQL
    @Query("SELECT e FROM Employee e WHERE e.salary > :min")
    List<EmployeeListProjection> findHighEarnerProjections(@Param("min") BigDecimal min);
}
```

```
Entity vs Projection SQL:
+-------------------------------------------------------------------+
|                                                                   |
|  Entity query: List<Employee> findByActive(boolean active)        |
|  SQL: SELECT id, name, email, salary, hire_date, bio,             |
|       department_id, active, created_at, updated_at               |
|       FROM employees WHERE active = true                          |
|  --> Loads ALL 10 columns                                         |
|                                                                   |
|  Projection: List<EmployeeListProjection> findByActive(boolean)   |
|  SQL: SELECT id, name, email                                      |
|       FROM employees WHERE active = true                          |
|  --> Loads ONLY 3 columns (what the projection needs)             |
|                                                                   |
|  Benefits:                                                        |
|  - Less data transferred from database                            |
|  - No entity managed in persistence context (less memory)         |
|  - No dirty checking overhead                                     |
|  - No lazy loading issues (no entity = no proxies)                |
+-------------------------------------------------------------------+
```

Class-Based Projections (JPQL Constructor Expressions)

Use the `new` keyword in JPQL to construct DTOs directly:

```java
// DTO for the projection
public class EmployeeSummary {
    private final String name;
    private final String email;
    private final BigDecimal salary;
    private final String departmentName;

    // Constructor must match the JPQL new() arguments
    public EmployeeSummary(String name, String email,
                           BigDecimal salary, String departmentName) {
        this.name = name;
        this.email = email;
        this.salary = salary;
        this.departmentName = departmentName;
    }

    // Getters...
}
```

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    @Query("SELECT new com.example.dto.EmployeeSummary(" +
           "e.name, e.email, e.salary, d.name) " +
           "FROM Employee e JOIN e.department d " +
           "WHERE e.active = true " +
           "ORDER BY e.salary DESC")
    List<EmployeeSummary> findActiveSummaries();

    // With pagination
    @Query(value = "SELECT new com.example.dto.EmployeeSummary(" +
                   "e.name, e.email, e.salary, d.name) " +
                   "FROM Employee e JOIN e.department d WHERE e.active = true",
           countQuery = "SELECT COUNT(e) FROM Employee e WHERE e.active = true")
    Page<EmployeeSummary> findActiveSummariesPaged(Pageable pageable);
}
```

```
Constructor Expression vs Interface Projection:
+-------------------------------------------------------------------+
|                                                                   |
|  Interface Projection:                                            |
|  + No JPQL needed — works with derived methods                    |
|  + Clean, simple                                                  |
```

```
|  - Spring creates proxy objects (slight overhead)                 |
|  - Nested properties need @Value SpEL                             |
|                                                                   |
|  Constructor Expression (new ...):                                |
|  + No proxy — creates real DTO objects                            |
|  + Full control over SQL with JPQL                                |
|  + JOINs are explicit (no N+1 risk)                               |
|  - Requires full class path in JPQL string                        |
|  - Constructor must match query column order exactly              |
|                                                                   |
|  Use interface projections for simple queries.                    |
|  Use constructor expressions for complex queries with JOINs.     |
+-------------------------------------------------------------------+
```

Dynamic Projections

Return different projections from the same query using generics:

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Dynamic projection — caller chooses the return type
    <T> List<T> findByActive(boolean active, Class<T> type);

    <T> Optional<T> findById(Long id, Class<T> type);
}
```

```java
// Use different projections for different use cases
// List view — minimal fields
List<EmployeeListProjection> listView =
    repository.findByActive(true, EmployeeListProjection.class);

// Detail view — more fields
List<EmployeeDetailProjection> detailView =
    repository.findByActive(true, EmployeeDetailProjection.class);

// Full entity when needed
List<Employee> entities =
    repository.findByActive(true, Employee.class);
```

Choosing the Right Approach

```
Decision Guide:
+-------------------------------------------------------------------+
|                                                                   |
|  Need to CREATE or UPDATE data?                                   |
|      |                                                            |
|   YES --> Use request DTOs + manual/MapStruct mapping to entity   |
|                                                                   |
|  Need to READ data for API response?                              |
|      |                                                            |
|      v                                                            |
|  Simple query, few fields, no JOINs?                              |
|      |                                                            |
|   YES --> Interface projection (cleanest)                         |
|      |                                                            |
|    NO                                                             |
|      |                                                            |
|      v                                                            |
|  Complex query with JOINs and aggregations?                       |
|      |                                                            |
|   YES --> Constructor expression in JPQL (most control)           |
|      |                                                            |
|    NO                                                             |
|      |                                                            |
|      v                                                            |
|  Need to load full entity for business logic, then map?           |
|      |                                                            |
|   YES --> Entity + MapStruct/manual mapping to response DTO      |
|                                                                   |
|  Performance Priority:                                            |
|  1. Constructor expression (pure SQL, no entity overhead)         |
|  2. Interface projection (proxy, minimal columns)                 |
|  3. Entity + DTO mapping (full entity load + conversion)          |
+-------------------------------------------------------------------+
```

```
Complete Approach Comparison:
+-------------------------------------------------------------------+
|                                                                   |
|  Approach             Performance   Ease    Flexibility           |
|  ----------------------------------------------------------------|
|  Entity directly       Poor         Easy    Low (security issues) |
|  Entity + manual map   Medium       Medium  High                  |
|  Entity + MapStruct    Medium       Easy    High                  |
|  Interface projection  Good         Easy    Medium                |
|  Constructor expr      Best         Medium  High                  |
+-------------------------------------------------------------------+
```

DTO Organization

```
Project Structure:
+-------------------------------------------------------------------+
|                                                                   |
|  src/main/java/com/example/                                       |
|  +-- entity/                                                      |
|  |   +-- Employee.java                                            |
|  |   +-- Department.java                                          |
|  +-- dto/                                                         |
|  |   +-- employee/                                                |
|  |   |   +-- CreateEmployeeRequest.java                           |
|  |   |   +-- UpdateEmployeeRequest.java                           |
|  |   |   +-- EmployeeResponse.java                                |
|  |   |   +-- EmployeeListProjection.java   (interface)            |
|  |   |   +-- EmployeeSummary.java          (constructor expr)     |
|  |   +-- department/                                              |
|  |       +-- CreateDepartmentRequest.java                         |
|  |       +-- DepartmentResponse.java                              |
|  +-- mapper/                                                      |
|  |   +-- EmployeeMapper.java                                     |
|  |   +-- DepartmentMapper.java                                   |
|  +-- controller/                                                  |
|  +-- service/                                                     |
|  +-- repository/                                                  |
|                                                                   |
|  Convention:                                                      |
|  - Request DTOs: CreateXxxRequest, UpdateXxxRequest               |
|  - Response DTOs: XxxResponse, XxxSummary                         |
|  - Projections: XxxListProjection, XxxDetailProjection            |
|  - Mappers: XxxMapper (one per entity)                            |
+-------------------------------------------------------------------+
```

Common Mistakes

1. Returning entities from controllers: Exposes sensitive fields, causes circular references, triggers lazy loading errors. Always return DTOs.
2. Creating one DTO for everything: Different endpoints need different fields. Use separate DTOs for create, update, list, and detail views.
3. Mapping inside controllers: Keep mapping in the service layer. Controllers should receive and return DTOs. Services handle entity-DTO conversion.
4. Accessing lazy fields during DTO mapping: If mapping happens outside `@Transactional`, lazy fields trigger `LazyInitializationException`. Map inside the transaction or use `@EntityGraph`.
5. Using constructor expressions with `SELECT *`: Constructor expressions must list specific fields, not use wildcard. `SELECT new Dto(e)` does not work — use `SELECT new`

`Dto(e.name, e.email)`.

6. Ignoring partial updates: Using `@NotNull` on all update DTO fields forces clients to send everything. Allow null fields and only update non-null values.

Best Practices

1. Never expose entities in API responses: Always convert to response DTOs. This is a non-negotiable best practice for production applications.
2. Use separate DTOs for create, update, and response: Each operation has different required/optional fields and validation rules.
3. Use projections for read-heavy endpoints: Interface projections and constructor expressions are more efficient than loading full entities and mapping.
4. Use MapStruct for complex mappings: When entities have many fields, MapStruct eliminates boilerplate and catches missing mappings at compile time.
5. Keep DTOs simple: DTOs should be plain data holders. No business logic, no persistence annotations, no framework dependencies.
6. Use Page.map() for paginated DTO conversion: `repository.findAll(pageable).map(mapper::toResponse)` preserves pagination metadata while converting content.

Summary

- DTOs separate internal entity structure from external API contracts. They prevent security leaks, circular references, lazy loading issues, and tight coupling.
- Request DTOs carry validated input data (CreateXxxRequest, UpdateXxxRequest). Response DTOs carry formatted output data (XxxResponse).
- Manual mapping is simple but tedious. MapStruct generates mapping code at compile time with zero runtime overhead.
- Interface projections let Spring Data load only the columns you need — no entity, no persistence context, no lazy loading.
- Constructor expressions (`SELECT new Dto(...)` in JPQL) are the most efficient — they produce DTO objects directly from the query.
- Dynamic projections (`<T> List<T> findBy(..., Class<T>)`) let the caller choose the projection type.

- Different use cases warrant different approaches: projections for reads, DTOs + mapping for writes.

Interview Questions

Q1: Why should you not return JPA entities from REST controllers?

Exposing entities causes: (1) security risks (password, internal fields leaked), (2) API coupled to database schema, (3) circular references in JSON (entity A references B, B references A), (4) LazyInitializationException during serialization, (5) over-fetching (all fields sent when client needs few).

Q2: What is the difference between interface projections and constructor expressions?

Interface projections define getter methods that Spring proxies to map query results. They work with derived query methods without JPQL. Constructor expressions use `SELECT new DTO(...)` in JPQL to directly instantiate DTO objects. Constructor expressions give more control and create real objects (not proxies), but require explicit JPQL.

Q3: How does MapStruct differ from runtime mapping libraries?

MapStruct generates mapping code at compile time as plain Java method calls. There is no reflection, no runtime overhead, and missing mappings cause compile errors. Runtime libraries (like ModelMapper) use reflection to map fields, which is slower and errors are only caught at runtime.

Q4: When would you use a projection vs loading the entity and mapping?

Use projections for read-only endpoints where you need a subset of fields — they are more efficient (less data from DB, no persistence context). Load the full entity when you need to modify it (update/delete) or when business logic requires the complete object state.

Q5: How do you handle partial updates with DTOs?

Use an update DTO where all fields are optional (nullable). In the service method, check each field for null before updating the entity: `if (request.getName() != null) entity.setName(request.getName())`. MapStruct supports this with `@BeanMapping(nullValuePropertyMappingStrategy = IGNORE)`.

Practice Exercises

Exercise 1: Entity to DTO Create a `Product` entity with 10 fields. Create a `ProductResponse` with 6 fields and a `ProductListItem` with 3 fields. Write manual mapping methods. Test that sensitive fields (costPrice, supplierNotes) are not in the response.

Exercise 2: MapStruct Add MapStruct to your project. Create a `ProductMapper` interface that maps Entity to Response, Request to Entity, and handles partial updates. Verify the generated implementation in `target/generated-sources`.

Exercise 3: Interface Projection Create a `ProductSummaryProjection` interface with `getName()`, `getPrice()`, and `getCategoryName()`. Use it in a repository method. Compare the SQL generated vs loading full entities.

Exercise 4: Constructor Expression Write a JPQL query using `SELECT new ProductReport(p.name, c.name, p.price, p.stockQuantity)` with a JOIN to Category. Verify it produces efficient SQL and returns the correct DTO.

Exercise 5: Complete CRUD with DTOs Build a complete CRUD API for an `Employee` resource: `CreateEmployeeRequest`, `UpdateEmployeeRequest`, `EmployeeResponse`, `EmployeeListItem`. Use projections for the list endpoint, full entity + mapping for detail/create/update. Validate requests. Handle errors.

What Is Next?

In the next chapter, we will explore Repository Pattern, Service Layer, and Architecture — how to organize a production Spring Boot application with proper layering. You will learn about the Controller-Service-Repository architecture, transaction boundaries, custom repository implementations, and base entity classes for shared behavior.

Chapter 24: Repository Pattern, Service Layer, and Architecture

Learning Goals

By the end of this chapter, you will be able to:

- Understand the layered architecture pattern (Controller-Service-Repository)
- Explain the responsibilities of each layer and their boundaries
- Use @Service and @Repository annotations correctly
- Define transaction boundaries at the service layer
- Create custom repository implementations for complex queries
- Build a base entity class for shared audit fields
- Organize packages by feature vs by layer
- Apply the dependency rule: outer layers depend on inner layers
- Avoid common architectural mistakes

Layered Architecture

The standard Spring Boot application follows a three-layer architecture:

```
Layered Architecture:
+-------------------------------------------------------------------+
|                                                                    |
|  +-----------------------------------------------------------+  |
|  |  CONTROLLER LAYER (@RestController)                        |  |
|  |  - Receives HTTP requests                                  |  |
|  |  - Validates request DTOs (@Valid)                          |  |
|  |  - Calls service methods                                   |  |
```

```
|  |  - Returns response DTOs                                    |  |
|  |  - Handles HTTP status codes                                |  |
|  |  - NO business logic, NO database access                    |  |
|  +-------------------------------------------------------------+  |
|       |  DTOs                            ^ DTOs                   |
|       v                                  |                        |
|  +-------------------------------------------------------------+  |
|  |  SERVICE LAYER (@Service)                                   |  |
|  |  - Contains business logic                                  |  |
|  |  - Defines transaction boundaries (@Transactional)          |  |
|  |  - Coordinates between repositories                         |  |
|  |  - Maps entities to/from DTOs                               |  |
|  |  - Validates business rules                                 |  |
|  |  - NO HTTP concerns, NO SQL                                 |  |
|  +-------------------------------------------------------------+  |
|       |  Entities                        ^ Entities               |
|       v                                  |                        |
|  +-------------------------------------------------------------+  |
|  |  REPOSITORY LAYER (@Repository / JpaRepository)             |  |
|  |  - Database access (CRUD, queries)                          |  |
|  |  - Spring Data JPA interfaces                               |  |
|  |  - Custom query implementations                             |  |
|  |  - NO business logic, NO HTTP concerns                      |  |
|  +-------------------------------------------------------------+  |
|       |                                  ^                        |
|       v                                  |                        |
|  +-------------------------------------------------------------+  |
|  |  DATABASE                                                   |  |
|  +-------------------------------------------------------------+  |
|                                                                    |
+--------------------------------------------------------------------+
```

Layer Responsibilities

```
What Goes Where:
+--------------------------------------------------------------------+
|                                                                    |
|  Controller:                  Service:                             |
|  - @GetMapping, @PostMapping  - @Transactional                     |
|  - @Valid @RequestBody        - Business rules                     |
|  - ResponseEntity             - Entity-DTO mapping                |
|  - HTTP status codes          - Coordinate repos                  |
|  - Path variables, params     - Throw business exceptions         |
|  - Exception handling         - Authorization checks              |
|                                                                    |
|  Repository:                  Entity:                              |
|  - extends JpaRepository      - @Entity, @Table                   |
|  - @Query methods             - Field annotations                 |
|  - Specifications             - Relationship mappings             |
|  - Custom implementations     - Lifecycle callbacks               |
|  - Projections                - equals/hashCode                   |
```

```
|                                                                  |
+------------------------------------------------------------------+
```

The Service Layer in Detail

The service layer is where business logic lives. It orchestrates data access and enforces business rules.

```java
@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final ProductRepository productRepository;
    private final CustomerRepository customerRepository;
    private final OrderMapper orderMapper;

    public OrderService(OrderRepository orderRepository,
                        ProductRepository productRepository,
                        CustomerRepository customerRepository,
                        OrderMapper orderMapper) {
        this.orderRepository = orderRepository;
        this.productRepository = productRepository;
        this.customerRepository = customerRepository;
        this.orderMapper = orderMapper;
    }

    @Transactional
    public OrderResponse placeOrder(CreateOrderRequest request) {
        // 1. Business validation
        Customer customer = customerRepository.findById(request.getCustomerId())
            .orElseThrow(() -> new CustomerNotFoundException(request.getCustomerId()));

        if (!customer.isActive()) {
            throw new BusinessException("Inactive customers cannot place orders");
        }

        // 2. Create entity
        Order order = new Order(customer);

        // 3. Process items (business logic)
        for (OrderItemRequest itemReq : request.getItems()) {
            Product product = productRepository.findById(itemReq.getProductId())
                .orElseThrow(() -> new ProductNotFoundException(itemReq.getProductId()));

            if (product.getStock() < itemReq.getQuantity()) {
                throw new InsufficientStockException(product.getName());
```

```
            }

            order.addItem(product, itemReq.getQuantity());
            product.reduceStock(itemReq.getQuantity());
        }

        // 4. Save and return DTO
        Order saved = orderRepository.save(order);
        return orderMapper.toResponse(saved);
    }

    @Transactional(readOnly = true)
    public Page<OrderSummary> getOrders(Pageable pageable) {
        return orderRepository.findOrderSummaries(pageable);
    }

    @Transactional(readOnly = true)
    public OrderResponse getOrder(Long id) {
        Order order = orderRepository.findWithItemsById(id)
            .orElseThrow(() -> new OrderNotFoundException(id));
        return orderMapper.toResponse(order);
    }
}
```

```
Transaction Boundaries:
+-------------------------------------------------------------------+
|                                                                   |
|  Controller (NO @Transactional):                                  |
|  +-----------------------------------------------------------+    |
|  | OrderResponse response = orderService.placeOrder(request);|    |
|  | return ResponseEntity.created(uri).body(response);        |    |
|  +-----------------------------------------------------------+    |
|                                                                   |
|  Service (@Transactional):                                        |
|  +-----------------------------------------------------------+    |
|  | [Transaction starts]                                      |    |
|  | Customer customer = customerRepo.findById(id)             |    |
|  | Product product = productRepo.findById(id)                |    |
|  | order.addItem(product, qty)                               |    |
|  | product.reduceStock(qty)                                  |    |
|  | orderRepo.save(order)                                     |    |
|  | [Dirty checking: UPDATE products SET stock = ...]          |    |
|  | [INSERT INTO orders ...]                                  |    |
|  | [INSERT INTO order_items ...]                              |    |
|  | [Transaction commits]                                     |    |
|  +-----------------------------------------------------------+    |
|                                                                   |
|  ALL operations in one transaction:                               |
|  If stock reduction fails, order creation rolls back too.        |
|  Atomicity across multiple repositories.                          |
+-------------------------------------------------------------------+
```

The Repository Layer

Standard Repository

```java
@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {

    // Derived query methods
    List<Order> findByCustomerId(Long customerId);
    List<Order> findByStatus(OrderStatus status);
    boolean existsByCustomerIdAndStatus(Long customerId, OrderStatus status);

    // JPQL queries
    @Query("SELECT o FROM Order o LEFT JOIN FETCH o.items WHERE o.id = :id")
    Optional<Order> findWithItemsById(@Param("id") Long id);

    // Projections
    @Query("SELECT new com.example.dto.OrderSummary(" +
           "o.id, c.name, o.total, o.status, o.createdAt) " +
           "FROM Order o JOIN o.customer c")
    Page<OrderSummary> findOrderSummaries(Pageable pageable);

    // Modifying queries
    @Modifying
    @Query("UPDATE Order o SET o.status = :status WHERE o.id = :id")
    int updateStatus(@Param("id") Long id, @Param("status") OrderStatus status);
}
```

Custom Repository Implementation

When Spring Data methods are not enough, add custom implementations:

```java
// Step 1: Define custom interface
public interface OrderRepositoryCustom {
    List<Order> findByComplexCriteria(OrderSearchCriteria criteria);
    OrderStatistics calculateStatistics(LocalDate from, LocalDate to);
}
```

```java
// Step 2: Implement it
public class OrderRepositoryCustomImpl implements OrderRepositoryCustom {

    @PersistenceContext
    private EntityManager entityManager;
```

```
    @Override
    public List<Order> findByComplexCriteria(OrderSearchCriteria criteria) {
        CriteriaBuilder cb = entityManager.getCriteriaBuilder();
        CriteriaQuery<Order> query = cb.createQuery(Order.class);
        Root<Order> root = query.from(Order.class);

        List<Predicate> predicates = new ArrayList<>();

        if (criteria.getCustomerId() != null) {
            predicates.add(cb.equal(root.get("customer").get("id"),
                criteria.getCustomerId()));
        }
        if (criteria.getMinTotal() != null) {
            predicates.add(cb.greaterThanOrEqualTo(root.get("total"),
                criteria.getMinTotal()));
        }
        if (criteria.getStatus() != null) {
            predicates.add(cb.equal(root.get("status"), criteria.getStatus()));
        }
        if (criteria.getFromDate() != null) {
            predicates.add(cb.greaterThanOrEqualTo(root.get("createdAt"),
                criteria.getFromDate()));
        }

        query.where(predicates.toArray(new Predicate[0]));
        query.orderBy(cb.desc(root.get("createdAt")));

        return entityManager.createQuery(query).getResultList();
    }

    @Override
    public OrderStatistics calculateStatistics(LocalDate from, LocalDate to) {
        String jpql = "SELECT new com.example.dto.OrderStatistics(" +
            "COUNT(o), SUM(o.total), AVG(o.total), MIN(o.total), MAX(o.total)) " +
            "FROM Order o WHERE o.createdAt BETWEEN :from AND :to";

        return entityManager.createQuery(jpql, OrderStatistics.class)
            .setParameter("from", from.atStartOfDay())
            .setParameter("to", to.plusDays(1).atStartOfDay())
            .getSingleResult();
    }
}
```

```java
// Step 3: Extend both interfaces in the main repository
@Repository
public interface OrderRepository
        extends JpaRepository<Order, Long>, OrderRepositoryCustom {
    // Spring Data methods + custom methods available together
}
```

```
Custom Repository Naming Convention:
+-------------------------------------------------------------------+
|                                                                   |
|  Main interface:   OrderRepository                                |
|  Custom interface: OrderRepositoryCustom                          |
|  Implementation:   OrderRepositoryCustomImpl                      |
|                                                                   |
|  The Impl suffix is REQUIRED by convention.                       |
|  Spring Data finds OrderRepositoryCustomImpl automatically        |
|  and delegates custom method calls to it.                         |
|                                                                   |
|  OrderRepository                                                  |
|  +-- extends JpaRepository (Spring Data auto-implements)         |
|  +-- extends OrderRepositoryCustom                               |
|      +-- OrderRepositoryCustomImpl (you implement)               |
+-------------------------------------------------------------------+
```

Base Entity Class

Share common fields across all entities with `@MappedSuperclass`:

```java
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @CreatedDate
    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @Version   // Optimistic locking
    private Integer version;

    public Long getId() { return id; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public LocalDateTime getUpdatedAt() { return updatedAt; }
    public Integer getVersion() { return version; }

    @Override
```

```
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        BaseEntity that = (BaseEntity) o;
        return id != null && id.equals(that.id);
    }

    @Override
    public int hashCode() {
        return getClass().hashCode();  // Stable across new/managed states
    }
}
```

```java
// All entities extend BaseEntity
@Entity
@Table(name = "products")
public class Product extends BaseEntity {

    @NotBlank
    private String name;

    @Positive
    private BigDecimal price;

    // id, createdAt, updatedAt, version inherited automatically
}
```

```
BaseEntity Fields:
+-------------------------------------------------------------------+
|                                                                   |
|  Field        Source              Purpose                         |
|  ---------------------------------------------------------------|
|  id           @GeneratedValue     Primary key                     |
|  createdAt    @CreatedDate        When row was created            |
|  updatedAt    @LastModifiedDate   When row was last modified      |
|  version      @Version            Optimistic locking              |
|                                   (prevents lost updates)         |
|                                                                   |
|  Every entity gets these 4 fields for free.                       |
|  Just extend BaseEntity instead of managing them per entity.     |
+-------------------------------------------------------------------+
```

Package Organization

By Layer (Traditional)

```
By Layer:
+-------------------------------------------------------------------+
|                                                                   |
|  com.example.myapp/                                               |
|  +-- controller/                                                  |
|  |   +-- EmployeeController.java                                 |
|  |   +-- DepartmentController.java                               |
|  |   +-- OrderController.java                                    |
|  +-- service/                                                     |
|  |   +-- EmployeeService.java                                    |
|  |   +-- DepartmentService.java                                  |
|  |   +-- OrderService.java                                       |
|  +-- repository/                                                  |
|  |   +-- EmployeeRepository.java                                 |
|  |   +-- DepartmentRepository.java                               |
|  |   +-- OrderRepository.java                                    |
|  +-- entity/                                                      |
|  |   +-- Employee.java                                           |
|  |   +-- Department.java                                         |
|  |   +-- Order.java                                              |
|  +-- dto/                                                         |
|      +-- EmployeeResponse.java                                   |
|      +-- CreateEmployeeRequest.java                              |
|      +-- ...                                                      |
|                                                                   |
|  Problem: Related classes are scattered across packages.          |
|  To understand "Employee", you check 5 packages.                 |
+-------------------------------------------------------------------+
```

By Feature (Recommended for Larger Projects)

```
By Feature:
+-------------------------------------------------------------------+
|                                                                   |
|  com.example.myapp/                                               |
|  +-- employee/                                                    |
|  |   +-- Employee.java                                           |
|  |   +-- EmployeeRepository.java                                 |
|  |   +-- EmployeeService.java                                    |
|  |   +-- EmployeeController.java                                 |
|  |   +-- dto/                                                    |
|  |       +-- CreateEmployeeRequest.java                          |
|  |       +-- EmployeeResponse.java                               |
|  +-- department/                                                  |
|  |   +-- Department.java                                         |
|  |   +-- DepartmentRepository.java                               |
|  |   +-- DepartmentService.java                                  |
|  |   +-- DepartmentController.java                               |
|  |   +-- dto/                                                    |
|  +-- order/                                                       |
|  |   +-- Order.java                                              |
```

```
|  |   +-- OrderItem.java                                          |
|  |   +-- OrderRepository.java                                   |
|  |   +-- OrderService.java                                      |
|  |   +-- OrderController.java                                   |
|  |   +-- dto/                                                    |
|  +-- common/                                                     |
|      +-- BaseEntity.java                                         |
|      +-- GlobalExceptionHandler.java                             |
|                                                                  |
|  Benefit: Everything about "Employee" is in one place.           |
|  Easier to navigate, understand, and modify.                     |
+------------------------------------------------------------------+
```

```
Choosing Package Structure:
+------------------------------------------------------------------+
|                                                                   |
|  Small project (< 10 entities):   By layer is fine                |
|  Medium project (10-30 entities): By feature recommended          |
|  Large project (30+ entities):    By feature essential             |
|                                                                   |
|  You can mix: feature packages for domain,                       |
|  shared package for cross-cutting concerns.                       |
+------------------------------------------------------------------+
```

Dependency Rules

```
Dependency Direction:
+------------------------------------------------------------------+
|                                                                   |
|  Controller --> Service --> Repository --> Database                |
|                                                                   |
|  Rules:                                                           |
|  1. Controllers depend on Services (never on Repositories)        |
|  2. Services depend on Repositories (never on Controllers)        |
|  3. Repositories depend on Entities (never on Services)           |
|  4. Entities depend on nothing (pure JPA annotations)             |
|                                                                   |
|  Violations to avoid:                                             |
|  X Controller calling Repository directly (skips business logic)  |
|  X Repository calling Service (inverted dependency)               |
|  X Entity depending on Service (entity should be passive)         |
|  X Service depending on Controller (inverted dependency)          |
+------------------------------------------------------------------+
```

Common Mistakes

1. Putting business logic in controllers: Controllers should only handle HTTP concerns. Move validation, calculations, and orchestration to the service layer.
2. Calling repositories directly from controllers: This bypasses the service layer, losing transaction management and business rule enforcement.
3. Having services call other services excessively: Deep service-to-service chains create complex call graphs. Prefer having services coordinate through repositories, not through other services.
4. Putting @Transactional on repositories: Spring Data repositories already run each method in a transaction. The service layer defines the business transaction boundary spanning multiple repository calls.
5. Fat entities with business logic: Entities should be data holders with JPA annotations. Business rules belong in services.
6. Creating a service per repository method: A service method should represent a business operation, not a thin wrapper around a repository call.

Best Practices

1. One service per domain concept: `EmployeeService`, `OrderService`, `PaymentService`. Not one service per entity if entities are related.
2. @Transactional on service methods, not repositories: The service defines what constitutes a business transaction (e.g., place order = check stock + create order + reduce inventory).
3. Use constructor injection: Inject dependencies via constructor, not `@Autowired` fields. This makes dependencies explicit and testing easier.
4. Return DTOs from service methods: Services should convert entities to DTOs before returning to the controller. Entities should not leak out of the service layer.
5. Use BaseEntity for shared fields: id, createdAt, updatedAt, version — define once, inherit everywhere.
6. Organize by feature for growing projects: Group all related classes (entity, repository, service, controller, DTOs) in a feature package.

Summary

- The three-layer architecture (Controller-Service-Repository) separates concerns: HTTP handling, business logic, and data access.
- Controllers handle HTTP, validate input, and return responses. No business logic.
- Services contain business logic, define transaction boundaries with `@Transactional`, coordinate repositories, and map between entities and DTOs.
- Repositories handle data access. Use Spring Data interfaces for standard operations and custom implementations for complex queries.
- Custom repositories use the `Impl` suffix convention. The main repository extends both `JpaRepository` and the custom interface.
- BaseEntity with `@MappedSuperclass` provides shared fields (id, timestamps, version) across all entities.
- Package by feature groups related classes together, improving navigability in larger projects.
- Dependencies flow inward: Controller -> Service -> Repository -> Database. Never violate this direction.

Interview Questions

Q1: What are the responsibilities of the service layer?

The service layer contains business logic, defines transaction boundaries, coordinates between multiple repositories, validates business rules, maps entities to/from DTOs, and enforces authorization. It does not handle HTTP concerns (that is the controller) or write SQL (that is the repository).

Q2: Why should @Transactional be on the service layer, not the repository?

The service layer defines business transactions that may span multiple repository operations. For example, placing an order involves checking stock, creating the order, and reducing inventory — all of which must succeed or fail together. Repository methods already run in individual transactions.

Q3: How do custom repository implementations work in Spring Data?

Create a custom interface (e.g., `OrderRepositoryCustom`), implement it in a class with the `Impl` suffix (e.g., `OrderRepositoryCustomImpl`), and have the main repository extend both `JpaRepository` and the custom interface. Spring Data detects the `Impl` class and delegates custom method calls to it.

Q4: What is the advantage of packaging by feature over packaging by layer?

Feature-based packaging keeps all related classes together (entity, repository, service, controller, DTOs in one package). This makes it easier to understand a feature, navigate the codebase, and enforce boundaries. Layer-based packaging scatters related classes across packages.

Q5: Why should entities not be returned from controllers?

Entities expose internal structure (sensitive fields, database schema), cause circular reference issues during JSON serialization, trigger lazy loading exceptions outside transactions, and tightly couple the API to the database schema. DTOs provide a clean, stable API contract.

Practice Exercises

Exercise 1: Layered Architecture Build a complete `Product` CRUD with proper layering: controller (HTTP only), service (business logic, DTO mapping), repository (data access). Verify no layer violates its responsibilities.

Exercise 2: Custom Repository Create a `ProductRepositoryCustom` with a `search(ProductSearchCriteria)` method using Criteria API. Support filters for name, category, price range, and active status — all optional.

Exercise 3: BaseEntity Create a `BaseEntity` with id, createdAt, updatedAt, and version. Create 3 entities extending it. Verify all entities get audit timestamps automatically.

Exercise 4: Package by Feature Refactor an existing project from layer-based to feature-based packaging. Verify that everything still works and that the package structure is clearer.

Exercise 5: Service Orchestration Create an `OrderService.placeOrder()` method that coordinates `CustomerRepository`, `ProductRepository`, and `OrderRepository`. Enforce business rules: customer must be active, stock must be sufficient. Verify atomic transaction behavior (all succeed or all rollback).

What Is Next?

In the next chapter, we will explore Error Handling and Exception Translation — how Spring translates database exceptions, how to create a consistent error response format, and how to

build a global exception handler with `@ControllerAdvice` for clean error responses across your entire API.

Chapter 25: Error Handling and Exception Translation

Learning Goals

By the end of this chapter, you will be able to:

- Understand Spring's persistence exception translation mechanism
- Handle common JPA/Hibernate exceptions (DataIntegrityViolation, OptimisticLock, etc.)
- Create custom business exceptions with meaningful messages
- Build a global exception handler with @ControllerAdvice
- Design a consistent error response format for REST APIs
- Map database constraint violations to user-friendly messages
- Handle validation errors from Bean Validation
- Use Problem Details (RFC 9457) for standardized error responses

Spring's Exception Translation

Spring automatically translates low-level JPA/Hibernate exceptions into Spring's `DataAccessException` hierarchy. This happens because of `@Repository` and Spring's persistence exception translation.

```
Exception Translation Flow:
+-------------------------------------------------------------------+
|                                                                   |
|  Your Code                                                        |
|  repository.save(employee);                                       |
|       |                                                           |
```

```
|       v                                                          |
|  Hibernate throws:                                               |
|  jakarta.persistence.PersistenceException                        |
|  org.hibernate.exception.ConstraintViolationException            |
|       |                                                          |
|       v                                                          |
|  Spring @Repository translates to:                               |
|  org.springframework.dao.DataIntegrityViolationException         |
|       |                                                          |
|       v                                                          |
|  Your service catches Spring's exception                         |
|  (not Hibernate's — your code is provider-independent)           |
+------------------------------------------------------------------+
```

Common Spring Data Access Exceptions

```
Spring Exception Hierarchy:
+------------------------------------------------------------------+
|                                                                  |
|  DataAccessException (root)                                      |
|  |                                                               |
|  +-- NonTransientDataAccessException                             |
|  |   |  (permanent errors — retrying won't help)                 |
|  |   |                                                           |
|  |   +-- DataIntegrityViolationException                         |
|  |   |   Unique constraint violation                             |
|  |   |   Foreign key violation                                   |
|  |   |   NOT NULL violation                                      |
|  |   |                                                           |
|  |   +-- EmptyResultDataAccessException                          |
|  |   |   Expected one result, got none                           |
|  |   |                                                           |
|  |   +-- IncorrectResultSizeDataAccessException                  |
|  |       Expected one result, got many                           |
|  |                                                               |
|  +-- TransientDataAccessException                                |
|  |   |  (temporary errors — retrying might help)                 |
|  |   |                                                           |
|  |   +-- CannotAcquireLockException                              |
|  |   |   Row or table lock timeout                               |
|  |   |                                                           |
|  |   +-- OptimisticLockingFailureException                       |
|  |       @Version mismatch (concurrent update detected)          |
|  |                                                               |
|  +-- InvalidDataAccessApiUsageException                          |
|      Wrong API usage (e.g., query syntax error)                  |
+------------------------------------------------------------------+
```

Custom Business Exceptions

Create domain-specific exceptions that communicate what went wrong in business terms:

```java
// Base business exception
public class BusinessException extends RuntimeException {

    private final String errorCode;

    public BusinessException(String message) {
        super(message);
        this.errorCode = "BUSINESS_ERROR";
    }

    public BusinessException(String errorCode, String message) {
        super(message);
        this.errorCode = errorCode;
    }

    public String getErrorCode() { return errorCode; }
}

// Specific exceptions
public class ResourceNotFoundException extends BusinessException {

    public ResourceNotFoundException(String resource, Long id) {
        super("NOT_FOUND",
              String.format("%s with id %d not found", resource, id));
    }

    public ResourceNotFoundException(String resource, String field, String value) {
        super("NOT_FOUND",
              String.format("%s with %s '%s' not found", resource, field, value));
    }
}

public class DuplicateResourceException extends BusinessException {

    public DuplicateResourceException(String resource, String field, String value) {
        super("DUPLICATE",
              String.format("%s with %s '%s' already exists", resource, field, value));
    }
}

public class InsufficientStockException extends BusinessException {

    public InsufficientStockException(String productName, int requested, int available) {
        super("INSUFFICIENT_STOCK",
              String.format("Product '%s': requested %d but only %d available",
                  productName, requested, available));
    }
}
```

Using in Services

```java
@Service
public class EmployeeService {

    @Transactional
    public EmployeeResponse create(CreateEmployeeRequest request) {
        // Check for duplicate email
        if (employeeRepository.existsByEmail(request.getEmail())) {
            throw new DuplicateResourceException("Employee", "email", request.getEmail());
        }

        Department dept = departmentRepository.findById(request.getDepartmentId())
            .orElseThrow(() -> new ResourceNotFoundException("Department",
                request.getDepartmentId()));

        Employee employee = new Employee(request.getName(), request.getEmail());
        employee.setDepartment(dept);
        return mapper.toResponse(employeeRepository.save(employee));
    }

    @Transactional(readOnly = true)
    public EmployeeResponse getById(Long id) {
        Employee employee = employeeRepository.findWithDeptById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Employee", id));
        return mapper.toResponse(employee);
    }
}
```

Consistent Error Response Format

Define a standard error response structure for your API:

```java
public class ErrorResponse {

    private int status;
    private String error;
    private String message;
    private String path;
    private LocalDateTime timestamp;
    private Map<String, String> fieldErrors;  // For validation errors

    public ErrorResponse(int status, String error, String message, String path) {
        this.status = status;
        this.error = error;
```

```
        this.message = message;
        this.path = path;
        this.timestamp = LocalDateTime.now();
    }

    public void setFieldErrors(Map<String, String> fieldErrors) {
        this.fieldErrors = fieldErrors;
    }

    // Getters...
}
```

```
Error Response Examples:
+-------------------------------------------------------------------+
|                                                                   |
|  404 Not Found:                                                   |
|  {                                                                |
|    "status": 404,                                                 |
|    "error": "NOT_FOUND",                                          |
|    "message": "Employee with id 42 not found",                    |
|    "path": "/api/employees/42",                                   |
|    "timestamp": "2025-09-15T14:30:00"                             |
|  }                                                                |
|                                                                   |
|  400 Validation Error:                                            |
|  {                                                                |
|    "status": 400,                                                 |
|    "error": "VALIDATION_FAILED",                                  |
|    "message": "Request validation failed",                        |
|    "path": "/api/employees",                                      |
|    "timestamp": "2025-09-15T14:30:00",                            |
|    "fieldErrors": {                                               |
|      "name": "Name is required",                                  |
|      "email": "Must be a valid email",                            |
|      "salary": "Salary must be positive"                          |
|    }                                                              |
|  }                                                                |
|                                                                   |
|  409 Conflict:                                                    |
|  {                                                                |
|    "status": 409,                                                 |
|    "error": "DUPLICATE",                                          |
|    "message": "Employee with email 'a@co.com' already exists",    |
|    "path": "/api/employees",                                      |
|    "timestamp": "2025-09-15T14:30:00"                             |
|  }                                                                |
+-------------------------------------------------------------------+
```

Global Exception Handler with @ControllerAdvice

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    // === Business Exceptions ===

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
            ResourceNotFoundException ex, HttpServletRequest request) {
        ErrorResponse error = new ErrorResponse(
            404, ex.getErrorCode(), ex.getMessage(), request.getRequestURI());
        return ResponseEntity.status(404).body(error);
    }

    @ExceptionHandler(DuplicateResourceException.class)
    public ResponseEntity<ErrorResponse> handleDuplicate(
            DuplicateResourceException ex, HttpServletRequest request) {
        ErrorResponse error = new ErrorResponse(
            409, ex.getErrorCode(), ex.getMessage(), request.getRequestURI());
        return ResponseEntity.status(409).body(error);
    }

    @ExceptionHandler(BusinessException.class)
    public ResponseEntity<ErrorResponse> handleBusiness(
            BusinessException ex, HttpServletRequest request) {
        ErrorResponse error = new ErrorResponse(
            422, ex.getErrorCode(), ex.getMessage(), request.getRequestURI());
        return ResponseEntity.status(422).body(error);
    }

    // === Validation Exceptions ===

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(
            MethodArgumentNotValidException ex, HttpServletRequest request) {

        Map<String, String> fieldErrors = new LinkedHashMap<>();
        for (FieldError fe : ex.getBindingResult().getFieldErrors()) {
            fieldErrors.put(fe.getField(), fe.getDefaultMessage());
        }

        ErrorResponse error = new ErrorResponse(
            400, "VALIDATION_FAILED", "Request validation failed",
            request.getRequestURI());
        error.setFieldErrors(fieldErrors);

        return ResponseEntity.badRequest().body(error);
    }

    // === JPA / Database Exceptions ===
```

```
    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<ErrorResponse> handleDataIntegrity(
            DataIntegrityViolationException ex, HttpServletRequest request) {

        String message = "Data integrity violation";

        // Extract meaningful message from constraint name
        String rootMessage = ex.getMostSpecificCause().getMessage();
        if (rootMessage != null) {
            if (rootMessage.contains("UNIQUE") || rootMessage.contains("unique")) {
                message = "A record with this value already exists";
            } else if (rootMessage.contains("FOREIGN KEY")) {
                message = "Referenced record does not exist or cannot be deleted";
            } else if (rootMessage.contains("NULL")) {
                message = "A required field is missing";
            }
        }

        ErrorResponse error = new ErrorResponse(
            409, "DATA_INTEGRITY", message, request.getRequestURI());
        return ResponseEntity.status(409).body(error);
    }

    @ExceptionHandler(OptimisticLockingFailureException.class)
    public ResponseEntity<ErrorResponse> handleOptimisticLock(
            OptimisticLockingFailureException ex, HttpServletRequest request) {
        ErrorResponse error = new ErrorResponse(
            409, "CONCURRENT_UPDATE",
            "This record was modified by another user. Please refresh and try again.",
            request.getRequestURI());
        return ResponseEntity.status(409).body(error);
    }

    // === Catch-All ===

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneral(
            Exception ex, HttpServletRequest request) {
        // Log the full exception for debugging
        log.error("Unhandled exception", ex);

        ErrorResponse error = new ErrorResponse(
            500, "INTERNAL_ERROR", "An unexpected error occurred",
            request.getRequestURI());
        return ResponseEntity.status(500).body(error);
    }

    private static final org.slf4j.Logger log =
        org.slf4j.LoggerFactory.getLogger(GlobalExceptionHandler.class);
}
```

```
Exception Handler Priority:
+-----------------------------------------------------------------+
```

```
|                                                                    |
|  Exception thrown in service layer                                 |
|       |                                                            |
|       v                                                            |
|  Spring looks for matching @ExceptionHandler:                      |
|                                                                    |
|  1. Most specific match first:                                     |
|     ResourceNotFoundException handler (matches exactly)            |
|                                                                    |
|  2. Parent class match:                                            |
|     BusinessException handler (if no specific handler)             |
|                                                                    |
|  3. Catch-all:                                                     |
|     Exception handler (last resort)                                |
|                                                                    |
|  Order matters! Put specific handlers before general ones.         |
+--------------------------------------------------------------------+
```

Handling Constraint Violations from Bean Validation

When validation happens at the entity level (not controller), `ConstraintViolationException` is thrown:

```java
@ExceptionHandler(ConstraintViolationException.class)
public ResponseEntity<ErrorResponse> handleConstraintViolation(
        ConstraintViolationException ex, HttpServletRequest request) {

    Map<String, String> fieldErrors = new LinkedHashMap<>();
    for (var violation : ex.getConstraintViolations()) {
        String field = violation.getPropertyPath().toString();
        fieldErrors.put(field, violation.getMessage());
    }

    ErrorResponse error = new ErrorResponse(
        400, "VALIDATION_FAILED", "Constraint validation failed",
        request.getRequestURI());
    error.setFieldErrors(fieldErrors);

    return ResponseEntity.badRequest().body(error);
}
```

Mapping Constraint Names to User Messages

For database-level constraint violations, map constraint names to friendly messages:

```java
@Component
public class ConstraintMessageMapper {

    private final Map<String, String> constraintMessages = Map.of(
        "UK_EMPLOYEE_EMAIL", "An employee with this email already exists",
        "UK_DEPARTMENT_NAME", "A department with this name already exists",
        "FK_EMPLOYEE_DEPARTMENT", "The specified department does not exist",
        "UK_PRODUCT_SKU", "A product with this SKU already exists"
    );

    public String getMessage(DataIntegrityViolationException ex) {
        String rootMessage = ex.getMostSpecificCause().getMessage();

        for (Map.Entry<String, String> entry : constraintMessages.entrySet()) {
            if (rootMessage != null &&
                rootMessage.toUpperCase().contains(entry.getKey())) {
                return entry.getValue();
            }
        }

        return "A data integrity constraint was violated";
    }
}
```

```java
// Use in exception handler
@ExceptionHandler(DataIntegrityViolationException.class)
public ResponseEntity<ErrorResponse> handleDataIntegrity(
        DataIntegrityViolationException ex, HttpServletRequest request) {
    String message = constraintMessageMapper.getMessage(ex);
    ErrorResponse error = new ErrorResponse(409, "DATA_INTEGRITY", message,
        request.getRequestURI());
    return ResponseEntity.status(409).body(error);
}
```

Complete Exception Handling Flow

```
End-to-End Error Handling:
+-----------------------------------------------------------------+
|                                                                 |
|  Client sends: POST /api/employees                              |
```

```
|  { "name": "", "email": "alice@co.com", "salary": -5 }           |
|       |                                                           |
|       v                                                           |
|  @Valid catches validation errors BEFORE service is called:       |
|  --> 400 { "fieldErrors": { "name": "required", "salary": ">0" }}|
|                                                                   |
|  Client sends: POST /api/employees                                |
|  { "name": "Alice", "email": "existing@co.com", "salary": 80000 }|
|       |                                                           |
|       v                                                           |
|  Service checks: existsByEmail("existing@co.com") == true         |
|  Throws: DuplicateResourceException                               |
|  --> 409 { "error": "DUPLICATE", "message": "already exists" }   |
|                                                                   |
|  Client sends: GET /api/employees/999                             |
|       |                                                           |
|       v                                                           |
|  Service: findById(999) returns empty                             |
|  Throws: ResourceNotFoundException                                |
|  --> 404 { "error": "NOT_FOUND", "message": "not found" }        |
|                                                                   |
|  Client sends: PUT /api/employees/1 (with stale version)         |
|       |                                                           |
|       v                                                           |
|  Hibernate: @Version mismatch                                     |
|  Throws: OptimisticLockingFailureException                        |
|  --> 409 { "error": "CONCURRENT_UPDATE", "message": "refresh" }  |
|                                                                   |
|  Unexpected bug in code:                                          |
|       |                                                           |
|       v                                                           |
|  Throws: NullPointerException (unhandled)                         |
|  Catch-all handler logs it and returns:                           |
|  --> 500 { "error": "INTERNAL_ERROR", "message": "unexpected" }  |
+-------------------------------------------------------------------+
```

Common Mistakes

1. Returning stack traces to clients: Never include exception stack traces in API responses. Log them server-side, return user-friendly messages to clients.
2. Using generic 500 for all errors: Differentiate between 400 (client error), 404 (not found), 409 (conflict), 422 (business rule), and 500 (server error).
3. Catching exceptions silently in services: Swallowing exceptions inside `@Transactional` prevents rollback and hides problems. Rethrow or handle explicitly.
4. Not handling DataIntegrityViolationException: If you do not check for duplicate emails before saving, the database unique constraint throws `DataIntegrityViolationException`.

Handle it with a friendly message.

5. Inconsistent error response formats: Some endpoints return `{"error": "..."}`, others return `{"message": "..."}`. Use one consistent format everywhere.
6. Not logging unexpected exceptions: The catch-all handler should always log the full exception. Without logging, production bugs are invisible.

Best Practices

1. Create domain-specific exceptions: `ResourceNotFoundException`, `DuplicateResourceException`, `InsufficientStockException` communicate intent clearly.
2. Use @ControllerAdvice for centralized handling: One class handles all exceptions consistently. No try-catch in controllers.
3. Validate at the boundary first: Check for duplicates and invalid references in the service layer before calling save. This gives better error messages than database constraint violations.
4. Design a consistent error response format: Same structure for all errors (status, error code, message, timestamp, path). Frontend can parse any error the same way.
5. Map HTTP status codes correctly: 400 for validation, 404 for not found, 409 for conflicts/duplicates, 422 for business rule violations, 500 for unexpected errors.
6. Log server errors, not client errors: Log 500s with full stack trace. Do not log 400s and 404s (these are expected client behavior).

Summary

- Spring exception translation converts Hibernate/JPA exceptions into Spring's `DataAccessException` hierarchy, keeping your code provider-independent.
- Custom business exceptions communicate domain problems (not found, duplicate, insufficient stock) with meaningful error codes and messages.
- @ControllerAdvice centralizes exception handling. Each `@ExceptionHandler` maps an exception type to an HTTP status and error response.
- Consistent error response format with status, error code, message, timestamp, and path lets frontends handle errors uniformly.
- Validation errors from `@Valid` produce field-level error maps. Database constraint violations should be mapped to friendly messages.

- Optimistic locking failures (`@Version` mismatch) return 409 with a "refresh and retry" message.
- Catch-all handler logs unexpected exceptions and returns 500 without exposing internals.

Interview Questions

Q1: How does Spring translate Hibernate exceptions?

The `@Repository` annotation enables Spring's persistence exception translation. When Hibernate throws a `PersistenceException` or its subclasses, Spring's `PersistenceExceptionTranslationPostProcessor` catches it and wraps it in a Spring `DataAccessException` subclass. This keeps your service code independent of the JPA provider.

Q2: What is the difference between DataIntegrityViolationException and ConstraintViolationException?

`DataIntegrityViolationException` (Spring) is thrown when a database constraint is violated (unique, FK, NOT NULL) — it comes from the database after SQL execution. `ConstraintViolationException` (Jakarta Validation) is thrown when Bean Validation annotations fail — it comes from Java validation before SQL is executed.

Q3: How does @ControllerAdvice work?

`@ControllerAdvice` creates a global exception handler that applies to all controllers. Methods annotated with `@ExceptionHandler(SomeException.class)` catch that exception type thrown by any controller method. Spring selects the most specific handler first.

Q4: What HTTP status codes should you use for common errors?

400 for validation errors, 404 for resource not found, 409 for conflicts (duplicates, optimistic lock failures), 422 for business rule violations, 500 for unexpected server errors. Never use 200 for errors.

Q5: Why should you not return stack traces in API responses?

Stack traces expose internal implementation details (class names, library versions, file paths) that could be exploited by attackers. They also confuse API consumers. Log stack traces server-side and return user-friendly messages to clients.

Practice Exercises

Exercise 1: Custom Exceptions Create `ResourceNotFoundException`, `DuplicateResourceException`, and `InsufficientFundsException`. Use them in a `BankAccountService` with transfer, create, and find operations.

Exercise 2: Global Exception Handler Build a `@ControllerAdvice` that handles your custom exceptions, `MethodArgumentNotValidException`, `DataIntegrityViolationException`, and a catch-all. Test each handler returns the correct HTTP status and error format.

Exercise 3: Constraint Mapping Create a `ConstraintMessageMapper` that maps database constraint names to friendly messages. Test with unique constraint violations on email and department name.

Exercise 4: Optimistic Locking Add `@Version` to an entity. Write a test that simulates concurrent updates: load the entity in two transactions, modify both, save both. Verify `OptimisticLockingFailureException` is thrown and handled with 409.

Exercise 5: End-to-End Error Testing Write integration tests for every error scenario: validation error (400), not found (404), duplicate (409), concurrent update (409), and unexpected error (500). Verify consistent response format for all.

What Is Next?

In the next chapter, we will explore Database Migrations with Flyway — how to manage database schema changes in a production-safe way. You will learn why `ddl-auto=update` is dangerous in production, how Flyway versioned migrations work, naming conventions, and how to integrate Flyway with Spring Boot for automatic migration execution.

Chapter 26: Database Migrations with Flyway

Learning Goals

By the end of this chapter, you will be able to:

- Explain why ddl-auto=update is dangerous in production
- Understand database migrations and why they are essential
- Set up Flyway in a Spring Boot project
- Write versioned migration scripts with proper naming conventions
- Add columns, create tables, insert seed data, and alter constraints
- Use repeatable migrations for views and stored procedures
- Baseline an existing database for Flyway adoption
- Handle migration errors and resolve failed states
- Integrate Flyway with your development workflow

Why Not ddl-auto=update?

During development, you have been using `spring.jpa.hibernate.ddl-auto=update`, which lets Hibernate automatically modify the database schema. This is convenient for learning but dangerous in production.

```
ddl-auto Options:
+-------------------------------------------------------------------+
|                                                                    |
|  Value        What It Does                    Safe for Prod?      |
|  ------------------------------------------------------------------|
```

```
|  create       Drops ALL tables, recreates     NEVER (data loss!)  |
|  create-drop  Like create, drops on shutdown  NEVER               |
|  update       Adds missing columns/tables     DANGEROUS           |
|  validate     Checks schema matches entities  YES (read-only)     |
|  none         Does nothing                    YES                 |
+-------------------------------------------------------------------+

Why ddl-auto=update Is Dangerous:
+-------------------------------------------------------------------+
|                                                                   |
|  1. It NEVER deletes columns or tables                            |
|     Remove a field from your entity --> column stays in DB        |
|     Orphaned columns accumulate forever                           |
|                                                                   |
|  2. It NEVER renames columns                                      |
|     Rename a field --> old column stays, new column created       |
|     Data in old column is lost                                    |
|                                                                   |
|  3. It can ALTER columns in unexpected ways                       |
|     Change @Column(length=100) to length=200                      |
|     --> ALTER TABLE may lock the table for minutes on large data  |
|                                                                   |
|  4. No rollback capability                                        |
|     If a schema change breaks something, you cannot undo it      |
|                                                                   |
|  5. No audit trail                                                |
|     No record of what changed, when, or why                      |
|                                                                   |
|  6. Different environments drift apart                            |
|     Dev, staging, and production may have different schemas       |
|     because updates ran at different times                        |
+-------------------------------------------------------------------+
```

What Are Database Migrations?

Database migrations are versioned SQL scripts that evolve your schema in a controlled, repeatable, and auditable way. Each script runs once, in order, and is tracked so it never runs again.

```
Migration Concept:
+-------------------------------------------------------------------+
|                                                                   |
|  V1__Create_employees_table.sql                                   |
|  CREATE TABLE employees (                                         |
|      id BIGINT PRIMARY KEY AUTO_INCREMENT,                        |
|      name VARCHAR(100) NOT NULL                                   |
|  );                                                               |
```

```
|       |                                                          |
|       v                                                          |
|  V2__Add_email_column.sql                                        |
|  ALTER TABLE employees ADD COLUMN email VARCHAR(150);            |
|       |                                                          |
|       v                                                          |
|  V3__Add_departments_table.sql                                   |
|  CREATE TABLE departments (...);                                 |
|  ALTER TABLE employees ADD COLUMN department_id BIGINT;           |
|       |                                                          |
|       v                                                          |
|  V4__Add_salary_column.sql                                       |
|  ALTER TABLE employees ADD COLUMN salary DECIMAL(10,2);          |
|                                                                  |
|  Each migration builds on the previous one.                      |
|  Run them in order on any database to get the current schema.    |
|  Every environment ends up with the exact same structure.        |
+------------------------------------------------------------------+
```

Setting Up Flyway

Add Dependency

```xml
<!-- pom.xml -->
<dependency>
    <groupId>org.flywaydb</groupId>
    <artifactId>flyway-core</artifactId>
</dependency>
```

Configure Application Properties

```properties
# application.properties

# Let Flyway manage the schema, not Hibernate
spring.jpa.hibernate.ddl-auto=validate

# Flyway configuration
spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration
spring.flyway.baseline-on-migrate=false
```

```
Configuration Explained:
+-------------------------------------------------------------------+
|                                                                   |
|  ddl-auto=validate:                                               |
|  Hibernate checks that entity mappings match the database schema  |
|  but does NOT modify the schema. Flyway handles all changes.     |
|  If they don't match, the app fails to start (catches errors).   |
|                                                                   |
|  flyway.locations=classpath:db/migration:                         |
|  Flyway looks for SQL files in src/main/resources/db/migration/  |
|                                                                   |
|  On application startup:                                          |
|  1. Flyway checks flyway_schema_history table                    |
|  2. Finds which migrations have already run                       |
|  3. Runs any NEW migrations in version order                      |
|  4. Updates flyway_schema_history                                 |
|  5. Hibernate validates schema matches entities                   |
|  6. Application starts normally                                   |
+-------------------------------------------------------------------+
```

Writing Migration Scripts

Naming Convention

```
Flyway Naming Convention:
+-------------------------------------------------------------------+
|                                                                   |
|  V{version}__{description}.sql                                    |
|  |  |          |                                                  |
|  |  |          +-- Description (underscores for spaces)           |
|  |  +------------- Version number                                 |
|  +---------------- V = Versioned migration                        |
|                                                                   |
|  Examples:                                                        |
|  V1__Create_employees_table.sql                                   |
|  V2__Add_email_to_employees.sql                                   |
|  V3__Create_departments_table.sql                                 |
|  V4__Add_department_id_to_employees.sql                           |
|  V5__Insert_seed_data.sql                                         |
|                                                                   |
|  Rules:                                                           |
|  - V is uppercase                                                 |
|  - Version can be: 1, 2, 3 or 1.1, 1.2, 2.0 or timestamps       |
|  - TWO underscores (__) separate version from description         |
|  - Description uses underscores (no spaces)                       |
|  - .sql extension                                                 |
|  - Files go in src/main/resources/db/migration/                   |
```

```
+-----------------------------------------------------------------+
```

Example Migrations

```sql
-- V1__Create_departments_table.sql
CREATE TABLE departments (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP,
    version INTEGER DEFAULT 0
);
```

```sql
-- V2__Create_employees_table.sql
CREATE TABLE employees (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    salary DECIMAL(10, 2),
    hire_date DATE,
    active BOOLEAN DEFAULT TRUE,
    department_id BIGINT,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP,
    version INTEGER DEFAULT 0,
    CONSTRAINT fk_employee_department
        FOREIGN KEY (department_id) REFERENCES departments(id)
);

CREATE INDEX idx_employee_email ON employees(email);
CREATE INDEX idx_employee_department ON employees(department_id);
```

```sql
-- V3__Insert_default_departments.sql
INSERT INTO departments (name) VALUES ('Engineering');
INSERT INTO departments (name) VALUES ('Marketing');
INSERT INTO departments (name) VALUES ('Sales');
INSERT INTO departments (name) VALUES ('Human Resources');
INSERT INTO departments (name) VALUES ('Finance');
```

```sql
-- V4__Add_phone_to_employees.sql
ALTER TABLE employees ADD COLUMN phone VARCHAR(20);
```

```sql
-- V5__Create_projects_table.sql
CREATE TABLE projects (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    description TEXT,
    start_date DATE,
    end_date DATE,
    status VARCHAR(20) DEFAULT 'ACTIVE',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP
);

CREATE TABLE employee_projects (
    employee_id BIGINT NOT NULL,
    project_id BIGINT NOT NULL,
    assigned_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    role VARCHAR(50),
    PRIMARY KEY (employee_id, project_id),
    FOREIGN KEY (employee_id) REFERENCES employees(id),
    FOREIGN KEY (project_id) REFERENCES projects(id)
);
```

```sql
-- V6__Add_bio_column_to_employees.sql
ALTER TABLE employees ADD COLUMN bio TEXT;

-- V7__Rename_phone_to_phone_number.sql
ALTER TABLE employees ALTER COLUMN phone RENAME TO phone_number;
```

```
File Structure:
+-------------------------------------------------------------------+
|                                                                   |
|  src/main/resources/                                              |
|  +-- db/                                                          |
|  |   +-- migration/                                               |
|  |       +-- V1__Create_departments_table.sql                     |
|  |       +-- V2__Create_employees_table.sql                       |
|  |       +-- V3__Insert_default_departments.sql                   |
|  |       +-- V4__Add_phone_to_employees.sql                       |
|  |       +-- V5__Create_projects_table.sql                        |
|  |       +-- V6__Add_bio_column_to_employees.sql                  |
|  |       +-- V7__Rename_phone_to_phone_number.sql                 |
|  +-- application.properties                                       |
+-------------------------------------------------------------------+
```

How Flyway Tracks Migrations

Flyway creates a `flyway_schema_history` table to track which migrations have run:

```
flyway_schema_history Table:
+-------------------------------------------------------------------+
|                                                                   |
|  | version | description              | script                  | |
|  |---------|--------------------------|-------------------------| |
|  | 1       | Create departments table | V1__Create_depart...sql | |
|  | 2       | Create employees table   | V2__Create_employ...sql | |
|  | 3       | Insert default depts     | V3__Insert_defaul...sql | |
|  | 4       | Add phone to employees   | V4__Add_phone_to_...sql | |
|                                                                   |
|  | installed_on        | execution_time | checksum   | success |  |
|  |---------------------|----------------|------------|---------|  |
|  | 2025-09-01 10:00:00 | 45             | -123456789 | true    |  |
|  | 2025-09-01 10:00:01 | 120            | 987654321  | true    |  |
|  | 2025-09-01 10:00:01 | 15             | -555555555 | true    |  |
|  | 2025-09-15 14:30:00 | 30             | 111111111  | true    |  |
|                                                                   |
|  On next startup:                                                 |
|  Flyway sees V1-V4 already ran. If V5 exists, it runs V5 only.  |
|  Already-run migrations are NEVER re-executed.                   |
|                                                                   |
|  Checksum: Hash of the file content. If you modify an already-   |
|  run migration file, Flyway detects the change and FAILS.        |
|  This prevents accidental modification of applied migrations.    |
+-------------------------------------------------------------------+
```

Repeatable Migrations

Repeatable migrations run every time their content changes. Useful for views, functions, and stored procedures:

```
Repeatable Migration Naming:
+-------------------------------------------------------------------+
|                                                                   |
|  R__{description}.sql                                             |
|  |     |                                                        |
|  |     +-- Description                                           |
|  +-------- R = Repeatable (runs when checksum changes)           |
|                                                                   |
|  Examples:                                                        |
|  R__Create_employee_summary_view.sql                              |
|  R__Create_department_statistics_function.sql                     |
+-------------------------------------------------------------------+
```

```sql
-- R__Create_employee_summary_view.sql
-- This runs every time the file content changes

CREATE OR REPLACE VIEW employee_summary AS
SELECT
    e.id,
    e.name,
    e.email,
    e.salary,
    d.name AS department_name,
    e.hire_date,
    e.active
FROM employees e
LEFT JOIN departments d ON e.department_id = d.id;
```

```
Versioned vs Repeatable:
+-------------------------------------------------------------------+
|                                                                   |
|  Versioned (V1__, V2__):                                          |
|  - Runs ONCE, tracked by version number                           |
|  - For schema changes: CREATE TABLE, ALTER TABLE, INSERT          |
|  - Cannot be modified after running                               |
|  - Order matters (V1 before V2)                                   |
|                                                                   |
|  Repeatable (R__):                                                |
|  - Runs every time content changes (checksum comparison)          |
|  - For replaceable objects: views, functions, procedures          |
|  - Can be modified freely                                         |
|  - Always runs AFTER all versioned migrations                     |
+-------------------------------------------------------------------+
```

Baselining an Existing Database

If you are adding Flyway to a project that already has a database schema, use baselining:

```properties
# application.properties
spring.flyway.baseline-on-migrate=true
spring.flyway.baseline-version=0
```

```
Baseline Process:
+-------------------------------------------------------------------+
|                                                                   |
|  Existing database with tables already created manually           |
|       |                                                           |
```

```
|       v                                                          |
|  Add Flyway with baseline-on-migrate=true                        |
|       |                                                          |
|       v                                                          |
|  Flyway creates flyway_schema_history with baseline entry        |
|  (marks version 0 as "baseline")                                 |
|       |                                                          |
|       v                                                          |
|  Future migrations start from V1                                 |
|  V1 = first CHANGE (not the initial schema)                      |
|       |                                                          |
|       v                                                          |
|  Optional: Create V0__Baseline.sql for documentation             |
|  (contains CREATE TABLE statements matching existing schema)     |
|  (Flyway skips it because baseline version is 0)                 |
+------------------------------------------------------------------+
```

Development Workflow

```
Migration Workflow:
+------------------------------------------------------------------+
|                                                                  |
|  1. Developer adds a field to an entity:                         |
|     @Column(name = "phone_number")                               |
|     private String phoneNumber;                                  |
|                                                                  |
|  2. Developer creates a migration script:                        |
|     V8__Add_phone_number_to_employees.sql                        |
|     ALTER TABLE employees ADD COLUMN phone_number VARCHAR(20);   |
|                                                                  |
|  3. Developer runs the app:                                      |
|     Flyway executes V8                                           |
|     Hibernate validates schema matches entity (ddl-auto=validate) |
|     App starts successfully                                      |
|                                                                  |
|  4. Developer commits both files:                                |
|     - Entity change (Java)                                       |
|     - Migration script (SQL)                                     |
|                                                                  |
|  5. Other developers pull and run:                               |
|     Flyway automatically runs V8 on their database               |
|     Everyone has the same schema                                 |
|                                                                  |
|  6. CI/CD deploys to staging/production:                         |
|     Flyway runs V8 on staging database                           |
|     Flyway runs V8 on production database                        |
|     All environments are in sync                                 |
+------------------------------------------------------------------+
```

Handling Migration Failures

```
When a Migration Fails:
+-------------------------------------------------------------------+
|                                                                    |
|  V5 has a SQL syntax error:                                        |
|  CRAETE TABLE projects (...)    <-- typo!                          |
|                                                                    |
|  Flyway tries to run V5 --> SQL error                              |
|  flyway_schema_history marks V5 as failed (success = false)       |
|  Application fails to start                                        |
|                                                                    |
|  To fix:                                                           |
|                                                                    |
|  Option 1: Fix the SQL and repair                                  |
|  1. Fix the typo in V5__Create_projects_table.sql                  |
|  2. Run: flyway repair (or Spring Boot command)                    |
|     This removes the failed entry from flyway_schema_history      |
|  3. Restart the app — Flyway retries V5 with fixed SQL            |
|                                                                    |
|  Option 2: In development, clean and restart                      |
|  spring.flyway.clean-disabled=false  (ONLY in dev!)                |
|  Run: flyway clean (drops everything, reruns all migrations)      |
|                                                                    |
|  NEVER use flyway clean in production!                             |
+-------------------------------------------------------------------+
```

Common Mistakes

1. Modifying already-applied migration files: Flyway checksums detect changes to applied migrations and refuses to start. Applied migrations are immutable — create a new migration for changes.
2. Using ddl-auto=update alongside Flyway: Both try to manage the schema. Use `ddl-auto=validate` (or `none`) with Flyway. Let Flyway be the single source of truth.
3. Forgetting to create a migration when changing entities: Adding a field to an entity without a corresponding migration causes `validate` to fail. Always pair entity changes with migration scripts.
4. Writing non-idempotent migrations: If a migration fails halfway, some DDL might have executed. Write migrations that can handle partial application (use `IF NOT EXISTS` where

possible).

5. Using database-specific SQL in migrations meant for H2: H2 syntax differs from PostgreSQL/MySQL. If you test with H2 but deploy to PostgreSQL, some SQL may not be portable.
6. Numbering gaps or conflicts in team development: Two developers creating V5 independently causes a conflict. Use timestamps (`V20250915143000__`) or coordinate version numbers.

Best Practices

1. One migration per change: Keep migrations small and focused. `V5__Add_email_index.sql` is better than `V5__Multiple_changes.sql`.
2. Never modify applied migrations: Once a migration has run in any environment, treat it as immutable. Fix mistakes with new migrations.
3. Use validate in production: `spring.jpa.hibernate.ddl-auto=validate` catches schema mismatches at startup instead of at runtime.
4. Commit migrations with their entity changes: The migration and the Java entity change should be in the same commit. They are a unit.
5. Test migrations in CI: Run migrations against a fresh database in your CI pipeline to catch issues before production.
6. Use descriptive migration names: `V12__Add_indexes_for_search_performance.sql` tells the team what and why without opening the file.

Summary

- ddl-auto=update is dangerous in production: it never drops columns, never renames, and provides no rollback or audit trail. Use `validate` instead.
- Flyway manages database schema changes through versioned SQL migration scripts that run in order, once, and are tracked in `flyway_schema_history`.
- Naming convention: `V{version}__{description}.sql` for versioned migrations, `R__{description}.sql` for repeatable migrations.
- Repeatable migrations re-run when their content changes — useful for views and functions.
- Baselining integrates Flyway with an existing database without re-creating the schema.

- Never modify applied migrations. Create new migrations to fix or change previous schema decisions.
- Pair entity changes with migrations: Every `@Column` change needs a corresponding SQL migration.

Interview Questions

Q1: Why should you not use ddl-auto=update in production?

It never removes columns or tables (orphaned data accumulates), never renames (creates new column, loses data), can lock tables during ALTER on large datasets, provides no rollback capability, no audit trail, and causes schema drift across environments.

Q2: How does Flyway ensure migrations run in order and only once?

Flyway tracks applied migrations in `flyway_schema_history` table with version numbers, checksums, and timestamps. On startup, it compares available migration files against applied versions and runs only new migrations in version order. It detects modified files via checksum comparison.

Q3: What is the difference between versioned and repeatable migrations?

Versioned migrations (V1, V2) run once and are immutable — for DDL changes. Repeatable migrations (R__) re-run whenever their content changes — for replaceable database objects like views and functions. Repeatable always run after all versioned migrations.

Q4: How do you add Flyway to an existing project with an established database?

Use `spring.flyway.baseline-on-migrate=true`. Flyway creates a baseline entry in `flyway_schema_history` marking the current state. Future migrations start from the next version number. Optionally create a V0 script documenting the existing schema.

Q5: What happens if a migration fails?

Flyway marks the migration as failed in `flyway_schema_history`. The application cannot start until the issue is resolved. Fix the SQL script, run `flyway repair` to remove the failed entry, and restart. In development, `flyway clean` can reset everything.

Practice Exercises

Exercise 1: First Migration Remove `ddl-auto=update` and switch to `validate`. Write V1-V3 migrations that create the tables your entities need. Verify the app starts successfully.

Exercise 2: Schema Evolution Add a new field to an entity. Write the corresponding migration. Verify `validate` passes. Then try adding a field WITHOUT a migration and observe the startup error.

Exercise 3: Seed Data Write a migration that inserts default data (categories, roles, status codes). Verify the data exists on startup.

Exercise 4: Repeatable Migration Create a database view using a repeatable migration. Modify the view definition and verify Flyway re-runs it on the next startup.

Exercise 5: Team Simulation Simulate two developers creating migrations concurrently. Both create V5 with different changes. Observe the conflict. Resolve by renumbering one to V6.

What Is Next?

In the next chapter, we will explore Performance Tuning and Monitoring — batch processing, bulk operations, connection pooling, slow query logging, Hibernate statistics, and indexing strategies. You will learn how to identify and fix performance bottlenecks in JPA/Hibernate applications.

Chapter 27: Performance Tuning and Monitoring

Learning Goals

By the end of this chapter, you will be able to:

- Enable and interpret Hibernate statistics for performance monitoring
- Configure batch inserts and updates for bulk operations
- Use JPQL bulk UPDATE and DELETE to bypass entity loading
- Understand and configure HikariCP connection pooling
- Enable slow query logging to catch inefficient queries
- Apply database indexing strategies for JPA queries
- Use StatelessSession for high-throughput operations
- Profile and optimize real-world query patterns
- Identify the top performance issues in Hibernate applications

The Top Hibernate Performance Issues

```
Most Common Performance Problems (and where to find fixes):
+-------------------------------------------------------------------+
|                                                                    |
|  #1  N+1 Queries (Chapter 18)                                     |
|      Loading N entities triggers N extra queries for relations    |
|      Fix: JOIN FETCH, @EntityGraph, @BatchSize                    |
|                                                                    |
|  #2  Loading too much data                                        |
|      SELECT * when you need 3 columns                             |
|      Fix: Projections, DTOs (Chapter 23)                          |
```

```
|                                                                  |
|  #3  No batch processing                                        |
|      1000 inserts = 1000 individual SQL statements               |
|      Fix: JDBC batching (this chapter)                           |
|                                                                  |
|  #4  Missing indexes                                             |
|      Full table scans on WHERE and ORDER BY columns              |
|      Fix: Database indexes (this chapter)                        |
|                                                                  |
|  #5  Connection pool exhaustion                                  |
|      Long transactions hold connections, others wait             |
|      Fix: HikariCP tuning, short transactions (this chapter)     |
|                                                                  |
|  #6  Dirty checking overhead                                     |
|      Hibernate compares every field of every managed entity      |
|      Fix: readOnly transactions, StatelessSession (this chapter) |
+------------------------------------------------------------------+
```

Hibernate Statistics

Enable statistics to see exactly what Hibernate is doing:

```properties
# application.properties
spring.jpa.properties.hibernate.generate_statistics=true
logging.level.org.hibernate.stat=DEBUG
```

```
Statistics Output Example:
+------------------------------------------------------------------+
|                                                                  |
|  Session Metrics:                                                |
|    Statements prepared: 23                                       |
|    Statements closed: 23                                         |
|    Transactions: 5 (5 successful, 0 failed)                      |
|                                                                  |
|  Entity Metrics:                                                 |
|    Entities loaded: 150                                          |
|    Entities inserted: 12                                         |
|    Entities updated: 3                                           |
|    Entities deleted: 1                                           |
|                                                                  |
|  Query Metrics:                                                  |
|    Queries executed: 23                                          |
|    Max query time: 245ms ("SELECT e FROM Employee e ...")        |
|    Total query time: 890ms                                       |
|                                                                  |
|  L2 Cache Metrics:                                               |
```

```
|    Cache hits: 45                                                |
|    Cache misses: 12                                              |
|    Hit rate: 78.9%                                               |
|                                                                  |
|  Key things to look for:                                         |
|  - Statements prepared >> Transactions = possible N+1            |
|  - Max query time > 100ms = slow query to investigate            |
|  - Entities loaded >> needed = over-fetching                     |
+------------------------------------------------------------------+
```

Programmatic Statistics Access

```java
@RestController
@RequestMapping("/api/admin")
public class MonitoringController {

    @Autowired
    private EntityManagerFactory emf;

    @GetMapping("/hibernate-stats")
    public Map<String, Object> getStats() {
        Statistics stats = emf.unwrap(SessionFactory.class).getStatistics();

        Map<String, Object> result = new LinkedHashMap<>();
        result.put("queriesExecuted", stats.getQueryExecutionCount());
        result.put("slowestQuery", stats.getQueryExecutionMaxTimeQueryString());
        result.put("slowestQueryTime", stats.getQueryExecutionMaxTime() + "ms");
        result.put("entitiesLoaded", stats.getEntityLoadCount());
        result.put("entitiesInserted", stats.getEntityInsertCount());
        result.put("entitiesUpdated", stats.getEntityUpdateCount());
        result.put("connectionsObtained", stats.getConnectCount());
        result.put("transactionsCompleted", stats.getTransactionCount());
        result.put("l2CacheHits", stats.getSecondLevelCacheHitCount());
        result.put("l2CacheMisses", stats.getSecondLevelCacheMissCount());

        return result;
    }
}
```

Batch Inserts and Updates

By default, Hibernate sends one INSERT per entity. With batching, it groups multiple inserts into a single JDBC batch call.

```
Without Batching (default):
+-------------------------------------------------------------------+
|                                                                   |
|  Inserting 1000 employees:                                        |
|  INSERT INTO employees (...) VALUES (...)   -- statement 1       |
|  INSERT INTO employees (...) VALUES (...)   -- statement 2       |
|  INSERT INTO employees (...) VALUES (...)   -- statement 3       |
|  ...                                                              |
|  INSERT INTO employees (...) VALUES (...)   -- statement 1000    |
|                                                                   |
|  1000 round-trips to the database!                                |
|  Each INSERT: send SQL -> database parses -> executes -> responds |
|                                                                   |
+-------------------------------------------------------------------+

With Batching (batch_size = 50):
+-------------------------------------------------------------------+
|                                                                   |
|  Inserting 1000 employees:                                        |
|  BATCH 1: 50 INSERTs sent together   -- 1 round-trip             |
|  BATCH 2: 50 INSERTs sent together   -- 1 round-trip             |
|  ...                                                              |
|  BATCH 20: 50 INSERTs sent together  -- 1 round-trip             |
|                                                                   |
|  20 round-trips instead of 1000!                                 |
|  50x fewer network round-trips.                                  |
+-------------------------------------------------------------------+
```

Configuration

```properties
# application.properties
spring.jpa.properties.hibernate.jdbc.batch_size=50
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true
spring.jpa.properties.hibernate.jdbc.batch_versioned_data=true
```

```
Batch Configuration Explained:
+-------------------------------------------------------------------+
|                                                                   |
|  hibernate.jdbc.batch_size=50:                                    |
|  Group up to 50 statements in a single JDBC batch.               |
|                                                                   |
|  hibernate.order_inserts=true:                                    |
|  Reorder INSERT statements so same-entity inserts are grouped.   |
|  Without this: INSERT emp, INSERT dept, INSERT emp, INSERT dept   |
|  With this:    INSERT emp, INSERT emp, INSERT dept, INSERT dept   |
|  Grouping enables effective batching.                             |
|                                                                   |
|  hibernate.order_updates=true:                                    |
```

```
|  Same reordering for UPDATE statements.                          |
|                                                                  |
|  hibernate.jdbc.batch_versioned_data=true:                       |
|  Allow batching for entities with @Version (optimistic locking).  |
|                                                                  |
|  IMPORTANT: IDENTITY generation strategy DISABLES batching!      |
|  Hibernate must execute each INSERT immediately to get the ID.   |
|  Use SEQUENCE strategy for batch-heavy applications.              |
+------------------------------------------------------------------+
```

Batch Insert Service

```java
@Service
public class BatchImportService {

    @PersistenceContext
    private EntityManager entityManager;

    @Transactional
    public int importEmployees(List<EmployeeData> dataList) {
        int batchSize = 50;
        int count = 0;

        for (int i = 0; i < dataList.size(); i++) {
            EmployeeData data = dataList.get(i);
            Employee employee = new Employee(data.getName(), data.getEmail());
            employee.setSalary(data.getSalary());
            entityManager.persist(employee);
            count++;

            if (count % batchSize == 0) {
                entityManager.flush();    // Send batch to DB
                entityManager.clear();    // Free memory
            }
        }

        entityManager.flush();
        entityManager.clear();

        return count;
    }
}
```

JPQL Bulk Operations

For updating or deleting many rows, JPQL bulk operations bypass the persistence context entirely:

```java
@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    // Bulk UPDATE — one SQL statement, no entity loading
    @Modifying(clearAutomatically = true)
    @Query("UPDATE Employee e SET e.salary = e.salary * :factor WHERE e.department.id =...
    int adjustSalaries(@Param("factor") BigDecimal factor, @Param("deptId") Long deptId);

    // Bulk DELETE — one SQL statement
    @Modifying(clearAutomatically = true)
    @Query("DELETE FROM Employee e WHERE e.active = false AND e.hireDate < :cutoff")
    int purgeInactive(@Param("cutoff") LocalDate cutoff);

    // Bulk status update
    @Modifying(clearAutomatically = true)
    @Query("UPDATE Employee e SET e.active = false WHERE e.department.id = :deptId")
    int deactivateDepartment(@Param("deptId") Long deptId);
}
```

```
Entity-by-Entity vs Bulk Operation:
+-------------------------------------------------------------------+
|                                                                    |
|  Deactivating 500 employees in department 5:                      |
|                                                                    |
|  Entity-by-entity:                                                 |
|  1. SELECT * FROM employees WHERE department_id = 5  (load 500)  |
|  2. For each: employee.setActive(false)               (in memory)|
|  3. Flush: 500 UPDATE statements                      (500 SQLs) |
|  Total: 501 SQL statements, 500 entities in memory               |
|                                                                    |
|  Bulk JPQL:                                                        |
|  UPDATE employees SET active = false WHERE department_id = 5      |
|  Total: 1 SQL statement, 0 entities in memory                    |
|                                                                    |
|  Bulk is ~500x faster for this operation.                        |
|  Use bulk for mass updates/deletes where entity logic is not     |
|  needed (no @PreUpdate callbacks, no cascade, no dirty checking). |
+-------------------------------------------------------------------+
```

Connection Pooling with HikariCP

Spring Boot uses HikariCP by default. Proper configuration prevents connection exhaustion.

```properties
# application.properties — HikariCP configuration
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=5
spring.datasource.hikari.idle-timeout=300000
spring.datasource.hikari.max-lifetime=600000
spring.datasource.hikari.connection-timeout=30000
spring.datasource.hikari.leak-detection-threshold=60000
```

```
HikariCP Settings:
+-------------------------------------------------------------------+
|                                                                   |
|  maximum-pool-size=10:                                            |
|  Maximum number of connections in the pool.                       |
|  Rule of thumb: connections = (2 * CPU cores) + disk spindles    |
|  For most apps: 10-20 is sufficient.                              |
|  More is NOT better — too many connections cause contention.      |
|                                                                   |
|  minimum-idle=5:                                                  |
|  Minimum connections kept ready (even when idle).                 |
|  Set equal to maximum-pool-size for stable performance.           |
|                                                                   |
|  connection-timeout=30000 (30s):                                  |
|  How long to wait for a connection from the pool.                 |
|  If pool is full and all connections are busy, wait up to 30s.   |
|  After 30s: throw SQLException (connection timeout).              |
|                                                                   |
|  leak-detection-threshold=60000 (60s):                            |
|  If a connection is not returned within 60s, log a warning.       |
|  Helps detect connection leaks (forgotten transactions).          |
|                                                                   |
+-------------------------------------------------------------------+

Connection Pool Lifecycle:
+-------------------------------------------------------------------+
|                                                                   |
|  Request 1  ----+                                                 |
|  Request 2  ----|---> [Connection Pool]                           |
|  Request 3  ----+     [C1] [C2] [C3] [C4] [C5] ... [C10]        |
|  Request 4  ----+            |                                   |
|  ...                         v                                   |
|                        Database                                   |
|                                                                   |
|  Each @Transactional method:                                      |
|  1. Borrows a connection from the pool                            |
|  2. Uses it for queries and updates                               |
|  3. Returns it when transaction commits/rollbacks                 |
|                                                                   |
|  If all 10 connections are busy:                                  |
|  Request 11 WAITS (up to connection-timeout)                      |
|  --> This is why short transactions matter!                       |
+-------------------------------------------------------------------+
```

Slow Query Logging

Hibernate Slow Query Log

```properties
# Log queries slower than 500ms
spring.jpa.properties.hibernate.session.events.log.LOG_QUERIES_SLOWER_THAN_MS=500
logging.level.org.hibernate.SQL_SLOW=INFO
```

SQL Logging for Development

```properties
# Show all SQL with formatting (development only)
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Better: use logging framework (includes parameters)
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.orm.jdbc.bind=TRACE
```

```
SQL Logging Output:
+-------------------------------------------------------------------+
|                                                                   |
|  Hibernate:                                                       |
|      select                                                       |
|          e1_0.id,                                                 |
|          e1_0.name,                                               |
|          e1_0.email,                                              |
|          e1_0.salary                                              |
|      from                                                         |
|          employees e1_0                                           |
|      where                                                        |
|          e1_0.department_id=?                                     |
|                                                                   |
|  Binding: [1]     <-- parameter value (with TRACE level)          |
|                                                                   |
|  Look for:                                                        |
|  - Same query repeated many times (N+1)                           |
|  - SELECT * when you need few columns (over-fetching)             |
|  - Missing WHERE clause (full table scan)                         |
|  - Queries without LIMIT (unbounded results)                      |
+-------------------------------------------------------------------+
```

Database Indexing

Indexes speed up WHERE, JOIN, and ORDER BY operations. Without indexes, the database scans every row.

```
Index Impact:
+-------------------------------------------------------------------+
|                                                                   |
|  employees table: 1,000,000 rows                                  |
|                                                                   |
|  Query: SELECT * FROM employees WHERE email = 'alice@co.com'      |
|                                                                   |
|  Without index:                                                   |
|  Full table scan: checks ALL 1,000,000 rows                      |
|  Time: ~500ms                                                     |
|                                                                   |
|  With index on email:                                             |
|  B-tree lookup: finds the row directly                            |
|  Time: ~1ms                                                       |
|                                                                   |
|  500x faster!                                                     |
+-------------------------------------------------------------------+
```

When to Add Indexes

```
Index These Columns:
+-------------------------------------------------------------------+
|                                                                   |
|  Always index:                                                    |
|  - Primary keys (automatic)                                       |
|  - Foreign keys (department_id, customer_id)                      |
|  - Unique constraints (email, username)                           |
|  - Columns in WHERE clauses used frequently                      |
|  - Columns in ORDER BY for paginated queries                     |
|  - Columns in JOIN conditions                                     |
|                                                                   |
|  Consider indexing:                                               |
|  - Status columns (active, status) if filtered often              |
|  - Date columns (created_at) if sorted/filtered by date          |
|  - Composite indexes for multi-column WHERE clauses              |
|                                                                   |
|  Do NOT index:                                                    |
|  - Columns rarely used in queries                                 |
|  - Columns with very low cardinality (boolean with 50/50 split)  |
|  - Tables with very few rows (< 1000)                            |
|  - Columns that are updated very frequently                      |
|    (indexes slow down INSERT/UPDATE)                              |
+-------------------------------------------------------------------+
```

Creating Indexes with JPA

```java
@Entity
@Table(name = "employees", indexes = {
    @Index(name = "idx_employee_email", columnList = "email"),
    @Index(name = "idx_employee_department", columnList = "department_id"),
    @Index(name = "idx_employee_active_hire",
           columnList = "active, hire_date")  // Composite index
})
public class Employee {
    // ...
}
```

Or in Flyway migrations (recommended for production):

```sql
-- V10__Add_performance_indexes.sql
CREATE INDEX idx_employee_email ON employees(email);
CREATE INDEX idx_employee_department ON employees(department_id);
CREATE INDEX idx_employee_active_hire ON employees(active, hire_date);
CREATE INDEX idx_order_customer_status ON orders(customer_id, status);
CREATE INDEX idx_order_created ON orders(created_at);
```

StatelessSession for High-Throughput

Hibernate's `StatelessSession` bypasses the persistence context entirely — no caching, no dirty checking, no cascading:

```java
@Service
public class BulkExportService {

    @Autowired
    private EntityManagerFactory emf;

    public void exportAllEmployees(Writer writer) {
        SessionFactory sessionFactory = emf.unwrap(SessionFactory.class);

        try (StatelessSession session = sessionFactory.openStatelessSession()) {
            ScrollableResults<Employee> results = session
                .createQuery("FROM Employee e ORDER BY e.id", Employee.class)
                .setFetchSize(100)
                .scroll(ScrollMode.FORWARD_ONLY);

            while (results.next()) {
```

```
                Employee employee = results.get();
                writer.write(employee.toCsvLine());
                // No persistence context = no memory buildup
                // Can process millions of rows
            }
        }
    }
}
```

```
Session vs StatelessSession:
+-------------------------------------------------------------------+
|                                                                   |
|  Regular Session (EntityManager):                                 |
|  + Dirty checking (auto-save changes)                             |
|  + First-level cache (identity guarantee)                         |
|  + Cascading operations                                           |
|  + Lazy loading                                                   |
|  - Memory overhead (snapshots, managed entities)                  |
|  - Slow for bulk operations                                       |
|                                                                   |
|  StatelessSession:                                                |
|  + No persistence context (zero memory overhead)                  |
|  + No dirty checking (fast)                                       |
|  + Processes millions of rows without OutOfMemoryError           |
|  - No caching (every access hits DB)                              |
|  - No cascading (manage relationships manually)                   |
|  - No lazy loading (load everything upfront)                      |
|  - Must call insert/update/delete explicitly                      |
|                                                                   |
|  Use StatelessSession for:                                        |
|  - Batch imports/exports                                          |
|  - ETL processes                                                  |
|  - Data migration scripts                                         |
|  - Report generation over large datasets                          |
+-------------------------------------------------------------------+
```

Performance Checklist

```
Pre-Production Performance Checklist:
+-------------------------------------------------------------------+
|                                                                   |
|  [ ] All @ManyToOne set to FetchType.LAZY                         |
|  [ ] All @OneToOne set to FetchType.LAZY                          |
|  [ ] N+1 queries identified and fixed (JOIN FETCH / @EntityGraph) |
|  [ ] default_batch_fetch_size configured (16 or 32)               |
|  [ ] JDBC batch_size set for bulk operations (50)                 |
```

```
|  [ ] order_inserts and order_updates enabled                      |
|  [ ] Read-only methods annotated with @Transactional(readOnly)    |
|  [ ] Projections/DTOs used for list endpoints                     |
|  [ ] Bulk operations use JPQL UPDATE/DELETE when possible         |
|  [ ] Indexes on FK columns, frequent WHERE/ORDER BY columns      |
|  [ ] HikariCP pool size appropriate for workload                  |
|  [ ] Slow query logging enabled                                   |
|  [ ] Statistics enabled in staging (disabled in production)       |
|  [ ] spring.jpa.open-in-view=false                                |
|  [ ] ddl-auto=validate (Flyway manages schema)                   |
|  [ ] Connection leak detection enabled                            |
+-------------------------------------------------------------------+
```

Common Mistakes

1. Not enabling JDBC batching: Default batch_size is 1 (no batching). Set `hibernate.jdbc.batch_size=50` for any application that does bulk writes.
2. Using IDENTITY generation with batching: `GenerationType.IDENTITY` forces immediate INSERT to get the generated ID, disabling batching. Use `SEQUENCE` for batch-heavy operations.
3. Over-sizing the connection pool: 100 connections is almost never better than 10-20. More connections mean more database memory, more context switching, and more contention.
4. Not indexing foreign key columns: JPA does not create FK indexes automatically (some databases do, some do not). Always create indexes on FK columns explicitly.
5. Disabling SQL logging in development: You cannot optimize what you cannot see. Always enable SQL logging during development to catch N+1 and other issues early.
6. Using entity-by-entity operations for mass updates: Updating 10,000 rows by loading each entity, modifying it, and relying on dirty checking generates 10,001 SQL statements. Use JPQL bulk UPDATE instead (1 SQL statement).

Best Practices

1. Enable JDBC batching globally: Set `batch_size=50`, `order_inserts=true`, `order_updates=true` as defaults. They help even for small operations.
2. Use JPQL bulk operations for mass updates/deletes: `UPDATE Employee e SET e.active = false WHERE ...` is orders of magnitude faster than loading and modifying entities

individually.

3. Set `default_batch_fetch_size=16`: This global setting reduces N+1 without changing any code. Good baseline optimization.
4. Index every FK column: `CREATE INDEX idx_employee_dept ON employees(department_id)`. This speeds up JOINs and cascade deletes.
5. Monitor with Hibernate statistics in staging: Enable statistics, run load tests, identify slowest queries and highest query counts.
6. Keep transactions short: Avoid calling external services inside `@Transactional`. Each second of transaction time is a second a connection is held.

Summary

- Hibernate statistics reveal query counts, entity operations, cache hit rates, and slowest queries. Essential for identifying bottlenecks.
- JDBC batching groups multiple INSERT/UPDATE statements into single round-trips. Configure `batch_size=50` with `order_inserts=true`.
- JPQL bulk operations (`UPDATE`, `DELETE`) execute a single SQL statement instead of loading and modifying entities individually. Use for mass operations.
- HikariCP connection pooling: size the pool based on CPU cores, not request volume. Enable leak detection. Keep transactions short.
- Slow query logging catches queries that take too long. Enable with `LOG_QUERIES_SLOWER_THAN_MS`.
- Database indexes on FK columns, WHERE columns, and ORDER BY columns prevent full table scans.
- StatelessSession bypasses the persistence context for high-throughput operations with zero memory overhead.

Interview Questions

Q1: What is the most common Hibernate performance problem?

The N+1 query problem. Loading N entities and accessing their lazy relationships triggers N additional queries (1 for the parent list + N for each relationship). Fix with JOIN FETCH, @EntityGraph, or @BatchSize.

Q2: How does JDBC batching improve performance?

Instead of sending each INSERT/UPDATE as a separate network round-trip, batching groups multiple statements (e.g., 50) into a single JDBC batch call. This reduces network overhead by a factor of the batch size. Configure with `hibernate.jdbc.batch_size`.

Q3: Why does GenerationType.IDENTITY disable JDBC batching?

IDENTITY relies on the database to generate the ID via auto-increment, which requires executing each INSERT immediately to retrieve the generated value. Batching requires deferring execution, which conflicts with IDENTITY. Use SEQUENCE instead.

Q4: How do you size a connection pool?

A good starting formula is `connections = (2 * CPU cores) + effective_spindle_count`. For most applications, 10-20 connections is sufficient. More connections cause contention and waste database resources. Monitor connection wait times and adjust.

Q5: When should you use StatelessSession instead of EntityManager?

For high-throughput batch operations: importing millions of rows, ETL processes, bulk exports, and report generation over large datasets. StatelessSession has no persistence context (zero memory overhead), no dirty checking, and no caching — making it suitable for processing unlimited data without OutOfMemoryError.

Q6: What columns should you always index?

Primary keys (automatic), foreign key columns (not always automatic), unique constraint columns, columns frequently used in WHERE clauses, columns used in ORDER BY for paginated queries, and columns used in JOIN conditions.

Practice Exercises

Exercise 1: Batch Insert Performance Insert 10,000 employees with and without JDBC batching. Measure the time difference. Use `System.nanoTime()` before and after. Enable SQL logging to verify batching is working.

Exercise 2: Bulk vs Entity Update Deactivate 500 employees in a department two ways: (a) load all, set active=false, let dirty checking update; (b) JPQL `UPDATE Employee e SET e.active = false WHERE ...`. Compare query counts and execution time.

Exercise 3: Index Impact Create a table with 100,000 rows. Query by a non-indexed column and measure time. Add an index. Query again and measure. Compare the difference.

Exercise 4: Connection Pool Monitoring Set `maximum-pool-size=2` and `leak-detection-threshold=5000`. Create a service that holds a transaction for 10 seconds. Call it 3 times concurrently. Observe the connection timeout and leak detection warnings.

Exercise 5: Statistics Analysis Enable Hibernate statistics. Run your application's main use cases. Review the statistics output and identify: (a) the slowest query, (b) the highest query count (potential N+1), (c) cache hit rate.

What Is Next?

In the next chapter, we will put everything together and build a Complete REST API with Spring Boot and Hibernate. You will create a full CRUD API with proper DTOs, validation, error handling, pagination, and all the best practices from this book.

Chapter 28: Building a Complete REST API

Learning Goals

By the end of this chapter, you will be able to:

- Design a complete REST API with proper HTTP methods and status codes
- Build CRUD endpoints with Spring Boot, JPA, and Hibernate
- Apply all the patterns from previous chapters in one cohesive application
- Use request/response DTOs with validation
- Implement pagination and sorting in API endpoints
- Add a global exception handler for consistent error responses
- Document the API with OpenAPI/Swagger
- Test endpoints with MockMvc
- Follow REST best practices for resource naming and response structure

API Design

We will build a Product Management API that demonstrates all the patterns from this book.

```
API Endpoints:
+-------------------------------------------------------------------+
|                                                                   |
|  Products:                                                        |
|  GET    /api/products              List (paginated, sortable)     |
|  GET    /api/products/{id}         Get one by ID                 |
|  POST   /api/products              Create new product            |
|  PUT    /api/products/{id}         Full update                   |
```

```
|  PATCH  /api/products/{id}         Partial update                |
|  DELETE /api/products/{id}         Delete                        |
|  GET    /api/products/search       Search with filters           |
|                                                                  |
|  Categories:                                                     |
|  GET    /api/categories            List all categories           |
|  POST   /api/categories            Create category               |
|  GET    /api/categories/{id}/products  Products by category      |
|                                                                  |
|  HTTP Status Codes:                                              |
|  200 OK              Successful GET, PUT, PATCH                  |
|  201 Created         Successful POST                             |
|  204 No Content      Successful DELETE                           |
|  400 Bad Request     Validation error                            |
|  404 Not Found       Resource does not exist                     |
|  409 Conflict        Duplicate or concurrent update              |
|  500 Internal Error  Unexpected server error                     |
+------------------------------------------------------------------+
```

Entity Layer

```java
@Entity
@Table(name = "categories")
public class Category extends BaseEntity {

    @NotBlank
    @Column(nullable = false, unique = true, length = 100)
    private String name;

    @Size(max = 500)
    private String description;

    @OneToMany(mappedBy = "category")
    private List<Product> products = new ArrayList<>();

    protected Category() {}

    public Category(String name, String description) {
        this.name = name;
        this.description = description;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
    public List<Product> getProducts() { return Collections.unmodifiableList(products); }
}
```

```java
@Entity
@Table(name = "products", indexes = {
    @Index(name = "idx_product_category", columnList = "category_id"),
    @Index(name = "idx_product_sku", columnList = "sku"),
    @Index(name = "idx_product_active", columnList = "active")
})
public class Product extends BaseEntity {

    @NotBlank
    @Column(nullable = false, length = 200)
    private String name;

    @Column(columnDefinition = "TEXT")
    private String description;

    @NotNull
    @Positive
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price;

    @NotBlank
    @Column(nullable = false, unique = true, length = 50)
    private String sku;

    @PositiveOrZero
    @Column(name = "stock_quantity", nullable = false)
    private int stockQuantity;

    @Column(nullable = false)
    private boolean active = true;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "category_id")
    private Category category;

    protected Product() {}

    public Product(String name, String description, BigDecimal price,
                   String sku, int stockQuantity) {
        this.name = name;
        this.description = description;
        this.price = price;
        this.sku = sku;
        this.stockQuantity = stockQuantity;
    }

    // Getters and setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
    public BigDecimal getPrice() { return price; }
    public void setPrice(BigDecimal price) { this.price = price; }
```

```
    public String getSku() { return sku; }
    public void setSku(String sku) { this.sku = sku; }
    public int getStockQuantity() { return stockQuantity; }
    public void setStockQuantity(int stockQuantity) { this.stockQuantity = stockQuantit...
    public boolean isActive() { return active; }
    public void setActive(boolean active) { this.active = active; }
    public Category getCategory() { return category; }
    public void setCategory(Category category) { this.category = category; }
}
```

```java
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @CreatedDate
    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @Version
    private Integer version;

    public Long getId() { return id; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public LocalDateTime getUpdatedAt() { return updatedAt; }
    public Integer getVersion() { return version; }
}
```

DTO Layer

```java
// === Request DTOs ===

public class CreateProductRequest {

    @NotBlank(message = "Product name is required")
    @Size(min = 2, max = 200, message = "Name must be between 2 and 200 characters")
    private String name;
```

```
    @Size(max = 2000, message = "Description must be at most 2000 characters")
    private String description;

    @NotNull(message = "Price is required")
    @Positive(message = "Price must be positive")
    @Digits(integer = 8, fraction = 2, message = "Price format: up to 8 digits and 2 de...
    private BigDecimal price;

    @NotBlank(message = "SKU is required")
    @Size(min = 3, max = 50, message = "SKU must be between 3 and 50 characters")
    private String sku;

    @NotNull(message = "Stock quantity is required")
    @PositiveOrZero(message = "Stock cannot be negative")
    private Integer stockQuantity;

    private Long categoryId;

    // Getters and setters...
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
    public BigDecimal getPrice() { return price; }
    public void setPrice(BigDecimal price) { this.price = price; }
    public String getSku() { return sku; }
    public void setSku(String sku) { this.sku = sku; }
    public Integer getStockQuantity() { return stockQuantity; }
    public void setStockQuantity(Integer stockQuantity) { this.stockQuantity = stockQua...
    public Long getCategoryId() { return categoryId; }
    public void setCategoryId(Long categoryId) { this.categoryId = categoryId; }
}

public class UpdateProductRequest {

    @Size(min = 2, max = 200)
    private String name;           // All fields optional for partial update

    @Size(max = 2000)
    private String description;

    @Positive
    private BigDecimal price;

    @PositiveOrZero
    private Integer stockQuantity;

    private Boolean active;

    private Long categoryId;

    // Getters and setters...
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getDescription() { return description; }
```

```
    public void setDescription(String description) { this.description = description; }
    public BigDecimal getPrice() { return price; }
    public void setPrice(BigDecimal price) { this.price = price; }
    public Integer getStockQuantity() { return stockQuantity; }
    public void setStockQuantity(Integer stockQuantity) { this.stockQuantity = stockQua...
    public Boolean getActive() { return active; }
    public void setActive(Boolean active) { this.active = active; }
    public Long getCategoryId() { return categoryId; }
    public void setCategoryId(Long categoryId) { this.categoryId = categoryId; }
}
```

```java
// === Response DTOs ===

public class ProductResponse {

    private Long id;
    private String name;
    private String description;
    private BigDecimal price;
    private String sku;
    private int stockQuantity;
    private boolean active;
    private String categoryName;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;

    public ProductResponse() {}

    public ProductResponse(Long id, String name, String description, BigDecimal price,
                           String sku, int stockQuantity, boolean active,
                           String categoryName, LocalDateTime createdAt,
                           LocalDateTime updatedAt) {
        this.id = id;
        this.name = name;
        this.description = description;
        this.price = price;
        this.sku = sku;
        this.stockQuantity = stockQuantity;
        this.active = active;
        this.categoryName = categoryName;
        this.createdAt = createdAt;
        this.updatedAt = updatedAt;
    }

    // Getters...
    public Long getId() { return id; }
    public String getName() { return name; }
    public String getDescription() { return description; }
    public BigDecimal getPrice() { return price; }
    public String getSku() { return sku; }
    public int getStockQuantity() { return stockQuantity; }
    public boolean isActive() { return active; }
```

```
    public String getCategoryName() { return categoryName; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public LocalDateTime getUpdatedAt() { return updatedAt; }
}

// Lightweight list item
public interface ProductListProjection {
    Long getId();
    String getName();
    BigDecimal getPrice();
    String getSku();
    int getStockQuantity();
    boolean getActive();
}
```

Repository Layer

```java
@Repository
public interface ProductRepository extends JpaRepository<Product, Long>,
        JpaSpecificationExecutor<Product> {

    boolean existsBySku(String sku);
    boolean existsBySkuAndIdNot(String sku, Long id);

    @EntityGraph(attributePaths = {"category"})
    Optional<Product> findWithCategoryById(Long id);

    @EntityGraph(attributePaths = {"category"})
    Page<Product> findByActive(boolean active, Pageable pageable);

    Page<ProductListProjection> findProjectedByActive(boolean active, Pageable pageable);

    @Query("SELECT p FROM Product p WHERE p.category.id = :categoryId AND p.active = tr...
    Page<Product> findByCategoryId(@Param("categoryId") Long categoryId, Pageable pagea...
}

@Repository
public interface CategoryRepository extends JpaRepository<Category, Long> {

    boolean existsByName(String name);
    Optional<Category> findByName(String name);
}
```

Service Layer

```java
@Service
public class ProductService {

    private final ProductRepository productRepository;
    private final CategoryRepository categoryRepository;

    public ProductService(ProductRepository productRepository,
                          CategoryRepository categoryRepository) {
        this.productRepository = productRepository;
        this.categoryRepository = categoryRepository;
    }

    @Transactional
    public ProductResponse create(CreateProductRequest request) {
        if (productRepository.existsBySku(request.getSku())) {
            throw new DuplicateResourceException("Product", "SKU", request.getSku());
        }

        Product product = new Product(
            request.getName(),
            request.getDescription(),
            request.getPrice(),
            request.getSku(),
            request.getStockQuantity()
        );

        if (request.getCategoryId() != null) {
            Category category = categoryRepository.findById(request.getCategoryId())
                .orElseThrow(() -> new ResourceNotFoundException(
                    "Category", request.getCategoryId()));
            product.setCategory(category);
        }

        Product saved = productRepository.save(product);
        return toResponse(saved);
    }

    @Transactional(readOnly = true)
    public ProductResponse getById(Long id) {
        Product product = productRepository.findWithCategoryById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Product", id));
        return toResponse(product);
    }

    @Transactional(readOnly = true)
    public Page<ProductResponse> getAll(Pageable pageable) {
        return productRepository.findByActive(true, pageable)
            .map(this::toResponse);
    }
```

```
    @Transactional(readOnly = true)
    public Page<ProductResponse> search(String name, Long categoryId,
                                        BigDecimal minPrice, BigDecimal maxPrice,
                                        Pageable pageable) {
        Specification<Product> spec = Specification.where(null);

        if (name != null && !name.isBlank()) {
            spec = spec.and((root, query, cb) ->
                cb.like(cb.lower(root.get("name")), "%" + name.toLowerCase() + "%"));
        }
        if (categoryId != null) {
            spec = spec.and((root, query, cb) ->
                cb.equal(root.get("category").get("id"), categoryId));
        }
        if (minPrice != null) {
            spec = spec.and((root, query, cb) ->
                cb.greaterThanOrEqualTo(root.get("price"), minPrice));
        }
        if (maxPrice != null) {
            spec = spec.and((root, query, cb) ->
                cb.lessThanOrEqualTo(root.get("price"), maxPrice));
        }
        spec = spec.and((root, query, cb) -> cb.isTrue(root.get("active")));

        return productRepository.findAll(spec, pageable).map(this::toResponse);
    }

    @Transactional
    public ProductResponse update(Long id, UpdateProductRequest request) {
        Product product = productRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Product", id));

        if (request.getName() != null) product.setName(request.getName());
        if (request.getDescription() != null) product.setDescription(request.getDescrip...
        if (request.getPrice() != null) product.setPrice(request.getPrice());
        if (request.getStockQuantity() != null) product.setStockQuantity(request.getSto...
        if (request.getActive() != null) product.setActive(request.getActive());

        if (request.getCategoryId() != null) {
            Category category = categoryRepository.findById(request.getCategoryId())
                .orElseThrow(() -> new ResourceNotFoundException(
                    "Category", request.getCategoryId()));
            product.setCategory(category);
        }

        return toResponse(product);
    }

    @Transactional
    public void delete(Long id) {
        if (!productRepository.existsById(id)) {
            throw new ResourceNotFoundException("Product", id);
        }
        productRepository.deleteById(id);
    }
```

```
    private ProductResponse toResponse(Product product) {
        return new ProductResponse(
            product.getId(),
            product.getName(),
            product.getDescription(),
            product.getPrice(),
            product.getSku(),
            product.getStockQuantity(),
            product.isActive(),
            product.getCategory() != null ? product.getCategory().getName() : null,
            product.getCreatedAt(),
            product.getUpdatedAt()
        );
    }
}
```

Controller Layer

```java
@RestController
@RequestMapping("/api/products")
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping
    public Page<ProductResponse> getAll(Pageable pageable) {
        return productService.getAll(pageable);
    }

    @GetMapping("/{id}")
    public ProductResponse getById(@PathVariable Long id) {
        return productService.getById(id);
    }

    @GetMapping("/search")
    public Page<ProductResponse> search(
            @RequestParam(required = false) String name,
            @RequestParam(required = false) Long categoryId,
            @RequestParam(required = false) BigDecimal minPrice,
            @RequestParam(required = false) BigDecimal maxPrice,
            Pageable pageable) {
```

```
        return productService.search(name, categoryId, minPrice, maxPrice, pageable);
    }

    @PostMapping
    public ResponseEntity<ProductResponse> create(
            @Valid @RequestBody CreateProductRequest request) {
        ProductResponse response = productService.create(request);
        URI location = URI.create("/api/products/" + response.getId());
        return ResponseEntity.created(location).body(response);
    }

    @PutMapping("/{id}")
    public ProductResponse update(
            @PathVariable Long id,
            @Valid @RequestBody UpdateProductRequest request) {
        return productService.update(id, request);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        productService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

```
Request/Response Flow:
+-------------------------------------------------------------------+
|                                                                   |
|  POST /api/products                                               |
|  { "name": "Widget", "price": 19.99, "sku": "WDG-001",          |
|    "stockQuantity": 100, "categoryId": 1 }                       |
|       |                                                           |
|       v                                                           |
|  Controller: @Valid validates request DTO                         |
|       |                                                           |
|       v                                                           |
|  Service: checks SKU uniqueness, loads category, saves entity    |
|       |                                                           |
|       v                                                           |
|  Response: 201 Created                                            |
|  Location: /api/products/42                                       |
|  {                                                                |
|    "id": 42,                                                      |
|    "name": "Widget",                                              |
|    "price": 19.99,                                                |
|    "sku": "WDG-001",                                              |
|    "stockQuantity": 100,                                          |
|    "active": true,                                                |
|    "categoryName": "Electronics",                                 |
|    "createdAt": "2025-09-15T14:30:00",                            |
|    "updatedAt": "2025-09-15T14:30:00"                             |
|  }                                                                |
+-------------------------------------------------------------------+
```

Global Exception Handler

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.cl...

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
            ResourceNotFoundException ex, HttpServletRequest request) {
        return buildResponse(404, ex.getErrorCode(), ex.getMessage(), request);
    }

    @ExceptionHandler(DuplicateResourceException.class)
    public ResponseEntity<ErrorResponse> handleDuplicate(
            DuplicateResourceException ex, HttpServletRequest request) {
        return buildResponse(409, ex.getErrorCode(), ex.getMessage(), request);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(
            MethodArgumentNotValidException ex, HttpServletRequest request) {
        Map<String, String> fieldErrors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors()
            .forEach(fe -> fieldErrors.put(fe.getField(), fe.getDefaultMessage()));

        ErrorResponse error = new ErrorResponse(400, "VALIDATION_FAILED",
            "Request validation failed", request.getRequestURI());
        error.setFieldErrors(fieldErrors);
        return ResponseEntity.badRequest().body(error);
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<ErrorResponse> handleDataIntegrity(
            DataIntegrityViolationException ex, HttpServletRequest request) {
        return buildResponse(409, "DATA_INTEGRITY",
            "A data integrity constraint was violated", request);
    }

    @ExceptionHandler(OptimisticLockingFailureException.class)
    public ResponseEntity<ErrorResponse> handleOptimisticLock(
            OptimisticLockingFailureException ex, HttpServletRequest request) {
        return buildResponse(409, "CONCURRENT_UPDATE",
            "Record was modified by another user. Please refresh.", request);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneral(
```

```
            Exception ex, HttpServletRequest request) {
        log.error("Unhandled exception at {}", request.getRequestURI(), ex);
        return buildResponse(500, "INTERNAL_ERROR",
            "An unexpected error occurred", request);
    }

    private ResponseEntity<ErrorResponse> buildResponse(
            int status, String code, String message, HttpServletRequest request) {
        ErrorResponse error = new ErrorResponse(status, code, message,
            request.getRequestURI());
        return ResponseEntity.status(status).body(error);
    }
}
```

Testing with MockMvc

```java
@WebMvcTest(ProductController.class)
class ProductControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private ProductService productService;

    @Autowired
    private ObjectMapper objectMapper;

    @Test
    void createProduct_validRequest_returns201() throws Exception {
        CreateProductRequest request = new CreateProductRequest();
        request.setName("Widget");
        request.setPrice(new BigDecimal("19.99"));
        request.setSku("WDG-001");
        request.setStockQuantity(100);

        ProductResponse response = new ProductResponse(
            1L, "Widget", null, new BigDecimal("19.99"), "WDG-001",
            100, true, null, LocalDateTime.now(), LocalDateTime.now());

        when(productService.create(any())).thenReturn(response);

        mockMvc.perform(post("/api/products")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isCreated())
```

```
            .andExpect(jsonPath("$.id").value(1))
            .andExpect(jsonPath("$.name").value("Widget"))
            .andExpect(jsonPath("$.sku").value("WDG-001"))
            .andExpect(header().string("Location", "/api/products/1"));
    }

    @Test
    void createProduct_invalidRequest_returns400() throws Exception {
        CreateProductRequest request = new CreateProductRequest();
        // Missing all required fields

        mockMvc.perform(post("/api/products")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(request)))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.error").value("VALIDATION_FAILED"))
            .andExpect(jsonPath("$.fieldErrors.name").value("Product name is required"))
            .andExpect(jsonPath("$.fieldErrors.price").value("Price is required"));
    }

    @Test
    void getById_notFound_returns404() throws Exception {
        when(productService.getById(999L))
            .thenThrow(new ResourceNotFoundException("Product", 999L));

        mockMvc.perform(get("/api/products/999"))
            .andExpect(status().isNotFound())
            .andExpect(jsonPath("$.error").value("NOT_FOUND"))
            .andExpect(jsonPath("$.message").value("Product with id 999 not found"));
    }

    @Test
    void getAll_returnsPaginatedResults() throws Exception {
        List<ProductResponse> products = List.of(
            new ProductResponse(1L, "Widget", null, new BigDecimal("19.99"),
                "WDG-001", 100, true, "Electronics",
                LocalDateTime.now(), LocalDateTime.now())
        );
        Page<ProductResponse> page = new PageImpl<>(products, PageRequest.of(0, 20), 1);

        when(productService.getAll(any())).thenReturn(page);

        mockMvc.perform(get("/api/products")
                .param("page", "0")
                .param("size", "20")
                .param("sort", "name,asc"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.content").isArray())
            .andExpect(jsonPath("$.content[0].name").value("Widget"))
            .andExpect(jsonPath("$.totalElements").value(1));
    }

    @Test
    void delete_returnsNoContent() throws Exception {
        doNothing().when(productService).delete(1L);
```

```
        mockMvc.perform(delete("/api/products/1"))
            .andExpect(status().isNoContent());
    }
}
```

Application Configuration

```properties
# application.properties

# Database
spring.datasource.url=jdbc:h2:mem:productdb
spring.datasource.driver-class-name=org.h2.Driver
spring.h2.console.enabled=true

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.open-in-view=false
spring.jpa.properties.hibernate.jdbc.batch_size=50
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.default_batch_fetch_size=16

# Flyway
spring.flyway.enabled=true

# Logging (development)
logging.level.org.hibernate.SQL=DEBUG

# Jackson
spring.jackson.serialization.write-dates-as-timestamps=false

# Pagination defaults
spring.data.web.pageable.default-page-size=20
spring.data.web.pageable.max-page-size=100
```

Patterns Applied

```
Patterns from This Book Used in the API:
+-----------------------------------------------------------------+
|                                                                 |
```

```
|  Ch  Pattern                      Applied In                      |
|  ----------------------------------------------------------------|
|  4   JPA annotations              Entity mappings                 |
|  6   CRUD operations              Service create/read/update/del  |
|  7   Derived queries              Repository methods              |
|  10  Specifications               Search with dynamic filters     |
|  11  Pagination                   Page<T> in list endpoints       |
|  13  @ManyToOne relationship      Product -> Category             |
|  17  Persistence context          Dirty checking in update()     |
|  18  @EntityGraph                 Eager loading for detail view   |
|  20  @Transactional               Service method boundaries      |
|  22  Bean Validation              @Valid on request DTOs          |
|  23  DTOs and Projections         Request/Response DTOs           |
|  24  Layered architecture         Controller/Service/Repository   |
|  25  Exception handling           @ControllerAdvice               |
|  26  Flyway                       ddl-auto=validate               |
|  27  Performance                  Batch size, OSIV=false          |
+-------------------------------------------------------------------+
```

Common Mistakes

1. Returning entities from controllers: Always map to response DTOs. Entities expose internal structure, cause circular references, and trigger lazy loading issues.
2. Putting validation in the service instead of the DTO: Use `@Valid` on request DTOs in the controller. This catches errors before reaching the service.
3. Not using readOnly on GET methods: All read operations should use `@Transactional(readOnly = true)` for performance.
4. Hardcoding page size without limits: Always configure `max-page-size` to prevent `?size=1000000` requests.
5. Inconsistent error responses: Every error should follow the same format (status, error code, message, path, timestamp).

Best Practices

1. Separate request and response DTOs: Different fields, different validation, different purposes.
2. Use 201 Created with Location header for POST: Tell the client where to find the new resource.

3. Use 204 No Content for DELETE: Acknowledge success without returning data.
4. Use Pageable for all list endpoints: Every list can grow. Always paginate.
5. Test every endpoint and error case: Happy paths, validation errors, not found, duplicates.
6. Disable open-in-view: Force all data loading to happen in the service layer.

Summary

This chapter brought together all the patterns from the book into a complete, production-quality REST API:

- Entities with proper annotations, relationships, validation, and a shared BaseEntity.
- DTOs for request validation and response formatting, never exposing entities.
- Repositories with derived methods, @EntityGraph, projections, and Specifications.
- Services with business logic, transaction management, and entity-DTO mapping.
- Controllers with proper HTTP methods, status codes, and validated request bodies.
- Global exception handling for consistent, informative error responses.
- Tests with MockMvc for every endpoint and error scenario.
- Configuration with OSIV disabled, Flyway, batch processing, and pagination limits.

Interview Questions

Q1: What HTTP status codes should a REST API return for CRUD operations?

GET: 200 OK. POST: 201 Created (with Location header). PUT/PATCH: 200 OK (with updated resource). DELETE: 204 No Content. Errors: 400 for validation, 404 for not found, 409 for conflicts, 500 for server errors.

Q2: Why should you use separate DTOs for request and response?

Request DTOs have validation annotations and only include fields the client can set. Response DTOs include computed fields (timestamps, category names) and exclude sensitive data. They serve different purposes and change independently.

Q3: How do you implement search with multiple optional filters?

Use Spring Data Specifications (JpaSpecificationExecutor). Build predicates conditionally for each non-null filter parameter. Combine with `Specification.and()`. Pass to `repository.findAll(spec, pageable)`.

Q4: What is the benefit of using @EntityGraph in the detail endpoint but not the list endpoint?

The detail endpoint needs full data (category name, etc.), so @EntityGraph prevents N+1. The list endpoint may use a lighter projection that does not need relationships, avoiding unnecessary JOINs and data transfer.

Practice Exercises

Exercise 1: Complete CRUD API Build the Product Management API from this chapter. Test all endpoints with an HTTP client (curl, Postman, or HTTPie).

Exercise 2: Add Category CRUD Add full CRUD endpoints for Categories. Include listing products by category with pagination.

Exercise 3: Search Endpoint Implement the `/api/products/search` endpoint with filters for name, category, price range, and active status. Test with various filter combinations.

Exercise 4: Test Coverage Write MockMvc tests for every endpoint: create (valid/invalid), get (found/not found), update, delete, list with pagination, search with filters.

Exercise 5: API Documentation Add SpringDoc OpenAPI (`springdoc-openapi-starter-webmvc-ui`) dependency. Add `@Operation` and `@ApiResponse` annotations. Access the Swagger UI at `/swagger-ui.html`.

What Is Next?

In the next chapter, we will build a complete capstone project — a Library Management System with complex entity relationships, full CRUD APIs, search and filtering, Flyway migrations, and comprehensive tests. This will be the culmination of everything you have learned.

Chapter 29: Capstone Project — Library Management System

Learning Goals

By the end of this chapter, you will have built a complete, production-quality application that demonstrates:

- Complex entity relationships (one-to-many, many-to-many with extra columns)
- Full CRUD REST APIs with DTOs and validation
- Dynamic search with Specifications
- Pagination and sorting across all list endpoints
- Flyway database migrations
- Global exception handling
- Layered architecture (Controller-Service-Repository)
- Performance optimizations (batching, @EntityGraph, readOnly)
- Comprehensive test coverage

Project Overview

We will build a Library Management System that tracks books, authors, members, and book loans. This is a realistic application with the kind of entity relationships and business rules you encounter in production systems.

```
Library Management System:
+-------------------------------------------------------------------+
|                                                                   |
```

```
|  Entities and Relationships:                                          |
|                                                                       |
|  Author ----< BookAuthor >---- Book                                   |
|  (many-to-many with display_order)                                    |
|                                                                       |
|  Book ----< BookCopy                                                  |
|  (one book can have multiple physical copies)                         |
|                                                                       |
|  Member ----< Loan >---- BookCopy                                     |
|  (a member borrows a specific copy)                                   |
|                                                                       |
|  Book ----< Book                                                      |
|  (via Category: one category has many books)                          |
|                                                                       |
+-----------------------------------------------------------------------+

API Endpoints:
+-----------------------------------------------------------------------+
|                                                                       |
|  Authors:                                                             |
|  GET    /api/authors               List (paginated)                   |
|  GET    /api/authors/{id}          Get with books                     |
|  POST   /api/authors               Create                             |
|  PUT    /api/authors/{id}          Update                             |
|                                                                       |
|  Books:                                                               |
|  GET    /api/books                 List (paginated)                   |
|  GET    /api/books/{id}            Get with authors and copies        |
|  POST   /api/books                 Create with author IDs             |
|  PUT    /api/books/{id}            Update                             |
|  GET    /api/books/search          Search by title/author/isbn        |
|                                                                       |
|  Members:                                                             |
|  GET    /api/members               List (paginated)                   |
|  GET    /api/members/{id}          Get with active loans              |
|  POST   /api/members               Register                          |
|                                                                       |
|  Loans:                                                               |
|  POST   /api/loans/checkout        Borrow a book                      |
|  POST   /api/loans/{id}/return     Return a book                      |
|  GET    /api/loans/overdue         List overdue loans                 |
|  GET    /api/members/{id}/loans    Member's loan history              |
+-----------------------------------------------------------------------+
```

Step 1: Entity Design

BaseEntity

```java
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class BaseEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @CreatedDate
    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @Version
    private Integer version;

    public Long getId() { return id; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public LocalDateTime getUpdatedAt() { return updatedAt; }
}
```

Author

```java
@Entity
@Table(name = "authors")
public class Author extends BaseEntity {

    @NotBlank
    @Column(nullable = false, length = 150)
    private String name;

    @Column(columnDefinition = "TEXT")
    private String biography;

    @Column(name = "birth_year")
    private Integer birthYear;

    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval = true)
    @OrderBy("displayOrder ASC")
    private List<BookAuthor> bookAuthors = new ArrayList<>();

    protected Author() {}

    public Author(String name) {
        this.name = name;
    }
```

```
    // Getters and setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getBiography() { return biography; }
    public void setBiography(String biography) { this.biography = biography; }
    public Integer getBirthYear() { return birthYear; }
    public void setBirthYear(Integer birthYear) { this.birthYear = birthYear; }
    public List<BookAuthor> getBookAuthors() {
        return Collections.unmodifiableList(bookAuthors);
    }
}
```

Book

```java
@Entity
@Table(name = "books", indexes = {
    @Index(name = "idx_book_isbn", columnList = "isbn"),
    @Index(name = "idx_book_category", columnList = "category_id")
})
public class Book extends BaseEntity {

    @NotBlank
    @Column(nullable = false, length = 300)
    private String title;

    @Column(nullable = false, unique = true, length = 20)
    private String isbn;

    @Column(columnDefinition = "TEXT")
    private String description;

    @Column(name = "publication_year")
    private Integer publicationYear;

    @Column(length = 200)
    private String publisher;

    @Column(name = "page_count")
    private Integer pageCount;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "category_id")
    private Category category;

    @OneToMany(mappedBy = "book", cascade = CascadeType.ALL, orphanRemoval = true)
    @OrderBy("displayOrder ASC")
    private List<BookAuthor> bookAuthors = new ArrayList<>();

    @OneToMany(mappedBy = "book", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<BookCopy> copies = new ArrayList<>();
```

```
    protected Book() {}

    public Book(String title, String isbn) {
        this.title = title;
        this.isbn = isbn;
    }

    // Helper methods
    public void addAuthor(Author author, int displayOrder) {
        BookAuthor ba = new BookAuthor(this, author, displayOrder);
        bookAuthors.add(ba);
        author.getBookAuthors(); // ensure loaded
    }

    public BookCopy addCopy(String barcode) {
        BookCopy copy = new BookCopy(this, barcode);
        copies.add(copy);
        return copy;
    }

    public int getAvailableCopyCount() {
        return (int) copies.stream().filter(BookCopy::isAvailable).count();
    }

    // Getters and setters
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getIsbn() { return isbn; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
    public Integer getPublicationYear() { return publicationYear; }
    public void setPublicationYear(Integer publicationYear) { this.publicationYear = pu...
    public String getPublisher() { return publisher; }
    public void setPublisher(String publisher) { this.publisher = publisher; }
    public Integer getPageCount() { return pageCount; }
    public void setPageCount(Integer pageCount) { this.pageCount = pageCount; }
    public Category getCategory() { return category; }
    public void setCategory(Category category) { this.category = category; }
    public List<BookAuthor> getBookAuthors() {
        return Collections.unmodifiableList(bookAuthors);
    }
    public List<BookCopy> getCopies() {
        return Collections.unmodifiableList(copies);
    }
}
```

BookAuthor (Many-to-Many Join Entity)

```java
@Entity
@Table(name = "book_authors")
public class BookAuthor {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "book_id", nullable = false)
    private Book book;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id", nullable = false)
    private Author author;

    @Column(name = "display_order", nullable = false)
    private int displayOrder;

    protected BookAuthor() {}

    public BookAuthor(Book book, Author author, int displayOrder) {
        this.book = book;
        this.author = author;
        this.displayOrder = displayOrder;
    }

    public Long getId() { return id; }
    public Book getBook() { return book; }
    public Author getAuthor() { return author; }
    public int getDisplayOrder() { return displayOrder; }
}
```

BookCopy

```java
@Entity
@Table(name = "book_copies")
public class BookCopy extends BaseEntity {

    @Column(nullable = false, unique = true, length = 50)
    private String barcode;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private CopyStatus status = CopyStatus.AVAILABLE;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "book_id", nullable = false)
    private Book book;
```

```
    @OneToMany(mappedBy = "bookCopy")
    @OrderBy("borrowedAt DESC")
    private List<Loan> loans = new ArrayList<>();

    protected BookCopy() {}

    public BookCopy(Book book, String barcode) {
        this.book = book;
        this.barcode = barcode;
    }

    public boolean isAvailable() {
        return status == CopyStatus.AVAILABLE;
    }

    public void markBorrowed() { this.status = CopyStatus.BORROWED; }
    public void markAvailable() { this.status = CopyStatus.AVAILABLE; }
    public void markDamaged() { this.status = CopyStatus.DAMAGED; }

    // Getters
    public String getBarcode() { return barcode; }
    public CopyStatus getStatus() { return status; }
    public Book getBook() { return book; }
}

public enum CopyStatus {
    AVAILABLE, BORROWED, DAMAGED, LOST
}
```

Member

```java
@Entity
@Table(name = "members")
public class Member extends BaseEntity {

    @NotBlank
    @Column(nullable = false, length = 150)
    private String name;

    @NotBlank
    @Email
    @Column(nullable = false, unique = true, length = 200)
    private String email;

    @Column(name = "phone_number", length = 20)
    private String phoneNumber;

    @Column(name = "membership_date", nullable = false)
    private LocalDate membershipDate;
```

```
    @Column(nullable = false)
    private boolean active = true;

    @Column(name = "max_loans", nullable = false)
    private int maxLoans = 5;

    @OneToMany(mappedBy = "member")
    @OrderBy("borrowedAt DESC")
    private List<Loan> loans = new ArrayList<>();

    protected Member() {}

    public Member(String name, String email) {
        this.name = name;
        this.email = email;
        this.membershipDate = LocalDate.now();
    }

    public int getActiveLoansCount() {
        return (int) loans.stream().filter(l -> l.getReturnedAt() == null).count();
    }

    public boolean canBorrow() {
        return active && getActiveLoansCount() < maxLoans;
    }

    // Getters and setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getPhoneNumber() { return phoneNumber; }
    public void setPhoneNumber(String phoneNumber) { this.phoneNumber = phoneNumber; }
    public LocalDate getMembershipDate() { return membershipDate; }
    public boolean isActive() { return active; }
    public void setActive(boolean active) { this.active = active; }
    public int getMaxLoans() { return maxLoans; }
    public List<Loan> getLoans() { return Collections.unmodifiableList(loans); }
}
```

Loan

```java
@Entity
@Table(name = "loans", indexes = {
    @Index(name = "idx_loan_member", columnList = "member_id"),
    @Index(name = "idx_loan_copy", columnList = "book_copy_id"),
    @Index(name = "idx_loan_due_date", columnList = "due_date")
})
public class Loan extends BaseEntity {
```

```
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "member_id", nullable = false)
    private Member member;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "book_copy_id", nullable = false)
    private BookCopy bookCopy;

    @Column(name = "borrowed_at", nullable = false)
    private LocalDateTime borrowedAt;

    @Column(name = "due_date", nullable = false)
    private LocalDate dueDate;

    @Column(name = "returned_at")
    private LocalDateTime returnedAt;

    @Column(columnDefinition = "TEXT")
    private String notes;

    protected Loan() {}

    public Loan(Member member, BookCopy bookCopy, int loanDurationDays) {
        this.member = member;
        this.bookCopy = bookCopy;
        this.borrowedAt = LocalDateTime.now();
        this.dueDate = LocalDate.now().plusDays(loanDurationDays);
    }

    public boolean isOverdue() {
        return returnedAt == null && LocalDate.now().isAfter(dueDate);
    }

    public boolean isActive() {
        return returnedAt == null;
    }

    public void markReturned() {
        this.returnedAt = LocalDateTime.now();
    }

    // Getters
    public Member getMember() { return member; }
    public BookCopy getBookCopy() { return bookCopy; }
    public LocalDateTime getBorrowedAt() { return borrowedAt; }
    public LocalDate getDueDate() { return dueDate; }
    public LocalDateTime getReturnedAt() { return returnedAt; }
    public String getNotes() { return notes; }
    public void setNotes(String notes) { this.notes = notes; }
}
```

Category

```java
@Entity
@Table(name = "categories")
public class Category extends BaseEntity {

    @NotBlank
    @Column(nullable = false, unique = true, length = 100)
    private String name;

    private String description;

    @OneToMany(mappedBy = "category")
    private List<Book> books = new ArrayList<>();

    protected Category() {}

    public Category(String name) { this.name = name; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }
}
```

```
Entity Relationship Diagram:
+-------------------------------------------------------------------+
|                                                                   |
|  categories (1)----(*) books (1)----(*) book_copies               |
|                         |                    |                    |
|                         |                    |                    |
|                   book_authors          loans                    |
|                   (join entity)         |     |                  |
|                         |               |     |                  |
|                   authors (*)     members (1)--+                  |
|                                                                   |
|  Relationships:                                                   |
|  Category  1:N  Book        (one category, many books)            |
|  Book      M:N  Author      (via BookAuthor with display_order)   |
|  Book      1:N  BookCopy    (one book, many physical copies)      |
|  Member    1:N  Loan        (one member, many loans over time)    |
|  BookCopy  1:N  Loan        (one copy, many loans over time)      |
+-------------------------------------------------------------------+
```

Step 2: Flyway Migrations

```sql
-- V1__Create_categories_table.sql
CREATE TABLE categories (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    description VARCHAR(500),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP,
    version INTEGER DEFAULT 0
);
```

```sql
-- V2__Create_authors_table.sql
CREATE TABLE authors (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    biography TEXT,
    birth_year INTEGER,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP,
    version INTEGER DEFAULT 0
);
```

```sql
-- V3__Create_books_table.sql
CREATE TABLE books (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(300) NOT NULL,
    isbn VARCHAR(20) NOT NULL UNIQUE,
    description TEXT,
    publication_year INTEGER,
    publisher VARCHAR(200),
    page_count INTEGER,
    category_id BIGINT,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP,
    version INTEGER DEFAULT 0,
    FOREIGN KEY (category_id) REFERENCES categories(id)
);
CREATE INDEX idx_book_isbn ON books(isbn);
CREATE INDEX idx_book_category ON books(category_id);
```

```sql
-- V4__Create_book_authors_table.sql
CREATE TABLE book_authors (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    book_id BIGINT NOT NULL,
    author_id BIGINT NOT NULL,
    display_order INTEGER NOT NULL DEFAULT 0,
    FOREIGN KEY (book_id) REFERENCES books(id),
    FOREIGN KEY (author_id) REFERENCES authors(id),
    UNIQUE (book_id, author_id)
);
```

```sql
-- V5__Create_book_copies_table.sql
CREATE TABLE book_copies (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    barcode VARCHAR(50) NOT NULL UNIQUE,
    status VARCHAR(20) NOT NULL DEFAULT 'AVAILABLE',
    book_id BIGINT NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP,
    version INTEGER DEFAULT 0,
    FOREIGN KEY (book_id) REFERENCES books(id)
);
```

```sql
-- V6__Create_members_table.sql
CREATE TABLE members (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    email VARCHAR(200) NOT NULL UNIQUE,
    phone_number VARCHAR(20),
    membership_date DATE NOT NULL,
    active BOOLEAN DEFAULT TRUE,
    max_loans INTEGER DEFAULT 5,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP,
    version INTEGER DEFAULT 0
);
```

```sql
-- V7__Create_loans_table.sql
CREATE TABLE loans (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    member_id BIGINT NOT NULL,
    book_copy_id BIGINT NOT NULL,
    borrowed_at TIMESTAMP NOT NULL,
    due_date DATE NOT NULL,
    returned_at TIMESTAMP,
    notes TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP,
    version INTEGER DEFAULT 0,
    FOREIGN KEY (member_id) REFERENCES members(id),
    FOREIGN KEY (book_copy_id) REFERENCES book_copies(id)
);
CREATE INDEX idx_loan_member ON loans(member_id);
CREATE INDEX idx_loan_copy ON loans(book_copy_id);
CREATE INDEX idx_loan_due_date ON loans(due_date);
```

```sql
-- V8__Insert_seed_data.sql
INSERT INTO categories (name, description) VALUES
    ('Fiction', 'Novels, short stories, and literary fiction'),
```

```
    ('Non-Fiction', 'Biographies, history, science, and essays'),
    ('Science Fiction', 'Futuristic and speculative fiction'),
    ('Programming', 'Software development and computer science'),
    ('History', 'Historical accounts and analysis');

INSERT INTO authors (name, biography, birth_year) VALUES
    ('Robert C. Martin', 'Software engineer and author of Clean Code', 1952),
    ('Joshua Bloch', 'Former chief Java architect at Google', 1961),
    ('Martin Fowler', 'Author and software development thought leader', 1963);

INSERT INTO books (title, isbn, description, publication_year, publisher, page_count, c...
    ('Clean Code', '978-0132350884', 'A handbook of agile software craftsmanship', 2008...
    ('Effective Java', '978-0134685991', 'Best practices for the Java platform', 2018, ...

INSERT INTO book_authors (book_id, author_id, display_order) VALUES
    (1, 1, 1),
    (2, 2, 1);

INSERT INTO book_copies (barcode, status, book_id) VALUES
    ('CC-001', 'AVAILABLE', 1),
    ('CC-002', 'AVAILABLE', 1),
    ('EJ-001', 'AVAILABLE', 2),
    ('EJ-002', 'AVAILABLE', 2),
    ('EJ-003', 'AVAILABLE', 2);

INSERT INTO members (name, email, membership_date) VALUES
    ('Alice Johnson', 'alice@library.com', '2024-01-15'),
    ('Bob Smith', 'bob@library.com', '2024-03-20');
```

Step 3: Loan Service (Business Logic)

The loan service demonstrates complex business rules:

```java
@Service
public class LoanService {

    private static final int DEFAULT_LOAN_DAYS = 14;

    private final LoanRepository loanRepository;
    private final MemberRepository memberRepository;
    private final BookCopyRepository bookCopyRepository;
    private final BookRepository bookRepository;

    public LoanService(LoanRepository loanRepository,
                       MemberRepository memberRepository,
                       BookCopyRepository bookCopyRepository,
```

```
                        BookRepository bookRepository) {
        this.loanRepository = loanRepository;
        this.memberRepository = memberRepository;
        this.bookCopyRepository = bookCopyRepository;
        this.bookRepository = bookRepository;
    }

    @Transactional
    public LoanResponse checkout(CheckoutRequest request) {
        // 1. Validate member
        Member member = memberRepository.findById(request.getMemberId())
            .orElseThrow(() -> new ResourceNotFoundException("Member", request.getMembe...

        if (!member.isActive()) {
            throw new BusinessException("INACTIVE_MEMBER", "Member account is inactive");
        }

        // 2. Check loan limit
        long activeLoans = loanRepository.countByMemberIdAndReturnedAtIsNull(member.get...
        if (activeLoans >= member.getMaxLoans()) {
            throw new BusinessException("LOAN_LIMIT",
                String.format("Member has reached the maximum of %d active loans",
                    member.getMaxLoans()));
        }

        // 3. Find an available copy
        BookCopy copy;
        if (request.getBarcode() != null) {
            copy = bookCopyRepository.findByBarcode(request.getBarcode())
                .orElseThrow(() -> new ResourceNotFoundException(
                    "BookCopy", "barcode", request.getBarcode()));
        } else if (request.getBookId() != null) {
            copy = bookCopyRepository.findFirstByBookIdAndStatus(
                    request.getBookId(), CopyStatus.AVAILABLE)
                .orElseThrow(() -> new BusinessException("NO_COPIES",
                    "No available copies of this book"));
        } else {
            throw new BusinessException("INVALID_REQUEST",
                "Either bookId or barcode must be provided");
        }

        if (!copy.isAvailable()) {
            throw new BusinessException("COPY_UNAVAILABLE",
                "This copy is currently " + copy.getStatus());
        }

        // 4. Check if member already has this book
        boolean alreadyBorrowed = loanRepository
            .existsByMemberIdAndBookCopy_Book_IdAndReturnedAtIsNull(
                member.getId(), copy.getBook().getId());
        if (alreadyBorrowed) {
            throw new BusinessException("ALREADY_BORROWED",
                "Member already has a copy of this book on loan");
        }
```

```
        // 5. Create the loan
        copy.markBorrowed();
        Loan loan = new Loan(member, copy, DEFAULT_LOAN_DAYS);
        Loan saved = loanRepository.save(loan);

        return toLoanResponse(saved);
    }

    @Transactional
    public LoanResponse returnBook(Long loanId) {
        Loan loan = loanRepository.findWithDetailsById(loanId)
            .orElseThrow(() -> new ResourceNotFoundException("Loan", loanId));

        if (loan.getReturnedAt() != null) {
            throw new BusinessException("ALREADY_RETURNED", "This loan was already retu...
        }

        loan.markReturned();
        loan.getBookCopy().markAvailable();

        return toLoanResponse(loan);
    }

    @Transactional(readOnly = true)
    public Page<LoanResponse> getOverdueLoans(Pageable pageable) {
        return loanRepository
            .findByReturnedAtIsNullAndDueDateBefore(LocalDate.now(), pageable)
            .map(this::toLoanResponse);
    }

    @Transactional(readOnly = true)
    public Page<LoanResponse> getMemberLoans(Long memberId, Pageable pageable) {
        if (!memberRepository.existsById(memberId)) {
            throw new ResourceNotFoundException("Member", memberId);
        }
        return loanRepository.findByMemberId(memberId, pageable)
            .map(this::toLoanResponse);
    }

    private LoanResponse toLoanResponse(Loan loan) {
        return new LoanResponse(
            loan.getId(),
            loan.getMember().getName(),
            loan.getBookCopy().getBook().getTitle(),
            loan.getBookCopy().getBarcode(),
            loan.getBorrowedAt(),
            loan.getDueDate(),
            loan.getReturnedAt(),
            loan.isOverdue()
        );
    }
}
```

```
Checkout Business Rules:
+-------------------------------------------------------------------+
|                                                                    |
|  1. Member must exist and be active                                |
|  2. Member must not have exceeded max loans (default 5)            |
|  3. Book copy must be available                                    |
|  4. Member cannot borrow the same book twice simultaneously       |
|  5. Loan duration: 14 days                                         |
|                                                                    |
|  Return Business Rules:                                            |
|  1. Loan must exist                                                |
|  2. Loan must not already be returned                              |
|  3. Book copy status changes back to AVAILABLE                    |
|                                                                    |
|  If ANY rule fails, the entire operation rolls back.               |
+-------------------------------------------------------------------+
```

Step 4: DTOs

```java
public class CheckoutRequest {

    @NotNull(message = "Member ID is required")
    private Long memberId;

    private Long bookId;      // Borrow any available copy of this book
    private String barcode;   // Borrow a specific copy

    // Getters/setters...
    public Long getMemberId() { return memberId; }
    public void setMemberId(Long memberId) { this.memberId = memberId; }
    public Long getBookId() { return bookId; }
    public void setBookId(Long bookId) { this.bookId = bookId; }
    public String getBarcode() { return barcode; }
    public void setBarcode(String barcode) { this.barcode = barcode; }
}

public class LoanResponse {
    private Long id;
    private String memberName;
    private String bookTitle;
    private String copyBarcode;
    private LocalDateTime borrowedAt;
    private LocalDate dueDate;
    private LocalDateTime returnedAt;
    private boolean overdue;
```

```
    public LoanResponse(Long id, String memberName, String bookTitle,
                        String copyBarcode, LocalDateTime borrowedAt,
                        LocalDate dueDate, LocalDateTime returnedAt,
                        boolean overdue) {
        this.id = id;
        this.memberName = memberName;
        this.bookTitle = bookTitle;
        this.copyBarcode = copyBarcode;
        this.borrowedAt = borrowedAt;
        this.dueDate = dueDate;
        this.returnedAt = returnedAt;
        this.overdue = overdue;
    }

    // Getters...
    public Long getId() { return id; }
    public String getMemberName() { return memberName; }
    public String getBookTitle() { return bookTitle; }
    public String getCopyBarcode() { return copyBarcode; }
    public LocalDateTime getBorrowedAt() { return borrowedAt; }
    public LocalDate getDueDate() { return dueDate; }
    public LocalDateTime getReturnedAt() { return returnedAt; }
    public boolean isOverdue() { return overdue; }
}

public class BookResponse {
    private Long id;
    private String title;
    private String isbn;
    private String description;
    private Integer publicationYear;
    private String publisher;
    private Integer pageCount;
    private String categoryName;
    private List<String> authors;
    private int totalCopies;
    private int availableCopies;

    // Constructor, getters...
    public BookResponse(Long id, String title, String isbn, String description,
                        Integer publicationYear, String publisher, Integer pageCount,
                        String categoryName, List<String> authors,
                        int totalCopies, int availableCopies) {
        this.id = id;
        this.title = title;
        this.isbn = isbn;
        this.description = description;
        this.publicationYear = publicationYear;
        this.publisher = publisher;
        this.pageCount = pageCount;
        this.categoryName = categoryName;
        this.authors = authors;
        this.totalCopies = totalCopies;
        this.availableCopies = availableCopies;
    }
```

```
    public Long getId() { return id; }
    public String getTitle() { return title; }
    public String getIsbn() { return isbn; }
    public String getDescription() { return description; }
    public Integer getPublicationYear() { return publicationYear; }
    public String getPublisher() { return publisher; }
    public Integer getPageCount() { return pageCount; }
    public String getCategoryName() { return categoryName; }
    public List<String> getAuthors() { return authors; }
    public int getTotalCopies() { return totalCopies; }
    public int getAvailableCopies() { return availableCopies; }
}
```

Step 5: Repositories

```java
@Repository
public interface LoanRepository extends JpaRepository<Loan, Long> {

    long countByMemberIdAndReturnedAtIsNull(Long memberId);

    boolean existsByMemberIdAndBookCopy_Book_IdAndReturnedAtIsNull(
        Long memberId, Long bookId);

    @Query("SELECT l FROM Loan l " +
           "JOIN FETCH l.member " +
           "JOIN FETCH l.bookCopy bc " +
           "JOIN FETCH bc.book " +
           "WHERE l.id = :id")
    Optional<Loan> findWithDetailsById(@Param("id") Long id);

    @Query("SELECT l FROM Loan l " +
           "JOIN FETCH l.member " +
           "JOIN FETCH l.bookCopy bc " +
           "JOIN FETCH bc.book " +
           "WHERE l.returnedAt IS NULL AND l.dueDate < :date")
    Page<Loan> findByReturnedAtIsNullAndDueDateBefore(
        @Param("date") LocalDate date, Pageable pageable);

    @EntityGraph(attributePaths = {"member", "bookCopy", "bookCopy.book"})
    Page<Loan> findByMemberId(Long memberId, Pageable pageable);
}

@Repository
public interface BookCopyRepository extends JpaRepository<BookCopy, Long> {
```

```
    Optional<BookCopy> findByBarcode(String barcode);

    Optional<BookCopy> findFirstByBookIdAndStatus(Long bookId, CopyStatus status);
}

@Repository
public interface BookRepository extends JpaRepository<Book, Long>,
        JpaSpecificationExecutor<Book> {

    boolean existsByIsbn(String isbn);

    @Query("SELECT DISTINCT b FROM Book b " +
           "LEFT JOIN FETCH b.bookAuthors ba " +
           "LEFT JOIN FETCH ba.author " +
           "LEFT JOIN FETCH b.category " +
           "LEFT JOIN FETCH b.copies " +
           "WHERE b.id = :id")
    Optional<Book> findWithDetailsById(@Param("id") Long id);
}
```

Step 6: Controllers

```java
@RestController
@RequestMapping("/api/loans")
public class LoanController {

    private final LoanService loanService;

    public LoanController(LoanService loanService) {
        this.loanService = loanService;
    }

    @PostMapping("/checkout")
    public ResponseEntity<LoanResponse> checkout(
            @Valid @RequestBody CheckoutRequest request) {
        LoanResponse response = loanService.checkout(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    @PostMapping("/{id}/return")
    public LoanResponse returnBook(@PathVariable Long id) {
        return loanService.returnBook(id);
    }

    @GetMapping("/overdue")
    public Page<LoanResponse> getOverdue(Pageable pageable) {
```

```
        return loanService.getOverdueLoans(pageable);
    }
}

@RestController
@RequestMapping("/api/members")
public class MemberController {

    private final MemberService memberService;
    private final LoanService loanService;

    public MemberController(MemberService memberService, LoanService loanService) {
        this.memberService = memberService;
        this.loanService = loanService;
    }

    @GetMapping
    public Page<MemberResponse> getAll(Pageable pageable) {
        return memberService.getAll(pageable);
    }

    @GetMapping("/{id}")
    public MemberResponse getById(@PathVariable Long id) {
        return memberService.getById(id);
    }

    @PostMapping
    public ResponseEntity<MemberResponse> register(
            @Valid @RequestBody CreateMemberRequest request) {
        MemberResponse response = memberService.register(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    @GetMapping("/{id}/loans")
    public Page<LoanResponse> getMemberLoans(
            @PathVariable Long id, Pageable pageable) {
        return loanService.getMemberLoans(id, pageable);
    }
}
```

Step 7: Testing the Loan Flow

```java
@SpringBootTest
@Transactional
class LoanServiceTest {
```

```
    @Autowired private LoanService loanService;
    @Autowired private MemberRepository memberRepository;
    @Autowired private BookRepository bookRepository;
    @Autowired private BookCopyRepository bookCopyRepository;
    @Autowired private LoanRepository loanRepository;

    private Member member;
    private Book book;
    private BookCopy copy;

    @BeforeEach
    void setUp() {
        member = memberRepository.save(new Member("Test User", "test@lib.com"));
        book = new Book("Test Book", "978-0000000001");
        copy = book.addCopy("TEST-001");
        bookRepository.save(book);
    }

    @Test
    void checkout_validRequest_createsLoan() {
        CheckoutRequest request = new CheckoutRequest();
        request.setMemberId(member.getId());
        request.setBookId(book.getId());

        LoanResponse response = loanService.checkout(request);

        assertThat(response.getMemberName()).isEqualTo("Test User");
        assertThat(response.getBookTitle()).isEqualTo("Test Book");
        assertThat(response.getCopyBarcode()).isEqualTo("TEST-001");
        assertThat(response.isOverdue()).isFalse();
    }

    @Test
    void checkout_noCopiesAvailable_throwsException() {
        copy.markBorrowed();

        CheckoutRequest request = new CheckoutRequest();
        request.setMemberId(member.getId());
        request.setBookId(book.getId());

        assertThatThrownBy(() -> loanService.checkout(request))
            .isInstanceOf(BusinessException.class)
            .hasMessageContaining("No available copies");
    }

    @Test
    void checkout_memberAtLoanLimit_throwsException() {
        member = new Member("Limited", "limited@lib.com");
        // Create a custom member with maxLoans = 1 for testing
        memberRepository.save(member);

        // First checkout succeeds
        CheckoutRequest req1 = new CheckoutRequest();
        req1.setMemberId(member.getId());
        req1.setBookId(book.getId());
```

```
        loanService.checkout(req1);

        // Create another book for second checkout
        Book book2 = new Book("Second Book", "978-0000000002");
        book2.addCopy("TEST-002");
        bookRepository.save(book2);

        // Second checkout should fail if limit is 1
        // (default limit is 5, so this test uses 5 loans)
    }

    @Test
    void returnBook_validLoan_marksReturned() {
        // Checkout first
        CheckoutRequest request = new CheckoutRequest();
        request.setMemberId(member.getId());
        request.setBarcode("TEST-001");
        LoanResponse checkout = loanService.checkout(request);

        // Return
        LoanResponse returned = loanService.returnBook(checkout.getId());

        assertThat(returned.getReturnedAt()).isNotNull();

        BookCopy updatedCopy = bookCopyRepository.findByBarcode("TEST-001").orElseThrow();
        assertThat(updatedCopy.isAvailable()).isTrue();
    }

    @Test
    void returnBook_alreadyReturned_throwsException() {
        CheckoutRequest request = new CheckoutRequest();
        request.setMemberId(member.getId());
        request.setBarcode("TEST-001");
        LoanResponse checkout = loanService.checkout(request);

        loanService.returnBook(checkout.getId());

        assertThatThrownBy(() -> loanService.returnBook(checkout.getId()))
            .isInstanceOf(BusinessException.class)
            .hasMessageContaining("already returned");
    }
}
```

Project Checklist

```
Capstone Project Completeness:
+-------------------------------------------------------------------+
|                                                                   |
```

```
|  [ ] Entities with proper JPA annotations and relationships      |
|  [ ] BaseEntity with audit fields and @Version                   |
|  [ ] Flyway migrations (V1-V8) creating all tables and seed data |
|  [ ] ddl-auto=validate (Flyway manages schema)                   |
|  [ ] Request DTOs with Bean Validation                           |
|  [ ] Response DTOs (never expose entities)                       |
|  [ ] Repository layer with @EntityGraph and JOIN FETCH           |
|  [ ] Service layer with @Transactional and business rules        |
|  [ ] Controller layer with proper HTTP methods and status codes  |
|  [ ] Global exception handler (@ControllerAdvice)                |
|  [ ] Pagination on all list endpoints                            |
|  [ ] Search with Specifications (books)                          |
|  [ ] N+1 prevention (JOIN FETCH, @EntityGraph)                   |
|  [ ] FetchType.LAZY on all @ManyToOne                            |
|  [ ] spring.jpa.open-in-view=false                               |
|  [ ] readOnly=true on all read operations                        |
|  [ ] Tests for business rules and error cases                    |
|  [ ] Indexes on FK and frequently queried columns                |
+------------------------------------------------------------------+
```

Summary

In this capstone project, you built a complete Library Management System that applies patterns from every chapter in this book:

- 6 entities with complex relationships: one-to-many, many-to-many with join entity, and enums
- Flyway migrations for schema management with seed data
- Business rules enforced in the service layer (loan limits, availability checks, duplicate prevention)
- DTOs for all API input and output, never exposing entities
- Pagination on every list endpoint
- @EntityGraph and JOIN FETCH to prevent N+1 queries
- Global exception handling with consistent error responses
- Tests covering happy paths and error scenarios
- Performance optimizations: lazy loading, readOnly transactions, proper indexing

This is a production-quality application structure that you can use as a template for real projects.

Practice Exercises

Exercise 1: Book Search Implement `GET /api/books/search` with Specifications. Support filters: title (contains), author name (contains), isbn (exact), category, publication year range. All filters optional.

Exercise 2: Overdue Notifications Write a service method that finds all overdue loans and returns a list of notification DTOs with member name, email, book title, due date, and days overdue.

Exercise 3: Member Dashboard Create `GET /api/members/{id}/dashboard` that returns: member info, active loans count, total loans ever, overdue loans count, and a list of currently borrowed books.

Exercise 4: Book Availability Create `GET /api/books/{id}/availability` that returns: total copies, available copies, borrowed copies, and a list of expected return dates for borrowed copies.

Exercise 5: Statistics Endpoint Create `GET /api/admin/statistics` that returns: total books, total members, active loans, overdue loans, most borrowed book (this month), and most active member (this month). Use JPQL aggregate queries.

What Is Next?

The final chapter is the Glossary of JPA and Hibernate Terms — an alphabetical reference of every term, annotation, and concept covered in this book. Use it as a quick lookup when you encounter unfamiliar terminology in code reviews, documentation, or interviews.

Chapter 30: Glossary of JPA and Hibernate Terms

This glossary provides a quick reference for every key term, annotation, and concept covered in this book. Terms are grouped by category for easier browsing, then listed alphabetically within each group.

Core Concepts

Cascade: The propagation of persistence operations (persist, merge, remove, refresh, detach) from a parent entity to its related entities. Configured with `CascadeType` on relationship annotations.

Dirty Checking: Hibernate's automatic mechanism for detecting changes to managed entities. At flush time, Hibernate compares the current state of each entity to a snapshot taken when it was loaded. Changed entities generate UPDATE statements automatically.

Entity: A Java class mapped to a database table using `@Entity`. Each instance represents a row in the table. Entities have a primary key (`@Id`) and are managed by the persistence context.

Entity Lifecycle States: The four states an entity can be in: New (created but not persisted), Managed (attached to a persistence context), Detached (was managed but the context is closed), Removed (scheduled for deletion).

First-Level Cache (L1 Cache): The persistence context itself. Every entity loaded within a transaction is cached. Loading the same entity twice by ID returns the same object instance. Always enabled and cannot be disabled.

Flush: The process of synchronizing the persistence context with the database. Hibernate writes all pending INSERT, UPDATE, and DELETE statements to the database. Happens automatically before queries, at transaction commit, or manually via `flush()`.

Impedance Mismatch: The fundamental difference between how Java represents data (objects, inheritance, references) and how relational databases represent data (tables, rows, foreign keys). ORM frameworks bridge this gap.

JDBC (Java Database Connectivity): The low-level Java API for interacting with databases. Requires manual SQL, connection management, and result mapping. Hibernate uses JDBC internally but abstracts it away.

JPA (Jakarta Persistence API): The official Java specification for object-relational mapping. Defines standard annotations (`jakarta.persistence.*`), the `EntityManager` interface, JPQL, and the Criteria API. JPA is a specification, not an implementation.

JPQL (Java Persistence Query Language): A SQL-like query language that operates on entities and their fields rather than tables and columns. Example: `SELECT e FROM Employee e WHERE e.salary > 50000`. Database-independent and validated at startup.

Lazy Loading: A fetching strategy where related entities or collections are loaded from the database only when first accessed. Configured with `FetchType.LAZY`. Prevents loading unnecessary data but can cause N+1 problems or `LazyInitializationException`.

Managed Entity: An entity that is currently attached to a persistence context. Changes to managed entities are automatically detected and persisted at flush time.

N+1 Problem: A performance issue where loading N entities triggers N additional queries to load their lazy relationships. Solved with `JOIN FETCH`, `@EntityGraph`, or `@BatchSize`.

ORM (Object-Relational Mapping): A programming technique that automatically converts data between Java objects and relational database tables. Hibernate is the most popular Java ORM framework.

Orphan Removal: Automatic deletion of a child entity when it is removed from a parent's collection. Configured with `orphanRemoval = true` on `@OneToMany` or `@OneToOne`.

Owning Side: The side of a bidirectional relationship that controls the foreign key column. For `@OneToMany`/`@ManyToOne`, the `@ManyToOne` side is always the owning side. The owning side does NOT have `mappedBy`.

Persistence Context: The first-level cache managed by an `EntityManager`. It tracks all loaded entities, detects changes, and manages entity identity (only one instance per primary key).

Persistence Unit: A logical grouping of entity classes and configuration that defines a persistence context. In Spring Boot, configured via `application.properties`.

Second-Level Cache (L2 Cache): An optional, shared cache that spans multiple transactions and sessions. Stores entity data beyond the persistence context lifecycle. Requires a cache provider like Ehcache or Caffeine.

Session: Hibernate's equivalent of the JPA `EntityManager`. Provides the same functionality plus Hibernate-specific features. In modern code, prefer `EntityManager`.

JPA Annotations (jakarta.persistence.*)

@Basic: Marks a field as a basic (simple) type. Optional for most fields. Use `@Basic(fetch = FetchType.LAZY)` for lazy loading of large fields.

@Cacheable: Marks an entity as cacheable in the second-level cache. Requires `@EnableCaching` and a cache provider.

@Column: Specifies column details. Attributes: `name`, `nullable`, `unique`, `length`, `columnDefinition`, `updatable`, `insertable`.

@ConstructorResult: Used within `@SqlResultSetMapping` to map native query results to a DTO constructor.

@Convert / @Converter: Defines a JPA attribute converter for custom type mapping between Java types and database column types.

@DiscriminatorColumn: Specifies the discriminator column used in single-table inheritance. Default name is `DTYPE`.

@DiscriminatorValue: Specifies the value stored in the discriminator column for a specific subclass.

@ElementCollection: Maps a collection of embeddable types or basic types (e.g., `List<String>`) to a separate table.

@Embeddable: Marks a class as an embeddable value object that can be embedded in entities. Does not have its own table or identity.

@Embedded: Marks a field as an embedded instance of an `@Embeddable` class.

@Entity: Marks a class as a JPA entity mapped to a database table.

@EntityListeners: Specifies callback listener classes for entity lifecycle events (e.g., `AuditingEntityListener`).

@Enumerated: Specifies how an enum is stored. `EnumType.STRING` stores the name; `EnumType.ORDINAL` stores the position (avoid ordinal).

@GeneratedValue: Specifies primary key generation strategy: `AUTO`, `IDENTITY`, `SEQUENCE`, `TABLE`, or `UUID`.

@Id: Marks the primary key field of an entity.

@IdClass: Specifies a composite primary key class for an entity with multiple `@Id` fields.

@EmbeddedId: Marks a field as an embedded composite primary key using an `@Embeddable` class.

@Index: Defines a database index on the entity's table. Used within `@Table(indexes = ...)`.

@Inheritance: Specifies the inheritance mapping strategy: `SINGLE_TABLE`, `JOINED`, or `TABLE_PER_CLASS`.

@JoinColumn: Specifies the foreign key column for a relationship. Used on the owning side.

@JoinTable: Specifies a join table for `@ManyToMany` relationships or unidirectional `@OneToMany`.

@Lob: Marks a field as a Large Object (CLOB for text, BLOB for binary).

@ManyToMany: Maps a many-to-many relationship. Requires a join table.

@ManyToOne: Maps a many-to-one relationship. The owning side that holds the foreign key. Default fetch: `EAGER` (change to `LAZY`).

@MappedSuperclass: Marks a class whose fields are inherited by entities but is not an entity itself. No table is created for it.

@NamedEntityGraph: Defines a reusable fetch plan for an entity. Referenced by name in `@EntityGraph`.

@NamedQuery: Defines a pre-compiled JPQL query on an entity. Validated at startup.

@OneToMany: Maps a one-to-many relationship. Typically the inverse side with `mappedBy`. Default fetch: `LAZY`.

@OneToOne: Maps a one-to-one relationship. Can be owning or inverse side.

@OrderBy: Specifies the default ordering for collection elements using JPQL field names.

@PostLoad: Lifecycle callback executed after an entity is loaded from the database.

@PostPersist: Lifecycle callback executed after an entity is persisted (INSERT).

@PostRemove: Lifecycle callback executed after an entity is removed (DELETE).

@PostUpdate: Lifecycle callback executed after an entity is updated (UPDATE).

@PrePersist: Lifecycle callback executed before an entity is persisted.

@PreRemove: Lifecycle callback executed before an entity is removed.

@PreUpdate: Lifecycle callback executed before an entity is updated.

@Query: Spring Data JPA annotation for defining custom JPQL or native SQL queries on repository methods.

@SequenceGenerator: Configures a database sequence for primary key generation.

@SqlResultSetMapping: Maps native query results to entities or DTOs with precise column-to-field control.

@Table: Specifies the table name, schema, and indexes for an entity.

@Temporal: Specifies the JDBC temporal type for `java.util.Date` or `java.util.Calendar` fields. Not needed for `java.time` types.

@Transient: Marks a field as not persisted. The field is ignored by JPA.

@Version: Marks a field for optimistic locking. Hibernate automatically increments it on each update and throws `OptimisticLockException` on concurrent modifications.

Hibernate-Specific Annotations (org.hibernate.annotations.*)

@BatchSize: Controls the number of lazy-loaded collections or entities fetched in a single SQL IN clause. Reduces N+1 by batching.

@DynamicInsert: Generates INSERT statements that include only non-null columns.

@DynamicUpdate: Generates UPDATE statements that include only changed columns.

@Fetch(FetchMode): Controls how a collection is fetched: `SELECT` (separate query), `JOIN` (join with parent), `SUBSELECT` (single subselect for all collections).

@Filter / @FilterDef: Defines a parameterized SQL filter that can be enabled or disabled at runtime. Useful for soft deletes.

@Formula: Maps a field to a SQL expression instead of a column. Computed on every load.

@NaturalId: Marks a field as a natural (business) key. Provides `byNaturalId()` lookup in Hibernate.

@SQLDelete: Overrides the default DELETE statement. Used for soft deletes: `@SQLDelete(sql = "UPDATE table SET deleted = true WHERE id = ?")`.

@Where: Adds a SQL WHERE clause to entity loading or collection fetching. Used for soft delete filtering.

Spring Data JPA Terms

CrudRepository: Base interface providing basic CRUD methods: `save`, `findById`, `findAll`, `deleteById`, `count`, `existsById`.

Derived Query Methods: Repository methods whose queries are automatically generated from the method name. Example: `findByNameContaining(String name)`.

JpaRepository: Extends `CrudRepository` with JPA-specific methods: `flush`, `saveAndFlush`, batch deletes, `findAll(Sort)`, `findAll(Pageable)`.

JpaSpecificationExecutor: Interface that adds Specification-based querying to a repository. Enables dynamic query composition.

Page: A paginated result that includes content, total elements, total pages, and navigation metadata. Requires an extra COUNT query.

Pageable / PageRequest: Defines pagination parameters: page number (0-based), page size, and sort order.

Slice: A paginated result that knows if more data exists but does not count the total. More efficient than Page for large datasets.

Sort: Defines ordering: field name, direction (ASC/DESC), and null handling.

Specification: A functional interface for building type-safe, composable query predicates using the Criteria API. Can be combined with `and()`, `or()`, `not()`.

Transaction and Performance Terms

ACID: Atomicity, Consistency, Isolation, Durability — the four guarantees of a database transaction.

@Transactional: Spring annotation that wraps a method in a database transaction. Key attributes: `readOnly`, `propagation`, `isolation`, `rollbackFor`.

Batch Processing: Executing multiple SQL statements in groups (batches) rather than one at a time. Configured with `hibernate.jdbc.batch_size`.

Connection Pool: A pool of pre-created database connections reused across requests. HikariCP is the default in Spring Boot.

Isolation Level: Defines how concurrent transactions interact. Common levels: `READ_COMMITTED`, `REPEATABLE_READ`, `SERIALIZABLE`.

Optimistic Locking: A concurrency strategy using a `@Version` field. The transaction fails if another transaction modified the entity first.

Propagation: Defines how transactions interact when methods call other transactional methods. Common: `REQUIRED` (default), `REQUIRES_NEW`, `SUPPORTS`.

Query Cache: An optional Hibernate cache that stores the results (entity IDs) of queries. Must be used with the second-level entity cache.

Read-Only Transaction: A transaction marked with `readOnly = true` that skips dirty checking, improving performance for read-heavy operations.

StatelessSession: A Hibernate-specific lightweight session that bypasses the persistence context. Used for bulk operations where caching overhead is unwanted.

Database Migration Terms

Baseline: Initializing Flyway on an existing database by marking the current schema as the starting point.

Flyway: A database migration tool that applies versioned SQL scripts to evolve the database schema. Runs automatically on Spring Boot startup.

Repeatable Migration: A Flyway migration (prefix `R__`) that is re-executed whenever its content changes. Used for views, stored procedures, and reference data.

Versioned Migration: A Flyway migration (prefix `V1__`, `V2__`) that runs exactly once and is never modified after execution.

Architecture Terms

Controller: The HTTP layer that handles requests, validates input, and returns responses. Annotated with `@RestController`.

DTO (Data Transfer Object): A plain object used to transfer data between layers. Entities are never exposed directly in API responses.

Repository: The data access layer that interacts with the database. Annotated with `@Repository`. In Spring Data JPA, typically an interface extending `JpaRepository`.

Service: The business logic layer that orchestrates operations, enforces rules, and manages transactions. Annotated with `@Service`.

This glossary covers the essential terms you will encounter when working with JPA and Hibernate. Return to it whenever you need a quick refresher on a concept, annotation, or pattern.